

The PreTeXt Guide

The PreTeXt Guide

Robert A. Beezer
University of Puget Sound

David Farmer
American Institute of Mathematics

Alex Jordan
Portland Community College

Mitchel T. Keller
Morningside College

Oscar Levin
University of Northern Colorado

June 1, 2026

©2013–2019 Robert A. Beezer, David Farmer, Alex Jordan, Mitchel T. Keller

Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.3 or any later version published by the Free Software Foundation; with no Invariant Sections, no Front-Cover Texts, and no Back-Cover Texts. A copy of the license is included in the section entitled “GNU Free Documentation License”.

Preface

Introduction. This part is the place to begin if you are new to PreTeXt. [Chapter 1](#) is the introduction, overview, and philosophy. Then [Chapter 2](#) intends to get you started quickly by showing how to set up a PreTeXt authoring environment and converting a document to HTML and $\text{\LaTeX}$ output formats. Notice that there are three parts which target different roles: the *Author's Guide* ([Part II](#)), the *Publisher's Guide* ([Part IV](#)) and the *Developer's Guide* ([Part V](#)).

Author's Guide. This guide will help you *author* a PreTeXt document. So it serves as a description of the PreTeXt XML vocabulary, along with the mechanics of creating the source and common output formats. [Chapter 3](#) is meant to be a short overview of the majority of PreTeXt's features, which can be skimmed to get a sense of PreTeXt's capabilities. Or it can be read quickly as you begin authoring and you can return as you need certain features. The roughly parallel [Chapter 4](#) is much more comprehensive and is the first place to go for details not addressed in the overview. Note that the *Author's Guide* is not concerned with *publishing* your document, which is described in the *Publisher's Guide*.

Basic Reference. This part provides a quick overview of the minimal syntax for a variety of key PreTeXt features. Unlike the sample article, which is designed to demonstrate and stress test all aspects of PreTeXt, this guide will illustrate only the key elements of some of the most universally-used features of the language. In many cases, in addition to features not discussed, there may be alternative structures that are not given here.

Publisher's Guide. Even if you intend to distribute your document with an open license, and you are both author *and* publisher, it is still helpful and instructive to understand, and separate, the two different steps and roles. So visit this part of the *Guide* to learn how you can present, distribute, and maintain what you have authored.

Developer's Guide. This part provides advice, suggestions, and conventions for contributing to PreTeXt. For anything not answered here please use the [pretext-dev](#) Google Group. Make a membership request and it will be processed quickly.

Appendices. In addition to the usual items you might expect in the back matter, such as an [open license](#), [glossary](#), [references](#), and an index, there are numerous more specialized additions, mostly describing the installation of, or effective use of, various technical tools that are independent of PreTeXt (but useful or necessary).

Contents

Preface	iv
I Introduction	
1 Why PreTeXt?	2
2 Getting Started Tutorial	6
II Author's Guide	
3 Overview of Features	15
4 Topics	27
5 Processing, Tools and Workflow	122
6 PreTeXt Vocabulary Specification	140
7 WeBWork Exercises	144
8 Authoring Advice	151
9 Author FAQ: Frequently Asked Questions	153
III Basics Reference	
10 About This Reference	159

11 Basic Formatting	160
12 Document Structure	161
13 Mathematics	163
14 Lists	165
15 Blocks	167
16 Exercises	173
17 Worksheets	181
18 Renaming Elements	186
19 Figures and Friends	187
20 Tables	193
21 SageMath Content	196
22 Small, But Useful	198
23 Modular Source	201
IV Publisher's Guide	
24 (*) Producing a Book	203
25 Copyright and Licensing	205
26 Conversions, Generally	210
27 Custom Versions	214
28 Further Customizations	218
29 Conversion to Online HTML	220

30	Conversion to PDF and Print	229
31	Conversion to EPUB/Kindle	236
32	Conversion to Runestone	239
33	Conversion to Braille	246
34	Conversion to Slides	247
35	(*) Conversion to Jupyter Notebooks	249
36	Instructor's Version	250
37	Ancillaries	252
38	WeBWork Exercises	257
39	Hosting Your Online Version	264
40	L^AT_EX Fonts	266
41	L^AT_EX Styles	271
42	(*) Cover Design	277
43	Print-On-Demand	278
44	Publication File Reference	280
45	Publisher FAQ: Frequently Asked Questions	299
V	Developer's Guide	
46	Processing with xsl_tproc	302
47	The pretext Script	309
48	Coding Conventions	315

49	Styling with CSS and SCSS	318
50	Contributing Localizations	322
51	Code Style	323
52	Debugging	324
53	Git	326

Appendices

A	Welcome to the PreTeXt Community	334
B	Best Practices	337
C	Python	338
D	Text Editors, Spell Check	341
E	Schema Tools	355
F	Node and npm	358
G	Offline MathJax	359
H	LibLouis	361
I	Revision Control: git	363
J	Conversion from L^AT_EX	364
K	MacOS Installation Notes	365
L	Installing the PreTeXt-CLI on Windows	366
M	Windows Installation Notes	372
N	Windows Subsystem for Linux	379

O Transitioning to the PreTeXt-CLI	381
P CLI versions 1 vs 2	384
Q List of Supported Journal Styles	388
R Example List of Notation	389
S GNU Free Documentation License	390
Back Matter	
Glossary	396
References	399
Index	400

Part I

Introduction

Chapter 1

Why PreTeXt?

Welcome to “the Guide” for PreTeXt. You are likely eager to get started, but familiarizing yourself with this chapter should save you a lot of time in the long run. We will try to keep it short and at the end of early chapters we will guide you on where to go next. Not everything we say here will make sense on your first reading, so come back after your first few trial runs. When you are ready to seek further help, or ask questions, please read the [Welcome to the PreTeXt Community](#) in [Appendix A](#).

1.1 Philosophy

PreTeXt is a **markup language**, which means that you explicitly specify the logical parts of your document and not how these parts should be displayed.

This is very liberating for an author, since it frees you to concentrate on capturing your ideas to share with others, leaving the construction of the visual presentation to the software. As an example, you might specify the content of the title of a chapter to be Further Experiments, but you will not be concerned if a 36 point sans-serif font in black will be used for this title in the print version of your book, or a CSS class specifying 18 pixel height in blue is used for a title in an online web version of your book. You can just trust that a reasonable choice has been made for displaying a title of a chapter in a way that a reader will recognize it as a name for a chapter. (And if all that talk of fonts was unfamiliar, all the more reason to trust the design to software.)

You are also freed from the technical details of presenting your ideas in the plethora of new formats available as a consequence of the advances in computers (including tablets and smartphones) and networks (global and wireless). Your output “just works” and the software keeps up with technical advances and the introduction of new formats, while you concentrate on the content of your book (or article, or report, or proposal, or ...).

If you have never used a markup language, it can be unfamiliar at first. Even if you have used a markup language before (such as HTML, Markdown, or basic L^AT_EX) you may need to make a few adjustments. Most word-processors are WYSIWYG (“what you see is what you get”). That approach is likely very helpful if you are designing the front page of a newspaper, but not if you are writing about the life-cycle of a salamander. In the old days, programs like [troff](#) and its predecessor, [RUNOFF](#) (1964), implemented simple markup languages to allow early computers to do limited text-formatting. Sometimes the old ways are the best ways.

PreTeXt is what is called an **XML application** or an **XML vocabulary** (I prefer the latter). That is, the source you write is “marked up” as XML, with specific **tags** that describe the semantic structure of your document. Authoring in XML might seem cumbersome at first, since some content will require more characters of markup than of content. Much of this markup can be quickly produced with a modern text editor, but it can still be overwhelming. We believe you will eventually appreciate the long-run economies, so keep an open mind. And if you are already familiar with XML, realize we have been very careful to design this vocabulary with human authors foremost in our mind.

Principles. The creation, design, development, and maintenance of PreTeXt is guided by the following list of principles. These will become more understandable as you become more familiar with authoring texts with PreTeXt and should amplify some of the previous discussion.

List 1.1.1 PreTeXt Principles

1. PreTeXt is a markup language that captures the structure of textbooks and research papers.
2. PreTeXt is human-readable and human-writable.
3. PreTeXt documents serve as a single source which can be easily converted to multiple other formats, current and future.
4. PreTeXt respects the good design practices which have been developed over the past centuries.
5. PreTeXt makes it easy for authors to implement features which are both common and reasonable.
6. PreTeXt supports online documents which make use of the full capabilities of the Web.
7. PreTeXt output is styled by selecting from a list of available templates, relieving the author of the burden involved in micromanaging the output format.
8. PreTeXt is free: the software is available at no cost, with an open license. The use of PreTeXt does not impose any constraints on documents prepared with the system.
9. PreTeXt is not a closed system: documents can be converted to $\text{\LaTeX}$ and then developed using standard $\text{\LaTeX}$ tools.
10. PreTeXt recognizes that scholarly documents involve the interaction of authors, publishers, scholars, curators, instructors, students, and readers, with each group having its own needs and goals.
11. PreTeXt recognizes the inherent value in producing material that is accessible to everyone.

1.2 Understanding Your Source

Almost all of your time authoring in PreTeXt will be spent editing your **source** files. We now briefly describe what these files will look like and how to edit them.

File Format and Text Editors. Your source will be plain text **ASCII files**, which you create and edit with any number of text editors. Files can be saved with the `.ptx` extension, which might tell your text editor what sort of file you are editing and will provide syntax highlighting and code completion, among other features. If your editor does not recognize `.ptx`, then you can use the `.xml` extension which has wider editor support (but with fewer PreTeXt-specific features).

Popular text editors include Visual Studio Code, Sublime Text, vi, emacs, Notepad, Notepad++, Atom, TextWrangler, and BBEdit. But in particular, you should not use word processing programs like Word, LibreOffice, Google Docs, WordPerfect, AbiWord, Pages, or similar programs. Sometimes these editors are known as a **programmer's editor** (though we will be doing no programming). Support for writing HTML sometimes translates directly to good support for XML.

Visual Studio Code has support for PreTeXt documents via a free extension, and the editor is open source and cross-platform (Windows, OS X, and Linux). The developers of PreTeXt have also had a very good experience with Sublime Text, which is cross-platform, and can be used for free, though it has a very liberal paid license if you want to avoid nagging.

There are **XML editors**, which might be too complex for authoring in PreTeXt. They do have some advantages and XML Copy Editor is one that you might find useful.

Some text editors (like VS Code) have spell checking extensions. More generally, recommendations for a spell checker can be found in [Section D.3](#).

Structure of your Source. If you start to think about the structure of a document (like an article or book) you will quickly realize that components are like blocks, stacked inside or next to other blocks. From the *outside* to *inside*,

a book will have a number of chapters (next to each other, but all inside the book), and each might have sections (adjacent but inside the chapter). In the section, there will be a title, paragraphs, images, examples, theorems, and so on. Examples will themselves contain paragraphs. A theorem might contain a statement, which contains some paragraphs, which might contain some displayed math, and adjacent to the statement, there could be a proof, itself containing paragraphs, etc.

The hierarchical nature of XML is perfect to capture the hierarchical nature of a scholarly document. Consider the start of a PreTeXt document shown in [Listing 1.2.1](#).

Listing 1.2.1 Source of a simple PreTeXt book project.

```
<?xml version="1.0" encoding="UTF-8"?>
<pretext>
  <book>
    <title>Hello world!</title>
    <chapter>
      <title>Getting Started</title>
      <p>Welcome to PreTeXt!</p>
      <!-- TODO: find something more to say... -->
    </chapter>
  </book>
</pretext>
```

The first line is boilerplate that lets various programs know the rest of the file is XML, and the line start `<!--` is an example of a comment that won't appear in the output. Besides this, you can start to see how the structure of the book is laid out.

Whitespace and Indentation. The term **whitespace** refers to characters you type but typically do not see. For us they are **space**, **non-breaking space**, **tab** and **newline** (also known as a “carriage return” and/or “line feed”). Unlike some other markup languages, PreTeXt *does not ever use whitespace* to convey formatting information.

However, it can be useful to use whitespace to indent the different levels of the XML (and document) hierarchy. Use two (or four) spaces for indentation; a good editor will visually respect this indentation, and help you with maintaining the right indentation with each new line. Line up opening and closing tags at the same level of indentation, and your editor should let you “fold” the code to visually hide blocks.

Whatever you do, use a style and stick with it. You could put titles on a new line (indented) after creating a new chapter or section; some people like them on the same line, immediately adjacent. You could put a single blank line before each new paragraph, but not after the last. And so on. The choice is yours, but consistency will pay off when you inevitably come back to edit something. You have put a lot of work and effort into your source. You will be rewarded with fewer problems if you keep it neat and tidy.

In some parts of a PreTeXt document, every single whitespace character is important and will be transmitted to your output, such as in the `<input>` and `<output>` portions of a `<sage>` element. Since Sage code mostly follows Python syntax, indentation is important and leading spaces must be preserved. But you can indent all of your code to match your XML indentation and the entire `<input>` (or `<output>`) content will be uniformly shifted left to the margin in your final output.

Never use tabs, they can only cause problems. You should be able to set your editor to translate the tab key to a certain number of spaces, or to translate tabs to spaces when you save a file (and these behaviors are useful). Most editors have a setting that will show whitespace as a small faint dots or arrows, so you can be certain there is no stray whitespace *anywhere*.

Learn to Use Your Editor. Because XML requires a closing tag for every opening tag, it feels like a lot of typing. The VS Code PreTeXt extension comes with many **snippets** (code completions) that can fill out lots of the markup for you. More generally, any editor should know what tag to close next and there should be a simple command to do that (for example, in Sublime Text on Linux, `Alt-Period` gives a closing tag). Not only is this quick and easy, it can help spot errors when you forget to close an earlier tag.

If your editor can predict your opening tag, all the better. VS Code can recognize what tags are allowed at a given position. Sublime Text recognizes if you already have a `<section>` elsewhere, so when you start a second section,

you very quickly (and automatically) get a short list of choices as you type, with the one you want at the top of the list, or close to it.

Invest a little time early on to learn, and configure, your editor and you can be even more efficient about capturing your ideas with a minimum of overhead and interference.

Revision Control. If you are writing a book, or if you are collaborating with co-authors, then you owe it to yourself and your co-authors to learn how to use revision control, which works well with PreTeXt since the source is just text files. The hands-down favorite is `git`. To fully understand it is beyond the scope of this guide but some information is provided in [Appendix I](#) which has hints on how to best use `git` together with a PreTeXt project.

If you use the workflow recommended in the [Chapter 2](#) using GitHub’s codespaces, you will get revision control via `git` automatically, and VS Code provides a graphical user interface for all the basic operations you need.

1.3 Converting Your Source to Output

Once you have content created in PreTeXt files (i.e., XML files), you will want to convert these files into a output format such as HTML, to be viewed in a web browser, or a PDF. Instructions for doing this will be discussed in [Chapter 2](#), and in even more detail in [Chapter 5](#). Here we provide an overview of how the conversion works to help you understand what is possible.

With PreTeXt “installed” (on your computer or in the cloud), converting PreTeXt XML into a full HTML website can be as simple as typing `pretext build html` in a terminal, or hitting `Ctrl+Alt+B` in VS Code. Behind-the-scenes, these commands read through your XML and use XSL 1.0 (**eXtensible Stylesheet Language**) to *transform* the XML source, using a number of xsl stylesheets that come with PreTeXt.

The recommended workflow for processing your source uses a python program we call the *PreTeXt-CLI* (CLI is **command line interface**). There are also a number of other free tools that can processes XML with XSL. For example, `xsltproc` is a command line program that is usually installed by default on Linux systems and MacOS. This was the recommended method in the early days of PreTeXt, and still works. Documentation for how to use `xsltproc` with PreTeXt can be found in [Chapter 46](#), but unless you are helping with the development of PreTeXt or are trying to do something fancy, you probably don’t need it.

Some features of PreTeXt, such as the inclusion of images described in source, or including WeBWorK exercises, requires the use of an additional processing, done in python. Some of these also require additional software (such as $\text{\LaTeX}$ or Sage). The PreTeXt-CLI does this automatically when building (and regenerates these assets if they have changed since the last build). There is also a python script that can accesses these functions directly for use in development. See [Chapter 47](#) if you are curious.

1.4 Where Next?

To start playing with PreTeXt right away, work through the [Chapter 2](#). It will guide you through a cloud-based setup (no software install required) and you will create, edit, convert, and deploy your first document.

If you would like a general, high-level overview of features skip ahead to [Chapter 3](#).

In-depth, comprehensive use of features is in [Chapter 4](#).

If you have an existing project authored in $\text{\LaTeX}$ you may be interested in the conversion process described in [Appendix J](#).

Chapter 2

Getting Started Tutorial

Objectives

At the end of this tutorial you will have...

- Created a free GitHub account.
- Created a GitHub Repository and Codespace for authoring PreTeXt in your web browser.
- Learned the first steps to editing a PreTeXt document.
- Converted your document to both $\text{\LaTeX}$ -generated PDF and accessible HTML.
- Deployed your HTML to the web via GitHub Pages.

This chapter serves as a tutorial for quickly getting started with PreTeXt in your web browser using free services provided by [GitHub](#). (Advanced users who'd prefer to install our free and open-source software to their own machine may choose to skip ahead to [Section 2.3](#).)

The community does its best to keep this guide updated, but for even more up-to-date advice, join us at our regular Zoom drop-ins announced at [our Google group](#) or watch a recording posted in [Section 2.4](#).

2.1 Using PreTeXt online

It is possible to write PreTeXt documents using nothing more than a web browser. This approach does not require you to install any software (other than a web browser), although it does require you have internet access. Options for installing PreTeXt on your own computer are discussed in [Section 2.3](#).

2.1.1 What is GitHub

GitHub is a freely-available service owned by Microsoft for authoring, sharing, and deploying documents and source code. It uses the free and open-source **Git** software for version management.

There are other services such as [CoCalc](#) and [GitLab](#) for managing PreTeXt documents online, as well as other ways to write PreTeXt that don't require anything besides installing the free and open-source PreTeXt software onto your own device (see [Section 2.3](#) to learn more).

We will use GitHub for this tutorial as it the most popular way to share and disseminate PreTeXt documents, and provides the easiest pathway to getting started writing in the PreTeXt language.

To create your free GitHub account, [follow the instructions on GitHub's sign-up page](#). You can also log into an existing GitHub account if you already have one. Be sure to note your GitHub username and password in your password manager (or however you usually keep track of login credentials).

Tip! Educators and non-profit researchers can get many of GitHub’s paid features for free. While this is not strictly required for the rest of the tutorial, it’s a useful way to increase GitHub’s free Codespaces usage quotas, and allows you to use GitHub’s free web hosting even for private repositories.

Apply at [Education.GitHub.com](https://education.github.com) to unlock these features. In our experience, applications are usually processed quickly for .edu email addresses, but you do not need to wait for approval to continue on with this tutorial.

2.1.2 Three GitHub concepts

This tutorial uses three GitHub services:

Codespaces (github.dev)	The Codespace for your project is an application run in your web browser that gives you access to a virtual computer with all the software recommended to author PreTeXt installed for you automatically. This Codespace is private to you, and lives at an address like <code>https://username-random-words-abc123.github.dev</code> .
Repository hosting (github.com)	The repository for your project represents the history of its edits that have been “committed and synced” from your Codespace to it. This repository can be public or private (though we encourage public repositories as they help the community provide support for each other), and lives at an address like <code>https://github.com/username/reponame/</code> .
GitHub Pages (github.io)	The GitHub Pages service provides free hosting for websites such as the HTML generated from a PreTeXt project. This website is public, and lives at an address like <code>https://username.github.io/reponame/</code> .

Broadly speaking, you “author” within your Codespace, which you periodically “commit and sync” to your repository, and then occasionally “deploy” to your public GitHub Pages website.

2.1.3 Creating your repository and Codespace

Follow the instructions at <https://github.com/PreTeXtBook/pretext-codespace> to get started creating your repository and Codespace. You’ll have the option to make your repository public (recommended if you want support from the rest of the PreTeXt community) or private. Either way, those instructions will also walk you through creating your private Codespace for authoring.

This takes a few moments, but is a one-time process. Take note of the `github.com` URL your new repository lives at so you can find it the next time you want to work on your project. (You can always access your `github.dev` Codespace link from there via the Code menu.) Then you’ll be ready for [Section 2.2](#).

2.2 Your First PreTeXt Document

At this point, you should have a PreTeXt project set up as a `github.com` repository with a `github.dev` Codespace. You can use the Code menu on the repository webpage to pull up the Codespace environment in your web browser if you haven’t already.

When you first created your repository from the template, there were very few files included. The first thing you will do when you open your Codespace is to create a new PreTeXt project. As the directions in the repository indicate, you can do this by selecting the “PreTeXt: New Project” command from the **command palette** (which you can open with `CTRL+SHIFT+P`, among other options).

You should be presented with a dialog asking for what sort of project you would like to create (article, book, course, slideshow, etc.). For purposes of this tutorial, you should select article or book. You will then be asked where you would like to create the project, and selecting the default suggested location is fine.

The window should now refresh and you will see a bunch of new files, including a folder called `source` that contains your `main.ptx` file. Now you are ready to build it!

2.2.1 Building for web

You can build your entire project in a few different ways.

- Click the “PreTeXt” button in the center left of the bottom toolbar of the VS Code window (see [Figure 2.2.1](#)). A dialog will pop up asking which PreTeXt command you want to run. Select `Build default target`, or choose `Build another target...` to get a menu of options to select a **target** to build (choose `web`).
- You can use the keyboard shortcut `CTRL+ALT+p` (replacing `CTRL` with `CMD` if you have a Mac) to get the same dialogs. Or to build in one step, use `CTRL+ALT+b`.
- Select a PreTeXt command from the VS Code command palette, which you can access by clicking the icon in the bottom left of the VS Code window. You can also access this by typing `CTRL+SHIFT+p` (again, replacing `CTRL` with `CMD` if you have a Mac). Start typing “pretext” to get a list of commands available.
- If you are comfortable entering commands in a terminal/command prompt, you can access one in your Codespace using `CTRL+``. Then you can run `pretext build web` to build your project.

The resulting HTML files will be available in the output/web directory of your project. However, to view it, you should NOT navigate there and open the files. Instead, read on.

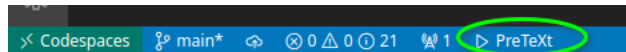


Figure 2.2.1 PreTeXt commands in Codespaces

2.2.2 Viewing

You can preview these HTML files you just built using the **View** command. Again, you can access this in multiple ways: PreTeXt button in the toolbar, `CTRL+ALT+p`, etc. Select `View` from the dialog. You may be given options on how to view the document, depending on what VS Code plugins you have available to you. Try one or another until you’re able to view your web build in either a new tab of your browser or a tab within VS Code.

The VS Code Live Preview is a good option, but it is buggy when used inside Codespaces. It seems to help to use the VS Code command palette to run `Live Preview: Show Preview (External Browser)`, then close the tab that opens, and start the process over. You may need to do this a few times before it works.

Now is a great time to try to make edits to your source files (maybe change the title). Note that these changes aren’t updated live in your preview: you will need to build again, and then refresh the preview window to see them. Note, you do not need to run the `View` command again unless you stop the preview server.

2.2.3 Building for print

To produce a PDF from your PreTeXt source, repeat the above instructions with the `print` target. If you use the PreTeXt button in the toolbar, select `Build another target...` and then `print`. Alternatively, you can enter `pretext build print` in a terminal. The resulting PDF will land in `output/print/main.pdf`, and can be downloaded by right-clicking in the VS Code file explorer, or previewed using a `View` command.

If you would like to see the intermediate $\text{\LaTeX}$ source files, you can add a new target to the project manifest (`project.ptx`) with a `@format` of `latex`. See [Subsection 5.2.7](#).

2.2.4 Saving your work

Using Codespaces will keep all your files “in the cloud”, saved automatically as you edit. As long as your Codespace is active, your files will be saved there for your private use. However, inactive Codespaces are periodically cleaned up by GitHub (as of writing, this happens after one month of inactivity), so you’ll need to periodically **commit & sync** your work to your repository where it will never be deleted.

Recall that your Codespace lives at `github.dev`, while your repository has a `github.com` address like `https://github.com/username/repo`. This repository serves as a backup of your work in the Codespace, and has the added benefit of allowing collaborators to access your files as well. As a bonus, if you made your repository public, members of the PreTeXt community who watch the [PreTeXt-support](#) Google group can create their own Codespace based on your public repository and easily answer any questions you have.

While Git and GitHub have a lot of features, there’s a very simple way to use them via Codespaces. As you edit files, you’ll notice that their filenames will turn orange, and new files will appear green. Likewise, a blue number will appear in the left sidebar.

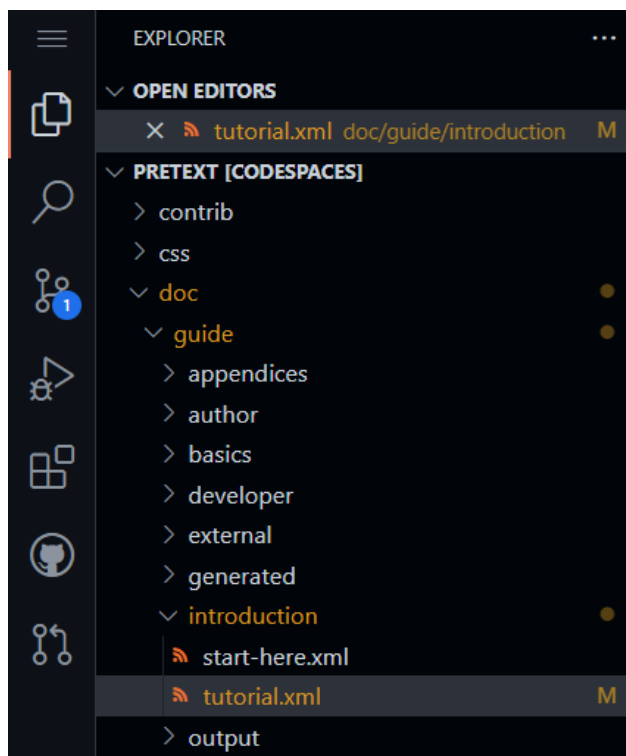


Figure 2.2.2 Filenames changing color as they are edited in Codespaces

This blue badge is next to the Source Control view. You will notice a list of files that were changed; you can click on any of these to see what the changes are.

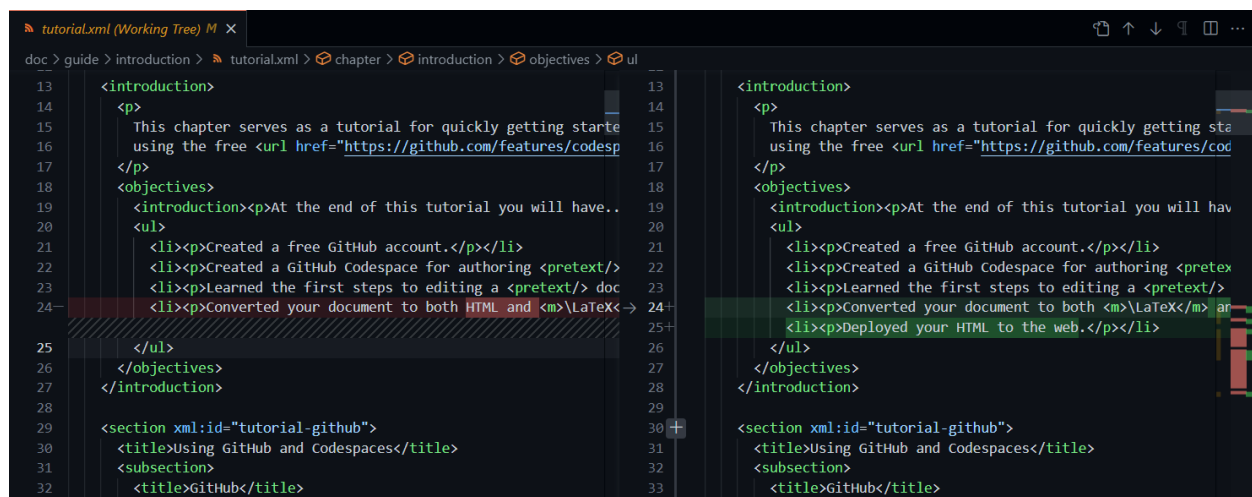


Figure 2.2.3 A “Git diff” showing changes in a file

Type a message describing the changes you’ve made then click the green “Commit and Sync” button. If it just says “Commit”, use the drop-down menu to choose “Commit and Sync”. (If you forget to type a message describing the changes you’ve made, then a new tab will open: “COMMIT_EDITMSG” where you can type the message. When you are done, close the tab.)

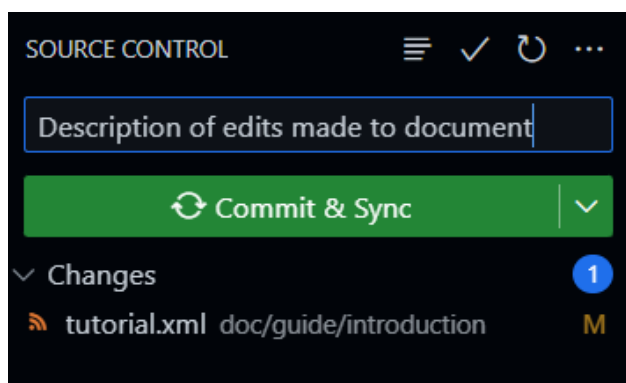


Figure 2.2.4 Committing and syncing changes

To see that this is successful, return to your `github.com` repository webpage. You should see your files with all your committed/synced changes. (That is, most of them: many files, such as log files and temporary build files that appear in gray within your Codespace, will not be synced. This is no problem: they are created during a build automatically and don't need to be, and really shouldn't be, saved or shared with others.)

2.2.5 Generating assets

If your document contains certain elements, you might need to **generate** their assets for use in certain output formats. Depending on your build target, these include:

- `<latex-image>`
- `<sagemath>`
- `<asymptote>`
- `<youtube>`
- `<webwork>`
- `<code\ens>`

Starting in CLI version 1.7, these assets will be automatically generated whenever you build your output. If you change the source of these assets, they will be regenerated when you build.

Regardless of which version of the CLI you are using, you can generate assets as a separate step in much the same way you run a build. You will see a **Generate** option in the PreTeXt command dialog, just below **Build**. Select your target and wait for the process to complete, then **Build** once more to incorporate your generated assets.

2.2.6 Deploy

So you have worked tirelessly to prepare course notes or a book, built and previewed, synced changes to your git repository, and now you are ready to share the results of your efforts with the world. It's time to **deploy** your project!

From the "PreTeXt Commands" dialog, select "Deploy". This will automatically take the most recent build of your web target and host it through [GitHub Pages](#). Watch the output pane for a link to your published site; unlike the preview link you've been using on `github.dev` which is private to only you, this `github.io` link is ready to share with the world. (It can take a few minutes for the site to get set up or updated; there should be another link to view the progress of the GitHub "action" that reports the progress.)

By default, doing a deploy will just publish your web target. It is also possible to deploy multiple targets along with a "landing" page directing a visitor of your site to the different versions of your project. See [Section 5.2](#) for more information.

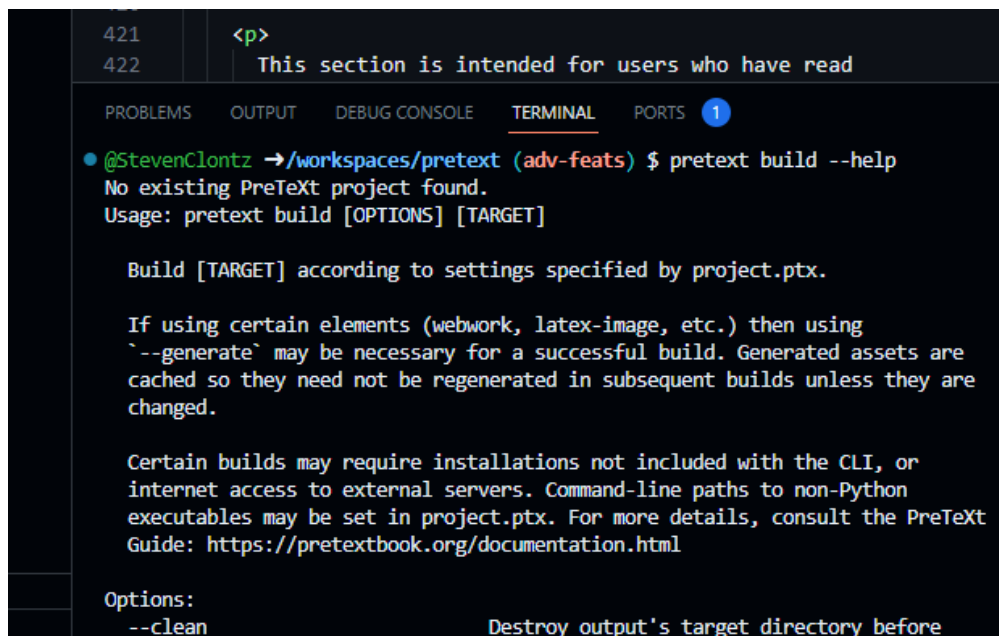
2.2.7 Using this guide and advanced features

The rest of this guide will help you on your way. However, keep in mind that this guide is the work of many volunteers over many years, and certain sections may assume the reader is using mechanisms for writing PreTeXt that have been around for much longer than the Codespaces environment recommended for this tutorial.

In particular, there are two advanced mechanisms used by many PreTeXt authors: the **PreTeXt developer script** [Chapter 47](#) (i.e. the pretext/pretext script) and the **PreTeXt CLI** [Section 5.2](#).

Under the hood, the PreTeXt CLI is what you're using in Codespaces, and it also has the ability to call the PreTeXt developer script as well. If you ever want to use a PreTeXt CLI command, you can open a **Terminal** in your Codespace using the menus, or by pressing `Ctrl`+``` (the backtick key, found in upper left of many keyboards).

From the terminal, you can type in any PreTeXt CLI commands directly. For example, typing in the CLI command `pretext build web` and running it by pressing `Enter` builds the web target.



```

421  <p>
422  This section is intended for users who have read

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  1
● @StevenClontz → /workspaces/pretext (adv-feats) $ pretext build --help
No existing PreTeXt project found.
Usage: pretext build [OPTIONS] [TARGET]

Build [TARGET] according to settings specified by project.ptx.

If using certain elements (webwork, latex-image, etc.) then using
`--generate` may be necessary for a successful build. Generated assets are
cached so they need not be regenerated in subsequent builds unless they are
changed.

Certain builds may require installations not included with the CLI, or
internet access to external servers. Command-line paths to non-Python
executables may be set in project.ptx. For more details, consult the PreTeXt
Guide: https://pretextbook.org/documentation.html

Options:
  --clean          Destroy output's target directory before
  
```

Figure 2.2.5 Using the PreTeXt CLI with a Codespaces terminal

The CLI should be sufficient to do nearly everything you want to do for your project, and using the developer script should be exercised with caution. Nonetheless, to access a pretext/pretext developer script feature, you can use `pretext devscript`. For example, if the documentation suggests a command like `pretext/pretext -foo bar`, you could try running `pretext devscript -foo bar`.

2.3 Installing PreTeXt

The browser-based GitHub Codespaces workflow described in [Section 2.1](#) is a quick and easy way to use PreTeXt without needing to install any additional software. If you prefer to have a local setup of PreTeXt, we recommend one of the following three setups.

2.3.1 Option 1: Installing a Docker container

The closest thing we have to a single click installer is to use a **Docker container** that includes all the software needed to run PreTeXt. In fact, the setup described in this section results in an identical environment to the one used in GitHub Codespaces, but it is all locally running on your own computer. The directions for all operating systems are the same (since the two programs you need to download and install are available for Windows, Mac, and Linux).

The one downside to this approach is that you will need a fair amount of disk space (around 5 GB all together), and some of that space will be used to install tools like $\text{\LaTeX}$ and SageMath that you might already have installed

anyway.

Here are the steps required to use this option.

1. Download and install [Visual Studio Code](#). This is the desktop version of the text editor used in Codespaces (and is great for all your text editing needs, not just PreTeXt).
2. Download and install [Docker Desktop](#). Once installed, start Docker Desktop and agree to the licensing terms. You should not need to create an account.
3. Download the [PreTeXt Codespace](#) zip file and extract it to a location of your choice.
4. Launch Visual Studio Code. Use the “File” menu to select “Open Folder...” and navigate to the folder where you extracted the zip file in the previous step. Select that folder and click “Open”.
5. You will likely get a popup in the lower right corner of the window asking if you want to install the devcontainer extension, or if you want to open the current workspace in a devcontainer. You do want to do both of these. If you don’t get this message, you can install the “Dev Containers” extension from the Extensions view (click the square icon on the left sidebar) and then use the Command Palette (**CTRL**+**SHIFT**+**P**) to run the command “Dev Containers: Reopen in Container”.

The last step will take a few minutes the first time since it must download the container image.

Upon completing the steps above, you can create a new project and build/view/deploy just like described in [Section 2.2](#).

Whenever you work on a project locally, you will want to open it in the container (but next time it will boot up much faster since you will already have the container image downloaded).

2.3.2 Option 2: Installing the PreTeXt CLI and Additional Software

The PreTeXt CLI is a Python package that can be installed on your computer. This option is more lightweight than using a Docker container, but it does require you to install several prerequisite programs (including Python itself).

Detailed instructions for installing the PreTeXt CLI with PIP are available in [Section 5.2](#). If you know what you are doing, you can just run `pip install pretext[all]` to get all the python and core parts of PreTeXt.

This option is great if you already have a lot of the software that would be duplicated in the docker image from Option 1. In the following list, only python is strictly required, but many features of PreTeXt will not work without the other software.

- **Python** (version 3.10 or later); required.
- **Node.js** (version 18 or later); required to build custom themes for HTML and to build Braille and some epub targets.
- **Git**; required to store your project on GitHub and use the `pretext deploy` command to host your project on GitHub Pages.
- **PreFigure**; required to build `<prefigure>` images. You can get this as part of the PreTeXt CLI installation, if you install the cli with `pip install pretext[prefigure]`, although you might need some additional system libraries installed. See [PreFigure’s website](#) for more information.
- **TeX** (any standard distribution, including TeXLive, MiKTeX or TinyTex will work); required to build PDF output and if your document includes TiKz images (in a `<latex-image>` element).
- **SageMath** (version 10.0 or later recommended); required to build SageMath output and if your document includes sageplots (in a `<sageplot>` element).

While you can use any text editor you like for authoring PreTeXt documents, **Visual Studio Code** is highly recommended due to the availability of the `pretext-tools` extension which provides syntax highlighting, autocompletion, and other features that make authoring PreTeXt documents much easier. You can install this extension from the Extensions view in Visual Studio Code (click the square icon on the left sidebar) or by searching for `pretext-tools` in the Extensions Marketplace (it is automatically installed if you use the Docker container from Option 1).

2.3.3 Option 3: Running PreTeXt “from source”

For advanced users, especially if you want to contribute to the development of PreTeXt, you can run PreTeXt without the CLI. To do this, you will need to clone the [PreTeXt repository](#) and install the required dependencies manually. This is required if you want to run the [PreTeXt script 47](#) or even just use [xsltproc 46](#) to process your PreTeXt documents. Additional information is available in the linked sections.

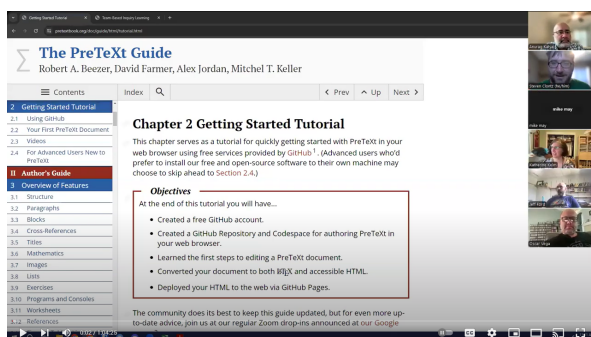
Some context for experienced Python developers: PreTeXt is an open-source XML language primarily powered by [XSLT](#) and [Python](#) tools. PreTeXt development is primarily split over two GitHub repositories: [PreTeXtBook/pretext](#) for the “core” functionality of PreTeXt, and the more recent [PreTeXtBook/pretext-cli](#) that packages up these resources into a Python package with several UX enhancements such as a simplified command line interface and project management that does not require the use of custom makefiles. Instructions for developing with the CLI are available in its repository’s README file.

If you’re interested in potentially contributing back to PreTeXt someday, please feel free to request to join [our developer Google Group](#) and say hello!

2.4 Videos

Occasional “Getting Started with PreTeXt” tutorials are offered to the community via Zoom. For updates on when these are available, subscribe to [the PreTeXt announcements Google Group](#).

The most recent recording of this tutorial is provided here for your reference.



Standalone
In Context

Figure 2.4.1 Getting Started with PreTeXt, 2023 July

Part II

Author's Guide

Chapter 3

Overview of Features

This chapter is a high-level view of the important concepts, features and design decisions that go into the creation of PreTeXt. For careful exact descriptions of details, we will direct you to one of the many sections in the [Topics](#) chapter. So this chapter should make you aware of what is possible and expand on the philosophy described earlier in [Section 1.1](#), while also giving you examples of many basic constructions you can use to get started quickly.

3.1 Structure

A PreTeXt document is a nested sequence of **structural divisions**. For a book, these would go `<part>`, `<chapter>`, `<section>`, `<subsection>`, and `<subsubsection>`. Using `<part>` is optional, but a book must always use `<chapter>` (or else it is not a book!). No skipping over divisions. For example, you cannot divide a `<section>` directly into several `<subsubsection>`s without an intervening `<subsection>`.

An `<article>` starts divisions from `<section>`, though it may choose to have no divisions at all. `<paragraphs>` are exceptional. They lack a full set of features, but can be used to divide anything, in books or in articles, though they are always terminal since you cannot divide them further. You will have noticed that we prefer the generic term **division** (rather than “section”) since a `<section>` is a very particular division.

A division may be unstructured, in which case you fill it with paragraphs and lists and figures and theorems and so on. But if you choose to structure a division it must look like the following:

- An optional `<introduction>`
- One or more divisions of the next finer granularity
- An optional `<conclusion>`.

Either version may have a single `<exercises>` division at the end, or other “specialized” divisions. The structured version may have more than one `<exercises>`, and more than one of each of the types of specialized divisions. For example a `<references>` is a second example of a specialized division. (See [Section 4.7](#).)

The `<introduction>` and `<conclusion>` divisions are meant to be short, and may not contain any other numbered tag. No exercises, theorems, listings, etc. If you want to have an introductory division with any of the numbered elements you are free to omit the `<introduction>` and use the next finer subdivision with a `<title>` of “Introduction”.

Every division tag can carry an `@xml:id` attribute, and it is a good practice to (a) provide one, (b) use a very short list of words describing the content, and (c) adopt a consistent pattern of your choosing. Do not use numbers, you may later regret it. These are optional, and with practice you will learn how best to use them. See [Section 3.4](#) just below for more on this.

The `<exercises>` and `<references>` tags are special divisions, see [Section 4.3](#) and [Subsection 4.7.1](#).

This explanation is expanded and reiterated at [Section 4.6](#) and is worth reading earlier rather than later.

3.2 Paragraphs

Once you have divisions, what do you put into them? Most likely, paragraphs. We use long, exact names for tags that are used infrequently, like `<subsubsection>`. But for frequently used elements, we use abbreviated tags, often identical to names used in HTML. So a paragraph is delimited by simply the `<p>` tag.

Lots of things can happen in paragraphs, some things can *only* happen in a paragraph, and some things are *banned* in paragraphs. Inside a paragraph, you can emphasize some text (`<em>`), you can quote some text (`<q>`), you can mark a phrase as being from another language (`<foreign>`), and much more. You can use almost any character your keyboard can produce, but need to be careful with the three XML exceptional characters: ampersand (&), less than (<), and rarely, greater than (>). (See [Section 3.14](#).) You must put a list inside a paragraph, and all mathematics ([Section 3.6](#)) will occur inside a paragraph. You cannot put a `<table>` or a `<figure>` in a paragraph, and many other structured components are prohibited in paragraphs.

Paragraphs are also used as part of the structure of other parts of your document. For example, a `<remark>` could be composed of several `<p>`. As you get started with PreTeXt, remember that much of your actual writing will occur inside of a `<p>` and you will have a collection of tags you can use there to express your meaning to your readers.

So early in your writing project, familiarize yourself with the components of a paragraph detailed in [Section 4.1](#).

3.3 Blocks

Besides paragraphs ([Section 3.2](#)) the most common object to include in a division is what we informally refer to as a **block**. These are self-contained units of text, almost always set-off visually, and likely with a number and a title. If you know $\text{\LaTeX}$, you may be in the habit of calling these **environments**. Mathematical results are one example, and you can start at [Section 3.20](#) to learn more. There are others that are more general-purpose, such as `<remark>` and `<example>`. While fundamentally different from these blocks that are textual with reflowable lines, objects like `<figure>` and `<table>` ([Section 3.13](#)) or `<program>` and `<console>` ([Section 3.10](#)) are blocks, even if their contents are more rigid or spatial. For a more precise description, see [Section 4.2](#).

3.4 Cross-References

Cross-references in a PreTeXt document are easy, powerful and flexible. So it is worth familiarizing yourselves with them early, here and then ahead in [Section 4.5](#).

Any element that you place a `@xml:id` on can become the target of a cross-reference. This could be a division, a remark, a bibliographic entry, or a figure. So for example, suppose your source had `<subsection xml:id="subsection-flowers">` and someplace else you wrote `<xref ref="subsection-flowers" />`. Then at the latter location you would get a reference to the Subsection that discusses flowers. In print this might just be the number for the subsection, but in various electronic output formats, these cross-references can be very powerful interactive ways to explore the content. And the mechanism is always the same, pair up an `@xml:id` on a target with a `@ref` on an `<xref>` cross-reference.

Since the value of an `@xml:id` is also used in a variety of ways, such as to construct some file names, some care should be taken in how you author them. We limit the possible characters to letters and numbers (a-z, A-Z, 0-9), with hyphens and underscores (-_) available as word-separators. Our advice is to stick to lowercase letters, though we are not yet aware of any problems with case-insensitivity. So in short, use **kebab-case** or **snake-case** for your `@xml:id` values.

For more, see [Section 4.5](#) because cross-references have many features. But first, here are two features you do not want to miss. In the early stages of writing, you can author `<xref provisional="subsection-flowers" />` to point to a subsection you are contemplating (but have not written yet) and you will get various polite reminders to get that straightened out eventually (see [Section 5.9](#) for details). Also the default behavior is to automatically provide the generic name of the target, so you will get something like “Subsection 4.3.2” without ever typing the “Subsection” part. If you move the target, the generic name will adjust if necessary, and if you switch to one of the supported languages, the generic name will switch language (see [provisional cross-reference: topic-on-languages]).

3.5 Titles

Divisions always require titles, you accomplish this with a `<title>` tag first thing. Almost everything that you can use in a paragraph can be used in a title, but a few constructions are banned, such as a displayed mathematical equation (for good reason). Try to avoid using footnotes in titles, even if we have tried to make them possible.

Since titles migrate to other places, such as a Table of Contents, there are options for variants of a title, such as a short version, or a markup-free version. Some (major) titles may also be structured as a sequence of `<line>` elements to control line-breaks for long titles.

Many, many other structures admit titles. Experiment, or look at specific descriptions of the structure you are interested in. Titles are integral to PreTeXt, much like cross-references. Titles migrate to the Table of Contents, get used in page headers for print output, can be used in lists (such as a List of Figures), and can be used as the text of a cross-reference, instead of a number. You might be inclined to not give a `<remark>` a title, but it would definitely be good practice to do so (study [Best Practice 4.8.1](#)). For more details consult [Section 4.8](#).

3.6 Mathematics

With experience, you may realize that PreTeXt utilizes three principal languages. One is the narrative of everyday sentences and paragraphs. Most of what you write in a paragraph, or a table cell, or a title, or a caption, or an index heading, is in this language. Then there is the structural language, which is the majority of the elements in PreTeXt, such as `<chapter>`, `<theorem>`, or `<figure>`. Then finally, there is the language of mathematical symbols and notation.

A key design decision is that mathematical symbols, expressions and equations are authored using $\text{\LaTeX}$ syntax. More precisely, we support the symbols and constructions provided by [MathJax](#), which quite closely follows the `amsmath` package maintained by the American Mathematical Society. Neither you nor I want to write [MathML](#) by hand!

The symbols and macros supported by MathJax can be found at their [Supported \$\text{\LaTeX}\$ commands](#) documentation. Look here to see which parts of $\text{\LaTeX}$ may be used in your mathematical expressions.

For **inline** mathematics, use the short `<m>` tag within a `<p>` (or within a `<title>` or `<caption>`). For example, `<m>\alpha^2 + \beta^4</m>` will do what you expect, in print and in electronic outputs. To get a single equation, centered, with some vertical separation before and after, use the `<md>` tag (“math display”) in the same way within a `<p>`, but do not try using it within a `<title>`. For example, `<md>\rho = \alpha^2 + \beta^4</md>`. If you want your equation numbered, add the `@number` attribute with the value set to yes.

There is a way to incorporate your own (simple) custom $\text{\LaTeX}$ macros within mathematics (only). They will be effective in your print and electronic outputs, and can be employed in graphics languages like TikZ and Asymptote. You can also author multi-line **display mathematics** using the `<md>` tag, but now it should contain a sequence of `<mrow>` elements holding $\text{\LaTeX}$ syntax. We defer the details to [Section 4.9](#).

3.7 Images

You can include an image via the `<image>` tag, using the `@source` attribute to provide a filename, likely prefixed by a relative path from the top-level of the appropriate directory. Read [Section 5.6](#) for details on how to set these directories correctly. If you are starting a new project, using the PreTeXt-CLI (with the command `pretext new book`, for example), then most of the setup portion is done for you and the top-level directory for images that are created external to the project is called `assets`, and it is a sibling of the source directory. It is your responsibility to locate that file properly relative to this directory, and that the file format is compatible. So, for example, suppose your source contained `<image source="images/butterflies.jpg"/>`. Then you would want to have a directory named `images` below wherever you set the `@external` top-level directory, and you would place the `butterflies.jpg` file inside of the `images` directory.

The `@width` attribute can be used to control the size of the image. Widths are expressed as a percentage of the available width, such as `width="60%"`. Instead of a width, you can also specify margins and the width will be deduced.

The optional `@rotate` attribute controls the angular rotation of the image about its center, for example `image/@rotate="30"` will rotate the image 30° counterclockwise.

You may want to wrap your image in a `<figure>` to have it centered, and to have some vertical separation above and below. A `<figure>` must also have a `<caption>`, and the figure will be numbered.

You can also place an “anonymous” image (no caption, no number) almost anywhere you might place a paragraph (but not *within* a paragraph). Note also that the `<sidebyside>` tag provides some very flexible options for placing several images (Section 3.19) together, or combining figures with subcaptions.

If you wish to construct technical diagrams, with editable source, and perhaps including the use of $\text{\LaTeX}$ macros, PreTeXt provides support for authoring with graphics languages such as Asymptote, TikZ, PGF, PSTricks, and xy-pic in addition to using Sage code to describe a plot or image. In most cases output can be obtained as smoothly-scalable SVG images, in addition to other formats like PDF or PNG. Making all this happen is one of the more technical aspects of PreTeXt, so read the details in Section 4.14 along with frequent references there to the pretext script described in Chapter 47.

For accessibility, every `<image>` should either have a description or it should explicitly declare itself to be a decorative image setting `@decorative` to the value `yes`. Descriptions are of two types. A `<shortdescription>` should minimally describe the important information in the image that is not already present in the surrounding text. It should have no element children except possibly `<var>` children. This content will be used as the `@alt` attribute for the HTML `<img>`. Some screen readers may cut off reading this content after the first 100–140 characters, therefore you should keep this element short. A `<description>` element should be structured with `<p>` and `<tabular>` children and has no technical limit on length. It may describe the image more completely. The content of the `<description>` should describe the image, but should not be necessary for a sighted reader to take in the important features of the image. (If there are features of the image that need to be explained to all readers, this should happen in the main body of the text.) An `<image>` may have one or both types of description, but it is incorrect to have neither unless `@decorative` is set to value `yes`.

3.8 Lists

Ordered lists (numbered), unordered lists (bullets) and description lists (defined terms) are all supported, and syntax generally follows HTML. Lists usually live within a paragraph (`<p>`), though there are limited exceptions. Their structure is given by the `<ol>`, `<ul>`, `<dl>` tags (respectively). These can specify a variety of options for the labels via attributes, as described in Section 4.11.

List items, for any of the three types, are delimited with the `<li>` tag. What is different from HTML is that the contents of a list item may be structured, with paragraphs (`<p>`) being the most likely and frequent element. So to nest lists you begin a paragraph in a list item of the outer list, then begin the inner list within that paragraph. However, a simple list item may be authored just like you were authoring within a paragraph, much like writing sentences elsewhere. A structured list item may begin with `<title>` for ordered and unordered lists, and is mandatory for a description list. In this latter case, the text of the `<title>` will become the text that is being described (the label of the list item). For the optional uses, the title will be rendered as its own paragraph, with a different font (perhaps italics or oblique). A description list cannot be contained within another list. In other words, it is a “top-level” list.

Lists are more complicated than they appear, so be sure to read the details at Section 4.11 before you start designing really involved lists.

3.9 Exercises

Textbooks in many disciplines have exercises for the reader. In PreTeXt there are five places where you can set a question for the reader to pursue.

Divisional There is a special `<exercises>` division and an `<exercise>` placed there is then known as a **divisional exercise**. This division supports extra features designed for exercises, such as an `<exercisegroup>` for short exercises with common instructions. The `<exercises>` division can be used at any level. In other words, it can be a peer of any other division.

Inline	Immediately within any division, you can interrupt the narrative with an inline exercise . It will be rendered similar to a <code><theorem></code> or other block, with a number, and a optional <code><title></code> .
Reading Question	Another specialized division, <code><reading-questions></code> , can be used to house <code><exercise></code> designed to test or guide a reader's comprehension of the material in that division.
Worksheet	The main component of a <code><worksheet></code> is an <code><exercise></code> (Section 3.11). Notably the exercises in a worksheet may be arranged with a <code><sidebyside></code> element (Section 3.19).
Project	A <code><project></code> is similar to an inline exercise, other than the type name and the fact that it can run on a separate counter from theorems, figures, etc. (When running on a separate counter, the same counter is used for <code><activity></code> , <code><exploration></code> , and <code><investigation></code> .)

If an `<exercise>` is simply a statement of the question, then it may be authored with paragraphs (`<p>`) and similar elements. If an `<exercise>` has hints, answers, or solutions, then it must be structured with a `<statement>`, followed by (possibly several) optional `<hint>`, `<answer>` and/or `<solution>`. Conceptually, an `<answer>` is a short final result, while a `<solution>` provides details about the route to the answer. Each of these four components is structured further, with paragraph-like elements, and the exercise itself may have a `<title>`. A title is strongly encouraged for inline exercises, and nearly-mandatory if you plan to have inline exercises rendered in knows in a conversion to HTML.

There is a wide variety of interactive exercise types you may specify, such as multiple choice, Parson problems, matching, and more. See [Section 4.12](#) for descriptions of each type and the details of markup for each.

You need (and want) to have the hints, answers and solutions grouped with the statement as you author, but there is a lot of flexibility on making these available at the location of the exercise, or in the back matter. See [Section 4.13](#) for more.

An inline exercise typically gets a fully qualified unique number and is rendered similar to an `<example>` or a `<remark>`. A divisional exercise (including reading questions and worksheet exercises) only gets a sequential number, though this can be overridden with the `@number` attribute if you want to maintain stable numbering in response to edits. (Be careful, once you override the sequential numbering, you probably need to manually specify every subsequent number, so save overrides for when your project matures.)

Within a run of divisional exercises a subgroup can be delimited as an `<exercisegroup>`, which requires an `<introduction>` and allows a `<conclusion>` to explain some commonality. A `<title>` is optional, and a default will be provided otherwise. An `<exercisegroup>` should be rendered in some way that makes it clear to the reader that they are a group.

3.10 Programs and Consoles

If you are writing about computer science, or more general scientific or engineering topics, you may wish to include sample computer programs, or command-line sessions. A `<program>` will contain a complete computer program, or a portion of the code from a program, while a `<pf>` ("program fragment") will display a chunk of code inline with the text. Some `<program>` can execute in an online output format, while others are both editable and executable. This behavior depends on the language used and the host employed. A `<console>` holds a command-line session. These elements feature monospace fonts, preservation of whitespace, and syntax highlighting. These may also be placed in a `<listing>` to be more prominent, or as the target of a cross-reference. See [Section 4.16](#) for details.

3.11 Worksheets

Another division is a `<worksheet>`. It is similar to a `<chapter>`, `<section>`, and so on, but with some variations to support a worksheet or in-class activity. Here we recognize the primacy of printed output (perhaps to bring into a classroom), and the online version is a less-capable representation.

There is no limit to what you can place in a `<worksheet>` division: objectives, an introduction, theorems, figures, images, and so on. But the principal element is an `<exercise>`, which mostly behaves like an `<exercise>` in an `<exercises>` division, but with additional capabilities.

An `<exercise>` in a `<worksheet>` can have a specified width when included in a `<sidebyside>`, and in any case may have a specified additional blank working space of specified height. Page breaks can be specified, and the four margins on a page can be independently controlled. So if you want to create ancillary worksheets for your project, and you like to use *all* of the space on a printed page, then there is some layout control to support that.

Notice that all of this layout control is an exception to the philosophy of PreTeXt. So in particular, margins and working space do not appear in the HTML output. We do give a visual indication where a page break in a `<worksheet>` is placed. An author might wish to collect all of the worksheets in a book, for printing as an “activity book”, and so there are plans (2018-08-11) to support and automate that process. Details for authoring worksheets can be found in [Subsection 4.7.3](#).

3.12 References

Like `<exercises>`, a `<references>` division may go anywhere a more typical division could go. This allows for things like a “Further Reading” list at the end of every chapter of a book. These are populated with `<biblio>` items that are individual bibliographic entries. Support is presently very minimal, but is planned to improve.

3.13 Figures, Tables, Listings, Lists

Some elements in PreTeXt are **containers**, meaning they do not stand by themselves, but are meant to be filled with other elements. Other elements are **atomic**, meaning they cannot be decomposed into smaller elements. A canonical example is the case of a `<figure>`, which is a container meant to be filled with other elements. One such element is an `<image>`, which is atomic. Note that the figure can be filled with other items, and that an image may appear inside other elements, including as a child of a division, perhaps as a peer within a run of paragraphs. A purpose of the container is typically to provide a number and some text (a title, caption, or similar) as identification or description, in addition to indicating to conversions the necessity for some visual formatting (such as small amounts of separating vertical space above and below). Note that an author provides the caption, while PreTeXt provides the vertical spacing.

Here we provide very brief descriptions of these four containers. Be sure to consult the in-depth topics for more details on specifics. Generally, the common thread here is that these containers contain text or graphic elements that has a two-dimensional quality to it. To different degrees the content is rigid. Unlike a paragraph, which could be unwound into a single (linear) long sequence of characters, *something* about the contents of these containers would be lost if stretched out in one dimension. To reflect this rigid two-dimensional flavor, we refer to these objects (and their containers) as **planar**.

A `<figure>` is the most general planar container. It can hold an `<image>`, a `<audio>`, a `<video>`, and more. A `<caption>` is authored early as metadata, but will likely render below. A `<title>` can be used for cross-references or in lists of figures, but may not render where authored. See [Section 4.18](#) for full details.

A `<table>` is the container for a `<tabular>`, an atomic element which is the only allowed content of a table. A `<title>` is the identifying information, and renders above the rows and columns of the table. A variety of notes (to appear below the table) are possible (not implemented as of 2021-12-10). The elements we provide to describe a table are heavily influenced by the discussion in *Chicago Manual of Style* [1, Chapter 13], which is worth reading if your project has many important tables. See [Section 4.19](#) for full details.

A `<listing>` is a container for computer code or programs, to support projects in computer science and other technical disciplines. These languages often rely on indentation (Python) or even exact column numbers of text (FORTRAN), hence a planar quality. Other languages can be written syntactically-correct as one long reflowable line (C, Pascal), but are impractical to do so as part of an exposition. So the allowed content is a `<program>` or `<console>`, which will respect indentation, use monospace fonts, and include syntax highlighting. Otherwise, a listing is very similar to a figure in how a caption and title are handled. See [Section 4.20](#) for full details.

A `<list>` may not be what you think it is. An actual list (be it ordered, unordered, or description) is a common and popular device for organizing information. Start at [Section 3.8](#) for details on lists. Since these lists are considered part of a sentence (within a paragraph), or a part of a paragraph, it is hard to create a cross-reference to them. So when you have a list that is important to mention elsewhere, you can create a **named list** with the `<list>` element, a container that has an optional `<introduction>`, followed by a actual list, and then an optional `<conclusion>`. An example might be a laboratory procedure, such as the steps necessary to dissect a frog. Like a `<table>` this element

should have a `<title>`, and will be given a number. Using a new line for each new list item, and conveying nesting with indentation gives a list a planar quality. See [Section 4.21](#) for full details.

We use a `<table>` to summarize these similar containers.

Table 3.13.1 Containers for Planar Content

	Figure	Table	Listing	List
Element	figure	table	listing	list
Contents	various	tabular	program, or console	introduction ol, ul, or dl conclusion
Title	×	×	×	×
Caption	×		×	
Notes		×		
Details	4.18	4.19	4.20	4.21

3.14 Exceptional Characters

An advantage of XML syntax is that very few characters are reserved for the language’s use, and thus very few characters need to be escaped. Of course, there is always the need to escape the escape character.

The escape character for XML is the ampersand, `&`. The other dangerous character is the left angle bracket, the “less than,” `<`. If you like to be symmetric, you can also handle the right angle bracket, the “greater than,” `>`, similarly. Single and double quotation marks are used to delimit attributes, so are part of the XML specification, but do not present difficulties in narrative text.

These first two characters are interpreted by the XML processor very early in the analysis of your source. So they need to be authored specially via the XML entities `&` and `<`; . In practice, escaping `>` is rarely necessary. So fundamentally within PreTeXt there are just two characters to type carefully or exceptionally.

If you consistently follow the prescription in the previous paragraph you will avoid a descent into escape-character hell and avoid a lot of head-scratching. In particular, you should have no need of the `<![CDATA[ ]]>` mechanism of XML, so *please* just forget we even mentioned it. But see [Subsubsection 4.1.4.2](#) if you are curious, or want a more thorough discussion.

3.15 Nontrivial Characters

Quality typography is more expansive than the limited capabilities of computer keyboards, whose history is rooted in the machines known as typewriters. Some characters are never present on a keyboard (e.g., a pilcrow), while others are ambiguous. Is a diagonal line a slash used to separate information (either/or), or is it a solidus used to form a simple fraction such as $\frac{3}{4}$? [Table 3.15.1](#) is a *sample* of how PreTeXt addresses this. (The last two are the technical exceptions imposed by using a markup language, see [Section 3.14](#).) See [Section 4.1](#) for a more comprehensive and detailed discussion.

Table 3.15.1 Nontrivial Characters

To get this:	Type this:	To get this:	Type this
...	<code><ellipsis/></code>		<code><fillin/></code>
	<code><icon name="gear"/></code>		<code><icon name="wrench"/></code>
	<code><kbd>PrtScn</kbd></code>		<code><kbd name="shift"/></code>
—	<code><ndash/></code>	—	<code><mdash/></code>
®	<code><registered/></code>	©	<code><copyright/></code>
‰	<code><per mille/></code>	¶	<code><pilcrow/></code>
§	<code><section-mark/></code>	·	<code><midpoint/></code>
™	<code><trademark/></code>	~	<code><swungdash/></code>
×	<code><times/></code>	±	<code><plusminus/></code>
/	<code><solidus/></code>	÷	<code><obelus/></code>
°	<code><degree/></code>		
′	<code><prime/></code>	"	<code><dblprime/></code>
&	<code>&amp;</code>	<	<code>&lt;</code>

3.16 Verbatim and Literal Text

Typesetting literal text, usually in a monospace font, can sometimes be tricky. For short bits of such text, as part of a sentence in a paragraph, or in a caption or title, use the `<code>` tag, which is short for “code.” For much longer blocks of literal text, with line breaks that are to be preserved, use the `<cd>` element within a paragraph (“code display”). Outside a paragraph, most anywhere you could place a regular paragraph, use the `<pre>` tag, which is short for “pre-formatted”.

For the content of a `<pre>` element, the indentation will be preserved, though an equal amount of leading white-space will be stripped from every line, so as to keep the code shifted left as far as possible.

The behavior of these two tags is to preserve characters exactly. Certainly the ASCII character set will behave as expected, and Unicode characters will migrate successfully to output formats based on HTML. As mentioned in [Section 3.14](#) the ampersand and left angle bracket will confuse the initial XML processing. So use the XML entities `&` and `<` to represent these characters to the PreTeXt conversion tool, the PreTeXt-CLI. See [Section 4.4](#) for further details.

3.17 Sage

[Sage](#) is an open source library of computational routines for symbolic, exact and numerical mathematics. It is designed to be a “viable free open source alternative to Magma, Maple, Mathematica, and Matlab.” PreTeXt contains extensive support for including example Sage into your document.

A typical use of the `<sage>` tag is to include an `<input>` element, followed by an `<output>` element. The content of the `<input>` element may be presented statically in PDF output, or dynamically as a Sage Cell in an output format based on HTML. Of course, for output as a CoCalc or Jupyter worksheet, the Sage code is presented in the worksheet’s native format.

The content of the `<output>` element is included in PDF output, but not in dynamic instances, since it can be re-computed. Notably, there is a conversion which pairs input and output into a single file in the format used by Sage’s doctest framework. So if expected output is provided, it becomes automatic to identify when Sage has diverged from your expectations, and you can adjust your examples accordingly.

The Sage Cell Server can also be configured to interpret different languages, because Sage by default contains everything needed to evaluate code in these languages. This is done by providing a `@language` attribute, where possible values are `sage`, `gap`, `gp`, `html`, `macaulay2`, `maxima`, `octave`, `python`, `r`, and `singular`. The default is `sage`.

Note that the dynamic formats (including the Sage Cell) may run Sage “interacts,” so that is possible to embed interactive demonstrations into your dynamic output formats.

3.18 Interactives

We strive to make it simple for authors to incorporate interactive demonstrations in the online output of their projects. Of course, this prompts the question of what to do with this content in less-capable formats like PDF or braille.

The `<interactive>` element provides a consistent way to specify these demonstrations. There are many possibilities, but perhaps they can be grouped mostly into three broad classes.

Server Hosted	Interactive demonstrations hosted at external sites, such as those from GeoGebra or Desmos, can be included simply by providing the appropriate identifying information, much like the way you would specify a YouTube video (Subsection 4.33.4).
Source Code	Some interactives can be described by source code that you include in your PreTeXt source. Examples include JessieCode, CircuitJS, and Sage Interacts. We also support GeoGebra this way. This is similar to how we employ Sage Cells, but without as much specialization.
Roll Your Own	If you know HTML, CSS, and Javascript, you can provide your own routines and libraries to incorporate any sort of demonstration you can imagine and can code.

See [Section 4.23](#) for details.

For output formats where executing an `<interactive>` would be impossible, we manufacture a static version. This includes a screenshot of the demonstration (automatically generated, or supplied by the author) and a Quick Response (QR) code that will point to a **standalone** HTML page that contains the interactive. Again, see [Section 4.23](#) for details.

3.19 Side-by-Side Panels

A `<sidebyside>` is a useful organization of elements in a horizontal layout, and so begins to blur the line between content and presentation. While we default to organizing information in a vertical sequence, it is often desirable to organize smaller elements adjacent to each other horizontally. Specifically, images, tabular, figures, tables, paragraphs, and more, may all be combined and there is good control over vertical and horizontal alignment. Captioning, both overall and individually, is especially flexible. An `<sbsgroup>` (“side-by-side group”) collects multiple `<sidebyside>` to stack vertically, which allows for displays in grids. See [Section 4.24](#) for details.

3.20 Mathematical Results

Definitions, theorems, corollaries, etc. are supported by the tags: `<theorem>`, `<corollary>`, `<lemma>`, `<algorithm>`, `<proposition>`, `<claim>`, `<fact>`, `<definition>`, `<conjecture>`, `<axiom>`, and `<principle>`. Each may have a `<title>` (strongly encouraged), and then contains a `<statement>` which is a sequence of paragraphs and other elements. As appropriate, some of these elements (such as a `<lemma>`) may contain an optional `<proof>` (or several), while other elements may not have a `<proof>` (such as a `<conjecture>`).

A `<definition>` is a natural place to define notation as well (see [Section 3.23](#)), and to use the `<term>` tag to identify the terminology being defined.

In order to assist readers locating numbered items, these items are all numbered consecutively in a group that includes `<example>`s, `<remark>`s and inline `<exercise>`s.

3.21 Front Matter

In the beginning of your `<book>` or `<article>` you can have a `<frontmatter>` element that contains various items that would precede your first `<chapter>` or `<section>` (respectively). Possibilities include `<bibinfo>` (to hold **bibliographic information** about your document), `<titlepage>`, `<colophon>`, `<biography>`, `<abstract>`, `<dedication>`, `<acknowledgement>`, `<foreword>`, and `<preface>`. Some of these may be duplicated (e.g., several prefaces for multiple editions), many of these items are restricted to books (e.g., a foreword), and some items are restricted to articles (e.g., an abstract). The schema ([Chapter 6](#)) will help you place them in the right order in your source. See [Section 4.25](#) for details.

The `<frontmatter>` is also employed in other types of documents, such as a `<slideshow>`, in similar, but not identical, ways.

3.22 Back Matter

Similar to front matter, there is material you might wish to include after your book’s final `<chapter>` or your article’s final `<section>`. Possibilities to place in a `<backmatter>` include `<appendix>`, `<references>`, `<glossary>`, `<solutions>`, `<index>`, and `<colophon>`. There are empty tags you can place into an appendix to generate lists of notation, or lists of particular elements of your choice, such as a list of figures. A similar empty element actually generates the index, `<index-list>`, but you will almost always want to place it into the `<index>` division. See [Section 4.26](#) for details on the back matter generally. Also, be sure to read about the powerful and flexible automatic list feature ([Section 4.29](#)), which is not restricted to just the back matter.

3.23 Index and Notation Entries

Construction of an index and a list of notation is accomplished by placing information into your text in the appropriate places in the right way.

The `<idx>` tags denote an index entry. These should be placed within the element that they describe. By this we mean that an `<idx>` element can be placed within a `<theorem>` to refer to just that theorem, or it might be placed within a `<subsection>` to refer to that subsection. When you do this, a natural place to place the `<idx>` is right after the `<title>` and similar metadata. In this way, electronic versions of your work can have an index that is more informative than a traditional index that uses just page numbers, since it will be apparent while reading and using the index just what type of object the entry refers to. (See the end of this Guide in an electronic format for an example, Index). Note that the text contained within the `<idx>` tags does not actually appear in the article—it only serves to mark the location the index entry points to. You can have several levels of headings by structuring your `<idx>` element with up to three `<h>` tags (“h” for “heading”). Additionally, you can use `<see>` or `<seealso>` for cross-references within the index.

See [Section 4.27](#) for more details and the finer points of creating index entries.

A similar device is used to create a list of notation for a technical (mathematical) work. Place a `<notation>` element as close as possible to the place where notation is first introduced. If you use the `<definition>` tag for your definitions, then this is a very natural place to also introduce notation. Inside of `<notation>` use the `<usage>` tag to include a short example of the notation in use, wrapped in an `<m>` tag and use $\text{\LaTeX}$ syntax (as usual). The `<description>` tag should contain a very short description in words of what the notation is for. So “center of a group” would be a good description to accompany the usage “ $Z(G)$.”

See [Section 4.28](#) for more details and the finer points of creating notation entries.

3.24 WeBWork Exercises

It is possible to include WeBWork/PG problems in your source. These can be authored using PreTeXt, included from local PG files, or referenced from a host WeBWork course. A static version of the exercise will be rendered in your static output formats such as $\text{\LaTeX}$ /PDF. A “live” version will be rendered in HTML output. If the HTML is hosted on a Runestone server, the student’s progress completing the live exercises can be tracked. You can also extract the problems into a single archive suitable for uploading into a WeBWork course. This is a big topic, so see the dedicated [Chapter 7](#) for details.

3.25 MyOpenMath Exercises

MyOpenMath is a hosted online homework system with hundreds of thousands of prebuilt questions. These questions can be embedded in PreTeXt using the `<myopenmath>` tag. See [Section 4.30](#) for details.

In the web output, the MyOpenMath question will be embedded and interactive though not tied to any student or faculty account. In static output, a static version of the question will be included. Note that there are some limitations on which problems will display correctly in web and static output. See [Section 4.30](#) for best practices.

3.26 STACK Exercises

STACK is an online assessment system, and these exercises can be embedded in a PreTeXt document. STACK is an acronym for “System for Teaching and Assessment using a Computer algebra Kernel.” See [Section 4.31](#) for details.

3.27 URLs and External References

The `<url>` tag always requires an `@href` attribute. Usually this will be a complete address for some external web page, or other external resource. The `@visual` attribute is sometimes mandatory, but sometimes optional. It should provide a simplified version of the URL for use in print, or similar situations. Finally, you can provide content for the `<url>` element, which will become the clickable text in most realizations.

If the `<url>` element is empty (no content), then the value of the `@href` attribute or the optional `@visual` attribute will be the link text, with a preference for the latter. When you instead provide content, you can use PreTeXt elements much like any other piece of text that would occur in a paragraph. In this case, a `@visual` attribute is now highly recommended, as an alternative to the content, providing information about the actual URL for non-electronic formats like print. A default version of the value of the `@href` attribute will be used in its absence. This visual version of the URL will appear in a trailing parenthetical for print PDF versions.

See [Section 4.32](#) for an example and full details. There is a similar `<dataurl>` element for pointing to supporting files, see [Subsection 4.32.2](#).

3.28 Video

Videos, either author-hosted, via a URL, or hosted on YouTube, may be embedded in a PreTeXt document that is converted to HTML, and may be optionally “popped-out” to view on another page. For a YouTube video, it is simplicity itself, as an author need only supply the identification string, and all the details of the embedding are handled by PreTeXt. See [Section 4.33](#) for details.

3.29 Scientific Units

If you are writing about science or engineering, or even if you are not, there is extensive support for scientific units. So, for example, you could author a force in metric units as

```
<quantity>
  <mag>20.7</mag>
  <unit prefix="kilo" base="gram" />
  <unit base="meter" />
  <per base="second" exp="2" />
</quantity>
```

This would be rendered as $20.7 \frac{\text{kg}\cdot\text{m}}{\text{s}^2}$. More in [Section 4.35](#).

3.30 Localization

We are interested in helping authors produce documents with open licenses around the entirety of the world. PreTeXt provides infrastructure for doing so (or **internationalization**) in accordance with guidelines of the [W3C Internationalization \(I18n\) Activity](#) at w3c (www.w3.org).

In order to actually adapt PreTeXt to another language (**localization**), there are two requirements:

- A file for localization into the desired locale.
- The author adds an `xml:lang` attribute on the `<pretext>` element.

See [Section 4.40](#) for more details.

3.31 Accessibility

The [Web Accessibility Initiative](#) at w3c (www.w3.org) says:

The Web is fundamentally designed to work for all people, whatever their hardware, software, language, culture, location, or physical or mental ability. When the Web meets this goal, it is accessible to people with a diverse range of hearing, movement, sight, and cognitive ability.

Thus the impact of disability is radically changed on the Web because the Web removes barriers to communication and interaction that many people face in the physical world. However, when websites, web technologies, or web tools are badly designed, they can create barriers that exclude people from using the Web.

Since we are interested in helping authors produce documents with open licenses, and we concentrate on employing open standards for the HTML output we create, we are ideally positioned to help you easily create highly-accessible documents. There are many technical features which happen automatically, and there are some features which we make available for your use as an author and which only an author can provide, or elect to use. Before getting too deep into your project, review [Section 4.41](#) for full details and ways you can make the HTML version of your document more accessible, and more useful for a wider audience.

3.32 Slides

In addition to articles and books, support for authoring slideshows is also available, but is experimental (so tags and attributes are subject to change).

Currently, support for producing [Reveal.js](#) and [Beamer L^AT_EX](#) slideshows is available. Details on authoring may be found in [Section 4.42](#), and publishing details are in [Chapter 34](#).

Also, future support for annotating PreTeXt books and articles to export PreTeXt slides is in the works, but not yet available.

3.33 Literate Programming

If you know what **literate programming** is, then you may not be surprised to learn that PreTeXt provides excellent support primarily with two additional elements: `<fragment>` and `<fragref>`.

If you have never heard of literate programming, it is a way to mix code and documentation for a computer program in a single file. But rather than typographically simple code comments, you have the full power of every other feature in PreTeXt and all of the possible output formats. Furthermore, you can arrange your code in an order that might make more sense to a human (top-down, bottom-up, a mixture, or ...) and PreTeXt will rearrange the code into an order that the compiler or interpreter understands.

So the idea is that from one file you get a program for the compiler, and a beautiful, typeset explanation for a human reader in any format PreTeXt supports. An accessible introduction is Knuth's description of his WEB system for the Pascal programming language [2], or many more resources are at [the literate programming site](#). Full details on the PreTeXt implementation are at [Section 4.43](#).

Chapter 4

Topics

This long chapter provides the main documentation for a variety of the features of PreTeXt. Some sections are just stubs and need to be written. Requests for sections to prioritize are welcome, though some sections are waiting for features to stabilize.

4.1 Paragraphs

Much of your writing will happen in paragraphs, delimited by the simple tag, `<p>`. You are reading one right now. They are a basic building block of divisions, and also a basic building block of other structures. For example, an ordered list, `<ol>`, contains a sequence of list items, `<li>`, and a typical list item might be a sequence of paragraphs. (Do not confuse this element with the anomalous `<paragraphs>` subdivision, [Section 4.6](#)).

Paragraphs are a choke-point of sorts. Many tags can *only* be used within paragraphs, and many others *cannot* be used within paragraphs. Notice too, that we do a certain amount of manipulation of whitespace in a paragraph, in ways that you may not even notice.

The following subsections together contain allowed, or encouraged, markup within a paragraph. Many of these may be used in captions and titles, but some of the more complicated constructions (which appear later here) cannot be used in captions or titles.

One more comment: typewriters, computer keyboards, and the ASCII character set have together conspired to limit the full range of characters that typographers and printers have used historically. A case in point is the hyphen, which is a single key on a keyboard. However, there are at least three common dashes of differing lengths (hyphen, en dash, and em dash), and in the context of mathematics or a computer program, the hyphen might be the binary operation of subtraction or the unary operation of negation. Another example is the “upright”, or “dumb”, quote mark that is a single key on a keyboard, while careful typography will employ “smart”, or “curly”, quote marks that have left and right variants. (Sometimes called “66” and “99” based on their shape.) PreTeXt will help you navigate this complexity, but you will want to use keyboard characters or markup appropriately. So if you care about communicating clearly, and making your writing easy for a reader to use, absorb the details that follow and the philosophy they implement.

We will say it again. PreTeXt is a markup language, and our various output formats (L^AT_EX, HTML, EPUB, Jupyter notebooks) in turn employ markup languages. These use different escape characters and give different characters special meanings. Our job is to insulate you from this variety, so you can concentrate on authoring your *ideas*.

We begin with some simple “grouping” elements which contain several excellent examples of the importance and utility of careful markup. There is a plethora of empty tags for individual characters, and these are very important (see [Subsection 4.1.4](#)). We defer them to the end of this section, since they are not as instructive, but do not think this means they are an afterthought. They can be extremely critical for successful conversions. Also do not miss [Best Practice 4.1.8](#) in the conclusion of this section.

4.1.1 Simple Markup in Paragraphs

Beyond empty tags that translate to various characters, there are relatively simple tags that can call attention to various *portions* of a sentence, or generate more complicated constructions than described above.

Most, if not all, of the markup in this subsection may also be used within titles and captions, though they might lose some of their features when used in a title, especially when the title is duplicated in other contexts, such as a Table of Contents.

<q>, quotes, “group”. This is the first of several grouping tags, using characters with left and right variants, and some of the most common markup in your writing. Presentation uses double quote marks that are **smart quotes**, meaning that they look different in their opening and closing variants. (See `<blockquote>` for extensive runs of quoted text that can stand alone, and which can carry an attribution.)

<sq>, single quotes, ‘group’. Perhaps less-often used than `<q>`, so a couple more characters to type. Presentation is paired single-quotes, opening and closing.

<angles>, angle brackets, <group>. Left and right angle brackets to enclose a phrase. This is not for creating a set of generators in mathematics, use the appropriate mathematics tag and syntax for that. Note also that the characters used here are definitely distinct from the inequality symbols, `<` and `>`.

<dblbrackets>, double square brackets, [[group]]. Double left and right square brackets to enclose a phrase. This is not for grouping expressions in mathematics, use the appropriate mathematics tag and syntax for that. These are used in the analysis of texts to note various restorations or deletions. Inquire if you feel there should be more semantic markup for this purpose.

, emphasis, *important*. Use this element to surround characters in a phrase that is to be emphasized. This will typically be rendered in italics, though this choice is left to the implementation of a particular conversion. See also, `<alert>`.

If you are new to using a markup language, this is a place to stop and think. As a PreTeXt author you are never able to say, “I want this text to appear in italics.” Rather, you specify that certain text has a certain purpose or meaning. Emphasis is a way of *calling attention* to a portion of a sentence or paragraph. A font change (to italic) is a common and effective device. But a particular format might have a better, or different, way to achieve this. Perhaps in an electronic format, the letters are animated and dance up and down. (Just kidding. But you may be reminded of frequent blinking text in the early days of web design, supported by a non-standard `<blink>` element.) Seriously, now would be a good time to review [Section 1.1](#).

<alert>, alert, *critical*. Use this to heavily emphasize something to a greater degree than just emphasis. Maybe think of it as *SHOUTING*. Bold italic font, or a bright color, or both, would be normal choices for presentation. Overuse of this tag will dilute its effectiveness.

<term>, terminology, larvae. Use this to identify a word or phrase that is being defined, in contrast to actually using a structured `<definition>`. Typical presentation is a bold font. Caution: the use of this tag is to communicate a defined term and converters may make use of this interpretation, given the importance of definitions in scholarly work. It would be considered **tag abuse** to use this tag to simply bold a word or phrase for some other reason, perhaps as an alternative to `<em>` or `<alert>`.

<foreign>, foreign words, idioms, phrases, *Hola*. This tag is used to identify words or phrases from a language other than the main one used for the overall document. It is best practice to use a `@xml:lang` attribute to identify the language, since this will assist screen readers and hyphenation algorithms. We may also recognize the need for a different script (font). Usual presentation is italics for languages using a Latin script. This should not be used for entire paragraphs as a way of assisting with a translation of an entire document.

Note that when we use italics for emphasis *and* to point out foreign words or phrases, there is a loss of information in our output. In other words, we can no longer reliably (in an automated way) convert our output back to equivalent PreTeXt source from its visual representations. *C’est la vie*. See [Section 1.1](#) again.

<pubtitle>, <articletitle>, titles of books and articles. These provide the ability to typographically distinguish the title of another work, and are not a replacement for careful bibliographies and citations. Use <pubtitle> for longer, complete works, such as books, plays, or entire websites. Use <articletitle> for shorter, component works, such as a chapter of a book, a research article, or a single webpage.

Presentation for a longer work will be italics or an oblique (slanted) font, and for a shorter work, the title will simply be quoted.

<abbr>, <init>, <acro>, abbreviation, initialism, acronym, Mr., XML, SCUBA. An abbreviation is a condensed or shortened version of some word or phrase, such as MR. for “Mister.” Converters should take care with periods (full stop) inside an <abbr> as distinguished from the end of a sentence (which may not always be clear given the absence of a tag delimiting sentences). An initialism is an abbreviation read as a sequence of letters, often the first letter of words in a phrase, such as HTML for “HyperText Markup Language.” An acronym is much like an initialism, but the letters are read as a pronounceable word (which sometimes actually enters the language as a word, such as “radar” which began as RAdio Detection And Ranging). An example is scuba which stands for “Self-Contained Underwater Breathing Apparatus.” Initialisms and acronyms may be presented in a small-capitals font or as regular capitals reduced in size.

<delete>, <insert>, <stale>, editing assistance, ~~gone~~, new, old. These denote portions of a text that is being changed in some way, presumably as part of an editing process. Conceivably, they could be managed by some other tool acting on your source. Stale text is that which is slated for removal eventually, but is left in place so that it may be consulted. There is no support presently for actually deleting or incorporating text, though that would be a reasonable feature request.

Red and green, for leaving and entering, are natural choices for presentation. But in consideration of those readers who cannot always distinguish different colors, other devices, such as strikethrough or underlining, should also be employed.

<tag>, <tage>, <attr>, tag, empty tag, attribute, <section>, <hash/>, @width. These are PreTeXt tags for when we write about PreTeXt and need to discuss tags, empty tags, and attributes. Given how we design PreTeXt tags the content of these elements should only be the 26 lower-case letters and a dash/hyphen. These should render in ways that make the three types of language elements obvious without much further discussion. Just a bit self-serving, but not unjustified.

These are not meant for more elaborate constructions. For example, an attribute-value pair should not be authored as

```
<attr>data="/home/data"</attr>
```

and instead I would write

```
set the <attr>data</attr> attribute to the value <c>/home/data</c>
```

Similarly, do not use the <tag> element for long XPath constructions like

```
<tag>docinfo/prefigure-preamble/line</tag>
```

The <c> or <cd> element will likely work better.

<taxon>, scientific names, *Escherichia coli*. This element may surround a full scientific name, resulting in presentation in italics. There are subelements <genus> and <species> which may be used to delineate those components.

A @ncbi attribute on <taxon> accepts an identifier from the [National Center for Biotechnology Information](#). Feature requests for ways to make this more useful are welcome.

<fn>, footnotes. A footnote can be inserted in a paragraph and a mark will be left behind. Where the content of the footnote goes depends on the capabilities of the output format. Because a footnote allows you to begin a new piece of text *anywhere*, it can be difficult to handle technically. For this reason it is banned from places like titles and its possible content is limited (for openers, no paragraphs).

A footnote is the farthest thing from structured writing that we can think of. It can go anywhere. Resist the temptation to use it, and your writing will improve. We frequently entertain the thought of making footnotes impossible in PreTeXt. See the `<aside>` element for a possible alternative.

<m>, mathematics, $x^2 + y^2$. Simple, inline expressions using mathematical notation may be used in paragraphs, and in titles and captions. The syntax is L^AT_EX. See [Section 4.9](#) for full details.

<c>, code, verbatim text, literal text, import. Short bursts of raw, or verbatim, text can be wrapped in the `<c>` element. Strictly speaking, “code” is a misnomer, as the text may be anything you need to communicate exactly as one would type it at a keyboard or as input to a computer program. Anything longer than a handful of characters, or needing accurate line breaks should consider the `<cd>`, `<pre>`, `<program>` or `<console>` tags. If you like to have your source word-wrapped with hard line-breaks (newline characters), we will replace those for you with a space character. This is the only modification made to the content of a `<c>` element.

Presentation is normally a monospaced sans serif font, perhaps of a slightly heavier weight, and designed for the job with features such as unambiguous zeros (versus the letter ‘oh’). See [Section 4.4](#) for details.

<email>, email address, [nobody@example.com](#). Very similar to the `<c>` tag, this may be used to get a monospace presentation of an email address, possibly as an active link in some formats.

4.1.2 Cross-References and Paragraphs

There are several devices for creating cross-references. Generally, these are unwise (or banned) in titles, and if allowed may be inactive in certain portions of an electronic output format (such as when migrating to a Table of Contents).

<url>, linking external resources. This is a cross-reference *to* some item separate and distinct for your document.

A Uniform Resource Locator (URL) is, loosely speaking, an Internet address for some item. Capabilities for providing links to such items are provided and take into account usability in different output formats. See [Section 4.32](#) for full details on how to use `<url>` for links to external resources.

Note also that this tag is meant for *external* resources, so see the `<xref>` element (below) or [Section 4.5](#) for ways to link internally (i.e. within your document).

<xref>, cross-references. This is a cross-reference *to* some item contained within your document.

Extensive and intuitive capabilities for cross-referencing are a primary feature of PreTeXt. Typical use is an empty tag with the `@ref` attribute specifying the value of an `@xml:id` on the **target** of the cross-reference. This should work easily without much more instruction, but familiarize yourself with the details in [Section 4.5](#) to get the most out of some the available options.

<idx>, index target. This indicates that the containing structure (theorem, example, etc.), or top-level paragraph, should be the *target* of an entry of the index (a special sort of cross-reference). See [Section 3.23](#) and [Section 4.27](#) for general details.

<notation>, index target. This indicates that the containing definition, or top-level paragraph, should be the *target* of an entry of the list of notation (a special sort of cross-reference). See [Section 3.23](#) and [Section 4.28](#) for general details.

4.1.3 Structured Markup in Paragraphs

There are three categories of items which typically are structured further, and which are almost entirely restricted to appearing in a paragraph. Given their complexity, details are covered in other sections of this guide.

Lists. With only one major exception (and three minor ones), a list *must* appear within a paragraph. See [Section 3.8](#) for an introduction and [Section 4.11](#) for precise details.

Display Mathematics. Displayed mathematics, which is a single equation or a sequence of (aligned) equations, can only be placed within a paragraph. The relevant tag is `<md>`. It can hold a single expression or be structured with `<mrow>` elements. See [Section 3.6](#) for an introduction and [Section 4.9](#) for precise details.

Display Verbatim. The `<cd>` tag, by analogy with the `<md>` tag for displayed mathematics, may be used to display one or more lines of verbatim text (such as a series of commands), possibly structured with the `<cline>` tag. See [Section 3.16](#) for an introduction and [Section 4.4](#) for precise details.

This should not be confused with the `<pre>`, `<console>`, or `<program>` tags, which have slightly different uses, and all of which must be used *outside* of a paragraph.

4.1.4 Characters in Paragraphs

Some keyboard characters are unambiguous, for example, the percent sign, `%`. Other keyboard characters are poor replacements for several different characters. Is a slash, `/`, being used to separate information/ideas, or is it a **solidus** being used to form a fraction such as $\frac{3}{4}$? Other characters, such as per-mille, `‰`, are not present on keyboards at all. We organize this section according to these types of distinctions.

4.1.4.1 Unambiguous Keyboard Characters

The keyboard characters ```, `~`, `!`, `@`, `#`, `$`, `%`, `^`, `*`, `(`, `)`, `_`, `=`, `+`, `[`, `]`, `{`, `}`, `\`, `|`, `:`, `;`, and `,` are entered as-is and are only rendered one way. Easy.

Of course, the fifty-two Latin letters, and ten decimal digits, are also in this category. If you have an international, or bilingual, or country-specific keyboard, then common accented versions of Latin letters (as used in Europe and the Western Hemisphere) may also be used directly from your keyboard.

4.1.4.2 Exceptional Keyboard Characters

XML is a **markup language**, which in part means that some keyboard characters are co-opted to signal the start of markup. For XML this character is the less-than symbol, `<`. It signals the start of a **tag**, and then an opening tag ends with a greater-than symbol, `>`, while a closing tag has an extra `/` right after the `<`.

This begs the question: if a `<` is used in our XML source to signal the start of a tag, then how did we get one to appear here in this sentence without mistakenly starting a tag? Once a markup language gives some characters special meanings, then there needs to be an **escape character**. For XML the escape character is the ampersand, `&`. So to author the `<` and `>` symbols, we type **escaped** versions: `<`; and `>`;

I hear you now say, “But now we just took the `&` out of the running and gave it a special meaning. How do we get an ampersand?” Easy, use the escaped version: `&`;

So the short answer is: never, ever type the `<` or `&` keyboard characters in isolation. The very beginning of the processing of XML (i.e. PreTeXt) will fail fatally on these characters. Instead, always use the sequences `<`; and `&`; and then very early the XML processing will convert them to characters, *without* interpreting them as signals for aspects of the markup.

It does not seem necessary to author `>` as `>`; , though there is no real harm in doing so. The two other characters with escaped versions are the single and double quotes, `'` and `"`, which have escaped version of `'`; and `"`; (respectively). These are only necessary for attribute values, and we have been careful to design PreTeXt so that they are not necessary.

Remark 4.1.1 Excessive Escaped Characters. If you know another markup language, such as $\text{\TeX}$, $\text{\LaTeX}$, Markdown, JSON, or PGML, think about how many characters have been given special meanings, and the subsequent necessity to use escaped versions. And if you want to write about computer languages, realize that each such language also gives certain keyboard characters special meanings.

XML only has five exceptional characters, and in your daily use, PreTeXt really only requires you to be aware of two, the minimum necessary for a markup language.

Best Practice 4.1.2 A CDATA Section is Never Necessary. We hate to mention it, but sooner or later, we need to have an uncomfortable discussion about the misunderstood CDATA section, and risk confusing the rest of this subsection. And this is the place. But you can come back later, if you wish.

You will read other places about very special markup known as a CDATA section. The name stands for **character data**, which means “all characters, no markup”. Think of it as switching off the XML processing for a while, so in particular, &, <, > no longer have any special meaning at all. That *could* be nice, but realize that now there is no opportunity to have any markup present using XML syntax, since it is ineffective.

A CDATA section is always a convenience and is never necessary.

When would it be convenient? Maybe you have some $\LaTeX$ inside an `<md>` with a large matrix that uses lots of ampersands to separate the entries. Inside a CDATA you can author it with bare & rather than a plethora of `&` or `\amp`. But you lose the ability to include an `<xref>` in that CDATA, so you need to be surgical about its scope. Perhaps a Tikz diagram in a `<latex-image>` has a multitude of `<->` or a chunk of Sage code in an `<input>` has a lot of finitely-generated algebraic structures authored as `R.<x> = ...` (which is not even legal Python syntax either!). These places where there is little, or no, markup could be *convenient* places to use a CDATA. Be sure to read the warning at [Item 7](#) in [Section 5.10](#) before you go all-in.

4.1.4.3 Ambiguous Keyboard Characters

Some keyboard characters have a primary interpretation, and are imitations of other typographic characters. Your output will be of higher quality if you understand these distinctions and employ the proper variant.

Table 4.1.3 Ambiguous Keyboard Characters and Alternatives

Keyboard	Primary	Notes
/	(forward) slash	<code><solidus/></code> is a fraction bar,/
'	apostrophe	<code><rsq/></code> is a right single quote, ’
`	backtick	<code><lsq/></code> is a right single quote, ‘
.	period	abbreviations <i>and</i> end-of-sentence
-	hyphen	See dashes, and arithmetic
"	upright double quote	<code><lq/></code> is “, <code><rq/></code> is ”

Note that the four quote marks (left/right, single/double) are meant for the actual characters. Always use the grouping constructions described above (i.e. `<q>` and `<sq>`) when grouping a phrase with quote marks. Note, too, that there is never a good reason to use the keyboard quote character (") unless you are creating some sort of verbatim text, such as a program listing or describing literal keyboard input.

When creating print or PDF via $\LaTeX$ a period may get different trailing space depending on location and context, generally being its use in abbreviations or to conclude a sentence. We do not yet have this dual-use under control.

4.1.4.4 Extraordinary Characters

Some characters or symbols are typically not available on a keyboard, so we provide empty elements. Many of these may be entered directly into your source as Unicode characters, and they will do well in your HTML output. However, these may fail entirely if you create print or PDF via $\LaTeX$ using the `pdf $\LaTeX$` engine. Furthermore, even for HTML output there may be several Unicode characters that are very similar.

So again, for the best quality output be aware of these elements and use them. Please suggest additions if you do not find what you need and are resorting to Unicode characters.

`<ellipsis/>`, ..., ellipsis. Typically three low dots with no intervening space, to indicate a continuation. This will always perform better than three consecutive periods.

`<midpoint/>`, ·, midpoint. A small centered (vertically) dot, which can be used to separate pieces of information, especially in displayed text (i.e. outside of paragraphs). Not to be confused with a bullet preceding a list item, or multiplication in mathematics.

`<swungdash/>`, ~, swung dash. Another decorative separator, not to be confused with the keyboard tilde character since it is wider and thicker.

<permille/>, ‰, **per mille**. Like per cent, but now a number expressed as its product with 1000 (rather than with 100).

<pilcrow/>, ¶, **pilcrow**, **paragraph mark**. Mark used historically to indicate the start of an internal paragraph, and in a more modern use, to indicate a permalink.

<section-mark/>, §, **section mark**. Used to prefix the number of a section, or other division. (So the word section is being used generically here.)

<copyright/>, ©, **copyright**. The symbol used in publishing, legal, or business contexts. For a PreTeXt project, copyright information can be specified within the <bibinfo> portion of the <frontmatter>.

<trademark/>, ™, **trademark**. The symbol used in legal or business contexts.

<registered/>, ®, **registered**. The symbol used in legal or business contexts.

Table 4.1.4 Extraordinary Characters and Their Empty Elements

Character	Name	Element
...	ellipsis	<ellipsis/>
·	midpoint	<midpoint/>
~	swung dash	<swungdash/>
‰	per-mille	<permille/>
¶	pilcrow	<pilcrow/>
§	section-mark	<section-mark/>
©	copyright	<copyright/>
™	trademark	<trademark/>
®	registered	<registered/>

4.1.4.5 Accented Characters

The second 128 Unicode characters (hex 80 to FF) contain many of the most frequently-used accented characters in Western languages, along with niceties such as the German *eszett*, ß, or the Scandinavian *æsc*, æ, an a-e ligature. Like the fifty-two Latin letters (part of the *first* 128 Unicode characters), these may be used as-is. They may be present on your keyboard, or you may need to learn keyboard shortcuts or specifics of your operating system to enter them as Unicode characters. In a pinch, you can often cut-and-paste a few characters from web pages.

This table is indexed by the Unicode number, in hexadecimal notation. The first 32 of the 128 (U+0080–U+009F) are control codes and U+00A0 is a non-breaking space, so is invisible, while U+00AD is a soft hyphen (which we have not implemented and so is excluded).

Table 4.1.5 Latin-1 Supplement, Unicode U+00A0–U+00FF

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00A_		¡	¢	£	¤	¥	¦	§	¨	©	ª	«	¬		®	¯
00B_	°	±	²	³	´	µ	¶	·	¸	¹	º	»	¼	½	¾	¿
00C_	À	Á	Â	Ã	Ä	Å	Æ	Ç	È	É	Ê	Ë	Ì	Í	Î	Ï
00D_	Ð	Ñ	Ò	Ó	Ô	Õ	Ö	×	Ø	Ù	Ú	Û	Ü	Ý	Þ	ß
00E_	à	á	â	ã	ä	å	æ	ç	è	é	ê	ë	ì	í	î	ï
00F_	ð	ñ	ò	ó	ô	õ	ö	÷	ø	ù	ú	û	ü	ý	þ	ÿ

4.1.4.6 Arithmetic

If you are writing about technical subjects, then you will want to avail yourself of PreTeXt's extensive support for mathematics. Otherwise, you may wish to write *really simple* arithmetic within sentences without extra formatting. Notice that there is no provision for preventing line-breaks in the midst of these expressions.

So you can author $(2 \times 6) \div 3 + 10 - 15 = -1$, but that is about the limit of the complexity of expressions you should author without using the extensive capabilities designed for *mathematics*, rather than *arithmetic*. Note that the spaces around the equal sign have been supplied in the source, but no spaces have been provided around the operators. Also, the minus sign and the negative are slightly different because the subtraction uses the `<minus/>` element, while the negative answer uses a plain keyboard hyphen/dash.

Using the `<m>` element instead, the above is $(2 \times 6) \div 3 + 10 - 15 = -1$. Note the more careful spacing, and the appropriate symbols for subtraction and negation, with no special care in the $\text{\LaTeX}$ syntax used in the source.

Note also that the plus sign, + and the equals sign, =, can be provided in text as the unambiguous keyboard characters.

The `<degree/>`, `<prime/>`, `<dblprime/>` elements support simple coordinates with degrees, minutes, seconds, or temperature, or distance in feet and inches. “We parked the car at $36^{\circ}16'0.83''\text{N}$, $122^{\circ}35'47.27''\text{W}$, and since it was 93°F , we walked $505'3.6''$ so we could swim in the bay.”

`<minus/>`, −, minus, subtraction, negation. For simple arithmetic expressions in text, this symbol may be used. Note that the keyboard hyphen (or dash) might be acceptable for your purposes, but they are different.

`<times/>`, ×, times, multiplication. For simple arithmetic expressions in text, this symbol may be used. Or it may be used to specify dimensions, as in “I bought a 2×4 at the lumber yard.”

`<solidus/>`, /, solidus, virgule, fraction bar. For simple arithmetic expressions in text, this symbol may be used to form a fraction. It should appear to have a significantly shallower slope than the forward slash, /.

`<obelus/>`, ÷, obelus, division sign. For simple arithmetic expressions in text, this symbol may be used to indicate division.

`<plusminus/>`, ±, plus-minus sign. For simple arithmetic expressions in text, this symbol may be used to indicate a tolerance or a choice of two values, one the negative of the other.

`<degree/>`, °, degree symbol. A raised open circle for temperature or for angles used in coordinates.

`<prime/>`, ', prime symbol. A straight mark that is placed like an exponent. For use in coordinates or statements of linear measure in feet and inches. Not an apostrophe, and not mathematics (like, say, not to denote a derivative).

`<dblprime/>`, ", double prime symbol. Two straight marks that are placed like an exponent. For use in coordinates or statements of linear measure in feet and inches. Not an apostrophe, and not mathematics (like, say, not to denote a second derivative).

4.1.4.7 Separators

`<ndash/>`, –, en dash. A dash, the width of a lowercase ‘n’, or exactly half the width of the em dash. This is typically used to express a range, such as $1955\text{--}1975$, with no intervening spaces. It is often expressed as two hyphens when typed. Bringhurst suggests an ndash surrounded by spaces – thusly – when setting off phrases.

`<mdash/>`, —, em dash. A dash, the width of a lowercase ‘m’, or twice the width of the en dash. This is typically used to express a secondary part of a phrase, much like the use of a semi-colon or parentheses.

Style guides suggest that there should be no spaces, before or after, an em dash, while some allow for a “thin” space on either side. You should always leave no space around an `<mdash/>` element in your PreTeXt source. Then a publication file entry can be used to elect the automatic addition of a thin space, should your publisher so desire. See [Subsection 44.1.5](#) for the syntax of the publisher file entry.

<nbsp;/>, non-breaking space. A space, but which ties two words together and discourages a line break when formatted, such as Summer />1967. This can also be used to discourage a period in an abbreviation from being interpreted as the end of a sentence, such as C.R. />Darwin.

<midpoint/>, <swungdash/>, , , ~, midpoint, swung dash. These can be used as more decorative ~ separators.

4.1.5 Keyboard Keys

If you are writing a software manual, or writing about how to use a calculator as part of a science textbook, then you might want to make it very clear which keys to press on a keyboard. The `<kbd>` element can hold content like `Z` or `Caps Lock` and your output will have a very nice looking keyboard key with the desired label. For example, `<kbd>Z</kbd>` and `<kbd>Caps Lock</kbd>`. For keys labeled with graphics, like the arrow keys, instead of content, provide the `@name` attribute with a value from the following table. Request additions to this table if you are using this feature (2019-11-22).

Table 4.1.6 Keyboard Keys Specified by `@name` Attribute

left		right		up		down	
enter		shift		solidus			
plus		minus		times		left-paren	
obelus		squared		inverse		right-paren	

4.1.6 User-Interface Icons

When writing about software, you may wish to call attention to icons used in the interface. Similar to graphical keyboard keys, use the `<icon>` element with a `@name` attribute with a value from the following table. See [Section 30.15](#) for information about using icons as part of $\text{\LaTeX}$ output.

Table 4.1.7 User-Interface Icons available with `<icon/>`

Name	Icon	Name	Icon	Name	Icon
file-save		arrow-left		media-rewind	
arrow-up		arrow-right		media-fast-forward	
menu		arrow-down		media-skip-to-start	
wrench		media-stop		media-pause	
gear		media-play		media-skip-to-end	
power					

4.1.7 Other Markup in Paragraphs

<today/>, <timeofday/>, June 1, 2026, 14:34:29 (-07:00). Values at the time of XML processing. Useful for marking drafts or other frequently revised material such as online versions.

<tex/>, <latex/>, <xetex/>, <xelatex/>, <pretext/>, <webwork/>, $\text{\TeX}$, $\text{\LaTeX}$, $\text{\XeTeX}$, $\text{\XeLaTeX}$, $\text{\PreTeXt}$, $\text{\WebWork}$. Conveniences for frequently-mentioned accessories to $\text{\PreTeXt}$.

<fillin/>, _____, fill-in blank. A “fill in the blank” blank. May be used in normal text or within mathematics contexts. The `@characters` attribute may be used to hint at how long the line will be. Here we have set `@characters` to the value 5.

If used inside math, a `@fill` attribute (in lieu of `@characters`) may be some string of math that will be used to determine width, height, and depth of the blank. In text, the default value of 10 is used for `@characters`. In math, the default value of XXX is used for `@fill`.

If used in normal text, a `@rows` and/or `@cols` attribute may be present, each a positive integer. When either is greater than one, there will be an indication that the expected content to fill in the blank is a `@rows × @cols` array.

`<ie/>`, `<eg/>`, `<ca/>`, `<vs/>`, `<etc/>`, i.e., e.g., ca., vs., etc.. A small collection of frequently-used Latin abbreviations, with attempts to handle the tricky periods wisely in $\text{\LaTeX}$ output. Strictly speaking BC is not Latin, but we include it for completeness. Tags are always lowercase, no punctuation, usually two letters.

Tag	Realization	Meaning
ad	AD	<i>anno Domini</i> , in the year of the Lord
am	AM	<i>ante meridiem</i> , before midday
bc	BC	English, before Christ
ca	ca.	<i>circa</i> , about
eg	e.g.	<i>exempli gratia</i> , for example
etal	et al.	<i>et alia</i> , and others
etc	etc.	<i>et caetera</i> , and the rest
ie	i.e.	<i>id est</i> , in other words
nb	NB	<i>nota bene</i> , note well
pm	PM	<i>post meridiem</i> , after midday
ps	PS	<i>post scriptum</i> , after what has been written
vs	vs.	<i>versus</i> , against
viz	viz.	<i>videlicet</i> , namely

SI Units. *Système international (d’unités)* (International System of Units) is a system of measurement used universally in science. $\text{\LaTeX}$ has comprehensive support for this system and its notation and abbreviations. See [Section 3.29](#) for a short introduction and [Section 4.35](#) for detailed descriptions of the relevant elements and their use.

$\times$, $\sharp$, $\natural$, $\flat$, $\sharp\sharp$, **music notation.** Notes, chords, and other notation may appear within sentences as part of a discussion. See [Section 4.34](#) for detailed descriptions of the relevant elements.

Best Practice 4.1.8 Understand the Importance of Careful Markup. There is a lot of detailed information in this section. Much of it is critically important. If you are new to thinking in terms of markup (rather than WYSIWYG tools), it might be overwhelming, a lot to digest, and hard to separate the wheat from the chaff. Careful here means using the necessary markup, not using it for other purposes different than its intent (**tag abuse**), planning ahead for different output formats, but not becoming a slave to over-doing it.

So come back here often for a re-read. And keep in mind that $\text{\LaTeX}$ is designed around principles ([List 1.1.1](#)), and that it is markup ([Item 1.1.1:1](#)) which enables multiple outputs ([Item 1.1.1:3](#)) and effective and beautiful online versions ([Item 1.1.1:6](#)).

4.2 Blocks

4.2.1 Introduction to Blocks

A division ([Section 3.1](#)), that is not further subdivided, is primarily, but not exclusively, composed of paragraphs and **blocks**. We document the types of blocks here, even though we do not intend to ever provide a rigorous definition of the term. Here is a list of characteristics, which is not prescriptive.

List 4.2.1 Characteristics of a Block

- Visually set-off from a run of “plain” paragraphs. Often earning a number, and ideally provided a title or caption ([Best Practice 4.8.1](#)).
- Reflowable lines of text, such as an `<example>`, or a more rigid, more spatial, more **planar** object, such as a `<figure>` or `<table>`.
- Usually a child of a division ([Section 4.6](#)). But see just below.
- Typically a block does not contain another block, except that the more planar ones can appear as part of a more textual one.

- When numbered, all blocks generally run consecutively. (In $\text{\LaTeX}$ -speak, “on the same counter.”) Numbering FIGURE-LIKE and PROJECT-LIKE independently is in transition at this writing (2021-07-07).
- Depending on the output format of your document, some block types may be initially hidden to improve the visual flow. The reader must click on the heading for the block to reveal its contents. You can change this behavior by configuring your publication file. See [Section 44.4](#). We will make note of the elements that are hidden by default for HTML output.

The following is somewhat general, and we have not extensively cross-referenced to the particular types of blocks, so use the Table of Contents or the Index to learn more specifics.

4.2.2 Isomorphic Blocks

The structure of a block is described carefully in the schema ([Chapter 6](#)). There are approximately forty blocks that are arranged into ten groups, within which they behave identically, except for their displayed names. An exception is the group of four “figure-like” items which are very similar, but have differences beyond just their displayed names. These groupings are defined in the `xsl/entities.ent` file, which we summarize in the next table. The category name is taken from the entities file, and the notes are meant to describe the distinctive capabilities of the category.

List 4.2.2 Summary of Blocks

REMARK-LIKE	<code><remark></code> , <code><convention></code> , <code><note></code> , <code><observation></code> , <code><warning></code> , <code><insight></code> The most basic, generic, block.
EXAMPLE-LIKE	<code><example></code> , <code><question></code> , <code><problem></code> A worked problem meant as exposition. It can be structured with <code><task></code> , <code><hint></code> , <code><answer></code> , and <code><solution></code> just like an <code><exercise></code> or PROJECT-LIKE, but the <code><hint></code> , <code><answer></code> , and <code><solution></code> cannot be electively removed from output, and they do not migrate to collections of solutions elsewhere. Hidden by default for HTML output formats.
PROJECT-LIKE	<code><project></code> , <code><activity></code> , <code><exploration></code> , <code><investigation></code> These are similar to an <code><exercise></code> , but the name suggests a slightly different undertaking, and they cannot be placed in an <code><exercises></code> division. The current default is that they are numbered independently from other blocks, but this is planned to switch to elective behavior. The <code><hint></code> , <code><answer></code> , and <code><solution></code> behave more like those for an <code><exercise></code> and can be removed from output, and can migrate to collections of solutions.
FIGURE-LIKE	<code><figure></code> , <code><table></code> , <code><listing></code> , <code><list></code> An object that is a container for other atomic objects, which are typically somewhat rigid (not reflowable) or two-dimensional. Typically with a number, and provided with a title or caption. But each is slightly different in what it can contain and how it is rendered. A <code><table></code> can only hold a <code><tabular></code> , while a <code><listing></code> is meant for <code><program></code> and <code><console></code> . A <code><list></code> is not the list itself, but a container for one of the three possible lists (see Section 4.11 and especially Section 4.21). A <code><figure></code> is the most liberal, allowing a wide variety of contents, with <code><image></code> being the prototypical example. These are not designed with the expectation that they can be renamed.
THEOREM-LIKE	<code><theorem></code> , <code><corollary></code> , <code><lemma></code> , <code><algorithm></code> , <code><proposition></code> , <code><claim></code> , <code><fact></code> , <code><identity></code> Mathematical results, which can have an (optional) <code><proof></code> . Proofs are hidden by default in HTML output.

AXIOM-LIKE	<code><axiom></code> , <code><conjecture></code> , <code><principle></code> , <code><heuristic></code> , <code><hypothesis></code> , <code><assumption></code> A mathematical statement, which does not have a proof.
DEFINITION-LIKE	<code><definition></code> A definition of a (mathematical) object.
ASIDE-LIKE	<code><aside></code> , <code><biographical></code> , <code><historical></code> Parenthetical content that is structured beyond what a footnote can contain.
COMPUTATION-LIKE	<code><computation></code> , <code><technology></code> , <code><data></code> For descriptions of activities or data for use with computers, calculators, or other devices.
GOAL-LIKE	<code><objectives></code> , <code><outcomes></code> These are structured primarily as lists, and may only appear early (<code><objectives></code>) or late (<code><outcomes></code>) within a division.

4.2.3 Other Blocks

There are other blocks which can be achieved using just one element. Examples are `<poem>` and `<assemblage>`. An `<exercise>` can take on different behaviors, depending on its placement (see [Section 4.3](#)). One such placement is as a child of a division, which we call an **inline exercise**, and this would be regarded as a block, very similar to a PROJECT-LIKE. Inline exercises are hidden by default in HTML output.

A paragraph (`<p>`) can appear many places and is a primary component of blocks. But when it is a child of a division, it shares some characteristics with other blocks. There are a number of peers of a `<p>` which would then also qualify: `<pre>`, `<blockquote>`, `<image>`, `<video>`, `<program>`, `<console>`, and `<tabular>`. None of these can have a title or number, making them less useful, but widening the possibilities for placement.

4.2.4 Renaming Blocks

A common desire of authors new to PreTeXt is a new block. Authors extending PreTeXt to add a new one is not supported, and it is not even straightforward for a PreTeXt developer to provide comprehensive support for a new block. One of the reasons for multiple versions of blocks, with common behaviors, is that you can appropriate one for use with your project and give it a new name. So for example, a biology textbook might want to use `<activity>` for a laboratory, but rename it to “Laboratory”. Of course, this means you will forego having any “Activity” in your book. (But if you *are the first* to write a biology textbook in PreTeXt perhaps you should talk to us about a real `<Laboratory>` element that behaves well for all the physical and biological sciences!) Here is the procedure:

1. Choose a block from [List 4.2.2](#) that has behavior similar to your intended use, and which you do not foresee using as-is in your project.
2. In the `<docinfo>` section use a `<rename>` element. The content should be the new name (“Laboratory” above), while the `@element` attribute should be the name of the element renamed (`activity` above).
3. The `@xml:lang` attribute should be used to specify a value of the code for use with documents authored in languages other than US English. If absent, the default value `en-US` is used. Multiple `<rename>` elements for the same element, in different languages, is supported.

So, continuing and extending our example, an author would use

```
<rename @element="activity" xml:lang="fr-FR">Laboratoire</rename>
```

The most popular block to rename is `<exercise>`. We have exercises inside `<exercises>` divisions, which we call **divisional exercises**, and exercises inside divisions, which we call **inline exercises**. It is possible to have one of each that have identical numbers. So in cross-references the former is an “Exercise”, while the latter is a “Checkpoint”. If you only have inline exercises, you might prefer that they be called Exercise rather than Checkpoint. For the `@element` attribute of the `<rename>` element, use a **pseudo-element**, in this case `inlineexercise`.

Other pseudo-elements which can be used to distinguish the various types of exercises are: `divisionexercise`, `worksheetexercise`, and `readingquestion`.

Choose the element you rename carefully, trying to match it to the purpose of your content. It can be useful for other purposes, such as automatic lists ([Section 4.29](#)), and you may decide later to use other properties of the element you have chosen.

4.3 (*) Exercises, Inline and Divisional

4.4 Verbatim and Literal Text

This section expands on parts of [Section 3.16](#). For descriptions of more involved uses, such as program listings and console sessions, see [Section 4.16](#) and [Section 4.15](#).

The tags described here contain *only raw characters*. By that we typically mean the first 128 characters of the ASCII code. Unicode characters are likely to migrate to HTML output just fine, but results for $\TeX$ output will be variable. The restriction to character data has two consequences. First, any markup mistakenly included will have its content silently ignored and dropped. Second, you need to observe the rules on exceptional characters and escaped characters for XML for literal text, which are mercifully simple.

In your source, use `&` for an `&`, use `<` for `<`, and optionally use `>` for `>`. Otherwise, every other ASCII character will render faithfully across all possible formats.

See [Subsubsection 4.1.4.2](#) for more detail and explanation.

4.4.1 Short, Inline, Verbatim Text

The `<c>` tag is a mnemonic for “code”, but is really meant to be any chunk of literal characters that you want to emphasise that way. So a “typewriter” font of fixed-width characters would be a typical presentation. It is meant for use within a sentence or caption (“inline”) so its use is limited to those situations, and others that are similar, such as a title or a cell of a table. Typically these pieces of text do not hyphenate words, and so can lead to spillover into a right margin. In these situations, consider options below for longer pieces of text.

There is also a `<pf>` tag, which is a mnemonic for “program fragment”. It is used in the same way as `<c>`, but will apply syntax highlighting so that the displayed text matches the formatting of a `<program>`. See [Subsubsection 4.16.1.3](#) for more information.

4.4.2 Longer, Inline, Verbatim Text

For longer pieces of verbatim text, use the `<cd>` tag, which is short for “code display”, analogous to the `<md>` for mathematics. It is used within sentences of a paragraph and will be presented with a vertical break above and below, but without interrupting the paragraph. Because of the display presentation, it cannot be used other places, such as a `<title>`, where a vertical gap is not appropriate. All of the previous discussion about exceptional characters applies for this tag.

You have two options in use. You may author inline with the rest of a sentence, with no extra newlines or whitespace before, after, or within the content. The result will be a single displayed line.

Or you may structure the content using one, or more, of the `<cline>` tag, which is meant to be similar to the `<line>` tag used elsewhere. You should still take care to not place any extra whitespace before or after the `<cd>` element, but in between the `<cline>` you may use as much visual formatting of your source as you wish, especially if you like your source to mirror your output.

4.4.3 Blocks of Verbatim Text

If you want to isolate large chunks of verbatim text outside of paragraphs, the `<pre>` tag is the one to use. It can be used as a peer of paragraphs (and other structures) as a child of a division, or it can be placed into a `<listing>` to receive a caption, title and number.

You can structure the contents with `<cline>` in exactly the same manner as for `<cd>`. But you may find this tedious. Instead, you can make the content of `<pre>` a sequence of lines separated by newlines. So that you can preserve the indentation of your source, the line closest to the left margin is taken to actually be the left margin, and a corresponding amount of leading whitespace will be removed from every line. This will work well if you recognize two caveats. First, results will be unpredictable if you mix spaces and tabs for indentation. Sticking with spaces is best. Second, if your first characters of content immediately follow the `<pre>` tag then there is no leading whitespace and it is as if that line is already at the left margin. Then subsequent indentation may seem too severe to you.

As previously mentioned, [Section 4.15](#) and [Section 4.16](#) discuss the `<console>` and `<program>` tags which are more specific, and hence more capable. Review the possibilities before you decide between `<pre>`, `<console>`, and `<program>`.

4.5 Cross-References and Citations

When you read a novel, you would likely read it cover-to-cover (in one sitting?) and then put it away and never read it again. But for a textbook, you may read cover-to-cover, but you may also frequently skip around, especially at exam time. And once read, it might become a reference work for you, since you know it so well. So years later you might come back looking for something. Cross-references help with all this, so use them liberally. Recognize that an index is really just a specialized way to provide an abundance of cross-references.

4.5.1 Creating a Cross-Reference

It is a two-step process to make a cross-reference.

1. Put an `@xml:id` attribute on any element you think you might want to reference later. Be organized about the values of these attributes, and in particular do not number them, as this has no place in your source, and you do not want to maintain the changing numbers over the life of your project. See the advice in [Section 3.4](#) about banned characters. Some elements do not accept this attribute because the element has nothing to identify it (no number, no title). Typically these are **containers** such as `<sidebyside>`, `<statement>`, or `<ol>`. In these cases, put the attribute on the closest enclosing element.
2. To make a cross reference, you create an `<xref/>` element with a `@ref` attribute with the same value as `@xml:id` attribute on the element you want to reference.

Simple. It is meant to be, so you can use it liberally. But we also know authors want some flexibility.

Best Practice 4.5.1 Use `@xml:id` Frequently. Use the `@xml:id` attribute liberally, on any object you might want to reference later, and on every division. It is easier to do as you author and will be very valuable later. (Trust us on this one.) Develop a system so you can recall them predictably, but keep them readable. Don't use numbers, they will change. Then make frequent cross-references. They are relatively easy for you and will be incredibly useful for each and every one of your many readers, over and over and over again.

4.5.2 Text of a Cross-Reference

By default, a cross-reference will visually be text like Theorem 5.2. Depending on your output format, it may have various devices to help you locate that theorem. Maybe a page number, or it is a hyperlink, or the whole theorem is the content of a knowl. You can change the default look of cross-references by setting the `@text` attribute in `docinfo/cross-references`. But you can also change the visual appearance of a cross-reference on a case-by-case basis. Add a `@text` attribute to your `<xref/>` element to override the document-wide setting. The first column of this table lists the possible attribute values, either document-wide, or on a per-cross-reference basis. The second column has live cross-references to a section of an earlier chapter (that is far away). The third column has live cross-references to another section of this chapter (which is close by). For the fourth column, we have placed content ("Extra") into the `<xref>` element as an override of, or addition to, some of the text for the cross-references of the preceding column. Study the table and then read some more explanation following. Note that `type-global` is the default.

Table 4.5.2 Cross-reference visual text styles

@text	Far Away	Close By	With Content
type-local	Section 5	Section 6	Extra 6
type-global	Section 3.5	Section 4.6	Extra 4.6
type-hybrid	Section 3.5	Section 6	Extra 6
local	5	6	Extra 6
global	3.5	4.6	Extra 4.6
hybrid	3.5	6	Extra 6
type-local-title	Section 5 Titles	Section 6 Divisions	Extra 6 Divisions
type-global-title	Section 3.5 Titles	Section 4.6 Divisions	Extra 4.6 Divisions
phrase-global	Section 5 of Chapter 3	Section 6 of Chapter 4	Warning
phrase-hybrid	Section 5 of Chapter 3	Section 6	Warning
title	Titles	Divisions	Warning
custom			Extra

Note that local/global describes the uniqueness of the number (and is affected by your choice of numbering schemes), while type refers to an automatic prefix of the number. The text of the type will vary according to the document's language. If a cross-reference and its target are close to each other, a number like 5.8.2.4 might be overkill, when just a 4 would suffice. A hybrid scheme will use the shorter number whenever it makes sense. There are two phrase schemes, and it should be clear what text title will produce (though realize there must be a title for the object, possibly a default provided by PreTeXt). Finally, if desired, the text can be customized with any text you like.

You can also override the text used in a cross-reference link. You do this by providing content to the `<xref>` element. In other words, do not use an empty tag, but put some (simple) text in the element. Generally, this additional text becomes a prefix of a number or a replacement of a type. It is better to use these overrides, since in electronic formats, the text of the override will be incorporated into the “clickable” portion of the resulting link, making a larger item to hit. Recognize that this extra content will not benefit from automatic internationalization.

Here are careful examples of providing content inside the `<xref>` element, where we have provided the content “Division” in the live examples. The list is not exhaustive.

Table 4.5.3 Cross-reference text overrides

@text	Example
'global'	Division 4.5
'type-global'	Division 4.5
'custom'	Division

4.5.3 Variations

There are some variant uses for the `<xref>` tag.

- Replace `@ref` by `@provisional` and give it a value with some simple text like subsection on eagle habitat and you will get reminders that once you write this future subsection you need to link it in right here. This is just a convenience for authors during the early stages of a writing project (see [Section 5.9](#) for details).
- Replace `@ref` by the pair `@first` and `@last`. Provide attribute values just as you would for `@ref`. The code will check that the targets have the exact same tag, and that the order is correct. You will get a link that looks like a range, separated by an en dash. As a link, only `@first` will be used for the linking mechanism (i.e., one link is generated, not two). Experiment with `@text` and content overrides.
- The `@ref` attribute may be a list of `@xml:id`, separated by commas or by spaces. Then you will get back a list of cross-references. This is meant for a list of citations, producing a look like [5, 9, 22], but it makes no restriction to this case. Use it generally, but it is unlikely to get any more capable. If you want a different list, just use multiple `<xref>` and format as you desire.

You can create many different combinations with the text options and the variants. Here is one example. You want to say [Chapters 1–6](#). As a range you use the variant with two references. You would get “Chapter” out front

automatically with the `type-global` scheme, but a plural form makes more sense. So you use that text as an override. We could use either `type-global` or `global` to get the same text, and possibly `type-hybrid` depending on the place where you built the cross-reference. So possibly, one of these schemes might be your document-wide setting and you do not need to specify it now. Here is what we just used:

```
<xref first="start-here" last="schema" text="global">Chapters</xref>
```

You can place a cross-reference into a `<title>` element, but a particular conversion is free to simply render it as text, and not as an active link.

Finally, there is fairly robust error-checking to protect against typographical errors in the attribute values that need to match up for all this to work. Also, there is a check that the `@xml:id` are unique. But all this checking happens at processing-time, not at the validation step. Any suggestions for improvements that make these checks even more robust are welcome.

4.5.4 Citations

Citations are just specialized cross-references to `<biblio>` elements, and so behave the same way as other cross-references. However they will always visually look like [23], and there is no notion of “type name.”

4.5.5 Equations

Similar to citations, references to equations (`<md>` and `<mrow>` elements) will visually look like (4.2), where the type name is implied by the parentheses. Notice that you cannot cross-reference a bare `<md>` element that is not numbered nor an `<md>` element structured with `<mrow>` (it is just a container, filled with `<mrow>` that you can target if you give them numbers). Consult [Subsection 4.9.3](#) for details about controlling the numbering of equations within an `<md>`.

4.5.6 Lists

Roughly, you can target a list item for a cross-reference, but not the list itself, since it is a container. See [Subsection 4.11.4](#) for precise details about using list content as the target of a cross-reference. Note also, that an entire named list may be the target of a cross-reference, see [Section 4.21](#). Here we concentrate on the text of the cross-reference itself.

First, note that if you cross-reference a list item of an anonymous list, there is a very real possibility that the number will be ambiguous, and there is no option for `@text` will save you, and never can be. See the middle column of [Table 4.5.5](#) for the demonstration. We assiduously try to make it *impossible* to ever create ambiguous text for cross-references, especially in consideration of print output. Use the feature carefully.

Best Practice 4.5.4 Take Care Referencing Anonymous Lists. Cross-references to list items of anonymous lists can easily be ambiguous and then useless for readers of print. Keep such a cross-reference close to its target, ideally within the same list, and/or perhaps using additional, unambiguous clues about location (which you expect will survive later editing):

1. See [Item 2](#) of this list.
2. See [Item 1](#) in the list appearing in [Best Practice 4.5.4](#).

The `local` option, discussed generally above, behaves differently for a cross-reference to a list item of an ordered list that is contained in a named list. As seen in the table just below, the local portion of the number is the part that comes from the list item, without the part that comes from the location of the `<list>` block.

Table 4.5.5 Cross-references to list items, visual text styles

@text	Named List	Anonymous List	With Content
type-local	Item B.c	Item 2.III	Extra B.c
type-global	Item 4.11.4:B.c	Item 2.III	Extra 4.11.4:B.c
type-hybrid	Item 4.11.4:B.c	Item 2.III	Extra 4.11.4:B.c
local	B.c	2.III	Extra B.c
global	4.11.4:B.c	2.III	Extra 4.11.4:B.c
hybrid	4.11.4:B.c	2.III	Extra 4.11.4:B.c
phrase-global	Item B.c of List 4.11.4	Warning	Warning
phrase-hybrid	Item B.c of List 4.11.4	Warning	Warning
title	A Test Title	Exquisite Fish	Warning
custom			Extra

The hybrid options employ a different definition of when the distance between a cross-reference and its target is close enough that the number can be shortened, without becoming ambiguous. When an `<xref>` and its target are part of the same `<list>`, then the common part of the number derived from the `<list>` is not needed. To illustrate we need to make a small `<list>` with cross-references contained within.

List 4.5.6 Small test

1. (type-global) Flowers are not [Item 4.5.6:6](#).
2. (type-hybrid) Fish are not [Item 6](#).
3. (hybrid) Bacteria are not [6](#).
4. (phrase-global) Fungi are not [Item 6](#).
5. (phrase-hybrid) Trees are not [Item 6](#).
6. Mammals.

Best Practice 4.5.7 Be Rational About Numbering Variations. With distinct numbering schemes for divisions, theorems, figures, equations, and citations, along with ten different text styles for a cross-reference, plus variants, per-cross-reference settings, and text overrides, there is a huge supply of combinations. Likely you can create some really ugly cross-references. Use the flexibility sensibly.

4.6 Divisions

A **division** (or more carefully, a **structural division**) is a structured component of a book or article that would be recognized by most any reader. They are essential to the organization of a PreTeXt project. Notice that we use the generic term division, since a `<section>` is just one example of a division.

Divisions are `<book>`, `<article>`, `<part>`, `<chapter>`, `<section>`, `<subsection>`, `<subsubsection>`, and `<paragraphs>`. Their use is fairly intuitive, though there are some restrictions, so please read on.

A `<book>` must contain at least one `<part>` or at least one `<chapter>`, which may contain `<section>`, `<subsection>`, and `<subsubsection>`. A `<part>` simply contains a sequence of `<chapter>` and functions in two user-selectable ways: structural (e.g. numbering will reset), or decorative (merely inserting a decorative page between two chapters and sectioning the Table of Contents).

An `<article>` is simpler and shorter than a book. It might be really simple and have no divisions at all, or it may have `<section>`s. It cannot have `<chapter>`s, as that would be a `<book>`. Within a `<section>`, `<subsection>`s and `<subsubsection>`s may follow.

Divisions must nest properly and may not be skipped. So a `<section>` cannot contain a `<chapter>` and a `<subsection>` may not be contained in a `<chapter>` without an intervening `<section>`.

A division *must* contain a `<title>`, and may contain one or more index entries (see [Section 4.27](#)), which should appear before anything else. Any division may be unstructured, with just a sequence of **top-level content** such as paragraphs, figures, lists, theorems, etc. Or a division may be structured, and in this case it must follow a prescribed pattern. There may be a single, optional `<introduction>`, filled with top-level content, followed by a sequence of at least one of the appropriate divisions, ending with a single, optional `<conclusion>`, filled with top-level content. It is an error to begin with a run of top-level content inside a division and then begin to use divisions. (The solution is to make the initial content an `<introduction>` and/or one or several divisions.)

A `<book>` may be structured into parts. After the `<frontmatter>` and before the `<backmatter>` there may be a sequence of `<part>`. These elements may carry a `<title>`, and not much else, besides a substantial sequence of `<chapter>`. The main effect is to get an extra level of division in the Table of Contents. For print and PDF there is an entire page devoted to the title and number of the part. The default numbering is **decorative**, which means that the chapters are numbered consecutively from the start of the book and do not reset to one at the start of each part. It is as if the parts were not even there. The alternative is that parts are **structural**. Now each part begins with Chapter 1. There are other more subtle differences, such as cross-references use a part number if, and only if, the trip from the cross-reference to its target crosses a boundary of two parts. The two approaches to part structure may be set via the publication file, see [Section 44.2](#).

There are exceptions to the above. For one, `<paragraphs>` is an anomalous division, as a sort of lightweight sectioning command. It may appear in any division, at any location within a division, it may not be divided further (it is a leaf of the document tree), it never gets a number, and its title is formatted in a subsidiary way. I especially like to use this in a two- or three-page `<article>` that has no other divisions at all. It is also very useful as a way to subdivide portions of the front matter ([Section 4.25](#)), such as a `<preface>`. Typical presentation has the title in bold, without much change in font size (if at all), inline with the first paragraph, and perhaps a bit of vertical space as it begins and ends. Despite the name, it may contain more than just paragraphs, so may contain any top-level-content that would go in any other division.

4.7 Specialized Divisions

There are six divisions that have specialized functions, and therefore have less generic names than ones like `<chapter>` or `<section>`. They are `<exercises>`, `<reading-questions>`, `<solutions>`, `<references>`, `<glossary>`, and `<worksheet>`. They have some features in common, such as having a `<title>`, but each is different from the other in substantial ways.

Generally, a specialized division may be placed within any other division ([Section 4.6](#)), and it will behave like a subdivision of that division. Some may be placed in the back matter and may behave as a version relevant to the entire document. This section describes the specifics for each type of specialized division.

4.7.1 (*) References (Lists of Works Cited)

4.7.2 Glossary

A glossary is a list, in alphabetical order, of foreign or unfamiliar words, along with definitions. [\[1, 1.87\]](#)

A `<glossary>` division may be placed inside any main matter division, and at most once in the `<backmatter>`. In either event, the *Chicago Manual of Style* [\[1, 1.87\]](#) indicates that it should be placed right before a `<references>`.

After a `<title>`, index entries, and other metadata, a `<glossary>` division may begin with an optional `<headnote>`, which can use paragraphs to explain anything unusual about the construction of a particular glossary.

The remainder of a glossary is a sequence of items to explain. Typically these are words, phrases, initialisms, or acronyms. Each item is a “glossary item”, enclosed in a shorthand `<gi>` element. The element must lead with a `<title>`, which is the term being explained. PreTeXt will provide a period after each defined word, so there should not be any punctuation in your source at this location. The term should not have any markup, *unless* the markup is used in every occurrence of the term in the text. Similarly, a term is capitalized only if it is capitalized routinely in the text.

The explanation itself follows, typically in a sequence of paragraphs, but unnumbered items, such as an `<image>` may also be used. It is the author’s responsibility to create the list in alphabetical order. Automatic groupss (according

to initial letter) are a pending feature request, perhaps especially for a final, overall, back matter glossary, much like an index. See [GitHub #1971](#).

Many of the preceding recommendations can be found in *Chicago Manual of Style* [1, 2.28]. For an example, see the [glossary](#) in the back matter of this Guide.

4.7.3 (*) Worksheets

4.8 Titles

Divisions always need titles, you accomplish this with a `<title>` tag first thing. Almost everything that you can use in a paragraph can be used in a title, but a few constructions are banned, such as a displayed mathematical equation (for good reason). Try to avoid using footnotes in titles, even if we have tried to make them possible.

A division will also support a single optional `<shorttitle>` and/or a single optional `<plaintitle>`. The full `<title>` will be used where the division is born, but in other places where the title is used for navigation, such as a Table of Contents, print page header, or HTML summary page, when horizontal space may be at a premium, the `<shorttitle>` will be used preferentially.

A `<plaintitle>` is similar, but slightly different, so you might want both. In limited situations, PreTeXt markup does not travel well in certain conversions. The best example is mathematics, which might be in a title of a division, and then in a conversion to HTML, will fail to render in the tab of a browser, or the list of open tabs for a browser window. Two examples

```
<title><m>q</m>-crystalline cohomology</title>
<title><m>\delta</m>-rings</title>
```

PreTeXt will automatically do a nice job for you with the first, but the second will retain `\delta` in a browser tab. However, you can add a `<plaintitle>` element where the δ can be replaced by a Unicode delta (U+03B4), which will be used preferentially and be fine in HTML output.

To avoid confusing your readers, use these alternate titles sparingly, and ideally only when you have a really, really, really long title and then use a short title that is easily recognizable as a variant of the long title, or when markup is behaving poorly in situations such as a browser tab. Your first option should be to ask if your long title must absolutely be so long, or if the markup is strictly necessary.

Many, many other structures admit titles. Experiment, or look at specific descriptions of the structure you are interested in. Titles are very integral to PreTeXt, much like cross-references. Titles migrate to the Table of Contents, get used in page headers for print output, can be used in lists (such as a List of Figures), and can be used as the text of a cross-reference, instead of a number. You might not be inclined to give a `<remark>` a title, but it would be good practice to do so. If you use `knowls` in your HTML output for structures such as `<example>` (or if somebody else may someday choose to), then your readers will be spared a lot of confusion if you supply informative titles for each. Your electronic outputs will be much more useful to your readers if you routinely title every structure that allows it (perhaps excepting `<exercise>` which can be known by their number).

If a title is very long, the `<line>` element can be used to indicate how the title should appear on multiple lines. Note: does not apply to all output formats.

Best Practice 4.8.1 Provide Informative Titles Liberally. Provisions for titles in many situations is a key PreTeXt feature. And then they are used for various purposes to benefit readers. A good example is when the HTML conversion is used to place content in a `knowl`, a unit of content that begins hidden, but can be revealed (and hidden again) with one click of a mouse. Since a reader cannot see the content originally, we will migrate a title into the clickable text. But we cannot read your mind—it is your job as the author to provide a title, and to provide a good one.

Even if you are not yet sure what a `knowl` is, and even if you think you do not want to use them, there are other good reasons to have a title (such as automatic lists, see [Section 4.29](#)). Constructing them on-the-fly is much easier than making a big chore out of going back and doing it later.

Bad Example 5.10

Better Example 5.10 A cool lizard trick.

Best Example 5.10 Various colors and markings of a chameleon.

4.9 Mathematics

As mentioned in the overview, [Section 3.6](#), we use $\text{\LaTeX}$ syntax for mathematics. In order to allow for quality display in HTML, and other electronic formats, this limits us to the subset of $\text{\LaTeX}$ supported by the very capable [MathJax](#) Javascript library. Generally this looks like the `amsmath` package maintained by the American Mathematical Society at their [AMS-LaTeX page](#). For a complete and precise list of what MathJax supports, see MathJax's [Supported \$\text{\LaTeX}\$ commands](#). Once you have digested this more general section, be sure to also consult [Section 4.10](#) for some very specific suggestions.

4.9.1 Inline Mathematics

Use the `<m>` to place variables or very short expressions within a sentence of a paragraph, the content of a `<title>`, a `<cell>` of a table, a footnote, or other similar locations of sentence-like text. You cannot cross-reference this text, nor make a `knowl` with it. Though you can typically cross-reference a containing element.

Do not use $\text{\LaTeX}$ -isms like `\displaystyle` to try to end-run the inline nature. It will just lead to poor results.

Best Practice 4.9.1 Keep Inline Mathematics Short. Longer mathematical expressions in an `<m>` element can lead to awkward line breaks, both in HTML output, and especially in PDF generated from $\text{\LaTeX}$. And complicated fractions or integrals can introduce abnormal line-spacing that is distracting to a reader. As a rough rule-of-thumb, keep an inline expression shorter than a moderately-long regular word and avoid tall constructions. This should allow $\text{\LaTeX}$'s line-breaking algorithms the best chance of success.

So a simple, short equality such as $x = 2$ should not cause a problem, but if you want to claim that the probability distribution of the normal distribution has the right scaling factors, $\int_{-\infty}^{\infty} \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\frac{(x-\mu)^2}{\sigma^2}} dx = 1$, there is a good chance it will do less harm to your paragraph if you display it

$$\int_{-\infty}^{\infty} \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\frac{(x-\mu)^2}{\sigma^2}} dx = 1$$

using the `<md>` element described next.

4.9.2 One-Line Display Mathematics

The `<md>` element can be used for longer expressions or a single equation. Typically you will get vertical separation above and below, and the contents will be centered. See below about concluding periods (and other punctuation), and alignment. To number the expression, use the `@number` attribute with the value set to `yes`. And once numbered, the expression can be the target of a cross-reference (`<xref>`), if you also add a `@xml:id` attribute.

A symbolic local tag may instead be requested with the `@tag` attribute, using the same set of symbol names as on an `<mrow>`; see [Subsection 4.9.3](#). Since a tag stands in for a number, the `@tag` and `@number` attributes are mutually exclusive on a one-line `<md>`.

4.9.3 Multi-line Display Mathematics

We begin with the `<md>` element, but now used as a container. Within the container, content, on a per-line basis, goes into a `<mrow>` element. You can think of `<mrow>` as being very similar to a one-line `<md>` that does not have additional structure. In a $\text{\LaTeX}$ `align` environment, the `\` marks the end of a displayed line. In PreTeXt each `<mrow>` delimits a displayed line, and there are no `\`s. Use `\amp` to mark the alignment point.

On any given `<mrow>` you can place the `@number` attribute, with allowable values of `yes` and `no`. These will typically be used to override the behavior you might place on the `<md>` container, via an overall `@number` attribute, but there is no harm if they are redundant. A given line of the display may be the target of a cross-reference (`<xref>`) via an `@xml:id` attribute, though the numbering flexibility means you can try (and fail) to target an unnumbered equation.

An `<mrow>` may have a `@tag` attribute in place of a `@number` attribute. This will create a “number” on the equation which is just a symbol. This is meant for situations where you do not want to use numbers, and the resulting cross-

reference is “local.” In other words, the `<xref>` and its target are not far apart, such as maybe within the same `<example>` or the same `<proof>`. Allowable values for the attribute are:

```
star, dstar, tstar,
dagger, ddagger, tdagger,
daggerdbl, ddaggerdbl, tdaggerdbl,
hash, dhash, thash,
maltese, dmaltese, tmaltese
```

These are the names of symbols, with prefixes where the prefix `d` means “double”, and the prefix `t` means “triple”. Cross-references to these tagged equations happens in the usual way and should behave as expected. See [Section 3.4](#) and [Section 4.5](#) for more on cross-references.

Finally, the default is to not number displayed expressions. If you want every displayed expression numbered, or you find yourself electing to number the majority of the displayed expressions, then there is a global default. In your `<docinfo>` element, put a `@equations` attribute on the `<numbering>` element, and set its value to `yes`, rather than the default `no`.

4.9.4 Exceptional Characters

The $\TeX$ macros, `\amp`, `\lt`, and `\gt` are always available within these mathematics elements, so that you can avoid the exceptional XML characters `&`, `<` and `>`. See [Section 3.14](#) for this same information, but in the broader context of your entire document.

4.9.5 Text in Mathematics

Once in a while, you need a little bit of “regular” text within an expression and you do not want it to look like a product of a bunch of one-letter variables. Use the `\text{}` macro for these. Only. Other ways of switching out of math-mode and into some sort of “regular” text will appear inferior, and can raise errors in certain conversions.

- Do place surrounding spaces inside the `\text{}` macro.
- Do not place any mathematics inside the `\text{}` macro.
- Do not use the `\mbox{}` macro as a substitute.
- Do not use font-changing commands (e.g. `\rm`) as a substitute.

For example,

```
<me>f(x) = \begin{cases} x^2 \amp \text{if } x \gt 0 \\ -7 \amp \text{otherwise} \end{cases}</me>
```

produces

$$f(x) = \begin{cases} x^2 & \text{if } x > 0 \\ -7 & \text{otherwise.} \end{cases}$$

This example amply illustrates the use of macros for XML exceptional characters (twice), appropriate use of the `\text{}` macro (twice), spaces in the `\text{}` macro (once), sentence-ending punctuation (see the source, the period is *not* inside the `<md>` element) and yes, we did think twice about the `\` (an exception to the rule).

4.9.6 Color in Mathematics

There is a temptation to use color to indicate or highlight portions of mathematics, especially for electronic outputs where color is easy and cheap. But before you leap, how will this work in black-and-white printed output? How will it work for a blind reader using a screen-reader or a braille version?

With careful use of $\TeX$ grouping (`{ . . . }`) you can make the two behaviors of `\color` effective. For example, go:

```
{\color{blue}{x^2}}
```


4.9.7 Cross-References in Display Mathematics

A cross-reference is achieved with the `<xref>` element, see [Section 3.4](#). You can place an `<xref>` inside a `<mrow>`, and remarkably, it will do the right thing. This is one of only two XML elements you can mix-in with $\LaTeX$ syntax. A typical use is to provide a justification or explanation for a step in a proof, derivation, or simplification. And it works best with alignment, see below.

4.9.8 Alignment in Display Mathematics

Displayed mathematics is implemented with the AMS- $\LaTeX$ `align` environment. Ampersands are used to control this, so use the `\amp` macro for these. The first ampersand in a line or row is an alignment point, typically a symbol, like an equality. The next ampersand is a column separator, then the next is an alignment point, then a column separator, then... The moral of the story is you should have n alignment points, with $n - 1$ column separators, for a total of $2n - 1$ ampersands—always an odd number.

For example,

```
<md>
  <mrow>A \amp = B \amp D \amp = E \amp \amp \text{Because}</mrow>
  <mrow> \amp = C \amp \amp = F \amp \amp <xref ref="txo" /></mrow>
</md>
```

produces

$$\begin{array}{ccc} A = B & D = E & \text{Because} \\ = C & = F & \text{Table 4.5.3.} \end{array}$$

Sometimes you want several short equations on one line. Do not use the bare `<md>`. Instead use a single `<mrow>` inside an `<md>`, and use alignment to spread them out evenly.

For multi-line display mathematics with no ampersands present, each line will be centered. This is implemented with the AMS- $\LaTeX$ `gather` environment.

You can fool the alignment behavior by hiding all your ampersands in macro definitions, so there is the optional `@alignment` attribute for the `<md>` element, in order to force the right kind of alignment. Allowable values are `gather`, `align`, and `alignat`. The latter is similar to `align`, but no space is automatically provided between columns. You can leave it that way, or explicitly add your own. For example, this allows you to precisely arrange individual terms of a system of linear equations, especially when terms with zero coefficients are omitted. When using the `alignat` option $\text{\LaTeX}$ tries to count ampersands to see how many columns you intend, since $\text{\LaTeX}$ needs this number (we are not sure why). This detection can be fooled too, especially if you have something like a matrix with lots of ampersands for other purposes. So set the `@alignat-columns` attribute to the *number of intended columns*, if necessary.

If you have many aligned equations in a single `<md>`, but want to add a short sentence or two somewhere in the middle, place it in an `<intertext>` element, and keep it simple. Just some words and punctuation, plus maybe some elements like `<em>` or `<q>`, but nothing like a list. In PDF output constructed via $\text{\LaTeX}$ the alignment will be preserved across the break for the `intertext` (since this feature is inspired by $\text{\LaTeX}$'s `\intertext{}` macro). Unfortunately, this is too complicated in other (electronic) output formats, so we just do the best we can to preserve your intent. Perhaps some day, MathJax will be more capable, and we can make your source do better.

4.9.9 Commutative Diagrams

Commutative diagrams may be authored using the AMS $\text{\LaTeX}$ `amscd` package. While restricted in some ways, such as the lack of sloped or curved arrows, it has one important advantage over more general drawing tools. Support for HTML output comes from MathJax, and hence has accessible versions included in the output.

Typical use would be within an `<md>` element, so starting with `\begin{CD}`. Despite this being multi-line output, we have not chosen to integrate it within the more general `md/mrow` structure, but that decision can be revisited.

4.9.10 Fill-In Blanks in Mathematics

The other mix-in XML element is `<fillin>`. It may use a `@fill` attribute with some math string to determine width, height, and depth of the blank. Or it may use a `@characters` attribute that takes an integer value to hint at the width.

4.9.11 Page Breaks for Tall Display Mathematics

For print output, do nothing additional and $\text{\LaTeX}$ will do its best to break your display between lines. You can turn this behavior off by setting the `@break` attribute on the `<md>` to the value `no`. Once you do this, you can then selectively allow a page break after a given `<mrow>` by setting the `@break` attribute on the `<mrow>` to the value `yes`.

4.9.12 Your Macros

These go in the `<docinfo>` section, wrapped in a `<macros>` element. Keep them simple—one or two arguments, and one-line definitions. This is not the place to be fancy, and not the place to try to end-run the structural aspects of $\text{\LaTeX}$. The idea is to define something like `\adjoint{A}` for the matrix A to be a superscript asterisk, and later you can change your mind and use a superscript dagger instead. Keep in the spirit of $\text{\LaTeX}$ and use readable, semantic macros. For example, do not use `\a{A}` for the adjoint of A . Repeating: keep your $\text{\LaTeX}$ macros simple, and to a single line.

$\text{\LaTeX}$ will use your macros correctly for print and for HTML, after erasing whitespace from the left margin, and stripping $\text{\LaTeX}$ comments.

The name of your macros also cannot contain any numbers, otherwise MathJax will “silently” fail and may not read any subsequent macros you might have. This is important because $\text{\LaTeX}$ will place custom macros for you at the end of your own, defined at [Subsection 4.9.4](#), to be used. Those would fail to be processed by MathJax if your own macros caused it to stop reading.

4.9.13 Semantic Macros

We have resisted using overly-verbose MathML for mathematics, or worse, inventing our own XML vocabulary for mathematics. $\text{\LaTeX}$ syntax generally works great, but to work even better within $\text{\LaTeX}$ an author needs to take a few extra steps. Your work will translate better to a variety of formats, and will be easier to maintain, with well-designed macros. A well-designed macro will convey the mathematical *meaning* of the object to a reader of your source, without them looking at your *definition* of the macro. In situations where a mathematical object might be written with different notation, it should be trivial to change the macro’s definition and preserve the mathematical meaning. For example, consider two versions of a binomial coefficient:

$$\binom{n}{r} \qquad C(n, r)$$

which could both equally well be the realization of `\binomial{n}{r}`.

4.9.13.1 A Cautionary Tale

For a memorable illustration of what can go wrong without meaningful macro names, consider the $\text{\LaTeX}$ macro `\be`. David Farmer collected, from papers in the arXiv preprint server, sixty-six different definitions of `\be`. Across that collection, `\be` stands for at least sixteen entirely distinct mathematical objects (and one for which it is a stretch to call the object mathematical at all). Without the definition close at hand, a reader of the source has no way to know which is meant. We reproduce his list below, grouped into sixteen equivalence classes, with each class given a short descriptive phrase that the macro name `\be` conspicuously fails to convey.

List 4.9.2 Sixty-six definitions of `\be`

1. the equation environment, or display math:
 - `\begin{equation}`

- `\begin{equation}`
 - `\begin{equation*}`
 - `\begin{equation}\begin{array}{l}`
 - `\setcounter{equation}{\value{theorem}} \begin{equation}`
 - `\begin{displaymath}`
 - `\begin{equs}`
 - `\protect\l`
2. the `eqnarray` environment:
- `\begin{eqnarray}`
 - `\begin{eqnarray*}`
 - `\begin{eqnarray}\label{#1}`
 - `\begin{eqnarray} \label{#1}`
 - `\begin{eqnarray#1}`
3. the `gather` environment:
- `\begin{gather}`
4. the `enumerate` environment:
- `\begin{enumerate}`
5. an example-like environment:
- `\begin{exmple}`
 - `\begin{example}\rm\label{#1}`
 - `\begin{Example}`
6. the Greek letter beta, possibly decorated:
- `\beta`
 - `{\beta}`
 - `\ensuremath{\beta}`
 - `\overrightarrow{\beta}`
 - `\boldsymbol{\beta}`
 - `$\beta\text{ }$`
 - `\beta_{\text{ep}}`
 - `{\bar{\beta}}`
 - `\smash{\mkern2mu\overline{\mkern-2mu\phantom{\eta}}\mkern-9.583mu}\eta`
7. a bold lowercase letter:
- `\mathbf{e}`
 - `{\mathbf{e}}`
 - `{\mathbf{e}}`
 - `\boldsymbol{b}`
 - `\boldsymbol{e}`

- `\boldsymbol{e}`
- `{\boldsymbol{e}}`
- `\mbox{\boldmath $e$}`
- `\bm{e}`
- `{\bf e}`
- `{\bf{e}}`
- `{\bf \textbf{e}}`
- `\mbox{\bf e}`
- `{\bold {\hat{e}}}`

8. a bold or blackboard-bold uppercase E:

- `\mathbb E`
- `\mathbb{E}`
- `\mathbf E`
- `\mbox{$\bf E$}`
- `\breve{E}`
- `\boldsymbol{\mathcal{E}}`

9. a bold or barred digit:

- `\mathbf{1}`
- `\bar{5}`

10. a barred, fraktur, sans-serif, vector, or matrix lowercase letter:

- `\bar e`
- `\bar{e}`
- `\bar{\e}`
- `\bar{\varepsilon}`
- `\mathfrak b`
- `\textfrak{B}`
- `\mathfrak{s}`
- `\mathsf{o}`
- `\vec{e}`
- `\mtx{e}`

11. horizontal spacing:

- `\mbox{ }\hskip10pt`

12. a partial derivative:

- `\partial B_\lambda`

13. a specific functor:

- `\Bord_n(H_n)`

14. the Greek letters alpha or gamma:

- `\alpha`
 - `\gamma`
15. the infinity symbol:
- `\infty`
16. the phrase “broken ergodicity”:
- `broken ergodicity`

Sixteen meanings, one name—and that is just for one macro on one preprint server. Now imagine reading source written by a colleague who has used `\be` consistently, but never told you which of the sixteen is meant. The remedy is to use macro names that say what they mean: `\equation`, `\beta`, `\expectation`, and so on. The remaining subsections of this subsection give specific advice for several common situations where a single piece of notation carries multiple mathematical meanings.

4.9.13.2 Vertical Bars

Vertical bars are used for a variety of mathematical objects. Paired to create functions of expressions: absolute value, $|x - 1|$; norm of a vector, $\|\mathbf{v}\|$; cardinality of a set, $|X|$; and the determinant of a matrix, $|A^k|$. As relations: division, $a \mid b$; parallel lines $L_1 \parallel L_2$. Sets are another use: $E = \{x \in \mathbb{Z} \mid x \equiv 0 \pmod{2}\}$.

$\text{\LaTeX}$ `\vert`, `\Vert`, `\lvert`, `\rvert`, `\lVert`, `\rVert` are the delimiters, where `l` and `r` refer to left and right, and the capitalized versions are a pair of vertical lines. The qualifiers `\left` and `\right` can be used to have the length of the bar grow to match what it encloses. Note that there is a `\middle` that we have used above with `\vert` for the set E , and we have added space on either side. `\mid` and `\parallel` are relations, used above to indicate divisibility and parallel lines, and so automatically get an extra bit of spacing on either side.

When using `\left` or `\right` in isolation, `\left.` or `\right.` can be used to define a group that only has a bar on one end. For example:

$$\int_0^1 \sin^{-1}(x) dx = \frac{1}{\sqrt{1-x^2}} \Big|_{x=0}^{x=1}.$$

4.9.13.3 Times

The “times” symbol sees many uses that are different: dimensions, multiplication, and more complicated products (such as a Cartesian product of two sets). Macros `\product`, `\times`, and `\times` could carry different meanings, even if each one is defined by the `\times` symbol, $\times$. For example:

Chess is played on an `<m>8 \by 8</m>` grid, which contains `<m>8 \times 8 = 64</m>` little squares.

If `<m>G</m>` and `<m>H</m>` are finite groups, then `<m>\card(G \product H) = \card(G) \times \card(H)</m>`.

4.9.14 Punctuation After Display Math

If a chunk of displayed math concludes a sentence, then the sentence-ending punctuation should appear at the conclusion of the display. (And certainly not at the start of the first line after the display!) But do not author the punctuation within the mathematics element, put it afterwards, where it logically belongs.

More specifically, place a sentence-ending period (say) *immediately* after the closing of an `<md>` element. $\text{\LaTeX}$ will place the period in your output in the right place and in the right way. (By using $\text{\LaTeX}$ ’s `\text{}` macro, if you are curious to know the details.) Here is an example. The XML source

```
<md>
  <mrow>(a+b)^2</mrow>
</md>. Now...
```

will render as

$$(a + b)^2.$$

Now...

This all applies more generally to clause-ending punctuation, such as a comma. Take notice of the requirement that the punctuation must be *immediately* after the closing tag of the math element, otherwise it will not migrate properly. For example, do not interrupt the flow with whitespace, or an XML comment, or anything else.

For inline mathematics (the `<m>` element) the same authoring principle holds, though you would likely do this naturally. Author the punctuation *outside* the element, where it will remain.

Here is a technical subtlety that will demonstrate some of the inner machinery of PreTeXt and our conversions. In your work, locate a theorem that has some numbered display mathematics (`<md>`) which is at the end of a sentence, and which you have authored as described above. In HTML output, test a cross-reference (`xref`) to the theorem and you will see the period for the end of the sentence at the end of the display, where it should be. Now test a cross-reference (`xref`) to one of the numbered equations. First, the knowl will contain the entire display, to provide context, but it also will not contain the period, since the rest of the sentence is not in the knowl and so the period is not necessary.

Best Practice 4.9.3 Authoring Punctuation after Mathematics. *Always* follow the instructions in [Subsection 4.9.14](#) about placing all punctuation following mathematics *after* the math element, not inside it. PreTeXt will do the right thing for display math for you. But furthermore, there are some special situations where the output format is not visual, such as braille or audio, where the placement of the punctuation is both different and important to not confuse “reader.” You can help ensure your various outputs are of the highest quality by observing these sorts of details.

4.9.15 Lists of Mathematical Expressions

It is common to make lists of expressions, equations, or identities. Think of the definitions of trigonometric functions, a collection of antiderivatives, or a compendium of generating functions.

In these situations, author a list item, `<li>`, within an `<ol>` or `<ul>`, by using *only* the necessary `<m>` element. Do not use an intervening `<p>`, and do not include any adjacent text. Whitespace is OK. Then PreTeXt will add `\displaystyle`’s `\displaystyle` command to improve the visual appearance of the mathematics, and so you do not need to.

If you prefer to not have this behavior, insert an intervening `<p>`, and output will be identical, but without the `\displaystyle`.

Note that *any* text, other than whitespace, outside the `<m>` tag will disable this feature, *including punctuation*. However, according to the *Chicago Manual of Style* [1, 6.127], “Items carry no closing punctuation unless they consist of complete sentences.” So that comma at the end of your equation probably doesn’t belong there anyway.

4.9.16 L^AT_EX Packages, MathJax Extensions

L^AT_EX, which we use as the syntax for mathematics, allows extensive customization through **packages**, which are enabled through a `\usepackage{}` macro. Packages also provide extensive customization or control over the document structure. Thus, PreTeXt provides support for additional packages *to enhance mathematical content*, but does not have any facility beyond this (e.g. to influence or support document structure).

We use L^AT_EX to create PDF and print output. For most other electronic formats, we use MathJax to embed mathematics onto a web page or to make images of mathematics used in EPUB, and so on. MathJax has **extensions**, which mimic certain L^AT_EX packages. For a list of supported extensions see MathJax’s [TeX/LaTeX Extension List](#). Note that some of these extensions are technical (not supporting mathematical syntax) and some we load already as part of setting up MathJax, so carefully evaluate how a given package is selected to support certain macros.

Suppose there is a L^AT_EX math-mode macro you would like to use. We will illustrate with the `\cancelto` macro that is part of the fairly standard [cancel](#) package. It will draw an arrow through a mathematical formula, with a replacement value. Conveniently, MathJax has a `cancel` extension, and this extension *also* defines the `\cancelto`

macro. You need to check this, as a $\LaTeX$ package and a MathJax extension are not guaranteed to define the same collection of macros. Nor are the two guaranteed to have the same name (though this is best practice).

To make the `\cancelto` macro available and active in your mathematical elements (`<m>` or `<md>`) add the following to your `<docinfo>` element:

```
<math-package latex-name="cancel" mathjax-name="cancel"/>
```

Now you may freely use the `\cancelto` macro in your mathematics, confident that it will render correctly in all output formats.

Note that both attributes `@latex-name` and `@mathjax-name` must be present. They may be identical (as above, and typical) or they may be different (rare). There are situations where one might be empty. Of course, you can use these macros in other definitions of mathematics macros you might make ([Subsection 4.9.12](#)). However, we do not guarantee the absence of conflicts with other packages in use, even if employed by PreTeXt. Nor do we support debugging such conflicts.

If there are $\LaTeX$ macros you desire, but without a MathJax implementation, then there is not much we can do, short of suggesting you write your own extension (not recommended, see [A Custom Extension](#)). There are some workarounds which we may document shortly.

4.9.17 Mathematics in Titles

Titles are a key feature of PreTeXt, and migrate to various places in different conversions, where they might be mixed with bold text, or where they do not render properly. Please report any substandard situations. A good example of this phenomenon is mathematics in division titles, where we provide a `<plaintitle>` alternate. See [Section 4.8](#) for a complete discussion.

4.9.18 Extras

We were once in the business of supporting “extra” macros for mathematics, but have discontinued that practice (in a mostly backward-compatible way).

We are moving away from built-in automatic support for the `\sfrac{ }{ }` macro. As of 2023-10-18 you may use this macro, but results will just look like regular inline fractions. However, you can get superior typesetting in PDF output by loading the `xfrac` $\LaTeX$ package with the following in `<docinfo>`:

```
<math-package latex-name="xfrac" mathjax-name="" />
```

See the [Subsection 4.9.16](#) for more details.

Historically, we provided good internal support for the $\LaTeX$ `extpfeil` package of extensible arrows, even though the $\LaTeX$ version has some severe shortcomings. As of 2023-10-19, it is now an author’s responsibility to *elect* this package, along with the reliable MathJax extension of the same name. And as of 2026-03-04 we have removed all attempts at backward-compatibility. Add to `<docinfo>`:

```
<math-package latex-name="extpfeil" mathjax-name="extpfeil"/>
```

See the [Subsection 4.9.16](#) for more details. Note that this package can have several adverse impacts on the conversion to PDF via $\LaTeX$.

4.9.19 Notes

As mentioned at the start of this section, your use of $\LaTeX$ needs to also be supported by MathJax so that it may be rendered as part of an HTML page displayed in a web browser. In addition to the information at the start of [Section 4.9](#), this subsection has some notes that may help you navigate this situation.

1. Generally, MathJax supports commands available in the `amsmath` package.
2. You can construct, and use, your own macros, *but only for mathematics*, not for document structure or document management. See [Subsection 4.9.12](#).
3. Support for loading “extra” packages is extremely limited. See [Subsection 4.9.16](#).

4. The `\matrix{}` command, and its friends (such as `\pmatrix{}`) are not supported by $\text{\LaTeX}$, despite being recognized by MathJax. So use environments like `\begin{matrix}` and `\begin{pmatrix}` (with their corresponding `\end{}`, of course) and you will get accurate results for both formats.

4.10 Mathematics Best Practices

This section addresses some finer points of authoring mathematics with $\text{\LaTeX}$, motivated in large part by helping MathJax create the most accessible output possible, which in many instances will also create a superior typographical result across all conversions. We try to provide justification and explanation, though that might not be easy in all cases. Some are definitely technical, and we are only aware of them from authors' experience and reports. Many are meant to help in the conversion to braille. This is an incomplete list so additional reports and additions are welcome.

Large Numbers. Using a separator to aid in reading a large number should avoid using a space as a separator. So 234,766,544 or 234.766.544 is preferable to 234 766 544. We have used braces around the commas in the first instance, and a $\text{\LaTeX}$ “thin space”, `\,`, in the last instance.) For 234,766,544, braces around each comma prevents automatic trailing space that would normally be desirable in a list of numbers. In other words, author as:

```
<m>234{,}766{,}544</m>
```

Here is the version with commas and no effort to distinguish the big number from a list of smaller numbers: 234, 766, 544.

Ratios. If you really intend to communicate a ratio with a colon, keep it super-simple, like 3 : 2 or $a : b$. Do not wrap it in parentheses or other decorations, since then the colon will be communicated literally in braille. For example, the index of a subgroup, $[G:H]$, will not be confused with a ratio, due to the brackets. But $(4:5)$ will not be output in a conversion to braille in a way that communicates that it is a ratio. If you keep your ratios simple, braille output will use special syntax designed for ratios.

Function and Operator Names. Common mathematical functions, like $\sin$ (`\sin`) and $\det$ (`\det`), are built into $\text{\LaTeX}$, well-known, and commonly used. Compare with the versions authored as simple sequences of letters *sin* and *det*, which $\text{\LaTeX}$ interprets (and typesets) as a product of three individual variables.

Instead, use the $\text{\LaTeX}$ `\DeclareMathOperator` macro as we will illustrate. Suppose you want to discuss the set of homomorphisms from the vector space U to the vector space V , $\text{Hom}(U, V)$. Define the macros

```
\DeclareMathOperator{\homop}{Hom}
\newcommand{\Hom}[2]{\homop\left(\#1,\,#2\right)}
```

and employ as `\Hom{U}{V}`, which yields $\text{Hom}(U, V)$. Compare with the no-macro, no-special-care, version, $\text{Hom}(U, V)$. Grouping Hom as a unit will prevent $\text{\LaTeX}$ from rendering it as a product of three variables, use the correct font, and will preserve the meaning in Nemeth braille. Notice that the `\homop` macro is never used in your source.

Directly contradicting this general advice, Sean Fitzpatrick reports on 2021-01-06 that the $\text{\LaTeX}$ macro construction

```
\newcommand{\foo}{\operatorname{foo}}
```

behaves better in Asymptote diagrams than

```
\DeclareMathOperator{\foo}{foo}
```

So perhaps you need to define a second macro for use in Asymptote diagrams.

Permutations in Cycle Notation. The permutation in cycle notation, $(1246)(35)$, is difficult to distinguish from a product of two integers and gets entirely different treatment than intended as Nemeth braille. And when the points of the permutation group involve multiple digits, some other notation will become necessary anyway. Commas work, as in $(1, 2, 4, 6)(3, 5)$. Or if commas look too cluttered, then spaces are possible, as in $(1\ 2\ 4\ 6)(3\ 5)$, where we have used the $\text{\LaTeX}$ medium space, `\,`.

Text in Math. We have discussed this one already, but it is important for a conversion to braille. If you use a word inside of math, such as describing a condition for membership in a set, put any spaces inside of a `\text{}` macro, not on either side of it. So `\text{ and }` is preferable to `\ \text{and}\` . Similarly, do not put any math inside a `\text{}` macro. For example,

```
\text{ for -3 \lt x \lt -2 }
```

will lead to poor results due to the inequalities mixed in with the text. A better version would be

```
\text{ for }-3 \lt x \lt -2
```

and now the numbers will be treated as mathematics. Review [Subsection 4.9.5](#) for more details.

Math as Text. Do not use symbols from mathematics in non-mathematical situations such as the $\TeX$ construction `73^\circ` to indicate a temperature in degrees, outside of a mathematical or scientific setting, such as in a <poem> about springtime weather. In this particular case, we provide a <degree/> element that can be used in non-mathematical contexts, so we can get 73°, which will behave well in a variety of conversions (and can also be used for latitude and longitude in your poem). Similarly, we advocate writing ordinal numbers with text on the baseline, rather than a $\TeX$ construction like `2^\mathrm{nd}`. So use 1st, 2nd, 3rd, 4th, etc.

Avoid Nesting. It may seem convenient to nest custom macro definitions. For example:

```
\newcommand{\makered}[1]{\color{red}{#1}}
\newcommand{\MAKERED}[1]{\textbf{\makered{#1}}}
```

The second macro definition uses the first macro. At least one PreTeXt conversion isolates macro definitions for use only on an as-needed basis. So here, if the author used `\MAKERED` without also using `\makered` close nearby, it would lead to an issue under that conversion, because `\makered` would be locally undefined. So we recommend defining each macro from the ground up. In this case:

```
\newcommand{\MAKERED}[1]{\textbf{\color{red}{#1}}}
```

This also contradicts our advice above on operator names.

4.11 Lists

A **list** is an unusual construction, even if everybody knows exactly what one is. We view the list itself as a container of various chunks of text, while those chunks of text are the **list items**. Each item has a **marker** to identify it.

Markup and processing is complicated by the possibility of a list item containing a list, resulting in **nested lists**. We simplify this problem by *requiring* that a list appear within a paragraph (<p>), see [Subsection 4.1.3](#). One of the three exceptions is the possibility to place a list into a block that earns a caption and a number, using the <list> element, see [Section 4.21](#).

The final subsection contains some examples that you may wish to consult as you read this section.

4.11.1 Ordered, Unordered, Description Lists

An **ordered list** has items with markers that are naturally ordered (usually numerically or alphabetically). We borrow from HTML, and use the tag to construct an ordered list. Some commentators suggest an ordered list should only be used when the order of the content is important. So the steps in a recipe would belong in an ordered list, but the shopping list when you go to the store need not be an ordered list.

An **unordered list** has items with markers that have no inherent order and so are usually symbols like circles, disks, squares, etc. We borrow from HTML, and use the tag to construct an unordered list.

A **description list** has items that have *short* pieces of text as their markers. We borrow from HTML, and use the <dl> tag to construct a description list. However, unlike the HTML <dl>, the items in a description list are tagged with . A <title> element within each provides the term to be described (the <dt> in HTML terms) and the remaining elements of the provides the description (the <dd> in HTML.)

Ordered lists are used as part of <objectives> ([provisional cross-reference: topic-objectives]) and exercises (Section 4.13). Any of the three lists may occur inside the <list> element (below, Section 4.21). Otherwise, a list must occur within a paragraph, <p>. This means that to place a list within a list item of another list, the list item must contain a paragraph.

4.11.2 List Markers

Do nothing, and your ordered and unordered lists will get sensible default markers. They are consistent in the following sense. If your list has two items, and each of the two items contains a list, then these two lists will use the same type of marker.

For a description list, you author each marker as part of each list item, as discussed below in the discussion of list items.

If you want to change how an ordered list is marked, then you use the @marker attribute on the , whose value is a **format code**. This string contains one of five codes (a single character), which may be surrounded by other characters, excluding the five codes. For example marker="(A)" will produce uppercase letters wrapped in parentheses: (A), (B), (C), The extra formatting works well in a conversion to $\text{\LaTeX}$, but is not possible technically in a conversion to HTML, so it should be considered decorative, and not relied upon for meaning. The formatting does not carry through to the numbers of list items in cross-references.

If you want to change how an unordered list is marked, then you use the @marker attribute on the , whose value is a **format code**. This string contains one of three codes in the table below. Then every item of the list will have that symbol as its marker.

Code	Realization	Code	Realization
1	Arabic numerals	disc	Filled small circle, aka a bullet
a	Lowercase letters	circle	Small circle
i	Lowercase Roman numerals	square	A square
A	Uppercase letters		
I	Uppercase Roman numerals		
0	Arabic numerals, from zero		

Table 4.11.2 Unordered List Markers

Table 4.11.1 Ordered List Markers

Default marker types are assigned to each level of nested lists in the order shown in the table, and cycle back to the top of the table if necessary, though zero-based Arabic numerals will be skipped for ordered lists.

Start with the defaults, and experiment as needed. See the examples below for some extreme (and unwise) customizations of markers.

For a description list, possible markers are more varied than what you can express within an attribute. So the list item must have a <title> element (see below). This should be very short text, and may contain inline mathematics. It is often rendered in bold, so be aware that some markup may get lost. Perhaps for obvious reasons, do not include footnotes, cross-references, or display mathematics. The <dl> element has a @width attribute, with possible values narrow, medium, and wide. The default is narrow. This is a *hint* about how much text you have in these markers, and in certain presentations may make better use of horizontal space on a page.

4.11.3 List Items

So now you have a list all organized as a container. What do you put in it? List items, using the tag, again borrowed from HTML, and independent of the type of list.

A list item could be really simple, maybe just one or two words. Then you can use, and conceptualize, an element as not much different from a <p> element, and the rules about content are not much different. Even several full sentences, with some intermediate displayed mathematics, is fine.

But once you want two paragraphs in a list item, then you need to structure the contents of the item. So a list item might have five paragraphs in it, requiring five <p> elements. Notice that this is how you nest lists. Make a list item, include a paragraph, then put the subsidiary list into the paragraph. Indeed, this is the only way to nest lists. A consequence of this is that the only way to have an unstructured list item is if it is a terminal item, like the leaf of a tree.

Other items may be interspersed among the paragraphs of a list item, such as a chunk of verbatim text delimited by a `<pre>` tag. But anything with a number, such as a `<figure>` or `<remark>` is banned, in part because the consequences for numbering and organization become too complicated. Imagine a remark, and a paragraph of the remark has a list. Fine so far. But if the items of that list can again contain remarks, the possibilities become endless. You should be able to include non-textual items, like an `<image>`, and work is underway to improve this. You can use a `<sidebyside>` in a structured list item, which can in turn hold an `<image>`, `<tabular>`, or similar. But you cannot place items in such a `<sidebyside>` that are numbered, so a `<figure>` or `<table>` is not possible. A general rule is no numbered components in a list item. Computational components, such as `<sage>` are also banned from list items due to the difficulty of converting them into electronic computational notebooks with a relatively flat structure.

A list item of a description list must have a `<title>` element, to provide the text of the marker. Now that the list item has some structure, the remainder must also be structured, typically with some paragraphs, as discussed above. In other words, the earlier option of employing an `<li>` element just like a `<p>` element is not available in a description list. Further, given the complexity of presenting a description list, it can *only be a top-level list*. It can contain the two other types nested within its list items.

For ordered and unordered lists, you may optionally include a `<title>` when you have structured the `<li>`. This will be rendered as a heading of sorts for the list item, though the only distinction might be a change to an italic or oblique font. As an example, this might be a good way to author a Frequently Asked Questions (FAQ).

Note that a list item holding *only* an `<m>` element will get special treatment. See [Subsection 4.9.15](#).

4.11.4 Cross-References to List Content

Note that a list is a container, so it cannot be the target of a cross-reference, and so the three types of lists cannot have an `@xml:id` attribute. But you may well be able to point at some other structure (e.g., a `<remark>`) with a paragraph containing a list of interest. If this seems overly restrictive, read below about named lists.

By contrast, a list item, `<li>`, is not a container, and does contain content. Further, a list item of an ordered list has a marker that is natural text for a cross-reference. So in this situation, the list item can have an `@xml:id` attribute. But note that the “number” of a list item of an ordered list, which is nested inside a list item of an unordered list, is not defined, so a cross-reference by number can fail.

The “number” of a list item, mostly for the purposes of a cross-reference, is the concatenation of all of the individual markers in the containing lists, outermost first. For example, from the example lists below, the list item with content “Walleye” has number [2.I](#). These are indivisible, there is no way to get a component, excepting leading subsequences obtained by using an `@xml:id` on a containing list item. Note that the format codes never become part of the number.

4.11.5 Lists in Columns

You can control the number of columns used to layout an ordered or unordered list (but not a description list). On the `<ol>` or `<ul>` use a `@cols` attribute with values 2 through 6. (1 is the default.)

We do not yet (2018-03-28) have enough technical confidence to allow an author to specify a row-major order versus a column-major order for the layout. So understand that this is can be an implementation choice for a particular conversion, and can vary across implementations. If this is critical to conveying *meaning*, and not an aesthetic preference, then maybe consider using a `<table>` or `<tabular>` ([Section 4.19](#)).

Best Practice 4.11.3 Use Only a Few Columns for Lists. Anything more than three columns tends to get very crowded horizontally. Think twice about using more than that, and realize that six columns should be a ridiculously generous upper limit, and not a promise of good behavior in final output.

4.11.6 Exceptional Lists

We use the tags for lists in a few situations outside of anonymous lists inside paragraphs and named lists. These include the items within an objectives, subparts of an exercise, and within panels of a side-by-side. See those topics to learn about subtle differences in use.

4.11.7 Examples of Lists

To illustrate this section, we offer three too-elaborate examples. Take these as compact examples of what is possible, and not best practice in your writing. We also use these to illustrate cross-references to list items, see [Subsection 4.5.6](#).

We have a paragraph that begins with anonymous list of species that live in water (maybe partially), which necessarily is placed inside a paragraph. The roman numerals purposely do not have any extra adornment in the $\text{\LaTeX}$ version (but may for HTML output).

- 1) Amphibians
 - a. Frog
 - b. Salamander
 - c. Newt
 - d. Toad
- 2) Freshwater Sport Fish
 - I Walleye
 - II Bass
 - III *Exquisite Fish*.
Trout
- 3) Saltwater Sport Fish
 - (A) Salmon
 - (B) Halibut
 - (C) Marlin

Within the same paragraph, we transition to an unordered, two-column, list of some germs:

- | | |
|--|--|
| <ul style="list-style-type: none"> • Bacteria <ul style="list-style-type: none"> ◦ <i>Staphylococci</i> ◦ <i>Streptococci</i> ◦ <i>Salmonella</i> | <ul style="list-style-type: none"> • Viruses <ul style="list-style-type: none"> ◦ <i>Varicellovirus</i> ◦ <i>Orthopoxvirus</i> |
|--|--|

This sentence concludes our (small) paragraph on small and large organisms.
A named list, only to test cross-references.

List 4.11.4 A two-deep ordered list

- A: i. A and i
 ii. A and ii
- B: (a) B and a
 (b) B and b
 (c) *A Test Title*.
 B and c
- C: <I> C and I
 <II> C and II
 <III> C and III

An example of a description list, anonymously in a paragraph.

Red	The color of the sun at sunset.
Blue	The color of a clear sky.
Aqua	The color of shallow tropical waters.
$x^2 + y^2$	Definitely not a color

Remark 4.11.5 Best Practice. Lists are a very attractive device. Hopefully the discussion above has convinced you that they are more complicated than they first appear. Think carefully before using one, and consider if some other structure (`<paragraphs>`, `<sidebyside>`, a subdivision) might do a better job of organizing and communicating your meaning. And if a list is really necessary, consider if it should be named or anonymous, heavily-nested or nearly-flat, with columns, or with long or short content in the items. Cross-references *from* the items of a list to more complicated structures is another device that works well.

4.12 Interactive Exercises

PreTeXt has markup for a variety of interactive problem types, described individually in the subsections to follow. Generally, for a regular HTML build ([Chapter 29](#)), these will be interactive, informing the reader when their answer is correct, and providing feedback when their answer is not correct. When HTML is built to host on a Runestone server ([Chapter 32](#)), then student progress as part of a course will be recorded on the server. Some features, which will be noted below, such as execution of computer code, is more capable on a Runestone server.

Generally, an interactive exercise is authored much like a “regular” `<exercise>` ([Section 3.9](#)) with `<statement>`, `<hint>`, `<answer>`, `<solution>`. However, there is also some additional markup which serves as a **signal** that the exercise is more than a short-answer, or free-response, question. Usually, but not always, this signal is an additional element following `<statement>`. One consequence of this is that all but one type of interactive exercise will require a `<statement>` element to contain the question text.

Each type of interactive question has a **static** version for use in output formats like print, PDF, or braille. Details are given below. Note that since the ability for instant evaluation of a reader’s answer means an author provides the solution and other feedback, this can then be incorporated into a static version as an automatically-generated `<answer>` and/or `<solution>` (in addition to any others an author provides). Note that an author can provide a `<hint>` for use in all output formats, but there are not any automatically-generated `<hint>` for static versions of interactive problems. Visibility of these solutions can be controlled via the mechanism applicable to all exercises, see [Section 26.4](#).

Note the opportunity to provide feedback to the reader using a `<feedback>` element in various locations. These are generally optional, but highly encouraged as a way to improve the quality of your reader’s experience.

These interactive questions are enabled by software from Runestone Academy ([Chapter 32](#)) and have extra capabilities when the online version of our output is hosted on Runestone servers. But you *must* use a `@label` attribute. This is described more carefully next, so that we can make reference to it from other locations.

Best Practice 4.12.1 Use `label` attribute for Runestone Components. Many of our interactive exercises and programming environments are powered by software from Runestone Academy ([Chapter 32](#)). Not suprisingly, when you build online HTML for hosting on Runestone servers these components have more features than otherwise. This is managed by locating each component in a (massive) database. And so each exercise needs an identifier.

You accomplish this by placing a unique value in a `@label` attribute on an outermost element, such as an `<exercise>`, `<task>`, `<project>`, `<video>`, `<program>`, etc. You should choose these strings carefully, as they should not be changed, lest the database entries become confused. Under the hood, we get the content of `docinfo/document-id` element, and the `@edition` attribute of `<document-id>`, to form a database identifier such as `AATA_2_easy-exercise`. The `<document-id>` distinguishes your book from other books, and the edition allows you some flexibility. In other words, if you declare a new edition, you can change some of your `@label` as part of that process and you will get new database entries while preserving the old.

Short answer: *before* hosting your project on Runestone, decide on permanent values for `<document-id>`, `@edition`, and all necessary `@label`. (We expect to add a warning for Runestone builds when these values are not set.)

4.12.1 True/False Exercises

A True/False exercise *must* have a `<statement>` element, and this element *must* have a `@correct` attribute, whose value is yes or no (there is no default value). That's it. The presence of the `@correct` attribute is the signal that this is a True/False exercise.

The text of the statement is the assertion the reader must determine is true or not. The `@correct` attribute is how an author describes if the statement is true (yes) or false (no). This is enough information for a conversion to formulate a version of the question. An optional `<feedback>` element may follow the `<statement>`, and should provide more than a binary explanation of the exercise.

Presentation as an interactive element will vary cosmetically, according to the output type targeted.

A static version gets an automatic `<answer>` that is simply “True” or “False”. The automatic `<solution>` is the same, plus the content of `<feedback>`.

4.12.2 Multiple-Choice Exercises

A multiple-choice exercise is signaled by a `<choices>` element (plural) following a `<statement>` and preceding an optional `<hint>`. The `<choices>` element is structured by a sequence of `<choice>` elements, whose content is a potential answer for the reader to choose. So the `<statement>` is the prompt or question, and the `<choice>` are the possible answers.

At least one `<choice>` has an attribute `@correct` set to the value yes. The default value of `@correct` is no. There may be several correct answers, indicated with this attribute. The presentation as an exercise with one answer or many is automatic. However, in the event there is exactly one correct answer, but you wish the reader to consider the possibility of multiple correct answers, you may set the `@multiple-correct` attribute on the `<choices>` element to yes. The default value is no.

Each `<choice>` element must be further structured with a `<statement>` and a `<feedback>`, which each can contain items such as paragraphs (`<p>`). In this way, the highly-encouraged feedback can be associated with each correct and incorrect choice.

The order of the `<choice>` as authored, is the order they will be given in a static version. To present the choices in different orders in an interactive version, set the `@randomize` attribute on the `<choices>` element to yes.

An automatic `<answer>` for the static version is simply the list markers for the correct choices. An automatic `<solution>` has an indication for each choice if it is correct or incorrect, along with the choice's feedback.

4.12.3 Parsons Problems

Parsons problems are named for Dale Parsons, one of their creators, along with Patricia Haden. They could also be called **mixed-up blocks**. A reader is presented with a set of blocks containing text, either computer code or natural language, and their goal is to assemble the blocks into a correct order. This could be a computer problem with a stated purpose, or could be a logical argument such as proof, or it could be a procedure such as a recipe. An interactive drag-and-drop interface is a very efficient presentation for a reader.

Similar to multiple-choice exercises, a `<statement>` is followed by a `<blocks>` element containing a sequence of `<block>` elements. The `<blocks>` element is the signal that this is a Parsons problem. The overall `<exercise>` element may have several attributes:

- `@numbered` set to left or right indicates the blocks should be numbered, and on which side the numbers go. The default value is no.
- `@language` set to natural indicates the text of the blocks is natural language, while the use of a computer language should be indicated by naming the actual language employed. A list of languages will soon be added in [Section 4.16](#). The default is natural.
- `@indentation` set to hide indicates that a computer code exercise will not include indentation common to each line of a block. In other words, the problem is slightly harder, as the interactive interface will require the reader to adjust the block horizontally to provide common indentation for the block. The default is show.
- `@adaptive` set to yes enables an adaptive mode: after three incorrect attempts, the problem will offer help. After continued failures, the problem will be incrementally simplified: distractors will be removed, one at a time. For coding problems, requirements for indentation may be removed.

A `<block>` that has natural content is authored as usual, with a sequence of paragraphs (`<p>`) or similar. If the content is a computer language, then it should be authored as a sequence of lines, using the `<cline>` element, which allows for precise interpretation of indentation and non-standard characters. Include *all* indentation necessary for each line, no matter how `@indentation` is set. The content of the blocks, arranged properly, should form a syntactically correct program. If the reader is to provide indentation, PreTeXt will strip away the common amount of indentation.

A block may be a **distractor**, meaning it is not to be used at all in the solution. Indicate this with a `@correct` attribute on the `<block>` set to `no` (the default being `yes`). Furthermore, a given block of the solution can be authored with several alternatives, only one of which is correct. Indicate this by structuring a `<block>` with a sequence of `<choice>`. Each `<choice>` should be authored similarly to a `<block>`, and the one correct choice should have the `<correct>` attribute set to `yes` (the default being `no`).

The blocks should be authored in the correct order and the interactive interface will control the randomization. A block that is a distractor may be placed in any location amidst the other blocks. Each block should have an `<@order>` attribute that is a whole number, counting from 1. This is the fixed, mixed-up, order that will be presented in any static rendition. In static versions, sequences of blocks are in lists, which are numbered if the attribute has been set, but the left/right distinction is lost—all numbers are to the left. An automatic `<answer>` is provided if blocks are numbered, and it is just the block numbers in the correct order. An automatic `<solution>` is always generated with the text of the blocks listed in the correct order. For an exercise with computer code, the PreTeXt `<program>` element and the exercise's `@language` attribute will produce a syntax-highlighted listing.

There is no provision for a `<feedback>` element. Further explanation may be provided in a `<hint>`, `<answer>`, or `<solution>`. Also, a good interactive interface can provide assistance to a reader by telling them if they have too few or too many blocks, or combining or removing blocks, and so on.

It is possible to specify a partial ordering of blocks using the `@name` and `@depends` attributes of `<block>`. `@name` is used to assign the block a string identifier. `@depends` should be a space-separated list of the `@name` of each block that must precede the current block. The interactive version of the exercise will mark as correct any ordering of blocks that is valid according to the described dependencies. Static versions of the answer will only display the canonical answer as determined by the order of the blocks in the source document.

If a Parsons problem that has a `@language` set to a value that is valid for an activecode ([Subsection 4.16.3](#)), it can be made runnable in interactive versions of the book - when the student completes the exercise, they can run the code they built. To do so, define a `<program>` element as a sibling to the `<blocks>`. The program can have all of the normal attributes for an activecode, but does not need the `@interactive` attribute to be set. It should have an empty body as the student work will be used as its contents. If it is necessary to add additional code to the program, you can add either or both a `<program-preamble>` and `<program-postamble>` element as siblings to the `<program>`. The `<program-preamble>` will be placed before the student's code, and the `<program-postamble>` will be placed after the student's code. Both elements can have an `@indent` with some text to use as the indentation for every line in the block; use this to align the added code to the expected format of a correct answer.

4.12.4 Horizontal Parsons Problems

A **horizontal Parsons problem** is very similar to the traditional *vertical* problems just described in [Subsection 4.12.3](#). Except they are horizontal. Which means the blocks are very short (a word or symbol) and they are rearranged by the reader to form a horizontal bit of text. They are ideal for constructing a single line of computer code, such as a regular expression, or a short sentence. While similar, they have a few features which are different.

To indicate a horizontal problem, set the `@layout` attribute on the `<blocks>` element to the value `horizontal`. The default value is `vertical` and so is not necessary for the traditional versions.

The syntax for blocks is the same, and they are authored in an order that yields a correct answer. The use of an `@order` attribute on each `<block>` indicates a rearrangement to be used for a static version. Blocks may be reused. To do this, place an `<@xml:id>` on a single instance of a block to be reused. Then, to indicate its reuse as part of a correct version, use a `@ref` attribute on a `<block>`. Since the duplicate instance is not shown to the reader, the use of `@order` and `@ref` is mutually-exclusive. You can set the `@reuse` attribute on the `@blocks` element to the value `yes` to force the interface to allow the reader to reuse blocks, even if there is no duplication in the correct version. Otherwise, the interface reacts to the presence or absence of blocks with the `@ref` attribute.

Distractors may be included in the same manner as for vertical Parsons problems, by setting the `@correct` attribute on a `<block>` to the value `no`. The default is `yes`. It does not make much sense to indicate a reusable block

(in either form) as a distractor, so don't.

Another option on the `@blocks` element is the `@randomize` attribute. The default value is yes. When the value is set to no, then the blocks are always presented to the reader in the same order, which is the one given by the `@order` attribute.

As of 2022-11-28, the implementation of these problems assumes that the blocks rearrange to be computer code. So you *can* set the `@language` attribute on the `<exercise>` element to `natural`, but it will not have much effect. Markup like `<em>` on the text of a block will be unpredictable, as will attempts to use mathematics. And the text of blocks will always be in a monospaced font, perhaps with some syntax highlighting if `@language` is set properly.

4.12.5 Matching Exercises

A **matching exercise** asks a reader to pair a **premise** with a **response**. Similar to multiple-choice exercises and Parsons problems, a `<matches>` element follows a `<statement>` and this is the signal. The `<matches>` element is structured as a sequence of `<match>` elements, each of which has a `<premise>` element and the matching `<response>` element. Since the content of each premise and response is best kept short and simple as a phrase, the elements may also be simple phrases without the additional structure of `<p>` elements, or similar. That's it.

An interactive interface should randomize at least one of the lists of premises and responses, consulting the authored version for the correct pairings. For a static version, an author should put an `<order>` element on each `<match>` whose value is a whole number, starting from 1. Then the `<premise>` will appear in the authored order, while the `<response>` will be re-ordered according to the attribute.

A single `<feedback>` element may be given, as a peer of `<statement>`, in addition to authored `<hint>`, `<answer>`, or `<solution>`. For a static version an automatic `<solution>` presents the problem in the order the `<match>` were authored.

4.12.6 Clickable Area Exercises

Clickable Area is a misnomer, assuming an interactive version of this type of problem. Perhaps "Select Text" would be better. In any event "area" is not meant to connote a geometric notion, instead this is about a sequence of characters, either in natural language or in computer code.

A `<statement>` is a prompt, describing which sort of areas should be selected by the reader. Such as: Locate all the nouns in the following paragraph. This is followed by an `<areas>` element, which is the signal for this type of exercise. The `<areas>` element holds either (a) PreTeXt paragraphs or similar, or (b) a sequence of `<cline>`, understood to be program code. In the latter case, a `@language` attribute on the `<areas>` element will enable the right syntax highlighting in some output formats.

Within the content of the `<areas>` element an `<area>` element can be used to surround a run of characters. These cannot cross XML boundaries, such as a `<cline>` or `<p>`. A `@correct` attribute, with values yes or no, indicates the text is to be selected (clicked on) or that it is a distractor (tempting to click on). The default is yes, so only distractors need to be indicated.

A single overall `<feedback>` may be placed following the `<areas>` element.

Now an interactive version will allow the reader to click on and off the marked areas of text, and provide information about which are correct and which are not, in addition to general feedback.

Generally, a static version of the problem will not be clickable. `<wink/>` So your prompt might say something generic like "select" or "locate" since a reader might work the problem on paper by circling or underlining the areas. A static version includes an automatic `<answer>` which is a list of the correct areas as text, followed by a list of the incorrect areas as text. An automatic `<solution>` repeats the text in the `<areas>` element, and uses (accessible) visual cues to note the correct and incorrect text.

4.12.7 Fill-in-the-Blanks Exercises

A **fill-in-the-blank** problem allows the reader to submit free text that is evaluated for correctness. The signal for a fill-in problem is a `<statement>` containing a sequence of `<fillin>` elements with a sibling `<evaluation>` element. Randomized elements to appear in the statement and solution and to be used in evaluation of reader responses requires the presence of an additional `<setup>` element that is also a sibling to the `<statement>`. The presence of the

`<setup>` provides the ability for the statement and evaluation of the responses to incorporate random, dynamically generated elements.

The body of the `<statement>` is the same as for any exercise-like statement, with the addition of two elements of markup. A `<fillin>` is placed at the location a blank would be required for the reader's response. Details for this element are described in the next paragraph. When the question includes a `<setup>`, content can reference the string or TeX representation of the object (if defined) by using a `<eval>` element, where the name of the object is provided by a `@object` attribute.

The `<fillin>` element is required to have an `@answer` attribute that contains an answer that would be evaluated as correct, in the case that the question has a known, static, valid answer, or an `@ansobj` attribute that contains the variable name of a dynamically generated object representing a correct answer, in the case that the problem has an associated `<setup>`. In addition, the `<fillin>` is required to have a `@mode` that characterizes the mode by which the submitted answer will be parsed. Current possible modes are `string`, `number`, and `math` (only for the dynamic implementation). The `<fillin>` element may have a `@width` attribute specifying the number of characters for the static-version blank. For questions that involve dynamic evaluation, a `@name` attribute may be used to identify a variable name that will reference the reader's response during dynamic evaluation of *other* blanks. (A standard `ans` variable will always be available in the context of each individual response's evaluation.)

The `<evaluation>` block will have a sequence of `<evaluate>` elements, one associated with each of the corresponding `<fillin>` elements appearing in the `<statement>`. By default, the association is by order. If each `<fillin>` has an associated `@name`, the association does not need to be in order. Within each `<evaluate>` element, there will be a sequence of `<test>` elements that define a comparison rule for the test and the associated feedback provided when the test evaluates as true. Default behavior is that the `@answer` or `@ansobj` attribute of the corresponding `<fillin>` is used for comparison to evaluate a correct response. If the author wishes to override this default or wishes to provide specific feedback for a correct response which will also be used for the automatic solution for static versions, the author will identify one `<test>` element with the attribute `@correct="yes"`.

Within each `<test>`, the author needs to specify which type of comparison will be used. For non-dynamic questions, three comparisons are available: `<numcmp>`, `<strcmp>`, and `<jscmp>`. For dynamic questions that include `<setup>`, additional options include `<mathcmp>` comparisons involving math objects and `<logic>` elements to construct more complex boolean tests involving these objects, described later. Feedback that is provided to the reader when the comparison is true, beyond saying the response is correct/incorrect, is provided in a `<feedback>` element which can itself contain structured PreTeXt text including `<eval>` elements to reference dynamically generated objects.

A `<numcmp>` and a `<strcmp>` can specify that the comparison should be to match the provided answer for the corresponding `<fillin>` by including the attribute `@use-answer="yes"`. Otherwise, these comparisons must specify the value being compared. A `<numcmp>` provides this with an attribute `@value` that indicates the matching value. If the `<numcmp>` includes the optional `@tolerance` attribute, any response in the range of from the given value plus or minus the tolerance will be accepted. Alternatively, the `<numcmp>` can directly provide a `@min` and `@max` attribute to specify an arbitrary interval. A `<strcmp>` that does not include the `@use-answer` will have content that defines a matching string or more generally a matching regular-expression. By default, PreTeXt augments the provided string to match the entire response after stripping leading and trailing white space. The attribute `@strip="no"` overrides this behavior so that, for example, the match can look for the presence of a pattern within the response rather than matching the entire response. In addition, the attribute `@case="insensitive"` makes the regular expression ignore case during the comparison.

A comparison involving `<jscmp>` allows the author to provide arbitrary Javascript code that will evaluate to either true/false or a string containing feedback for a valid match, where the reader's evaluated response is available as a variable `ans` and all of the reader's responses are in an array `ans_array`. This allows custom evaluation that can allow the evaluation of one blank to depend on what was answered in another blank. The Javascript is evaluated in the context of a local, anonymous function. Nevertheless, the author should take care that no commands are included that invoke mutable side-effects.

To create dynamic, randomizable versions of a fill-in-the-blank question, a `<setup>` element is required as a sibling of the `<statement>` and `<evaluation>` elements. For the interactive version of these questions, the problem will be created with a random number generator initialized with a seed that is stored by the Runestone Component for the reader. This results in the reader seeing the same version the next time they open the question. A button allowing the reader to request a new randomized version changes the seed and regenerates the question. Static representations need to provide consistent versions each time, so the `<setup>` requires a `@seed` attribute.

Within the `<setup>`, the author provides statements that will initialize Javascript objects that will be accessible in the `<statement>`, `<solution>`, and `<evaluation>` elements. The author can provide Javascript code within a `<setupScript>` child of the `<setup>`. It is likely that the script contents will need to be escaped to avoid XML conflicts, for example by individually escaping problematic characters or surrounding the code with `<![CDATA[...]]>`. In the context in which the script runs, there is an object `v` representing the dynamic variable namespace. Any object that the author wants to access should be created as a member of that object, such as `v.myObject = ...`. The name `myObject` could then be used (without reference to `v`) in an `@ansobj` attribute of a `<fillin>`, in an `@object` attribute of an `<eval>` or directly in a `<jscmp>` comparison.

To provide randomization, a random number generator based on the current seed is available in the `<setupScript>` using the object name `RNG`. The `RNG` object includes the following methods for generating various types of random values.

1. `RNG.random()`: Uniform random values between 0 and 1.
2. `RNG.randSign()`: Random values +1 and -1.
3. `RNG.randint(a,b)`: Random integer from $a, a + 1, a + 2, \dots, b$.
4. `RNG.randUniform(a,b)`: Random floating value $a < x < b$.
5. `RNG.randDiscrete(a,b,dx,nonzero)`: Random value x from $a, a+dx, a+2dx, \dots$ with $x \leq b$. If `nonzero=true` (optional parameter), then $x = 0$ is excluded.

Using this generator will ensure that subsequent visits by the reader to the question will see the same version.

If the author wishes for reader responses to be parsed to assess format prior to evaluating correctness, `<setupScript>` should include a declaration of an entry `v.types`. The value can either be a single parser function applied to all blanks or an array of separate parser functions, one for each `<fillin>`. A parser function takes a string representing the response for the blank and returns the parsed object. If there is an error in the structure of the response, the function should throw `new TypeError(errMsg)`, and the `errMsg` will be provided as feedback for the response without performing any of the tests.

The Runestone Component that implements interactivity takes the HTML generated from the `<statement>` after the `<setup>` is processed and performs substitutions of any `<eval>` elements. The recomputed HTML is then inserted into live webpage. A `<postRenderScript>` entry in `<setup>` can be used as a second script that runs as a call-back after this substitution occurs. Such a script would be useful if the script requires access to the actual webpage elements and not just defining objects for the evaluation context.

Math Objects in Dynamic Problems. A Javascript library that supports some mathematical objects in the dynamic setup and evaluation of fill-in-the-blank problems. In order to access these objects, an XML-interface is provided to interact with the objects in the `<setup>` and `<evaluation>` elements. Any `<fillin>` elements expecting to parse as mathematical objects should use `@mode="math"` and assign `@ansobj` to the name of a dynamic expression object created in `<setup>` that represents a correct answer.

For the `<setup>` stage, a sequence of `<de-object>` elements can be used to define mathematical dynamic expression objects. Each `<de-object>` will have a `@name` attribute representing the object name. The mathematical nature of the `<de-object>` is indicated with a `@context` attribute. Currently supported expression types are to represent numbers using `@context="number"` and formulas using `@context="formula"`.

The content of the `<de-object>` element is an XML element that defines the actual expression. The following choices are available, described in the following paragraphs:

1. `<de-random>`: create a randomly generated number
2. `<de-number>`: create a number by providing its value or formula
3. `<de-expression>`: create an expression or formula involving variables by directly providing a formula or by performing a substitution or a derivative on a previously-defined expression object
4. `<de-evaluate>`: create a number by evaluating an expression object by replacing each variable with a specified number value

As elements are defined in sequence, formulas used at any stage may include the name of any previously generated object as a valid symbol in the formula.

To define a random number, use a `<de-random>` element. The same random number generation routines provided by the RNG object are accessible through this element using a `@distribution` attribute and additional attributes for the distribution parameters, some of which come with default values.

1. `@distribution="uniform"`: Parameters are `@min=0`, `@max=1` as decimal values. Gives a value from the interval (min, max).
2. `@distribution="sign"`: No parameters. Gives ± 1 as possible values.
3. `@distribution="integer"`: Parameters are `@min`, `@max` as integer values. Value is an integer chosen inclusively between min and max.
4. `@distribution="discrete"`: Parameters include `@min`, `@max`, `@by="1"`, and `@nonzero="yes"`. Values chosen from the sequence $\min + k \cdot \text{by}$, $k = 0, 1, 2, \dots$ that are not greater than max.

In practice, all distributions can be obtained using just the uniform and discrete distributions.

To define a number by formula, including defining rational numbers as fractions, use a `<de-number>` element. The content of the element should be a single number or a formula that results in a constant value. Symbols of previously defined numbers as well as mathematical constants like `pi` or `e` may be part of the formula. Any undefined symbols will result in an error.

To define an expression using a formula that might involve variables, use a `<de-expression>` with `@mode="formula"`. The content of the element can be a mathematical formula that includes both numbers and variables. Previously defined numbers and expressions can be used in the formula by referencing their object name.

To define an expression that is the partial derivative of another expression with respect to a variable, use `<de-expression>` with `@mode="derivative"`. Such an expression must have a `<formula>` child that contains an object representing the original expression. The content can be either a `<eval>` with `@object` to reference a previously-defined object or a separate `<de-expression>` defining an expression not saved as a separate object. In addition, the element must contain a `<variable>` element with `@name` indicating the name of the variable of differentiation.

Formula evaluation and substitution follow a similar pattern. Formula evaluation to compute a number is performed with a `<de-evaluate>` element, which requires that every variable in the expression is assigned a specific numerical value. Formula substitution is like function composition and allows for any subset of the involved variables to be assigned either number or expression values and uses `<de-expression>` with `@mode="substitution"`. In either event, the element requires a `<formula>` child element to specify a starting expression using a `<eval>` element to specify a previously-defined `<de-object>` or a `<de-expression>` element to generate a new formula expression. For each variable that will be replaced by a value or another expression, we will include a `<variable>` with `@name` specifying which variable is being replaced. The content of the `<variable>` element can be an `<eval>` to reference an existing object or any of the elements that can be used to generate a `<de-object>`.

Automatic reduction and simplification can be applied to elements created by `<de-evaluate>` and `<de-expression>` by including the attribute `@reduce="yes"`. Elements created by `<de-number>` always reduce the resulting number. Reductions include reducing arithmetic involving numerical values and simple algebraic identities. The identities that are applied include the following where u and w represent any expression:

1. $u \pm 0 \rightarrow u$
2. $0 \cdot u \rightarrow 0 \rightarrow u$
3. $0 \div u \rightarrow 0 \rightarrow u$
4. $1 \cdot u \rightarrow u \rightarrow u$
5. $-1 \cdot u \rightarrow -u \rightarrow u$
6. $u \div 1 \rightarrow u \rightarrow u$
7. $u^1 \rightarrow u \rightarrow u$
8. $u^0 \rightarrow 1 \rightarrow u$

9. $1^u \rightarrow 1 \rightarrow u$
10. $u + (-u) \rightarrow 0$
11. $u - u \rightarrow 0$
12. $u + -w \rightarrow u - w$
13. $u - (-w) \rightarrow u + w$
14. $-(-u) \rightarrow u$
15. $u \cdot (-w) \rightarrow -u \cdot w$
16. $u \div (-w) \rightarrow -u \div w$
17. $(-u) + (-w) \rightarrow -(u + w)$

For the `<evaluation>` block, two comparison elements are accessible—`<mathcmp>` for testing whether two math objects are equivalent and `<logic>` for constructing compound tests involving simple Boolean logical operations. The `<mathcmp>` comparison is performed by numerically evaluating the two expressions being compared over a range of values for each variable present and verifying that they are numerically within a comparison tolerance.

To compare the reader's submitted response to the provided correct answer with `<mathcmp>`, use `@use-answer="yes"`. To compare the submitted response to any other object, the `<mathcmp>` element should have a single evaluated math object as content, which could include a newly generated expression using `<de-expression>` that is based on the submitted responses. To compare two different math objects, the `<mathcmp>` should have two math objects. For convenience, there are some implicit representations available as well, illustrated in [Listing 4.12.2](#).

Listing 4.12.2 Possible structures for using `<mathcmp>` within a `<test>`. The `<de-expression>` would be replaced by any valid construction as described for the `<setup>`.

```

<!-- Compare submitted response with fillin answer -->
<mathcmp use-answer="yes"/>

<!-- Compare submitted response with pre-computed object -->
<mathcmp obj="objName"/>

<!-- Compare submitted response with pre-computed object (alternate) -->
<mathcmp>
  <eval obj="objName"/>
</mathcmp>

<!-- Implicit mathcmp with submitted response (alternate) -->
<eval obj="objName"/>

<!-- Compare submitted response with newly generated object -->
<mathcmp>
  <de-expression/>
</mathcmp>

<!-- Implicit mathcmp with submitted response (alternate) -->
<de-expression/>

<!-- Compare with newly generated object with pre-computed object -->
<mathcmp>
  <de-expression/>
  <eval obj="objName"/>
</mathcmp>

<!-- Compare two newly generated objects -->
<mathcmp>
  <de-expression/>
  <de-expression/>
</mathcmp>

```

Compound logical structures are created using the `<logic>` element in the place of a comparison. To specify which operation is being used, the `@op` attribute is set to one of `@op="and"`, `@op="or"`, or `@op="not"`. The operations are considered *n*-ary, meaning that the contents should be one or more comparison elements, which might include additional logic. The `@op="and"` will evaluate as true when *all* of the children comparisons evaluate as true. The `@op="or"` will evaluate as true when *at least one* of the children comparisons evaluates as true. The `@op="not"` is technically implemented as the negation of `@op="and"`, and so will evaluate as true when *at least one* of the children comparisons evaluates as *false*. In the absence of any `<logic>`, there is always an implicit `@op="and"` so that any sequence of comparisons within a single `<test>`, all comparisons are required to evaluate as true.

Static Representations. A static version will include an automatic `<solution>` which “fills in” each blank with the correct answer that is provided by the `<fillin>`. The feedback text specified in the `<evaluation>` that associated with a correct response for each of the `<fillin>` blanks are then provided in sequence. Consequently, the author should ensure that the feedback for a `<test>` with `@correct="yes"` makes sense in the context of a back-of-the-book solution and not just as immediate feedback in an interactive setting.

If the author provides an optional `<solution>` element, this text will also be included as a solution. This solution may contain `<eval>` elements in the same manner as described for the `<statement>` element. The automatic solution can be disabled by including `@include-automatic="no"` as an attribute of the `<solution>`.

To build the static version, PreTeXt needs to generate the dynamic substitution rules. Each dynamic question that includes `<setup>` is processed using its corresponding static seed to evaluate the potential substitutions. The resulting substitutions are stored in one of the generated files. Using the PreTeXt-CLI, the substitution file is generated using the command `pretext generate dynamic-subs -t static-target` where `static-target` is replaced by

a static project target such as PDF.

As of 2022-06-14 a major effort is underway to provide comprehensive markup for **fill-in-the-blank** problems. Until then, there is transitional markup intended only to supply a migration path for projects originally authored for Runestone servers ([Chapter 32](#)). So (a) our documentation is sparse, and (b) there will be no backward-compatible improvements. So in particular, new projects should wait for the new markup. Also, studying examples may be a useful way to augment what is described here.

A `<statement>` is enriched with empty `<var/>` elements which will render as the blanks in the problem.

The signal for a fill-in problem is a `<setup>` element containing a sequence of `<var>` elements. Each `<var>` contains a sequence of `<condition>` elements that describe possible values (via regular expressions) which might appear in a blank. The first condition describes the correct answer(s), and then the subsequent conditions are descriptions of probable incorrect answers. Each `<condition>` has a `<feedback>`. So the first condition to match an entry provided via a blank will be noted as correct or incorrect, and its feedback will be relayed.

The `<var>` of the statement and the `<var>` of the setup are in a 1-1 correspondence, which establishes how the setup is associated with a blank. The `<var>` in the statement may have a `@width` attribute whose value controls how many characters would be visible in the blank.

A static version will include an automatic `<solution>` which “fills in” each blank with a correct answer, and then duplicates the feedback text, in order.

4.12.8 Coding Exercises and Projects

A **Coding Exercise** is formed by placing a `<program>` element *after* the `<statement>` of an `<exercise>`. The main distinction is that this is a signal that the interactivity is provided by the `<program>`, and therefore the `<exercise>` will not be understood as a short answer exercise ([Subsection 4.12.9](#)). For this reason, the `<program>` should be requested as one of the interactive realizations ([Subsection 4.16.2](#), [Subsection 4.16.3](#)).

The identical construction may be used with any PROJECT-LIKE ([List 4.2.2](#)) such as an `<activity>`. As of 2022-06-17 this only applies to the form that uses a `<statement>`, but will soon also apply to `<task>` within PROJECT-LIKE.

Realize that it is always possible to place a `<program>` *inside* of a `<statement>`, and if there is no `<program>` that is *after* the `<statement>` (or another signal for an interactive exercise) then the `<exercise>` will be classified as a short-answer exercise. It may be instructive to understand that in a static realization, a `<program>` at the *end* of a `<statement>` may be visually identical to an `<exercise>` where the `<program>` is *after* the `<statement>`, even though the former is a short-answer exercise and the latter is a coding exercise (which will render differently for different output formats and hosting platforms).

4.12.9 Short Answer Exercises

Short Answer questions might also be known as **Free Response** questions, or **Essay** questions. A PreTeXt `<exercise>`, or a PROJECT-LIKE that is not structured by `<task>`, is implicitly of this nature. But you still need to signal that you wish such a problem to be interactive, typically with a text box where the student can enter an answer. So, similar to other types of exercises, add a `<response/>` element immediately after the `<statement>`. (We expect to add attributes for this element to influence the behavior of the text box.)

In an online setting, it is a simple matter to provide a place for a reader to type in an answer, response, or essay. But then what? Until artificial intelligence is brought to bear, somebody (not something), such as an instructor for a course of enrolled students, will need to read a response and provide a score and/or comments that can be saved and distributed back to the students. So the first prerequisite is that HTML output is being built for a capable platform. As of 2022-06-15, this means a Runestone server ([Chapter 32](#)), but it could easily also be a WeBWorK server.

An author can also supply an `@attachment` attribute on the `<exercise>` element. When set to yes the short answer question will allow the student to upload a single document in support of their answer. This document might be a diagram or an entire essay in PDF format.

A publisher can control when a response area is created in HTML output ([Subsection 44.4.16](#)). The default is to only have this area present when it is possible for a response to be graded and scored. However, an option will cause a response box to be created always. A reader can reflect on the question by typing in a response, and the text will be saved *on that particular device* only. When it is impossible for a response to be graded, placeholder text will warn the reader.

4.12.10 Timed Assessments

A collection of `<exercise>` may be optionally selected to be part of a timed assessment. A timed assessment will show each `<exercise>` separately with a navigation mechanism to allow the student to move from exercise to exercise. A student may “flag” an exercise for later review if they are unsure of their answer. When a student completes the assessment they will receive varying amounts of feedback on their performance, depending on how the assessment is configured.

To enable this behavior, place a `@time-limit` attribute on the `<exercises>` element, with a value indicating the time limit in seconds. In addition you can allow the student to pause the timer by including `@pause` attribute on the `<exercises>` element with a value of “yes”. You can control the amount of feedback provided to the student with two additional attributes. `@results` and `@feedback`. If the `@results` attribute is set to “yes”, the student will receive a summary of their performance after completing the assessment. If the `@feedback` attribute is set to “yes”, the student will receive feedback on their answers.

For books that will be hosted on Runestone Academy, you should include a `@label` attribute on the `<exercises>` element to provide a short name for the assessment. This label will be used to identify the assessment in the grade-book. It will also allow you as the instructor to reset the exam for a student to give them another try if they need it for any reason, including some kind of technical glitch with their browser as they are taking the exam.

4.12.11 Group Work Exercises

A collection of `<exercise>` may be optionally selected for work by a group of readers. This behavior is only available when hosted on a Runestone server. We re-purpose a `<worksheet>` specialized division for groups of such exercises.

To enable this behavior, place a `@groupwork` attribute on a `<worksheet>` element, with a value of yes. Then all of the contained `<exercise>` will be provided by Runestone as group work.

4.13 Exercises and their Solutions

As described in [Section 3.9](#) an `<exercise>` can be placed in many different locations, and a `<project>` has similar features. It is critical to understand that you want to author any hints, answers, or solutions immediately following the statement of an exercise. If your PreTeXt source is public, and you would like to keep some aspects of the solutions private, then read [Section 36.1](#) for some practical advice. See [Section 37.2](#) as well for information on creating a standalone *Solutions Guide*. We concentrate here on techniques for controlling visibility and location of the components of exercises within your primary output.

4.13.1 Exercises, Original Versions

In a conversion to HTML, a `<hint>` to an `<exercise>` renders nicely in a knowl, right below the exercise statement. For a conversion to L^AT_EX/PDF/print, you might wish to display a hint, visibly, as part of the exercise, or you may wish to park the hint in a *Hints to Exercises* division in the back matter. To control visibility of the components of exercises (and projects) there are twenty switches you can use. See [Section 26.4](#) for more.

4.13.2 Exercises, Solutions Versions

Exercises, and their components may be duplicated easily, to provide a back matter appendix with solutions, or within each division. For example, you can easily create an end-of-chapter division with solutions to every inline exercise throughout the chapter and solutions to all the divisional exercises from each section of the chapter.

The `<solutions>` element will create an entire division, semi-automatically. You can provide a `<title>`, an `<introduction>`, and `<conclusion>`. The remaining content is statements, hints, answers, and solutions to exercises (and projects).

If `<solutions>` is a child of `<backmatter>`, then an appendix will be generated, and covering `<exercise>` from the entire `<book>` or `<article>`. If `<solutions>` is a child of a division, then a new subdivision is created and the scope is all `<exercise>` for the division. So, for example, a `<solutions>` placed inside a `<chapter>` will render as a division that looks like a `<section>` and will include components of all the exercises (at any level) contained within the `<chapter>`.

An optional attribute is `@scope`, whose value is the `@xml:id` of a division. Then it is this division which is scanned for exercises and their solutions (rather than defaulting to the enclosing parent of the `<solutions>`). This allows for much greater flexibility. For a simple example, suppose a `<chapter>` contains two `<exercises>`, and you want to have two `<solutions>` within the chapter, each covering just one of the `<exercises>`. This can be accomplished with `@scope`, and you can arrange the four divisions (two `<exercises>` and two `<solutions>`) however you wish within the chapter.

An author filters the types of exercises, and their components, through attributes of the `<solutions>` element. For example

```
reading="hint answer"
```

would cause every `<exercise>` within each `<reading-questions>` to have its `<hint>` and `<answer>` displayed, but not its `<statement>` nor its `<solution>`. These are the attribute names and the possible values.

Table 4.13.1 Attributes (left) and Values (right) for `<solutions>` element

<code>inline</code>	<code>statement</code>
<code>divisional</code>	<code>hint</code>
<code>reading</code>	<code>answer</code>
<code>worksheet</code>	<code>solution</code>
<code>project</code>	

Note that the `@worksheet` filter only refers to `<exercise>` within any `<worksheet>`. This is because worksheets are built around `<exercise>` that are slightly different than others, such as allowing `@workspace`, and being allowed in a `<sidebyside>` for layout purposes. Similarly, a `<project>` (or more precisely PROJECT-LIKE) is a child of a division, and so can appear inside a `<worksheet>`, or outside of a `<worksheet>`. The `@project` filter applies equally to both instances.

So, PreTeXt source like

```
<section>
  <title>Tropical Bird of Paradise</title>
  ...
  <solutions worksheet="hint solution" project="hint answer">
    <title>Hints and Solutions to Worksheets and Projects</title>
  </solutions>
</section>
```

would generate an entire subsection in the section on Birds of Paradise, containing with hints and solutions to every `<exercise>` in any worksheet in the section, and hints and answers for every project, located anywhere (including in subsections, subsubsections, and worksheets). There is a great deal of flexibility, though not every imaginable scenario may be possible.

An `@admit` attribute specifies some feature of an exercise's serial number to determine whether its components are admitted into the solutions division. (For example, the "serial number" of Exercise 1.2.3 is 3.) Presently, the only options are odd, even, and the default all. So, PreTeXt source like

```
<solutions divisional="answer" admit="odd">
```

would generate a subsection with answers to only the odd-numbered divisional exercises.

4.14 Images

4.14.1 Raster Images

A **raster image** is an image described pixel-by-pixel, with different colors and intensities. Photographs are good examples. Common formats are Portable Network Graphics (PNG) and Joint Photographic Experts Group (JPEG, JPG), which will both work with pdf $\LaTeX$ and modern web browsers. JPEG is a good choice for photographs since

they are compressed on the assumption they will be viewed by a human, while PNG is a lossless format and good for line art, diagrams and similar images (if you do not have vector graphics versions, see below).

To use these images, you simply provide the complete filename, with a relative path. A subdirectory such as `images` is a good choice for a place to put them. It is your responsibility to place these images where the $\TeX$ output will find them or where the HTML output will find them. Your PreTeXt source would look like:

```
<image source="images/crocodiles.png" width="50%" />
```

Typically you would wrap this in a `<figure>` that might have an `@xml:id` attribute for cross-references, with or without a caption. There is no `@height` attribute, so the aspect ratio of your image is your responsibility outside of PreTeXt. The `@width` attribute is a percentage of the available width of the text (outside of a `<sidebyside>` panel).

You should also provide a `<description>` and/or `<shortdescription>` as a child of the `<image>`, or else explicitly declare the image to be decorative with an attribute `@decorative` set to `yes`. A `<shortdescription>` should be text-only (but perhaps with `<var>` children) and be less than 100–140 characters long. A `<description>` should be structured with `<p>` and `<tabular>` elements. For example:

```
<image source="images/crocodiles.jpeg" width="50%">
  <shortdescription>Five crocodiles partially submerged.</shortdescription>
  <description>
    <p>
      Five crocodiles are in a pond. Three of them have their eyes above
      the water line, looking in the direction of the camera. The other
      two are in the background and only their tails are visible above
      the water line.
    </p>
  </description>
</image>
```

See [Subsection 4.41.3](#) for advice on writing effective image descriptions.

4.14.2 Vector Graphics

An image is a **vector graphic** if the file describes the geometric shapes that constitute the image. So a simple diagram would be a good candidate, but a photograph would not. Popular formats are Portable Document Format (PDF) and Scalable Vector Graphics (SVG). You will get the best results with PDF images in $\TeX$ output and SVG images for HTML. The principal advantage of these formats is that they scale (big or small) smoothly, along with fonts. This is critical when you cannot predict the screen size for a reader of an electronic version.

Unless you describe these images with a language (see [Subsection 4.14.3](#)), you are responsible for providing the PDF and SVG versions. The `pdf2svg` utility is very useful if you have PDF images only. To have these different images used for different output formats, you simply follow the instructions above, but *do not include a file extension*. This alerts the conversion to use the best possible choice for any given output, and to embed it correctly. So presuming you made available the files `images/toad-life-cycle.pdf` and `images/toad-life-cycle.svg`, the following example would incorporate the PDF version with $\TeX$ output and the SVG version for HTML output.

```
<image source="images/toad-life-cycle" width="85%">
  <description><p>The four stages of a toad's life.</p></description>
</image>
```

Vector graphics images can be created with source code in different languages ([Subsection 4.14.3](#)) or with applications, such as Inkscape ([Section 5.7](#)). If you are creating non-technical graphics that have lots of geometric shapes and simple text (a look like a movie poster), then using a tool like Inkscape is a great choice since its native file format is an enhanced version of SVG and a faithful PDF is easy to create.

4.14.3 Images Described by Source Code

There are various languages which may be used to describe diagrams, geometric objects, or data plots. A key strategy enabled by PreTeXt is to put these specifications of such images *directly in your document's source* rather than losing track of them over time.

So we have various elements which are children of `<image>` that hold these source code descriptions. Then PreTeXt provides techniques for realizing these in the best formats for various devices and print. So if you are accustomed to the idea of a `@source` attribute pointing to a file, think of these elements as alternative specifications.

4.14.3.1 Asymptote

[Asymptote](#) is a vector graphics language that produces high-quality output in WEBGL, SVG, PNG, and PDF formats. You can describe 2-D or 3-D objects, and the 3-D objects are interactive in online output as HTML WEBGL files. $\LaTeX$ may be used, and your macros are automatically available for use.

Authoring is straight-forward. Inside an `<image>` include a child `<asymptote>` to hold the code. For example:

```
<image>
  <description>A histogram of Gaussian data.</description>
  <asymptote label="gaussian-histogram">
    import graph;
    import stats;

    size(400,200,IgnoreAspect);

    int n=10000;
    real[] a=new real[n];
    for(int i=0; i < n; ++i) a[i]=Gaussrand();

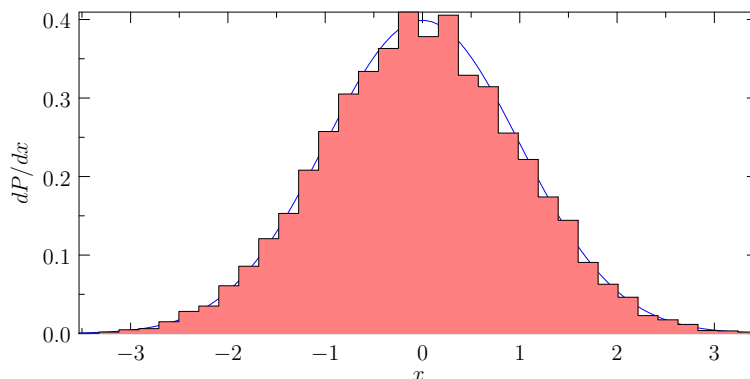
    draw(graph(Gaussian,min(a),max(a)),blue);

    // Optionally calculate "optimal" number
    // of bins a la Shimazaki and Shinomoto.
    int N=bins(a);

    histogram(a,min(a),max(a),N,normalize=true,low=0,lightred,black,bars=false);

    xaxis("$x$",BottomTop,LeftTicks);
    yaxis("$dP/dx$",LeftRight,RightTicks(trailingzero));
  </asymptote>
</image>
```

Here is the result. Look elsewhere for examples of 3-D output from Asymptote.



Notes:

- Notice the necessity of escaping the less-than in the for-loop. See [Subsubsection 4.1.4.2](#).
- Setting a `@label` on the `<asymptote>` tag is necessary to have a stable name for graphics files that will be generated.

- The `<description>` is an important part of making your output accessible.
- Notice the use of $\LaTeX$ for the label on the vertical axis. All of your macros defined in `docinfo` are available for use, so you can keep notation consistent.
- You need to produce PDF versions of your diagrams for use in a conversion to $\LaTeX$.
- You need to produce HTML versions of your diagrams for use in a conversion to an electronic format based on HTML. For a 2-D diagram these are a thin wrapper around an SVG image. For a 3-D diagram these are interactive WebGL objects.

It is very important to note that these HTML versions contain the height and width of the diagram and these are queried by a conversion of your document to HTML format in order to compute the aspect ratio. Therefore they need to be available with your other source files (typically in an `images` directory). So in a very real sense these files become part of your source.

- You may want to produce SVG versions of your diagrams for conversion to EPUB, and PNG versions for conversion to the EPUB precursor for Kindle format.
- PDF versions produced by the `pretext` script will not include the RPC extensions. So “rotatable” 3D images rendered by the proprietary viewer, Adobe Acrobat, are not created, consistent with our open source philosophy.
- Colors in Asymptote can be hard-coded using `rgb` syntax. Colors can also be defined at the top of an Asymptote file, to be referred to later. You may wish to produce PDF in both color (electronic) and black and white (print on demand) formats, and you probably do not want to maintain parallel source for both versions. Rather than writing (for example) `pen p=rgb(0,0,.7);` in your Asymptote code, you can write `pen p=curvepen1;`. Then, in the `<docinfo>` section of your document, you can add an `<asymptote-preamble>` and include the line `pen curvepen1=rgb(0,0,.7);`. Once you are ready to produce your black and white version, you need only change the definition of `curvepen1` in your `<asymptote-preamble>`.

One note of caution: if your preamble includes Asymptote code that only works once certain libraries are loaded, you must include lines to import those libraries in your preamble. For example, to define a `material` you must first have the line `import three;`.

Asymptote may be run as a program installed locally, but the project also has an on-demand online server written by Supakorn Jamie Rassameemasuang. By default, the `pretext/pretext` script ([Chapter 47](#)) will interface automatically with this server to create your diagrams. Furthermore, Asymptote provides a very useful [web application](#) written by Pedram Emami. This is a great place to learn, experiment, and iterate as you become more skilled at building high-quality graphics to illustrate the concepts in your document.

Best Practice 4.14.1 Build 3-D Asymptote Figures. If your project uses geometric or mathematical objects that are three-dimensional, invest some time in learning the Asymptote vector graphics language. The interactive diagrams for your HTML output produced by Asymptote, in WebGL format, are outstanding and will greatly enhance your project. (And the other static formats are similarly excellent.) The `pretext/pretext` script will create these diagrams, in the necessary formats, with no extra software by using an online server.

4.14.3.2 Images in $\LaTeX$ Syntax

There are a variety of $\LaTeX$ packages for authoring a diagram, plot, or graph. Examples include: TikZ, PGF, Xy-pic, and PSTricks. Generally, the `<latex-image>` tag allows you to incorporate this code into your source and PreTeXt realizes these descriptions as images in your output.

For $\LaTeX$ output the procedure is transparent—PreTeXt simply incorporates the preamble information and the image’s code in the correct places in the $\LaTeX$ output, scaled to fit whatever space is described on the `<image>` element. Then traditional $\LaTeX$ processing will do the right thing. For output to other non- $\LaTeX$ formats, such as HTML or EPUB, we need some help from the CLI to generate other formats. This tool will isolate the image’s code and bundle it up with the necessary preamble to make a complete single-purpose $\LaTeX$ file. Once converted by $\LaTeX$ to a PDF version, other tools can convert the image into other formats, such as SVG. In this way, you can use $\LaTeX$ packages for describing images, use mathematically-correct labels in $\LaTeX$ syntax, and use your own macros for consistency.

in notation, yet also employ the resulting images in more modern output formats. Note that as of 2020-07-24, limited testing indicates that PSTricks needs to be processed with the `xelatex` engine, and the `pstricks-add` package might also be necessary. Any updates, especially using `pdflatex` would be appreciated. Finally, processing with `xelatex` might be necessary if your labels use Unicode characters.

Much like the `<asyptote>` tag, the `<latex-image>` tag is used as a child of `<image>` and can be thought of as an alternative to the `@source` attribute of `<image>`. Adding a `@label` to the `<latex-image>` tag will ensure a stable file name for the graphics files that will be produced. The contents need to be a complete specification of the image. For example, a TikZ image will typically begin with `\begin{tikzpicture}`. Inside of your document's `<docinfo>` you will likely need to employ a `<latex-image-preamble>` element to hold necessary `\usepackage` commands and any global settings, such as the style for tick-marks and labels on axes of graphs. The source code in this next example is greatly abbreviated and mildly edited, see the source for the complete example.

```
<docinfo>
  <latex-image-preamble>
    \usepackage{tikz}
  </latex-image-preamble>
</docinfo>

<figure xml:id="rna-codons-figure">
  <caption>RNA Codons Table, by Florian Hollandt</caption>
  <image width="100%">
    <description><p>A table of the RNA codons.</p></description>
    <latex-image label="rna-codons-table">
      \begin{tikzpicture}
        \footnotesize
        \tikzstyle{every node}=[inner sep=1.7pt,anchor=center]
        % to_x and from_x styles denote bonds terminating
        % or starting in labeled nodes. x denotes the
        % number of letters in the node label.
        \tikzstyle{to_1}=[shorten >=5pt]
        \tikzstyle{to_1i}=[shorten >=6pt]
        \tikzstyle{to_2}=[shorten >=7pt]
        \tikzstyle{to_3}=[shorten >=8pt]
        ...
        \begin{scope}[scale=0.5] % Asparagine
          \draw[ultra thick,shorten >=2pt,shorten <=2pt] (90-2*5.625:8.2)
            arc(90-2*5.625:90-4*5.625:8.2);
          \path (90-3.5*3.625-3:13.3) node (zero) {};
          \draw[to_2] (zero.center) -- ++(30:1) node (C0) {}
            -- ++(330:1) node [anchor=base] {O$^{\mbox{-}}$};
          \draw[to_1] (C0.center) -- ++(90:1) node (Od) {O};
          \draw[to_1i] (C0.30) -- ++(90:1);
          \draw[to_3] (zero.center) -- ++(150:1) node {NH$_{\mbox{3}}^{\mbox{+}}$};
          \draw[to_2] (zero.center) -- ++(270:1) node (Cb){}
            -- ++(330:1) node (Cc) {}
            -- ++(30:1) node (Cd) {NH$_{\mbox{2}}$};
          \draw[to_1i] (Cc.center) -- ++(270:1) node (O) {};
          \draw[to_1] (Cc.210) -- (O.150);
          \path (O.center) node {O};
        \end{scope}
        ...
        \node at (90-55*5.625:4.5) {C};
        \node at (90-58*5.625:4.5) {S};
        \node at (90-61*5.625:4.5) {L};
        \node at (90-63*5.625:4.5) {F};
      \end{tikzpicture}
    </latex-image>
  </image>
</figure>
```

```

\end{tikzpicture}
</latex-image>
</image>
</figure>

```

This will result in:

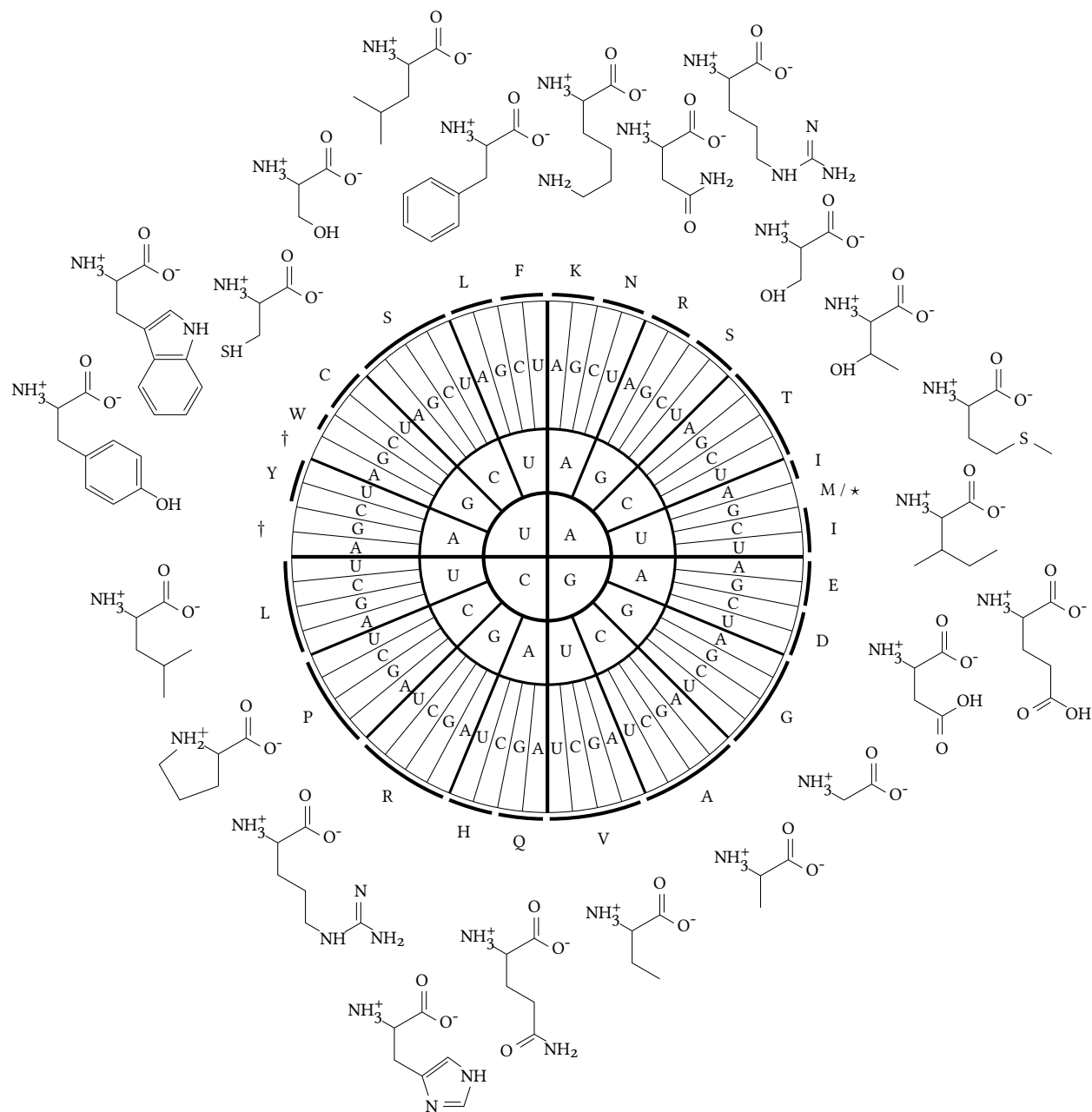


Figure 4.14.2 RNA Codons Table, by Florian Hollandt, from [TikZsample.net](https://www.tikzsample.net)

4.14.3.3 Scaling TikZ Images

Images authored in [TikZ](https://www.tikzsample.net) are the most popular. Text (nodes) in a TikZ picture are at whatever the current $\text{\LaTeX}$ font size is. The other parts of the picture (lines, circles, rays, etc.; the “line art”) can be scaled as part of an overall scale factor. The point being, the scale factor will not scale the text simultaneously. It is not unlike a map application on your mobile device. The name of a road is too small to read, so you zoom in on the map, making the street bigger,

but the name stays in the same font size and is still unreadable. This means some manual labor is involved when you place a TikZ picture into a PreTeXt document.

For many authors, the goal is to have the text in their TikZ picture have the same size as the surrounding text, both in a PDF and in HTML. We now explain how to accomplish this consistently.

Preparation. Well *before* designing many TikZ images, answer the following questions.

1. For your $\text{\LaTeX}$ output, what will the overall font size be?
2. For your $\text{\LaTeX}$ output, what will the width of the text block be? Note that this will normally be computed by PreTeXt, dependent on your chosen font size. A larger font will mean a greater width. You can generate the $\text{\LaTeX}$ source file and look early in the preamble to see what width is being used. It is also possible that you may be setting this with yourself (Section 30.6). The ratio of line width to font size is always 34 : 1.

TikZ in $\text{\LaTeX}$. Every image in PreTeXt may be constrained by width and/or margins, or may be restricted to a panel of a <sidebyside> with a certain width. So the TikZ code you author will create an image that is then scaled by PreTeXt to fit the constraints (much as any other image is scaled). Except this is done in a way that scales both the font and the line art. Your main goal is to have this scaling use a scale factor of 1.0. Which, of course, sounds like a waste of effort, but it is critical for how the image behaves in HTML (next).

To accomplish this unit scaling, follow this procedure for each TikZ picture.

1. Determine the width of the TikZ picture itself, in physical units of length. Typically, the lengths used for larger portions are described in centimeters. But note that an overall scale factor is sometimes applied for convenience (or as a result of poor planning!). Also, the default unit length (centimeters) can be changed. Note also, that text may “push out” to the right and left, defining the boundaries on the sides, and these lengths can be hard to compute or predict.
2. Recall the width of your text (above). Recognize that list items will be indented (reducing width), and perhaps there are multiple indents if a list has multiple levels.
3. Now you want the width of your picture as a percentage of the overall available width. By default, your overall width will be points, and your picture width will be in centimeters. You may be familiar with a “big point” (or “desktop publishing point”) which is 72 points to the inch. $\text{\TeX}$ however uses 72.27 points to the inch, which makes a $\text{\TeX}$ point equal to 0.03514598 centimeters. Convert to whatever common unit makes sense to you, since it is the dimension-less ratio you are after.
4. Use this percentage as the @width attribute on the <image> (with a percent sign).

Now produce a PDF and you will find that the font in the surrounding text, and the font in image, will match identically. I like to check this carefully by zomming in on the PDF and using an on-screen pixel ruler to check the heights of identical letters. [KRuler](#) is one such example for Linux, suggestions for other operating systems are welcome.

Note that in practice you will envision your picture as large or small, and you will begin with some overall physical width in mind, relative to the line width.

TikZ in HTML. For HTML output, the goal is to not edit your source. In other words to not change the @width attributes that have been so carefully computed and to not edit the TikZ code. But you will want to maintain fidelity with the surrounding font.

HTML output is designed to behave very similarly (not identically) to how $\text{\LaTeX}$ output behaves. In other words, the ratio of line width to font size is 34 : 1. In this way, line length and font size are such that a long paragraph will usually have an identical number of lines in $\text{\LaTeX}$ (at any font size) and in HTML.

Our tools produce Scalable Vector Graphics (svg) versions of TikZ pictures for use in HTML output. Being scalable means a reader can zoom in without any pixelation of the images. This is helpful for those with low vision, or if some fine point of a picture needs to be examined closely. It also means an SVG can be scaled by any factor when placed in PreTeXt HTML. However, the work done for a unit scaling for $\text{\LaTeX}$ output will continue to provide the correct scaling for HTML! (Provided the text width used for the PDF production is the one automatically computed from the font size via the 34 : 1 ratio.)

Example 4.14.3 Case Study: Scaling a TikZ picture. The PreTeXt source below describes a simple TikZ picture, and is followed by the picture itself. The rectangle is 8 centimeters wide. The Guide is produced as a PDF with 10 point text and a text width of 6.5 inches. (This is too wide for comfortable reading, and contrary to our recommendations. See [Best Practice 30.6.1](#).) Normally, a choice of 10 point text would result in a width of 340 point, or about 4.7 inches.

So we compute the fraction of the available width required, as a percentage:

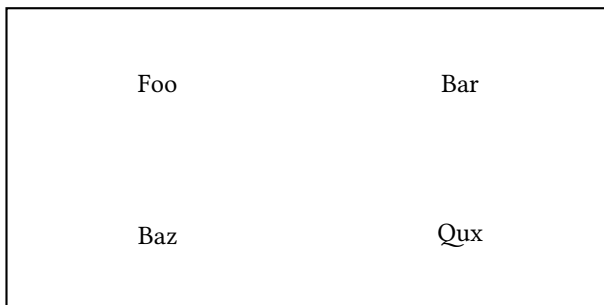
$$\frac{8 \text{ cm}}{6.5 \text{ in}} = \frac{8 \text{ cm} \left(\frac{1 \text{ in}}{2.54 \text{ cm}} \right)}{6.5 \text{ in}} = 0.4846 = 48.46\%$$

and we use that as the width of the image.

```
<image width="48.46%">
  <latex-image label="scaling-tikz">
    \begin{tikzpicture}
      % 1 cm is default unit of length
      % a rectangle: 8 cm wide, 6 cm tall
      \draw[draw=black, thick] (4,2) rectangle (-4,-2);

      \node at (-2, 1) {Foo};
      \node at ( 2, 1) {Bar};
      \node at (-2, -1) {Baz};
      \node at ( 2, -1) {Qux};
    \end{tikzpicture}
  </latex-image>
</image>
```

Some characters for comparison: Foo Bar Baz Qux



In the PDF version, the text matches between the image and the surrounding text almost identically. We could slide the image right and left by adjusting the margins (the default is to be centered). But if we want the image bigger and smaller, we need to adjust the TikZ code and recompute the @width attribute.

Now for HTML we need to produce an SVG version that is a close match. The HTML version is a close match for L^AT_EX built with a computed text width (for any font size). We do not want to change the percentage of the width devoted to the TikZ picture, and we do not want to change the TikZ code itself. If we had not chosen a different text width (the 6.5 inches, versus a computed 340 point), then we could generate the svg by supplying the same publication file, so as to use the same font size. However, our text width is 38% larger in the L^AT_EX version,

$$\frac{6.5 \text{ in}}{340 \text{ pt}} = \frac{6.5 \text{ in} \left(\frac{72.27 \text{ pt}}{1 \text{ in}} \right)}{340 \text{ pt}} = 1.3816$$

The font size needs to increase by a similar percentage,

$$10 \text{ pt} \times 1.3816 = 13.816 \text{ pt} \approx 14 \text{ pt}$$

So we generate the svg image with a *different* publication file, giving a font size of 14 point. The HTML font in the text may be very different from the L^AT_EX font used in the TikZ picture, but their *sizes* are nearly identical. Note that

our use of $\text{\LaTeX}$ only supports 8 different font sizes, so it was fortuitous in this example that the 38% increase was so close to the supported 14 point font size. Note also, that since we used a different text width for the PDF, the resulting 40% increase in the font size for the SVG could play havoc with text that has been placed carefully not to overlap other components of the picture. ▲

There are myriad ways to scale and transform a TikZ picture. You might choose to intentionally use a smaller font size than the surrounding text, as in [Figure 4.14.2](#). Or, fidelity with the surrounding text might not be important to you. Or you might prefer that images perform better in HTML. But hopefully the above discussion and example provide enough insight into how the various constructions behave. The important points are:

- TikZ uses physical units for the overall width of a picture, and nodes have text using the ambient font size of the PreTeXt $\text{\LaTeX}$ file (unless prescribed otherwise).
- PreTeXt scales a TikZ picture uniformly (text *and* line art) to fit into constraints given in the source.
- The SVG version of a TikZ picture is also uniformly scalable and at the same width as the original will have text of the correct font size. However, when used in HTML output, it is scaled on the assumption that the ratio of the line width to the font size is 34:1. This is the default width computed by PreTeXt for all supported font sizes. Changes in this ratio for PDF production requires an equivalent change in font size during SVG construction, via the publication file.

4.14.3.4 Images in Sage Syntax

Sometimes the necessary computations for an image are not part of the capabilities of whatever system is actually realizing the image. We have good support for Sage in other parts of your document, and Sage has an extremely wide variety of computational capabilities, in addition to letting you program your own computations in Python syntax with the full support of the Sage library. Rather than translating Sage output as input to some other graphics program, we simply capture the graphics output from Sage. So if your graphics are derived from non-standard, or intensive, computation this might be your best avenue.

Use the `<sageplot>` element, in a manner entirely similar to the `<asymptote>` element and the `<latex-image>` element, as a child of `<image>`, and containing the necessary Sage code to construct the image. There is one very important twist. The last line of your Sage code *must* return a Sage Graphics object. The pretext/pretext script ([Chapter 47](#)) and PreTeXt-CLI ([Section 5.2](#)) will isolate this last line, use it as the right-hand side of an assignment statement, and the Sage `.save()` method will automatically be called to generate the image in a file. Like `<asymptote>` and `<latex-image>`, putting a `@label` on the `<sageplot>` tag will determine the name of this file. Note that there are four different file types, depending on if the graphic is 2D or 3D, and the output format of the conversion.

The `@variant` attribute of the `<sageplot>` element may be 2d or 3d, since PreTeXt is not capable of analyzing your Sage code. The default value is 3d so can be skipped for 2D plots. For technical reasons, it is also necessary to specify the aspect ratio of a graphic for the 3D case using the `@aspect` attribute. The value can be a positive real number (decimal) or a ratio of two positive integers separated by a colon. The default is a square (1.0 or 1:1).

Table 4.14.4 File formats for `sageplot` images

	2D	3D
$\text{\LaTeX}$	PDF	PNG
HTML	SVG	HTML (for <code>iframe</code>)
EPUB	SVG	PNG
Kindle	PNG	PNG

Note that the PNG images in the 3D case are not very good. This needs help on the Sage side. And since 3D images in HTML output are inserted via an HTML `iframe` they can misbehave if you do not get the aspect ratio close to right. On the plus side, the 3D HTML images may be manipulated interactively with keyboard arrow keys, a mouse scroll wheel, and by dragging with a mouse using both a left and a right mouse press. Note that EPUB output used to make a Kindle book needs to use PNG images for both the 2D and 3D cases.

Pay very careful attention to the requirement that the last line of your code evaluates to be a graphics object. In particular, while `show()` might appear to do the right thing during testing, it evaluates to Python's `None` object and that is just what you will get for your image. The example below illustrates creating two graphics objects and

combining them into an expression on the last line that evaluates to the graphics object that will be created in the desired graphics files.

```
<figure>
  <caption>Negative multiple of a curve</caption>
  <image width="65%">
    <description><p>Plot of  $x^4 - 1$  and its negative.</p></description>
    <sageplot label="negative-curve">
      f(x) = x^4 - 1
      g(x) = -x^4 + 1
      up = plot(f, (x, -1.5, 1.5), color='blue', thickness=2)
      down = plot(g, (x, -1.5, 1.5), color='red', thickness=2)
      up + down
    </sageplot>
  </image>
</figure>
```

This will result in:

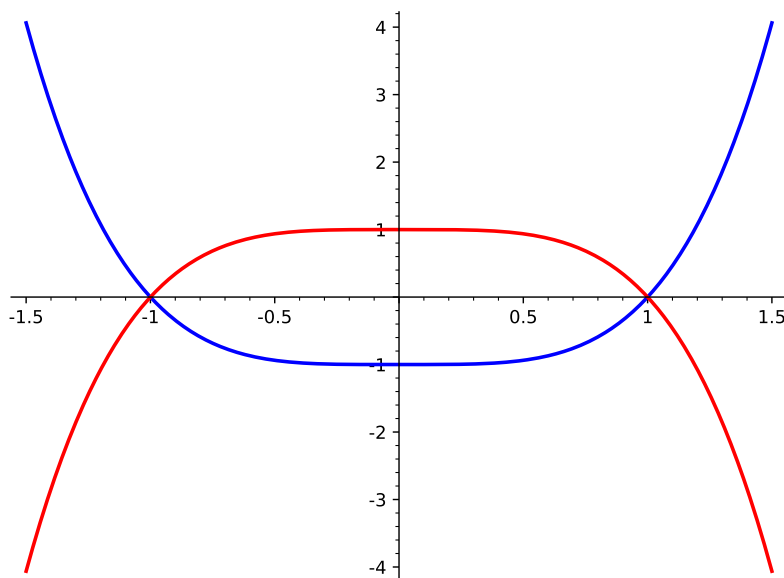


Figure 4.14.5 Negative multiple of a curve

Note the necessity of using the pretext script ([Chapter 47](#)) to independently invoke Sage, no matter what sort of output is being created for your document.

4.14.3.5 Mermaid

[Mermaid](#) is a Markdown-inspired tool for authoring various kinds of diagrams. One kind of diagram - a git commit visualization - is shown below. For a full listing of diagram types, see the [Mermaid Documentation](#). The [Mermaid live editor](#) is a great tool for testing the syntax of your mermaid diagrams.

You can specify global settings for diagrams via an entry in the publication files, see [Subsection 44.1.7](#). The available options are the same as the ones available in the config frontmatter section of a diagram. Note that the neutral theme will produce grayscale images, which could be useful for controlling the cost of a print-on-demand version.

To author a Mermaid diagram, use a `<image>` that contains a `<mermaid>` element. A `@label` on the `<mermaid>` element will be used to construct a filename—you will want to utilize this feature.

```
<figure xml:id="figure-mermaid-git">
  <caption>Mermaid Git Diagram</caption>
```

```

<image>
  <mermaid label="mermaid-git-image">
  ---
  title: Example Git diagram
  config:
    theme: default
    layout dagre
  ---
  gitGraph
    commit
    commit
    branch develop
    checkout develop
    commit
    commit
    checkout main
    merge develop
    commit
    commit
  </mermaid>
</image>
</figure>

```

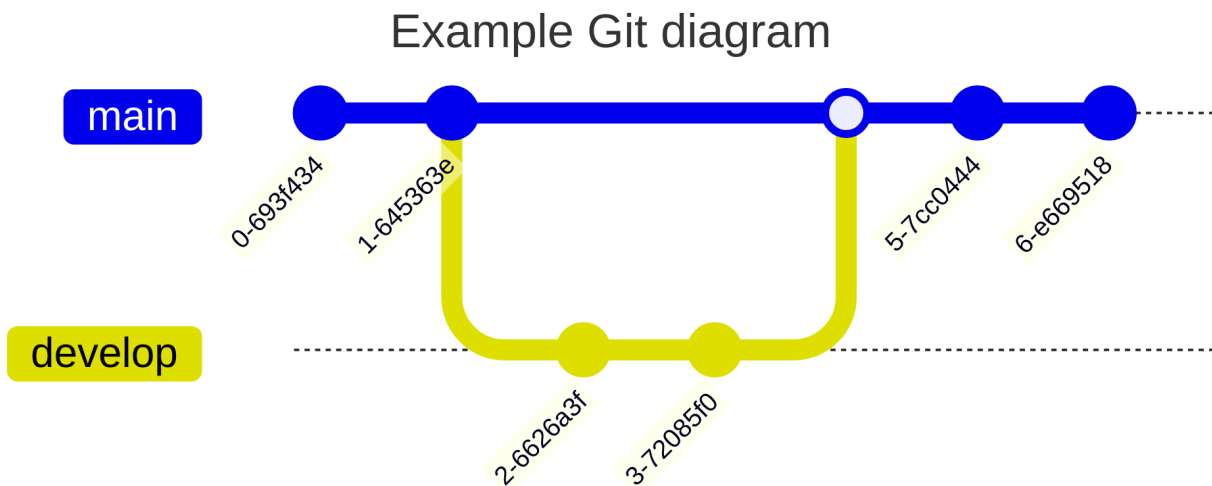


Figure 4.14.6 Mermaid Git Diagram

Mermaid has two layout engines—the default and `elk`. The ELK engine often does a better job with complex diagrams. You can specify it as a default by setting `@common/mermaid/@layout-engine` to `"elk"`, or by using standard Mermaid config frontmatter as shown in [Figure 4.14.7](#) below.

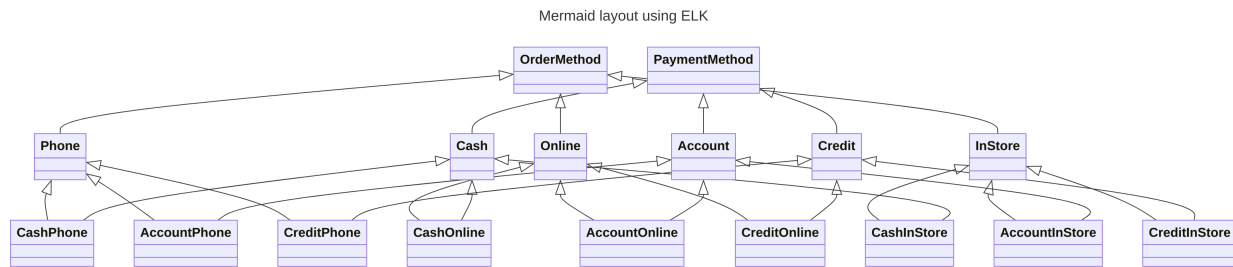


Figure 4.14.7 Mermaid Class Diagram using ELK layout engine

4.14.4 Image Formats

Best Practice 4.14.8 Preferred Image Formats. The best formats for images, in order, are:

SVG	Vector graphics format ideal for HTML output formats. Scalable and compact. Converts to other formats, such as PDF.
PDF	Vector graphics format ideal for print and PDF output formats. Good tools exist to convert back-and-forth between SVG and PDF.
PNG	Lossless and compressible format for raster images. May be used for both HTML and PDF outputs.
JPEG	Compressed lossy format which works well for photographs. May be used for both HTML and PDF outputs. PNG should be preferred when there is a choice, except in the case of a photograph. Converting between these formats is unlikely to be an improvement.

4.14.5 Image Archives

As an instructor, you might want to recycle images from a text for a classroom presentation, a project handout, or an examination question. As an author, you can elect to make images files available through links in the HTML version, and it is easy and flexible to produce those links automatically.

First, it is your responsibility to manufacture the files. For making different formats, the pretext script can sometimes help ([Chapter 47](#)). The Image Magick convert command is a quick way to make raster images in different formats, while the pdf2svg executable is good for converting vector graphics PDFs into SVGs (although now the pretext script uses the pyMuPDF library for these tasks instead). Also, to make this easy to specify, different versions of the same image must have identical paths and names, other than the suffixes. Finally, the case and spelling of the suffix in your PreTeXt source must match the filename (e.g. jpg versus JPEG). OK, those are the ground rules.

For links for a single image, add the @archive attribute to the <image> element, such as

```
<image ... archive="pdf svg">
```

to get two links for a single image.

To have every single image receive an identical collection of links, in docinfo/images place an <archive> element whose content is the space-separated list of suffixes/formats.

```
<archive>png JPEG tex ods</archive>
```

will provide four links on every image, including a link to an OpenDocument spreadsheet.

For a collection of images that is contained within some portion of your document, you can place an @xml:id on the enclosing element and then in docinfo/images place

```
<archive from="the-xml-id-on-the-portion">svg png</archive>
```

to get two links on every image *only* in that portion (chapter, subsection, side-by-side, etc.). The @from attribute is meant to suggest the root of a subtree of your hierarchical document. If you use this, then *do not* use the global form that does not have @from.

You may accumulate several of the above semi-global semi-local forms in succession. An image will receive links according to the last `<archive>` whose `@from` subtree contains the image. So the strategy is to place general, large subtree, specifications early, and use refined, smaller subtree specifications later. For example,

```
<archive from="the-xml:id-on-a-chapter">svg png</archive>
<archive from="the-xml:id-on-the-introduction">jpeg</archive>
<archive from="the-xml:id-on-a-section-within" />
```

will put two links on every image of a chapter, but just one link on images in the introduction, and no links at all on every image within one specific section. Again, do not mix with the global form. You can use the root document node (e.g. `<book>`) for `@from` to obtain a global treatment, but it is unnecessary (and inefficient) to provide empty content for the root node as first in the list—the same effect is the default behavior.

Notice that this facility does not restrict you to providing files of the same image, or even images at all. You could choose to make data files available for each data plot you provide, as spreadsheets, or text files, or whatever you have, or whatever you think your readers need.

Finally, “archive” may be a bit of a misnomer, since there is no historical aspect to any of this. Maybe “repository” would be more accurate. Though for a history textbook, it might be a perfect name.

4.14.6 Copies of Images

Sometimes you want to use the same image more than once. Putting it in a `<figure>` and then making a cross-reference (`<xref>`) can work well in HTML output since it will be available as a `knowl`. However in a static format (PDF, print) the reader will need to chase the cross-reference.

For a raster image, you can just point to the file again with the `@source` attribute. You are free to wrap it in a figure and thus change the caption. It will get a new number as a new figure, and you will need to assign a new unique `@xml:id` attribute. Maybe appending `-copy-2`, or similar, to the `@xml:id` will be helpful.

If you have a figure generated from source code (such as in TikZ) you really do not want to edit and maintain two copies that may not stay in sync. Instead, you can place the code into a file and `xinclude` it twice. Study [Section 4.39](#) carefully, and note that this is an excellent place to take advantage of setting the `@parse` attribute to text (no need to escape problematic XML characters). Notice that when you generate images, you will have two image files with identical contents, but different names. This is no place for false optimization. Bits are cheap and your time is valuable. It is far more important to only maintain a single copy of the source, than to be caught up with the “waste” of having two copies of the same file (and which are managed for you). We demonstrate this with the sample book, since it is all set up with the `xinclude` mechanism. See the two plots of the 8-th roots of unity in the complex numbers section of the chapter on cyclic groups.

4.15 Consoles

4.15.1 Content and Placement of Consoles

A `<console>` is a transcript of an interactive session in a terminal or console at a command-line. It is a sequence of the following elements, in this order, possibly repeated many times as a group: `<input>`, and `<output>`. The `<output>` is optional. The content of these two elements is treated as verbatim text (see [Section 3.16](#)), subject to all the exceptions for exceptional characters (see [Section 3.14](#)). A `@prompt` attribute on the `<input>` can be supplied to provide a system prompt distinct from the actual commands. The default prompt is a dollar sign followed by a space. If it is more convenient `@prompt` may be supplied on the `<console>`, to be used in each enclosed `<input>`. If you do not want any prompts at all, just use an empty value for `@prompt`. A `@continuation` attribute may also be supplied at the `<input>` or `<console>`. If present, it will apply to the start of each input line following the first.

A `<console>` may be wrapped in a `<listing>`, see [Section 4.20](#). This will behave similar to a `<figure>`, with the `<caption>` displayed below, and a number assigned. So, in particular, if your program is important enough to cross-reference, it is an enclosing `<listing>` that serves as the target.

4.16 Programs and Program Fragments

4.16.1 Content and Placement of Programs

4.16.1.1 Content and Placement

A `<program>` will be treated as verbatim text (see [Section 3.16](#)), subject to all the exceptions for exceptional characters (see [Section 3.14](#)). Indentation will be preserved, though an equal amount of leading whitespace will be stripped from every line, so as to keep the code shifted left as far as possible. So you can indent your code consistently along with your XML indentation. For this reason it is best to indent with spaces, and not tabs. A mix will almost surely end badly, and in some programming languages tabs are discouraged (e.g. Python).

The text contents of a program can be surrounded with `<code>`, but that is only necessary if you have other elements inside of the `<program>` like `<preamble>` or `<tests>` used by an interactive program (see [Subsection 4.16.3](#) below for details).

A `<program>` or `<console>` may be wrapped in a `<listing>`, see [Section 4.20](#). This will behave similar to a `<figure>`, with the `<title>` displayed above, and a number assigned. So, in particular, if your program or console is important enough to cross-reference, it is an enclosing `<listing>` that serves as the target.

The `@language` attribute may be used to get some degree of language-specific syntax highlighting and/or interactive behavior. The current known languages are:

- | | | |
|--------------|------------|----------|
| • basic | • sas | • matlab |
| • c | • sage | • octave |
| • cpp | • splus | • ml |
| • go | • vbasic | • ocaml |
| • java | • vbscript | • fsharp |
| • javascript | • clojure | • css |
| • lua | • lisp | • latex |
| • pascal | • clisp | • html |
| • perl | • elisp | • tex |
| • python | • scheme | • xml |
| • python3 | • racket | • xslt |
| • r | • sql | |
| • s | • llvm | |

In some output formats, notably HTML, the syntax highlighter can add lines or highlight individual lines of code. Highlighting lines is not supported for $\text{\LaTeX}$ output. To display line numbers, set the `@line-numbers` attribute to yes. To highlight particular lines, set `@highlight-lines` to a comma-separated list consisting of individual lines and/or ranges indicated with dashes. Some examples are: 5, 2, 5, 2, 5-8, 10-15, 15.

Listing 4.16.1 A static Java program with highlighted lines

```

1  import javax.swing.JFrame;    //Importing class JFrame
2  import javax.swing.JLabel;    //Importing class JLabel
3  public class HelloWorld {
4      public static void main(String[] args) {
5          JFrame frame = new JFrame();           //Creating frame
6          frame.setTitle("Hi!");                 //Setting title frame
7          frame.add(new JLabel("Hello, world!")); //Adding text to frame
8          frame.pack();                          //Setting size to smallest
9          frame.setLocationRelativeTo(null);     //Centering frame
10         frame.setVisible(true);               //Showing frame
11     }
12 }

```

For interactive versions hosted on Runestone servers, the `@label` attribute is critical, just like for interactive exercises. So be certain to read [Best Practice 4.12.1](#).

Finally, when authoring programs, it may be helpful to keep the source code as plain text in separate files. This can avoid the need to manually escape characters that have special meaning in XML and can facilitate testing and updating the code samples. See [Section 4.39](#) for how to do so.

4.16.1.2 Default Attributes for Programs

Book-level default values for some program attributes can be set in `docinfo/programs`. If a value for an attribute is set in that location, it will be used for any `<program>` that lacks the corresponding attribute.

The following attributes can be given default values - some only are applicable to interactive programs (see [Subsection 4.16.2](#) and [Subsection 4.16.3](#)): `@compiler-args`, `@linker-args`, `@interpreter-args`, `@download`, `@language`, `@linenumbers`, and `@timelimit`.

4.16.1.3 Program Fragments

Fragments of a program can be added inline with text using the `<pf>`. A program fragment will be displayed similar to a `<c>`, but will be syntax highlighted to match a `<program>`. The language to interpret the fragment as can be specified with `@language` or by relying on a document level default for programs (see [Subsubsection 4.16.1.2](#)).

4.16.2 Interactive Programs, CodeLens

CodeLens is an interactive version of a computer program, which can be visualized by stepping through the code one statement at a time, watching output, variables, and other data structures change. So it is similar to a debugger, except the reader does not set breakpoints or modify program data on-the-fly. This is possible automatically for several different languages when your HTML is hosted on a Runestone server ([Chapter 32](#)). This may also be accomplished “in browser” when hosted on any old generic web server. The catch is that for a generic server a publisher must generate **trace data** in advance, typically with the `PreTeXt-CLI` ([Section 5.2](#)). Place the `<interactive>` attribute on a `<program>` element with the value `codeLens` to elect this behavior (`no` is the default value). Also, be sure to specify a language from the supported languages: Python, Java, C, and C++. Consult [Table 4.16.5](#) below for a summary of various combinations. When an output format does not support an interactive CodeLens instance, the fallback is a static program listing. The `<program>` should have an `@xml:id` that will be used as a unique identifier for the generated trace file needed by the codeLens.

Sometimes, there is uninteresting setup to get through before the portion of the program the author wants to illustrate. The author can use `@starting-step` to specify the instruction number that the interactive should start at. Code before that will be fast-forwarded through and execution will begin at the indicted step. Note that `@starting-step` is a “Step” number in the execution sequence, not a line number in the source code. You may have to run the CodeLens once to find the right step to start at.

An author may add checkpoints to a CodeLens that will ask the user a question when execution reaches a particular line of the program. To do so, add one or more `<checkpoint>`s. The `<checkpoint>` should have a `@line` specifying what line to ask it on and either an `@answer` or `@answer-variable` attribute to indicate the correct answer (see the next paragraph for more on these). The `<checkpoint>` must have a child `<prompt>` with the question to

ask the reader and optionally may have a child `<feedback>` element with Simple markup can be used to format the contents of both the prompt and the feedback.

To determine if the user's answer is correct, the `@answer` or `@answer-variable` will be used. `@answer` will be compared directly to the user response via simple string comparison. An `@answer-variable` will be compared to the current value of a variable in the program. This is especially useful for checkpoints inside a loop or function that will be asked multiple times with a different correct answer each time. To specify the name of a variable, use `globals.x` (for globally scoped variables) or `current_frame.x` (for locally scoped ones). More advanced names (array elements, object members, etc...) can be decoded by examining the trace file generated for the code lens. Find the line that will contain the checkpoint, and examine the `globals` field or the `encoded_locals` field of the last stack frame in that line.

For examples, see [the CodeLens section of the Sample Book on Runestone](#).

4.16.3 Interactive Programs, ActiveCode

4.16.3.1 Authoring ActiveCode Programs

ActiveCode is an interactive environment where a reader may work on code through repeated edit-compile-test cycles. Code can be provided by an author as a complete program to be modified, a partial program to be completed, or nothing at all. One good example is that maybe header files, import statement, and similar are provided, and a skeleton of a main entry-point procedure is also provided. Then a reader can concentrate on the more conceptual parts of the programming. Some languages will be executable “in browser” on any old generic web server, while others must be on a Runestone server ([Chapter 32](#)) where a [Jobe Server](#) is running to support the execution.

Place the `@interactive` attribute on a `<program>` element with the value `activecode` to elect this behavior (no is the default value). Also, be sure to specify a language from the supported languages. Consult [Table 4.16.5](#) below for a summary of various combinations. When an output format does not support an interactive ActiveCode instance, the fallback is a static program listing.

Warning 4.16.2 Labels and XML IDs. Properly placing labels and `xml:id`'s can get tricky for programs. Understanding the purpose of these two different identifiers is important:

- `@xml:id`s are used to link to a program from other parts of the book source code. Any program that you want to use as the target of an `@add-files` or `@include` attribute in some other program must have a unique (within the book) `@xml:id`.
- `@labels` are used to provide an identifier for items in the Runestone database. They too should be unique within a book. But a program that is a direct child of an exercise-like element is considered to be a **programming exercise**. Programming exercises do not need a label as they are included in the database as part of the exercise.

So best practices are:

- Every program should have a `@label` attribute unless it is the direct child of an exercise like element. In that case, the label goes on the exercise and the program does not get a label. It is OK to leave a label off of a non-interactive program no other programs try to link to it (e.g. `@addfiles`).
- Any program you want to link to from other places should get an `@xml:id`. It always goes on the program itself.

It is perfectly fine to use the same value for both `@label` and `@xml:id`, although `xml:ids` are more restrictive in terms of what characters are allowed.

Inside the `<program>` of an ActiveCode, you can use the following elements:

preamble	Code that will be part of the program but not available for editing by the user and precedes the contents of <code><code></code> . This code will be visible unless the <code>@visible</code> is set to <code>no</code> . Note that the contents of this element will be combined with the <code><code></code> before the indentation is adjusted. So any code in this block must be indented to the same baseline as the contents of <code><code></code> .
code	The body of the program that will be editable by the user.

postamble	Code that will be part of the program but not available for editing by the user and follows the contents of <code><code></code> . This code will be visible unless the <code>@visible</code> is set to no. Note that the contents of this element will be combined with the <code><code></code> before the indentation is adjusted. So any code in this block must be indented to the same baseline as the contents of <code><code></code> .
tests	Tests may take the form of either structured input-output tests (see Subsubsection 4.16.3.2) or as a block of code that will be added after the <code><code></code> (as well as any <code><postamble></code>). The assumption is that code in <code><tests></code> is doing unit testing that the user should not see, so this code will be hidden unless the <code>@visible</code> is set to yes. This code is processed by PreTeXt separately from the <code><code></code> , so indentation does not need to be consistent between it <code><code></code> and <code><tests></code> . Some languages have support for unit testing frameworks. If you use one of them, the results of the tests will be parsed and displayed to the user as a visual results table. The following languages have support for unit testing frameworks: • python supports <code>unittest</code> • java supports <code>junit</code> • c and cpp support <code>doctest</code> (which tends to build and run significantly faster) and <code>catch</code>
stdin	If this element is provided, a text box will be created for the user to enter text that will be fed to the program via the <code>stdin</code> stream. If the <code><stdin></code> has text content, that will be provided to the user as a default input. Multiple lines of input are permissible. The text will be sanitized in the same way that the <code><program></code> itself is.

Here is a sample of a program with preamble and postamble. For more examples, see [the Activecode section of the Sample Book](#)

Listing 4.16.3 A Python program with preamble/postamble

```
def add(a, b):
    # TODO - complete the add function

# Use the function
result = add(2, 3)
if result == 5:
    print("Test passed")
else:
    print("Test failed")
```

There are a wealth of attributes that can be used to control the behavior of ActiveCode. (These are in addition to the standard ones allowed for `<program>`. see [Section 4.16](#) for those.) Below is a list of the extra attributes available. Any attribute that lists other elements for inclusion will take a list of `@xml:ids` for those elements.

autorun	If set to yes the program will run on page load.
chatcodes	Set to yes to enable users to talk about this code snippet with others.
codelens	In languages that support Codelens (see Subsection 4.16.2), there will be a button that allows the user to run the program via Codelens. Set this to no to remove it. This may be desirable if you know the code uses features that are not supported by Codelens (e.g. turtle graphics or image).
compiler-args	(C/C++ only) A comma separated list of strings to be passed to the compiler. Ex: <code>-Wall, -std=c++17</code> . A book-level default can be specified - see Subsubsection 4.16.1.2 .
extra-compiler-args	(C/C++ only) A comma separated list of strings to be passed to the compiler in addition to the <code>@compiler-args</code> . Mostly of interest to add an argument to a particular program without overriding any book-level default arguments.

database	(SQL only) SQL based programs can make use of an SQLite database file. Use this attribute to specify the file to load as a string relative to the @external top-level directory.
filename	For server based programs. What name to use when the code from this file is written to the server. This is generally only required if using the @add-files attribute on some other program to make the contents of this program available (as a .h file for example).
download	Set to yes to enable users a download button that triggers downloading of the user code to a file. A book-level default can be specified - see Subsubsection 4.16.1.2 .
hidecode	Set to yes to initially hide the code. When code is hidden, the 'Run' button is also disabled. To run the code, a user either presses 'Show Code', then 'Run'. You might use this if you want to put an ActiveCode block in the page in order to include it in another ActiveCode block, but you don't need or want students to see it right away. Or to hide the code of an ActiveCode with @autorun where you initially want to focus on the output instead of the program.
include	A comma separated string identifying one or more other ActiveCodes that are to be prepended to this program when it is run. This allows you to write examples that build on each other without having to duplicate all the code in each successive block. For example, if you write a function definition in one ActiveCode, you can include it in a later ActiveCode block that has code to call the function without having to visibly include the definition.
add-files	(Server based code only) A comma separated string identifying one or more other programs or Datafiles by xml:id that are to be made available to this code. Each of those elements should have an @filename. For server based programs, the other elements will be copied into files based on their @filename attributes. This allows you to split code into multiple "files" and thus create more complex inter-connections than when using @include.
compile-also	(C/C++ only) A comma separated string identifying one or more other programs by xml:id that are also to be compiled when this program is compiled. They will be assumed to also be a part of the @add-files collection. Each of those programs should have an @filename.
interpreter-args	(Python2/3/Java/octave only. Only applies to Python that is run on the server. See Table 4.16.5 .) A comma separated list of strings to be passed to the compiler. Ex: -Xrs, -Xss8m, -Xmx200m. A book-level default can be specified - see Subsubsection 4.16.1.2 .
language	Same options as <program>. A book-level default can be specified - see Subsubsection 4.16.1.2 .
linker-args	(C/C++ only) A comma separated list of strings to be passed to the compiler. Ex: -lm, -g. A book-level default can be specified - see Subsubsection 4.16.1.2 .
timelimit	A maximum time allowed, in milliseconds, for a program to compile and run. If not provided, the default is 25000 (25 seconds). A book-level default can be specified - see Subsubsection 4.16.1.2 .

4.16.3.2 Tests for Interactive Programs

There are currently two ways to make automated tests for ActiveCode programs. The first way is to simply provide code in the <tests> element of the <program> as described in [Subsection 4.16.3](#). It will be assumed this code is unit tests and it will be run automatically with the user's submission.

Below is an example of unit tested Python. For more examples, see [the Coding Exercises section of the Sample Book](#)

Checkpoint 4.16.4 Coding Exercise, with Unit Tests. Fix the following code so that it always correctly adds two numbers. [Ed. Unit test support is experimental.]

```
def add(a,b):
    return 4
```

Answer. We're not really sure. But it would begin as follows:

```
#include <stdio.h>

int main(void)
```

The second way to automatically test code is to provide a set of structured input-output pair tests. For each test, the program will be run and fed the specified input. Then the program output will be compared against the test output to decide if the program handled the test correctly. This mechanism is useful for testing simple programs without functions and for languages that do not have a unit testing framework. A particular program can only have one of these two testing mechanisms.

IO tests are generally only available for programs that run on a Runestone server. (See [Table 4.16.5](#) for a list of which languages support IO tests.). To preview them, see [the Coding Exercises section of the Sample Book hosted on Runestone](#)

Structured IO tests are added as `<iotest>s` to the `<tests>` element. Each `<iotest>` element must have a single `<input>` and `<output>` element. The `<input>` element contains the input that will be fed to the program, and the `<match>` element contains the expected output of the program. Indentation for both will be normalized in the same way that programs are (so multiple-line sample output can be indented in your source document).

When evaluated, the program output and test `<output>` will have leading and trailing spaces removed. Other than that spacing, the two must match exactly to be considered a "passed" test.

4.16.4 Interactive Program Capabilities

This table lists which types of interactivity are available on various combinations of servers and programming languages. The entry "AC + CL" means that both ActiveCode and CodeLens instances are available, but the ActiveCode instance will have a CodeLens button enabled. Although `<tests>` can be used to append code to any program, only in situations listed as "UT" below will the test output be parsed and interpreted by the server

Note that python is generic Python with the standard libraries (version 3.x). On the other hand python3 is a Runestone server installation (only) with a number of additional popular Python packages available, such as numpy and pandas.

Table 4.16.5 Interactive Programs

Language	@language	Generic	Server Runestone
Python	python	AC + CL + UT	AC + CL + UT
Python 3	python3		AC + IO
Java	java	CL	AC + CL + IO + UT
Kotlin	kotlin		AC
C	c	CL	AC + CL + IO + UT
C++	cpp	CL	AC + CL + IO + UT
JavaScript	javascript	AC	AC
HTML	html	AC	AC
SQL	sql	AC	AC
Octave	octave		AC + IO

AC = ActiveCode, CL = CodeLens, UT = Unit Tests (parsed), IO = IO Tests

4.17 Data Files

In concert with interactive programs (see [Section 4.16](#)) you can define a file of data that may be employed by those programs. The necessary element is `<datafile>`. It requires a `@label`. To allow programs to link to the datafile, it will also need an `@xml:id` (which can be the same as the `@label`). A `@filename` is also required and is the name

the file is known by in an ActiveCode program. Do not try to impose any sort of directory structure on this name. Just a filename. In the case of a text file (see below), the `@editable` attribute is optional. The value `no` is the default, with `yes` as the other option. The attributes `@rows` and `@cols` are optional for text files, and default to 20 and 60 respectively. Finally, a non-editable text file (only) may have its contents hidden by setting the `@hide` attribute to `yes`, rather than the default value of `no`.

Where might you place a `<datafile>`? Lots of places are possible, such as in an `<example>` or a programming `<exercise>`, close to an ActiveCode `<program>`. So, in expository material or in activities for readers to work through. The purpose-built COMPUTATION-LIKE block, `<data>`, which will get you a heading, number, title, cross-reference target, etc. (see [List 4.2.2](#)), is an option if the file itself needs more prominence or dedicated explanation. Notice that this feature is very powerful, and thus requires a bit of machinery to support. If you just want to point your reader to a file (and leave them to work with it outside of your project), either globally or locally, the read about the `<dataurl>` element at [Section 4.17](#).

Text as Data Files. Inside of a `<datafile>` place an `<pre>` element. There are then two options: provide the contents of the text file right in your source PreTeXt file, as you might for other preformatted text, or supply a `@source` attribute whose value is the name of an external text file you provide. The former is appropriate for “toy” examples, while the latter may be used for “serious” files with many lines, or with long lines. Note that if you provide the file as the content of the `<pre>` element, it can be indented to match your source file indentation, and will undergo some manipulation, such as removing leading whitespace, and ensuring a final newline, but preserving any *relative* indentation. If provided via a `@source` attribute, there is no manipulation.

Such a text file may be declared editable by the reader, presumably to allow them to witness the resulting behavior of a some employing program. The `@rows` and `@cols` attributes describe the viewport into the file provided in the HTML output. Typically scrollbars will allow the reader to survey all of a large file. In static outputs, the first few lines are shown, given by the value of `@rows`, and lines are truncated according to the value of `@cols`.

Images as Data Files. Inside of a `<datafile>` place an `<image>` element with a `@source` attribute. As usual, this attribute should be the name of an external file you provide. Most common formats are supported, but it is important to use standard extensions, so the format can be discerned. Now this file may be explored programmatically by opening the file using the name provided in `@filename`.

Keep the size of the image small, say 300–400 pixels in each direction. You may also supply the usual layout controls, such as `@width`, and these will be consulted in the formation of output formats. Ideally, you should use a width that scales the image to look something like its “native” resolution, since part of an image-processing exercise may depend on this aspect of the input. HTML output uses a 600 pixel overall width, so a percentage can be computed based on this parameter.

Notes on Data Files. Some notes that apply to each type of data file.

- Note that the name of the data file in a `@source` attribute need not have any resemblance to the new name given to the file via the `@filename` attribute. In other words, the reader will never know (or care) what `@source` was.
- Whenever the `@source` attribute is used, there needs to be an advance step performed by the CLI [Section 5.2](#) or the pretext/pretext script ([Chapter 47](#)) to generate an auxillary file (yes, a third file!) to aid the transistion from an external file to a file that can be used by the reader in programs.
- For a program to use a data file, it must be able to find the appropriate file. This is accomplished by specifying an `@add-files` attribute on the `<program>` element that is a list (separated by commas or space) of the `xml:id`’s for files that are available for use in the program. For programs that run strictly in the browser as opposed to on a Runestone server (see [Subsection 4.16.4](#) for what languages run in the browser), a program may be able to open a file by name without specifying its `xml:id` in the `@add-files`. However, for all programs, best practice is to make sure that the `xml:id` of any datafile a program should have access to is correctly listed in that program’s `@add-files`
- In all cases, for an HTML build the contents of the data file live within an HTML page, as text for a text file, and as a base-64 encoding for an image file. Hence for a non-Runestone build, any employing program *must* be

on the same page, and an author should think ahead about the granularity of how a project might be chunked into pages ([Section 44.1](#)).

In a build for use on Runestone Academy ([Chapter 32](#)), the file will be in the Runestone database and usable throughout.

4.18 Figures

A `<figure>` is the most generic and flexible container for planar content. But be sure to read [Section 3.13](#) so you are aware of the other possibilities. A figure has a `<caption>`, which will typically render below the content (even if authored early as metadata) and serves to provide an extra description of the content. So it may be several sentences long. There is also a `<title>`, which is typically not rendered as part of the figure. Instead it is used for cross-references, or in a list of figures, to identify the figure. So it should be very short and might just be a phrase, such as “Life Cycle of a Salamander.”

An `<image>` is likely the most frequent content in a `<figure>`. But you may also place a `<video>`, `<audio>`, `<sidebyside>`, or `<sbsgroup>`. Once completely implemented, an `<interactive>` is another possibility. (See [Section 4.24](#) for more about the side-by-side construction.)

A special situation is when a `<figure>` is a panel of a `<sidebyside>`, which is itself inside a `<figure>`. Then the interior figure is **subnumbered**. For example, the exterior figure might be Figure 4.12, and if a panel of the `<sidebyside>` is the second interior figure it will be Figure 4.12(b). For example,

```
<figure>
  <caption>Salamanders at different life stages</caption>
  <sidebyside>
    <figure>
      <caption>Hatchling</caption>
      <image source="salamader-hatchling.jpg"/>
    </figure>
    <figure>
      <caption>Juvenile</caption>
      <image source="salamader-juvenile.jpg"/>
    </figure>
    <figure>
      <caption>Adult</caption>
      <image source="salamader-adult.jpg"/>
    </figure>
  </sidebyside>
</figure>
```

could result in the entire figure being Figure 4.12 and then the juvenile salamander photograph would be inside of Figure 4.12(b).

4.19 Tables and Tabulars

A `<table>` is a container that houses a `<tabular>`, which is the actual rows and columns of table entries.

Note that `<tabular>` may be constructed using the [L^AT_EX Complex Table Editor](#) tool online and then exported in PreTeXt syntax. This produces verbose PreTeXt syntax that is usually equivalent to much simpler PreTeXt syntax once you understand the borders and alignment considerations below.

4.19.1 Tables

A `<table>` is similar to other blocks in PreTeXt ([Section 4.2](#)) and is most similar to a `<figure>`. It will earn a number, which is likely to be a part of the text of a cross-reference pointing to the table. Rather than a `<caption>`, it will have a `<title>`. The main difference is that the principal content *must* be a `<tabular>`. Only.

4.19.2 Tabular

A `<tabular>` is the actual headers, rows, and columns of a table. As discussed above, a typical use is to place it inside a `<table>`, though it can be placed all by itself, typically in among a run of paragraphs.

Fundamentally a `<tabular>` is a sequence of `<row>` and each `<row>` is a sequence of `<cell>`, which could also be called “table entries.”

4.19.3 Table Cells

A given cell can span multiple columns, by providing the `@colspan` attribute with a value that is a positive number, the cell will extend to occupy additional columns.

4.19.4 Table Rows

A `<row>` of a table is a sequence of `<cell>` elements. Each row should occupy the same number of cells, when considering the `@colspan`, as discussed above.

To achieve *column* headers, you indicate that a *row* contains headers. Typically, the contents of every cell in this row will then be rendered in bold, or some other style. The `<row>` element accepts a `@header` attribute with possible values of `no` (the default), `yes`, or `vertical`. The latter is useful if space is at a premium (which always seems to be the case with tables), and the cells of a column are narrow and the header is long. Note that only the first (top) rows can be treated as column headers and these rows must be contiguous. If you think you need column headers mid-tabular, maybe you really have two tables?

4.19.5 Table Columns

Prior to all of the `<row>` within a `<tabular>`, there may be a sequence of empty `<col/>` elements. Having these is optional, but once there is one, then there needs to be as many as the number of columns of the table. These elements do not have any content that appears in the table, but are used to hold attributes that influence the borders or alignment of the cells within a column. These are described below.

So it should now be clear that, after much consideration, that we have chosen a “row first” approach to describing a table.

To encourage good style, we only support row headers as the first column. So this is a property of the entire `<tabular>`. So the attribute `@row-headers` on `<tabular>` can have values `no` and `yes`, with the former as default. Note that “major” and “minor” row headers should be accomplished in the first column by using indentation for the minor headers. Please make a feature request if you would find this useful.

4.19.6 Table Borders and Rules

You can view each cell of your table as having four borders. Or you can imagine rows and columns separated by horizontal or vertical rules. These additions to your table do not change the arrangement of information into rows and columns (a doubly-indexed data set), though you may think it makes the presentation clearer. But less is actually more.

Best Practice 4.19.1 Vertical Rules in Tables. One of the goals of PreTeXt is to gently guide authors towards good choices in the design of their documents, even if we do not claim to know it all ourselves. Take a close look at [Table 4.1.3](#). What’s missing? No vertical rules. Try living without them, you will not really miss them. If you think you need to divide a table into two halves, maybe you really need two tables (and then see the “side-by-side” capabilities, [Section 4.24](#)).

In the documentation for his excellent $\text{\LaTeX}$ package, [booktabs](#), Simon Fear gives two rules for what he calls “formal tables”: (1) Never, ever use vertical rules, and (2) Never use double rules. We have resisted the temptation to enforce the former and have provided an alternative to the second (three thicknesses). He refers to using tables for layout as creating “tableau.” If you are finicky about the look of your work, the first three pages of the documentation is recommended reading.

A given `<cell>` can have a border on its bottom edge, and on its right edge. This is accomplished with the `@bottom` and `@right` attributes. The possible values are `minor`, `medium`, and `major`, which control thickness. (Not

every conversion can produce three distinct thicknesses, so this should be considered a hint to the conversion.) A value of none is the default behavior when the attribute is not used, but can be given explicitly.

How to get a left border on the first cell of a row? The `<row>` element allows a `@left` attribute which will put a border on the left end of the row, which is also the left border of the first cell.

How to get a top border on a cell? Put a bottom border on the cell above it. But what if the cell is already in the top row and has no cell above it? The relevant `<col>` element allows a `@top` attribute which will place the necessary border on the top-row cell.

Borders and rules verge on presentation, so we are not concerned about which cell a border (or rule) belongs to. So, generally `@bottom` and `@right` can be used in many places, and the exceptional `@top` and `@bottom` maybe used to get the missing border $n + 1$ for a vertical or horizontal sequence of n cells.

The attributes described for cells may also be used on `<row>`, `<col>`, and `<tabular>`. For example a thick horizontal rule after two rows of headers could be accomplished with

```
<row header="yes">...</row>
<row header="yes" bottom="major">...</row>
```

We will not detail all the combinations that are possible, so experiment and you should be able to create any rational look (and some irrational ones).

4.19.7 Table Cell Alignment

The horizontal alignment of the contents of a `<cell>` can be influenced by the `@halign` attribute with values `left`, `right`, `center`, and for “paragraph cells,” `justify`. Similarly the `@valign` attribute will influence the vertical alignment through values `top`, `middle`, and `bottom`. Default alignments are `left` and `middle`.

To align the cells of an entire `<row>`, `<col>`, or `<tabular>` identically, place the relevant attribute on the relevant element. Note that these choices can be overridden by different values on individual constituents.

4.19.8 Multi-line Cells

A cell of a table may contain more text that fits onto one line. If you know exactly where you want the line-breaks to be, then structure the entire cell as a sequence of `<line>` elements.

Or, if you want the contents of a cell to look and feel more like a paragraph (or several), structure the cell as a sequence of `<p>`, which can contain the usual content of a `<p>`, excepting “larger” content such as display mathematics or lists. Now, in this case, you must constrain the width of the cell’s column, to force the line-breaking necessary to render a paragraph as several lines. Use the relevant `<col/>` element, and specify a percentage of the tabular’s overall width, like this:

```
<col width="40%"/>
```

A paragraph cell can be right-justified with the `@halign` attribute set to `justify`. But be aware that if the column is skinny, this can lead to awkward inter-word spaces.

4.19.9 Breakable Tabulars

A `<tabular>` may be specified as **breakable**, inside of a `<table>` or not. Use the attribute `@break` set to `yes`. (The default is `no`.) This only affects conversions to formats with page breaks, such as `PDF`. Usually the motivation will be a `<table>` or `<tabular>` that is too long for a page, but even a shorter table can be allowed to page break.

As of 2022-07-28 this is effective for simple tables, but introduces some variations for more complex constructions. This is implemented with the `TeX` `longtable` package, which suggests it may take up to four passes with `TeX` to obtain the final version. It is also not effective for a `<tabular>` that is a side-by-side panel. Consult the sample article for examples where more progress is necessary.

4.19.10 Table Philosophy

The *Chicago Manual of Style* [1, 13.1] says:

A table offers an excellent means of presenting a large number of individual similar facts so that they are easy to scan and compare. A simple table can give information that would require several paragraphs to present textually, and it can do so more clearly. ... A table should be as simple as the material allows and understandable on its own; even a reader unfamiliar with the material presented should be able to make general sense of a table.

If you review the twenty tables presented in Chapter 13 of CMOS, that are of the type we implement, you will notice several things.

- Only the first column is ever used for row headings.
- Cells do not span multiple rows. (There is no analogue for @colspan.)
- Column headings appear at the top, other than **cut-in heads**, which have a very particular form. (We have not implemented these, but would entertain a feature request.)

While our implementation allows for some presentational elements (borders, rules, alignment) our conversions will presume your table hews to the purposes described by CMOS. In particular, it is not a device for spatial layout of complex elements. You might find that the <sidebyside> and <sbsgroup> layout devices will suit that purpose better (see [Section 4.24](#)).

Best Practice 4.19.2 Tables are Difficult. Width is always at a premium, and then when a <tabular> has more than a few columns, the width becomes even more dear. When a <cell> has text that looks like a phrase or a sentence, rather than numerical data or symbols, it can be even harder to pack it all in. A common example is a schedule of talks at a small professional conference where each time slot (rows) might have two or three talks simultaneously in parallel sessions (columns).

We offer **paragraph cells** which automatically break lines, but you need to specify a @width on the <col> as a percentage to indicate where line-breaking happens. For manual line-breaking, a <cell> can be structured entirely by <line> elements.

The next complication is that the $\text{\LaTeX}$ used for PDF output tends to make columns as wide as necessary and will not break lines without the devices mentioned in the previous paragraph. The HTML output can sometimes be a bit more forgiving and flexible. So we suggest building the $\text{\LaTeX}$ output first and getting that right, and then the HTML is likely to follow along and not need much further refinement.

In contrast to most of PreTeXt, you may need to experiment, refine your approach, iterate, and maybe do things contrary to usual best practices elsewhere. For example, the clickables for URLs and knows might need to be short and less-informative in order to save some width. Abbreviations, initialisms, and acronyms can also save some width.

4.19.11 Summary: Table Reference

Finally, we summarize the available options for a table with...a table. Because it would take too much text to describe fully.

This table describes how to construct tables via the tabular element. The table element may be used to enclose the raw table, so as to associate a title and get vertical separation with horizontal centering.

The tabular element contains a sequence of row elements, and must contain at least one. Each row contains a sequence of cell elements and must have the same number in each row (accounting for the use of the colspan attribute). The contents of the cell elements are the text to appear in entries of the table.

A sequence of col elements may optionally be used. But if one appears, then there must be the right number for the width of the table. They are empty elements always, and just carry information about their respective column.

Where the body of the table below has an entry, it means the attribute may be used on the element, and affects the range of the tabular described by the element. Employment of an attribute on elements to the right in the table will supersede use on elements to the left. Generally, every cell has right and bottom borders, but only cells at the left side of the table have a left border and only cells across the top have a top border. Only one cell has four borders.

Table 4.19.3 Tabular Elements and Attributes (p = potential)

Attributes	Elements				Values * = default
	tabular	col	row	cell	
top	x	x			none*, minor, medium, major
left	x		x		none*, minor, medium, major
bottom	x		x	x	none*, minor, medium, major
right	x	x		x	none*, minor, medium, major
halign	x	x	x	x	left*, center, right, justify
halign		p			decimal, character
header			x		yes/vertical/no*
row-headers	x				yes/no*
valign	x		x		top, middle*, bottom
colspan				x	1*, positive integer
width		x			percentage
colors	p	p	p	p	

4.20 Program Listings

A `<listing>` is really a specialized type of `<figure>`, whose purpose is to hold computer code. It has an optional `<title>` which is rendered above the listing. However, the enclosed planar content is limited to a `<program>` or `<console>` (see [Section 4.16](#) and [Section 4.15](#)).

4.21 Named Lists

As mentioned above, it is not possible to have a list be the target of a cross-reference. Should an entire list be so *important* that you need to point to it from elsewhere, then make it a **named list** by wrapping it in the `<list>` tag.

This element can begin with an optional `<introduction>`, then has a single, required list, which may be any of the three types. It concludes with an optional `<conclusion>`. It can have an `@xml:id` attribute, which in a way is the whole *raison d'être* for this construction. It will be numbered when rendered, and so also requires a `<title>`. You might think of this as similar to a `<table>`—bits of information organized spatially, via indentation and line breaks.

Since this element associates a number, title, to an entire list, we call it a “named list”. What should we call a list that is authored within a paragraph and cannot be the target of a cross-reference? We call it an **anonymous list** when we want to make the distinction.

4.22 Sage

Until we can expand this section, get some ideas from [Section 3.17](#). We will also collect a few items here, to be cleaned-up later.

For online output formats, sometimes the output of a Sage command can be overwhelming, and a bit complicated to parse. Many objects in Sage also have a $\text{\LaTeX}$ representation, which can be used to create a superior output format (for some purposes). Begin a cell with the “magic”:

```
%display latex
```

Experiment with the following Sage code on the next line

```
integral(x^9*cos(x), x)
```

Boom! Very nice. Try replacing `latex` with `None`, `plain`, `ascii_art`, or `unicode_art`.

4.22.1 Sage Cell Server Design

The ability to execute, and edit, chunks of Sage code is provided by a distinct project, the [Sage Cell Server](#). Simplifying somewhat, the Sage code a reader sees (or has edited!) is shipped out to a running instance of Sage (on a server *somewhere*) and the code is executed there. The results of that computation are shipped back to the reader for display below the code.

Two implications of this design are

- It is not within your power to add additional packages for the supported languages.
- You cannot read a (data) file hosted on your project's site.

Fortunately, there are workarounds.

If your code needs a Python package, or an R package, or similar, and it is a standard open source package, then make a request on the [Sage Cell](#) Google Group. Likely, it can be added/installed.

Unfortunately, the ability to read files *anywhere* on the internet was abused, so this capability had to be restricted to a finite list of servers. These include [DropBox](#) and [GitHub](#) where you might find it convenient to place files supporting your code. Note that for GitHub, you likely want to use a URL which is a “raw” file such as for the PreTeXt repository [README](#) file, written with Markdown.

4.23 Interactives

TODO: until then examine copious examples in the sample article.

4.24 Side-by-Side Panels

Documents, pages, and screens tend to run vertically from top to bottom. But sometimes you want to control elements laid out horizontally. A `<sidebyside>` is designed to play this role. It is best thought of as a container, enclosing **panels**, and specifying their layout. Examples include three images, all the same size and equally spaced. Or a poem occupying two-thirds of the available width, with commentary adjacent in the remaining third. Or an image next to a table. But the most common use may be a single image (with no caption, and hence no number), whose width and horizontal placement are controlled by the layout.

See the schema for the exact items that are allowed in a `<sidebyside>`. To author, just place these items within `<sidebyside>` in the order they should appear, left to right. Then you add attributes to the `<sidebyside>` element to affect placement.

Instead of placing a `@width` attribute on each item, instead place this on the `<sidebyside>` element. A single `@width` will use the same value for each panel. For different widths, use the plural form `@widths` and provide a space-separated list of percentages. The default is to give each panel the same width, and as large as possible, which will result in no gap between panels.

The margins can be specified with the `@margins` attribute, which if given as a single percentage will be used for both the left and right sides. You may also specify asymmetric left and right margins with two percentages, separated by a space, in the same attribute. An additional option is to use the value `auto` which will set each margin to half of the (common) space between panels. This is also the default. In the case of a single panel, the left margin, right margin, and panel width should all add up to 100%.

Once the widths and margins are known, any additional available width is used to create a common distance separating panels. (Which is not possible when there is just a single panel.)

Independent of horizontal positioning, individual panels may be aligned vertically. The attribute is `@valign` and its value is a space-separated list of `top`, `middle`, and `bottom`. The singular version, `@valign`, is used to give every panel the same alignment, using the same keywords. The default is to have every panel at the top.

We could give lots of examples, but instead it might be best to just experiment. Error-checking is very robust, so it is hard to get it too wrong. OK, we will do just one to help explain. Suppose a `<sidebyside>` contains three panels and has layout parameters given by

```
<sidebyside widths="20% 40% 25%" margins="auto" valign="middle">
```

Then there will be 15% of the width left to space out the panels. The two gaps are each 5% of the width, and the remaining 5% is split between the margins at 2.5% each. And the vertical midlines of each panel are all aligned.

For a single panel with no attributes, the panel will occupy 100% of the width. A single panel with a specified width will get equal (auto) margins, resulting in a centered panel.

Captioned items as panels deserve special mention. These will continue to be numbered consecutively, with one exception. If you place a `<sidebyside>` inside of a `<figure>`, then the `<figure>` will be numbered, and the captioned items inside the `<sidebyside>` will be **sub-captioned**. In other words, the second captioned panel of a `<sidebyside>` inside Figure 5.2 would be referenced as Figure 5.2.b.

An `<sbsgroup>` (“side-by-side group”) contains only `<sidebyside>`, which are displayed in order. However, all of the layout parameters allowed on a `<sidebyside>` may be used on an `<sbsgroup>`. This might allow a collection of fifteen images to be laid out in three rows of five images each, with widths and spacing identical for each row because the parameters are specified on the `<sbsgroup>` element. In this way, simple grids can be constructed. Note that any layout parameters given on an enclosed `<sidebyside>` will take priority over those given on the `<sbsgroup>`. Captioning behavior extends to an entire `<sbsgroup>`.

Since `<sidebyside>` and `<sbsgroup>` are containers they cannot be referenced and so do not have an `@xml:id`. However, you can reference their individual contents if they are captioned, and you can reference an enclosing `<figure>`.

Generally, a `<sidebyside>` or `<sbsgroup>` can be placed as a child of a division, or within various blocks, such as `<proof>` for example. See the schema for (evolving) specifics.

It should be clear now that a `<sidebyside>` is more about presentation than most PreTeXt elements, though there is some semantic information being conveyed by grouping the panels with one another.

4.25 Front Matter

A single `<frontmatter>` element can be placed early in your `<book>` or `<article>`. It is optional, but likely highly desirable. The following subsections describe the items that may be employed within the `<frontmatter>`. Most are optional, and some may be repeated. An `<article>` differs in that it must contain a `<bibinfo>` and `<titlepage>` and then may *only* contain an `<abstract>`. Generally, these divisions will get default titles, localized in the language of your document, but these defaults may also be replaced by giving a `<title>` element. None of these divisions themselves is numbered, so precluding any content within that might normally be numbered. So, for example, no `<figure>` may be included. But you could choose to include an `<image>`, perhaps within a `<biography>`.

If a component of the front matter cannot be numbered, how best to subdivide something like a `<preface>`? This is a good use of the `<paragraphs>` element. It allows for a (minimal) title, but cannot be subdivided further. See the later part of [Section 4.6](#) for more about this exceptional element.

These elements must appear in your source in the order given below, and will appear in your output in the same order, which is a generally accepted order used in the production of books. So, for example, even if you author an `<acknowledgement>` between two `<preface>`, your output may (will?) place the Acknowledgement before the first Preface.

Exceptions to the above general information are noted below.

4.25.1 Bibliographic Information

Required. The `<bibinfo>` element may contain metadata about your document, including `<author>`, `<editor>`, `<credit>`, `<date>`, `<keywords>`, `<edition>`, `<website>` and `<copyright>`. Additional elements to capture bibliographic information are planned. This is an exception to the divisions described next, as this is a mini-database of sorts, which contains various pieces of information which get used different ways, but primarily via the `<titlepage>` element.

Authors and editors. Each author and editor should be described in their own `<author>` or `<editor>` element, which are structured identically (in output, authors are listed first, followed by editors, sometimes using less prominent formatting). An author can be designated as the **corresponding author** using the `@corresponding` attribute.

The name of an author or editor should be enclosed in a `<personname>` element. Following this, affiliation information can be provided either by using elements `<department>`, `<institution>`, and `<location>` (each of

which can be further structured with `<line>` tags), or by enclosing these in an `<affiliation>` element. (Grouping affiliation details is useful when an author might have multiple affiliations.)

An author or editor can also have an `<email>`, `<biography>`, or `<support>` element. The `<support>` element can be used to describe funding sources particular to that author as required by some journals. Notice that the `<support>` element could also be a child of `<bibinfo>` itself, in which case it would appear as applying to the entire document, not just an individual author.

Keywords. Many journals require papers to contain a list of keywords or subject classification codes. These are both captured by the `<keywords>` element. This groups a collection of `<keyword>` elements, each of which is a single keyword or subject classification code.

To distinguish between author-provided keywords and subject classification codes, the `<keywords>` element can have attributes `@authority` and `@variant`. As of 2025-02-18, the values of these attributes that are recognized are given in [Table 4.25.1](#).

Table 4.25.1 Recognized keyword attribute values

Type	@authority	variant
Author-provided keywords	author or none	none
Math Subject Classification (MSC) codes	msc	2020 or 2010 or etc.

For subject classification codes, to distinguish between primary or secondary codes, the `<keyword>` element can have the optional attribute `@primary` (with value `yes` or `no`). Put `primary="yes"` on the *first* primary keyword, and `primary="no"` on the *first* secondary keyword. Alternatively, using `secondary="yes"` on the first secondary keyword is also acceptable.

If you are writing a paper that needs a different classification from those currently available, please submit a request.

4.25.2 Title Page

Optional, but usually desirable. When present, the `<titlepage>` should contain a single empty element `<titlepage-items/>`, that will collect the appropriate elements from `<bibinfo>` to generate a titlepage. So, this is an exception to the following list of divisions, as its production is automatic.

4.25.3 Abstract

Optional, and only available for an `<article>`.

4.25.4 Colophon

Optional; only available within a `<book>`. The *front* colophon. (There is also a *back* colophon, see [Subsection 4.26.6](#)). Sometimes this is also called the **copyright page**. The `<colophon>` element can be given a `@label` to produce a specific file name in HTML builds. The only allowable (and required) child of the `<colophon>` is the empty element `<colophon-items/>`, which will automatically bring in appropriate elements from `<bibinfo>`. So, like the `<titlepage>`, this is an exception due to its automatic production.

4.25.5 Biographies

Multiple `<biography>` elements, perhaps one per author. The `<title>` can be used in the case of multiple authors to distinguish one from another.

4.25.6 Dedication

A single `<dedication>` element, that might include multiple dedications (perhaps by different authors). The content is simply a sequence of `<p>`, with multiple elements for multiple dedications.

4.25.7 Acknowledgements

A single <acknowledgement> element (note spelling), that becomes a division, and so can contain paragraphs, lists, etc. The Chicago Manual of Style [1, 1.52] suggests that if these are short, they may be contained in a preface.

4.25.8 Forewords

As of 2021-07-16 the <foreword> element is not fully implemented. Please make a feature request if you need it.

A <foreword> is written by somebody other than the author. The name of the writer of the foreword needs to be included—at the end is a good location.

4.25.9 Prefaces

Multiple prefaces are a distinct possibility, and in this case providing a different <title> for each would be essential. Examples might include: “Preface to the Third Edition”, “How to Use this Book”, or “To the Student”. More ad-hoc material, such as a translator’s note, can be handled as a preface.

Best Practice 4.25.2 Understand the Role of a Preface. *Chicago Manual of Style* [1, 1.49] begins with “Material normally contained in an author’s preface includes reasons for undertaking the work, method of research, ...” Note that a preface is not introductory content and is not an introduction. It is written from the author’s point-of-view, and may include information about why they are qualified to write on the topic of the book. If there are several editions, the prefaces to the newer editions are placed first. See the related [Best Practice 26.3.1](#).

4.26 (*) Back Matter

4.26.1 (*) Appendices

Automatic lists ([Section 4.29](#)) can appear anywhere, but an appendix is a very natural place to place one.

4.26.2 (*) Glossary

4.26.3 (*) References

4.26.4 (*) Solutions

4.26.5 (*) Index

4.26.6 (*) Colophon

The *back* colophon, what most authors think of as *the* colophon. (There is also a *front* colophon, see [Subsection 4.25.4](#)).

4.27 Index

Continuing our basic discussion from [Section 3.23](#), we discuss some details of making and using index entries. We will begin with how you *procedurally* author an index entry with PreTeXt syntax, and then move to general principles about how to use these constructions to create an effective index. So these two subsections are intimately linked.

4.27.1 Syntax and Placement of Index Entries

Best Practice 4.27.1 Capitalization of Index Entries. The headings (entries) of an index are authored entirely in lower-case, unless it is a proper noun (name, place, etc.) which would normally be capitalized in the middle of a sentence. We are not able to provide any enforcement of this advice, nor any assistance. It is the author’s responsibility to provide quality source material in this regard. We do sort entries so that an entry with an initial capital letter arrives at the right location in the index.

Where you place an `<idx>` entry is critical. With $\text{\LaTeX}$ output, you will get the traditional page number as a locator in your index. With HTML output we can be more careful. We will look to see which sort of structure contains the `<idx>`. Maybe it is an `<example>` or a `<subsection>`. If so, the index will contain a locator that is a knowl of the example, or a link to the subsection. The distinction is the size of the object, we do not knowl divisions. The exception is a paragraph (`<p>`) that is a child of a division, and then the locator is a knowl of the entire paragraph. Remember that a knowl contains an “in-context” link which can take the reader to the original location of the content in the knowl.

A lot happens in a PreTeXt paragraph, especially when producing HTML. Sometimes an `<idx>` can get in the way. Our recommendation is to put `<idx>` entries *between* sentences, and not at the start or end of the paragraph. They can be authored with each on their own line. If you do not need the specificity of a paragraph, then locate the appropriate structure and author the `<idx>` right after the `<title>` (or where one would be).

A **cross-reference** in an index is a pointer to another index entry. These are rendered as “See” and “See also.” You can add `<see>` and `<seealso>` elements within an `<idx>`, so long as it is structured with `<h>`. Then it is placed after the last `<h>`. A “see” cross-reference is a direct pointer to another entry in the index. It cannot have a locator as well. When you build the HTML output, we will recognize this situation and produce a warning. A “see also” cross-reference is an additional pointer, and so it must have a locator to go with it (you will author two `<idx>` with identical headings, the first without a `<seealso>` to create the locator, the second with the `<seealso>` to create the cross-reference. Again, when you build the HTML output, we will recognize a `<seealso>` without a locator and produce a warning.

Follow these directions and PreTeXt will format cross-references for you, in the style suggested by the *Chicago Manual of Style* [1] for HTML output, and according to $\text{\LaTeX}$ ’s style for print and PDF.

(2019-03-04) We have consciously not said anything specific about what to place inside a `<see>` or `<seealso>` element. At this writing, you need to supply the text. Of course, this is error-prone and you will need to consult CMOS for formatting guidance. But we have plans to do this the PreTeXt way. First, the `ref/xml:id` mechanism will be used to automatically create the correct text for the cross-reference, both content and format. Second, these will become live links in electronic formats.

Certain index entries do not sort very well, especially entries that begin with mathematical notation. Our first advice is to avoid this situation, but sometimes it is necessary. The `@sortby` attribute on an `<h>` element can contain simple text that will be used to override the content shown to the reader during the sorting of the index.

4.27.2 Advice on Indexing

An index is a navigational aid for your readers (and you). We do not assume that a reader remembers where anything is, nor that the Table of Contents is a replacement for part of the index. Some readers of the index may not have even read your book yet, and are looking to get a feel for the range of topics as part of the decision of whether or not to read your book at all, or if it will be useful to have. It should be comprehensive, including everything substantive.

Indexing is a job for a skilled professional, and most authors produce poor indexes. The tips in this section will help you avoid the most common pitfalls. We follow recommendations from the *Chicago Manual of Style* [1, Chapter 18], *Indexing for Editors and Authors: A Practical Guide to Understanding Indexes* [3], and Pilar Wyman of [Wyman Indexing](#).

Terminology. The basic element of an index is an **entry**, which consists of a piece of information and its **locator**. For example:

```
normal subgroup, 37
```

is an entry indicating that information about “normal subgroup” can be found on page 37. Indexes are (usually) organized alphabetically, with a **main heading** aligned with the left margin, and progressively indented **subheadings** below the main heading.

Often it is desirable to place the same locator under more than one heading, known as **double posting**. For example, a desirable addition to the sample entry above is

```
subgroup, 28
  normal, 37 .
```


An alternative to double posting is **cross referencing**, using *see* and *see also*. Typically cross references are used to avoid repeating a large number of entries, or to direct the reader to related topics.

An index may start with a **headnote** giving advice about using the index. Typically a headnote is not necessary unless the index has some unusual features.

Basic principles. The purpose of an index is to point the reader to information. Point to, not repeat. For example, acronyms should be indexed at the location where they are defined, not at every place they appear, and it is not necessary to define the acronym within the index. People and places should be indexed when information is given about them, not every time they are mentioned.

A good index has multiple ways to find the same information. Being redundant is desirable, because it increases the chance the reader finds what they seek in the first place they look.

Indexing is best done after the text has been written. Adding index entries while writing the text may seem to be a labor-saving device, but if you are not an experienced indexer, those entries will only be a small fraction of the final index.

Topics should be indexed in multiple ways. If a term is defined, you should also think of other words the reader might search for. For example, you may define “limit point” and consistently use only that term, but an index entry for “accumulation point” with a “*see limit point*” locator would be appropriate.

Use **disambiguation** to distinguish identical terms with different meanings. For example

isomorphism (of groups), 55
isomorphism (of rings), 123

Both of those entries should also be double posted under the main headings of “group” and “ring”, respectively. No disambiguation is needed for those entries.

Singular or plural forms of nouns should reflect the language in the text. So if a chapter is titled “Mammals”, then use a heading *mammals*. And if the chapter is titled “The Mammal Class”, then use a heading *mammal*.

An index is typically as long as 5% of the main text. With many figures, or other structures creating additional whitespace, the percentage may be lower. If your primary output is online, length may not be an issue. For print, there are strategies for pruning an index.

Once you have finished the text, and then finished the index, it is time for a thorough review of the index. There will be places for consolidation, often due to using variants of particular words. You may wish to remove subheadings which all appear within the range given in the heading. For example,

fish, 204–212
 bass, 208–209
 salmon, 210
 trout, 207

could have all of its subheadings removed, especially if space is an issue.

Common pitfalls. Sometimes it takes less than one second to determine that an index is poor. If a quick glance reveals that the index consists mostly of main headings with very few subheadings, then few readers will find it to be useful. Double posting, which may mean more than literally two entries with the same locator, will help readers find what they are looking for. Most of those entries will be in subheadings.

Another instantly recognizable problem is too many locators in one entry. This entry

asymptote, 37, 48

is probably fine. But once you have three or more locators in an entry, then your index may be improved by adding some subheadings. If the locators in the above example refer separately to “horizontal” and “vertical”, then probably two subheadings would be more useful than two undifferentiated locators in one entry.

An additional problem which can be seen at a glance if you know what to look for, is the absence of any main headings with a large number of subheadings. On almost any subject there are topics which are addressed repeatedly. This should be reflected in the structure of the index. For example, in a group theory textbook there should be several entries under “group, examples”. In an introductory calculus book the index should help the reader locate the derivative of many different elementary functions.

Index headings should be nouns, not adjectives. An adjective may be important, and you should use it, but it should not be the entire content of a heading since it is not an idea by itself. But it may be a subheading. For example, suppose you have a paragraph on “highland sheep.” Then *both* of the following should appear in your index, since a reader might consult both locations.

highland sheep, 45

sheep
 highland, 45

4.28 Notation

We continue the introduction at [Section 3.23](#). A notation list, like an index, is a specialized collection of cross-references. So some of the philosophy here applies equally well to the `<idx>` and `<index-list>` elements, and vice-versa. (See [Section 4.27](#).)

To generate a list of notation employed in a book or article, use the `<notation-list/>` element. This empty element belongs in an `<appendix>`. Likely it is the only content, or you might include some preliminary material. The title of the `<appendix>` is up to you and is not automatic.

Some authors like to make definitions inside of paragraphs, ideally using a `<term>` element. This is a natural place for a `<notation>` element. So this approach gives an author a lot of flexibility in location.

Other authors like to make definitions using the `<definition>` element, since it creates a heading and number, allows a `<title>`, and can easily serve as the target of a cross-reference. So this is another good place for a `<notation>` element. But now, associate it clearly with the `<definition>` by placing it in the metadata, early on, after the `<title>`. And not in some subsequent paragraph. The reason will be clear in just a bit.

How is a `<notation>` element constructed? It has two elements. The `<usage>` should be a sample piece of mathematics using the necessary symbols, and wrapped in a *single* `<m>` element. The second element is `<description>` and should be a short phrase, or sentence-like material, decoding the sample usage, and may include `<m>` elements. The reader sees nothing in the output at the location of the `<notation>` element.

The automatically-generated notation list is then a three-column table, in the order of appearance, with the sample usage, the description, and a **locator**. For output derived from $\text{\LaTeX}$, such as print or PDF, the locator will be the page number of wherever you placed the `<notation>` element. For HTML the locator is much better—it is a knowl, for either a paragraph or for an entire definition. The latter possibility explains why it is better to place the `<notation>` element inside a `<definition>`, if possible, rather than in a paragraph that is a constituent of a `<definition>`.

4.29 Automatic Lists

Sometimes it is useful to have an automatic list of various elements of one kind in a book, other than the ones already available in a PreTeXt document. The predefined ones include an index (see [Section 4.27](#)) and a list of notation (see [Section 4.28](#)). Examples of lists one might wish to create could include lists figures, computational listings, or theorems.

There is a very flexible way to make a list of various blocks (or perhaps other items) in your text. Use an empty `<list-of/>` element as a child of a division. A very natural use would be to create an `<appendix>` for the sole purpose of holding one such list. This is why this feature is frequently used in the back matter. But you could place an **automatic list** many other places.

We will illustrate with an example. Suppose you know your book has theoretical results only in `<theorem>` and `<lemma>` elements. So, for example, you never use `<corollary>` elements. Then you could author

```
<appendix xml:id="appendix-list-results">
  <title>List of Results</title>

  <list-of elements="theorem lemma" divisions="chapter" empty="yes"/>
</appendix>
```

The result will be a link to every `<theorem>` and `<lemma>` in the entire book, using a clickable with its type, number and title. See [Appendix B](#) for an example. In HTML output the clickables will usually be knowls, which is especially handy. The list will be organized with the titles of the chapters as headings. The `@divisions` attribute can have several types of divisions listed, such as both `<chapter>` and `<section>`. The `@empty` attribute set to `yes` indicates that a division heading should be used even if there is nothing on the list contained within. The default for `@empty` is `no`.

This feature is best used when the items in the list have been authored with titles, which greatly increases the utility of the list for your reader. Review [Best Practice 4.8.1](#) if this advice is new to you.

There is a `@scope` attribute, which should be the name of an element which is a division containing the location of the `<list-of>` element. Then the list is restricted to items within the specified division. For example, if you have the `<list-of>` inside a `<subsection>` built for this purpose, and you use `@scope="section"` then the list will have all the items from throughout the section containing the list.

There are four types of exercises, based on their location: inline, divisional, worksheet, and reading questions. These may be specified inside `@elements` by the **pseudo-elements**: `'inlineexercise'`, `'divisionexercise'`, `'worksheetexercise'`, `'readingquestion'`. (These are just strings meant for this purpose, and are not *real* PreTeXt elements.)

There may be an argument for a `@ref` attribute that would behave similar to `@scope`. Make a feature request if you need it?

4.30 MyOpenMath Exercises

MyOpenMath is a hosted online homework system with hundreds of thousands of prebuilt questions.

In the web output, the MyOpenMath question will be embedded and interactive though not tied to any student or faculty account. In static output, a static version of the question will be included. Note that there are some limitations on which problems will display correctly in web and static output. Best practices are below.

To include a MyOpenMath question, you'll first need an account on [MyOpenMath.com](https://myopenmath.com) where you can browse the question libraries and look up the question ID. Then in your PreTeXt document include a `<myopenmath>` element with a `@problem` attribute with the MyOpenMath question ID.

To further control the web behavior of the embedded question, you can optionally include a `@params` attribute containing a comma-separated list of parameters, like `maxtries=1, showhints=0`, which can include:

maxtries	(default 0): Set to more than 0 to set the max tries on a question part before it gets disabled, and before a scaffolded question will move on to the next part.
hidescoreval	(default 0): Set to 1 to hide the "score: #" that displays after submitting a question. Score markers (check or x) will still show.
showansafter	(default 1, or maxtries if set): Set to have the answer show after this many tries. Set to 0 to have answers never show.
showhints	(default 3): Set to 0 to suppress help features, like hints and video buttons.
allowregen	(default 1): setting this to 1 will show a "Try another version" button after submitting.
submitall	(default 0): Set to 1 for all parts to get submitted, regardless of whether all parts are answered.
seed	(default random): To set a specific seed (1-9999) to force a specific version of the question to display

While almost all problems will display correctly in web output and many will display correctly in static output, it is recommended that you avoid using problems with the following features. Note, you can usually make a copy of the problem, edit out the offending feature, and use your substitute.

- Scaffolded problems
- Problems with extensive formatting (e.g., use of an html table to control display)
- Lists containing images or line breaks (`<br/>`)

If you really love a MyOpenMath problem that does not display correctly, you might contact PreTeXt support; likely someone knows how to produce a conforming version.

4.31 STACK Exercises

STACK is an assessment system with online interactive exercises. This section will soon have instructions on how to embed these exercises.

4.32 URLs and External References

4.32.1 URLs to External Web Pages

The `<url>` element is used to point to *external* web pages, or other online resources (as distinct from other internal portions of your current document, which is accomplished with the `<xref/>` element, [Section 3.4](#)). The `@href` attribute is always necessary, as it contains the full and complete address of the external page or resource. Include everything the URL needs, such as the protocol, since this will be most reliable, and as you will see it never needs to be visible. The element always allows, and then employs, a `@visual` attribute for a provided more-friendly version of the address. Finally, the content of the element, which becomes the clickable text in electronic formats, can be authored with the full range of PreTeXt markup generally available in a title or sentence. A typical use might look like

```
<url href="https://example.com/" visual="example.com">Demo Site</url>
```

This will render as [Demo Site](#). If a `<url>` has content, and no `@visual` attribute is given, then the `@href` will be placed in a trailing parenthetical for print versions of PDF, though there will be an attempt to remove standard protocols. Compare

```
<url href="https://example.com/">Demo Site</url>
```

which will render as [Demo Site](#) versus

```
<url href="mailto:nobody@example.com">Bouncing Email</url>
```

which will render as [Bouncing Email](#).

If you do not provide any content for a `<url/>` element, then the clickable text will be the actual URL with a preference for the (optional) `@visual` attribute, rather than the mandatory `@href` attribute. This should be considered as disruptive to the flow of your text, and so a poor alternative to the content version just discussed (see [Best Practice 4.32.1](#)). But it might be a good choice in something like a list of interesting web sites. Whether or not a simplified version of the address, via the `@visual` attribute, is desirable will depend on the application. As an example, using the optional `@visual` attribute we have

```
<url href="https://example.com/" visual="example.com"/>
```

This will render as [example.com](#). Note that for the print version of PDF, there is no trailing parenthetical since the visual version is already apparent.

If you want to squelch the automatic trailing parenthetical for the print version of PDF on a `<url>` element with content, you can explicitly set the `@visual` attribute to an empty string as `visual=""`. This signal will inhibit the automatic trailing parenthetical. This should be a *very rare* occurrence, since you are denying readers of some formats from seeing even a hint of the actual URL.

A `<url>` inside a `<title>` has been accounted for, but should be used with caution.

As with the rest of PreTeXt we have taken care to handle all of the exceptional characters that might arise in a `<url>`. So author normally, using the necessary keyboard characters, only taking care with the two XML characters, `<` and `&`, which need escaping (see [Section 3.14](#)). Use **percent-encoding** (aka **URL encoding**) for the `@href` attribute, if necessary, to include special characters, such as spaces. See [Subsection 4.32.4](#) below for a common need for the ampersand character, and a further caution about percent-encoding of URLs.

Finally, for conversion to $\text{\LaTeX}$ /PDF output it gets extremely tricky to handle all the various meanings of certain escape characters in URLs in more complicated contexts (such as tables, footnotes, and titles), so there may be some special cases where the formatting is off or you get an error when compiling your $\text{\LaTeX}$. We have anticipated most of these situations, but we always appreciate reports of missed cases.

4.32.2 Data URLs

A `<dataurl>` element is very similar to the `<url>` element just described. The purpose is to point to an actual file that will be of use to your readers. What actually happens when a reader clicks on it is dependent on the format of the $\text{\LaTeX}$ output and that reader's environment. Maybe the file will be downloaded, or maybe a particular application will open the file. That part is out of our hands. Use an `@href` attribute in the same way as for `<url>`, and the content and the `@visual` attribute also behave similarly.

The one key difference is that you can also use a `@source` attribute in place of `@href` and point to a file that you provide as part of your project (not unlike providing a photographic image via the `<image>` element). Place the file in your collection of external files (see [Section 5.6](#)) and provide the path to your file from below the directory of external files in the `@source` attribute. For HTML output, $\text{\LaTeX}$ will do the rest. For more static formats, you can set a base URL (see [Subsection 44.4.2](#)) and you will get a complete URL that points to the instance of your file hosted with the rest of your HTML output.

Notice that this element provides limited functionality, at best just a hyperlink to a file. For data files that you want a reader's in-browser computer program to process, read about the `<datafile>` element at [Section 4.17](#).

4.32.3 Visual URLs

By a **visual URL** we mean a version of a URL that is simpler than the “real” URL, but that provides enough information that a reader can type the URL into some other device with a minimum of effort, and with success. Consider that your project may someday be a print (hardcopy) book, or that your project will be converted to braille for a blind reader. These are some ideas about making a URL simpler. We welcome more ideas.

- Remove standard/default protocols like `http://` and `https://` which most browsers will furnish in their absence.
- Sites like [StackExchange](#) list posts with a long identifying number, followed by something that looks like the title. In practice, the number is enough.
- Experiment with dropping a trailing slash—they are frequently unnecessary.
- Often a leading `www.` in a domain name is not necessary.
- Try providing just a domain name in place of a top-level landing page, it will often redirect to a longer URL.
- You could use a [URL shortener](#), though some thought should be given to [its longevity](#). Will you remember where your short URLs point once they are no longer functional? Safer to have your long URLs in an `@href` in your source, and use $\text{\LaTeX}$ to make them friendlier.

Best Practice 4.32.1 Craft URLs Carefully. Your writing will be smoother, and easier on your readers, if you do not interrupt a sentence with a long URL, unless somehow it is really of interest and relevant right there. So provide content (the “clickable” text) when you use the `<url>` element (rather than an empty `<url/>`). This obligates you to provide a `@visual` attribute, which feels a little like a tedious exercise. But this will be very welcome to some of your readers, those who are unable or prefer not to use electronic formats. Just above ([Subsection 4.32.3](#)), we provide suggestions for crafting these to be more pleasing, but still useful, versions of URLs.

4.32.4 Characters in URLs

A URL can have a **query string**, which has a list of parameters following a question-mark. The parameters are separated by ampersands (`&`), which will need to be escaped, so as to not confuse the XML processor. So use `&` anywhere the ampersand *character* is necessary, such as a `@source` attribute, or a monospace version of a URL achieved with a `<c>` element. Also, the question-mark character should *not* be URL-encoded (`%3F`) (despite advice

just given above), so if necessary edit it to be the actual character. General advice about exceptional characters in XML source can be found in [Section 3.14](#).

4.33 Video

A video is a natural way to enhance a document when rendered in an electronic format, such as HTML web pages. It might be additional information that is hard to communicate with text (marine invertebrates swimming), a lecture or presentation that augments your text, or even some artistic work, such as a symphony legally hosted on YouTube, when you could never hope to get copyright clearance yourself.

PreTeXt supports videos you own and distribute with your source, videos shared openly on the Internet via stable URLs, and videos available on YouTube. Go straight to to the end of this section to see how easy it is to incorporate a YouTube video.

HTML5 web browsers are able to play video files in three formats, summarized in the following table.

Table 4.33.1 HTML5 video formats

Format	Extension	Reference
Ogg, Theora	.ogg	Free and open, Wikipedia
WebM	.webm	Royalty-free, Wikipedia
MPEG-4	.mp4	Patent encumbered, Wikipedia

4.33.1 Video Element

The `<video>` element is used to embed a video in output formed from HTML. Subsections below describe the different ways to indicate the source of the video. The video may be placed inside a `<figure>` or can be a panel of a `<sidebyside>`. The former will have a caption, be numbered, and hence can be the target of a cross-reference (`<xref>`). The latter is anonymous, but allows for horizontal layout, and combinations with other panels.

Size is controlled by a `@width` attribute expressed as a percentage (on the `<video>` element when used in a figure, or as part of the `<sidebyside>` layout parameters). Height is controlled by giving the **aspect ratio** with the `@aspect` attribute on the `<video>` element. The value can be a ratio expressed like 4:3 or a decimal number computed from the width divided by the height, such as 1.333. The default for videos is a 16:9 aspect ratio, which is very common, so you may not need to specify this attribute.

Options include specifying a `@start` and an `@end` in seconds as integers (no units) if you only want to highlight a key portion of a video. The `@play-at` attribute can take the following values

embed	Play in place (the default action).
popout	Play in new window or tab, at 150% width.
select	Provide the reader the choice of the other two options.

In an educational setting, sometimes the preview images provided by YouTube can be distracting, or for an author-provided video you may wish to provide your own preview image. The `@preview` attribute can take on the following values

generic	PreTeXt supplies a Play-button image.
default	Whatever the video playback provides. This is identical to simply not including <code>@attribute</code> at all
Path to an image file	
	Typically, this will be a relative path, starting with <code>images/</code> . This image will be used as preview for the online version and the print version.

4.33.2 Author-Provided Videos

If you own and possess your video content, then you can distribute it with your PreTeXt source, and it can be hosted

as part of your HTML output. Then the @source should be a relative file name that points to the file containing the video. If you are able to provide more than one of the three formats in [Table 4.33.1](#), then you can provide the filename *without an extension*. If a browser cannot play one format, it may be able to play another. PreTeXt will write the code to make that happen, preferentially in the order of the table (more open formats first!). In other words, you can provide files in more than one format and increase the likelihood that a reader's browser will find a format it can playback.

4.33.3 Network-Hosted Videos

If a video is shared openly on the Internet, you can simply provide the full URL in the @href attribute. All the other attributes are the same as for the author-provided case, above. Read [Subsection 4.32.4](#) for some considerations when authoring a URL, since there are a few gotchas.

You can frequently discover the URL of a video by first playing it, and then using a context menu (e.g. via a right-click) to reveal an option to copy the video's location. However, note that there are various techniques sites use to make such a URL temporary, or otherwise unusable. So do some research about potential uses and test carefully. Our example below is provided from a United States government site.

Also, some video-sharing sites do their best to never make it easy to access the video as a file, so the construction described here will not be successful. Instead, they have links to "Share" or to "Embed" which will run their own (proprietary) player in your web browser. YouTube and Vimeo are two popular examples, and we provide explicit support, see [Subsection 4.33.4](#) for YouZTube, while Vimeo is similar. For other sites, you have limited success with the <interactive> element, with a construction such as:

```
<interactive iframe="https://www.dailymotion.com/embed/video/x8mc0pr"/>
```

Here is one example of a network-hosted video in MP4 format (from the [HTML5 Video Test Page](#) at [Tek Eye](#)):

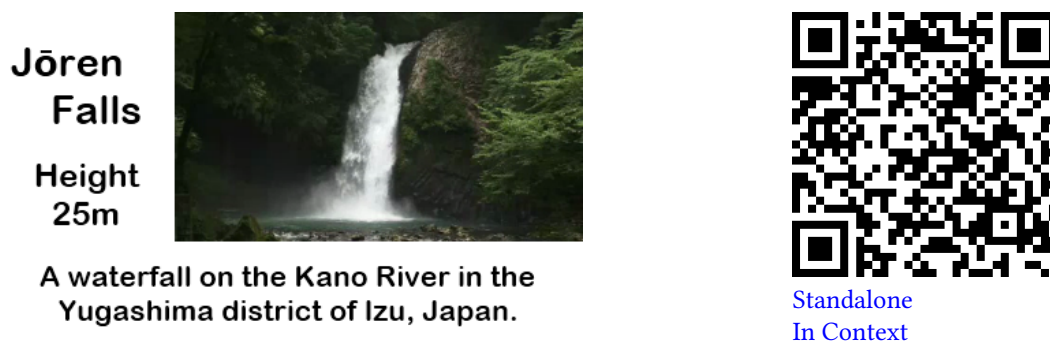


Figure 4.33.2 Jōren Falls, Kano River, Japan

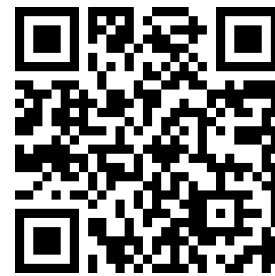
4.33.4 YouTube Videos

For a video hosted at YouTube, find the 11-character identification string in the address of a video you are viewing. It will look something like hAzdGU_kpGo. Then, instead of the @source attribute, simply provide this identification string as the @youtube attribute, such as

```
youtube="hAzdGU_kpGo"
```

That's it. All of the options above are then implemented and realized with YouTube's embedded player.

This can be a great way to incorporate popular or artistic content, legally, which might be difficult or costly to acquire through copyright clearance.



[Standalone](#)
[In Context](#)

Figure 4.33.3 The Eagles, “Hotel California”



[Standalone](#)
[In Context](#)

Figure 4.33.4 Mozart, Piano Sonata in C Major, K. 545, II

The pretext script ([Chapter 47](#)) may be used to download the provided preview images for YouTube videos (only). Filenames will be formed from the `@xml:id` of the `<video>` element. These will be used in static versions of output, such as print. Once custom preview images are implemented for author-hosted video, their static representation will improve.

Additionally, a YouTube **playlist** can be included in one of two ways. You may set the `@youtube` attribute to be a space-separated list of several video IDs (an “itemized” playlist). Alternatively, you may set the `@youtubeplaylist` attribute to a YouTube playlist ID (a “named” playlist). At present, a named playlist will not get a thumbnail image from Youtube, and either the “generic” thumbnail will be used or you can supply your own `@preview`.



[Standalone](#)
[In Context](#)

Figure 4.33.5 YouTube Playlist

Make a feature-request if a scheme similar to the one for YouTube, but for some other video-hosting service, would be useful for your project.

Note that when a project is hosted at Runestone Academy ([Chapter 32](#)) a YouTube video becomes an activity that is part of a student’s reading assignment. So just like an interactive exercise, it needs a `@label` attribute. See [Best Practice 4.12.1](#) for more.

4.34 (*) Music

TODO: Scholarly works discussing music may use notes and chords in text, and displays of sheet music are easily supported. (TODO: add some discussion to [Chapter 3](#).)

4.35 (*) Units of Measure

4.36 Unicode Characters

PreTeXt supports (and encourages) the use of Unicode characters. Here are some relevant comments.

- Unicode characters will migrate well to any output format based on HTML. Most browsers will have a variety of fonts with glyphs to realize these characters.
- $\text{\LaTeX}$ will not always behave as smoothly. For openers, you definitely will want to use the `xelatex` engine to build a PDF. Then you need to be sure your system has a font with the necessary characters and you make the font known to `xelatex`. We are working out the details of the best way to accomplish this.
- How do you get a Unicode character into your source? In part this is specific to your operating system and editor, so is outside the scope of this guide, but we have hints below for popular operating systems.
- You can always place a Unicode character in your source using XML syntax. The first thing an XML parser will do is convert this syntax into a character. The number of the SECTION SIGN in hexadecimal is A7, so the syntax `§`; is identical to the character §. Of course, this will get tedious fast.
- The [Full Unicode Input](http://www.cs.tut.fi/~jkorpela/fui.html) utility at www.cs.tut.fi/~jkorpela/fui.html will allow you to specify a chunk of 256 consecutive Unicode numbers and then you can click on characters to make a string of several or many. You can cut/paste these into your source, or convert the whole lot to XML syntax all at once.
- Unicode characters have standardized names. You can find these, and more information, including font support, at the Unicode section of FileFormat.info. If you are struggling to find a specific character, then using this site's name in a search will often quickly locate what you need. Be sure to experiment with the test pages there for browser and font support (including checking your local configuration).
- **Warning:** do not use Unicode characters as a way to get mathematical symbols (that is delegated to our use of $\text{\LaTeX}$ syntax). And do not use Unicode when we have provided an empty element for a character. These empty elements are conveniences, which spare you from looking up Unicode numbers and make your source more readable. We also sometimes fine-tune these characters in ways that are not possible if you embed them as Unicode. An example is `<times />`, for use outside of a strictly mathematical setting: “I bought a 2×4 at the lumberyard.”

4.36.1 Unicode Support in OSX

Mitch Keller reports on 2017-01-12 a way to get some popular characters with OSX. Use the Keyboard preference pane under System Preferences. In there, you can enable

Show Keyboard, Emoji, & Symbols Viewers in menu bar

Once you activate the keyboard viewer, you get a keyboard on your screen. When you hold down `opt`, it shows you what other symbol you would get if you push `opt+letter`. For instance, `opt+w` gives an upper-case Greek sigma and `opt+=` gives a not-equals sign (neither of which we can handle when processing the latex version of this guide). To get ä, you type `opt+u` and then hit `a`. This is illustrated by the keys for diacritical marks being highlighted in orange while holding `opt`. The shift key can have an effect to produce variations of some characters, such as quote marks (dumb versus smart).

4.36.2 (*) Unicode Support in Linux

4.36.3 (*) Unicode Support in Windows

4.37 Braille Best Practices

This is an evolving list of best practices for authoring (and publishing) so that a conversion to braille is as useful as possible for the blind reader.

Many recommendations for mathematics will be useful to any reader, but perhaps even moreso for a blind reader, so read [Section 4.10](#). Recommendations here may also improve your project for all readers. See [Chapter 33](#) for more about the mechanics of producing output as braille.

Division Numbering. Braille uses various devices to indicate division headings, since font weight, size, and color are not available. These include starting on a new page, centering text, preceding with a blank lines, standard levels of indentation (4 or 6 cells), and combinations of these devices.

In a conversion from PreTeXt, the text of each heading is the number of the division, followed by the title. So the formatting and the presence of a hierarchical number are together good clues that a new division is starting. And the number of parts in the hierarchical number will also serve as a precise indicator of the depth of the division.

As a publisher, you can “turn off” division numbering below some level ([Section 44.2](#)). Think carefully about the impact this will have on a blind reader, since lesser division headings will be harder to recognize without a leading number.

Side-By-Side Layout. A `<sidebyside>` can be a very useful device, but think carefully about its suitability. As of 2022-11-30, we have yet to even handle them carefully in a braille conversion. And if your panels hold images, that is even harder, since we do not have good support for tactile images yet.

In any event, we will likely “unwind” a `<sidebyside>` into a series of its panels running *down* the page, rather than *across*, along with a note about how many panels to expect. So when you author a `<sidebyside>`, consider how this alternate presentation in braille will be received by the reader.

4.38 (*) Testing Sage Examples

4.39 Xinclude Modularization

The `xinclude` mechanism is not part of PreTeXt, *per se*. It is of some use for organizing your authoring, so you do not have mammoth files open in your text editor. As discussed in [Section 5.3](#) there is very little value in modularizing so much that you have many very small files, and also almost no benefit whatsoever to using directory structure to duplicate the inherent tree-like structure of XML. Many small files, or deeply-nested directories, seem to be of little help and can cause more headaches than they are worth.

The `xinclude` mechanism automatically introduces a `@xml:base` attribute, which we need to account for in the RELAX-NG schema ([Chapter 6](#)). So we limit which PreTeXt elements may be the root element of an included file. The rough, general rule is that if an element *can* have a title, then it can be the root element of an included file. So in particular each of the divisions (`<chapter>`, `<section>`, etc.) is a candidate.

One special exception to this restriction is the use of text files, containing absolutely no markup at all. Two good examples are the `<code>` child of a `<program>` or the `<latex-image>` element used to describe an `<image>` by source code that $\text{\LaTeX}$ understands.

In both cases you can put the text content of these elements in a separate file, use the `@href` attribute of `<xi:include>` to point to the file, and then the twist is to set the `@parse` attribute to the value `text`. This has two general benefits. First, you now *cannot* have any XML in the file, so you do not have to have a single root element for the file (and so the schema imposes no restrictions). Second, you do not need to escape any problematic characters like ampersands and angle brackets ([Section 4.4](#)), nor use the misunderstood `CDATA` mechanism.

There is one caveat about this device. Normal XML processing will normalize line endings. Usually this means XML source edited on Windows will have line endings that are two characters, a carriage return (CR) and a line feed (LF). Effectively, this is replaced by one character, the line feed, which is what is common for Linux and Mac.

However, when you use the `@parse` attribute with value `text`, this is now an **unparsed entity** and the normalization of line endings does not happen. The likely result may be a lot of extra blank lines interleaved into your content. See [End-of-Line Handling](#) in the XML specification for details.

Additionally, in the case of `<latex-image>` you can park unsightly code away in files so you do not have to look at it, or you can create a small driver $\text{\LaTeX}$ program to test each one, or even better, you may want to use the same image more than once (maybe in different figures?) and can just include it repeatedly, while only ever editing the single copy.

Finally, the `<input>` and `<output>` children of `<sage>`, and `<console>` as well as the `<code>` of `<program>` are also candidates for this device. In particular, you may want to have the code for a program in its own file where you can test it easily with an interpreter or compiler. There is one gotcha. If you were to put a newline between `<code>` and `<xi:include>` there is the very real potential of unwanted whitespace bleeding into your PreTeXt output. Our suggested remedy uses an example from Bob Plantz. Convert

```
<program language="c">
  <code>
    <xi:include parse="text" href="intAndFloat.c"/>
  </code>
</program>
```

to

```
<program language="c">
  <code><xi:include parse="text" href="intAndFloat.c"/></code>
</program>
```

There are some fancy XSLT tricks you can employ to use more complicated content repeatedly. Your source will be less portable, and we do not support or recommend these techniques. But if you want to go anyway, see hints at www.sagehill.net/docbookxsl/DuplicateIDs.html. Note the reliance on `xpointer()`, and that the final technique is restricted to DocBook, a different XML vocabulary.

4.40 Localization

We briefly introduced PreTeXt support for authoring documents in multiple languages in [Section 3.30](#). Here are some more details.

First, authors are encouraged to write the actual text of their documents using the language of their choice. See [Section 4.36](#) for details. For HTML this should be straightforward, for other output formats (especially PDF) the publisher may need to have various fonts installed (see [Chapter 40](#)).

Localization for PreTeXt itself is restricted to the items PreTeXt names for you. This includes things like `<theorem>`, but also items such as the labels of navigation buttons. To have this in your language requires two things, described in the following subsections.

4.40.1 Localization Files

The prerequisite is that a localization file for your language exists. Fifteen languages are already supported, such as `es-ES` (Peninsular Spanish), `pt-BR` (Brazilian Portuguese), and `hu-HU` (Hungarian/Magyar); the default language code is `en-US` (American English). For a up-to-date list of which languages are available, see [the localization stylesheet folder on GitHub](#).

If such a file does not exist, you will need to create one or find someone to help you do so. Brief instructions for this [are in the README file](#) for this directory. These instructions suggest using the `en-US` file as a template and as a source of more detailed instructions. We highly encourage anyone who creates such a file to contribute it to our growing list of localization stylesheets; see [Section A.4](#) in the Appendices for details.

4.40.2 Changing the Document Language

In order to use a localization, simply place `xml:lang="es-ES"` (with language code as appropriate for your project) as an attribute on your `<pretext>` element. For example:

```
<pretext xml:lang="hu-HU"/>
```

Then you can process your source as usual. Be sure to check that the translations look appropriate in your output. While processing your source, you may encounter warnings for missing translations, like this:

```
MBX:WARNING: could not translate string with id  
"hypothesis" into language for code "pt-PT"
```

In this case, the localization stylesheet is missing a translation for this tag, and it will remain in en-US. We welcome contributions to keep stylesheets fully complete; see [Section A.4](#) for details.

4.40.3 Multilingual Documents

As of the start of 2023, we are improving support to allow for *parts* of a document to be in different languages. Much as you place an `@xml:lang` attribute on the overall `<pretext>` element, you can place this attribute, *with a different supported language* onto any element, and all of the content within that element should react according to the new language. A division would be most natural, but perhaps also a block or a paragraph? Development continues, and in particular not every part of an HTML page always reacts.

4.41 Accessibility

4.41.1 PreTeXt provided accessibility features

Continuing our discussion from [Section 3.31](#) we begin by listing features of our conversion to HTML which happen automatically. These come in part from the recommendations at the [Web Content Accessibility Guidelines](#) of the Web Accessibility Initiative.

HTML Wherever possible we supply HTML elements and attributes that will be interpreted sensibly by a screen reader in the absence of the visual styling provided by CSS. This means we are very careful about the role of headings (h1 through h6) for screenreaders, both for divisions and the block elements they contain. We provide HTML that passes validation checks. And so on. Employing attributes from the [Accessible Rich Internet Applications](#) suite of web standards (ARIA) will go a long way to improving accessibility. This work is on-going, as of 2021-11-03.

Mathematics

MathJax (mathjax.org) is the JavaScript library we use to render mathematics within the HTML output. It provides extensive capabilities for screen readers to render the mathematics audibly, and by default your project's output is configured to take advantage of these features. We refer the reader to the MathJax documentation of [Accessibility Features](#) for details. But here is a simple experiment you can do yourself right now to simulate how a blind reader could experience mathematics with the combination of PreTeXt, MathJax, and a screen reader.

1. Find some moderately complicated mathematics, such as in the “Mathematics” section of the sample article, or your own project, or the sample from MathJax copied below.
2. Bring up the context menu on that display (a mouse right-click for most).
3. Turn on the Accessibility > Explorer > Activate menu item. The page will reload, and the Explorer menu item will earn many more menu items. This setting is reasonably sticky, so you should not have to do this repeatedly. Having this on will incur some processing time as part of each page load, so you may want to turn it off later.
4. Turn on the Accessibility > Explorer > Speech Output menu item.
5. Turn on the Accessibility > Explorer > Subtitles menu item. (If Subtitles is “greyed out”, try toggling Speech Output.)
6. TAB until some mathematics is given focus (a discrete border appears).
7. SHIFT-SPACE will activate exploration of the mathematics with the Explorer. A subtitle, with an aural rendering of the mathematics, will appear below the display.
8. You can navigate (explore) the expression tree with the up, down, left, and right arrow keys. The subtitles will change as you do this.

From the MathJax demonstration page, Maxwell's equations for practice:

$$\begin{aligned}\nabla \times \vec{\mathbf{B}} - \frac{1}{c} \frac{\partial \vec{\mathbf{E}}}{\partial t} &= \frac{4\pi}{c} \vec{\mathbf{j}} \\ \nabla \cdot \vec{\mathbf{E}} &= 4\pi\rho \\ \nabla \times \vec{\mathbf{E}} + \frac{1}{c} \frac{\partial \vec{\mathbf{B}}}{\partial t} &= \vec{\mathbf{o}} \\ \nabla \cdot \vec{\mathbf{B}} &= 0\end{aligned}$$

3D Images Asymptote is a language for describing 2D and 3D images, which we support as much as possible. The 3D images produced are rotatable for exploration via a mouse or finger. For those with motor limitations, the images may also be manipulated with keyboard controls. (Many assistive technologies rely on, or emulate, keyboards.)

Skip to Main Content

Repeatedly pressing the Tab key will move a reader from one location to the next in a web document. Since your Table of Contents in the left sidebar is a series of many links, a reader will need to tab through *all* of these to eventually reach the interesting content on a page.

However, we support a common device. The first link on every page is hidden from all readers, but an initial Tab will present a link labeled Skip to Main Content which when executed will take the reader past the Table of Contents and to the start of the content at the top of the page.

Links

Hyperlinks have colors, styles (such as underlining), and effects (such as mouse hover) which are consistent with WCAG recommendations. Rather than being underlined by default, we instead use high-contrast color choices.

Colors We are sensitive to the fact that some readers have difficulty distinguishing between certain colors. So we do our best to distinguish text, or other elements, without relying exclusively on color. For example, the `<delete>` and `<insert>` elements may render text with strike-through and underlining (respectively) to show the distinction.

But you can help as you author. For example, see [Subsection 4.9.6](#).

Justified Text

Right-justified text (an even right margin) can sometimes lead to spaced-out text that is difficult for some readers. For print, our use of $\text{\LaTeX}$ as an intermediate format, leads to PDF output where right-justified text can be superior to the alternative, **ragged right** text. For more see [Section 30.3](#) and [Section 41.10](#).

Watermarks

If a document has a watermark ([Section 26.5](#)), then a screen reader will announce its presence at the beginning of each page of HTML content.

4.41.2 Author Provided Accessibility Features

Here are features which are PreTeXt helps facilitate, but require your participation as the author.

Image Description

Images you author or supply will be invisible to some readers. Within every `<image>` element you can provide both a `<shortdescription>`, which will migrate to the HTML `@alt` attribute, as well as a `<description>` element (structured with `<p>` and `<tabular>` children) that will be provided as a longer description. The content will be picked up by screen readers.

For the `<shortdescription>`, make the content clear and concise. Do not use any markup whatsoever, just simple characters, and avoid quotation marks, and limit the text to at most 125 characters. You can learn more at sites such as the one provided by the Web Accessibility Initiative (WAI) at [Text alternatives for non-text content](#).

We cannot do this one for you, this is for the author only, as image descriptions must take into account author intent and their context. But we can give you the tools to do it as easily and as correctly as possible.

See [Subsection 4.41.3](#) for further advice on authoring image descriptions.

Image Formats

University offices that provide services for students with disabilities are often interested in the images themselves from a text, as standalone files. For example, they might be able to manufacture tactile versions. You could use the pretext script to produce a variety of different formats and bundle these up in a single archive file for distribution at your book's website. Or you can make each image available through adjacent links placed automatically. We call these "image archives." See [Subsection 4.14.5](#).

Furthermore, as described above, a 3D image authored with Asymptote code can be superior for those with motor disabilities. So this functionality begins with an author's choice to employ Asymptote.

Cross-References

[Section 4.5](#) describes a variety of ways to customize the look and content of a cross-reference. You can create a larger target for clickable items by making the text as long as possible. So for example an `<xref>` authored as

```
<xref ref="theorem-FTC" text="type-global" />
```

would cause the clickable portion to be something like "Theorem 5.16", whereas

```
<xref ref="theorem-FTC" text="global" />
```

would then cause the clickable portion to be simply the much shorter "5.16". Of course, you can set a default style for your entire document, so it is not necessary to continually provide the `@text` attribute.

Link Text Default link text, such as “Theorem 4.15” has been chosen to be informative. But for internal links (`<xref>`) or external links (`<url>`) you can choose alternative content for the clickable portion of the text. Think carefully about your choices here and try to avoid text like “here” or “click here.” For a `<url>`, the default content is the `@href`, which can always be improved by providing content. For formats such as the print version of PDF, the `@visual` is provided automatically as a trailing parenthetical.

Hyperlink Underlining in PDF

In an electronic PDF, hyperlinks are distinguished from surrounding text by color alone, which may be insufficient for readers with color vision deficiencies. A publisher option can add an underline beneath all hyperlinks (`<xref>` and `<url>`), providing a second visual cue independent of color. (A print PDF does not color links, but a publisher may still choose to underline them.) See [Subsection 4.4.3.8](#) for the publisher file syntax.

Commutative Diagrams

Whenever possible, author commutative diagrams using the syntax of the `amscd` $\text{\LaTeX}$ package. Then online and braille output will be more accessible. See [Subsection 4.9.9](#) for more.

Punctuation after Mathematics

Periods, commas, and semi-colons that follow directly after mathematics are handled differently by PreTeXt for visual formats versus non-visual ones (e.g. braille, audio). But this only happens if you author the punctuation in the *logically* correct location and let PreTeXt do the rest for you. See [Best Practice 4.9.3](#) and [Subsection 4.9.14](#) for details.

4.41.3 Advice for Writing Image Descriptions

Oscar Levin

Acknowledgement: the following content is based on a presentation delivered by Michael Cantino in December 2022. Michael has served as the BVIS specialist for Oregon’s K-12 public schools, an accessibility specialist at Portland Community College, and is a Library of Congress certified Braille Transcriber. Many thanks to Michael for sharing his expertise with the PreTeXt community.

Accessibility standards dictate that all (non-decorative) images included in websites are accompanied by a text description which a screen reader can read aloud to a non-sighted user. This **alternative text** (or **alt text**) is also displayed in place of the image if the image fails to load. Essentially alt text is a brief description of what the image is meant to convey. Of course, determining how to concisely convey information of an image can be a challenge for any image, and this is especially true for images you would find in a mathematics or science textbook. The following suggestions and examples are intended to help an author create quality descriptions of their images.

First, a few technical considerations. The usual way a non-sighted user will access a pretext book is through a PreTeXt-generated HTML webpage. Screen readers will read the text contained in the `@alt` attribute of the `<img>` element on the page. (PreTeXt uses the text the author provides in the `<shortdescription>` element to create this content.) Technical limitations of the `@alt` attribute and most screen reader software mean that descriptions presented in this way must conform to particular specifications.

The content of an alt-text description must be plain text and should not exceed a total of 125 characters (although the exact character limit is a matter of some debate). Some screen readers will read more than this, but others will simply stop reading at this point. Further, it is more complicated to get screen readers to navigate alt text compared to how they would navigate the rest of text on the page. Since the alt text is plain text, it cannot include text styles, links, MathML, or other markup.

Tip 4.41.1 VS Code, and presumably most modern text editors, will display the character count of selected text in the status bar.

A separate way to provide information about an image to non-sighted users is to use some sort of **long description**. There is not wide agreement among platforms on how to implement long descriptions, but generally they are available near the image (not assigned to the image directly). In PreTeXt, long descriptions are authored inside a `<description>` element, structured with `<p>` (or `<tabular>`) children. These descriptions *can* include basic markup, including mathematics inside `<m>` tags.

In the following sections, we will consider how to write image descriptions that achieve the goal of providing the same information as the image does. Depending on the particular case, this could be done with a short or long

description. When a long description is needed, the image should still have a short description that give a brief overview of the image and alert the reader to the presence of a long description. The long description will likely repeat the information from the short description (although this is a bit of a style decision).

From this point on, we will not distinguish between the types of descriptions and simply consider what content would be appropriate to include in a description.

Images in Context. To convey the information of an image requires an understanding of what information the image is *intended* to convey. Two identical images can naturally have completely different descriptions depending on what the purpose of the image is. For example, consider the following image and think about how you might describe it.

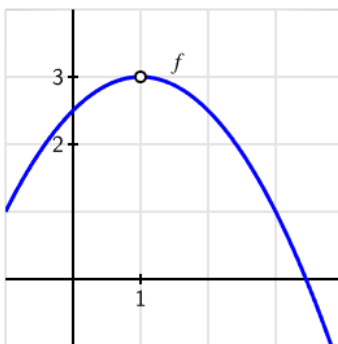


Figure 4.41.2 Graph from *Active Calculus*

The long description I gave to this image was: “The graph of a downward opening parabola labeled f , with vertex at the point $(1, 3)$. At the vertex, the graph shows an open circle, indicating that the function is not defined at this point.” This was informed by what I assumed the author was trying to accomplish with the image: to illustrate various ways that a function might not be continuous. I also know that the textbook has described parabolas as “opening downward,” rather than having negative leading coefficient or some other standard.

But what if this same image was in an exercise asking the reader to determine the equation of the parabola? The description above didn’t mention anything about axes since that wasn’t relevant to the point the author was trying to make. If the reader wanted to find the equation, then we might write a description like the following:

The graph of a function plotted on coordinate axes. Both horizontal and vertical axes range from -1 to 4 . The graph appears to start at the point $(-1, 1)$, crosses the vertical axis between labeled values 2 and 3 , reaches a maximum value of 3 at $x = 1$, and then decreases again, crossing the horizontal axis between $x = 3$ and $x = 4$.

Even this might change depending on the context. Should readers be able to identify whether the graph is a polynomial or a line? Or is the goal to be able to identify what the vertical intercept is from the graph?

Along with the pedagogical context, it is also worth considering the other information that is available to the reader: some of the information in the image may be described in the surrounding text or in the caption of the image. It is not necessary to repeat this information in the description.

Once you have an understanding of what the image is meant to convey, you can start to write the description itself. The following are some general guidelines to keep in mind.

- Give an overview. Before describing various elements in an image, give a brief overview. This will allow the reader to determine whether they need to continue reading the description, or jump ahead. It also orients the reader to the image and helps them understand the context of the description.
- Create *structure*. A reader will need to piece together all the different components of an image, so anything the author can do to facilitate this is helpful. For example, work from left to right, or clockwise from the top, and say that this is what you are doing. Use the relevant structure of an image to group descriptions of related components: “A set of coordinate axes, in which the horizontal axis ranges from -10 to 10 and the vertical axis ranges from -5 to 5 .”

- Be clear and concise. After writing your description, read it back and look for ways to improve brevity, clarity, and structure. This needs to be done while still considering the reading level and vocabulary of the target audience. A nice resource of this is the [Alt Text as Poetry](#) website.
- Leverage the reader’s specialized knowledge. Again with an understanding of your target audience, you may be able to assume they know what the graph of a particular function looks like. For example, an advanced college math textbook could have as a description, “graph of a quadratic function with a vertex at $(2, 3)$ and a y -intercept at $(0, 1)$.”
- Avoid visual shorthand. Sighted authors are used to using comparisons between how things look. Some basic shapes should be okay for most readers: a ball, a cup, U-shaped, etc.

As an example of the structure of a description, consider the recommendations for describing a “chart or graph” (data visualization) provided by the [Do No Harm Guide: Centering Accessibility in Data Visualization](#). Chapters 3 and 4 of that guide are particularly helpful.

Examples. We conclude with a few more examples of images and their descriptions.

Example 4.41.3 Venn diagram.

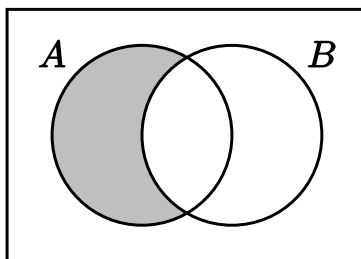


Figure 4.41.4 A Venn diagram from [Discrete Mathematics: an Open Introduction](#)

Short description	Venn diagram with two overlapping circles labeled A and B; region in A not overlapping B is shaded.
Long description	A Venn diagram with two overlapping circles labeled A and B, respectively. Each circle contains a region which does not overlap with the other circle, and a region which does overlap. The region contained in circle A that does not overlap with region B is shaded.
Context	The image is illustrating how Venn diagrams can be shaded to illustrate set operations. This was the forth Venn diagram in the text, as such the second sentence of the long description is likely unnecessary; readers should understand what a 2 set Venn diagram looks like.



Example 4.41.5 Kernel and pre-image.

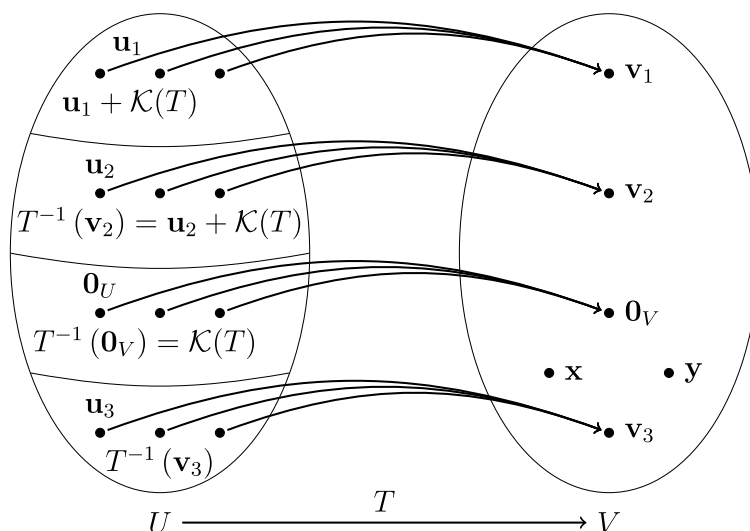


Figure 4.41.6 Kernel and pre-image diagram from [A First Course in Linear Algebra](#)

Short description Two ovals representing vector spaces containing points representing vectors, with arrows associating points.

Long description Two ovals representing vector spaces U and V with arrows connecting points in oval U to oval V , representing the transformation T . Oval U is split into 4 roughly equal sections, each with 3 points; arrows point from each individual point to a shared, corresponding point in oval V . Each section's points connect to a different point in oval V , which contains 6 points in total. In each section of oval U , a label appears above the first point and an expression appears to label the section.

- Section 1: first point is labeled $\mathbf{u}_1$, section labeled with $\mathbf{u}_1 + \mathcal{K}(T)$, arrows point to point $\mathbf{v}_1$ in oval V .
- Section 2: first point is labeled $\mathbf{u}_2$, section labeled with $T^{-1}(\mathbf{v}_2) = \mathbf{u}_2 + \mathcal{K}(T)$, arrows point to point $\mathbf{v}_2$ in oval V .
- Section 3: first point is labeled $\mathbf{o}_U$, section labeled with $T^{-1}(\mathbf{o}_V) = \mathcal{K}(T)$, arrows point to point labeled $\mathbf{o}_V$ in oval V .
- Section 4: first point is labeled $\mathbf{u}_3$, section labeled with $T^{-1}(\mathbf{v}_3)$, arrows point to point labeled $\mathbf{v}_3$ in oval V .

The two remaining points in V are labeled x and y .

Context The image is contained in a section on injective linear transformations in a proof-based linear algebra textbook. The goal of the image is to illustrate the equivalence between images of the linear transformation, their pre-images, and the kernel of the transformation.



Example 4.41.7 Combinatorial Graph (unlabeled).

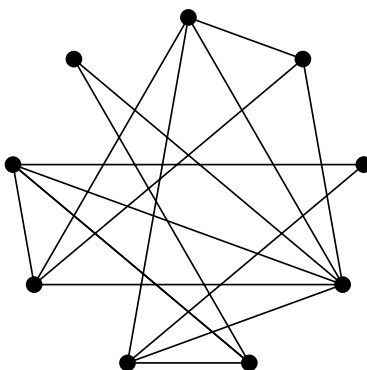


Figure 4.41.8 Unlabeled graph from *Discrete Mathematics: an Open Introduction*

Short description 9 vertices arranged in a circle; edges connect some pairs of vertices. See long description

Long description Graph with 9 vertices arranged in a circle. Edges connect pairs of vertices. Starting at the top and moving clockwise, the vertices have been numbered 1 through 9 so edges can be detailed in the table below.

Vertex	Connected to vertices
1	2, 4, 6, 7
2	1, 4, 7
3	6, 8
4	1, 2, 6, 7, 8, 9
5	6, 8, 9
6	1, 3, 4, 5
7	1, 2, 4, 8
8	3, 4, 5, 7
9	4, 5

Context This is from an exercise in a section on paths in graphs. The question reads, “Below is a graph representing friendships between a group of students (each vertex is a student and each edge is a friendship). Is it possible for the students to sit around a round table in such a way that every student sits between two friends? What does this question have to do with paths?”



Conclusion. Watching a blind reader navigate a web page can be a very enlightening experience. Or you might even undertake learning one yourself. Here are some suggestions for getting started (current on 2018-05-31).

- NVDA, www.nvaccess.org, Windows, open source via GPL
- Orca, help.gnome.org/users, Linux, open source via LGPL
- VoiceOver, included with Apple’s macOS and iOS
- ChromeVox, www.chromevox.com, ChromeOS, free from Google
- JAWS, www.freedomscientific.com, Windows, commercial

Much of the technical work for accessibility is accomplished by PreTeXt developers. But authors have responsibilities, too. So testing can be part of your workflow. One free tool is [axe](#) from [Deque Systems](#).

We have concentrated on making HTML output accessible, since that seems the most natural and best supported.

But we are aware of efforts for other formats.

PDF/L ^A T _E X	A PDF may not lend itself naturally to providing an accessible format. But there have been efforts. The T_EX Users Group on PDF Accessibility and PDF Standards is an excellent resource to start with. There is also a PreTeXt issue #1046 .
EPUB	The International Digital Publishing Forum has information on EPUB Accessibility .

4.42 Slides

Slideshows may be authored in PreTeXt by using the following tags.

- `slideshow` replaces the usual `article` or `book` tag to let PreTeXt know you are authoring a set of slides. As usual, you may then define your title, subtitle, and frontmatter.
- Similar to articles, your content should be organized into several sections.
- And within each section, a `slide` represents a full screen of information. Within a slide, you may author PreTeXt content as usual, such as paragraphs (`p`), lists (`ul`), and so on.
- Several elements support setting the `@pause` attribute to the value `yes` to allow stepping through parts of a slide (e.g. it inserts a Beamer `\pause`).

A minimal working example may be found in the [Examples folder of the PreTeXt repo on GitHub](#). See publishing details for various outputs in [Chapter 34](#).

4.43 Literate Programming

We continue (and do not repeat) the introduction at [Section 3.33](#). PreTeXt implements the literate programming paradigm with two primary elements, `<fragment>` and `<fragref>`.

A `<fragment>` is a chunk of code. Almost always it has a `@xml:id` attribute. It begins with a mandatory `<title>`. It may have index entries, but see below for advice using the `<list-of>` element. Then there is a mix of `<code>` and `<fragref>` elements. The `<code>` element holds the actual text in whatever computer language you are using. Line breaks are respected. Use the XML escape characters `<`; and `&`; if your code needs the `<` or `&` characters. Do not try to use XML markup inside the `<code>` element.

A `<fragref>` is a reference to some other `<fragment>`. This is accomplished via the `@ref` attribute which points to the target via an `@xml:id`. So this is very similar to an `<xref>` ([Section 3.4](#)), but not enough alike to have the same element name. The most important distinction is that the `xsl/pretext-litprog.xsl` stylesheet will *replace* the `<fragref>` with the *contents* of the target `<fragment>`. In other conversions, the `<fragref>` will be a visual expression of the target `<fragment>`, possibly with some active means to visit or examine the target (hyperlink, knowl), more similar to an `<xref>`.

So the `xsl/pretext-litprog.xsl` stylesheet will course through your fragments converting the tree-like structure given by the references in the `<fragref>` elements into a depth-first traversal that will assemble the `<code>` elements, and only the `<code>` elements, linearly into a program. Notice that it is your job as the author of the program to be certain that this rearrangement results in a syntactically correct program. (PreTeXt is good, but not that good.)

If you are with us this far, you are wondering just where the root of this tree is? In other words, just *which* `<fragment>` does this traversal begin with? Good question. At least one `<fragment>` must have a `@filename` attribute. Then a traversal will begin here and your program will be output to the file with the name you specify. You are not limited to one file/root, so if your program has multiple source files they may be documented/collected into a single PreTeXt source file. (Notice this implies that every `<fragment>` *must* have an `@xml:id` or a `@filename`.)

You can place a `<fragment>` most anywhere you might place any other numbered block (such as `<example>`), though we would tend to place them as children of divisions. The remainder of your document can have all the usual PreTeXt features: a table of contents, a preface, divisions, an index, references, etc. Then a conversion to PDF,

HTML, or other formats, will include your code, but in an order that might be more human-readable, and with careful documentation in close proximity.

Automatically migrating each `<fragment>` to an index is a bad idea. (We already experimented.) Instead you can try putting

```
<list-of elements="fragment"/>
```

into an `<appendix>`. Perhaps a pointer early on to its existence will help your reader. This list can be subdivided with the use of the `@divisions` attribute. See [Section 4.29](#). Note that you might still want to provide an index, but remember that its construction is a job for an author ([Subsection 4.27.2](#)).

Literate programming has been developed to support the authoritative RELAX-NG version of the PreTeXt schema. Since RELAX-NG is a declarative language, the rearrangement of code hunks is not quite as critical. But still, see `schema/pretext.xml` for a non-trivial example. As of 2020-11-11 there are a few caveats. Start a discussion on the development forum if you have a need.

- The conversion to a program has no explicit support for languages which interpret indentation meaningfully (e.g. Python). The `<code>` element makes *no changes*, so you could succeed if authored carefully. Some sort of relative indentation attribute might be a good solution.
- We do not syntax-highlight the code. A language attribute might allow us to recycle existing features.
- Numbering is serial from the start of the document. Raise a feature request if you think hierarchical numbering is indicated.

Chapter 5

Processing, Tools and Workflow

This chapter explains in full detail how to convert your source into various output formats, using both the simple PreTeXt-CLI as well as other methods to combine your source file with an XSL stylesheet that might provide greater flexibility and control. It expands on the simple cloud-based workflow in [Chapter 2](#), providing options for a local installation and processing.

5.1 Options for Processing

There are currently three supported options for converting (processing) your source:

1. The PreTeXt-CLI: very easy and friendly, but somewhat limited in customization. Requires Python 3.10 or later. Documentation appears in [Section 5.2](#), and throughout the Guide as needed.
2. The PreTeXt script (sometimes referred to as pretext/pretext): a Python script that has a large variety of utilities to process your source and its components. Requires Python 3.6 or later. This script is very useful for development and for users who prefer a “Swiss Army Knife” approach to their tools. In rare cases this may be needed to test new features that have not yet been exposed in the PreTeXt-CLI. See [Chapter 47](#).
3. xsltproc: an executable program that directly converts XML using a specified XSL file. The command-line program is easily available on Linux and MacOS, but harder to install on Windows (using Windows Subsystem for Linux is probably your best bet). Demonstrations of use in this Guide are being phased-out but will be collected in [Chapter 46](#).

The PreTeXt-CLI, can be installed on the command-line using the command `pip install pretext`, see [Section 5.2](#). To use options 2 and 3 above, you will need to get a copy of PreTeXt from its [GitHub repository](#). This can be done using the command `git clone https://github.com/PreTeXtBook/pretext.git`, or by downloading a zip file directly from the repository.

Wherever the functionality of the PreTeXt-CLI allows it, instructions in this guide will use that method. We will collect corresponding processing instructions using xsltproc in [Chapter 46](#). Information on the use of the PreTeXt script can be found in [Chapter 47](#). These tools are especially useful when developing new features for PreTeXt, so they are housed in the [Developer’s Guide](#).

If you prefer to use online, cloud-based tools, all of the processing options are available when using a Github Codespace or similar online platform. If you already use and have a license for CoCalc, it should have all the tools ready to go (although you will likely want to update the CLI).

5.2 The PreTeXt-CLI

Oscar Levin

Here we will outline the functionality of the PreTeXt-CLI. Instructions for using the CLI without any local installation can be found in [Chapter 2](#). An easy first step toward getting PreTeXt working locally would be to download VS

Code and install the pretext-tools extension (which will install the PreTeXt-CLI for you, assuming you have an appropriate version of python available).

We will work at the **command-line** inside of a **terminal** or **console**. If you do not know what this is, it will seem very primitive at first. This will be called a “Command Prompt” or “PowerShell” in Windows or a “Terminal” on a Mac. In Linux it may be known as a “console” or a “shell”. Whenever the guide says to enter something on the *command line* or in a *terminal*, or just to *enter the command*, this is what we are talking about.

5.2.1 Installing PreTeXt

Before you can install PreTeXt locally, make sure you have the following software on your computer, or else install it using instructions easily searched for online.

- Python, version 3.10 or later. In a terminal, type `python --version` to ensure you are already set up. On MacOS or Linux, your command for python might be called `python3`, so also try `python3 --version`.
- To use an HTML theme other than the default, you will need Node.js, version 18 or later. You can check if you have it installed by running `node --version` in your terminal.
- To produce a pdf, or to generate tikz images from source, you need $\text{\LaTeX}$. Try `xelatex --version` to see if you already have this.
- Any text editor. [Visual Studio Code](#) is an excellent choice (and has an extension, [PreTeXt-tools](#), with specific language support), but other editors such as SublimeText, atom, emacs, vi, etc. can also be used. See [Appendix D](#).

Now we can install PreTeXt. Open a terminal and type the following:

```
$ pip install pretext[all]
```

If this fails, try:

```
$ python -m pip install pretext[all]
```

(or, if `python3` worked above, do `python3 -m pip install pretext[all]`.)

The `python -m` helps in case Python is on your `PATH` but `pip` is not. This is a useful fix for the rest of the commands listed for the PreTeXt-CLI.

Including the `[all]` at the end of the installation command will install both optional dependencies with the CLI: `pelican` for generating a static landing page if you wish to deploy multiple targets, and `prefigure`, to include accessible prefigure graphics. You could install just one of these using `pip install pretext[prefigure]`, for example. If something goes wrong, you can always just include the required dependencies with `pip install pretext`.

Best Practice 5.2.1 Newer Linux distributions and MacOS versions may give an error or warning when using `pip` outside of a virtual environment. It is likely best practice to use a virtual environment, especially if you use python for other projects. Some instructions are available in [Section C.2](#).

Alternatively, you can use `pipx` to install the CLI with `pipx install pretext`, which will create and manage a virtual environment for you.

To verify that the CLI is installed, type `pretext --version` (or `python -m pretext --version`) and you should get back a number (2.13.5, for example).

For quick hints about what you can do, the CLI has built-in help. You can access this by entering `pretext --help` or `pretext build --help` (replacing `build` with `new` or `view` or `generate` etc.).

Upgrading. This documentation will assume you have version 1.0 or later of the CLI. You can upgrade the CLI to the most recent version through PIP using the command:

```
$ pip install pretext --upgrade
```

Versions of the CLI starting with 2.12 also allow you to simply run `pretext upgrade` to get the most recent version.

If you want to experiment with bleeding-edge features, you can install a nightly development version of the CLI by running:

```
$ pip install pretext --upgrade --pre
```

If you ever want to downgrade to previous version, you can do that with pip as well. For example, to install version 2.20.1:

```
$ pip install pretext=="2.20.1"
```

Warning 5.2.2 If you installed the PreTeXt-CLI before version 1.0, you need to manually uninstall the package pretextbook (which is what the CLI was called during early development). Run `pip uninstall pretextbook` and then `pip install pretext` to get caught up.

5.2.2 Starting a New Project: `pretext new`

To generate a new book or article, from the folder in which you want to project located, type:

```
$ pretext new book
```

This creates a new book in the folder “new-pretext-project” (which you can safely rename). For a new article, use `pretext new article`. You might also try `pretext new demo` to get a project that shows off more features available, or `pretext new course` for a collection of course documents. `pretext new slideshow` will create a new project with a simple slideshow template.

Inside the “new-pretext-project” folder, you will find the following folders and files:

assets	A folder to place static (external) assets, such as images or data files, that you will include in your project, for example, with <code><image source="frog.jpg"/></code> .
generated-assets	A folder that will hold assets (such as images) generated from source, using the <code>pretext build</code> or <code>pretext generate</code> commands. You should not manually edit any contents of this folder.
publication	A folder to hold your “publication” files used to customize how your output looks. One publication file is included, but you can have as many as you want. See Chapter 44 .
source	A folder to hold all your PreTeXt source files. These are the main files you will edit to control the content of your project.
project.ptx	An XML file called the project manifest . This specifies options for converting your source into different target outputs. We will describe the contents of this file in Subsection 5.2.7 .
README.md	A file that you can use to describe your project.
requirements.txt	A simple text file that contains the version of the CLI that is initially used to build the project. If you upgrade the CLI, you will be warned to also update the version in this file once you know that things build as expected.

Once you build your project, you will get a folder called `output` and possibly one called `.cache` (which stores cached generated assets). Do not edit the contents of these folders manually; such changes will be overwritten anyway.

Additional files and folders will be generated if your project is managed by git (for example, if you use GitHub). You can expect to see a file `.gitignore` and folders `.devcontainer` and `.github`.

Whenever you upgrade your installation of the CLI, you should run `pretext update` from inside your project folder. Doing so will update any managed files that you have not edited. This can be useful to get the latest features.

5.2.3 Converting: `pretext build`

To convert your source into one of the available output formats, say HTML, run the command:

```
$ pretext build web
```

Here “web” is the @name of one of the targets in the project manifest (project.ptx). To build different targets, replace it with the @name of another target, as in `pretext build print`.

You can also build the first (default) target of the manifest by omitting the target: just type `pretext build`.

5.2.4 Viewing output: pretext view

After you convert your source into, say HTML, you can view the output using the command:

```
$ pretext view web
```

This will direct you to open a file running in a local server (that the CLI started for you) at a provided address (perhaps <http://localhost:8128/output/web>). Once you have run `pretext view` once, you can navigate to other output targets by navigating to a different url (maybe <http://localhost:8128/output/print>) or just rerun the `pretext view` command with a different target name.

Running `pretext view` will attempt to open your default program for the type of file you are opening. You can prevent this attempt using the `--no-launch` flag.

Assuming you ran this command in a terminal, the easiest way to continue working is to open a new terminal tab to run `pretext build` again; the local server will continue in the original terminal until you stop it with `CTRL+C`.

Warning 5.2.3 When using `pretext view`, your project is only viewable on your local machine (even if you are working in a Codespace or on CoCalc). To make the output available to the public, you will need to copy the output to a web server, or use the `pretext deploy` command, described below.

5.2.5 Hosting your project: pretext deploy

When you are ready to share your project with the world, you can copy the contents of the `output/web` (or whatever you called the output of your html build) to any webserver. A convenient free option is to use [GitHub Pages](#), and the CLI makes this especially easy.

If you already track your source files using GitHub, all you need to do is enter the following command:

```
$ pretext deploy
```

If you watch the terminal, you will either get directions for how to set up your repository, or will simply be told what the live website for your project is.

Behind the scenes, this command copies the contents of the output folders you want to deploy to a folder `output/stage`, and then moves the entire contents of that folder to the `gh-pages` branch of your github repository. If you have not already created a `gh-pages` branch, the CLI will create one for you. If you have not already set up your repository to track your source files, the CLI will walk you through the steps you need to get set up.

Tip 5.2.4 If you don’t have your project set up with GitHub yet, the CLI will try to walk you through the process.

Sometimes authentication with GitHub can be tricky. Especially on Macs, it appears that conflicts can arise if you use different methods for authenticating in different applications. A fix that seems to work is to open Keychain Access, search for “github” and delete some or all of the entries you find (GitHub api seems to be a culprit). You will need to reauthenticate then, but after that, it should work.

Tip 5.2.5 You can deploy multiple targets, starting with the 2.0 release of the CLI. To do this, you will need to specify a `@deploy-dir` attribute in your project manifest for each target you want to deploy. Additionally, you should create a folder in the root of your project called `site` and create a landing page for your project. The `site` folder and the output folders for each specified project will all be copied to `output/stage`. We recommend using the flag `--stage-only` so you can preview you complete site before deploying it.

5.2.6 Generating assets: pretext generate

Some PreTeXt elements require special processing: `WeBWorK` exercises, `<prefigure>`, `<latex-image>`, `<sageplot>`, and `<asymptote>` images, as well as previews for embedded youtube videos or interactive elements. We call these elements “assets” or “generated assets”.

Prior to version 1.7 of the CLI, you would need to run `pretext generate -t [target]` (where “[target]” is the name of a target, like “web”) in order to generate assets. Since then, you can still do this, but it is usually not

necessary: whenever you build a target, the CLI will automatically generate any assets that are out of date. If you want to force the generation of assets, you can use the `-g` flag when building (e.g., `pretext build web -g`). If you really don't want to automatically regenerate assets, you can use the `--no-generate` flag when building (e.g., `pretext build web --no-generate`).

If you want to generate assets without building (or if you are using an older version of the CLI), the following instructions can help you limit what you generate.

- You can limit which sorts of assets you generate. For example, if you edit a few `<latex-image>` elements, you can just generate these (for the first target in the manifest) with `pretext generate latex-image`. To get the same assets for a different target, use the `-t` flag (e.g., `pretext generate -t print prefigure`). If you want to generate all assets for a specific target, use the `--all-assets` flag (e.g., `pretext generate -t web --all-assets`). This will generate all assets for the specified target, regardless of whether they are out of date or not.

Valid choices for what types of assets you can generate in this specific way are: `webwork`, `prefigure`, `latex-image`, `sageplot`, `asymptote`, `interactive`, `youtube`, `code-lens`, and `datafile`.

- To be even more precise, if the element you wish to generate has an `@xml:id`, you can generate just that element using the `-x` flag. For example, if your `<latex-image>` has id `"img-circle"`, generate it with `pretext generate -x img-circle`
- Finally, there might be times you would like to get all output formats for the assets you are generating. You can accomplish this using the `--all-formats` flag on `pretext generate`. Different output formats require different asset formats; to get just those formats for a particular target, you can use the `-t` flag to specify a target name.

5.2.7 The project manifest: `project.ptx`

The project manifest, always named `"project.ptx"`, contains information about each **target** you will build. Prior to version 2.0 of the CLI, it also contained the names of **executables** of external tools that might be needed for building targets and generating assets. (In 2.0 and later, if you need to change the default names or paths to executables, you will use a separate `executables.ptx` file.)

The CLI looks for this file, so you should have only one. If you have a project that was not created using pretext new, you can get a copy of the file by running `pretext init`. You can also get the most recent template version of the manifest by running `pretext init --refresh`, which will create a new versions of the files (and create a `.bak` backup version for those you have modified so you can compare then and update accordingly).

While we use the `.ptx` extension, the manifest is not technically a PreTeXt document, since it does not agree with the schema. However, it must have a specific structure to be used with the CLI. An example of a simple manifest is given in [Listing 5.2.6](#). Note this is version 2 manifest; for a comparison of "legacy" manifests and the current format, see [Appendix P](#).

Listing 5.2.6 Example of a very simple project manifest.

```
<?xml version="1.0" encoding="utf-8"?>
<project ptx-version="2">
  <targets>
    <target name="web" format="html" />
    <target name="rs" format="html" publication="publication-runestone.ptx" />
    <target name="print" format="pdf" />
  </targets>
</project>
```

The manifest contains a single `<project>` element with a `@ptx-version` attribute. This attribute is required and must be set to `"2"` for the CLI to recognize the file as a valid manifest. Additional optional attributes of `<project>` can also be specified, which we will describe below. The `<project>` element has a single child, `<targets>`, which contains a list of `<target>` elements.

Each `<target>` must have a `@name` attribute. The name (e.g., `web`, `print`) is what you specify when you run `build`, `view`, or `generate` for a specific target (so `pretext build web`, `pretext view web` or `pretext generate -t web`).

The second required attribute of `<target>` is `@format`, which must be one of `html`, `pdf`, `latex`, `custom`, `epub`, `kindle`, or `braille` (although the last three of these are still experimental, as of 9/1/2023).

With just these attributes, as with the manifest in [Listing 5.2.6](#), the CLI assumes your project uses default values throughout. The following defaults are constructed from the default value for an attribute of the root `<project>` element with the default value for an attribute of a particular `<target>` element. Each of the two pieces can be overridden. The net default folders are:

- The source is `source/main.ptx`.
- The publication file is `publication/publication.ptx`.
- Output is stored in `output/` (with each target in a subfolder identical to the target's name).
- The staging directory to preview deploys is `output/stage`.

Again, each of these can be modified using the appropriate attribute of either the root `<project>` element or a particular `<target>` element.

In [Table 5.2.7](#) and [Table 5.2.8](#) we give all the attributes you can have on the `<project>` and `<target>` elements, respectively. Some attributes can be specified for both elements. In such cases, the values will be paths, and the path given in the `<target>`'s attribute will be relative to the path given in the `<project>`'s attribute. This can be useful, for example, if all targets have publication files in the same folder, but the files for some targets are different.

All paths described in [Table 5.2.7](#) are given *relative to the root of your project* (the location of the `project.ptx` file).

Table 5.2.7 Attributes available for the `<project>` element.

Attribute	Description
<code>@ptx-version</code>	Required. Must have value 2.
<code>@source</code>	Optional, default: <code>source</code> . Path to folder holding main source file.
<code>@publication</code>	Optional, default: <code>publication</code> . Path to folder holding publication file.
<code>@output-dir</code>	Optional, default: <code>output</code> . Path to folder in which output files/folders for each target will be created.
<code>@site</code>	Optional, default: <code>site</code> . Path to folder holding user-provided landing page for deploying multiple targets.
<code>@stage</code>	Optional, default: <code>output/stage</code> . Path to folder where deployable targets will be collected before they are deployed.
<code>@xsl</code>	Optional, default: <code>xsl</code> . Path to folder holding custom xsl files.
<code>@asy-method</code>	Optional, default: <code>server</code> . Valid values: <code>server</code> or <code>local</code> . Used to specify whether asymptote images should be generated using the server asymptote version or a local asymptote install.
<code>@generated-cache</code>	Optional, default: <code>.cache</code> . Path to folder holding cached versions of some generated assets.
<code>@cname</code>	Optional, default: <code>None</code> . The custom domain you want to use for a deployed target through github pages. Only needed if you set a custom domain.

Any file paths described in attributes of a `<target>` element are relative to a corresponding attribute value from the root `<project>` element.

Table 5.2.8 Attributes available for the `<target>` elements.

Attribute	Description
@name	Required. The name you use when executing a CLI command.
@format	Required. Valid values: html, pdf, latex, epub, kindle, braille, webwork, and custom. The format the target will be built into.
@source	Optional, default: main.ptx. Path to the root source file of the project (relative to the value of the @source of <project>).
@publication	Optional, default: publication.ptx. Path to publication file (relative to the value of @publication attribute of <project>).
@output-dir	Optional, default: the value of @name. Path to folder in which output files/folders will be created (relative to the value of @output-dir attribute of <project>).
@output-filename	Optional, default: generated by pretext. Only valid for formats that produce a single output file. Path to output file to be built (relative to the value of @output-dir of the same <target> element).
@deploy-dir	Optional, no default. Path to subdirectory of deployed site where this target will live. If deploying multiple targets, then this attribute must have a value for it to be deployed.
@xsl	Optional, no default. Required when @format is custom. Path to custom xsl file (relative to the value of @xsl attribute of <project>).
@latex-engine	Optional, default: xelatex. Valid values: xelatex, pdflatex, or latex. Only used on targets that build with latex, to specify what latex command to call in that step.
@latex-source	Optional, default: no. Valid values: yes or no. Only used on pdf format targets. If yes, the LaTeX source file will be included in the output directory. Equivalent to using the --latex flag with pretext build.
@braille-mode	Optional, default: emboss. Valid values: emboss or electronic. Only used when @format is braille, to specify the mode for braille.
@platform	Optional, no default. Only valid when @format is html. Valid values: runestone. Used to specify that the target will be hosted on Runestone.
@compression	Optional, no default. Only valid when @format is webwork or html and @platform is not runestone. Valid values: scorm or zip. Results in output being compressed (as .zip file), and if the value is scorm, the required scorm manifest is also include in the archive.
@asy-method	Optional, default: server. Valid values: server or local. Overrides the @asy-method attribute on <project>.
@standalone	Optional, default: no. Valid values: yes or no. If yes, the target will be built as a standalone document. This will place the output adjacent to the source file, unless @output-dir is specified. Useful for creating a target for which you will specify a different source file (using the -i flag when building).

Example 5.2.9 Example Project Manifest. To illustrate how the relative paths work, with attributes on both `<project>` and `<targets>` elements, consider the following manifest:


```
<?xml version="1.0" encoding="utf-8"?>
<project ptx-version="2" source="src" output-dir="out">
  <targets>
    <target
      name="web"
      format="html"
      output-dir="web" />
    <target
      name="print"
      format="pdf"
      source="main-print.ptx"
      publication="pub/publication.ptx" />
  </targets>
</project>
```

Here, the web target, with format html, will build from the root PreTeXt file "src/main.ptx", using the publication file "publication/publication.ptx", and will place the output in the folder "out/web".

The print target, with format pdf, will build from the file "src/main-print.ptx", using the publication file "publication/pub/publication.ptx", and will place the output in the folder "out/print". ▲

In addition to the attributes available for <target> elements, there is one optional child element, <stringparams>, which can be used to specify string parameter options (see [Section 28.1](#)). These are specified using an attribute for the name of the string parameter, and the value of the attribute is the value of the string parameter. Multiple string parameters are set using multiple attributes on the single <stringparams> element. For example,

```
<stringparams author.tools="yes" html.css.extra="external/custom-style.css" />
```

5.2.8 Executables

PreTeXt uses several external tools to build and generate assets. The CLI will automatically find these tools if they are installed in the default location for your operating system. If you have installed them in a non-standard location, you can specify the path to the executable in a executables.ptx file, at the root of your project. In [Listing 5.2.10](#) we give the defaults and formatting of such a file.

Listing 5.2.10 Example of version 2 executables file

```
<?xml version="1.0" encoding="utf-8"?>
<executables
  latex="latex"
  pdflatex="pdflatex"
  xelatex="xelatex"
  asy="asy"
  sage="sage"
  pdfeps="pdftops"
  node="node"
  liblouis="file2brl" />
```

Warning 5.2.11 Prior to version 2 of the CLI, these executables were contained in the project.ptx file. If you are using a version prior to 2.0, see [Appendix P](#).

5.2.9 Using PreTeXt outside of a project

Most of the time you will want to set up a project folder that will hold all the files you need for your project, including the project.ptx and publication.ptx configuration files, your possibly modular source files, and external assets. However, it is also possible to use PreTeXt to process a single file, similar to how you might with L^AT_EX. Starting with the CLI version 2.19.0, this can be done easily.

Suppose a colleague sends you a file mypaper.ptx which you save to your ~/Desktop folder. You can process this file using the following command.

```
$ pretext build pdf -i ~/Desktop/mypaper.ptx
```

This will create a `mypaper.pdf` file in the `~/Desktop` folder (i.e., adjacent to the source `.ptx` file).

You can also get portable HTML output using `pretext build html -i ~/Desktop/mypaper.ptx`, which will create a folder `mypaper_html` in the `~/Desktop` folder. Note that often a portable HTML build will result in a single `.html` file, so this can easily be shared with others or uploaded to a web server.

5.2.10 Getting help

In addition to checking `pretext --help`, `pretext build --help`, `pretext view --help`, etc, you have a few options when you run into trouble. If you are getting errors that don't make sense, even after trying follow the suggestion of the error messages, look at the time-stamped log files inside the `logs` folder (or the `"cli.log"` file if you are using a CLI version prior to 2.0), which includes debug-level log messages. You can also run the CLI using this higher level of verbosity using the `-v debug` option, which must go right after `pretext` command, before the subcommands (`build`, `deploy`, etc.). That is, enter the following for example,

```
$ pretext -v debug build web
```

You can also ask for help on the [google support group](#). When you post there, please run `pretext support` from inside your project's folder and copy the output into your help request.

5.3 Modular Source Files

For a large project, such as a book, you will likely want to split up your source into logical units, such as chapters and sections. The PreTeXt-CLI supports an inclusion mechanism automatically (see [Chapter 46](#) for what you need to do different for `xsltproc`).

Suppose your book on animals has a chapter on mammals with a section on monkeys. Then you need to do the following:

1. For the file containing the `<chapter>` tag for the chapter on mammals, place the attribute

```
xmlns:xi="http://www.w3.org/2001/XInclude"
```

on the outermost tag in the file.

2. Within the `<chapter>` element for the chapter on mammals, add the line `<xi:include href="monkeys.xml" />` to "pull in" the section on monkeys at that location. The `@href` attribute can point to a file in a subdirectory, but will be interpreted relative to the location of the file containing the mammal chapter element.

Several comments are in order.

- Begin small and start a project *without* using modular files. Modularizing seems to add a layer of complexity that sometimes obscures other beginner's errors. So get comfortable with a single source file before branching out.
- The XML specification requires that a source file only contain a single outermost element. So for example, two `<chapter>` elements cannot go into the same file as simultaneous outermost elements.
- There will always be a "main" file that contains the `<pretext>` element as its single outermost element. In this Guide we will call this the **top-level** file.
- Any file that uses an `<xi:include>` element will need the `xmlns` declaration on the outermost element. So in our animal book example, the top-level file, which presumably includes several chapter files, would need this declaration on the `<pretext>` element.
- In practice, there is not a lot to be gained by creating a subdirectory structure mirroring your modularization—all your source files can go into one big directory and the XML hierarchy will take care of the organization. I do sometimes like to name my files accordingly, so for example `chapter-mammals.xml` and `section-monkeys.xml`.
- When you validate your source (see [Section 5.4](#) and [Chapter 6](#)) you will always point to the top-level source file (the one with the `<pretext>` tags).

The book generated by `pretext new demo` has modular source, so is a nice starting point to see how this works. Other examples are the sample book in `examples/sample-book` amply demonstrates different ways to modularize parts of a project (but in no way should be taken as best practice in this regard). This guide, in `doc/author-guide` is a simple example of modular source files, and might be a good template to follow for your book. See [Section 4.39](#) for some of the finer points of this topic.

5.4 Verifying your Source

A **schema**, in our case a RELAX-NG schema, is a formal specification of an XML vocabulary (the allowed tags and attributes), and how they relate to each other. So, for example, the restrictions that say you cannot nest a `<book>` inside of a `<chapter>`, nor can you nest a `<subsection>` in a `<chapter>` without an intervening `<section>`, are expressed and enforced by the schema. One of the beauties of the schema is that it is written using a very specific syntax and then there are tools that use a schema as input. In particular, a PreTeXt source file that conforms to the PreTeXt schema is said to be **valid**. You should strive to always, always, always have valid source files, and therefore you want to regularly verify that this is the case.

You can find the PreTeXt schema in the [schema folder of the GitHub repository](#). The version we author and maintain is `pretext.xml`, which is used to create `pretext.rnc`, which uses the compact syntax of the RELAX-NG specification. By providing the schema and your source to a program called a **validator** you can check if your source is valid, and if not, why. See [Chapter 6](#) for the details on doing this.

The PreTeXt-CLI also has some basic validating capability: after running `pretext build`, open the `.error_schema.log` file in the main folder of your project to see validation errors.

If you author source that is valid PreTeXt, then a conversion of your source to another format should succeed. And maybe in the future, somebody will create a new conversion to a new output format, and your source should still produce faithful output, with no extra effort from you. Think of the schema as a contract between authors of source files and developers of converters. This is different than performing a conversion and getting good-looking output—that can just be a happy accident and your source may not succeed with some other conversion.

We cannot stress enough the importance of setting up and performing regular validation and preventing many consistent errors of the same type. You will learn what elements are allowed where, and which are not, from the messages produced by validation errors. And when a conversion fails, or produces spectacularly incorrect output, validating your source should be your first reaction. Always.

The other beauty of a schema is that you can supply it to a text editor ([Section 1.2](#)) and then you will get context-sensitive help that greatly assists you in using only the tags and attributes that are allowed in a given location of your source. [XML Copy Editor](#) is the one editor like this we have tried, but we do not have extensive experience.

We have devoted an entire chapter ([Chapter 6](#)) to amplifying this introduction and providing more details, such as where to find details on installing a validator.

5.5 Customizations

There are some aspects of your output that are entirely divorced from the actual content, and are presumably all about how that content is presented. Two good examples are the size of the font used in $\text{\LaTeX}$ /PDF/print output, and the granularity of web pages in HTML output (by this we mean, is each web page a whole chapter, a whole section, a whole subsection?). Producing output with varying values of these parameters does absolutely nothing to change your content in any way, and so should not be a part of your writing, nor a concern while you concentrating on your writing.

Therefore, many aspects of how your writing is *presented* is accomplished by your publisher. Note that this is very different from *your* role as the author of your project. When your project is mature then you can consult with your publisher about how to best present your project to your audience. An entire part of this guide is devoted to this process ([Part IV](#)), but if you are curious, the first part of [Chapter 26](#) can serve as an introduction.

5.6 Directory Management

Organizing your source files, external assets (like images) and other support files, can become a challenge as your project grows. PreTeXt supports managing external and generated files (including images), but this requires some setup and understanding of the correct way to specify assets in your source. This section explains both those steps.

If you are starting a new project, using the PreTeXt-CLI (with the command `pretext new book`, for example), then most of the setup portion is done for you.

5.6.1 External and Generated Files

Early in your writing project, you will decide you want to add images, embedded YouTube videos, interactive demonstrations, or other enhancements that are more than just words on the page. Some of these objects will be created outside of your PreTeXt project, such as a photograph. But some images are described within your PreTeXt project, such as a diagram authored using the $\text{\LaTeX}$ TikZ package, and PreTeXt will help generate different versions of the diagram in different file formats for use in output formats that are not $\text{\LaTeX}$. For an embedded YouTube video we provide tools that will automatically get you a thumbnail preview image from a YouTube server, which will then appear in your non-interactive PDF version. We describe the photograph as **external**, since it comes from “somewhere else,” independent of anything you authored in your source. In contrast, an SVG image of your TikZ diagram for HTML output, and a preview image of a YouTube video for use in a print version, are described as **generated** since they are dependent on what you have put in your source, and PreTeXt automates almost all of their creation for you.

As these files are added to your project, you want to organize them in a specific (but flexible) way. First, make a directory (folder) for your *external* files. You can use any name you like, including `external`. Within reason you can place it wherever you want. Natural choices are as a peer of a source directory that holds your PreTeXt files, or as a subdirectory of your project’s top-level directory that may hold all your PreTeXt files. You can also organize this directory with subdirectories of your own choice, if that helps you stay organized.

Similarly, you need a separate directory for your *generated* files. As above, it can be named anything, including `generated`, and you can place it almost anywhere (close by). But now, it must have a precise directory structure, described below in [List 5.6.2](#), according to what sort of generation produced the files.

After you have read this section, see the discussion of generated and external files in [Subsection 27.2.4](#) for some good examples of why this flexibility is useful.

5.6.2 Managing Directories

The name, and location of the external and generated directories (both of them, always) are then specified in a publication file (see [Subsection 44.7.1](#) for the precise specification). The values of these attributes are *relative* pathnames to the directories, relative to the location of the *main* PreTeXt file for your project. Let us take a look at an example.

Example 5.6.1 Sample Directory Management. Suppose you are M. Jones, the author of the AOTA project, and so you might have the following directory and file structure. There is some sort of overall path from the root of your entire filesystem to your project, so on Linux this path could be

```
/home/mjones/books/
```

and on other operating systems it will be slightly different, but those differences are not relevant here. Inside of `books` there would be an `aota` directory for the AOTA project, structured as follows. We are most interested in the `ext` and `gen` directories.

```
aota
  source
    aota.ptx
    alligators.ptx
    dogs.ptx
  publication
    print.ptx
    online.ptx
    epub.ptx
```

```

ext
  photos
    slow-alligator.jpeg
    fast-whippet.png
  movies
    alligator-chases-whippet.mp4
  data
    deer-weights.csv
gen
  latex-image
    marsupial-life-cycle.svg
  youtube
    sloth.jpg

```

In your publication files you would then have the entry, as a sub-element of the source element,

```
<directories external="../ext" generated="../gen"/>
```

Notice that we have deliberately named our directories `ext` and `gen` as part of this illustration, so that they are not identical to the *attribute* names on the `<directories>` element. The two attribute values are *relative* to the location of the main PreTeXt file, in this case `aota.ptx`. The two periods, `..`, mean to go up a level, here to the `aota` directory, then the slash indicates a step down to either `gen` or `ext`. Note that `latex-image` and `youtube` within `gen` *must* be specified exactly that way. Now every PreTeXt production tool can deduce where your files are.

How do you specify in your PreTeXt source which file to use where? Suppose you want your two (external) photographs to be used in a `<sidebyside>` element. You would author

```

<sidebyside widths="50% 30%">
  <image source="photos/slow-alligator.jpeg"/>
  <image source="photos/fast-whippet.jpeg"/>
</sidebyside/>

```

Notice that the values of your `@source` attributes *do not* include `ext` or `../ext`. This gives you the freedom to move your `ext` directory or rename it, and then you only change the publication file and not your source. The decision to have the subdirectories `photos` and `movies` are not easily changed, since they will be in your source, but they may have value for the organization of your project.

The chase video is of your own creation, you have included it as part of your project's files, and will be hosting it on your web server, where the rest of your HTML version is made available for viewing. You might author

```

<figure xml:id="fig-chase">
  <caption>Alligator chases whippet</caption>
  <video source="movies/alligator-chases-whippet.mp4"/>
</figure/>

```

The situation with the generated files is presently a bit more complicated, but will eventually be transparent to an author. The marsupial life cycle diagram has been authored in your source using the language from the $\text{\LaTeX}$ TikZ package, which is delimited as a `<latex-image>` element. When building a PDF from $\text{\LaTeX}$ output this is a no-brainer—PreTeXt just puts your TikZ code into the $\text{\LaTeX}$ file. But how about HTML output? The pretext/pretext script will manufacture a standalone version of this image and then convert it to an SVG, and this version will work very well in the online version of your project. What do you author?

```

<figure xml:id="fig-life-cycle">
  <caption>Marsupial Life Cycle</caption>
  <latex-image xml:id="marsupial-life-cycle">
    -- some TikZ code here --
  </latex-image>
</figure/>

```

With the right options to the pretext/pretext script, the file `marsupial-life-cycle.svg` will be generated. Presently, you must explicitly tell the script to place the output in the directory

```
/home/mjones/books/gen/latex-image
```

but we plan to make this the default behavior. Or you could generate the file anywhere, and move it to the `latex-image` subdirectory of `gen`. Notice how much of the process of creating a companion SVG image (which excels in a web browser) is automated, and how the `@xml:id` attribute and the directories in the publication file are used to keep everything coordinated. The `latex-image` directory is not negotiable, it is the home for *every* image authored using a PreTeXt `<latex-image>` element (see [List 5.6.2](#)).

The slow sloth video was created by somebody else and posted to YouTube as a super slo-mo 240fps video. You authored it within a `<figure>` simply by using the ID of the video on YouTube.

```
<figure xml:id="fig-sloth">
  <caption>Slow Sloth</caption>
  <video xml:id="sloth" youtube="DJWEqYcxUl8"/>
</figure/>
```

With the right options to the `pretext/pretext` script, the file `sloth.jpg` will be generated by requesting a thumbnail preview from YouTube's servers. Note how this content is generated from your source (precisely, the YouTube ID) through an automated process. Presently, you must explicitly tell the script to place the output in the directory

```
/home/mjones/books/gen/youtube
```

but we plan to make this the default behavior. The `youtube` directory is not negotiable, it is the home for *every* server-generated preview image for an embedded YouTube video (see [List 5.6.2](#)). (Note that any `<video>` may have an author-provided image as its preview, and then this would be considered an external file.) ▲

As you can tell from the above, different output formats have different expectations for additional files and when producing an output, there may be different expectations for where exactly where to find files. A good example is the EPUB format, which despite being basically an HTML format, is still very rigorous about names, formats, and locations for image files. With the above procedures you provide just enough information for PreTeXt to handle the remainder of the complexity for you, without you becoming an expert with the EPUB standard. So the `pretext/pretext` script can automatically produce PDF, HTML, EPUB, Kindle, and braille, each in a single step.

We have skirted one finer point of all this. How does an author know if a file is external or generated and what are the supported subdirectories of the generated directory? A PreTeXt `@source` attribute will almost always point to a file that belongs in the external directory. An exception is the `@preview` attribute which is used to specify static images to use in outputs like PDF or print to represent more dynamic objects employed in more-capable electronic formats (videos, audio, interactives). (Please alert us to other exceptions!) The list below describes the subdirectories of the generated directory. The files that belong in these directories are all generated by the `pretext/pretext` script using aspects of your authored source.

List 5.6.2 Subdirectories of the Generated Directory

asymptote	Images described by Asymptote code in an <code><asymptote></code> element.
dynamic_subs	XML files needed for dynamic fill-in-the-blank exercises.
latex-image	Images described by $\text{\LaTeX}$ code in a <code><latex-image></code> element.
mermaid	Images described by Mermaid code in a <code><mermaid></code> element.
prefigure	Images described by PreFigure code in a <code><prefigure></code> element.
preview	Static images of dynamic content, created with automatic screenshots via a headless web server.
problems	MyOpenMath problems in static PreTeXt forms.
qrcodes	QR codes for PDF files that point to live versions of some interactive elements.
sageplot	Images described by Sage code in a <code><sageplot></code> element.
trace	Program trace data for <code><code lens></code> elements

youtube	Static images for YouTube videos, automatically downloaded from YouTube servers.
----------------	--

Just one more obscure situation to address, requiring a peek under the hood. Occasionally there are external files used in ways that PreTeXt cannot “see” them and provide the management necessary. One example is a file of data points used to generate a plot using the `<latex-image>` element. Suppose we have a file

```
/home/mjones/books/ext/data/deer-weights.csv
```

as shown in [Example 5.6.1](#). We will use this inside of a `<latex-image>` element employing PGF code with a `table {}`; construction. As part of building the $\text{\LaTeX}$ output, this file will be copied to a temporary directory as

```
external/data/deer-weights.csv
```

Note that your `ext` has been converted to `external` as files are copied—that is not a typo. PreTeXt is not going to examine all your code and try to guess where you have requested a file. So you need to author

```
\addplot table {external/data/deer-weights.csv};
```

using the filename created after the copy. Then the $\text{\LaTeX}$ compilation will locate the file successfully. The `external` directory after the file copy is not negotiable, that is what is always used.

5.6.3 Backward-Compatibility and Migration

Directory management was added in the middle of 2021. For projects existing at that time, it is an “opt-in” feature. In other words, existing projects can still be built by moving additional files in the same ways as always, so long as you do not have a `<directories>` element in your publication file.

However, it is absolutely necessary to opt-in to directory management (via the publication file) if you want to build EPUB or Kindle formats. You will also then be able to build PDF and HTML output with a single invocation of the `pretext/pretext` script, without any extra effort to place additional files in the right places.

New projects should definitely adopt directory management as their project develops. New features and tools will assume this. We now describe how existing projects can migrate.

Previously there was not a firm distinction between `external` and generated files, though they were specified differently in PreTeXt source. Many authors placed both types of files into a single directory, and since the default for locating generated files was a directory called `images`, this was often the single directory employed. Thus, `external` files would be specified with a `@source` attribute such as

```
<image source="images/bobcat.jpg"/>
```

A solution, if a bit clunky, is to make a new directory named `ext` and copy `images` and its contents to become a subdirectory of `ext`. Then place a relative path to `ext` in the `@external` attribute in your publication file. This means you will not have to edit your PreTeXt source at all. At some later time, you could remove all references to `images` in your source and move the files themselves directly into `ext`.

For generated files it will be a bit more involved. You need a new directory, such as `gen`, and you need to add it to your publication file with the right relative path. Then you will need to create as many of the relevant directories from [List 5.6.2](#) as necessary, as subdirectories of `gen`. Populate these subdirectories with the relevant files, which may have been located in `images` before you started migrating. There was a string parameter, `directory.images`, that could be used to specify an alternative directory to the default, `images`. That parameter is now announced as deprecated, but will still be effective until your project opts-in.

As another short-term option for Mac or Linux users, it is possible to take previously existing resource directories and make symbolic links from a previous image directory, such as `my_images`, to the names provided here. In that case, assuming the directory structure provided here, a command like `ln -s my_images external` (issued in the directory containing `my_images`) would create the correct symbolic link.

5.7 Creating Images with Inkscape

A strength of PreTeXt is the ability to create diagrams and images with editable source code, embedded alongside your other PreTeXt source material ([Subsection 4.14.3](#), [Section 5.8](#)). But sometimes you want, or need, to be more artistic. [Inkscape](#) is a great tool for creating images. It ticks all the boxes: open source, mature, cross-platform, standards-compliant. Bethany Llewellyn helped research Inkscape capabilities for this section.

Inkscape’s native file format is Scalable Vector Graphics (svg). As the name suggests, this translates into excellent support for vector graphics. (See [Subsection 4.14.1](#) and [Subsection 4.14.2](#) for the distinction.) Why use Inkscape?

- svg files are small and scale smoothly when you zoom in on a web page. So this format is our top choice for how images are realized in the HTML output. They will also work well in formats based on HTML, such as EPUB.
- The svg format is how Inkscape works with images internally. The file format it calls “Inkscape SVG” is valid svg, along with additional information to make editing better and/or easier. So any author may come back and easily edit an image saved in this format.
- Inkscape will export easily and efficiently to Portable Document Format (PDF), another vector format. This is our top choice for realizing images in L^AT_EX output, which will become print or PDF output.
- svg is an XML format, just like PreTeXt.

When would you use Inkscape?

- Your image is more free-form and artistic than what a source language like TikZ or Asymptote will provide, and you do not need to embed any mathematical symbols.
- You do not want to learn some new source language (there *is* a significant learning curve for most of these).
- You want to annotate screenshots (see [List 5.7.1](#)).

Note that when you save an image as svg from within Inkscape, you have two choices: “Plain SVG” and “Inkscape SVG”. The latter contains additions to the former which make subsequent editing better and/or easier. So, you can choose to use “Plain” as the format used for your HTML output, and it will be smaller and provide great performance. But you can also make an “Inkscape” version of the same name, and distribute it with your source files in a different directory, so you and others can edit it easily later.

List 5.7.1 Annotated Screenshot

Suppose you want to include a screenshot of some application, and point out some aspect of it.

1. View the application on the highest-resolution monitor you can get your hands on. Not a laptop and not a phone.
2. Zoom in, so the area of interest fills as much of the screen as possible. You are trying to capture as many pixels as possible.
3. Use a screen-capture tool that allows you to save the result as Portable Network Graphics (png). This is a lossless format, and you may be able to elect a high level of compression as part of the file-saving process.
4. The only raster format explicitly supported by Inkscape is png. So you can import the screenshot into Inkscape, and use the tools there to add arrows, red ovals, etc. to highlight key parts of the image.
5. Save the result as png, since it already has a significant raster component, for use in HTML conversions. And save again as pdf for use in print and pdf output.

5.8 Images Described in Source

We believe it is important to preserve a record of how diagrams and other graphics are produced. This can be easy when a graphics language is employed to describe the graphical elements, rather than creating a bit-mapped image with some other interface. So we have `<asymptote>`, `<latex-image>`, and `<sageplot>` for elements holding code to produce diagrams or images.

The upside to this is that small edits to the code can easily accomplish minor changes or corrections necessary for the images. The $\text{\LaTeX}$ macros provided by an author can be used in the text *and* in a diagram, leading to greater consistency between the two. Finally, starting from source, we can do the best possible job of producing image formats that are compatible with the document output formats and which scale smoothly in PDFs and in web browsers.

The downside is that processing these images requires various “helper” programs, such as `pdflatex` and `sage`. This requires installing the helper program and perhaps setting the path to it on your system. This is done inside the `project.ptx` manifest as children as the `<executables>` element.

Whenever you need to process images described in your source, run `pretext generate`. This will build the correct image output file for your specified target (use `-t [target]`). If you would like to build additional image formats, you can add `--all-formats` to the command.

5.9 Author Tools

While your writing project is getting underway, you may want to go in several directions at once. We have two devices, and three reports, which can help you manage this.

You may want to make a forward-reference to some future, not-yet-written material. So you can go

```
<xref provisional="a reminder of future material"/>
```

in your source. In your output, you will get a temporary place-holder of sorts.

Comments in the source code of a computer program, labeled `TODO`, is a common device to help a programmer remember tasks that need to be completed. You can use a similar device in your PreTeXt source. Use an XML comment, delimited by `<!--` and `-->`, and make the first four non-blank characters spell `todo`, using any combination of lower- and upper-case you like. Your Author’s Report (next) will look even better if you follow that with a colon and a space, but this is not required. So, for example, go

```
<!-- ToDo: include a section on salamanders and their life-cycle -->
```

As an XML comment, you can place this anywhere. Contents need to be plain characters, no XML will be active here. Remember to escape the two XML characters, and also be aware that `--` is banned in comments outside of the delimiters.

Use the `author.tools` parameter set equal to `yes` and your $\text{\LaTeX}$ and HTML output will be annotated. (See [Section 28.1](#) for more on parameters.) Provisional cross-references and todo-comments will be visible and highlighted, and in particular, the $\text{\LaTeX}$ output will display an abundance of extra information (maybe too much). The $\text{\LaTeX}$ -specific publication file entry for draft mode will activate the $\text{\LaTeX}$ draft option (see [Subsection 44.3.12](#)). The intent here is to make a rough draft, for an author or collaborator only, reporting as much as possible that is incomplete, pending, or hidden, in the usual output.

For users of `xsltproc`, the `authors-report.xsl` stylesheet, found in the `xsl/utilities` directory, will report all of the provisional cross-references and all of the properly prefixed todo-comments. See [Section 46.8](#).

5.10 Keeping Your Source Up-to-Date

Once in a while it becomes necessary to adjust how the PreTeXt vocabulary is arranged, which involves adding or removing elements or attributes, or changing their behavior. When elements or attributes are removed, or their relationships with other elements change, we say that certain items or behaviors are **deprecated**. Fortunately, we can often automate the changes.

When there is a deprecation, a warning is added so that any conversion will report the presence of the old use in the console. Sometimes we can preserve the old behavior, so there is no rush to make changes to your source.

Sometimes a change needs to be more urgent. And frequently old behaviors do not get updates or bug-fixes. Our warnings provide advice and information about what you need to do. There are also announcements on public discussion groups, clearly marked as deprecations. Also, the schema will change as part of any deprecation, so the old elements or old use will be reported.

For some suggestions about how to automate the process of adjusting your source, see [Section 46.9](#).

5.11 Testing HTML Output Locally

Certain complicated parts of HTML output will not always function when you look at PreTeXt output by just opening files in your web browser. These include knows, Sage cells, and YouTube videos. This is a consequence of security policies and so will vary from browser to browser. A solution is to run a web server on your own machine, which is much easier than it sounds.

In fact, this is exactly what the PreTeXt-CLI does when you type the following command.

```
$ pretext view web
```

You can replace “web” with any target name, although it is really HTML builds that benefit from this feature. The CLI uses python to spin up a local web server so you can see a copy of your output by going to the URL the CLI gives as output (usually <http://localhost:8128>, although if you are working in a GitHub codespace, this will be different).

There are two useful options that can be used with `pretext view` to speed up authoring. First, the `-b` flag will build your source before starting the server (i.e., runs `pretext build` in the background). Similarly, the `-g` flag will generate assets prior to viewing.

It is also possible to specify whether your server is public or private (on your local network) with the `-a` option, and to specify a port other than 8128 with the `-p` option. (Run `pretext view --help` for more.)

Another option, in the case where you know where your output lives (such as when you use the `pretext/pretext script`) is to use Python itself to start a simple web server. Ideally, first set your working directory to the location of your HTML output. Then in a console, at a command-line:

```
$ python3 -m http.server
```

```
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
```

Which tells you to open your browser using the address <http://0.0.0.0:8000/>. Ctrl-C in the terminal will kill the server when you are done. See the [Python documentation](#) for more options.

5.12 Testing HTML Output Globally

Just like testing files locally can sometimes be misleading, testing on your own server can sometimes be misleading. For example, on some university campuses, frequently accessed content (your textbook!) can be cached. So when you update your files, it might not look like anything has changed. An easy, free, quick solution is to place your HTML temporarily at [Netlify Drop](#) as a double-check on the source of any problems.

This can also be a great solution if you do not have a server setup and want help from the PreTeXt community with your HTML output. (See [Section A.3](#).)

5.13 Doctesting Sage Code

Adding computer code to your textbook is a tricky proposition. You can propose that it is merely an illustration, and not meant to have all the necessary details, or you can make it exact, correct and executable, and then risk inevitable changes to render your code obsolete. At least you have the option of editing and reposting online versions quickly and easily.

One of our main motivations for this project was mixing in code from the powerful, open source, mathematical software project, Sage ([Section 3.17](#)). When you add example Sage code to illustrate mathematical ideas, you are then encouraged to also include expected output in the `<output>` element. Here comes one of the powerful advantages of XML source and XSL processing.

This process currently requires `xslproc`, and is described in [Section 46.11](#).

The `pretext/xsl/pretext-sage-doctest.xsl` stylesheet, used in the usual way, will create one, or several file(s), in *exactly* the format Sage expects for automated testing. So all your words are gone, and all your Sage input and output is packaged so Sage can run all the `<input>` and compare the results to the expected `<output>`. See [Subsection 44.1.1](#) for details on obtaining more than one file.

We have many years' experience testing hundreds of non-trivial Sage examples from textbooks, for linear algebra and abstract algebra. Roughly every six months, we discover ten to twenty examples that fail. Frequently the failures are trivial (usually output gets re-ordered), but some are significant changes in behavior that leads us to re-word surrounding guidance in the text, and in a few cases the failures have exposed bugs introduced into Sage. It has been relatively easy to do this maintenance on a regular basis, and if it had not been done, the accumulated errors would be enough to greatly degrade confidence in the accuracy of the examples.

Exact details for this process can be found in [Section 4.38](#). Note that Sage is really just a huge Python library, so it might be possible to test pure Python code with this facility, but we have not tested this at all. Similar support for other languages can be considered if requested for use in a serious project.

Chapter 6

PreTeXt Vocabulary Specification

This Guide, along with the sample article and sample book distributed with the PreTeXt source, provide a wealth of examples of how to author in PreTeXt. However, at some point, you will undoubtedly encounter a situation where some of your text fails to appear in your output or the PreTeXt-CLI produces an error. Those are good moments to start investigating the formal specifications of the PreTeXt vocabulary, as most likely you tried to use something in a way incompatible with those specifications. This chapter will help you understand, and work with, the formal specification of PreTeXt.

6.1 RELAX-NG Schema

A **schema** is a set of patterns which describe how the elements of a language may be combined. The PreTeXt vocabulary is described by a RELAX-NG schema, which is included in the PreTeXt distribution. (RELAX-NG stands for “REgular LAnguage for XML Next Generation” and is pronounced “relaxing”.) In general terms, the schema tells you which elements are available, which attributes they may carry, and which other elements they may contain. You can then infer where you can place an element. The schema also indicates if an element is required or optional, if it may be repeated, or if it needs to appear in a prescribed order with other elements, and may limit attribute values.

Besides providing a concise formal description of the PreTeXt vocabulary, your XML source and the RELAX-NG schema can be provided to tools which will automatically **validate** your source against the formal definition. The best validators will provide accurate and helpful messages about errors in your source. Further, some editing tools will use a schema to provide context-sensitive assistance on available elements and attributes as you write, sparing you typographical errors, misplaced elements, and the need to frequently context-switch in order to consult reference material.

The schema does not tell you anything about how an element or attribute will behave. But hopefully there is not much ambiguity about the behavior of the content of a `<title>` element nested within a `<chapter>` element. You would not be surprised to see that content duplicated in the Table of Contents of a book. The purpose of this guide, and other documentation, is to help you understand what to expect. It is better to think of the schema as a contract between you and the developers of conversion tools. If your source conforms to the schema, then a conversion tool will produce reasonable output that conveys the structure and meaning of your writing. Twenty years from now, when GEF is the dominant document format, a conversion of your source will preserve your meaning, while also taking advantage of the amazing features of GEF. (GEF stands for “Great Electronic Format of the Future”.)

In summary, the RELAX-NG schema

- is the formal specification of the PreTeXt vocabulary,
- is a key input to validation,
- can be incredibly helpful in the editing process, and
- provides guidance to implementors of conversions.

As such, we are very deliberate about changes, and hope over time to make changes only very rarely.

6.2 Validation Plus

The RELAX-NG schema is very good at specifying “parent-child” relationships, in other words, which elements can nest directly under/within other elements. But we have situations where the possible elements depend on grandparents, great-grandparents, or older ancestors. An example is the `<var>` element, which is only useful if it is contained *somewhere* within a `<webwork>` element. You can describe these situations with RELAX-NG, but it becomes cumbersome and redundant. So our strategy is to allow some prohibited situations in the RELAX-NG schema, and use an additional stylesheet to identify the prohibited situations. Continuing our WeBWorK example, the RELAX-NG schema makes it appear that `<var>` can be used many places, but the “validation-plus” stylesheet provides a helpful message indicating you have used it outside the context of a WeBWorK problem.

You have put a lot of time and effort into your source, and we want to help you make the best possible output. A little more effort from you will allow us to make the fine distinctions that produce really high-quality output. So this stylesheet is our best attempt to help you make the very best possible source. It is full of (automated) advice and warnings.

To use this stylesheet, simply apply it at the command-line with `xsltproc` like any other stylesheet.

```
xsltproc -xinclude -o report.txt
    schema/pretext-validation-plus.xsl aota/animals.xml
```

The output will be a text file that indicates the suspect element by its location in the document tree.

You may get lots of output on first use, especially if your source was born “somewhere else,” not meant for use by PreTeXt. We could make improvements in managing all this output, but for now we have one suggestion. Sorting on the actual messages really, rather than a hodgepodge of messages in document order, can help you identify consistent situations that you might be able to fix in bulk. First, apply the stylesheet again, but now use the `stringparam single.line.output` set to the value `yes` (Section 28.1). As you suspect, this will put all the output on one line, and the message text will be in the third “field”, which can be used by the command-line utility `sort`,

```
cat report.txt | sort -k 3 > report-sorted.txt
```

We once used Schematron for this purpose. Its author, Rick Jelliffe, says “Schematron is a feather duster to reach the corners that other schema languages cannot reach.” Our additional stylesheet is similar.

Why do we have two tools for validation? We have explained the necessity of an *extra* stylesheet. Why not describe the entire grammar in this stylesheet? The reason is that RELAX-NG is a recognized standard, and so can be converted to other formats, and may also be utilized by XML editors or integrated development environments (IDE) to provide features like code completion. Besides, it would be very tedious to provide all the code for checking everything that is possible and everything that is not.

6.3 Versions of the Schema

The schema is born within a PreTeXt document, `schema/pretext.xml`, where surrounding text provides documentation and guidance on implementation. The literate programming support in PreTeXt (start at Section 3.33) is used to produce a file, `schema/pretext.rnc`, which is a RELAX-NG specification in compact syntax. HTML and PDF versions are available on the PreTeXt website as documentation. We provide some guidance below on reading the compact syntax.

The compact syntax is a faithful representation of the more verbose XML syntax. And vice-versa, so it is possible to translate back-and-forth between the two syntaxes. In practice, we convert the compact version to the XML version, producing a file `schema/pretext.rng`. Some tools require this latter (100% equivalent) version. We perform this conversion with `trang`, an open source program written by James Clark, one of the authors of RELAX-NG. (`trang` stands for “TRANslator for relax NG”.) The compact syntax is often indicated as `RNC` and the XML syntax is often indicated as `RNG`.

XSD (XML Schema Definition), from the World Wide Web Consortium (w3c), is an alternative to the RELAX-NG syntax. It cannot express as many situations as the RELAX-NG syntax, but we have created the PreTeXt schema in a way that `trang` can convert to an XSD specification without introducing any more-permissive “approximations.” But the XSD version seems to have a few small inaccuracies, and in particular should not be used for validation. That said, `schema/pretext.xsd` may be useful for tools (e.g. editors) that require it.

The files `pretext.xml`, `pretext.rnc`, `pretext.rng`, and `pretext.xsd` are all provided in the schema directory under revision control, and are updated by the `schema/build.sh` script when changes are made to `pretext.xml`. So as an author, you do not need to install or run `trang` and should just link to the (continually updated) copies in your `pretext` directory.

We once provided a **document type definition** (DTD) as a description of the PreTeXt vocabulary. [Mitch Keller](#) wrote an excellent initial version of this chapter to help others understand similar principles in the context of the DTD. However, the DTD was not sufficiently flexible to handle elements that behave differently in different locations, such as an `<introduction>` to a `<chapter>` versus an `<introduction>` to an `<exercisegroup>`. As further evidence, `trang` will knowingly refuse to convert the PreTeXt schema to a DTD since the DTD syntax is not sufficiently expressive to describe PreTeXt.

If you are interested in conversions, more tools can be found at relaxng.org and we have information on installation in [Appendix E](#). We would be pleased to learn more about authors' experiences with other converters.

6.4 Reading RELAX-NG Compact Syntax

The compact syntax might remind you of Java or C++ syntax. We do not provide a tutorial here, but do provide some hints on the various delimiters and special symbols, which may make it fairly easy to see your way through to the meaning. The fundamental object of the schema is a **pattern**, and **named patterns** can be reused in order to reuse common patterns and modularize the description. One approach to validation is to remove portions of your source that match required patterns until only optional material remains. Notice that if you were to chase your way through substituting the named patterns with their employment, you would have a single (large) pattern which every possible PreTeXt document would match, and by definition an XML document that does not match is not PreTeXt. (OK, that is a slight exaggeration, see [Section 6.2](#).)

Table 6.4.1 RELAX-NG Compact Syntax Summary

<code>element foo { }</code>	Define <code><foo></code> and children
<code>attribute bar { }</code>	Define <code>@bar</code> and values
<code>text</code>	Sequence of characters (any length)
<code>mixed { }</code>	Mixed-content, characters interleaved in pattern
	Exactly one (required)
<code>+</code>	One or more (required)
<code>*</code>	Zero or more (optional)
<code>?</code>	Zero or one (optional)
<code>,</code>	Sequence, in prescribed order
<code> </code>	Choice
<code>&</code>	Sequence, any order
<code>()</code>	Grouping
<code>=</code>	Define named pattern
<code> =</code>	Accumulate named pattern as a choice

6.5 Validation

We cannot stress enough the importance of validating your source early and frequently. Error conditions and messages can be built into processing (we have some anyway), but they are much better accommodated by tools built for just this purpose. If your processing with the PreTeXt-CLI suddenly fails, or if chunks of your content fail to materialize, it is highly likely that a validation check will tell you where the problem lies. If you integrate regular checks into your workflow, then sudden problems can be traced to recent changes. (Perhaps paired with using `git bisect`, in the extreme. You are using revision control, aren't you?)

We use `jing` for the first step, RELAX-NG validation. This is an open source companion to the `trang` converter described above. As a Java program, it should be cross-platform. It is also packaged for Debian and Ubuntu Linux, and is available in a GitHub codespace. It provides messages keyed to the line number and character number of

your source, and the messages are very clear and informative. See notes on installation in [Appendix E](#). We would be pleased to learn more about authors' experiences with other validators.

You might get a lot of error messages the first time you use `jing`. If so, it might be that many of them are the same situation. If you pipe the output of `jing` through `sort -k 2` then the output will group similar messages together.

Previously, we used `xsltproc` as the XSLT processor. We no longer recommend this option for authors, who should be using the PreTeXt-CLI. However, if you still use `xsltproc`, then you likely automatically also have the `xmlint` program, which will perform validation with RELAX-NG schema. Our experience is that it bails out at the first warning, or at least does not process the remainder of the current subtree, and that the error messages are often very misleading. We will not support questions about validation based on output from `xmlint`.

The second step is easier, since it is an XSL transform. In other words, it is just another stylesheet, which you run against your source, with a processor like `xsltproc`. This stylesheet encoding additional restrictions is unique to PreTeXt and will report exceptions that are too difficult to express with RELAX-NG. So validation is two complementary steps. See [Section 6.2](#) for the exact syntax for using this stylesheet.

6.6 Schema Browser

Through the efforts of [Jason Siefkin](#), and other contributors, we have a **schema browser** that lets you easily and naturally explore the schema through web pages, many with example uses. Look for a link in the [Documentation section](#) of the PreTeXt website.

Do not be intimidated by the list of roughly 300 elements available in the “All Tags” panel. Many are configuration options, many are special-purpose, and many you will never use. (Someday we will do lexical analysis on a substantial range of PreTeXt texts to see just which elements do get used most frequently). We hope to document elements which have complicated restrictions that cannot be expressed with the parent-child relationships of the schema as part of this tool.

An excellent way to contribute to PreTeXt is to help flesh out aspects of the schema browser with more, and better, information for all authors.

6.7 Editor Support for Schema

We collect summary information about editors that make use of schema. See [Appendix D](#) for more specific information, links, etc..

Table 6.7.1 Schema Support in Editors

Editor	Formats	Notes
Visual Studio Code	RNG	Validation and completion, open source
emacs	RNC	Schema-sensitive editing, open source
XML Copy Editor	RNG, XSD	Validation, “tag completion”, open source
oXygen	RNC, RNG	Validation and completion, commercial

Chapter 7

WeBWorK Exercises

Alex Jordan

With a WeBWorK server (version 2.16 or higher, or webwork-ptx.aimath.org) and a little setup work, you can embed WeBWorK exercises in your PreTeXt project. HTML output can have interactive problem cells or print problems in “static” form. PDF output will print static versions. And all such exercises can be archived into a file tree to be uploaded onto a WeBWorK server for use in the “traditional” way.

Although we expect that version 2.16 and above are supported, we recommend that you use the [current](#) (or one previous) version of WeBWorK whenever possible. Should bugs be discovered, it may be difficult to address them with older versions of WeBWorK.

See [Chapter 38](#) for how to configure a WeBWorK course and server to process your WeBWorK problems, and then how to process them.

7.1 WeBWorK Problems

You must extract WeBWorK content as described in [Chapter 38](#) before you will be able to see any WeBWorK content in your output. For most cases, you should be able to do this by running `pretext generate webwork` to process WeBWorK exercises.

A `<webwork>` tag must be inside an `<exercise>` or a PROJECT-LIKE block, *optionally* preceded by an `<introduction>`, and *optionally* followed by a `<conclusion>`.

```
<exercise label="mylabel">
  <introduction>
</introduction>

  <webwork>
</webwork>

  <conclusion>
</conclusion>
</exercise>
```

There are several methods for putting content into the `<webwork/>`. (Note that an empty `<webwork>` with no attributes will simply produce the camelcase WeBWorK logo.)

7.1.1 Using an OPL WeBWorK Problem

If a problem comes from the WeBWorK Open Problem Library and is accessible from the host course in the usual way using a `Library/...` path, then you can include it with a `@source` attribute. For example:

```
<webwork source="Library/.../foo.pg"/>
```

Warning 7.1.1 Not Every PG Problem is Compatible with PreTeXt. Some problems that work fine within WeBWorK are not compatible with PreTeXt. Typically, these are exercises that use older PG coding techniques. To be compatible with PreTeXt, all of the macros used by a problem must be updated to give PreTeXt output. We have done this with modern PG macros and macros that are frequently used. But still, not every PG problem is compatible with PreTeXt. In some cases, maybe it would be straightforward to train the macros to give PreTeXt output. But in many cases, older macros and problem files are not structured well and PreTeXt is all about good structure. So it could be a significant project to retrofit PreTeXt compatibility.

If you elect to use a problem that is incompatible with PreTeXt but you don't yet know that, a few things could happen. One is that when you process the problems to gather static representations, you will get an error message that the problem did not return valid XML.

Or you might coincidentally get valid XML back, but something is just missing or wrong. There's no automated check for that; you should read the output to check that the problem is complete. We recommend reading your PDF output with hints, answers, and solutions exposed, to be sure you are seeing the static version of the exercise.

If there is an incompatible problem that you would really like to use, you have three options:

- Author the problem in PreTeXt from scratch as described elsewhere in this section.
- Edit the code for that problem to use compatible macros.
- Edit the incompatible macros to be compatible with PreTeXt.

The last two options may involve contributing to repositories such as the OPL and PG on GitHub, so it is more expedient to use the first option.

7.1.2 Local PG Source

You can include PG source for a problem directly, as text content for the `<webwork>` element. For example:

```
<exercise>
  <webwork>DOCUMENT();

loadMacros(qw(PGstandard.pl PGML.pl PGcourse.pl));
$pi = Real("pi");

BEGIN_PGML
Enter a value for [\pi].

[_]{$pi}{5}
END_PGML

ENDDOCUMENT();
</webwork>
</exercise>
```

Note the attention paid here to whitespace. Any indentation or newline whitespace within the `<webwork>` will be kept in the PG file that results from this method. Such whitespace may affect the functionality of the problem.

If you have a problem as a PG file, you can place it in your project's folder tree and use `<xi:include>` to insert the PG code as text into the `<webwork>` element:

```
<webwork><xi:include parse="text" href="pg/my_problem.pg"/></webwork>
```

Two things to note. The file where you use `<xi:include>` has a root element, and that root element needs to declare the `xi` namespace. So you need to have an attribute `xmlns:xi="http://www.w3.org/2001/XInclude"` on that root element. Second, the `@href` specifying the path to the PG file needs to be relative to the file you are working in. In the example, there is a folder named `pg` adjacent to the file we are working in, and the PG file is within that folder.

7.1.3 Perl-free Problems

If you'd just like to rattle off a quick question with no randomization, you can do as in this example.

```
<exercise label="mylabel">
  <webwork>
    <statement>
      <p>
        <m>1+2=</m><var name="'3'" width="5"/>
      </p>
    </statement>
  </webwork>
</exercise>
```

The <exercise> above could be given an optional <title>, <introduction>, and <conclusion>. The <webwork> inside could be given a <hint> and <solution>. These are discussed in [Subsection 7.1.4](#).

In the above example, '3' is the @name attribute to a <var> element. There is actually no “variable” named “3”; we are just using the slot where more complicated exercises would place a Perl variable answer.

So the above is how to create an answer blank that is expecting 3 as the answer. What you give as a @name attribute will be passed to PG's Compute() constructor, so it needs to be valid input for Compute(). Note that you could pass a string encased in quotes, or a perl expression. Just be mindful of the differences. For example:

- 8^{**2} will process a perl real using exponentiation and lead to the MathObject Real 64.
- $8^{^2}$ will process a perl string and lead to the MathObject Real 64.
- 8^2 will process the perl real using bitwise XOR and lead to the MathObject Real 10.

The default context is Numeric, which understands numerical expressions and formulaic expressions in the variable x . You can activate some other context as in this example.

```
<exercise label="mylabel">
  <webwork>
    <pg-code>
      Context("ImplicitPlane");
    </pg-code>
    <statement>
      <p>
        The answer is <m>x+y=1</m>.
      </p>
      <p>
        <var name="'x+y=1'" width="8"/>
      </p>
    </statement>
  </webwork>
</exercise>
```

Many special contexts are automatically detected by PreTeXt, and it loads the appropriate macro file into the PG problem. However you may need to explicitly load a macro file as described in [Subsection 7.1.4](#).

7.1.4 PG code in Problems

To have randomization in problems or otherwise take advantage of the algorithmic programming capabilities of Perl and WeBWorK's PG language requires using a <pg-code> tag. Having at least a little familiarity with coding problems in WeBWorK is necessary, although for simpler problems you could get away with mimicking the sample article in pretext/examples/webwork/. A <statement>, *optional* <hint>, and *optional* <solution> follow.

```

<webwork>

  <pg-code>
  </pg-code>

  <statement>
  </statement>

  <hint>
  </hint>

  <solution>
  </solution>

</webwork>

```

If you are familiar with code for WeBWorK PG problems, the `<pg-code>` contains lines of PG code that would appear in the “setup” portion of the problem. Typically, this is the code that precedes the first `BEGIN_PGML`. If your code needs any special WeBWorK macro libraries, you may load them in a `<pg-macros>` tag prior to `<pg-code>`, with each such .pl file’s name inside a `<macro-file>` tag. However many of the most common macro libraries will be loaded automatically based on the content and attributes you use in the rest of your problem.

Here is a small example. Following the example, we’ll continue discussing `<statement>` and `<solution>`.

```

<webwork>
  <pg-code>
    Context("LimitedNumeric");
    $a = Compute(random(1, 9));
    $b = Compute(random(1, 9));
    $c = $a + $b;
  </pg-code>

  <statement>
    <p>
      Compute <m><var name="$a"/> + <var name="$b"/></m>.
    </p>
    <instruction>
      Type your answer without using the <c>+</c> sign.
    </instruction>
    <p>
      The sum is <var name="$c" width="2"/>.
    </p>
  </statement>

  <solution>
    <p>
      <m><var name="$a"/> + <var name="$b"/> = <var name="$c"/></m>.
    </p>
  </solution>
</webwork>

```

Within a `<statement>`, `<hint>`, or `<solution>`, reference perl variables using `<var>` tags with `@name`.

Within the `<statement>`, a `<var>` tag with either a `@width` or `@form` attribute creates an input field answer blank that expects the variable with that `@name` to be the answer.

A `<var>` can have `@form` with value `essay`, in which case it need not have a `@name` attribute. This is for open-ended questions that must be graded by a human. The form field will be an expandable input block if the question is

served to an authenticated user within WeBWorK. But for the WeBWorK cells in PreTeXt HTML output, there will just be a message explaining that there is no place to enter an answer.

A `<var>` can have `@form` with value `array`. You would use this when the answer is a Point, Vector, ColumnVector, or Matrix MathObject to cause the input form to be an array of smaller input fields instead of one big field.

A `<var>` can have `@form` with value `popup`, `buttons`, or `checkboxes` for multiple choice questions.

If you are familiar with PG, then in your `<pg-code>` you might write a custom evaluator (a combination of a custom answer checker, post filters, pre filters, etc.). If you store this similar to

```
$my_evaluator = $answer->cmp(...);
```

then the `<var>` can have `@evaluator` with value `$my_evaluator`.

An `<instruction>` is specific instructions for how the reader might type or otherwise electronically submit their answer. Contents of an `<instruction>` will be omitted from print and other static output forms. The `<instruction>` is a peer to `<p>`, but may only contain “short text” children.

Some general information on authoring WeBWorK problems can be found in a [set of videos at webwork.maa.org/wiki/Problem_Authoring](https://webwork.maa.org/wiki/Problem_Authoring). Not all of this is relevant to authoring within PreTeXt, but there are parts that will be helpful for constructing the Perl code necessary for randomized problems.

7.1.5 WeBWorK in an `<exercisegroup>`

An `<exercisegroup>` is a collection of exercises with common instructions that are put into an `<introduction>`. If you put WeBWorK exercises in an `exercisegroup`, then when the exercises are exported to `.pg` problem files for use as online homework from a WeBWorK server it makes sense that the instructions from the `<exercisegroup>`’s `<introduction>` should be included in the `.pg` file. And so they are included there. Note that they are *not* included when you are building HTML or L^AT_EX output for your project. (Rather, the `<exercisegroup>`’s `<introduction>` appears in its normal place.)

You should be aware of this when you write the `<exercisegroup>`’s `<introduction>`. It impacts the specific language you should use. For example, if you write “Differentiate the following functions.” or “Differentiate each of the functions below.”, then you have language that doesn’t fit the individual problem when it is used for homework on a WeBWorK server. Instead you might write “Differentiate the function”. It makes sense as common instructions for the `<exercisegroup>` as well as the instructions for an individual exercise.

7.1.6 Metadata

A `<webwork>` *without a source attribute* can have a plain text `<description>`. This should be a summary of what the exercise asks a user to do, including any relevant pedagogical details of the exercise. For example:

```
<webwork>
  <description>
    Add two fractions with distinct one-digit prime denominators.
  </description>

  ...

</webwork>
```

A longer description may be broken into lines where the lines are plain text.

```
<webwork>
  <description>
    <line>
      Add two fractions with distinct one-digit prime denominators.
    </line>
    <line>
      One fraction is always positive, the other always negative.
    </line>
```

```
</description>
```

```
...
```

```
</webwork>
```

The content of the description will be written into a PG COMMENT command, making the description visible in a WeBWorK Library Browser.

7.1.7 Reusing a <webwork> by @xml:id

If a <webwork> has an @xml:id, then another <webwork> can “copy” the first one simply by using a @copy attribute whose value is the first <webwork>’s @xml:id. The @seed of the first <webwork> is ignored, and the second <webwork> may set its own @seed. For example:

```
<exercise label="mylabel">
  <webwork xml:id="foo" seed="1">
    <pg-code>
      $a = random(1, 9);
      $answer = $a+1;
    </pg-code>
    <statement>
      <p>
        Enter <m><var name="$a"/>+1</m>. <var name="$answer" width="10"/>
      </p>
    </statement>
  </webwork>
</exercise>

<exercise label="myotherlabel">
  <webwork copy="foo" seed="2"/>
</exercise>
```

The @copy attribute should point to a <webwork> that has PreTeXt-authored source, not to a <webwork> with a @source attribute. (If you want to copy one with a @source attribute, just reuse the same @source value.)

7.1.8 Images

7.1.8.1 Using a Local image file

Planned, not implemented yet.

7.1.8.2 Using TikZ

In a problem statement (or hint, or solution), you may use an <image> with a <latex-image> child. See [Subsubsection 4.14.3.2](#) for generalities about using <latex-image>.

However, if a <latex-image> is inside a <webwork>, you must use $\dots$ instead of $\$ \dots \$$ to encase any inline math in the image. And if you have display math, that should use $\left[\dots \right]$ instead of $\$ \$ \dots \$ \$$.

Your latex-image code can use variables that you defined in the <pg-code> section. Scalar variables may be used simply by using their perl names including the dollar sign sigil. For example, $\text{\draw} (\$a, \$b) -- (\$c, \$d)$; where \$a, \$b, \$c, and \$d are variables. However, any instance of $\backslash \$$ in code will be interpreted as an escaped dollar sign first, so rare situations may require you to be more explicit. The alternative is to include variables just as you would anywhere else in a problem statement, using a <var> element like <var name="\$a"/>. You would also need to use a <var> element if you would like to insert a perl array, for example <var name="@a"/>.

In perl, \$, @, and % each have special meaning. So you may wonder about using them in your latex-image code. The short answer is that you should use them just as you would use them in a regular $\text{\LaTeX}$ document. So when you would like a dollar sign, write $\backslash \$$. For a percent sign, use $\backslash \%$. An “at” character does not need escaping, so write @.

As mentioned above, do not use a dollar sign to encase math. If you want to put a $\LaTeX$ comment in the code, you may write it in the usual way like `% This is a comment`.

Your project's `<docinfo>` may contain a `<latex-image-preamble>` element. If so, and if that preamble content should affect the latex-images inside the `<webwork>`, then you need to get that preamble content up on the WeBWorK host course. In many cases, you will *need* to get that preamble content up on the host course for the image to even compile. See [Section 38.4](#).

See the WeBWorK sample chapter for examples.

7.1.8.3 Using @pg-name

In a problem statement (or hint, or solution), you may use an `<image>` with a `@pg-name`. This attribute's values should be the name of an image created in the `<pg-code>` section. For example, an image created using `init_graph` from `PGgraphmacros.pl`.

See the WeBWorK sample chapter for examples.

Chapter 8

Authoring Advice

In this chapter we gather advice on authoring. In some cases it is discipline-specific.

8.1 Writing Your Student-Friendly Math Textbook

Kathy Yoshiwara

So you are writing a math textbook. You love your subject enough to put in the hours, and you probably have some ideas on how the standard presentation can be improved. You care about good pedagogy and want to engage your students.

You know that writing a text for undergraduates requires a different style from writing a research paper or a scholarly article for colleagues. But how to achieve that style? A good first step is to adjust your linguistic goal from

Elegant Argument

to

Illuminating Explanation

We can examine this strategy in three areas:

1. Language
2. Layout or Format
3. Content

Language. In the exposition, it is useful to adopt an informal voice. But, perhaps counter-intuitively, that does not mean a conversational voice. In conversation, you have gestures and tone of voice to help convey meaning: what are the main points, what are helpful hints, what are asides, and what are social interactions.

None of these prompts are available when communicating in print. The author must shape the material so that the reader can navigate the ideas on his or her own. It is best to be as brief and direct as possible. After writing a section or so, go back and omit any unnecessary words or phrases. For example, before presenting an Example, there is no need to say “Here is an example to illustrate the ideas we have just discussed.” Subconsciously, the reader must absorb and then jettison this comment as unimportant. Doing so constantly leads to “reader fatigue.”

Often an explanation can be improved just by (judiciously) making it shorter. In addition, while trying to understand a new concept, most students will not find helpful our philosophical musings or historical anecdotes. These can be presented in a sidebar or addendum. Do try to relate new ideas to previous ones and show how they fit into the overall scheme, but avoid references such as “we’ll need this for our later study of (whatever).”

The principles of good writing for any format apply equally to textbooks, mathematical or otherwise. Usually, active voice is more effective than passive voice, and a positive form is clearer than a negative one. Strunk and White’s classic *The Elements of Style* and, more recently, George Gopen’s reader expectation approach are standard resources for techniques of composition. (Although sometimes their advice is contradictory, which just goes to show that no rule is appropriate in every situation.) Steven Pinker’s *The Sense of Style* is also useful.

Layout or Format. In a textbook, it is good practice to deliver material in digestible portions. Try to keep blocks of uninterrupted text rather short. Break up the exposition visually with boxes, Examples, Cautions, Notes, and so on. Use bulleted or numbered lists to highlight important points. Consider whether it would be more effective to start a particular section with a motivating example, or perhaps with a few sentences explaining how the new topic arises naturally, or in some other way.

Instead of opening your textbook with a “Chapter 0” type catalog of all the notation and terminology needed in the entire book, it would be kinder to students to introduce new notation and terminology as needed. Also, it is not necessary to front-load all the information about a topic at one time; let students absorb and practice the fundamental ideas, then return later to elaborate, generalize, or present exceptions. Resist the temptation to proceed too quickly to general or abstract statements. Most people grasp abstract ideas more easily if they first see specific and concrete examples.

Choosing effective examples helps make your text student-friendly. A simple example has greater impact than a complicated one. If you are introducing asymptotes, the graph of

$$y = \frac{x - 3}{x - 2}$$

is a better example than

$$y = \frac{x^2 - 9}{x^2 - x - 2}.$$

It is also better than

$$y = \frac{1}{x}$$

because the latter may lead students to believe that asymptotes are identified with the coordinate axes. If you are introducing subgroups, do not limit your examples to subgroups of cyclic groups. Always consider your example from the students’ point of view! Is your example too general or too specific? Are there confounding features that may distract from the intended message?

What about proofs? These days a good background in mathematics is necessary for a wide variety of occupations, so that only a small number of your students may be headed to graduate school in mathematics, even in junior or senior level classes. Here is a good place to strive for “illuminating” rather than “elegant.”

Content. Choosing and organizing the material for your textbook requires more thought than any other aspect of the creative process. To make your textbook truly student-friendly, you may want to rethink the traditional or standard order of content. If you are used to presenting material by topic, you might want to consider how better to make connections or to take advantage of sound pedagogical principles.

For example, forty years ago high school algebra was taught by first covering all the “one-variable” material and then (if time permitted) considering graphs and other “two-variable” topics. Now it is considered more effective to study graphs throughout the course. Linear algebra courses used to start with the study of vector spaces. Now many texts prefer to begin with systems of linear equations. In trigonometry, maybe we can start with just three trig functions instead of six; maybe we can work in one quadrant first, then add the second quadrant, before treating all four.

In a sense, textbooks drive the curriculum. Think of the innovations introduced in the calculus renewal movement that are now incorporated into most calculus texts: the catalog of functions, the rule of four, the inclusion of conceptual exercises. Many instructors, especially adjunct or part-time instructors, rely on their textbooks to shape their courses. You have a real opportunity to influence the direction of your field and to shape the way it is taught. Now get to work.

Chapter 9

Author FAQ: Frequently Asked Questions

Technical questions first, followed by questions about markup.

1. *On my local machine, why is the _____ not working?*

The “_____” could be a knowl that appears empty. Or a sage cell that appears empty. Or an interactive element of some sort that seems broken. Or any component of the HTML page that is “fancy”.

Viewing an HTML file on your own computer is not the same as visiting it on a website. There is no web server, so you should assume that things will be different.

The cause of the difficulty is that some web browsers will behave differently when viewing a local file compared with viewing it on a website. If there are other resources to fetch (for example, the content of a knowl) the browser may consider it a security risk to fetch them while viewing a local file.

Try viewing your HTML file using a different browser, and you may find that the feature is working there. However the real test is to transfer the files to an actual web server, or to set up a local web server so that local files operate in the usual way.

To set up a local server, see [Section 5.11](#).

2. *How do I install xsltproc on Ubuntu or Debian Linux?*

```
sudo apt-get install xsltproc
```

3. *How do I install pdf2svg?*

As of July 2024, you no longer need pdf2svg to process latex-images; we now use the python pyMuPDF library instead.

4. *Why is there no tag for bold?*

Because the first principle of PreTeXt is that the markup captures the structure of the document, not the appearance.

A better question is: *why* do you want to print something in bold? Is it emphasis? (See .) Is it the volume number of a journal? (See <journal>.) Do you want to SHOUT? Try <alert>. And so on. There are lots of good answers, some of which are not yet implemented. We would love to hear about elements you need that are about expressing content, and not about altering presentation. See [Principle 1](#).

5. *Can I create PreTeXt source in Microsoft Word or Google Docs?*

Unlikely. PreTeXt is designed around XML markup, so using a text editor to create a text file is the intended process for creating source. When you create a file with other word processors it is likely to use different characters in some places, such as “smart quotes”, and convert other things you type, such as converting three periods into a single ellipsis character. And if you make font changes, such as italics for emphasis, that will not translate directly to PreTeXt.

Perhaps you know somebody technically-minded you want to convert your word-processor files for you? First, this is a tedious job. Second, they may not accurately translate your intent. Third, you cannot make any edits and you will be *reliant* on them alone to maintain your project for you.

If you insist on authoring with a traditional word-processor, then you should. But you will get whatever output that tool provides, and miss out on the benefits of the semantic markup that allows us to create all the various output formats PreTeXt provides.

6. *Why do I get no output and some warnings about bugs?*

There is a good chance you have “modularized” your source files and have not included an `<xi:include>` for each modularized file. (See [Section 5.3](#).) If that was your mistake, then you should have seen a warning. Please check to see if there was a warning you missed. (If not, we can improve the warning if you tell us how your source was organized. So please do, since we would love to hear about it.)

7. *I don't like surprises and have not updated in months. Why do I now have a problem?*

We almost never release mistaken code that breaks output produced from valid source ([Chapter 6](#)). And when we have, the cause is usually a small typo, or something that gets fixed easily and quickly. We do sometimes make backwards-incompatible changes, but you always get warnings, often the changes can wait, there are announcements on the mailing lists, and whenever possible, we update tools that automate the changes.

As volunteers, we cannot support problems from old versions, and sometimes we have to implement changes in reaction to third-party software (like MathJax), which is beyond our control. So while development is rapid, we implore you to remain on the dev branch of the repository and `git pull` regularly. Such as *daily*, with your morning coffee as you sit down to write, or with your last compilation of the evening. You will have *far fewer unpleasant surprises* this way, and we can help you better.

We understand that PreTeXt is a moving target, but we are iterating to a better state, and it is best to have everybody along for the *whole* ride.

8. *I have not updated in months, so I did a pull, and now I have a bunch of warnings. How do I catch up?*

Changes on our end which produce new warnings are almost always announced on the pretext-announce Google Group. This is a very low-traffic, announcement-only list, so you could skim the messages you missed. Also, read warnings carefully, as they often have explicit suggestions about what to do next.

9. *There are lots of different blocks, but my project needs one that does not exist..*

We do not support creating new blocks. But you can repurpose an existing one. See [Subsection 4.2.4](#).

10. *How do I put mathematics into my list markers?*

List markers are an organizational device for grouping small bits of information. It is not appropriate to use mathematics in this situation. (Even if $\LaTeX$ list item “labels” allow such a construction.) The alternative is to use a description list, with a `<dl>` element. Put your mathematics in the `<title>` and put the associated content into the remainder of the `<li>`. Or, put titles on the list items of an ordered or unordered list, and include mathematics there.

11. *I have errors when I try to validate my markup, but everything looks okay when I make the HTML and the PDF. Should I be worried?*

Occasionally the schema lags behind the code, so your first step is to post to `pretext-support@googlegroups.com` to find out if it is a problem with the schema.

Possibly there is another way to accomplish what you want, and that markup will fit the schema. Or maybe what you are doing meets the “common and reasonable” test and can become a feature request. The discussion on `pretext-support` should provide an answer.

12. *Why are theorems, definitions, examples, remarks, etc. all numbered using the same counter?*

The following is an argument in favor of using common counters for blocks of similar appearance. The argument is stronger in the context of using a printed copy of the book, where physical page flipping is necessary, but also applies to scrolling through a (long) page in a web browser.

Suppose your professor gave you a note to review Example 2.4.7 in your textbook. The “2.4” is useful information directing you to Chapter 2, Section 4. You can tell by the “7” that the example is probably not right at the beginning of the section, so you open to the middle of the section. You find yourself on a page with no examples, but you do see Remark 2.4.11. What do you do: flip forward or flip backward?

If examples and remarks are numbered using separate counters, you have no information about which way to go. You need to make a random decision, and flip pages until you find another example that you can use as a guidepost. And examples might be rare and sparse, so it may take quite a bit of page flipping to find that guidepost. You may end up at Example 2.4.8, telling you that you need to flip backward now. But how far? Will it be one page earlier or twenty?

For a more expansive discussion along these lines, see [Section 26.3](#).

Also, as an author, recognize that there is a very flexible mechanism for making lists of objects that may be included in the <backmatter> (or elsewhere). To continue the example here, you could make a list of all the examples in the book, and a separate list of all the remarks. Each list would be in the order of appearance, include the number (and a title if you provide one). In HTML output, each is a knowl which will quickly provide the content (independent of location), and also provides an “in-context” link to take you to the location for surrounding material. This useful feature requires very little additional effort, especially if you title your blocks as you author them.

13. *Why do I have L^AT_EX errors?*

If you have errors when you run pdf_latex or xelatex, that is more likely to be caused by a problem with your L^AT_EX installation than with PreTeXt. It is difficult to make legitimate PreTeXt that fails to compile; an exception being math markup inside a math element (<m>, <md>, and friends).

To check your L^AT_EX installation, run pdf_latex -version. If it reports something older than “TeX Live 2017”, then updating may help. If the trouble seems to be coming from a particular package, then check which version of the package is being used. For example, if the “tasks” package is causing a problem, run kpsewhich tasks.sty in the directory where you are compiling the L^AT_EX. If the package is in your personal texmf tree, that may be the problem.

If the PDF is created without errors, but something looks wrong in the PDF, then probably the PreTeXt source markup is wrong. Validate your source against the schema (see [Section 6.5](#)), and also carefully examine the HTML output. If that does not reveal the problem, seek expert help.

14. *I have great output, so why does validation produce hundreds of errors?*

Success with a tool like the PreTeXt-CLI (see [Section 5.2](#)), in terms of no errors and great-looking output, does not mean your source is correct. xslt processing can be forgiving, and many invalid constructions just work, and look great. But that is no guarantee this situation will continue, or the same happy accidents occur for a conversion to a different output format.

We do raise some errors during processing with the PreTeXt-CLI, but error-checking your input is a job for a validator, so in theory we should not ever produce any errors during a conversion. So strive for having 100% valid PreTeXt source, not simply great-looking results. See [Chapter 6](#) and [Appendix E](#).

15. *I’d like to keep using L^AT_EX markup for mathematics, in other words, \$...\$ and \[...\].*

If you *were* somehow able to mix-in a dollar sign into your PreTeXt document, how would you then write “The bus ride cost me \$2.50”? The L^AT_EX solution, \\$, is not going to help you. It is a shortcoming of T_EX that so many characters have special meanings in certain circumstances.

If you insist, you can still author using \$...\$ and \[...\], and before processing do a global search-and-replace. But converting a leading \$ to <m> and a trailing \$ to </m> will drive you mad unless you are really good with regular expressions. So perhaps better to use \ (and \) for inline mathematics and search-and-replace will go better. I count six key presses for that, including the shifts for the parentheses, while it just takes me seven key presses for the PreTeXt <m> element, when I use an editor that auto-completes elements with a two-key combination.

We realize change is hard—we used L^AT_EX for thirty years. But we believe the long-term advantages of PreTeXt markup are worth the short-term rearrangements.

16. *Why not $\LaTeX$? Why PreTeXt?*

$\TeX$ was first released in 1978. There was no Internet, no HTML, no Unicode, and no YouTube. There have been many attempts to convert $\TeX/\LaTeX$ to more modern document formats. They are not hard to find—none is satisfactory. We know because we have spent many years trying to adapt them to our purposes.

Many laud $\LaTeX$ for its ability to separate content from presentation. The key word is *ability*. It is possible to use $\LaTeX$ in a purely semantic way. But it very rarely happens in practice. And we suspect that when it does, the result looks much like XML anyway, such as the use of many `\begin{}/\end{}` pairs. $\LaTeX$ allows authors enough freedom that it is impossible to accurately discern intent in a totally automated way.

By contrast, an XML vocabulary defined by a schema (i.e. PreTeXt) forces authors to communicate intent and denies authors the opportunity to micro-manage presentation. The result is automated conversions to many useful output formats with no extra effort from the author, including future conversions to formats not yet imagined. And xsl, once understood, is a robust and powerful tool for the sorts of text-manipulation tasks necessary.

17. *Why do I have an “extra” period at the end of a title?*

Author your titles *without* punctuation at the end meant for spacing or separation (period, colon, semi-colon, hyphen). Do include punctuation which imparts meaning (question-mark, exclamation-point). PreTeXt will then add separation (a period, or spacing), as needed, in all the places where it is required. But PreTeXt will respect your question-marks and exclamation-points.

If you think you have an additional punctuation character that conveys meaning at the end of a title, please bring it to our attention.

18. *I’m a little confused about the less-than character, the ampersand, and “escape characters” in general.*

When you process XML, the less-than character, `<`, is a signal that the name of an opening or closing tag is coming next. How do you prevent this if you really need the character, especially for something like computer code? Answer: you use the “escaped” version, `<`. The ampersand is known as an **escape character**. It is a signal to the processor to escape from its usual rules. “But,” I hear you say, “we just gave the ampersand character a special meaning, now how do I get *that* character when I need to say ‘I went to the A&P store.’?” Simple—there is an escaped version, `&`.

So think of `<` and `&` as *being* the characters `<` and `&`. End of story. These two concessions should “work” throughout your PreTeXt source and in every conversion. Every markup language needs, and uses, an escape character, but XML and PreTeXt have you covered and really only need these two (infrequent) exceptions.

It is worth repeating: think of the escaped versions of the characters as actually being the characters themselves. That is the way the XML processor sees them and uses them. For more, see [Section 3.14](#), [Section 3.16](#), [Subsubsection 4.1.4.2](#), [Section 4.4](#), and [Subsection 4.32.4](#).

Postscript: XML has the escaped characters `>`, `'`, and `"` for `>`, `'`, and `"` (respectively). But they are rarely (never?) necessary within PreTeXt.

Extra-credit homework: think about how some of the above was authored and then look at the source to grade your own work.

19. *The titles of my description list are long, and the formatting is really bad..*

Use shorter titles? Read too about the `@width` hint for list items within [Section 4.11](#).

Or convert your list to an `<ol>` or `<ul>`. The titles will now render more like their own paragraphs.

20. *My math looks great in one output format ($\LaTeX$ or HTML) but is causing errors or looks bad in another output format (HTML or $\LaTeX$). What’s up?*

There are subtle and minor variations and restrictions around the interpretation of $\LaTeX$ syntax by $\LaTeX$ itself and the MathJax Javascript routines for rendering in a web page. Work backwards from [Subsection 4.9.19](#) for details.

21. *I would like to use a standard package in my Sage code.*

See [Subsection 4.22.1](#) for an explanation.

22. *I would like to use a (large) data file in my Sage code.*

See [Subsection 4.22.1](#) for an explanation.

23. *Processing my project takes a long time. Can I “comment out” some parts?*

First, we know some aspects of processing are inefficient and therefore slow for large projects. We *are* working on it.

Your PreTeXt source is not a program, nor is it $\text{\LaTeX}$, which can also be a program. So the idea of “commenting out” code, or mimicking conditional use of $\text{\LaTeX}$ ’s `\include{}` command, are injurious to your PreTeXt source. Processing expects to see *all* of it. Some aspects of producing output demand it.

We have the notion of a **version** which can eliminate some (large) portions of your source in controlled ways (see [Section 27.2](#)). Processing tools, such as the PreTeXt-CLI and the pretext/pretext Python script support the notion of restricting some processing to a subtree of your source, without *hiding* parts of your source from the processor. See [Section 47.10](#) for how to do this with the pretext/pretext Python script.

Part III

Basics Reference

Chapter 10

About This Reference

This *Basics Reference* supplements the more formal documentation in other parts of the Guide. Here you will find the *basics* of the most important and commonly used PreTeXt elements. In most instances, this will be exhibited by showing a chunk of PreTeXt code in a numbered Listing followed immediately by the exact output that PreTeXt code produces. (In some cases, such as with the code that produces a section, it is not practical to do this.) Wherever possible, the sample code and its output are accompanied by cross-references to the *Author's Guide* in [Part II](#).

There are some things that are considered beyond the scope of this reference:

- Many of the possible options that can be used with the different tags. This is a *basic* reference, and so we want to keep things simple. When you need a more advanced feature not discussed here, follow the link to the [Author's Guide](#). We welcome contributions of intermediate examples to this *Basics Reference*, but the initial goal is not to be comprehensive in adding such examples.
- Instructions on converting your PreTeXt XML to another format such as HTML or $\text{\LaTeX}$.
- Instructions on using WeBWorK in PreTeXt. (This reference does show the most basic of syntax for including a problem from the WeBWorK Open Problem Library. However, it assumes that you already have a project set up to compile correctly with WeBWorK problems, which is an involved task.)

Chapter 11

Basic Formatting

The [PreTeXt Principles](#) begin with “PreTeXt is a markup language that captures the structure of textbooks and research papers” ([Item 1.1.1:1](#)). By a **markup language**, we mean that the syntax describes the *structure* of the document and not the presentation of the document. Thus, PreTeXt does not provide, for instance, a way to make text bold or italic or in a larger font. If an author seeks a specific type of local typesetting, then they need to pause and think about the *reason* for that typesetting. Is the reason to emphasize a word or phrase? Is the reason to alert the reader to a common mistake? Is it to designate that a word is a new term being defined by the author? There are ways to mark up such structural ideas in PreTeXt, and authors should conscientiously ensure that they use this markup.

To illustrate some of the key structural markup that leads to formatting, we include the listing and paragraph below.

Listing 11.0.1 Some basic content of a paragraph

```
<p>
  This is a new <term>piece of terminology</term>.
  Here is a word we have chosen to <em>emphasize</em>.
  <alert>This is an alert to the reader!</alert> This is a piece of
  <c>code typeset in a different font</c>.
</p>
```

The code in [Listing 11.0.1](#) produces the following output:

This is a new **piece of terminology**. Here is a word we have chosen to *emphasize*. ***This is an alert to the reader!***
This is a piece of code typeset in a different font.

Chapter 12

Document Structure

Elements such as `<chapter>`, `<section>`, and `<subsection>` are called **divisions**. They are the key organizational elements of the structure of a PreTeXt document and all have (essentially) the same syntax. If a division does not contain any other divisions, then its structure looks like what we see in [Listing 12.0.1](#). (Plenty of other things can go inside other than paragraphs, including figures, etc.)

Listing 12.0.1 The general outline of a section as a model division

```
<section>
  <title>Mandatory</title>
  <p>
    First paragraph.
  </p>

  <p>
    Second paragraph.
  </p>
</section>
```

If a division has other divisions inside it, then the structure is a bit more complicated and regimented. In particular, if you want text before your first subdivision (`<subsection>` in this example), that text must go inside `<introduction>`. If you want to start with the `<subsection>`, then the `<introduction>` is optional. In the “division with subdivisions” model, everything *must* be contained inside `<introduction>`, `<subsection>` (or whatever your subdivision type is), `<exercises>`, `<references>`, or `<conclusion>`. This is illustrated in [Listing 12.0.2](#).

Listing 12.0.2 A <section> with <subsection>s.

```

<section>
  <title>Mandatory</title>
  <introduction>
    <p>
      Introductory text.
      (Optional.)
    </p>
  </introduction>

  <subsection>
    <title>Mandatory</title>
    <p>
      Subsection content.
    </p>
  </subsection>

  <subsection>
    <title>Mandatory</title>
    <p>
      Subsection content.
    </p>
  </subsection>

  <conclusion>
    <p>
      Concluding text.
      (Optional.)
    </p>
  </conclusion>
</section>

```

Limitations on introductions and conclusions. There are many tags that are *not* allowed in introductions and conclusions. In general, avoid things that would have numbers. For instance, one should not put an <example> or an <exercise> in an introduction or conclusion.

Paragraphs: like an un-numbered (sub)section. It can be useful to gather a few items and give them a title, but perhaps those items do not merit their own (sub)section. The <paragraphs> serves that purpose. A <title> is required.

This <paragraphs> contains two <p>s. Immediately before, and immediately after this <paragraphs> there are <paragraphs> which each contain one <p>. A <paragraphs> can contain pretty much anything except for a division.

The role of <p> tags. One of the things you'll need to keep an eye out for is when things must be wrapped in <p> (paragraph) tags. Notice that <title> tags do not have their content wrapped in <p>, which places some limits on the sorts of things that can be contained in a title. If you find text disappearing or displaying strangely, the culprit is likely an unnecessary or missing <p> tag. See [Chapter 6](#) for information on how to use some additional tools to see if your PreTeXt file is valid in terms of following the structural rules in the schema.

Chapter 13

Mathematics

Since PreTeXt was originally called MathBook XML, you will not be surprised to learn that it has robust support for mathematical formulas. Inside the tags that delimit math environments, your code is basically $\text{\LaTeX}$, with the caveat that you must be careful with $<$ and $&$ since they are special symbols for XML. When typing math in your PreTeXt code, use $\text{\textbackslash lt}$ for $<$, use $\text{\textbackslash gt}$ for $>$ (not strictly necessary, but good for symmetry), and use $\text{\textbackslash amp}$ for $&$. In HTML, MathJax is used to render math, so PreTeXt generally supports the things that MathJax does “out of the box” (with the addition of `amsmath`) without the need for too many additional packages to be loaded.

For inline math, just wrap things in the `<m>` tag. For example, $a^2 + b^2 = c^2$ is produced by `<m>a^2 + b^2 = c^2</m>`. We get displayed equations with the `<md>` tag; setting `@number` to `yes` on the `<md>` produces a numbered equation.

Listing 13.0.1 Displayed equations

```
<p>
  <md>
    \frac{d}{dx} \int_1^x \frac{1}{t} dt = \frac{1}{x}
  </md>
  <md number="yes">
    \int_a^b f(x) dx = F(b) - F(a)
  </md>
</p>
```

The code in [Listing 13.0.1](#) produces the following output:

$$\frac{d}{dx} \int_1^x \frac{1}{t} dt = \frac{1}{x}$$
$$\int_a^b f(x) dx = F(b) - F(a) \tag{13.0.1}$$

For a collection of equations all aligned at a designated point, use `<md>` and `<mrow>`. Number equations by setting `@number` to `yes` on the `<md>` (or on individual `<mrow>` children).

Listing 13.0.2 Aligned equations

```
<p>
  <md>
    <mrow>x \amp = r\cos\theta</mrow>
    <mrow>y \amp = r\sin\theta</mrow>
  </md>
</p>
```

The code in [Listing 13.0.2](#) produces the following output:

$$x = r \cos \theta$$

$$y = r \sin \theta$$

Because most of the early adopters of PreTeXt have been mathematicians, there are lots of additional features supported in terms of mathematics. See [Section 4.9](#) for further details.

Chapter 14

Lists

Lists are important in lots of contexts, and the desire to nest lists has led to some very, very complex discussions on the PreTeXt email lists. We'll keep it simple here. There are a variety of places that lists can live, but a good mental model is that a list must be put inside an element that's similar to `<p>`. *So for example, you can't put your list directly inside a `<subsection>`, but instead must wrap the list in a `<p>`.*

There are two common types of lists: ordered and unordered. (There's also the description list. See [Section 4.11](#) for more information.) As in HTML, an ordered list is produced with `<ol>` and an unordered list with `<ul>`. The items of your list are structured inside `<li>` tags. The example in [Listing 14.0.1](#) shows that these `<li>` tags can contain a paragraph in a `<p>` tag but you do not need to. If you're just putting a sentence or two inside your `<li>`, no `<p>` is required. However, if you also want to put an image or more complicated items inside the `<li>`, then the `<li>` must be structured with `<p>` by placing "loose" text inside one or more `<p>`.

Listing 14.0.1 An ordered list

```
<p>
  <ol>
    <li>First item.</li>

    <li>
      <p>
        Second item,
        showing can wrap content with <c>p</c>.
      </p>
    </li>
  </ol>
</p>
```

The code in [Listing 14.0.1](#) produces the following output:

1. First item.
2. Second item, showing can wrap content with p.

You can use the `@marker` attribute¹ on the `<ol>` tag to change the default markers. For instance, if the opening tag for the list above were `<ol marker="A">`, then the list items would be marked as A. and B. Sensible things to use with `@marker` are i, I, A, a, and 1. Nesting of lists is possible, and there are sensible default markers.

¹As illustrated here, an attribute is something that appears between the `<` and `>` of the opening tag. The convention in XML usage is to prefix an attribute name with `@` when referring to the attribute *outside* of the tag. You do not use the `@` in the tag itself.

Listing 14.0.2 An unordered list

```
<p>
  <ul>
    <li>First item.</li>

    <li>Another item.</li>
  </ul>
</p>
```

The code in [Listing 14.0.2](#) produces the following output:

- First item.
- Another item.

You can also use the @cols attribute to split a list (ordered or unordered) across multiple columns if the screen/page is suitably wide. The value of this attribute must be an integer between 2 and 6 (inclusive).

Chapter 15

Blocks

15.1 Theorem-Like Elements

The tags `<theorem>`, `<algorithm>`, `<claim>`, `<corollary>`, `<fact>`, `<identity>`, `<lemma>`, and `<proposition>` have the same structure in PreTeXt, so we will just illustrate `<theorem>` here.

Listing 15.1.1 A theorem

```
<theorem>
  <title>Optional</title>
  <creator>I. Newton</creator>
  <statement>
    <p>
      Here's the statement of the theorem.
    </p>
  </statement>

  <proof>
    <p>
      You don't actually need a proof,
      but put it inside the <c>theorem</c>.
      You can actually put another <c>proof</c> right after this one if you want to.
    </p>
  </proof>
</theorem>
```

The code in [Listing 15.1.1](#) produces the following output:

Theorem 15.1.2 Optional. (I. Newton) *Here's the statement of the theorem.*

Proof. You don't actually need a proof, but put it inside the theorem. You can actually put another proof right after this one if you want to. ■

The `<title>` is optional and typically used for theorems with names. To give an attribution, one can use the optional `<creator>` tag. Cross references (see [Section 22.1](#)) can be made using the name or the number, depending on how the author codes them.

A theorem-like element can contain multiple `<proof>` elements. In such instances, it would be useful to use the `<title>` tag within your proof. By default, a `<proof>` is hidden in a knowl when using HTML output. Click the “Proof.” heading to expand the proof. Click it again to hide the proof. A `<proof>` can also be divided into `<case>`s, each of which can have a title. Although it has not always been so, you can author a `<proof>` all on its own within a division. The structure of such a detached `<proof>` is the same as for a `<proof>` contained within a theorem-like element.

You can use `<definition>` essentially like `<theorem>`, but a `<definition>` does not have a proof. You are encouraged to use the `<term>` tag to set off the word being defined. If you wish to include a list of notation as an

appendix as your document, you might also add a `<notation>` tag such as shown in [Listing 15.1.3](#). A `<notation>` tag has no effect unless you have an `<appendix>` with a `<notation-list/>` in it.

Listing 15.1.3 A definition with notation

```
<definition>

  <notation>
    <usage><m>\binom{n}{k}</m></usage>
    <description>binomial coefficient</description>
  </notation>

  <statement>
    <p>
      The <term>binomial coefficient</term>
      <m>\binom{n}{k}</m> is the number of <m>k</m>-element subsets of an
      <m>n</m>-element set.
    </p>
  </statement>
</definition>
```

The code in [Listing 15.1.3](#) produces the following output:

Definition 15.1.4 The binomial coefficient $\binom{n}{k}$ is the number of k -element subsets of an n -element set. ◆

15.2 Example-Like Elements

PreTeXt provides three closely-related tags for things that are examples or similar. They are `<example>`, `<problem>`, and `<question>`. They all have the same syntax. The `<title>` element is optional, but encouraged as discussed in [Best Practice 4.8.1](#). You may either use a freeform example, as shown in [Listing 15.2.1](#), or an example structured with a `<statement>` and zero or more `<hint>`s, `<answer>`s, and `<solution>`s (in that order). This is illustrated in [Listing 15.2.3](#). Note that for HTML output, if your `<example>` has a `<solution>`, the solution will be hidden in a `knowl` by default. See [Section 29.2](#) for information on changing this behavior.

Author vs. publisher. PreTeXt strives to remind us that the roles of author and publisher should be kept mentally separate, even if there is one individual ultimately responsible for both roles. One way to think about this is that the **author** is the person writing the *content* of the book, article, etc., while the **publisher** is the person responsible for making the final product available to the world in one or more formats such as a physical print-on-demand book, an electronic book in EPUB format, or a web page.

Authors and publishers generally work collaboratively, but the publisher frequently comes in later in the process. Some authors act as their own publishers, but it is useful to switch mindsets when making decisions about how the final product will be presented to the world rather than the specific content on the page or screen.

Listing 15.2.1 A simple example

```
<example>
  <title>Differentiating a polynomial</title>
  <p>
    The derivative of the function
    <m>f(x) = 3x^5-7x+5</m> is <m>f'(x) = 15x^4-7</m>.
  </p>
</example>
```

The code in [Listing 15.2.1](#) produces the following output:

Example 15.2.2 Differentiating a polynomial. The derivative of the function $f(x) = 3x^5 - 7x + 5$ is $f'(x) = 15x^4 - 7$. ▲

If you are reading this in HTML with the default processing options applied, the example above will be *known*. This is the PreTeXt default *only* so that authors know that *knowls* exist. See [Section 29.2](#) for information on changing

this behavior.

Listing 15.2.3 A structured example

```
<example>
  <title>Differentiating a polynomial</title>
  <statement>
    <p>
      Differentiate the function <m> $f(x) = 3x^5 - 7x + 5$ </m>.
    </p>
  </statement>
  <answer>
    <p>
      <m> $f'(x) = 15x^4 - 7$ </m>
    </p>
  </answer>
  <solution>
    <p>
      We use the power, sum,
      and constant multiple rules and find that the derivative is <m> $f'(x) =$ 
       $15x^4 - 7$ </m>.
    </p>
  </solution>
</example>
```

The code in [Listing 15.2.3](#) produces the following output:

Example 15.2.4 Differentiating a polynomial. Differentiate the function $f(x) = 3x^5 - 7x + 5$.

Answer. $f'(x) = 15x^4 - 7$

Solution. We use the power, sum, and constant multiple rules and find that the derivative is $f'(x) = 15x^4 - 7$. ▲

15.3 Axiom-Like Elements

PreTeXt provides several tags that fall into the category of an “axiom”. They are `<assumption>`, `<axiom>`, `<conjecture>`, `<heuristic>`, `<hypothesis>`, and `<principle>`. The content of these tags is very simple. They allow an optional `<title>`, an optional `<creator>`, optional `<idx>` tags, and then a `<statement>` much like a `<theorem>` does.

Listing 15.3.1 An axiom

```
<axiom>
  <title>Optional</title>
  <creator>Peano</creator>
  <statement>
    <p>
      Here's the statement of the axiom.
    </p>
  </statement>
</axiom>
```

The code in [Listing 15.3.1](#) produces the following output:

Axiom 15.3.2 Optional. (Peano) *Here's the statement of the axiom.*

15.4 Remark-Like Elements

PreTeXt provides several tags that fall into the general category of a “remark”. They are `<convention>`, `<insight>`, `<note>`, `<observation>`, `<remark>`, and `<warning>`. The content of these elements is similar to what is allowed

inside the `<statement>` of a theorem-like element. They allow an optional title, optional `<idx>` tags, and then a mixture of `<p>`, `<blockquote>`, and `<pre>`.

Listing 15.4.1 A remark

```
<remark>
  <title>A little remark</title>
  <p>
    Remarks have restricted content,
    but they can be used to call out lots of important things for your readers.
  </p>
</remark>
```

The code in [Listing 15.4.1](#) produces the following output:

Remark 15.4.2 A little remark. Remarks have restricted content, but they can be used to call out lots of important things for your readers.

15.5 Project-Like Elements

There are four tags that PreTeXt considers to be “project-like”. They are `<activity>`, `<exploration>`, `<investigation>`, `<project>`. We will focus on `<project>` here. These four tags allow a general, freeform structure similar to the unstructured `<example>` in [Listing 15.2.1](#); a structure analogous to that of the structured `<example>` in [Listing 15.2.3](#); and the highly-structured `<introduction>`, `<task>`, `<conclusion>` model shown in [Listing 15.5.1](#).

Listing 15.5.1 A project

```
<project>
  <title>A structured project</title>
  <introduction>
    <p>
      Here is where we describe what the reader will accomplish in the project.
    </p>
  </introduction>

  <task>
    <statement>
      <p>
        The first step to do.
      </p>
    </statement>
    <hint>
      <p>
        A little hint.
      </p>
    </hint>
    <answer>
      <p>
        Just the answer.
      </p>
    </answer>
    <solution>
      <p>
        All the glorious details about how to do the first step.
      </p>
    </solution>
  </task>

  <task>
    <statement>
      <p>
        The second step to do.
        We'll be lazy and just include an answer.
      </p>
    </statement>
    <answer>
      <p>
        Just the answer.
      </p>
    </answer>
  </task>

  <conclusion>
    <p>
      A little wrap up.
    </p>
  </conclusion>
</project>
```

The code in [Listing 15.5.1](#) produces the following output:

Project 15.5.1 A structured project. Here is where we describe what the reader will accomplish in the project.

(a) The first step to do.

Hint. A little hint.

Answer. Just the answer.

Solution. All the glorious details about how to do the first step.

(b) The second step to do. We'll be lazy and just include an answer.

Answer. Just the answer.

A little wrap up.

Chapter 16

Exercises

PreTeXt provides a number of ways of collecting things that might be thought of as exercises for students to do. Some might be used for a student to check their ability to perform a skill. Others might be to check the understanding of what a student has just read, perhaps submitting responses to an instructor before class so that plans for in-class time can be adjusted. Another way of collecting exercises might be as a worksheet intended to be worked on during class.

16.1 <exercise> structure

An <exercise> can be rather freeform, containing elements such as <p>, <figure>, <image>, etc. However, an author will typically think about an exercise as having an associated correct answer, perhaps with a hint or a detailed solution. PreTeXt has tags to support that sort of content, but it does require that your <exercise> be structured. This is nearly identical to the structure of a project-like element.

Listing 16.1.1 An exercise

```

<exercise>
  <statement>
    <p>
      <idx><h sortby="statement"><tag>statement</tag></h><h sortby="of an
        exercise">of an <tag>exercise</tag></h></idx>
      <idx><h sortby="exercise"><tag>exercise</tag></h><h
        sortby="statement"><tag>statement</tag></h></idx>
      The <tag>statement</tag> is mandatory when any of
      <tag>hint</tag>, <tag>answer</tag>, or <tag>solution</tag>
      is included as a child of <tag>exercise</tag>,
      otherwise it may be omitted.
    </p>
  </statement>
  <hint>
    <p>
      <idx><h sortby="hint"><tag>hint</tag></h><h sortby="of an exercise">of an
        <tag>exercise</tag></h></idx>
      <idx><h sortby="exercise"><tag>exercise</tag></h><h
        sortby="hint"><tag>hint</tag></h></idx>
      Optional.
      Just an suggestion of what to try.
    </p>
  </hint>
  <answer>
    <p>
      <idx><h sortby="answer"><tag>answer</tag></h><h sortby="of an exercise">of an
        <tag>exercise</tag></h></idx>
      <idx><h sortby="exercise"><tag>exercise</tag></h><h
        sortby="answer"><tag>answer</tag></h></idx>
      Optional.
      Just the <q>final answer</q>.
    </p>
  </answer>
  <solution>
    <p>
      <idx><h sortby="solution"><tag>solution</tag></h><h sortby="of an
        exercise">of an <tag>exercise</tag></h></idx>
      <idx><h sortby="exercise"><tag>exercise</tag></h><h
        sortby="solution"><tag>solution</tag></h></idx>
      Optional.
      All the gory details.
    </p>
  </solution>
</exercise>

```

The code in [Listing 16.1.1](#) produces the following output:

Checkpoint 16.1.2 The <statement> is mandatory when any of <hint>, <answer>, or <solution> is included as a child of <exercise>, otherwise it may be omitted.

Hint. Optional. Just an suggestion of what to try.

Answer. Optional. Just the “final answer”.

Solution. Optional. All the gory details.

Note that you can have multiple <hint>, <answer>, and <solution> elements. But you must put all the <hint>s first, then all the <answer>s, and then all the <solution>s. There are a variety of options for determining where hints, answers, and solutions appear (at all). Check [Section 28.1](#) for information about stringparams.

An <exercise> can also have a more complicated structure that assigns a sequence of steps for a student to

complete. PreTeXt provides the same <task> tag that is used in project-like elements to give structure to such an <exercise>.

Listing 16.1.3 An exercise with tasks

```
<exercise>
  <title>A structured exercise</title>
  <introduction>
    <p>
      Here is where we give the student the background information
      required to start accomplishing tasks.
    </p>
  </introduction>

  <task>
    <statement>
      <p>
        The first step to do.
      </p>
    </statement>
    <hint>
      <p>
        A little hint.
      </p>
    </hint>
    <answer>
      <p>
        Just the answer.
      </p>
    </answer>
    <solution>
      <p>
        All the glorious details about how to do the first step.
      </p>
    </solution>
  </task>

  <task>
    <statement>
      <p>
        The second step to do.
        We'll be lazy and just include an answer.
      </p>
    </statement>
    <answer>
      <p>
        Just the answer.
      </p>
    </answer>
  </task>

  <conclusion>
    <p>
      A little wrap up.
    </p>
  </conclusion>
</exercise>
```

The code in [Listing 16.1.3](#) produces the following output:

Checkpoint 16.1.4 A structured exercise. Here is where we give the student the background information required to start accomplishing tasks.

(a) The first step to do.

Hint. A little hint.

Answer. Just the answer.

Solution. All the glorious details about how to do the first step.

(b) The second step to do. We'll be lazy and just include an answer.

Answer. Just the answer.

A little wrap up.

16.2 Inline exercises

You can put an `<exercise>` in the middle of a division, intermixed between theorems and paragraphs and figures. In this case, it is labeled as a “Checkpoint”.¹ To distinguish these `<exercise>s` from `<exercise>s` that are contained in the specialized divisions designed to collect `<exercise>s` that will be discussed in the next section, we refer to these as **inline exercises**.

16.3 Divisions of `<exercise>s`

You can also put several `<exercise>s` as part of an `<exercises>` element within a division, which is the typical way for creating a collection of exercises together at the end of a division such as a chapter or section. The content of an `<exercises>` division is rather limited. It can begin with an `<introduction>` (perhaps a set of common instructions), followed by a mixture of `<exercise>` and `<exercisegroup>` (see [Subsection 16.3.1](#)) elements, followed by an optional `<conclusion>`. The sample code in [Listing 16.3.1](#) illustrates this structure, which is rendered later as “16.4 Exercises”.

An alternative structure for an `<exercises>` division is to use a sequence of `<subexercises>` elements, optionally preceded by an `<introduction>` and followed by a `<conclusion>`. The content of a `<subexercises>` element is identical to what was described above for an `<exercises>` element, but we emphasize that a strong rationale for using `<subexercises>` (as opposed to `<exercisegroup>`) is that a `<subexercises>` element can begin with a `<title>`, providing a clear way of organizing the `<exercise>s` for the reader.

16.3.1 `<exercisegroup>`

Sometimes you have several exercises that should all have a common set of instructions, which is when you will use the `<exercisegroup>` tag. An `<exercisegroup>` can only be used as part of an `<exercises>` element or a `<subexercises>` element, however! The portion of this section headed as “16.4 Exercises” is produced using the code in [Listing 16.3.1](#).

¹See [Chapter 18](#) for information on how to use something different than “Checkpoint” as the name for these.

Listing 16.3.1 Using an <exercisegroup>.

```

<exercises xml:id="basics-s-sample-exercises">
  <exercisegroup>

    <introduction>
      <p>
        Here's where you put the common instructions.
      </p>
    </introduction>

    <exercise>
      <statement>
        <p>
          First exercise.
          <idx><h sortby="statement"><tag>statement</tag></h><h sortby="of an
            exercise">of an <tag>exercise</tag></h></idx>
          <idx><h sortby="exercise"><tag>exercise</tag></h><h
            sortby="statement"><tag>statement</tag></h></idx>
          You can add all the usual bells and whistles after,
          but we'll keep it short here.
          (The <tag>statement</tag> may be omitted if none of
          <tag>hint</tag>, <tag>answer</tag>, or <tag>solution</tag>
          are present as a child of the <tag>exercise</tag>.)
        </p>
      </statement>
    </exercise>

    <exercise>
      <statement>
        <p>
          Second exercise.
        </p>
      </statement>
      <hint>
        <p>
          The <tag>statement</tag> for this exercise is necessary because we
          have included at least one of
          <tag>hint</tag>, <tag>answer</tag>, or <tag>solution</tag>
          (namely, <tag>hint</tag>).
        </p>
      </hint>
    </exercise>

    <exercise>
      <p>
        Third exercise.
        (No <tag>statement</tag> necessary here because no
        <tag>hint</tag>, <tag>answer</tag>, or <tag>solution</tag>.)
      </p>
    </exercise>

  </exercisegroup>

  <exercise>
    <p>
      This <tag>exercise</tag> is not inside the <tag>exercisegroup</tag>.
    </p>
  </exercise>
</exercises>

```

If you want the contents of an <exercisegroup> to be put in multiple columns, you can add a @cols attribute to the <exercisegroup> with value (for example) 3. The integer value of @cols must be between 2 and 6 (inclusive).

The code in [Listing 16.3.1](#) produces the output seen in [Exercises 16.4](#).

16.3.2 Reading questions

Another specialized division, <reading-questions>, can be used to house <exercise>s designed to test or guide a reader's comprehension of the material in that division. The structure of a <reading-questions> element is similar to an <exercises> element, but without the grouping options of <subexercises> and <exercisegroup>. The portion of this section headed as "[16.3.3 Check your understanding!](#)" is produced using the code in [Listing 16.3.2](#).

Listing 16.3.2 Structure of <reading-questions>.

```

<reading-questions xml:id="basics-reading-questions">
  <title>Check your understanding!</title>
  <introduction>
    <p>
      Here is a spot to explain the purpose of these questions. It's
      optional, like most introductions.
    </p>
  </introduction>

  <exercise>
    <statement>
      <p>
        ^^IHere is a question.
      </p>
    </statement>
    <hint>
      <p>
        A little hint.
      </p>
    </hint>
    <answer>
      <p>
        Just the answer.
      </p>
    </answer>
    <solution>
      <p>
        All the glorious details about an answer.
      </p>
    </solution>
  </exercise>

  <exercise>
    <statement>
      <p>
        ^^IA second comprehension question. We don't bother with answers
        ^^Ior solutions.
      </p>
    </statement>
  </exercise>

  <conclusion>
    <p>
      A little wrap up, perhaps giving guidance or encouragement if
      the student struggled with the questions. Optional like most
      conclusions.
    </p>
  </conclusion>

</reading-questions>

```

16.3.3 Check your understanding!

Here is a spot to explain the purpose of these questions. It's optional, like most introductions.

1. Here is a question.
 Hint. A little hint.

Answer. Just the answer.

Solution. All the glorious details about an answer.

2. A second comprehension question. We don't bother with answers or solutions.

A little wrap up, perhaps giving guidance or encouragement if the student struggled with the questions. Optional like most conclusions.

16.4 Exercises

Exercise Group. Here's where you put the common instructions.

1. First exercise. You can add all the usual bells and whistles after, but we'll keep it short here. (The `<statement>` may be omitted if none of `<hint>`, `<answer>`, or `<solution>` are present as a child of the `<exercise>`.)
2. Second exercise.
Hint. The `<statement>` for this exercise is necessary because we have included at least one of `<hint>`, `<answer>`, or `<solution>` (namely, `<hint>`).
3. Third exercise. (No `<statement>` necessary here because no `<hint>`, `<answer>`, or `<solution>`.)
4. This `<exercise>` is not inside the `<exercisegroup>`.

16.5 WeBWorK Exercises

It is possible to embed WeBWorK exercises into a PreTeXt document. It is possible to build an HTML version in which readers can answer these exercises and find out if their answer is correct or incorrect. And if the book is hosted by Runestone, the reader's success can be recorded and incorporated into grades. Because WeBWorK content belongs inside an `<exercise>`, you can include WeBWorK exercises in any of the elements that can contain an `<exercise>` such as a `<worksheet>` and `<reading-questions>`. There's some configuration required to use WeBWorK. Please see [Chapter 7](#) and [Chapter 38](#) for more details. There is a WeBWorK *Sample Chapter* available elsewhere, with copious examples and output in multiple formats.

Chapter 17

Worksheets

A `<worksheet>` is a specialized division that can be a child of most divisions and can contain most PreTeXt tags. The general idea, however, would be to assemble a sequence of `<exercise>`s or project-like elements with some interspersed `<p>`, `<figure>`, and `<sidebyside>`. One of the few times that PreTeXt allows you to manipulate page layout in terms of giving blank space and specifying page breaks is in the `<worksheet>` tag. A `<worksheet>` is also the only place you can put an `<exercise>` inside a `<sidebyside>`. The sample code in [Listing 17.0.1](#) is not meant to demonstrate everything you can do, but rather to give you a skeleton to start exploring. The rendered output from the listing appears immediately after it.

Listing 17.0.1 Structure of <worksheet>.

```

<worksheet xml:id="basics-sample-worksheet">
  <title>A Skeletal Worksheet</title>
  <objectives>
    <ul>
      <li>Something really cool</li>
      <li>A less important thing you'll learn</li>
    </ul>
  </objectives>
  <introduction>
    <p>It can be helpful to say what the point of the worksheet is.</p>
  </introduction>

  <page>
    <exercise workspace="4in">
      <statement>
        <p>Here's a first exercise in this worksheet.
^^INotice how we set the workspace in inches.</p>
      </statement>
    </exercise>

    <exercise workspace="1cm">
      <introduction>
^^I<p>A second exercise, this time structured with tasks. The
^^Iworkspace specification is assigned to each task. </p>
      </introduction>
      <task>
^^I<statement>
^^I  <p>Here's the first task.</p>
^^I</statement>
^^I<hint>
^^I  <p>Why not give a hint here, we're nice authors, right?</p>
^^I</hint>
      </task>
      <task>
^^I<statement>
^^I  <p>The second task. No hint this time!</p>
^^I</statement>
      </task>
    </exercise>
  </page>

  <page>
    <p>OK, we're now onto a second page, one of the few times you can
    force this.</p>
    <figure>
      <caption>Just a little figure</caption>
      <image source="cross-square.png" width="50%" />
    </figure>
    <sidebyside margins="0%" widths="30% 60%" valign="top">
      <exercise workspace="4.5in">
        <statement>
^^I  <p>Only inside a worksheet can you put an exercise in a
^^I  sidebyside!</p>
        </statement>
^^I<answer>
^^I  <p>Sure, you can have an answer here.</p>
^^I</answer>
      </exercise>
      <exercise workspace="2in">
        <statement>
^^I  <p>Here's the second column. We also could have just put a
^^I  figure here if we needed for layout.</p>
        </statement>
      </exercise>
    </sidebyside>
    <exercise workspace="2.54cm">

```

A Skeletal Worksheet

Objectives

- Something really cool
- A less important thing you'll learn

It can be helpful to say what the point of the worksheet is.

1. Here's a first exercise in this worksheet. Notice how we set the workspace in inches.
2. A second exercise, this time structured with tasks. The workspace specification is assigned to each task.
 - (a) Here's the first task.

Hint. Why not give a hint here, we're nice authors, right?
 - (b) The second task. No hint this time!

OK, we're now onto a second page, one of the few times you can force this.

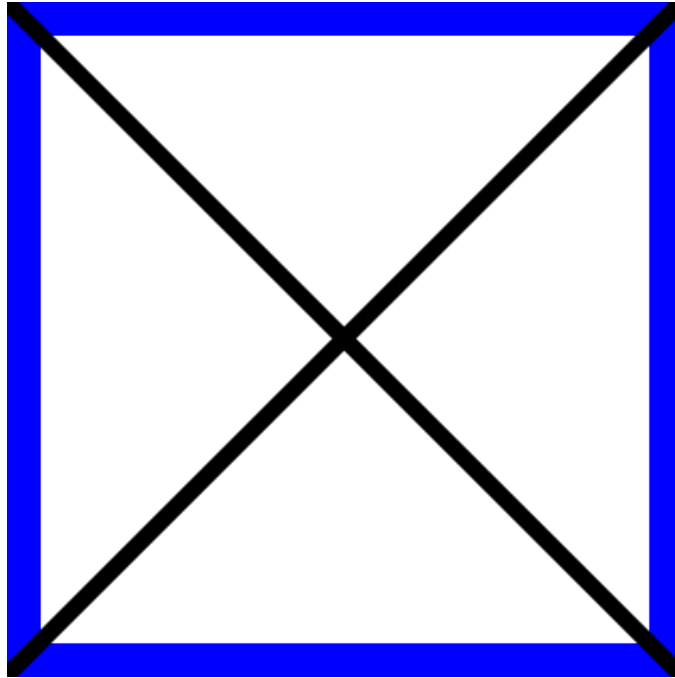


Figure 17.0.2 Just a little figure

3. Only inside a worksheet can you put an exercise in a sidebyside!

Answer. Sure, you can have an answer here.

4. Here's the second column. We also could have just put a figure here if we needed for layout.

5. One more exercise to do.

Final thoughts. Put something here because you might run out of time.

Chapter 18

Renaming Elements

The preceding sections have provided a lengthy list of PreTeXt tags that behave interchangeably. Perhaps you don't like one of their names. For instance, suppose your project will not involve any `<algorithm>`s, but you need another theorem-like tag whose name you would like to have rendered as “Porism”. To do this, you need to add a `<rename>` element to the `<docinfo>` element of your code. For our example, the necessary code would be `<rename element="algorithm">Porism</rename>`. A `<rename>` element generates a *global* change; it is not possible to rename a single instance of an element or to define your own tags (without writing your own XSLT code).

We have included this `<rename>` code in this guide's `<docinfo>`, and as such, we can do the following.

Listing 18.0.1 A porism generated using `<algorithm>` and `<rename>`

```
<algorithm>
  <statement>
    <p>
      This is a short little ditty that follows immediately from the previous proof.
    </p>
  </statement>

  <proof>
    <p>
      We'll still include a proof though.
    </p>
  </proof>
</algorithm>
```

The code in [Listing 18.0.1](#) produces the following output:

Porism 18.0.2 *This is a short little ditty that follows immediately from the previous proof.*

Proof. We'll still include a proof though. ■

A special note about renaming `<exercise>`s. Because of the range of divisions that can contain an `<exercise>` element with different names displayed (such as “Checkpoint” for an inline exercise), one cannot simply use `@element` with value `exercise` in a `<rename>` element. The value of `@element` to rename an `<exercise>` is as follows:

- `divisionalexercise` for an `<exercise>` inside `<exercises>`, `<subexercises>`, or `<exercisegroup>` with default “Exercise”
- `inlineexercise` for an inline exercise with default “Checkpoint”
- `worksheetexercise` for an `<exercise>` contained in a `<worksheet>` with default “Worksheet Exercise”
- `readingquestion` for an `<exercise>` contained in a `<reading-questions>` with default “Reading Question”

Chapter 19

Figures and Friends

19.1 <image>

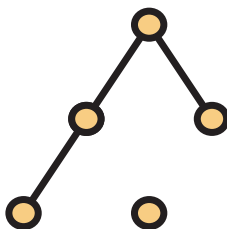
It is possible to include an image without a number or caption, centered on its own line by using <image>.

The gold standard for graphics to include in PreTeXt documents is, well, complicated. If you're only working with HTML or EPUB output then SVG is what you want. If you're producing PDF by using L^AT_EX, then you'll also want PDF graphics files. If you plan to produce an ebook that can be read on Kindle devices, you will need PNG graphics files. Fortunately, it's not too hard to convert between these formats on the command line.¹ In order to produce HTML, PDF, and Kindle, you will need to always have three files available. We recommend that you consistently pick PDF or SVG to be the one that you edit and then convert to the other format. Otherwise, you might have parabola.svg and parabola.pdf and parabola.png contain three different graphics, and then the different output formats for your project will have different images in them! PNG and JPEG are supported by modern web browsers, L^AT_EX, and Kindle, so that's a good option when vector graphic formats like SVG and PDF are not available or appropriate.

Listing 19.1.1 Code to include an image without a number or caption

```
<image source="small-graph" width="20%">
  <description><p>A graph on five vertices. One of the vertices is isolated,
    while the other four form a path on four vertices.</p></description>
</image>
```

Use of @width on an image must be a percentage, and for an <image> as in this example, the percentage is of the current line width. The code in [Listing 19.1.1](#) produces the following output:



The example in [Listing 19.1.1](#) also illustrates the use of <description>. We admit to not using a <description> for most of the other <image>s in this Guide, but doing so is strongly recommended for accessibility reasons. A reader who is unable to see the visual element of your book can use assistive technology to have the <description> read to them. As much as is practical, authors should endeavor to include <description>s for their <image>s.

Note that the path to the image file does not include the file extension. When you process your source, the output format you're generating will determine what gets added on so that the right file is used. If your browser says it can't find the image file, make sure that the svg file is in the correct location relative to the HTML file. Here, we need a directory called images that lives next to our HTML files with a file called small-graph.svg inside that directory.

¹Since conversion is a rare task, it may be easiest to do in a cloud environment like CoCalc.

If using a PNG or JPEG file across HTML, $\LaTeX$ /PDF, EPUB, and Kindle, put the extension in the filename so that the file is used for all. Many authors have an `images` directory that lives in the same directory as their PreTeXt source files and then produce the `.tex` file or HTML files in another directory. When doing so, you need to copy the `images` directory to be in the same directory as the `.tex` file before generating a PDF or in the same directory as your many `.html` files before viewing your HTML files in a web browser.

19.2 <figure>

To provide a number and caption for an `<image>`, the `<figure>` element is used. Notice that the `<caption>` is authored at the *beginning* of the `<figure>` but is displayed *below* the `<figure>`'s content.

Listing 19.2.1 Code to include a figure

```
<figure xml:id="fig-graph">
  <caption>A small graph (from <pubtitle>Applied
  Combinatorics</pubtitle> by Keller and
  Trotter)</caption>
  <image source="small-graph" width="20%" />
</figure>
```

The code in [Listing 19.2.1](#) produces the following output:

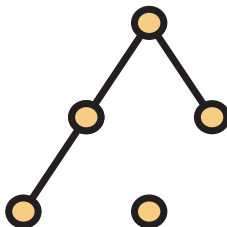


Figure 19.2.2 A small graph (from *Applied Combinatorics* by Keller and Trotter)

19.3 <sidebyside>

One of the more complex pieces of code in PreTeXt, by most accounts, is that used for positioning objects (frequently `<image>` and `<tabular>`, but also `<p>`) next to each other. If you've tried to do this in $\LaTeX$, you know that it can be challenging on a good day. Fortunately, PreTeXt does the heavy lifting for us.

We include two examples here. The first places the `<sidebyside>` directly in the current division and places a `<figure>` with a caption inside the `<sidebyside>`. The second puts the `<sidebyside>` inside `<figure>` and then uses an `<image>` not contained in a `<figure>` to include the graphic. It's possible to do all sorts of nesting and get nice subnumbering automatically. More information on the capabilities of `<sidebyside>` can be found in [Section 4.24](#).

Listing 19.3.1 Code to place things side by side

```

<sidebyside widths="25% 25%" valign="middle">
  <figure>
    <caption>Small graph with the caption only associated to the
    image. </caption>
    <image source="small-graph" />
  </figure>

  <p>
    We can put a paragraph here.
    Yes, a paragraph.
    Isn't this the most exciting paragraph that you've ever seen?
    It goes on and on and on and on and on and on.
    We want to put enough here to make it wrap, really, is all.
    Let's hope that this is enough.
  </p>
</sidebyside>

```

The code in [Listing 19.3.1](#) produces the following output:

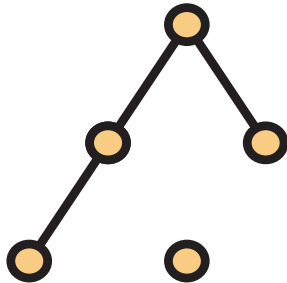


Figure 19.3.2 Small graph with the caption only associated to the image.

We can put a paragraph here. Yes, a paragraph. Isn't this the most exciting paragraph that you've ever seen? It goes on and on and on and on and on and on. We want to put enough here to make it wrap, really, is all. Let's hope that this is enough.

Listing 19.3.3 A few more bells and whistles for <sidebyside>

```

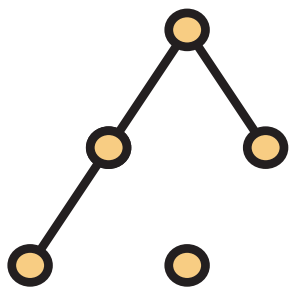
<figure>
  <caption>A graphic next to a paragraph of
  text with the caption assigned to both.</caption>
  <sidebyside widths="25% 35%" margins="0% 0%"
    valigns="bottom top">
    <image source="small-graph" />

    <p>
      We can put a paragraph here.
      Yes, a paragraph.
      Isn't this the most exciting paragraph that you've ever seen?
      It goes on and on and on and on and on and on.
      We want to put enough here to make it wrap, really, is all.
      Let's hope that this is enough.
    </p>
  </sidebyside>

</figure>

```

The code in [Listing 19.3.3](#) produces the following output:



We can put a paragraph here. Yes, a paragraph. Isn't this the most exciting paragraph that you've ever seen? It goes on and on and on and on and on and on. We want to put enough here to make it wrap, really, is all. Let's hope that this is enough.

Figure 19.3.4 A graphic next to a paragraph of text with the caption assigned to both.

For a layout with multiple rows (but the same @widths for each row, PreTeXt provides the side-by-side group using <sbsgroup>.

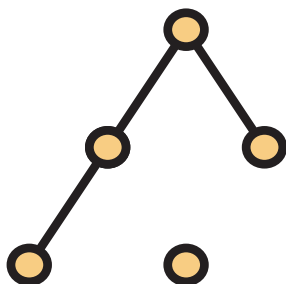
Listing 19.3.5 Use of <sbsgroup>

```
<sbsgroup widths="25% 50%" margins="5% 10%">
  <sidebyside>
    <image source="small-graph" />
    <p>
      A longer piece of text.
      It goes on and on and on.
      And on and on and on.
      And eventually we will let it end,
      but we want to show the width.
    </p>
  </sidebyside>

  <sidebyside>
    <p>
      Another little snippet of text.
      This time a bit longer so that it will wrap.
    </p>

    <p>
      We're going to keep this paragraph shorter.
    </p>
  </sidebyside>
  <sidebyside widths="75% 10%">
    <p>Here is some text in a row where we have overridden the widths
      specified above. Isn't this grand?</p>
    <p>Super duper narrow. Look at me!</p>
  </sidebyside>
</sbsgroup>
```

The code in [Listing 19.3.5](#) produces the following output:



A longer piece of text. It goes on and on and on. And on and on and on. And eventually we will let it end, but we want to show the width.

Another little snippet of
text. This time a bit longer so
that it will wrap.

We're going to keep this paragraph shorter.

Here is some text in a row where we have overridden the widths specified above. Super
Isn't this grand?

duper nar-
row. Look
at me!

19.4 $\text{\LaTeX}$ -generated images

PreTeXt makes it straightforward to embed $\text{\LaTeX}$ code that produces images (such as TikZ) into your source files. The PreTeXt-CLI basically just dumps your code out to your $\text{\LaTeX}$ file so that it compiles nicely. However, for HTML display, you will need SVG graphic files. This is where the pretext/pretext script comes in. Running the pretext/pretext script frequently requires patience, particularly on Windows, so settle in with an experienced user before attempting the steps in this section. See [Chapter 47](#) for complete details.

The pretext script. The pretext/pretext script requires a working Python installation, and works best with a virtual Python environment. See [Appendix C](#).

pdf2svg and ImageMagick are also pieces of the toolchain that may need to be installed separately, including on macOS. To generate graphics using SageMath, you'll need a working installation of it with a proper configuration of its path in the pretext.cfg file. Linux and CoCalc can probably get away without extra configuration here, but it will likely be required on macOS and Windows unless you've set up your SageMath install so you can run it from the command line by simply executing sage. More details on running the pretext/pretext script can be found in [Chapter 47](#)

Our example here just illustrates using TikZ to make a simple figure (the Hasse diagram of a poset), but lots of other $\text{\LaTeX}$ graphics packages can be used. One step required is to put the following three lines in the <docinfo> tag of your main file. <latex-image-preamble> is used to set up the preamble that should be used for making SVG images from your PreTeXt source. Macros that you wish to use more broadly should be put inside <macros> inside <docinfo>.

```
<latex-image-preamble>
\usepackage{tikz}
</latex-image-preamble>
```

Listing 19.4.1 How to use `<latex-image>` to invoke TikZ

```

<figure xml:id="fig-tikz">
  <caption>A figure generated with TikZ in
  <latex /></caption>
  <image width="15%" xml:id="poset">

    <latex-image >
      \begin{tikzpicture}
        \draw (0,0) -- (0,1);
        \draw (1,0) -- (1,1) -- (1,2) -- (1,3);
        \draw (0,0) -- (1,2);
        \draw (0,1) -- (1,0);
        \draw [fill] (0,0) circle [radius=0.1];
        \draw [fill] (0,1) circle [radius=0.1];
        \draw [fill] (1,0) circle [radius=0.1];
        \draw [fill] (1,1) circle [radius=0.1];
        \draw [fill] (1,2) circle [radius=0.1];
        \draw [fill] (1,3) circle [radius=0.1];
        \node [left] at (0,0) {$x$};
        \node [left] at (0,1) {$b$};
        \node [right] at (1,0) {$a$};
        \node [right] at (1,1) {$y$};
        \node [right] at (1,2) {$c$};
        \node [right] at (1,3) {$d$};
      \end{tikzpicture}
    </latex-image>
  </image>

</figure>

```

The code in Listing 19.4.1 produces the following output:

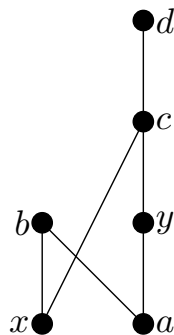


Figure 19.4.2 A figure generated with TikZ in L^AT_EX

Well, that's not 100% true for HTML. If you just run `pretext build web`, your terminal will display a few warnings about not generating assets such as images. To generate the assets, we have to use the `-g` flag. To generate the assets before building, on the command line we run the following command (on a single line). (Omitting the web target will, by default, use the first target) in the `project.ptx` file.)

```
pretext build web -g
```

For the same build using `xsltproc`, you would use the following:

```
$ /path/to/pretext/pretext -c latex-image -f
  svg -d /path/to/html/output/directory/images /path/to/yoursource.ptx
```

Chapter 20

Tables

After `<sidebyside>`, getting tables to lay out consistently between HTML and PDF is probably the second biggest headache that PreTeXt takes care of for us behind the scenes. Considerable effort has been taken in order to fix some of the challenges inherent to working with the `tabular` environment in $\text{\LaTeX}$, and so if you author in PreTeXt, you should be able to forget the hacks you had to learn to make nice tables in $\text{\LaTeX}$.

Tables should only be used to display data. Too often in other authoring systems, tables are used as a crutch to facilitate the visual layout of a page. Do *not* do that when authoring PreTeXt. A good question to ask yourself before using a `<tabular>` is “Do the xy -coordinates of a cell have semantic meaning in terms of my data?” If the answer is “yes”, then make an array of numbers with `<tabular>`. If not, find a more suitable tag. (Perhaps `<sidebyside>` and/or `<sbsgroup>`.) One of the many reasons for this is that screen readers used by individuals with visual impairments read tables in a very specific way that assumes the xy -coordinates of each cell are contributing to the meaning. Individuals who use screen readers will find a document that uses tables to do something other than present tabular data very confusing and frustrating.

Similar to $\text{\LaTeX}$, PreTeXt provides a `<table>` tag and a `<tabular>` tag. The `<tabular>` tag is used for producing the array of data, while the `<table>` tag provides the number and title.

Listing 20.0.1 Code to produce a table

```

<table>
  <title>A simple table</title>
  <tabular halign="center">
    <row header="yes" bottom="minor" >
      <cell>Variable <m>x</m></cell>
      <cell>Variable <m>y</m></cell>
      <cell>Conjunction <m>x\wedge y</m></cell>
      <cell>Disjunction <m>x\vee y</m></cell>
    </row>
    <row>
      <cell>T</cell>
      <cell>T</cell>
      <cell>T</cell>
      <cell>T</cell>
    </row>
    <row>
      <cell>T</cell>
      <cell>F</cell>
      <cell>F</cell>
      <cell>T</cell>
    </row>
    <row>
      <cell>F</cell>
      <cell>T</cell>
      <cell>F</cell>
      <cell>T</cell>
    </row>
    <row>
      <cell>F</cell>
      <cell>F</cell>
      <cell>F</cell>
      <cell>F</cell>
    </row>
  </tabular>
</table>

```

The code in Listing 20.0.1 produces the following output:

Table 20.0.2 A simple table

Variable x	Variable y	Conjunction $x \wedge y$	Disjunction $x \vee y$
T	T	T	T
T	F	F	T
F	T	F	T
F	F	F	F

Much like `<image>`, you can use `<tabular>` on its own to lay out a table of data, centered on a line, between (for instance) a couple of `<p>` elements. In many cases, the sort of data layout generated using `<tabular>` functions more as a figure than a table according to the definitions in the *Chicago Manual of Style* [1] (CMOS), which PreTeXt attempts to follow in the absence of other guiding principles. The quote below from David Farmer on the [pretext-dev Google Group](#) provides guidance on deciding if your `<tabular>` should be contained in `<figure>` or `<table>`.

“There is an entire chapter on tables in CMOS, so I’ll try to summarize how those are distinct from many uses of grids of numbers in PreTeXt. Some approximate quotes about tables from CMOS:

- facts that are easy to scan and compare
- a reader unfamiliar with the material should still be able to make sense of the table

- tables are numbered and have titles (and not captions)

My take-away is that a CMOS table is a supplement to what is written in the text and at any one time only a small amount of the table is relevant. Rephrasing: a CMOS table is not intended to be integrated with the narrative of the book, and the reader is not supposed to pore over a large fraction of the table when it is first encountered.”

—David Farmer

In reality, the `<tabular>` in [Listing 20.0.1](#) really should be contained in `<figure>` (and then the `<title>` must become a `<caption>`). Perhaps someday we will come up with an example `<tabular>` that meets the CMOS definition of a table to use instead.

See [Section 4.19](#) for more information about how to make more complicated tables including formatting columns and vertical and horizontal rules.

Chapter 21

SageMath Content

21.1 SageMathCells

Including computational SageMath cells is pretty easy with `<sage>`, `<input>`, and `<output>`. The last tag is useful for producing formats intended for offline reading so that they can include the result of the code's execution.

Listing 21.1.1 SageMath cell

```
<sage>
<input>
2+2
</input>
<output>
4
</output>
</sage>
```

The code in [Listing 21.1.1](#) produces the following output:

```
2+2
```

```
4
```

SageMathCells on a single HTML page are automatically linked so that a cell can use the results of computations done in earlier cells on the same page.

21.2 `<sageplot>`

Sometimes you don't want to provide an interactive SageMath environment in the middle of your book (or a chunk of code) but you would like to produce a figure to include in your project by using SageMath. The cleanest way to do this is to put the SageMath code right in your PreTeXt project and use the pretext script that we discussed in [Section 19.4](#) to produce the image files required for your chosen output formats. This is accomplished by using `<sageplot>` with the script. (The pretext script is fully discussed in [Chapter 47](#), but at least see the aside in [Section 19.4](#) about the additional packages that must be installed and configured to use it properly.)

Listing 21.2.1 <sageplot> to produce a graphic

```

<figure xml:id="fig-sage-cubic">
  <caption>A cubic plotted by SageMath on
  <m>[-3,2]</m></caption>
  <image xml:id="sageplot-cubic" width="50%">
    <description><p>A cubic function on the interval
    [-3,2]</p></description>

    <sageplot>
      f(x) = (x-1)*(x+1)*(x-2)
      plot(f, (x, -3, 2), color='blue', thickness=3)
    </sageplot>
  </image>

</figure>

```

We need to run the pretext script to actually make the image files required. If you want to make both HTML and PDF via L^AT_EX, you'll need to run it twice. The first command below (again, enter on one line) makes the SVG to use on the web, and the second makes what you need for L^AT_EX. There is an all option that can be passed after -f instead of svg or pdf, but that is more likely to raise errors because some source code cannot produce certain output formats. It's best to stay away from error-producing steps until you're comfortable with debugging your system.

```

$ /path/to/pretext/pretext/pretext -c sageplot -f svg -d ./images
/path/to/yoursource.ptx

$ /path/to/pretext/pretext/pretext -c sageplot -f pdf -d ./images
/path/to/yoursource.ptx

```

The code in Listing 21.2.1 produces the following output.

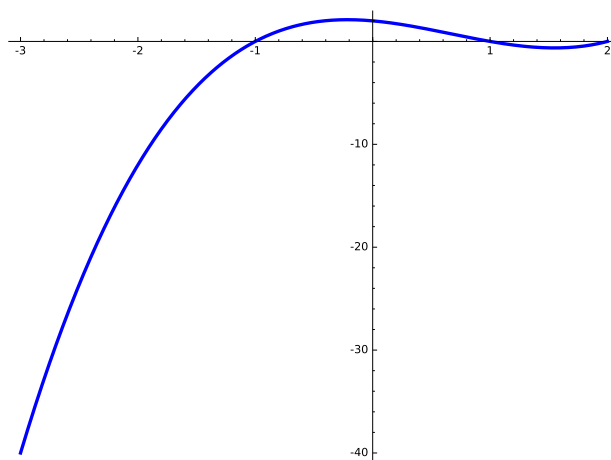


Figure 21.2.2 A cubic plotted by SageMath on $[-3, 2]$

Chapter 22

Small, But Useful

The topics in this section are not terribly structural or critical, but they fall in the category of “little things you want to do right from the outset”.

22.1 Cross references

PreTeXt provides a robust set of features for internal cross referencing. If you’re familiar with $\LaTeX$, the equivalent of `\label` is to use an `@xml:id`. For example, the opening tag for this section is `<section xml:id="basics-s-xref">`. Instead of the `\ref` used by $\LaTeX$, we use `<xref>` in PreTeXt. So we can type `<xref ref="basics-s-xref"/>` to create a reference to this subsection: [Section 22.1](#). There are lots of options to control what text and number appear when you use `<xref>`. The default is `@text` with value `type-global`, which produces something like “Subsection 3.3” or “Theorem 3.1.4”. The type is “Subsection” or “Theorem”, and the *global* number is 3.3 or 3.1.4. (Global is in contrast to *local*, which would be just 3 or 4, respectively, for these examples.) [Section 4.5](#) goes into greater detail about how to change settings for how cross references appear, which you can do for your entire document as well as for individual cross references that require different treatment.

22.2 External links

If you want to provide a link to a resource outside of your project, you will want the `<url>` tag. The code

```
<url href="https://pretextbook.org" visual="pretextbook.org">PreTeXt</url>
```

produces [PreTeXt](#). Note that in a print PDF version, a trailing parenthetical with the content of `@visual` is automatically created.

Extensive details are provided in [Section 4.32](#).

22.3 Footnotes and asides

Footnotes are not too hard, just use `<fn>`, but note that for the time being, what can go inside a footnote is very, very restricted.¹ For instance, you can’t put a `<p>` (and thus you can’t put lists) inside a footnote. Also, no displayed math via `<md>`. There’s a lot of care being taken because of the prospect of footnotes inside footnotes inside footnotes.

Because of the restrictions on footnotes, it is important to keep them short. A good alternative for longer things that are somewhat digressional is the *aside*, which comes in three flavors: `<aside>`, `<biographical>`, `<historical>`.

Each of these allows an optional title and then a variety of tags such as `<p>`, `<figure>`, and `<sidebyside>` (and many more).

¹This is a sample footnote, just so you can see how one looks.

Listing 22.3.1 A sample <aside>

```

<aside>
  <title>A Sample Aside</title>
  <p>
    Here is some text inside a sample aside.
    I'm going to put the Pythagorean identity below:
    <md>
      \sin^2(x) + \cos^2(x) = 1
    </md>.
  </p>

  <p>
    This aside has two paragraphs.
    You can't do that with a footnote.
  </p>
</aside>

```

The code in [Listing 22.3.1](#) produces the aside “A Sample Aside”. A less contrived example of an aside can be found in [Section 19.4](#).

A Sample Aside. Here is some text inside a sample aside. I’m going to put the Pythagorean identity below:

$$\sin^2(x) + \cos^2(x) = 1.$$

This aside has two paragraphs. You can’t do that with a footnote.

22.4 Index entries

PreTeXt does a good job of supporting index generation. You still need to tag everything that should get an index entry by hand, but then the index is produced automatically. For a simple index entry for the word “group”, you just use <idx>group</idx>. If you need an index entry involving subheadings, such as “normal” under “subgroup”, use <idx><h>subgroup</h><h>normal</h></idx>.

It is also possible to use “see” and “see also” entries for indices. For instance, in [Chapter 11](#), we use

```
<idx><h>font</h><see>formatting</see></idx>
```

to create an index entry for “font” that instructs the reader to “*see* formatting”. A “*see also*” can be created using the <seealso> tag instead of <see>.

If the index entry is in danger of being alphabetized incorrectly, you can specify how it should be sorted with the @sortby attribute. For example, to give an index entry to Σ in the “s” section, you would use <idx sortby="sigma"><m>\Sigma</m></idx>. Note that if the index entry has <h> tags, the @sortby attribute should go there, not on <idx>.

If you generate PDF output from L^AT_EX, the index is automatically generated.

22.5 Quotations

To ensure that quotation marks are properly typeset, it is important to use the correct PreTeXt code. To set something off in double quotes, use the <q> tag around what should appear in quotes. It will supply both the opening and closing quotation marks, as in: “This is a quotation.” If you need single quotes, use <sq>. Because the content of <q> and <sq> is quite restricted, you may find yourself needing to explicitly access the left and right single and double quotation marks. They are, quite sensibly, <lq>, <rq>, <lsq>, and <rsq>.

Longer quotes are best set off using <blockquote>

Listing 22.5.1 A sample <blockquote>

```
<blockquote>
  <title>Lorem ipsum</title>
  <p>
    <q>Lorem ipsum dolor sit amet,
    consectetur adipiscing elit,
    sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.
    Ut enim ad minim veniam,
    quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo
    consequat.
    Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu
    fugiat nulla pariatur.
    Excepteur sint occaecat cupidatat non proident,
    sunt in culpa qui officia deserunt mollit anim id est laborum.</q>
  </p>
  <attribution>Anonymous</attribution>
</blockquote>
```

The code in [Listing 22.5.1](#) produces the quotation below.

“Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.”

—Anonymous

Chapter 23

Modular Source

Once a project gets big, you may find yourself wishing to break your source into multiple files. This is well documented in [Section 5.3](#), so we refer you there for more details.

Part IV

Publisher's Guide

Chapter 24

(*) Producing a Book

You have finished using PreTeXt to write a textbook, research monograph, laboratory manual, writer's handbook, or some other scholarly work, and you want to share it openly with students and other readers. Good! That's why we created PreTeXt, and why we include this chapter. There is more to do.

This chapter will try to distill our personal experience, along with many conversations with other authors confronting these same decisions. But recognize there is a lot of *advice* here, and a bit of an attitude, simply for your consideration. The decisions are yours.

Open Source. PreTeXt is open source *software*, but that places no restrictions on how you use output that PreTeXt creates (see [Item 1.1.1:8](#)). You are welcome to sell your copyright to a commercial publisher or university press. But just once, here and now, we will encourage you to consider an open license (see [Chapter 25](#)) as a way of paying it forward.

24.1 (*) Front Matter

TODO

24.2 (*) Landing Page

TODO

24.3 (*) Source

TODO

24.4 (*) Marketing

Didn't think about that one? Write it, and they will come? Not necessarily. I have seen nice projects where authors make little extra effort to get the word out, and it shows. With the Internet, effective promotion can be accomplished without much effort or expense. And very soon your project can rise very high in search engine rankings.

24.4.1 (*) Social Media

TODO

24.4.2 (*) Reference Sites

TODO

24.4.3 (*) Analytics

TODO

24.4.4 (*) Discussion Groups

TODO

Chapter 25

Copyright and Licensing

The legal issues described here are based on the authors' experiences and study, which necessarily reflect the laws of the United States. But the [Berne Convention](#), which dates to 1886, has 172 parties, so law and practice are very similar the world over. The United States acceded to the convention in 1988.

25.1 Copyright

Copyright is a monopoly granted by the government. It gives the author control over reproductions, translations, adaptations, performances, communications, etc. of their work for a fixed time. Since 1978, it has not been necessary to register a copyright—it is automatic. So for example, every web page, no matter how simple or unrefined, is copyrighted by its author.

The phrase “All Rights Reserved” is used to assert that the copyright holder intends to exercise all the rights granted by copyright. It is not required to mark a document with the copyright symbol (©) but in case of disputes, it can be helpful.

So in PreTeXt you can go

```
<frontmatter>
  <colophon>
    <copyright>
      <holder>Thomas Jefferson</holder>
      <year>1776</year>
      <minilicense>All Rights Reserved</minilicense>
    </copyright>
  </colophon>
</frontmatter>
```

to assert a “traditional” copyright and the recommended information will then appear on the page after the title page.

In academic publishing, authors have usually transferred, or sold, their copyright to a publisher in return for distribution of their work, or for the promise of financial gain.

25.2 Open Licenses

Copyright allows an author to place a license on their work, granting others greater freedoms, sometimes along with certain specific obligations. So it is important to understand that copyright allows an author or publisher to be very restrictive, and it also allows an author or publisher to be less restrictive. A license makes these less restrictive terms explicit, and the ability to control these terms is made possible by copyright.

Generally an open license allows unlimited copying. It often allows the creation of derivative works, and the mixing of material from a variety of openly licensed documents. A **viral license** obliges the author of a derivative

work to grant the same license to the derivative work, rather than asserting more restrictive terms. The licenses are usually perpetual, so they do not expire at a fixed term.

It is easy to get distracted by legal jargon, obtuse arguments, and misunderstandings. We view an open license as statement of intent. The work is free to use forever. It will not go out of print. If you send the author a correction, suggestion, or contribution, it can be incorporated and enjoyed by others freely. And should an author lose interest in a project, or become unable to continue working on it, another individual may take it up and continue to maintain it.

25.3 Creative Commons Licenses

We describe the Creative Commons (cc) licenses first, since they have various options, which are a convenient way to compartmentalize and describe the features of other open licenses. They are known by abbreviations, so a license might be shortened to something like just CC BY-SA.

List 25.3.1 Creative Commons Options

Creative Commons, CC	All Creative Commons licenses allows unlimited copying, forever. Unless restricted, derivative works are allowed with no obligation.
Attribution, BY	A derivative work must make clear the contribution of the original author.
Share Alike, SA	A derivative work must be licensed with the same license as the original.
Non-Commercial, NC	Bans commercial uses. (Without this clause an open license would allow the sale of copies.)
No Derivatives, ND	Derivative works are not allowed.

Source versus Output. Creative Commons licenses are designed for a variety of media, and so are very popular. For example, images are often licensed with a Creative Commons license.

However, consider the case of a document distributed as a PDF which has been created from source, such as PreTeXt or $\LaTeX$ or Markdown. An author may put a CC BY-SA license on the PDF while retaining traditional copyright on the source file(s). While the license allows modification, how practical is it to modify a PDF? Worse, we have seen this situation for projects described as “open source.” This explains our use sometimes of the term “openly licensed content.”

If you are serious about your project being open source, and you want to send that signal to your readers, then mark your output with a cc license *and* mark your source files as also having the same cc license. This is usually accomplished by a statement at the top of each source file asserting copyright and then stating the license. Or this statement can point to a top-level text file, often named COPYING or LICENSE, with more precise license information. See the PreTeXt distribution for examples using the GPL software license.

Commercial Consideration. It is natural to consider that you have devoted considerable time and effort to your project, and with an open license you will now be donating it to the world to read for free. So you might think, “Why should somebody else profit?” Thus, the NC option is alluring. But think about it for a minute.

Suppose a commercial publisher hires an experienced copy editor to go through your book, correcting errors and tightening the prose. Then the publisher prints the improved version of your book in a nice hardback version, and sells the book for \$250. If you simply put the BY-SA options on your license, that publisher is obligated to (a) give you credit for authoring the original version of the book, and (b) make the copy-edited version available again with a CC BY-SA license. Now there is a business opportunity for anybody in the world: make a print-on-demand version of the copy-edited version, and sell it for \$200. Then it is a race to the bottom. Eventually the price will come down to the cost of manufacturing, plus some small compensation for managing the relationship with a print-on-demand service [Chapter 43](#). Just how much profit potential is there really available for others?

We assume you have written a book because you have something to say, and chosen an open license because you want to be read. If you do place an NC option on your cc license, then you have made it impossible for anybody

else to help you distribute your book in physical forms. A CC BY-SA license already makes your work unattractive to a commercial publisher who wants to charge an unreasonable price, and adding an NC clause simply chokes off some distribution options, limiting the reach of your work.

This discussion is meant to explain why we call it the “CC-NC mistake.”

Derivative Works. Will your monograph on the reptiles of the Mongolian steppes ever become a screenplay? As exciting as the topic is to you, we think Hollywood feels otherwise. An ND option probably makes little sense for scholarly works. An conversely, if you get hit by a pie truck tomorrow, it will be a lot easier for an enthusiastic reader to take over custody and maintenance of your project, and they will be *required* to continue with the same license if you have employed the SA option.

So in PreTeXt you can go

```
<frontmatter>
  <colophon>
    <copyright>
      <holder>Steve Jobs</holder>
      <year>1984</year>
      <minilicense>Creative Commons BY-SA</minilicense>
    </copyright>
  </colophon>
</frontmatter>
```

for a cc license with the Attribution and Share-Alike options. If it was not obvious already, this is our recommendation for scholarly work if you choose to use a Creative Commons license. This is an example of what is known as a **copyleft** license.

25.4 GNU Free Documentation License

The GNU Free Documentation License (GFDL) is a license designed for documentation of open source computer programs licensed by the GNU Public License (GPL). However, it explicitly mentions textbooks as one possible use. (GNU is a recursive acronym for “GNU’s Not Unix” and is the software project that originally built all the utilities which complement the Linux Kernel to make up an operating system.)

The GFDL is similar to a CC BY-SA license. It allows unlimited copying, forever. Modified versions that are distributed must acknowledge the original contributions and must also carry a GFDL license. So this is a viral license, always. And another example of copyleft.

However, the GFDL does not employ options like a Creative Commons license. More importantly, the GFDL is very explicit about source (“transparent” copies) and derived output (“opaque” copies), and the license applies to both versions. So the GFDL would say PreTeXt source is transparent, and a resulting PDF is opaque, and the license largely treats them identically.

Because the GFDL has the essential characteristics of CC BY-SA, and is so explicit about simultaneously licensing document source authored with a markup language along with output formats, it is our choice for textbook projects.

So in PreTeXt we might go something like

```
<frontmatter>
  <colophon>
    <copyright>
      <holder>Richard Stallman</holder>
      <year>1985</year>
      <minilicense>GNU Free Documentation License</minilicense>
    </copyright>
  </colophon>
</frontmatter>
```

The GFDL is also explicit about including the complete license with your document. You can find various places a version formatted for inclusion in a PreTeXt project, including as part of the source for this document.

25.5 Public Domain

Stating that your work is in the **public domain** basically means you relinquish all the rights you receive via your automatic copyright. So it is a very different situation from using copyright to provide an open license. Creative Commons uses CC0, “No Rights Reserved” to indicate this choice.

25.6 Remixing and License Compatibility

It is popular to describe the potential of remixing parts of open educational resources. For example, a literature professor might collect a variety of openly licensed poems into a reader for students in a course. When the licenses are viral, and different, there arises the problem of what license to put on the collection. Worse, one license might prohibit commercial uses, and another allow it, meaning the licenses are incompatible.

So some thought should go into the choice of a license when the work has the very real potential to be included in another, such as would be the case with a photograph. One solution is to provide more than one license (nothing about copyright prohibits this). Another solution is to avoid licenses with overly restrictive terms, such as restricting commercial use or derivative works.

Remember too, that in addition to multiple licenses, as the copyright holder you may offer your work to another project on different terms. So another author might ask if a chapter of your work may be included in their project, which might use a different license than yours (more or less restrictive), and you can grant permission for that use under that license. Now there are two versions of your chapter, which could diverge over time if derivatives are allowed, available to others on different terms.

For mathematics books, we do not concern ourselves too much with the potential for remixing. Notation and dependencies make it hard to collect parts of various textbooks and assemble them into something new (and coherent).

Creative Commons maintains a matrix showing compatibility between their own licenses at their FAQ question, [Can I combine material under different Creative Commons licenses in my work?](#), illustrating just how complicated this can become.

Finally, recognize that you can begin with a restrictive license and as you become more comfortable with the idea, change it to a more liberal license that applies to the work at the time of the change. Further, you can always change your license to a more restrictive version, but invariably, you cannot revoke an open license once granted. You could even stop offering an open license all together, and return to traditional copyright as you continue to improve your document. But the version that existed when you made that decision is still available for anyone to use, and possibly improve, independent of your own closed version.

After a while, you realize that openly licensing your writing project gives it an independence and freedom all of its own. It seems to be owned by everybody, and by nobody—at the same time.

25.7 A Final Comment

These discussions remind us of a [letter by Thomas Jefferson](#), the principal author of the United States’ *Declaration of Independence*, writing about another monopoly granted by governments—patents.

If nature has made any one thing less susceptible than all others of exclusive property, it is the action of the thinking power called an idea, which an individual may exclusively possess as long as he keeps it to himself; but the moment it is divulged, it forces itself into the possession of every one, and the receiver cannot dispossess himself of it. Its peculiar character, too, is that no one possesses the less, because every other possesses the whole of it. He who receives an idea from me, receives instruction himself without lessening mine; as he who lights his taper at mine, receives light without darkening me. That ideas should freely spread from one to another over the globe, for the moral and mutual instruction of man, and improvement of his condition, seems to have been peculiarly and benevolently designed by nature, when she made them, like fire, expansible over all space, without lessening their density in any point, and like the air in which we breathe, move, and have our physical being, incapable of confinement

or exclusive appropriation.

—Thomas Jefferson
Letter to Isaac McPherson
August 13, 1813

Chapter 26

Conversions, Generally

A main goal of PreTeXt is to provide a language for describing a scholarly document by its structure, with contained content, and with no description of the presentation. It then becomes possible to use software to produce different formats, where the presentation takes advantage of that format and enhances the meaning of the content through the expression of the structure.

But different output formats have different capabilities. For example, a conversion to HTML can take advantage of knowls to organize smaller chunks of content, while a conversion to PDF can take advantage of page numbers for cross-references. And in these two examples, the capability of the one output format is mostly impossible or silly in the other. Look here in this chapter for notes about options that are largely *independent* of the particular conversion. Subsequent chapters contain notes about options that are largely *specific* to a particular conversion.

26.1 Publication File

A key concept when using PreTeXt to describe your writing project is that an author should concentrate on *content* and then *later* a publisher can concentrate on *presentation*. (Of course, many PreTeXt authors are also their publisher, making this distinction more difficult than when a professional publisher was a necessity).

We isolate these publisher decisions in a file we call a **publication file**. Generally, use of this file will change *how* words look, or are arranged, on the page, but will not change the author's words *themselves*. some software might call this a "configuration file", but we think it is very important to indicate its role in the publication process.

In this section, we describe how to create and employ this file. Details on actual options can be found throughout this guide, with terse comprehensive reference material in [Chapter 44](#).

Create a separate XML file the same way you always would. Include the usual XML declaration as the first line. Now, instead of the overall element being <pretext>, use <publication>. That's it. Various elements within <publication> will be used to specify options, typically attributes. Name the file something that reminds you of its purpose, such as pod.xml for a print-on-demand version. Avoid using spaces in the filename, even if your operating system encourages it.

Entries that control aspects of the output are often attributes of various elements, but may also be the content of elements. When you read the reference material in [Chapter 44](#) be aware that we use a sort of shorthand to describe these entries, modeled on a specification called XPath. For example, if we say to set

```
/publication/foo/bar/@bazz
```

to possible values of yes, no, or maybe then the following will be the guts of a legitimate publication file that would somehow adjust your output in some way. In particular, note that the "at sign", @, indicates an attribute of the preceding element.

```
<publication>
  <foo>
    <bar bazz="maybe"/>
  </foo>
</publication>
```


When using the CLI ([Section 5.2](#)), the path to a publication file must be specified in the project manifest in the `<publication>` element for each `<target>`.

26.2 Levels Explained

Every PreTeXt document has a hierarchy, even if it might not be very deep. As an extreme example, for a sub-sub-section of a book, the `<subsubsection>` is contained in a `<subsection>`, that `<subsection>` is contained in a `<section>`, that `<section>` is contained in a `<chapter>`, and that `<chapter>` is contained in the `<book>`. Each division of a document has a **level**, and the overall **root** element is always at level 0 (the `<book>` in the example). Each other division is at some **depth**, computed by counting from the root. So the `<subsubsection>` in the example is at level 4.

Many aspects of the different outputs produced can be customized, typically via the publication file ([Section 26.1](#)), based on how much of the hierarchy is used or made visible. A good example is the Table of Contents. If the level of the Table of Contents of a book is set to 2, then the Table of Contents will be “two-deep” or have “two levels” of entries. More precisely, there will be titles (and maybe page numbers) for every `<chapter>` and every `<section>`.

When hierarchical numbering is customized by specifying a level, the number of an object will have as many separators (periods, typically) as the level given. Here’s why. Suppose equations are set to be numbered at level 2. Then two levels of the hierarchy will be used to create the initial part of the number. So in an `<article>`, Equation 5.2.34 will be in Subsection 2, of Chapter 5, and then will be the 34th equation of that sub-section. Two levels: one separator to describe the division, one less than the number of levels (this is the **structure number**) and a second separator to set off the count within the division (the **serial number**). So, $(2 - 1) + 1 = 2$ separators. See more on numbering at [Section 26.3](#).

Notice that you make no assumptions or decisions in your source about the depth of the Table of Contents, nor the numbering of equations. At any time, right up to the completion of your project (or later!), you can change this aspect of your output with nearly trivial edits in the publication file. Nice.

26.3 Numbering Explained

PreTeXt targets the production of “structured scholarly documents”—not novels, not magazine articles, not menus, and not travel brochures. (Though a novel could work well?) A research monograph might only be consulted for a portion of its content. A good textbook should be useful to a reader after a course is over, and it should be easy to locate certain portions of the material. A good textbook will foreshadow later material, and reinforce earlier material. So we provide tools that lead to a quality index, reduce the overhead of making an accurate cross-reference, and make an automatic Table of Contents. Page numbers can be a useful way to locate information for print output, but are less useful in an electronic PDF with hyperlinks, and are totally useless for online HTML and reflowable EPUB. So we rely on copious hierarchical numbering to assist with locating discrete pieces of content.

A PreTeXt document, like most any scholarly document has a hierarchy of **divisions**. These are always numbered to reflect that hierarchy. So Subsection 4.7.2 of a `<book>` is the second sub-section of the seventh section of the fourth chapter of the book. It is possible to specify that the numbering stops at some level, but that will limit how you can number smaller units of content.

Blocks are units of content held in a division. An `<example>` is a good example. These are always numbered, so that cross-references are as useful as possible in all output formats. The number begins with a **structure number** that is the number of a division. The division will contain the block, but does not need to be the closest containing division. For example, Example 5.2.65 of a book has structure number 5.2, indicating it is in Section 2 of Chapter 5. But this section might be structured as a sequence of sub-sections and Example 5.2.65 would be contained in one of these sub-sections. But if we started counting all the examples in this *section* we would find Example 5.2.65 as the sixty-fifth numbered block of the whole *section*, even if it might only be the twenty-second example of its sub-section. This final number is known as the **serial number**. The granularity of the structure number may be configured. This is a good place to suggest the complementary [Section 26.2](#) on the meaning of the term **level**.

Numbering of equations and footnotes may be configured in a manner entirely similar to that of blocks. See [Section 44.2](#) for details on how to control this.

As mentioned above, a number is a kind of **locator**—it should help a reader locate content, via a cross-reference, an index entry (a specialized cross-reference), or a Table of Contents. It should also help a reader (teacher) tell

another reader (student) where to find content, perhaps as part of a citation to a smaller item within a larger work. How would you locate Example 5.2.65? A Table of Contents, in any output format should get you to Section 5.2 quite easily. We claim that the `<remark>` immediately preceding Example 5.2.65 should be Remark 5.2.64. In other words, it will easier to scan the section and quickly home in on the example if the serial numbers count *all* the numbered blocks, rather than having one sequence of serial numbers counting examples, and a second sequence counting remarks. Not convinced? Suppose there were two such sequences of serial numbers. When you see Remark 5.2.23, should you move forward or backward in your search for Example 5.2.65?

If it is so important to not have separate sequences of serial numbers, then why do equations and footnotes get their own sequences? These items are substantially different visually, and even their numbers are formatted quite differently, so scanning for blocks or equations or footnotes should be very distinct visually. Notice that it is their *distinctive appearance* that is the criteria for an independent sequence of serial numbers.

We have implemented some flexibility for figures and tables, and for projects. This work is in flux, so we have not yet documented the possibilities. Our view is that figures and tables can be considered visually different enough to merit a separate sequence of serial numbers.

Divisions that are `<chapter>`s, and only `<chapter>`s, may begin with a number other than one. Primarily this is to accommodate books that need to be printed in multiple physical volumes, so numbering in a second (or subsequent) volume can be correct. We also understand the instructive value of a computer science text that wants to start counting from zero. We do not mean to encourage a Chapter 0 that is an introduction (go ahead and title Chapter 1 “Introduction”) or background preparatory material (make that an appendix). Understand that a `<preface>`, or multiple `<preface>`s, is the place to talk about how, or why, you wrote your book, and/or a place to instruct a reader or instructor about the best ways to use your book. See [Section 44.2](#) for details on how to accomplish this.

Best Practice 26.3.1 Use Chapter Zero Carefully. Chapter numbering may start with a number other than one, and zero is a popular choice. This should not be simply because the first chapter is introductory or preparatory, nor should it be a replacement for a preface, which has a well-defined purpose (see [Best Practice 4.25.2](#)).

26.4 Exercise Component Visibility

The `<statement>`, `<hint>`, `<answer>`, and `<solution>` elements are collectively the **components** of exercises, projects (and similar), and tasks they may contain. When you author an exercise, you may only want to have a solution appear at the end of a book, or in a solution manual, and not right where the exercise is born. There are twenty yes/no settings (four components for each of five types of exercises), which control visibility *at the location where the content is born*. The default value for each is yes, which means the component is visible. See [Subsection 44.1.3](#) for complete details.

This is a good example of settings you may wish to employ differently for different output. Since HTML output automatically puts these components into knows (Section 29.2), you may be less concerned about having them visible as part of the exercise itself. Conversely, for PDF/print output, you may wish exercise components to only be visible at the end of a book. Note that migrating solutions to another location is accomplished with a `<solutions>` specialized division ([Subsection 4.13.2](#)).

Finally, these settings are observed by the stylesheet which creates a solution manual (see [Section 37.1](#)). This might explain why controlling the visibility of a `<statement>` is of interest.

26.5 Watermarks

Output can contain a **watermark**, which is prominent text in the background of a document. You might use it to make a draft version (with a date?), or a CONFIDENTIAL version, or a document intended for limited distribution, such as an instructor’s solution manual.

In PDF output this text will render as large, light grey text, at an angle across the page. HTML output will have repeated svg images of the text below/behind the usual text.

To use, a publication file entry can be used to specify the text itself, along with a scaling factor that can be used to fill the page and prevent overruns. Note that the text is assumed to be simple (e.g. no markup) and when used with L^AT_EX output certain characters (e.g. a percent sign) may cause problems. Make a feature request if the handling of this text needs to be more robust. See [Subsection 44.1.6](#) for exact syntax.

Here is an example of a specification in the publication file.

```
<common>  
  <watermark scale="1.2">DRAFT 2034-05-25</watermark>  
</common>
```

Chapter 27

Custom Versions

An **edition** would be a copy of your project, different than one produced at some other time. For example, you might elect to make annual editions, with corrections, additions, and deletions. By contrast, a **version** is a copy of your project that shares content with another version, but differs in minor or substantial ways. We provide mechanisms for minor differences in [Section 27.1](#) and for substantial differences in [Section 27.2](#). Be sure to read about both mechanisms first, and understand their purposes, before committing to one or the other.

27.1 Customizations

The customization feature allows you to create two or more versions of your text for slightly different audiences. To do this, an author defines custom elements that find their translations in a file specified in the publication file. Multiple customization files can then be used to build different versions of the text. For example, we might want to create two versions of our AOTA book, one for zoologists and one for laypeople. For the zoologist edition we want animal names to be scientific names, but in the edition for laypeople we want to use common names. The rest of the text is identical. To execute this, we make every animal name a custom element and create a customization file for each of the two versions.

Say we want to write a sentence that reads “The IUCN red list classification of the western lowland gorilla is critically endangered” in the layperson’s edition and “The IUCN red list classification of the *Gorilla gorilla gorilla* is critically endangered” in the zoologist’s edition. We begin by writing the sentence in the source and creating a `<custom>` element as a place-holder for the name of the gorilla:

```
The <init>IUCN</init> red list classification of
the <custom ref="gorilla-name"/> is critically endangered.
```

Then the file of translations will contain a `<custom>` element, with a `@name` attribute that has value `gorilla-name`, and whose content is the translation.

Once you have placed `<custom>` elements in your source, you need to create one or more files of translations. To stay organized you might choose to place them in a directory of their own. The customization file opens with a special element and then contains a definition for each customization.

For the example above, here is the beginning of the customization file we might name `customizations/zoologist.ptx`.

```
<pi:customizations xmlns:pi="http://pretextbook.org/2020/pretext/internal">
  <!-- Name of Western mountain gorilla -->
  <custom name= "gorilla-name">
    <taxon>Gorilla gorilla gorilla</taxon>
  </custom>
</pi:customizations>
```

Note that employing a common value for the `@name` attribute and the `@ref` attribute makes the association for the replacement in the source. Next, we would also create a customization file named “`customizations/layperson.ptx`” that looks like the following.

```
<pi:customizations xmlns:pi="http://pretextbook.org/2020/pretext/internal">
  <!-- Name of Western mountain gorilla -->
  <custom name= "gorilla-name">western lowland gorilla</custom>
</pi:customizations>
```

Once the customization files are created, the element `<custom ref="gorilla-name"/>` can be used *throughout* the text and will populate automatically depending on which customization file is specified in the publication file.

When choosing names to use as values for the `@ref` and `@name` attributes, develop a consistent scheme that will make sense to other authors. Do not use spaces or capital letters. This name will never be visible to readers but should be easy for developers to understand. Placing careful comments in one “main” customization file can help other authors create new customizations that are accurate.

Note that this functionality is limited when it comes to irregular plurals and capitalization. Take care with the placement of custom elements: they will not, for example, capitalize automatically at the beginning of a sentence.

See [Subsection 44.7.3](#) for the mechanics of specifying a file of customizations via the publication file.

27.2 Versions

A **version** is formed by including or excluding content coming from your PreTeXt source. This could be optional content, or it could be content that varies between versions. See [Subsection 27.2.3](#) for specific examples.

27.2.1 Marking Your Source

The `@component` attribute is used by an author to identify elements in the PreTeXt source. The value of the attribute is any name that makes sense to a publisher. Examples might be `videos`, `labs`, `genome`, or `color`. There is a (huge) **un-component** which is the collection of all elements not contained in an element that is in some named component. You can think of it as a default component, or an un-named component—it is content that will be in *every* version, no matter what.

27.2.2 Forming a Version

Now, a publisher can elect to include or exclude all of the content of each component. This is accomplished with the `@include` attribute of the `source/include` element inside a publication file. The value of this attribute is a space-separated listing of some of the component names in use. An example minimal publication file could look like the following example. See [Subsection 44.7.2](#) for the specifics.

```
<publication>
  <source>
    <version include="videos labs"/>
  </source>
</publication>
```

This attribute is interpreted according to these rules:

- An element whose `@component` value *is* in the list in `@include` will be included in the source that will be converted.
- An element whose `@component` value *is not* in the list in `@include` will not be included in the source that will be converted.
- If there is no `@include` attribute in the publication file, the indication of the components are ignored and the entire source is processed. This would be the same as listing every component name in `@include`.
- Setting `@include` to an empty value (`include=""`) achieves the opposite effect and excludes every component from the source that will be converted.

You can nest one component inside another. But understand that once a PreTeXt element is excluded because the version in play does not include a certain component, then *all* of that element's contained source is gone and never coming back. In other words, an element nested inside an excluded element cannot be influenced by whatever other component it may be assigned to. However, the converse is possible: include an element via a component, and exclude contained elements via a different component.

27.2.3 Types of Versions

Here are three typical use cases for versions.

Additional Material. Some material may not be desired for every output format. Some interactive material might not make sense in a printed book. Or perhaps certain types of exercises are included at the end of each chapter, or not. By putting these elements in components, they may be included or excluded via a publication file. The idea here is that some versions contain a *subset* of all the available, authored material. For example, a version might include content in the videos and labs components, but exclude content in the genome component.

An “annotated” Instructor’s Version can be accomplished with additional material, perhaps in a selection of `<commentary>` elements looking like like

```
<commentary component="instructor">
```

Alternate Treatments. It may be possible to present a topic in two logically correct orders, but with substantial differences in how subtopics are treated. An example is the “early” or “late” treatment of transcendental functions in calculus. If the rearrangement is cosmetic, then an alternate main file can simply include divisions (chapters, sections) from separate files in a different order via the `xi:include` mechanism. See [Section 5.3](#) for details.

When the two pathways through the material have common and distinct material, then two components can be employed and the publication file would always include exactly one component.

Or for excluded material you might create some sort of placeholder text indicating what is missing. So all `<video>` elements might be excluded by placing them into the videos component. But you might want to indicate that there is a video available in some other format and include an indication of its title or topic. So you could write a short paragraph next to the `<video>` and place it in a novideos component. Now you would typically include exactly one of videos or novideos within each publication file in use. If your `<video>` live in numbered figures, you could exclude the `<figure>` and use a numbered block, such as a `<remark>` as the alternate and perhaps preserve numbering of later items.

Alternate Presentation. This is an example similar to one Sean Fitzpatrick uses. His `<docinfo>` configures the way color is used in his TikZ diagrams. But instead he has two configurations, one for full color (HTML, electronic PDF), and another for black-and-white (printed hardcopy). The former is in the `color` component, and the second is in the `nocolor` component. If his project has color photographs, he could make careful gray-scale versions with specialized tools, and then place the resulting pair of `<image>` into each of the two components separately.

Releasing Material over Time. Many authors are simultaneously publishers, and some are also instructors. And a few author-publisher-instructors like to “release” their material over time. This could be accomplished with versions. First there will be some base material like the front matter, back matter, and a preparatory Chapter 1. Now, mark Chapters 2 and 3 each with the component `week1`. Mark Chapter 4 with component `week2`. Mark Chapters 5, 6 and 7 with component `week3`. And so on.

Then the publisher file can be edited each week, or there can be multiple publisher files (one per week), which successively accumulate more components to include. For example,

```
<version include="week1"/>
<version include="week1 week2"/>
<version include="week1 week2 week3"/>
```

Note how the numbering of older material will not be affected by the addition of newer material, you will just want to be careful about forward cross-references from released material into un-released material. Of course, this example just uses chapters as the granular unit—you could use other divisions, or a mix.

27.2.4 Caveats for Creating Versions

There are some subtleties when you get fancy with manipulating your source this way, and we cannot protect you from every pitfall. You may get warnings for some of this. If you find new gotchas the hard way, please let us know.

Numbering. It is very possible that material that is common between versions ends up with different numbers. An exception is if you subset by excluding material *at the end of a division*. This may be very natural for optional material. Then elect a numbering option that resets at the next division. In this way later “higher” numbers go missing, and it does not affect the sequence of earlier “smaller” numbers.

Cross-References. Do not create a cross-reference into content you might exclude, there may not be a target for the `<xref>`. You should get a warning about this. When you have alternate versions, you will need to think carefully about `@xml:id` and `<xref>` for your two versions. It is possible that what you think is common material might really need to go into two components because an `<xref>` points at a target that actually resides in two different components.

Identifiers. Allied with cross-referencing, be careful not to create source that had duplicate identifiers that are meant to be unique. You may get warnings about this situation.

Generated and External Files. Some portions of your PreTeXt source get manipulated into additional files in particular formats (“generated”). Examples would include images given in $\text{\LaTeX}$ or Asymptote syntax, and trace files for Runestone CodeLens environments. Depending on what you include or exclude in different versions, these files could have different characteristics. As one example, suppose you define different color palettes for use in images described using the TikZ language (inside a PreTeXt `<latex-image>` element). And then you employ the two different palettes by using versions and components to control which palette is used for each version. If you want to save these images (say, by committing to a repository) rather than repeatedly regenerate them, then you need to save two different collections of generated images.

Versions are given by a publication file ([Chapter 44](#)), and the publication file allows you to specify which directory has these generated images ([Section 5.6](#)) so you can easily coordinate the generation and employment of these images with a coherent publication file.

You also sometimes bring “external” files to your project, such as JPEG images, which are not derived from your PreTeXt source. As in the example above, you could have color images for a version used to produce electronic formats and grayscale images for a version produced for physical printing. Again, the publication file could be used to employ a different collection/directory of external images for different versions via the options described in [Section 5.6](#).

Chapter 28

Further Customizations

The publication file is our primary vehicle for substantially affecting the way a project is produced, see [Section 26.1](#) for more. Varying small portions of text (on the order of a phrase) is accomplished with customizations to make different versions, see [Section 27.1](#). Producing different versions by including, or excluding, large portions of text, on the order of paragraphs, blocks, and divisions, is accomplished with support for versions, see [Section 27.2](#). Arranging material in different orders can be accomplished with thoughtful use of modularizing source files, see [Section 5.3](#).

In this chapter we describe two other ways to influence output, which are in some ways are techniques a PreTeXt developer might need to be familiar with ([Part V](#)), though a publisher might also find them useful. An author should not have any need for these techniques.

28.1 String Parameters

The majority of the conversions that PreTeXt supports are accomplished via an Extensible Stylesheet Language (xsl) stylesheet. This language has a **parameter** feature which allows an external value (as a small chunk of text) to be provided to the stylesheet externally. In this way, a single stylesheet can produce small changes in output in reaction to a parameter value, without the need for creating multiple stylesheets. We say that the stylesheet is **parameterized** by the parameter. These are often called **stringparam** since that is the command-line switch used by the `xsltproc` executable for the external communication ([Chapter 46](#)).

The conversion to $\text{\LaTeX}$ source, as a precursor of PDF output is a good example. A string parameter specifies the size of the font, which causes small changes to the $\text{\LaTeX}$ source file in the appropriate ways. Similarly, a PDF should be different if it is meant to become a hardcopy printed book, or if it is meant to be viewed on an electronic screen. This dichotomy is reflected by a simple string parameter (with values yes or no) and then a single stylesheet can produce two outputs that are different in substantial ways. Note that a publisher is insulated from any of this discussion for these two examples as the publisher file ([Section 26.1](#)) handles all the logistics. See the publisher file options at [Subsection 44.3.5](#) and [Subsection 44.3.2](#).

Some string parameters are just for internal use, especially when multiple stylesheets are chained together to accomplish a complicated conversion, such as to EPUB ([Chapter 31](#)) or braille. Others are used to allow developers to optionally test-drive some new features—these usually have debug in their name. We once used string parameters directly to accomplish publisher customizations. As of 2022-10-24 we are well along to moving these to the publication file, though we mention this here since this transition is not 100% complete.

The method for supplying an (external) string parameter to the processing of a PreTeXt project varies depending on the tool used for processing.

- PreTeXt-CLI, [Subsection 5.2.7](#)
- pretext/pretext Python script, [Section 47.8](#)
- xsltproc binary executable, [Section 46.4](#)

28.2 Extra Stylesheets

String parameters (Section 28.1) are an easy way to effect global changes in the presentation of your writing. But putting ten of them on every command-line gets old and cumbersome fast.

You may also wish to customize your output in some stylistic way. This might be especially true for $\text{\LaTeX}$ /PDF/print output. For example, you might wish to have every chapter heading of your book in a nice shade of light blue, with the title flush right to the margin, countered by a thick solid rule extending all the way right, to the edge of the paper. Notice that this does not affect your content, it is strictly presentation. This is our approach for styling $\text{\LaTeX}$ output, much as CSS is used to style HTML output (Chapter 41)

We have done several things to encourage such customizations. We have tried to put as much stylistic information as possible in the $\text{\LaTeX}$ preamble and keep as much as possible out of the body. (There is always room for improvement on this score, please be in touch if you have a need.)

You can start with a new small XSL file. You then tell the PreTeXt-CLI to use that XSL file instead of the standard one provided by PreTeXt through the `<xsl>` element in a `<target>` of the project manifest.

Assume that you have an XSL file called `custom-latex.xsl` located in the folder `xsl` inside the root of your project. In your `project.ptx` manifest file, use `<xsl>xsl/custom-latex.xsl</xsl>` as an element in the corresponding `<target>`.

The custom XSL file should import the stock PreTeXt file for the type of output you want to create. This is done using the line

```
<xsl:import href="./core/pretext-latex.xsl"/>
```

which should be placed near the top of the file; everything after it will redefine the various rules imported from the stock XSL. Note the `@href` attribute's value starts with `./core/`. This works because the CLI copies all the standard XSL to a subfolder `core` of the temporary directory holding your custom XSL so that you do not need to know the path to its location on your system.

See Section 47.9 to see how to use such a stylesheet with the `pretext/pretext` script.

The easiest thing to put in this file is elements like

```
<xsl:param name="latex.font.size" select="'120pt'" />
```

. Values given on the command-line supersede those given in an XSL file this way.

You can augment the $\text{\LaTeX}$ preamble with as much $\text{\LaTeX}$ code as you like in the following way.

```
<xsl:param name="latex.preamble.late">
  <xsl:text>% Proof environment with heading in small caps&#xa;</xsl:text>
  <xsl:text>\expandafter\let\expandafter\oldp\csname\string\proof\endcsname&#xa;</xsl:text>
  <xsl:text>\let\oldep\endproof&#xa;</xsl:text>
  <xsl:text>\renewenvironment{proof}[1][\proofname]{\oldp[\scshape #1]}{\oldep}&#xa;</xsl:text>
</xsl:param>
```

There are a variety of things you can do generally, by overriding the imported XSL templates to change behavior, but such modifications are beyond the scope of this guide.

Chapter 29

Conversion to Online HTML

This chapter describes way a publisher can adjust the presentation of PreTeXt content in an inline format, without actually changing that content. Similar to the case for $\text{\LaTeX}$ conversion ([Chapter 30](#)) there is a variety of options which may be configured. As of 2022-10-24 we are transitioning away from string parameters ([Section 28.1](#)) to using a publication file ([Section 26.1](#)), which is simultaneously being documented carefully. During the transition you can explore `xsl/pretext-html.xsl` by searching on the use of `<xsl:param>` at the outermost level (i.e. starting in column 1).

29.1 HTML Publisher Options

29.1.1 `index.html` Page

The conversion to HTML *always* creates a file named `index.html`. We do this because if a reader requests the URL

`platypus.mammal-institute.org/aota/html`

then most modern web servers will automatically return the page

`platypus.mammal-institute.org/aota/html/index.html`

So you can advertise the shorter version to potential readers. What is in `index.html`? Simple code to **redirect** to another one of your pages. Which one? Any one you like!

Within the `<publication>` element of your publication file ([Section 26.1](#)) include an `<html>` element, with a child element `<index-page>` having an attribute `@ref`. The value must be the `@xml:id` of a division which is rendered as an entire web page at the requested level of chunking. See [Subsection 44.4.8](#) for details on specifying this option.

For example, if a `<book>` is being chunked into `<chapter>`, and your source has

```
<chapter xml:id="birds">
```

then you can set `ref="birds"` and the page for that chapter will be the default page for the shorter URL. In practice, you probably really want a page that looks like the front matter or a Table of Contents.

The default is to first have `index.html` redirect to a page for the `<frontmatter>`, and if this is not possible, then it will redirect to a page for the top-level of your content. If your document is short or simple, you may just have a single web page. You could choose to not distribute the `index.html` file and then just use a concise and descriptive `@xml:id` for your top-level element (e.g. `<article>`) to fashion an attractive URL that points to your shorter work.

29.1.2 Banner Contents

The top of page banner normally features the title, subtitle, and author. If the subtitle or author list are long, or not particularly relevant for a given work, the publisher may wish to not render them at the top of each page. Of course, this information will still always be available at least once in places equivalent to a title page in the “front” of your document. See [Subsection 44.4.3](#) for details on specifying what to display.

29.1.3 Embedded Calculator

You may elect to have an embedded online calculator in each page of your online version. It will appear in the right margin, and will stay there as a reader scrolls the page up and down. A button near the masthead can be used to control visibility. By default no calculator is available, so you need to explicitly request this feature. As of 2020-05-30 there are four calculators available from the [GeoGebra Project](#). See [Subsection 44.4.4](#) for details on specifying this option.

29.1.4 HTML Favicon Configuration

A **favicon** is a graphical element (“icon”) that identifies a website. Perhaps its most recognizable use is its appearance in every browser tab open at that site. As publisher, you can associate an icon for your project to the HTML output of your project.

See [Subsection 44.4.5](#) for details on this specification. When the attribute value is none (the default), then your pages will not have a favicon.

If you set the attribute to `simple` then you *must* provide two versions of your icon, in PNG format, in pixel sizes 16×16 and 32×32 , with exactly the two filenames below (respectively), in the directory of provided external files ([Section 5.6](#)).

```
favicon/favicon-16x16.png
favicon/favicon-32x32.png
```

There are other ways to specify a favicon and some browsers expect different files. Try this scheme first, as it appears to have been sufficient since 2018. But if a new scheme needs implementation, we can consider a feature request.

29.1.5 ActiveCode Programming Windows

A window that allows for entering, and executing, computer programs can be made available for every page. The reader can click on a pencil icon to activate this window. Some languages can run in a web browser as part of any HTML output, while some other languages require infrastructure on a Runestone server to execute, and so are only available when you specify that hosting option. Note that the publisher will select a single language for use with the entire document. See [Subsection 44.4.7](#) for details on specifying this option.

29.1.6 Base URL

The base URL of a hosted version of HTML output may be specified as an entry in a publication file. Then certain aspects of other output (typically PDF/L^AT_EX) will link to corresponding aspects of the HTML output. See [Subsection 44.4.2](#) for the specifics of this entry. Examples of use include links from Asymptote graphics ([Section 30.8](#)), and URLs pointing to locally-hosted data files ([Section 4.17](#)).

29.1.7 Privacy Options for Video Embedding

When videos are embedded in HTML from sites like [YouTube](#) or [Vimeo](#), they come with whatever tracking cookies these sites want to include. Some of these can be helpful; for example, to let the viewer keep track of what they have watched. Others are designed to target advertising, and load when the page loads, rather than when the video plays, which can increase the time it takes for your book to load.

Currently YouTube offers an “enhanced privacy mode” that disables tracking cookies on page load. The assumption is that publishers will want to protect their readers’ privacy and optimize page load time, so this mode is turned on by default for YouTube videos. It is not known to be available for other platforms, but can be added if this changes. Note that the behavior and appearance of your videos may change slightly depending on which option you choose.

Within the `<publication>` element of your publication file include an `<html>` element, with a child element `<video>` having an attribute `@privacy`. The value must be either `yes` (use enhanced privacy, if available), or `no` (allow all tracking cookies). If your publication file does not have this element (or you do not have a publication file) you will get a warning message, and the default will be used. See [Subsection 44.4.15](#) for details on specifying this option.

29.1.8 Links to full-size versions of Asymptote diagrams

Asymptote diagrams are embedded in HTML as a canvas element within an `iframe`. Unlike image elements, web browsers will not provide the option to “view image”, and clicking on the image will not enlarge it. Since 3D Asymptote diagrams are interactive, a reader may wish that they could interact with a larger version of the graphic.

You may elect to place a “Link to full-sized image” link below each Asymptote diagram. Clicking on the link will open the Asymptote HTML file directly, and the diagram will be displayed at a much larger size.

See [Subsection 44.4.17](#) for the specifics of this entry. This feature is off by default.

29.1.9 HTML Feedback Button

You can elect to have a “Feedback” button in the footer of your HTML pages. You must provide a URL via an attribute. But where that URL points, and what happens there, *is your responsibility*. In other words, PreTeXt is no more help here, you just get a functional button pointing *somewhere* of your choosing or design. An example use case may be that you want to make it very easy for your readers to submit reports of small errors, like “typos.” So you setup an online form with a free service, which will help you manage this communication. Or maybe you want to run a reader survey via a form, so you point to that.

The default text on the button is Feedback, which will be in the language in effect for the page. You can override this choice (e.g., Bug Report), but it needs to be raw text (no markup) and then it will not react to language changes for your document. See [Subsection 44.4.18](#) for precise details.

29.1.10 HTML Navigation

There are publisher file entries to control how navigation between pages behaves. The “Up” button may be turned on and off. An entry for the logic of these buttons can be set to the values of `linear` (the default) or `tree`. The former causes the Previous and Next buttons to behave as if the divisions are arranged as in a printed book, i.e. linearly. The latter option means that when a reader comes to the last subdivision of a division, the Next button will not be active, and they will need to return (up the hierarchy/tree) to the division and move to the next division. Turning off the Up button while electing the tree model is likely to lead to a frustrating navigation experience.

The HTML target offers two views of the Table of Contents (ToC). The default is to fully expand the ToC to the depth indicated by the [common Table of Contents level 44.1.2](#) publisher setting. There is also a focused view that uses an expandable ToC and only initially fully expands the path to the current page.

See [Subsection 44.4.19](#) and [Subsection 44.4.20](#) for the exact syntax of these options.

29.2 Knowled Content

A **knowl** is a feature of the online conversion. It is text that you can click on, and nearby some content is revealed. You can click it again to hide that content. You can find a variety of examples in the Sample Article, or other PreTeXt projects.

Knowls come in two different types:

- A **born-hidden knowl** or **born-as knowl** is content that is contained in a knowl at the location where it first appears. Typical examples are footnotes, hints to exercises, or proofs of theorems. In other words, these small units of content are “hidden” behind knowls in the location where they naturally belong. Some content (e.g. footnotes and exercise solutions) are *always knowled*, while other content (e.g. an inline exercise) can be knowled, or not, as a choice made by the publisher via the Publisher’s File ([Section 26.1](#)). For exact details on influencing knowlization see the reference material in [Subsection 44.4.9](#).

An author does not need to be conscious of knowls. The structure and content of the document should not “be aware” that there will be knowls in the online output. But instead the publisher has control over whether or not certain medium-sized pieces of content (examples, exercises, figures, etc.) are born as knowls or not.

Note that some of these switches are for broad categories of items, for example, choosing to knowl theorems will also knowl `<lemma>s`, `<corollary>s`, `<fact>s`, and more. See [Subsection 44.4.9](#) for precise details. This choice applies document-wide, there is no plan to support electing this on a case-by-case basis.

- By contrast, a **cross-reference knowl** or an **xref knowl** is content that is a duplicate of some other content, in another location, as a result of an author making an `<xref>` cross-reference in their source. So a cross-reference in Chapter 8 to Example 4.6 will open the content of the example in the knowl, rather than transporting the reader backwards four chapters. If the reader *does* want to see the example in its original context in Chapter 4, then the knowl finishes with an **in-context** link that functions as a traditional hyperlink. Almost every cross-reference is a knowl, except when the target is a division—then a traditional hyperlink is used, since rendering an entire division as a knowl is unwieldy. Should knowls not be desired, there is a publisher level switch to render all `<xref>`s as traditional links. See [Subsection 44.4.21](#) for details.

The presence of the in-context link is one way to tell the difference between the two - **born-hidden knowls** will not have an in-context link, while **cross-reference knowls** will. Locators in the index ([Section 29.5](#)) are mostly cross-reference knowls. Knowls are used a few other places, such as in a list of notation.

29.3 Permalinks

If you place a mouse pointer in the left margin, to the left of a heading for an item, then an icon with two links of a chain will appear. This is a link to the item. Instead of a doing a right-click for a context menu that has an option like “Copy Link Location”, rather do a normal (left-) click. This will copy the heading (“Example 4.5 Chameleon Colors”), and also the link itself, to your clipboard, along with some formatting. You can then paste this into course materials, or email it to a reader who had a question.

You might want to make this feature known to your readers, perhaps via a preface explaining some of the more novel features of the HTML version of a PreTeXt document.

29.4 Lists

Lists with several columns are rendered in row-major order, as of 2018-02-28. In other words, the first list items (`<li>`) in your source will populate the first row.

29.5 Index Style

Start at [Section 3.23](#) to learn how to create an index. The realization of the index for online output is implemented within PreTeXt. We have made certain stylistic choices, in addition to taking advantage of certain features of the online format.

- A one-column format.
- Indented subheadings (not run-in).
- Maximum two levels of subheadings.
- Word-by-word sort order on headings.
- Locators for divisions are hyperlinks.
- Locators for smaller units are knowls.
- “Page ranges” use the initial location.
- No running heads (yet).
- A **headnote** can be accomplished with an `<introduction>`.

Some of these choices would be easy to adjust or extend as the result of a feature request.

Also, there can be significant differences between how PreTeXt implements the index for HTML and how the `imakeidx` package creates an index for $\text{\LaTeX}$. See [Section 30.16](#) and [\[3\]](#).

29.6 Styling

The PreTeXt conversion to HTML creates standard HTML elements, with styles controlled by CSS via class names (and not so much via the element names). As evidence of this, building HTML without the accompanying Javascript and CSS renders in a readable fashion, albeit quite plain (as one would expect).

This HTML is styled with CSS to create the output that a reader sees. There are multiple “themes” that a publisher can choose from to render the pages in different styles. Below are samples of available themes. See [Subsection 44.4.13](#) for the syntax for specifying a theme and options like the color palette to use.

Themes. There are multiple themes that define different appearances for HTML output. Currently available themes include:

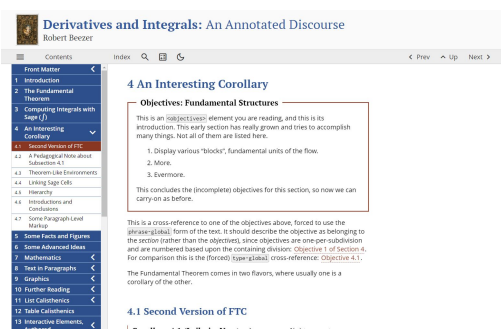


Figure 29.6.1 default-modern An updated version of the traditional PreTeXt theme.

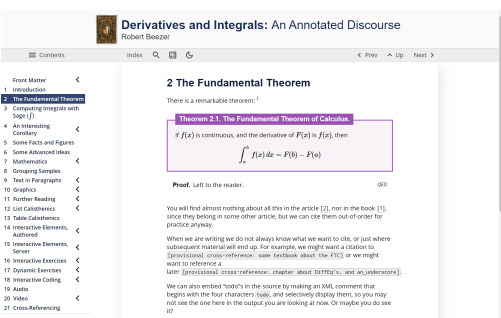


Figure 29.6.3 denver A theme that uses bolder structural elements and colors.

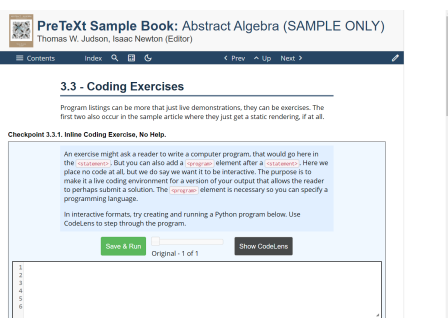


Figure 29.6.5 salem A theme that is optimized for displaying wide interactive elements (such as Active Code and Parsons).

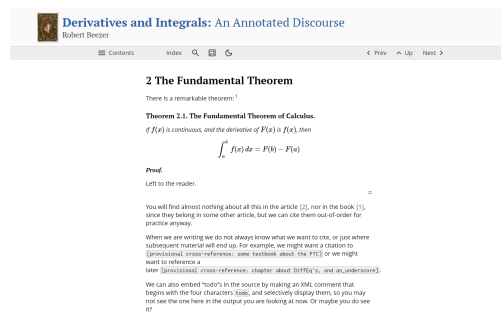


Figure 29.6.2 tacoma A theme with minimal decorations and colors. A minimalistic presentation of the contents.

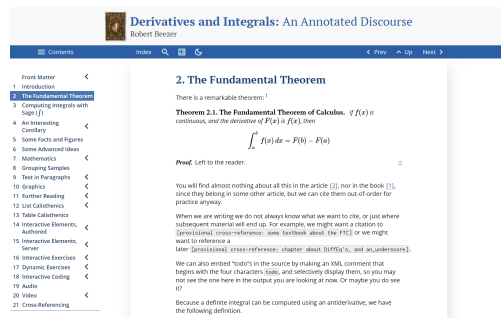


Figure 29.6.4 greeley A theme designed for short documents like academic papers.

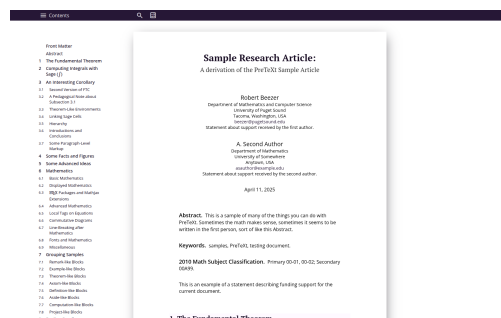


Figure 29.6.6 boulder A minimal theme designed to mimic a LaTeX pdf for academic paper (no masthead).

Development. The CSS for these themes is built from SCSS source. See [Chapter 49](#) for a developer’s tour of that

source, and please join us on the pretext-dev discussion group if you want to create alternate themes.

29.7 Analytics

PreTeXt makes it as easy as possible to have services provide data about visitors to the HTML version of your book. We support [Google Analytics](#) and [StatCounter](#).

In each case, use is similar. Begin at the service's site and follow their instructions for enabling data collection for your book's site. We will not document those instructions here. *Do not copy the identification numbers from another project—be sure to obtain your own for your project.* When completed, you can request (or will be shown) sample Javascript code to add to pages on your site. Except, PreTeXt is going to do that part for you. You only need to provide the unique identifiers used by the service that identify your project. Because these sign-ups are dependent on your site, this is a publisher activity, and hence configured with the publication file ([Section 26.1](#)), see [Subsection 44.4.1](#) for details.

StatCounter uses two identifiers, a project ID and a security code. *Do not copy the identification numbers from another project—be sure to obtain your own for your project.* You can find them in your “Project Config” under “Settings”. Or the code provided will contain lines such as

```
var sc_project=11538430;
var sc_invisible=1;
var sc_security="72e1390a";
```

In which case your publication file would have an element under html like

```
<analytics statcounter-project="11538430"
  statcounter-security="72e1390a"/>
```

Google Analytics comes in several flavors. We once supported Classic and Universal, but no longer, so we do not provide documentation here. For the replacement, Google **Global Site Tag** is a single ID. *Do not copy the identification numbers from another project—be sure to obtain your own for your project.* Google once called this a tracking ID, which may also be called a property ID; these looked like UA-6836004-1. Google has now switched to Google Analytics 4 and is exclusively using the form of a **Google Tag ID/Measurement ID** which will look similar to G-CQG9X37H61; you will need to upgrade your analytics if you used the older style tag. In any case your publication file would have an element under html like

```
<analytics google-gst="G-CQG9X37H61"/>
```

If you use both services, you can elect to use just one <analytics> element with several attributes.

We would be happy to provide support for additional major services. Just make a feature request.

29.8 Native Search

Client-side search is supported natively by a Javascript library. Search results are tuned to leverage the *structure* of your PreTeXt project. As such, development and support will favor this approach over our legacy support for an online service ([Section 29.9](#)).

There are two **tunings** of search.

Textbook	This is the default. Results are presented in order of appearance, on the assumption that a reader is following the text in a linear order and knows their current location. Visual clues will reflect the relevance of each result.
Reference	This assumes a document is more like a reference work and is accessed “randomly.” Results are not in order of appearance, but rather early results will likely be more relevant.

Employing a switch in the publication file will provide search boxes (top banner, lower-right corner) and all the supporting infrastructure. See [Subsection 44.4.14](#) for details.

29.9 Google Search

Note: as of 2022-10-27 we have search supported by an online service from Google and a client-side (“native”) version supported by Javascript ([Section 29.8](#)). Development and support will favor the latter.

Search facilities can be enabled through [Google Custom Search Engine](#). Please, please report any discrepancies in the following instructions as the setup interface at Google changes out from underneath us. These instructions have been updated as of 2020-10-01.

Besides being useful for search facilities, setting up a search engine might be a good way to alert Google of something newly available, and initiate your book’s rise up the search results rankings.

List 29.9.1 Configuring Google Custom Search

1. Create an account with Google (GMail, YouTube, etc.) and make sure you are signed in.
2. Visit [GCSE](#) and add a new search engine via New Search Engine.
3. Provide a URL for the top-level domain name/directory for your book/document. Everything below this will be indexed. We have taken some care to mark knowl content in a way compatible with the search facility, but there is more work to do here.
4. Give the engine a GCSE-specific name, so you can tell later which one it is when you have several.
5. Under Edit Search Engine in the Basics tab locate Search engine ID which has a string that uniquely identifies your new search engine. Save this, you’ll need to make it part of your PreTeXt document.
6. Under the Users tab add co-authors or trusted backup personnel.
7. Fiddle with Edit Search Engine > Look and Feel at your own risk! Only the defaults are tested and supported.
8. Provisions for removing advertisements for non-profit sites seem to have changed. If your university already contracts with Google, you should investigate having a “SuperAdmin” at your institution so this setup for you, and make you a trusted collaborator. Then it should be an easy matter to turn off advertisements.

List 29.9.2 Configuring PreTeXt for Google Search

1. Because these sign-ups are dependent on your site, this is a publisher activity, and hence configured with the publication file ([Section 26.1](#)), see [Subsection 44.4.14](#) for details.
2. The Search engine ID you saved from above is referenced in Google’s code as a cx number. An example looks like 482cf73dc05bed674 (older examples looked like 002673997130187229905:qjo2y0jplyu). In which case your publication file would have an element under html like


```
<search google-cx="482cf73dc05bed674"/>
```
3. The search/@google-cx attribute will alert the PreTeXt conversion and fully enable and implement search. You are done, and everything should just work. You should see a Google-branded search box to the top right of each of your pages. (We have no control over the branding.)
4. Time to rebuild your HTML output and make the improved version available.

29.10 Single Page HTML

Sometimes you would like a shorter work to be a *single* file in HTML format. Perhaps it is something self-contained, like a course syllabus, or you desire to attach it easily to an email or other post. This section contains advice and procedures you can use to achieve this.

The primary mechanism for limiting the number of files in an HTML build is to set the `@portable` attribute of `publication/html/platform` to "yes" (see [Subsection 44.4.12](#)). This will automatically take care of the following.

- Use hosted (CDN) versions of the CSS and javascript files rather than including them in your output directory.
- Embed generated SVG images in the HTML (anything from `<prefigure>` or `<latex-image>`).
- Embed the search code in the HTML (if you are using the)
- Set all `xref knows` to behave like links (same as setting `/publication/html/cross-references/@knowled` to `never`).
- Produce a single file with all your chapters/sections/subsections in it (the same as setting `/publication/common/chunking/@level` to `0`).
- Prevent the generation of most extra “standalone” pages (such as for most interactive elements and worksheets).
- Not produce the standard `index.html` file that is otherwise used to redirect to the main page.

Keep in mind that if your document has external assets, or uses generated assets that are not converted to SVG, these assets will still be needed in your output (regardless, you will get external and generated directories for your assets, but these might be empty or not needed). Additionally, some interactive elements work by creating an `iframe` that loads an HTML file, and these cannot be avoided currently.

Finally, depending on the purpose of your document, you might consider using a simpler theme. The boulder theme is designed to look like a $\text{\LaTeX}$ -produced PDF; it doesn't have any masthead, which makes sense for a single page document. Other style options are described in [Section 29.6](#).

For very small documents, you might also consider turning off the table of contents by setting `/publication/common/tableofcontents` to `0` (see [Subsection 44.1.2](#)).

29.11 HTML in an LMS

If you are using PreTeXt to author course materials, you may want to include the resulting HTML in your LMS (Learning Management System) course shell. While uploading PDFs can also be useful for students to print from, many universities require the documents to also be made available in accessible formats, so HTML is likely your best option.

Depending on which LMS you use, and possibly what settings have been applied to your course shell, you may be able to upload a portable HTML file, as described in [Section 29.10](#). This seems to work with Moodle, but not with Canvas, which blocks javascript inside uploaded files (the Javascript is required to render the MathJax math content). In this case, you have two options:

1. Host your HTML output on a web server (such as using `pretext deploy` to publish your files to GitHub pages) and then link to the content or embed it in your LMS using an `iframe`. To make embedding easier, you can turn on an “embed this page” button by setting `/publication/html/@embed-button` to `yes` in the publication file (see [Subsection 44.4.22](#)).

Once your page is published, click on the button and copy the code to your clipboard. In the LMS, create a page in your course shell. There is likely an “embed code” button in the editor interface of your LMS where you can paste what you copied from the PreTeXt page. When you save the page in your LMS, you should see just the *inner* content of the pretext page (no table of contents or navigation). However, the page should be fully functional, including interactive elements and MathJax rendered math (so it will be screen reader friendly).

2. If you need to keep your document private to your course (for example, maybe it is a set of exam solutions you don't want to publish on the open web), you can produce a SCORM¹ archive of your document. To do this, in your `project.ptx` manifest, set `@compression` to `scorm` for the `HTML` target you want to build.

PreTeXt will produce the HTML inside a `.zip` archive which will also include a SCORM manifest that tells your LMS how to display the content. You can then upload this SCORM file to your LMS and add it as a page. In Canvas, you can do this by adding SCORM to your course navigation in settings and then clicking the upload button on that page. Brightspace (D2L) lets you upload SCORM files from a page-creation menu. See the documentation for your LMS for additional help.

¹Sharable Content Object Reference Model

Chapter 30

Conversion to PDF and Print

This chapter describes ways a publisher can adjust the presentation of PreTeXt content for a print format, or an electronic facsimile (i.e. PDF).

30.1 Electronic versus Print

The stylesheet `pretext-latex.xsl` can produce two very similar outputs. Each is a file in $\text{\LaTeX}$ syntax, which can be converted to a PDF with a $\text{\LaTeX}$ executable (“engine”). However, there are two purposes for such a PDF. The first is a document which is meant to be read on a screen. We call this an **electronic PDF**. The second is meant to be printed as a physical book, so it would be the file you provide to a copy shop, campus bookstore, or print-on-demand service (see [Chapter 43](#)). We call this a **print PDF**. So as you read this, keep in mind that the $\text{\LaTeX}$ conversion is really two-converters-in-one.

To illustrate the difference, an electronic PDF will contain cross-references that are active, and colored to be obvious to the reader. For the print PDF the same cross-reference will be black, inactive, and by default contain the page number of the target. Another difference is the print PDF will include trailing parentheticals with the content of `@visual` when using `<url>`. This allows readers of the print version to visit a URL that has clickable text in other formats.

A wide `<figure>`, `<table>`, `<list>`, or `<listing>` will be presented horizontally by default, but including a `@landscape` attribute whose value is `yes` will rotate these blocks by 90° in *print PDF* output (only) to take advantage of the full page height. This is presently only supported by images given by a `@source` and with an extension provided in the filename.

The publication file entry `latex/@print` controls the selection of the two output modes of this single stylesheet. The default is `no` for the electronic version, and `yes` will change to the print version. See [Subsection 44.3.2](#) for the specifics of this entry.

In this chapter, we describe both of these two conversions together, since they are only slightly dissimilar.

30.2 Traditional Publishing

Suppose that you have engaged a publisher to help you manufacture a physical copy of your book or research article. Publishers vary greatly in their expectations for what you provide to them. (And maybe you are your own publisher, see [Chapter 43](#).)

If your publisher wants to run with **camera-ready copy** from you as a PDF, you should be in good shape. With the print option (versus electronic, see [Section 30.1](#)) and a bit of effort to control widows, orphans, and figure placement, you should be able to produce something very good by yourself.

In mathematics, computer science, physics, or economics, a publisher might want to work from your $\text{\LaTeX}$ files, which they will assume you have written from scratch. And they may expect to be able to swap in their house class or style file to achieve the look they want. A PreTeXt $\text{\LaTeX}$ file will look a bit unusual, see [Section 30.18](#). On 2022-11-22 we have a report of a very extensive [research article](#) written in PreTeXt and uploaded to the [arXiv](#) preprint server,

where it was successfully processed by pdf $\LaTeX$ on the server, with only minimal hand-edits to the intermediate $\LaTeX$ file. We can always improve in this area, so we welcome reports, good and bad, about the robustness of our $\LaTeX$ output when delivered to publishers.

30.3 Text Block Alignment

$\LaTeX$ is engineered around placing boxes on the page. Characters in boxes are built up to be words, then words in boxes become lines, and then lines in boxes become pages. Or something close to that. As a consequence $\LaTeX$ excels at right-justified text. For this reason, and also because we think it looks more like a commercial published book and therefore more professional, right-justified text is the default. If your opinions on this come from experience with some other word processor, keep an open mind. Note too, that paradoxically, sometimes a ragged-right alignment can lead to greater amounts of hyphenation.

Similarly, $\LaTeX$ can make the bottom of the text block *always* land at exactly the same place. This is especially pleasing if you are targeting two-sided printing, since you can have the two text blocks on either side of a single piece of paper match up *exactly*. But this comes at a cost—sometimes a page can have huge gaps between paragraphs just to place the bottom edge of the last paragraph at the bottom of the space meant for the text blocks. Further, $\LaTeX$ defaults to a flush bottom or ragged bottom, depending on if the document is a book or an article, or if it is one-sided or two-sided. Instead, we default to a ragged bottom for better out-of-the-box formatting, with an option to elect a flush bottom when you understand the risks and can watch out for them.

See details on specifying these options at [Subsection 44.3.6](#). See [Section 41.10](#) for greater control over some aspects of right-justified text and also read about justified text in [Section 4.41](#).

30.4 One-Sided or Two-Sided

An electronic PDF will default to page layout appropriate for a document printed single-sided, which makes the most sense for a document that may not ever be printed, or which possibly might be printed on a personal printer.

A print PDF will default to page layout appropriate for a document printed two-sided, which makes the most sense for a document that may be sent to a print-on-demand service or printed on a printer that will print on both sides of a sheet of paper.

These defaults may be overridden with the publisher entry `latex/@sides` switch with values one and two. See [Subsection 44.3.3](#) for the specifics of this entry.

One-sided layout will default to symmetric left/right margins, and page headers with the page numbers *always* placed in the upper-right corner as part of default page headers. There will be no blank pages between chapters of a book.

Two-sided layout will have asymmetric margins with the ratio of inner (adjacent to spine) to outer at 2:3. You need some extra space at the spine to compensate for the binding, but when a book is open, the two pages are separated by two inner margins, so these do not need to be as wide as the outer margin to give some distance between the pages. And readers will want more space to write in the outer margins, perhaps providing simple proofs of important results. (This ratio may be changed with the `hmarginratio` key of the `geometry` package.) Page headers will have page numbers on the outside of the page, with odd numbers on the right-side page. Chapters will possibly have a blank page between them, so they begin on a right/odd page. Behavior is similar in the front matter and back matter.

30.5 Font Size

The overall (document-wide) font size may be given as one of eight different values, ranging from 8 points to 20 points. The default is 10 points. These values influence multiple settings within $\LaTeX$ itself, and PreTeXt will also adjust some default values, such as the line-width (in order to preserve an optimal number of characters per line for reading ease). See [Subsection 44.3.5](#) for precise details.

30.6 Page Shape

The conversion to $\text{\LaTeX}$ defaults to US Letter paper with reasonable margins. However, the $\text{\LaTeX}$ [geometry package](#) has numerous options for paper sizes and margins. The publication file can be used to insert any configuration the package supports. See [Subsection 44.3.6](#) for the exact syntax.

Note that if you change the paper size, you may want to change the text width computed by PreTeXt, and the more changes you make, the more default settings you may need to adjust. However, if you are producing your book for print-on-demand ([Chapter 43](#)) getting the page size and margins just right is a key step of the process.

Note also that when you use the `<geometry>` element of the publication file, the *content* of the element (text between the opening and closing tags) will be duplicated *exactly* into a $\text{\LaTeX}$ source file. Being careless here is a good explanation for the $\text{\LaTeX}$ compilation step to fail.

Here is an example:

```
<latex>
  <page>
    <geometry>a4paper , total={16cm,25cm}</geometry>
  </page>
</latex>
```

twich will produce in the $\text{\LaTeX}$ output

```
\geometry{a4paper , total={16cm,25cm}}
```

An electronic PDF may be printed on physical paper, but perhaps you want to make a version that works well on a portable device that naturally supports a portrait orientation, such as an Android tablet, an iPad, a Kindle (device or application), smart phone, Sony Digital Paper, or a ReMarkable tablet. Aspect ratios vary across these devices, but once you settle on a target ratio, we have had good luck with the following algorithm and parameters:

1. Specify 10 point text
2. Text width of about 4.5 inches
3. Add quarter-inch left/right margins to compute text width
4. Use aspect ratio to compute an overall height (about 6.5 inches)
5. Subtract quarter-inch top and bottom margins to obtain text height

Then you can provide the `geometry` package the overall size as the `papersize` and the text width and text height as the `total` size of the body, resulting in equal (tight) margins all around, and good use of limited screen real estate. These parameters create a PDF that is very legible on a larger smart phone, and for fine detail, rotating the device to landscape works well. Really.

Suppose you are a publisher, or an author who is also your publisher, and you wish to manufacture a paperback copy of your book in a 6-inch wide by 9-inch tall version. It is unlikely your printer has paper of this size. Use the `geometry` option above to specify the logical size of the paper for your book and then set margins, etc. to determine exactly how the text block sits on the page. Then there is a `@crop-marks` option available though the publication file. You set it to a paper size describing the paper in your printer, likely `letter` (US) or `a4` (Europe). Then your logical page will be centered on the physical page, and indications in each corner will describe the boundaries of the page.

To see this in action, you need to explicitly set a smaller page size. So the value of the `geometry` could be set to

```
paperwidth=6in,paperheight=9in,total={5in,7in}
```

Do not forget to turn off any page-scaling in your printer configuration before printing onto paper. See [Subsection 44.3.6](#) for the exact syntax of how to enable crop marks.

Crop marks may be requested by a publisher, when it is understood that you are providing camera-ready copy. Or you may find it useful for visualizing how your text and margins will look at a smaller page size. The `crop` package has other useful options, such as framing the entire logical page. You can hand-edit these into your $\text{\LaTeX}$ file, or make a feature request.

Best Practice 30.6.1 Avoid Too Much Text on a Printed Page. With freedom comes responsibility. Resist the temptation to pack in as much text on the page as you can. PreTeXt varies the width of the text in reaction to the font size and is already very close to the maximum number of characters per line for comfortable reading by humans. Similarly, very narrow margins can be uncomfortable for reading (or holding a printed book).

30.7 Page Breaks

A publisher can insert page breaks beyond those that L^AT_EX creates automatically. This might relieve some awkwardness that L^AT_EX's otherwise excellent page breaking failed to relieve. The publisher should identify each source element for which they would like a page break to happen *before* that element appears in output. For each such element, there must be an @xml:id. Bring these @xml:id values together in the publication file, using whitespace to separate them. For example:

```
<latex>
  <insertions pagebreaks="xmlid1 xmlid2 ...">
</latex>
```

You may use multiple <insertions> elements to help organize page breaks by chapter, section, etc. For example:

```
<latex>
  <!-- chapter 1 page breaks -->
  <insertions pagebreaks="xmlid1 xmlid2">
  <!-- chapter 2 page breaks -->
  <insertions pagebreaks="xmlid3 xmlid4">
</latex>
```

At time of writing, the allowable elements which may be preceded by a page break using this feature are divisions, numbered blocks, and the following unnumbered elements: <answer>, <aside>, <audio>, <biographical>, <assemblage>, <blockquote>, <console> (when not a child of a <sidebyside>), <hint>, <historical>, <image> (when not a child of a <sidebyside>), <interactive>, <list-of>, <p>, <paragraphs>, <poem>, <pre>, <program> (when not a child of a <sidebyside>), <sage>, <sbsgroup>, <sidebyside> (when not a child of a <sidebyside>), <solution>, <tabular> (when not a child of a <sidebyside>), and <video>.

Note that over time, the precise size of many things in your PDF output may change. This may be because of an underlying change in how PreTeXt builds PDFs. It may be because of change in the L^AT_EX distribution. And of course, it may be because the source content and publisher specifications (like font) change. So all manually specified page breaks may need to be monitored over the life of a project.

See [Subsection 44.3.14](#).

30.8 Asymptote Links

An image described by an <asymptote> element will produce standalone HTML files as part of the conversion to HTML output. For 3D images, these are interactive (rotate, scale). If you set a base URL for your project ([Subsection 29.1.6](#)) and request this feature through a publisher switch, then each image in an electronic PDF ([Section 30.1](#)) will be a link to the HTML version. See [Subsection 44.3.11](#) for the specifics of this entry. This feature is off by default.

30.9 Cross-References

In an electronic PDF, cross-references will be hyperlinks that take the reader to the target. Color is used to make them visible, in addition to whatever indication a PDF viewer will provide on hover. The publication file entry controlling the use of numbers in cross-references ([Subsection 44.3.7](#)) is by default no in this case, but may be set to yes to include the page number of the target in the cross-reference.

In a print PDF, cross-references will not be active hyperlinks and will be the same color as the adjoining text. The publication file entry controlling the use of numbers in cross-references ([Subsection 44.3.7](#)) is by default yes in this case, but may be set to no to not include the page number of the target in the cross-reference.

See [Subsection 44.3.7](#) for the syntax of specifying the use of page numbers in cross-references.

For accessibility, an underline can be added beneath all hyperlinks (cross-references and URLs) to supplement color as a visual indicator. See [Subsection 44.3.8](#).

30.10 Page Number Fidelity

If you produce a “print” PDF, perhaps as a precursor to producing a physical book, the page numbers may be important to you and your readers. Thus, you may desire that an “electronic” PDF have faithful page numbers. And together we might consider this a canonical numbering of the pages of your project. (Long-term we hope to migrate these page numbers into other formats, such as braille. See [GitHub #1020](#).) Fundamentally, a PDF destined to become a physical book will be built with “two-sided” pages, while an electronic PDF will have “one-sided” pages. The two-sided version will often leave a page blank at the end of a division (e.g. parts or chapters), so a subsequent division begins (“opens”) on an odd-numbered page, also known as a **recto** page. We have publisher options for print versus electronic ([Subsection 44.3.2](#)), which subsequently provide (different) defaults for “sidedness” ([Subsection 44.3.3](#)).

Suppose you wish to have your electronic (one-sided) PDF have identical pages (i.e. identical content) with identical page numbers as your print (two-sided) PDF?

First decision point is: what do your cross-references look like? If every cross-reference looks like [Section 5.2](#), then presumably a reader of print (or embossed braille) can locate the target. But if you have used a cross-reference that looks like Fermat's Theorem then you have exhibited a preference for electronic formats at the expense of readers of physical formats. To remedy this, we allow for the automatic insertion of page numbers as part of every cross-reference (see [Subsection 44.3.7](#)). But all these page numbers will increase the length of your physical version and pages will never match up. So that your cross-references are workable in all output formats, or you should elect to have page numbers in every cross-reference, for each PDF produced.

Now a further complication is that a two-sided version will always start some larger divisions (parts, chapters, appendices) on an odd-numbered page, the **recto** page (versus the **verso** page). This sometimes requires an additional blank page on the verso side. But a one-sided version has no need for this practice, so the two-sided version will eventually have different (larger) page numbers on the identical content. The solution is to use a publisher option, which is only effective when a one-sided version is produced, that creates these blank verso pages or skips the page numbering ahead by one when there would have been a blank page. The result is that major divisions will always open on an odd-numbered page, matching the two-sided version. See [Subsection 44.3.4](#) for details. (Note that this is not the default and so must be a conscious choice.)

30.11 Nested Lists

TeX can fail if lists are nested too deeply. Maximums may be up to four nested ordered lists, and up to six overall (mixing in unordered lists). If you hit these limits, ask yourself if your situation is really that complicated, or ask us to consider a feature request adding a technical fix.

30.12 Multi-Column List Order

Lists with several columns are rendered in column-major order, as of 2018-02-28. In other words, the first list items (`<li>`) in your source will populate the first column.

30.13 Footnotes

Footnotes are the farthest thing imaginable from structured authoring. You can put them anywhere, and conceivably they can contain anything. But they do work great in HTML presentations as *knowls*. However, we do not let the tail wag the dog, and so have to make some *compromises* for footnotes output in print formats.

- As of 2019-01-04 we are fairly restrictive about content. No paragraphs, more like a few sentences, max. If you have more content, consider the `<aside>` element, and send feature requests for that.

- Many blocks, such as `<example>` and `<remark>`, are implemented with the L^AT_EX `tcolorbox` package, to make styling and layout much more capable and reliable. But footnotes get trapped within these boxes and render at the end of the box (not the bottom of the page) and have a different scheme for the marks (letters, not numbers). We have mitigated much of this behavior, but the cost is that these footnotes are delayed until the box finishes. So if you have a 4-page example, and use a footnote early, it may appear at the bottom of a page that is 3 or 4 pages away.
- Another consequence of the above is that hyperlinks in electronic PDFs, from the mark to the text, need to be disabled (they will never work within a `tcolorbox`). Cross-references to a footmark's text will still be active in an electronic PDF.

30.14 Cover Images

The publication file may be used to specify the filenames of a front-cover image and a back-cover image. The image must be a single-page PDF, and it will be scaled to fit an entire page. So it is your responsibility to supply an image which has the correct aspect ratio and sufficient resolution. These are only supported for the L^AT_EX conversion. See [Subsection 44.3.10](#) for exact syntax. Implementation is via the L^AT_EX `pdfpages` package.

This is meant to help you create a professional electronic PDF. A print-on-demand service ([Chapter 43](#)) will likely want a standalone image (possibly with the front and back, plus a spine, all rolled into one `wrap` image). So build your *real* cover images first ([Chapter 42](#)), and then modify them for this use.

30.15 Icons

The `<icon>` element is built around the **Font Awesome 5** font collection. The L^AT_EX `fontawesome5` package contains all the necessary font files (for use with both `pdflatex` and `xelatex`), and a style file that interfaces, in a semi-elaborate way, with the font files. If you have installed this L^AT_EX package, and the fonts still seem unavailable, figure out how to adjust your L^AT_EX setup or your system configuration. Simply installing a font file, such as the more recent **Font Awesome 5** is not going to work. See [Chapter 40](#) for more about fonts in L^AT_EX.

30.16 Index Style

Start at [Section 3.23](#) to learn how to create an index. The realization of the index is under the influence of the L^AT_EX `imakeidx` package, which results in certain stylistic choices, such as a two-column format. It also means that any options supported by this package could be easily implemented as the result of a feature request.

Also, there can be significant differences between how PreTeXt implements the index for HTML and how the `imakeidx` package creates an index for L^AT_EX. See [Section 29.5](#) and [\[3\]](#).

30.17 Styling

Note that some of these switches are about *style*. There are many more ways to influence the style of the L^AT_EX output, see [Chapter 41](#).

30.18 L^AT_EX File

The L^AT_EX file created by PreTeXt will contain the majority of your content in a form that you could use it in a new standalone L^AT_EX document, in accordance with Principle [1.1.1:9](#). However some constructions which are not natural in L^AT_EX, such as a `<sidebyside>`, may be cumbersome to reuse. We continue to improve and refine these situations, though.

Our philosophy is to create and use many new L^AT_EX environments, allowing styling and fine-tuning to occur in the preamble. This makes the body look more like simple L^AT_EX and allows for much greater flexibility in styling, along with greater reliability for successful L^AT_EX compilation.

The existence, variety, and quality of $\LaTeX$ packages changes continuously. We can, and will, swap out some packages for replacements, as needed or desirable. This is to your advantage, as you are absolved of the need to evaluate competing packages, and to insure that they do not clash with each other. So resist the temptation to modify the $\LaTeX$ output significantly prior to compilation, as it will inevitably lead to frustration. The $\LaTeX$ file is a means to an end—it allows us to create a PDF with excellent typography, and especially for the demands of technical disciplines, such as STEM and music. It is meant to be ephemeral, not archival.

If some other variety of $\LaTeX$ (or $\TeX$) file is desired, a new conversion could be created. Many of the more complicated aspects of any conversion are purposely isolated in the `pretext-common.xml` file so that they can be easily re-purposed and there is consistency across output formats.

Best Practice 30.18.1 Only Edit $\LaTeX$ Files Rarely. We want to stress that the $\LaTeX$ file created by various conversions is meant to be an *intermediate* format. In other words, it is *ephemeral*. We try to make it clean and organized, but it is not the $\LaTeX$ a human would write. You might be able to recycle a paragraph or two in other documents you create without PreTeXt. But it is not meant to be stable or archival, and no long-term use is supported in any way. In other words, it is not a supported output format, beyond compiling to a PDF without errors.

Because of that, you may find it necessary to manually adjust a file to control widows or orphans, or maybe the placement of a graphics file, or similar adjustments. Some of the most common adjustments used by PreTeXt authors are inserting `\newpage`, `\noindent`, or `\par` to avoid awkward transitions, or to remove an occasional `\leavevmode`; consult a good $\LaTeX$ reference before making such changes, as effects later in the output may be unpredictable.

We view this as the final step before making a new edition, which might be a PDF that you submit to a print-on-demand service ([Chapter 43](#)). So hand-editing might be an annual exercise, at the most frequent. One way to keep track of larger number of edits over long periods of time is to write a script (in an appropriate language), which looks for unique strings before or after trouble spots and replaces the nearby content. Another minimally invasive option is to keep a separate git branch of the $\LaTeX$ file which makes the desired changes, which can then be applied on those rare occasions it is necessary (ideally, with little rebasing needed).

30.19 Snapshot Record

We are careful about which $\LaTeX$ packages we use, trying to stick with well-established packages with active maintainers. But it can be useful to have a record of *exactly* which packages are in play, and better still, which versions of those packages are being used. So you can use a publication entry to request loading the [snapshot](#) package, which will then automatically generate such a record into a `<job>.dep` file. See [Subsection 44.3.13](#) for details on electing this feature.

If you suspect some packages are not playing well with each other, this record might be helpful for debugging this (rare) situation. It can also be useful if you wish to have a perfect archive of how some publication was produced. See the package documentation for more on the format and use.

Chapter 31

Conversion to EPUB/Kindle

EPUB is the standard format for electronic books. Books in EPUB format can be read using applications on a variety of platforms. (Apple Books across the entire Apple ecosystem. [Calibre](#) is open source and cross-platform for desktop usage. Android devices have options as well.) Amazon’s Kindle devices use a proprietary format that is derived from EPUB. Through much experimentation, the PreTeXt team has determined that SVG mathematics, generated by MathJax offline tools, works well in all ebook readers *other* than Kindle apps and devices. For Kindle, MathML is the best format. (Apple’s MathML implementation is very poor as of July 2021, and so we cannot recommend math formatted using MathML for situations where readers may use the Apple Books app.) Thus, we provide *two* pathways for production of electronic books. These formats are useful as an offline version, which is superior to a PDF in some ways, such as font face and font size being controlled by the software in an e-reader device. However, it is inferior to the online version ([Chapter 29](#)), since many interactive features cannot function within the EPUB version.

31.1 Prerequisites

There are a handful of prerequisites to build an EPUB version of a book.

- You must use either the PreTeXt-CLI or the `pretext/pretext` script, since creating an EPUB file is a multi-stage process; building with `xsltproc` is not supported.
- You must have node and npm must be installed. See [Appendix F](#) for more on node and npm.
- You must use a publication file (referred to below as `publication.ptx`) with

```
source/directories/@generated
```

and

```
source/directories/@external
```

so that images can be located and bundled (these are included by default if you use `pretext new` or `pretext init` for your project).

To use a non-generic cover image, the publication file must also have `epub/cover/@front` attribute that points to the cover image (JPEG or PNG, 2048 pixels tall, 1280 pixels wide). Absent an image provided, there will be an attempt to create a simple, generic cover image. See [Subsection 44.6.1](#) for details about specifying a cover image.

If you use the `pretext/pretext` script, you will need to generate any source-defined images in SVG format for standard EPUB, and PNG format for Kindle. Furthermore, Kindle processing requires PNG resolution to be at least 200 DPI, and 300 DPI is a good recommendation (from Mitch Keller). See [Kindle Help topic G202169030](#) for more.

Finally, if you are using the `pretext/pretext` script, you must install a local version of MathJax (the CLI will try to do this for you). We provide a bash script in `scripts/mjsre` that automates this process. See [Section G.1](#) for

instructions. As updates to the EPUB conversion are released, you may occasionally want to update your local copy of MathJax. Simply use the script referenced above.

To use the CLI, you will need to create a new target in the project manifest (project.ptx). The target should look something like the following.

```
<target name="ebook">
  <format>epub</format>
  <source>source/main.ptx</source>
  <publication>publication/publication.ptx</publication>
  <output-dir>output/epub</output-dir>
</target>
```

The @name can be whatever you wish, but the <format> must be epub or kindle. You won't be able to tell the output files apart, so if you want both an epub and a kindle, name the output directory differently for each.

31.2 Converting and validating

First we will describe how to convert to epub or kindle using the CLI. Assuming you have added a target with name "ebook" and format "epub", simply run:

```
$ pretext build ebook -g
```

If instead, you wish to use the pretext/pretext script, make sure to first generate images into the correct format (SVG for regular EPUB and PNG for kindle). Converting to EPUB with SVG math (used everywhere other than Kindle), run as a single command-line

```
/path/to/pretext/pretext/pretext -c doc -f epub-svg
-p publication.xml -d /path/to/output
/path/to/yourmainfile.ptx
```

For an EPUB file destined for Kindle, use the single command-line

```
/path/to/pretext/pretext/pretext -c doc -f epub-kindle
-p publication.xml -d /path/to/output
/path/to/yourmainfile.ptx
```

If you would like to name your output file something other than the name inferred from the root xml:id, you can use -o path/to/output/filename.epub instead of the -d option.

For the standard EPUB conversion, any standard EPUB reader should work. MacOS users will find the Apple Book app is likely their default. Calibre is useful for an alternative view and works on Windows and Linux as well. For the Kindle conversion, you will need to get [Amazon's Kindle Previewer app](#) (macOS and Windows only). For reasons we do not understand, some books crash the Kindle Previewer app, in which case you should try loading the EPUB file into the Amazon KDP web interface, which also offers a preview. (This preview is the only option for Linux users.)

Many EPUB marketplaces are strict about requiring that your EPUB file pass validation by the open source epubcheck validation tool. You may be able to install a command-line version of epubcheck using your operating system's package management tool. A free Java-based GUI version [released by Pagina](#) might be useful. There is also a (slow, limited) online version. More pointers can be found at the [W3 EPUBCheck](#) site. Please report validation errors and oddly-formatted electronic book output to the pretext-support Google group. Some aspects of PreTeXt have not yet been fully implemented for EPUB, but we will endeavor to support them as demand arises.

31.3 Distribution

Because the EPUB file built for Kindle is different than what we build for all other readers, you *must* distribute the Kindle version of your book through Amazon's KDP. Because Amazon and Kindle have the overwhelming majority of the electronic book market in the United States, if your readers are mainly in the United States, we highly recommend that you get your Kindle version in good shape.

Distribution of the EPUB file produced by the epub-svg conversion can be done through electronic book aggregators. There are many options available, and they all take an additional commission on top of what the electronic book marketplace through which a reader buys your book takes, but using such a service can save you from needing to post your electronic book on a variety of marketplaces. ***Do not allow an aggregator to distribute your book to Amazon, as the math will not render properly.*** It is as simple as *not* checking the box for Amazon when signing up with an aggregator to opt out of having the aggregator send your book to Amazon.

Chapter 32

Conversion to Runestone

[Runestone Academy](#) is a textbook hosting system originally designed for HTML textbooks about computer science. It is similar in design and goals to PreTeXt, but is also complementary in important ways. In particular, Runestone excels at providing very interactive versions where student work in the textbook can be graded (automatically or manually), scores recorded, and that information can be viewed by an instructor. Any instructor can easily spin-up an instance of your book for their course, hosted at Runestone Academy at no charge to the school or to the students. Could not be easier.

32.1 About Runestone Academy

Runestone textbooks are designed around interactive activities and active reading. We are very far along in the process of exposing these capabilities via PreTeXt markup. Publicly-accessible Runestone servers are available at [Runestone Academy](#).

- Every feature of the usual PreTeXt HTML output should function identically on a Runestone server.
- We have designed PreTeXt markup for the various types of problems available in Runestone: True/False, multiple choice, Parsons problems, matching, clickable area, (basic) fill-in, and ActiveCode programming exercises. Hosted on Runestone, readers have their responses graded and get instant feedback, plus their results are saved at the site. For other HTML output, hosted on your own server, these problems are automatically graded and the reader gets instant feedback, but of course, responses and scores are not saved anywhere. Please see [Section 4.12](#) for details of these exercises and their markup.
- Any `<exercise>` authored in PreTeXt can be electively rendered as a Runestone short answer question, when the Runestone platform is targeted as the host. A student can use $\LaTeX$ notation in formulating their answer, and Runestone will provide the instructor an efficient interface for reviewing student responses, provide feedback, and assign scores.
- Every feature described for an `<exercise>` is also available for PROJECT-LIKE, and for individual terminal `<task>` within these structures.
- As much as possible, non-interactive versions of these problems will render in less-capable formats, like PDF, EPUB, and braille.
- A `<program>` element with the attribute `@interactive` set to `activecode` (even outside of a an exercise) will be realized as a Runestone ActiveCode interactive program, where programs can be edited, compiled, and run. In some cases a CodeLens interactive trace utility is also available. The `@language` must be set. Supported values for the language when hosted at Runestone are: `python`, `python3`, `c`, `cpp` (C++), `javascript`, `java`, `kotlin`, `octave` (Matlab), `sql`, and `html`. When hosted on your own server, `python`, `javascript`, `sql`, and `html`, are supported with in-browser routines. So you do not need to configure *anything* server-side for this capability. See subsections of [Subsection 4.16.3](#) for details.

- Similarly, a `<program>` element with the attribute `@interactive` set to `codeLens` (even outside of an exercise) will be realized as a Runestone CodeLens interactive program. This allows a reader to step through the program, much like in a debugger, but with more informative displays of the intermediate state of the program (and nothing like breakpoints or changing variable's values). This ability varies by language, and by hosting location. See subsections of [Subsection 4.16.2](#) for details.
- All of the interactive exercises on a Runestone server can be worked by a student at the location in the book where they were authored, or a student may use the interface provided by the server (the **assignment page**) to locate exercises assigned by an instructor, at a location disjoint from the text itself.
- Additional activities are available when your book is hosted on a Runestone server, such as **peer instruction**. For example, you may choose an exercise, typically multiple-choice, for all the students to answer in class. Then the server can pair up students *with different answers* to discuss their responses before any more is known about the solution. This can be done through a chat window for a class where moving around is difficult.
- Students and instructors are provided extensive reports on progress.
- Instructors may download a spreadsheet of scores at any time.
- If you preview some books in Runestone's library, you will notice the use of ethical advertisements. This, along with donations, is how Runestone can offer free hosting to authors and courses. Once a student logs in as a member of a course, *advertisements are no longer shown*. So do not let an aversion to online advertising dissuade you from making courses possible on Runestone.
- Interactive exercises may not function fully inside of knowls. This bug is being tracked at [GitHub #1983](#).

We will not attempt to duplicate Runestone's documentation here. Visit the [Runestone Academy](#) site to learn more about all the features designed for hosting your textbook for everybody's courses.

32.2 Publishing to Runestone Academy

The usual PreTeXt HTML output ([Chapter 29](#)) only needs minor modifications to run profitably on a Runestone server. You accomplish this via a publication file ([Section 26.1](#)). The absolute simplest publication file to accomplish this is

```
<publication>
  <html>
    <platform host="runestone"/>
  </html>
</publication>
```

(See [Subsection 44.4.11](#).) Then perform the usual steps for a conversion to online HTML, as described in [Chapter 29](#) but also be sure to specify the correct publication file. The output should appear like a usual PreTeXt document, but will now include a new menu on each page. This has options which allow a reader or instructor to interact with the Runestone server, once hosted there. So it will not look entirely right when you view it locally, since you are not a Runestone server, but you should see subtle differences.

While this HTML may be manually deployed to any Runestone server, authors who wish to publish their work on the [Runestone Academy](#) server have a few more requirements.

- Hosting on Runestone assumes certain details conform to how a project is organized for use by the CLI ([Section 5.2](#)). So if you are managing your project with the CLI already, you are in good shape. If you did not create your project using `pretext new`, run `pretext init` to get started upgrading. You will also need to make your project available to the public via GitHub (instructions below).
- Copy the publication file you usually use for your `html-format` build target and name it as `publication/runestone.ptx` (or something similar that makes sense in your project). Edit this file so your `<html>` element matches the example at the start of this section.

- You should create a new `<target name="runestone"></target>` in your `project.ptx`. The contents of this tag will differ from your usual `html-format` build target in the following ways. 1) You should change your `<publication>` to point to the new file you just created, perhaps named `publication/runestone.ptx`. 2) You should change your `<output-dir>` to point to `published/document-id`, where `<document-id>` is defined in your `<docinfo>`. Note Runestone's preferred practice for the `<document-id>` is to have a simple lowercase string with no dashes or other special characters. This value is visible to readers who want to register for your course on Runestone outside of any formal setting.
- Update the `<docinfo>` section of your book so that it includes a `<blurb>` with a `@shelf`. The blurb should not contain any additional markup, just a simple string, that describes your book. Think of something like what you would read on a book jacket. It might even also be used automatically in the future for exactly that: a blurb on the back cover of a hardback book. So keep it simple—straight ASCII text, nothing fancy. The `@shelf` tells the Runestone software where your book belongs in the categories on the Runestone library page. Look at the Runestone library page to see what values are in use and copy an existing one *exactly* including capitalization. If you think a new shelf is necessary in the library, please seek advice on what to use.
- Use `pretext deploy` on the command line, or simply `git push` the changes described above to GitHub if you are comfortable with `git`. The `deploy` command will walk you through setup if you have not deployed your project to GitHub before. You do *not* have to enable GitHub Pages unless you want to. (GitHub Pages does not have the features of a Runestone server, and will not render a build for the Runestone target properly. You can host your regular `html-format` build on GitHub.)
- With Runestone Academy and the author interface you can see a draft of your book on

`author.runestone.academy`

When you are ready you can make a published version of your book available with the click of a button. We can set up access to the author interface when you open an issue (in the next step) requesting that your book be added to Runestone Academy.

- Log into GitHub and open an issue at github.com/RunestoneInteractive/rs/issues/new to request that your project be added to Runestone Academy. Be sure to provide the URL of your GitHub repository (e.g. `https://github.com/UserName/RepositoryName`). Runestone Academy administrators will communicate with you via GitHub to complete this process. When your book is first added to Runestone, it will clone your repository and build from your default branch. The default branch is configured on GitHub and is usually `main` or `master` but can be anything. However if you change your default branch after your book is added to Runestone you will need to get in touch with the Runestone Academy administrators to get them to re-clone your repository.

32.3 The Author Interface

32.3.1 Author Interface Basics

The author interface is a web-based interface that allows you to manage your book on Runestone Academy. It is designed to be easy to use and provides a number of features to help you manage your book.

When moving your book to Runestone Academy we will clone your repository to our server. We do this one time, after that we use `git pull` to get the latest changes from your repository. This means that you can make changes to your book on your local machine and push them to GitHub, then use the author interface to pull those changes into Runestone Academy and rebuild your book. Ideally your `document-id` and the name of your repository are the same.

Warning 32.3.1 Configuration Tip. If your `document-id` and repository name do not match, then you will need to update the “Path to existing repo” in the author interface metadata before you build your book. This should not be a GitHub URL, it is just the name of your GitHub repository (see [Figure 32.3.4](#)).

If your book has already been cloned on Runestone, then you cannot clone it again. If you think you have a use case where this is absolutely necessary please reach out to someone on the team to talk about it.

The author interface is available at author.runestone.academy/author. You can log in with your Runestone account. Once you have logged in you will see a list of your books. You can click on the book title to to edit metadata about your book. The author interface is also where you can build a new version of your book, see some analytics about your book, and publish your book to the Runestone Academy servers. You can even get an anonymized data set from a large sample of the classes using your book.

Books by test

Note: Click on the document-id to edit the meta data for your book

document-id/ basecourse	Last Build						
PTXSB	Build	View	Deploy	Show last log	Student Impact	Datashop Dump	2025-03-27 00:16:47.504459
WWSC	Build	View	Deploy	Show last log	Student Impact	Datashop Dump	None
ac-single	Build	View	Deploy	Show last log	Student Impact	Datashop Dump	2024-11-05 15:11:46.151808
dmoi-4	Build	View	Deploy	Show last log	Student Impact	Datashop Dump	2025-02-04 20:42:58.956721
fcla	Build	View	Deploy	Show last log	Student Impact	Datashop Dump	2025-04-08 16:45:56.377288
fcla_proteus	Build	View	Deploy	Show last log	Student Impact	Datashop Dump	None
thinkspy	Build	View	Deploy	Show last log	Student Impact	Datashop Dump	2024-12-06 00:18:37.750225

☐ Check the box if you want to generate assets for a PreTeXt book

Add a New Book

Your document id / basecourse name

Github Repo URL (Must use [https](#) url not ssh)

OR Path to existing repo ([/books/existingRepo](#))

[Add Book to Runestone](#)

Figure 32.3.2 The author interface main page

The main page of the author interface gives you access to all of the different functions. Clicking on the build button pulls the latest source from GitHub and builds your book. If you have a large book this can take a few minutes. When the build is complete you deploy your book to the runestone servers. You will see the build status change at the bottom of the page. If there is a failure for any reason you can usually see the cause by clicking the button to view the latest log.

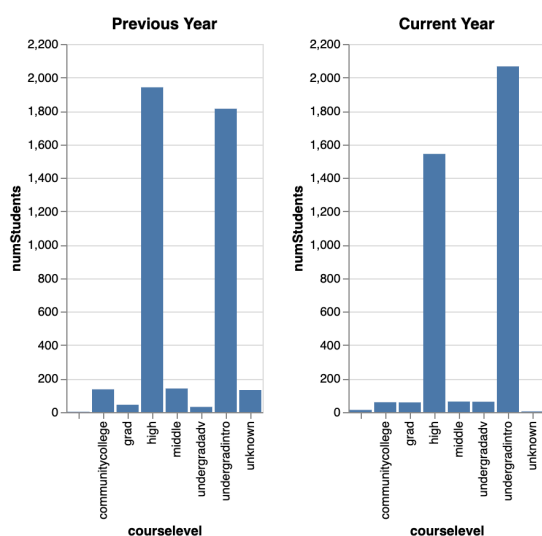
Another feature of the author interface is to provide you with some analytics. This page shows you the number

of students that have enrolled in a course using your book year to date. It also shows you usage patterns for the book by the week of the course. You can click on any of the shaded cells to drill down and see how students are viewing the sections of each chapter.

Runestone / PreTeXt Impact Report for Foundations of Python Programming

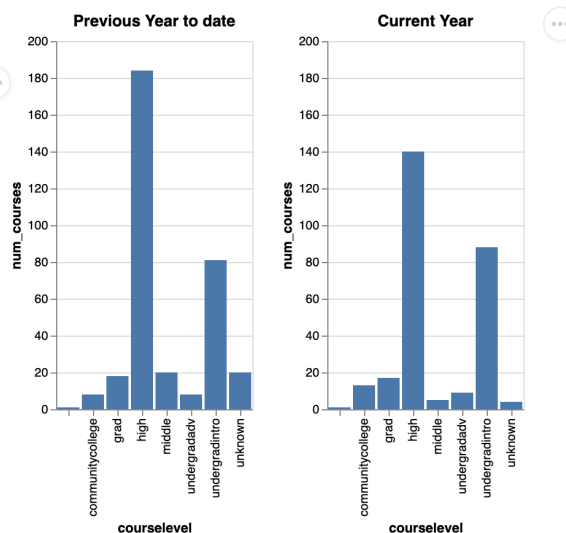
Registered Students to Date

This counts students from the start of the term (August 1) to today's date in both years.



Courses Created to Date

This counts courses from the start of the term (August 1) to today's date in both years.



Weekly Usage Patterns

Note click on any row in the heatmap to view the details for that chapter. It may take a few seconds. These graphs are current as of about 5 AM UTC each day.

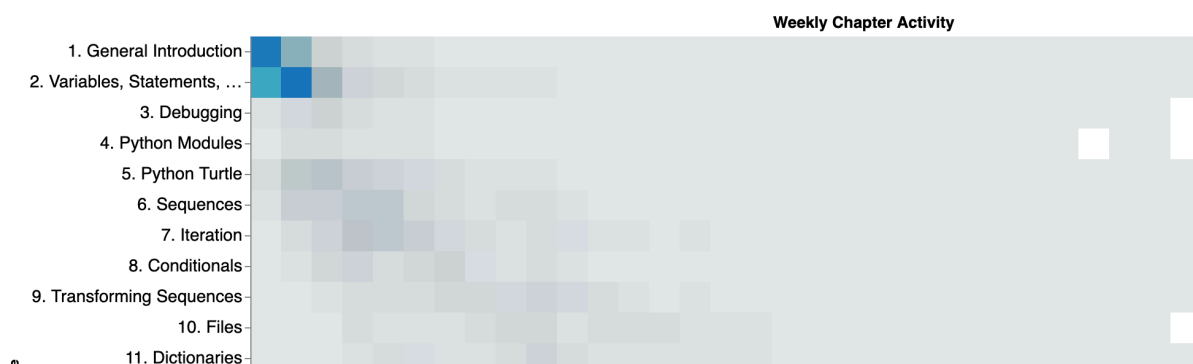


Figure 32.3.3 Some example analytics for your book

The Analytics are in their early stages so we encourage you to contact us with ideas for what kinds of data you would find useful.

If you click on the book's identifier you will be able to edit metadata about the book. This is reflected in our

library page so you can change the shelf section or update the **blurb** that describes your book to an audience. The Available for courses and Visible to Everyone in the Library checkboxes allow you to control the visibility of your book. If you are in the early stages of development you probably do not want others to see the book, but you can always view it via a direct link. When you are ready to make your book available to the world you can check the Visible to Everyone box and your book will appear in the Runestone Library. When you are ready to let others create a course around your book you can check the Available for Courses box. This will allow others to create a course using your book as the textbook. You can always change these settings later.

Edit book metadata for PTXSB

Title:

SubTitle:

Description:

Authors:

Shelf Section:

Base Course or Document ID:

Social URL:

Available for courses: ☐

Visible to Everyone in Library: ☒

Build System (PTX or Runestone):

Path to project.ptx file (relative to the root of repo):

PreTeXt project target (runestone):

Default Programming Language (for scratch activecode):

Github URL:

Path to an existing repo:

Main page:

Figure 32.3.4 Editing a book's metadata

32.3.2 Advanced Author Interface

The author interface also provides a number of advanced features to address the following scenarios:

- Your repository and your `<document-id>` do not match.
- Your repository is structured so that your `project.ptx` file is not in the root of your repository.
- You have a single repository with more than one book.
- You want to produce multiple books from a single repository using the PreTeXt versions feature.

If your repository name and your document-id do not match you will need to fill out the the “path to an existing repository” field so that it matches what you checked out from GitHub.

If your repository is structured so that your `project.ptx` file is not in the root of your repository you will need to configure the path to the `project.ptx` file. This is done by filling in the field for the path to the `project.ptx` file. This should be a relative path from the root of your repository to the `project.ptx` file.

If you have a single repository with more than one book you will need to do the following steps.

1. Create a new book for book number 1 on the main author page using the GitHub URL to your repository.
2. Make sure you have a target in your `project.ptx` file for this book and it is configured to use the publication file you created for Runestone.
3. Edit the metadata for your book in case any of the other advanced configuration situations apply.
4. To create book number 2 on the main author page use the “path to an existing repository” field. This should be `/books/<your-repo-name>`. Leave the GitHub URL field blank.
5. Depending on how your repository is structured you may need to make sure you have a second target for the second book with a different output folder configured and a different document-id. Alternatively you may have an entirely separate `project.ptx` file for the second book in a different folder of your repository. In this case you need to fill in the field for the path to the `project.ptx` file.

If you have a single repository and want to build two books using different versions then the process is the same as above, but you will need to have a second publisher file with the appropriate version information so that you will get a different output. This includes using the `@component` on the `<document-id>` element. This will allow you to have two different books in the same repository with different document-ids.

Chapter 33

Conversion to Braille

There is a conversion to braille which works very well, while also needing further improvements. Conceptually, it is not very different than the EPUB conversion ([Chapter 31](#)), except for the additional necessity of the `liblouis` library for the translation of literary text (see [Appendix H](#)). MathJax and Speech Rule Engine provide Nemeth Braille from your authored $\text{\LaTeX}$ mathematics. There is the option of embossable braille (designed for a physical paper page) or electronic braille meant for an electronic device like a one-line reader with mechanical raised pins. It is worth the effort to first make sure an EPUB conversion is successful, and then extend to the braille conversion.

Some extra care needs to go into the authoring of a PreTeXt project that creates the best possible output as braille. Some aspects are obvious, such as not being overly-reliant on video or interactive demonstrations. We can accomodate some constructions like a `<sidebyside>` in a way that makes sense to the reader. An allied project hopes to make it easy for authors to create diagrams that work well for both sighted and blind readers (tactile graphics). But constructions like abusing an `<m>` element to get a superscript “th” will just lead to confusion for a braille reader. With experience, we are collecting suggestions for authoring in [Section 4.37](#).

Please be in-touch in the PreTeXt discussion groups if you have a PreTeXt project you would like to convert. Of course, we want to improve the process and the result, but we are especially interested in the experience of blind and low-vision readers who can point us to areas that need improvement.

Chapter 34

Conversion to Slides

As discussed in [Section 3.32](#) and [Section 4.42](#), support is available for authoring slideshows in PreTeXt. Currently, slideshows must be authored explicitly, but we hope to eventually support annotating books and articles to export appropriate content as slides.

34.1 Reveal.js

To create a reveal.js slideshow using pretext, start by creating a new project as follows.

```
$ pretext new slideshow
```

You will notice a few differences from a standard project. In the project manifest, the format is “custom”, and there is an `<xsl>` tag that points to a custom xsl stylesheet in the xsl folder of the project. This will import the correct reveal.js stylesheet. After authoring your slides, you can build them with the following command.

```
$ pretext build web
```

To use xsltproc, run this to produce a Reveal.js slideshow:

```
xsltproc --xinclude -o path/to/output/slides.html
--stringparam publisher path/to/source/publication.xml
path/to/pretext/xsl/pretext-revealjs.xsl path/to/source/slides.xml
```

Reveal.js supports themes that affect the overall appearance of a slideshow. So in PreTeXt you can specify the name of theme in a publication file. See [Subsection 44.5.1](#) for details.

You might assume that your slideshow will be presented with the internet available, perhaps in a classroom, and will be updated for use the next semester. Or maybe you are presenting at a conference where you do not want to trust an unfamiliar internet connection, and you will later host an archival version of your presentation on your website and you want it to “just work” ten years from now. So you can configure your slideshow to obtain resources from a Content Delivery Network (CDN) online, or you may arrange to copy the necessary files from reveal.js and store them locally (your hard disk, your website). See [Subsection 44.5.4](#) for details on specifying these options, and see below for directions on how to organize the Reveal.js support files.

Reveal.js will render $\text{\LaTeX}$ syntax with MathJax, entirely similar to how MathJax is used in the PreTeXt conversion to HTML. This is accomplished with the Reveal.js math plugin, which is loaded automatically as part of your output. When you specify that resources come from a CDN, then this plugin will also get MathJax from a CDN. When you specify that resources are available locally (your hard disk, your website), then the plugin itself will be obtained locally, but MathJax will still be obtained from a CDN. So a local version may only be practical if you are careful not to include *any* mathematics in your document. (Reveal.js *can* be configured to use a locally installed copy of MathJax, but we have not provided any support for this scenario.)

We are not enthusiastic about PDF as an electronic format. But it might be a good choice as an archival format. So exporting your slideshow to a PDF could be a good choice for a long-term archive. On 2020-08-01 Andrew Rechnitzer suggests the [decktape](#) node (Javascript) program. The reveal plugin works well once you settle on a resolution (the

-s option). The generic plugin, along with the default key action (ArrowRight) can capture the behavior of slides built using the @pause attribute. Note that the grid option (see below) may not always work well for printing all slides, while default creates slides that decktape steps through properly. A local web server can also be employed to serve up the slides, see [Section 5.11](#).

Navigation Mode. Reveal.js has various options of the visibility of arrows a presenter can click on in order to move through a presentation. The visibility of these can be controlled via options in the publication file. See [Subsection 44.5.2](#).

Reveal.js imagines slides laid out on a 2-D grid. Each PreTeXt section gives rise to a title slide and these are organized left-to-right. *Below* each of these slides, arranged vertically, are the slides comprising the section. We use the attribute value grid to refer to this arrangement.

If public speaking makes you nervous and going left-to-right and top-to-bottom nearly simultaneously means you get lost and even less confident (we've seen it), we have an option for you. The attribute value linear arranges *all* your slides from left-to-right. Aah, that's better. See [Subsection 44.5.3](#) for details on setting the navigation mode for your slideshow.

Local Resources. To set up a Reveal.js slideshow to run locally, you need to have certain files available locally. We describe here the exact mechanics of doing this.

Suppose you have done the PreTeXt conversion, and have created a single slides.html file, which you have placed in a directory named talk. Now download or clone the git repository for Reveal.js (github.com/hakimel/reveal.js). This has a dist directory with four files, such as reveal.css, and also directory of themes, named theme. Copy these files and the directory to talk. Another directory in the repository is named plugin. Copy this directory to talk as well.

This process will duplicate more files than you need. Suppose your talk is produced to use the solarized theme ([Subsection 44.5.1](#)), and includes some math. Then as an example of how the copying should go, and as an example of the bare minimum necessary, your talk directory should be organized as follows.

```
talk
  slides.html
  reset.css
  reveal.css
  reveal.js
  theme
    solarized.css
  plugin
    math
      math.js
      plugin.js
```

34.2 Beamer $\LaTeX$

Run this to produce a Beamer $\LaTeX$ slideshow:

```
xsltproc --xinclude -o path/to/output/slides.tex
  path/to/pretext/xsl/pretext-beamer.xsl path/to/source/slides.xml
```

Of course, you should then run e.g. pdf_latex slides.tex to produce a PDF.

Chapter 35

(*) Conversion to Jupyter Notebooks

TODO

Chapter 36

Instructor's Version

Once your content is in place, you can begin thinking about various useful derivative versions. A natural example for a textbook is an “Instructor’s Version”, enhanced with additional material to help an instructor understand your organization and intent, or to provide advice and counsel about teaching the material.

36.1 Solutions

Philosophies about the purpose and use of exercises varies among authors and instructors. Some think hints, answers, and/or solutions, should be universally available to students to use responsibly. Others like to assign exercises to be graded as part of a course grade. Some are resigned to solutions that are distributed in a limited fashion eventually becoming public, or that some groups of students will distribute their own solutions, possibly not uniformly. Wherever you place yourself in this debate, distributing solutions to only instructors is one approach, and some instructors may find this a very helpful aid when they teach material new to them.

There is flexibility in which of `<hint>`, `<answer>`, and `<solution>` can be included or excluded in your text, which can be included or excluded in an Instructor’s Version, and which can be included or excluded in a Solution Manual (see [Section 37.1](#)). You can choose to author these or not, and then decide which to include in the student version, and which to include in an independently-produced Instructor Version, and which to include in a Solution Manual.

Read [Section 37.2](#) (and [Section 37.3](#)) for some practical advice about limiting distribution of solutions.

36.2 Notes and Commentary

The `<commentary>` element is designed primarily for the purpose of adding material to a document to make an enhanced version (see [Section 27.2](#)). It is similar in many ways to a `<paragraphs>` in that it can be placed within any division and must be titled.

Other distinctions are:

- Since it is often elective, you need to be careful about cross-references to and from a `<commentary>`. It is highly likely that you will want to make cross-references *within* a `<commentary>` *pointing to* other portions of your text, and this is always a good idea. You will want to avoid making cross-references *to* a `<commentary>` from other parts of the text, with the exception of a cross-reference that originates *within* some `<commentary>`.
- Numbered items are prohibited within a `<commentary>`, such as a `<figure>` or a `<theorem>`. Doing so would disrupt consecutive numbering in different versions, with or without, `<commentary>` included. Numbered equations are not prohibited in the schema, but should definitely be avoided anyway.

36.3 Adding or Removing Divisions

For an Instructor's Version you might wish to add additional material into the front matter (a specialized `<preface>` perhaps), or remove some material from the back matter (an `<appendix>` with solutions that duplicates solutions now placed within the exercises themselves). There may also be parts of each chapter you do not find necessary to include.

Modularizing your source files would allow for a different top-level XML source file to include different portions of the `<frontmatter>` or `<backmatter>`, perhaps just making a different title page. See [Section 5.3](#) for more on modularization.

Additional, minimal, XSLT stylesheets can be used to selectively “kill” portions of your source, such as every “Additional Reading” at the end of each `<chapter>` residing in a `<references>`. Consistent use of elements, leading strings in `@xml:id`, and/or leading strings in `<title>`, can make it a single-line exercise to selectively remove multiple portions of your source without removing other portions. See [Section 28.2](#) for more about additional XSLT stylesheets.

Think carefully about the effect of removals and additions on numbering. In HTML output all numbering is hard-coded and will be based on counts of the entire XML source file. So selectively killing content will not change numbering, but cross-references may point to divisions for which there is no content to serve as the target. Using a different top-level file can impact numbering throughout. Significant portions of the $\text{\LaTeX}$ output rely on $\text{\LaTeX}$'s automatic numbering via mechanisms like `\label{}` and `\ref{}`. So if portions of the text are killed, then the `\label{}` of a cross-reference may never be defined. A technical solution would be to provide an option to hard-code all numbering in $\text{\LaTeX}$ output.

Generally, removing portions of each division will have the least ill-effects on numbering if the portions removed are at the end of a division and no cross-references point there. So, for example, a `<references>` at the end of each `<chapter>` can be safely killed with no ill-effects if there are no cross-references elsewhere to the particular `<biblio>` contained in that `<references>`.

When a division is killed through the use of additional XSLT, knows and index entries will still be generated as usual for that division as part of the conversion to HTML. Thus, some care may need to be taken if certain knows should not be uploaded to a server. Using a consistent scheme for the values of the `@xml:id` might make this easy to script. The `<idx>` elements could be killed in a manner similar to the division with a use of the ancestor axis in a filter. Of course, the conversion to $\text{\LaTeX}$ will not create knows, and the index-creation process does not suffer from the shortcomings of the creation process for HTML.

36.4 Instructor's Notes

The `<commentary>` element, as described above, can be used effectively by an individual instructor to customize a personal version of a book. This is not as fine-grained as highlights or annotations, so is not meant as a replacement for tools that support more localized personal additions.

The ideal way to do this would be with a text having source distributed as a git repository, and with notes managed by git. Here is a rough outline, assuming a solid understanding of git.

1. Clone the author's repository to a local, personal, location.
2. Make a long-lived notes branch off the author's master branch.
3. Add `<commentary>` with commits on the notes branch.
4. Regularly pull master from the original repository to receive updates and fixes from the author.
5. Regularly merge master into notes so the enhanced version gets the author's changes without changing master.
6. Produce personalized output from the notes branch via PreTeXt as normal, with the switch enabling display of the `<commentary>`.

Chapter 37

Ancillaries

Similar to an Instructor's Version, for a textbook, or other work, a publisher might wish to provide ancillary documents with additional, or repackaged, material.

37.1 Solution Manual

An author may include a `<hint>`, `<answer>`, and/or `<solution>` as part of each `<exercise>` or `<project>`. Some of these may be designed for the reader, while some may be designed for the instructor. A separate conversion is available to make a PDF containing just these items.

To use the CLI, first create a small xsl file, called `solution-manual.xsl`, in the `xsl` folder at the root of your project, containing the following lines.

```
<?xml version="1.0" encoding="UTF-8" ?>
<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0">
  <xsl:import href="./core/pretext-solution-manual-latex.xsl"/>
</xsl:stylesheet>
```

Then add the following target in your project manifest.

```
<target name="solutions">
  <format>custom</format>
  <source>source/main.ptx</source>
  <publication>publication/publication.ptx</publication>
  <xsl>xsl/solution-manual.xsl</xsl>
  <output-dir>output/solutions</output-dir>
</target>
```

Then you can build the solution manual with the PreTeXt-CLI using `pretext build solutions`.

Or using `xsltproc`, execute

```
xsltproc -xinclude pretext-solution-manual-latex.xsl fauna.xml
```

with suitable paths in front of the stylesheet and the source file. The result will be a file like `fauna.tex`, which can be processed with a $\text{\LaTeX}$ engine such as `pdflatex`. By default, the result will include *all* `<statement>`, `<hint>`, `<answer>`, and `<solution>` for *every* `<exercise>` or `<project>` anywhere in your `<book>`. Note that this conversion is explicitly designed only for `<book>`, so send a request for support for an `<article>`. Division headings (`<chapter>`, `<section>`, ...) will be present, if and only if they have content. Page headers will help locate chapters and sections. Exercise numbers will be complete, to make it easier to locate individual problems. In other words, Exercise 3 from Exercises 5.6 will be labeled as 5.6.3, not just 3, as in the original text.

Any cross-reference that exists inside a solution will be honored and displayed faithfully. By that, we mean that if the author includes an `<xref>` as part of a solution to, say, Theorem 10.6, then that cross-reference will be

rendered visibly as “Theorem 10.6” in the solution manual. However, it will not be live (clickable) since the target (the `<theorem>` itself) is not part of the solution manual. We have not, and do not expect to, determine if a cross-reference points to part of another exercise which is visible in the remainder of the solution manual and then elect to make it live/clickable. In other words, all cross-references are static, even if there is the possibility to be more dynamic for a select few. If this is a severe shortcoming, consider producing an Instructor’s Version ([Chapter 36](#)), enhanced with additional solutions, where all cross-references are live, and targets are more likely to be available.

A set of switches allows a publisher to control including `<statement>`, `<hint>`, `<answer>`, or `<solution>`—for inline exercises, divisional exercises, worksheet exercises, reading questions, projects, and tasks within exercises and projects. So there is a total of $4 \times 5 = 20$ yes/no switches, for $2^{20} = 1\,048\,576$ supported scenarios. Start at [Section 26.4](#) for more on these settings. The $\text{\LaTeX}$ preamble is the same as for the full document, so besides being excessive, it should support any of the $\text{\LaTeX}$ styling options.

In practice, you will discover that the conversion will reproduce all of your `<frontmatter>` and `<backmatter>` *exactly* as if it was part of the entire text itself. The reason for this is that you may actually want a little bit of front matter, perhaps some back matter, and maybe a new title page that makes it clear that you have created a solution manual. Best practice would be to have your project already organized so that each of your chapters is in its own file, and incorporated into your document via a top-level file using the `<xi:include>` mechanism. (Read about modularity in [Section 5.3](#) if this is new to you.) So now make an *additional* top-level file for the solution manual, maybe with a new title, a new preface, and acknowledgements of any help creating exercises and solutions. Be sure to include *all* of the main matter, even if you know some parts may not have any `<exercise>`. You want numbering to be correct for your cross-references and this means having all the content available to be counted, even if it is not visible in the end product. This new top-level file is really the only overhead involved in getting a quality, reliable solution manual together.

The philosophy behind this conversion is that a publisher may wish to create a *different* range of solutions for instructors, for limited distribution in ways that students are unlikely to find. Thus, we have tried to produce a *functional* document by default, without too much attention to making a *beautiful* document. Of course, improvements and suggestions are always welcome, though here the priority will be ease-of-use.

Digression. This conversion illustrates some advantages of including all the content of your project in one source document, and then selecting a subset of that content for different audiences. The advantage of authoring `<hint>`, `<answer>`, and `<solution>` in close proximity to the `<statement>` should mean a higher probability that changes to one part of an exercise will be reflected in the other parts. And with standard processing tools, and provided switches, an author and publisher can easily decide which parts to show, and when.

By including all of your project’s content in one monolithic source document, it is possible to confidently reference supporting parts of the main text via cross-references from solutions. When a new edition is released, any variations in numbering will automatically be reflected in a new solution manual, created with no additional editing or proofreading.

37.2 Private Solutions File

Suppose an author distributes a textbook with an open license, and so makes the PreTeXt source available publicly (perhaps as a condition of the license). Perhaps the author also intends `<hint>` provided with `<exercise>` to assist students, and having them available as knows in HTML output is a great way to make them easily available, but not immediately visible. But the author has also written some, or many, `<solution>` for the `<exercise>`, but these are only meant for instructors, and not for students. See the discussion at [Section 36.1](#) for more background.

One approach is to distribute an Instructor Version or Solution Manual only on request, and only as a PDF. The ability to provide a watermark on every page (see [Section 26.5](#)) allows you to include a personalized message such as

Issued to Charles Darwin. Do Not Copy.

It would be a trivial technical exercise to remove this, but perhaps the moral imperative (in an extra `<preface>` as well?) would dissuade most from distributing further?

But with an open-source project, how can you distribute the exercise without distributing the `<solution>`? After multiple unsatisfactory experiments, we have arrived at the following *solution*. You author a *separate* PreTeXt

document with `<hint>`, `<answer>`, and `<solution>` that you wish to keep **private**. You might put this in a different, private, git repository that you only share with your co-authors. Here is how you can construct and employ this file. Here we use the word **solution** generically to mean any of `<hint>`, `<answer>`, or `<solution>`.

List 37.2.1 Private Solutions File

1. Start with an `<exercise>` that you wish to provide private solutions for. Give it an `@xml:id`, which you will need for [Item 4](#). Be sure the `<exercise>` is structured with a `<statement>` so a private solution may be appended.
2. We recommend leaving a comment inside the `<exercise>` to remind you that there is a private solution that may need editing if the problem `<statement>` changes.
3. Create a new PreTeXt file to hold the private solutions, presumably located *away* from your other source files (see [Warning 37.2.2](#)). Call it something like `privatesolutions.ptx`, which is what we will use below.

The first two lines of the file *must* be:

```
<?xml version="1.0" encoding="UTF-8" ?>
<pi:privatesolutions
  xmlns:pi="http://pretextbook.org/2020/pretext/internal">
```

(`pi` is the PreTeXt internal namespace.)

4. Now add as many `<hint>`, `<answer>`, or `<solution>` as you like, in any order you like, authored in the usual way. The missing piece is that you must link each solution back to the `<exercise>` it belongs to, with a `@ref` attribute whose value is the `@xml:id` you used to label the `<exercise>` in [Item 1](#).

For example, if there is an `<exercise>` with `@xml:id` having value `very-hard-problem`, you might author:

```
<answer ref="very-hard-problem">
  <p>42.</p>
</answer>
```

5. Finish the file by closing the overall `<pi:privatesolutions>` element.
6. Now you can have all of these private solutions incorporated into your source by specifying the filename in your publication file ([Section 26.1](#)). The `@private-solutions` attribute of the `/publication/source` element should be set to this filename ([Subsection 44.7.3](#)). So you would have

```
<publication>
  <source private-solutions="path/to/privatesolutions.ptx"/>
  ...
```

7. The private solutions file should be available to any conversion, but most likely you will be using it with the solution manual conversion (see [Section 37.1](#)).

For a large project you may have *many* private solutions and one big file is unwieldy. The solution is to modularize the file as described in [Section 5.3](#). Then you can organize your “file” of solutions into multiple files, perhaps organized by `<chapter>` or `<section>`. The catch is that the include facility requires each separate file to only have a *single* top-level (root) element. For this purpose use the `<pi:privatesolutionsdivision>` element, which is designed for only this purpose. You can nest this element arbitrarily deep.

Note that an `<exercise>` or `<task>` may be authored with no solutions and then does not need a `<statement>`. Now, if you add private solutions to such an exercise or task, the markup will be incorrect. As of 2020-06-24 we make no extra effort to warn, or fix, this situation. A likely consequence will be that these private solutions are not rendered (but you might see a problem number where they should appear). Similar advice applies to `<task>`, and especially to non-terminal `<task>` which can never hold solutions.

You will have noticed that you have a lot of freedom to make a completely disorganized private solutions file, or even *many* disorganized files. There is no structure to prevent this—you are on your own. Ourselves, we would keep the solutions in order of appearance, modularize a big project logically, and use comments liberally. But you can always search on the common value of `@xml:id` and `@ref` to locate the pieces.

Warning 37.2.2 Keep Your Private Solutions Private! If you are using git (and why wouldn't you?) it could be an easy mistake to include your private solutions in a public repository accidentally with a careless commit. We would place our private solutions in a directory close to our source tree, but not within the material tracked by git. And we would build our solutions manual with private solutions by copying the necessary files to some scratch directory, where they get deleted later, after the PDF result has been preserved for use.

37.3 Private Solutions with Git

(2020-06-06) We no longer advocate this approach, and find [Section 37.2](#) easier and more robust. This section is no longer maintained and may be removed.

Suppose an author distributes a textbook with an open license, and so makes the PreTeXt source available publicly (perhaps as a condition of the license). Perhaps the author also intends `<hint>` provided with `<exercise>` to assist students, and having them available as `knowls` in HTML output is a great way to make them easily available, but not immediately visible. But the author has also written some, or many, `<solution>` for the `<exercise>`, but these are only meant for instructors, and not for students. See the discussion at [Section 36.1](#) for more background.

One approach is to distribute an Instructor Version or Solution Manual only on request, and only as a PDF. The ability to provide a watermark on every page (see [Section 26.5](#)) allows you to include a personalized message such as

Issued to Charles Darwin. Do Not Copy.

It would be a trivial technical exercise to remove this, but perhaps the moral imperative (in an extra `<preface>` as well?) would dissuade most from distributing further?

But what about publicly available source code? After several unsatisfactory experiments, we have arrived at the following *solution*. Again it involves an intermediate understanding of the revision control software, git. And again, this is an outline.

- Create a private repository for authors, and other trusted contributors. In other words, if shared, read access is controlled via passwords or something similar.
- Create a branch off of master called solutions.
- Do all editing of private material, and only editing of private material, as commits to this branch. So a typical commit might just be `<solution>` elements inside existing `<exercise>`. Any script or top-level file for producing a solution manual might also be part of this branch.
- Do all authoring in this private repository, mostly as commits on master.
- Periodically, while the solutions branch is checked out, merge master to bring in new changes to the main content.
- Never ever merge solutions into master. In other words, solutions is a long-lived branch which never dies and is never merged into another branch. (Never rebase this branch if you have collaborators sharing the private repository.)
- Push and pull both master and solutions to and from the private repository by setting up tracking branches.
- Create a public repository which is a strict duplicate of the master branch. Periodically push the master branch of the private repository to the master branch of the public repository. Only. Its only purpose is for the next item. Use commands or a setup which makes it impossible to accidentally push solutions to this public repository.

- The commits in the public repository will be *identical* to those on master in the private repository. So anyone can clone or fork this repository and make pull requests, which authors can apply and manage via the private repository. But solutions will never be part of the interaction with this repository.

Chapter 38

WeBWorK Exercises

Alex Jordan

You can embed WeBWorK exercises in your PreTeXt project. This requires using a webwork2 server that is version 2.16 or later. The server could be the AIM WeBWorK server that has been set up for this purpose, so to get started, there is no need to establish your own webwork2 server.

Part of the process can optionally be sped up by using a local copy of the pg repository.

PDF and other static output formats will display static versions. HTML output can have interactive exercises or display problems in static form. If the HTML book is hosted on Runestone, reader interactions can be recorded so that scores can be given to students.

All such exercises can be archived into a file tree to be uploaded onto a webwork2 server for use in the “traditional” way.

See [Chapter 7](#) for how to include or create WeBWorK problems in your source.

38.1 Using a Local Copy of PG

WeBWorK problems can be processed using a local copy of the pg repository. This can significantly speed up processing time, compared with the other options that use a network connection.

To use this processing method, clone the pg repository. The code below places it in a default location, but you could place it somewhere else.

```
cd /opt
mkdir webwork
cd webwork
git clone https://github.com/openwebwork/pg
```

Note that you should end up with a version of pg that is 2.19 or later. To check, examine the VERSION file in the root of the repository.

Set the PG_ROOT environment variable to be the path to your copy of pg. If it is at the default location, you could run:

```
export PG_ROOT=/opt/webwork/pg
```

This will only define \$PG_ROOT for your current terminal session, so you might also do whatever is appropriate on your system to set PG_ROOT each time a terminal session begins.

In pg/conf/, make a copy of pg_config.dist.yml called pg_config.yml. In general you should read through all the settings and set them appropriately. At a minimum, set directories: root: to the absolute path to your clone of pg. You may need to set paths in externalPrograms: as well. And maybe more...

For PreTeXt in general you should already have ImageMagick installed. Somewhere in your file system is an XML file that ImageMagick uses to control permissions. The file is named policy.xml and might be located in a place like /etc/imagemagick/policy.xml or /etc/ImageMagick-6/policy.xml. (If your version of ImageMagick

is not 6, it's possible none of this is necessary. But also it should be harmless to continue.) In this file, within the <policymap> element, add the element:

```
<policy domain="coder" rights="read" pattern="PDF" />
```

Depending on your file, you may need to *replace* a similar line that is assigning rights="none". Make sure that this is not within a commented out portion of the file.

You need to have a recent enough perl version. Version 5.32 is known to work, and versions as early as 5.26 may work, but have not been thoroughly tested.

PG also uses perl packages that you may not yet have and need to install. These may be available from Linux package distributions, and the details differ from one flavor of Linux to the next. Alternatively there is a utility called cpanm (cpanminus) that is supposed to let you manage perl module installation across Linux flavors. One important difference is that cpanm only manages perl modules. And sometimes a perl module is actually an interface between perl and some other system utility. cpanm will not actually install that system utility that it depends on and will just quit. Whereas Linux package distributions will have a more sophisticated dependency understanding, and try to install non-perl tools when needed. So you want to try to use Linux package distributions where you can. Or else use cpanm and perhaps be prepared to read the log if it fails and attempt to install some non-perl tool that a particular module depends on.

Here is a list of all the perl modules you might need. Some are essential, and some are only needed for tools that many exercises will not use. Carp Class::Accessor Class::Tiny Data::Dumper DBI Digest::MD5 File::Copy File::Find File::Path File::Spec GD Getopt::Long HTML::Entities Locale::Maketext::Lexicon MIME::Base64 Mojo::Base Mojo::IO Loop Mojo::DOM Mojo::JSON Opcode Pandoc Perl::Tidy Pod::Simple::Search Scalar::Util Socket Tie::IxHash UUID::Tiny YAML::XS. If you don't have one of these that is needed, and you attempt to process WeBWorK problems using local PG processing, you will get a clear message about which module is missing that you must pursue installing it.

Here are some Linux package collections that we think are sufficient (but please report if you find them to be insufficient).

Debian/Ubuntu	liblocale-maketext-lexicon-perl, libyaml-libyaml-perl, libuuid-tiny-perl, libclass-tiny-perl, libdbi-perl, libgd-perl, libgd-text-perl
RedHat	List not yet gathered. Please report if you put together a good list.

To try to install using cpanm, of course first you must have cpanm. Try cpanm --version to see if you have it. If not, follow installation instructions for [Ubuntu](#), [Debian](#) (sudo apt install cpanminus), RedHat (sudo dnf install perl-CPAN), or MacOS (brew install cpanminus, requiring you first install [Homebrew](#)). Or you could install cpanm directly following [these instructions](#).

Once you have cpanm, you can opt to install the above modules one at a time, or all at once. Either way, it is probably adequate to install the modules as your own user. And yet we will recommend that you install the modules as root to avoid certain installation headaches and so that your system is set up to use webwork2 if that is ever desired. To install one at a time, run commands like: sudo cpanm Carp. But proceed through each module listed above. This will help you clearly see if there was an issue during installation. To install all at once: sudo cpanm Carp Class::Accessor ... YAML::XS, replacing the dots with the full list of modules above.

One known bump in the road may come with the GD module. You may need to actually install gd first, which is not a perl module. The perl GD module is the perl bridge to gd. For Ubuntu/Debian, install these packages: pkgconf, gdlib libgd-dev.

Now in a publisher file, you can set the <webwork> element to have @static-processing with value local (see [Subsection 44.8.1](#)). If you put the pg repository in the default location, this is all you need to do. Otherwise, you also need to add the attribute @pg-location with the absolute path, for example /opt/webwork/pg (see [Subsection 44.8.2](#)). And when you process WeBWorK exercises for static representations (following [Subsection 38.5.1](#)) your local copy of PG will be used instead of some remote server.

Note that you will still need a network connection for problems in your HTML output to render as live interactive exercises.

38.2 Configuring a webwork2 Server for PreTeXt

Note that the default webwork2 server for PreTeXt is webwork-ptx.aimath.org, and this server is already configured. If you are configuring your own WeBWorK server, read on.

We assume a mild familiarity with administrating a WeBWorK server. The version of WeBWorK needs to be 2.16 or later for use with PreTeXt.

The only thing you need to do at the serverlevel is set the web server to use certain headers on content that is fetched. These headers tell a web browser that you are authorizing it to display content from this web server as embedded content inside pages from another web server (in particular, where you are hosting you PreTeXt book).

WeBWorK 2.16 or 2.17. In `webwork2/conf/` there should be the apache configuration file `webwork-apache2.4-config`. This file needs to include the lines:

```
<IfModule mod_headers.c>
  <Location /webwork2/html2xml>
    Header set Access-Control-Allow-Origin "*"
  </Location>
  <Location /webwork2_files>
    Header set Access-Control-Allow-Origin "*"
  </Location>
</IfModule>
```

Note there is the distribution file `webwork-apache2.4-config.dist`, which has these lines (or similar lines) commented out.

You may also need to enable headers by executing something like `sudo a2enmod headers` from the command line.

WeBWorK 2.18. Edit `webwork2/conf/webwork2.mojolicious.yml` and uncomment these lines:

```
extra_headers:
# /webwork2/render_rpc:
#   Access-Control-Allow-Origin: '*'
/webwork2/html2xml:
  Access-Control-Allow-Origin: '*'
/webwork2_files:
  Access-Control-Allow-Origin: '*'
/pg_files:
  Access-Control-Allow-Origin: '*'
```

Note the `render_rpc` lines can remain commented out, unless you have some other reason to uncomment them.

You will need to restart the webwork2 service.

WeBWorK 2.19. Edit `webwork2/conf/webwork2.mojolicious.yml` and uncomment these lines:

```
extra_headers:
  /webwork2/render_rpc:
    Access-Control-Allow-Origin: '*'
# /webwork2/html2xml:
#   Access-Control-Allow-Origin: '*'
/webwork2_files:
  Access-Control-Allow-Origin: '*'
/pg_files:
  Access-Control-Allow-Origin: '*'
```

Note the `html2xml` lines can remain commented out, unless you have some other reason to uncomment them.

Also in this same file, set `allow_unsecured_rpc: 1`. You will need to restart the webwork2 service.

38.3 Configuring a WeBWorK Course for PreTeXt

To make use of WeBWorK in your PreTeXt project, you need a host WeBWorK course. If someone is providing a WeBWorK server for you, like the AIM server at webwork-ptx.aimath.org, then we assume they have already configured a course for use with PreTeXt, and you may skip this section and move on to [Section 38.5](#).

If you are configuring your own WeBWorK server to use with PreTeXt, we assume a mild familiarity with administering a WeBWorK server. The version of WeBWorK needs to be 2.16 or later for use with PreTeXt.

Using the admin course, create a course named anonymous. (You could name it something else, but we assume the name is anonymous in this guide.) In the course's Course Configuration menu, set all permissions to admin (or perhaps set some to the even more restrictive nobody). Except set "Allowed to login to the course" to login_proctor. Also if your server is new enough, it has "Allowed to view course home page" which should also be set to login_proctor.

In the Accounts Manager(Classlist Editor in older versions of WeBWorK), add a user named anonymous (again, you could use some other name), and set that user's permission level to login_proctor, the permission level one higher than student. Set that user's password to anonymous (again, you could use some other password). Note that because this is public information, anyone will be able to log into this course as this user. This is why restricting permissions in the previous paragraph is very important.

Add the following lines to the course.conf file (which lives in the parent folder of the templates/ folder.)

```
# Hide message about previewing hints and solutions for instructors
$pg{specialPGEEnvironmentVars}{ALWAYS_SHOW_HINT_PERMISSION_LEVEL} = 100;
$pg{specialPGEEnvironmentVars}{ALWAYS_SHOW_SOLUTION_PERMISSION_LEVEL} = 100;
```

WeBWorK 2.16, 2.17, or 2.18 (not needed for 2.19 or later). In the templates/macros/ folder, edit PGcourse.pl (or create it if need be) and add the following lines.

```
#### Replace essay boxes with a message
my $essay_message = 'If you were logged into a WeBWorK course '
    . 'and this problem were assigned to you, '
    . 'you would be able to submit an essay answer '
    . 'that would be graded later by a human being.';

sub essay_box {
    my $out = MODES(
        TeX => '',
        Latex2HTML => '',
        HTML => qq!<p>$essay_message</p>!,
        PTX => '',
    );
    $out;
};

sub explanation_box {
    return if ($envir{waiveExplanations});
    my $out = MODES(
        TeX => '',
        Latex2HTML => '',
        HTML => qq!<p>$essay_message</p>!,
        PTX => '',
    );
    $out;
};

#### Suppress essay help link
sub essay_help {};
```

```
#### How many attempts until hint is available
$showHint = -1;
# May be a bug that WeBWorK requires -1 instead of 0
# for immediate access to hints

1;
```

Now PreTeXt will be able to communicate with this course to retrieve what is needed.

38.4 PG Macros from the PreTeXt Source

The project's WeBWorK exercises may rely on PG macros that are written into the project's source. For example, the exercises might have TikZ images that rely on `docinfo/latex-image-preamble`.

For this, a PG macro library file must be built and placed in the host course's `templates/macros/` folder *before* attempting to process the WeBWorK exercises. To build this macro library, run:

```
pretext -c pg-macros aota.ptx
```

`aota.ptx` in the example is the root file for your PreTeXt project. You could also specify a location to place the resulting macro library file:

```
pretext -c pg-macros -d some/file/path aota.ptx
```

Once you have the macro library file, upload it to the host WeBWorK course's `templates/macros` folder. If your project relies on the AIM WeBWorK server and you need to supply a macro library file to a host course on that server, post to pretex-support@googlegroups.com and we can help with that.

38.5 Processing WeBWorK Exercises

38.5.1 Extraction and Processing

Before anything else can be done, a PreTeXt project with WeBWorK problems must first have its WeBWorK content extracted and processed into multiple representations, which are then collected into an auxiliary XML file. Using the CLI, this is done with `pretext generate webwork` (if you want to reference a particular target, add `-t [targetname]`).

If instead you want to use the `pretext/pretext/pretext` script, you might need to install the Python `requests` module. It is not uncommon for your computer to not have `requests` installed (although it comes with the CLI), so you should check if it is there and install it if need be. You can check if it is installed from the command line with:

```
python -c "import requests"
```

And if it isn't, you can install it with `pip`, specifically with:

```
pip install requests
```

(If you don't have `pip` installed, you could use:

```
easy_install pip
```

to install it.)

Processing WeBWorK problems is best accomplished if you are using directory management. With a publication file, declare the external and generated directories as describe in [Section 5.6](#). Then use the pretext script to extract PreTeXt content from the WeBWorK server. It will be placed in a webwork subfolder of your generated folder. For example:

```
$ pretext -c webwork -p <publisher> aota.ptx
```

aota.ptx in the example is the root file for your PreTeXt project.

Warning 38.5.1 File Paths. In the previous example and those that follow, you should specify paths as needed. For example, the pretext script is typically at `~/pretext/pretext/pretext`. And the `-p` option is specifying a publication file.

`-c webwork` means you are processing the WeBWorK components.

`-p` specifies the publication file (see [Chapter 44](#)). In the publication file, the element `<webwork>` may have attributes:

- `@server` ([Subsection 44.8.3](#))
- `@course` ([Subsection 44.8.4](#))
- `@user` ([Subsection 44.8.5](#))
- `@password` ([Subsection 44.8.6](#))

If absent, these default to `https://webwork-ptx.aimath.org`, `anonymous`, `anonymous`, `anonymous`, and `anonymous` respectively. If you specify a server, you must correctly specify the protocol (`http` versus `https`). And it must be version 2.16 or later. Do not include a trailing slash.

38.5.2 HTML output

We assume you are using managed directories, and have WeBWorK representations file as above. Build the HTML with `pretext build TARGET` where `TARGET` is the name of a target that has `format="html"`. You can also use `pretext/pretext` to build HTML. For example:

```
$ pretext -c doc -f html -p <publisher> -d <destination> aota.ptx
```

`-p` specifies the publication file, as described in [Chapter 44](#).

`-d` specifies a folder to place all of the HTML output. If unspecified, this will be the current working directory.

There are five publisher file entries which control how a WeBWorK problem is realized within HTML output. These are divided based on where the exercise (or project-like) resides: `inline`, within a division of exercises, within a reading questions division, within a worksheet, or if it is a project or similar. If the class of problems is declared `dynamic`, then each problem has a button that readers can click to make the problem interactive. Conversely, if the class of problems is declared `static`, then the problems render with a static preview.

By default, inline exercises and project-like will be `dynamic`, under the assumption that these are meant to be worked as a reader works through the material. The others will be `static`, under the assumption that they will be placed on a WeBWorK server where they will be worked for scores and grades. See [Subsection 44.4.6](#) for the precise syntax for these switches.

If an exercise is subdivided into tasks, then by default they will only be revealed incrementally as the reader answers each one correctly. If you would like to have all tasks revealed from the start, then this option may be elected in the publication file (see [Subsection 44.8.7](#)).

38.5.3 L^AT_EX output

We assume you are using managed directories, and have WeBWorK representations file as above. Then build as normal with the CLI. Or you may use `pretext/pretext` to build a L^AT_EX PDF. For example:

```
$ pretext -c doc -f pdf -p <publisher> -d <destination> aota.ptx
```

-p specifies the publication file, as described in [Chapter 44](#).

-d specifies a folder to place the PDF output. If unspecified, this will be the current working directory.

38.5.4 Creating Files for Uploading to WeBWork

All of the `<webwork>` that you have written into your project can be “harvested” in such a way that you could send them to a WeBWork server for use within WeBWork. The components you would want are PG files for the exercises, PG macros that you wrote, set definition files corresponding to the sections (or other chunks) of your project, and header files for those sets. All of this was placed into your project’s `generated/webwork/pg` folder when you executed `pretext -c webwork`, so you could find it there and package it all up to send to your WeBWork server.

Alternativley, you may run the following:

```
$ pretext -c all -f webwork-sets -p <publisher> -d <destination> aota.ptx
```

This will specify a destination for that folder tree other than within `generated/webwork/pg`. If you additionally use a `-z` flag, you will instead get a compressed tarball (`.tgz` file).

38.6 Unachievable Conversions

By authoring WeBWork problems within PreTeXt you do not need to learn all the ins and outs of PGML markup and you can concentrate on simply becoming proficient with PreTeXt. However, there are a few PreTeXt constructions which are not achievable in a WeBWork problem for one reason or another. We list exceptions here, and also try to use source-checking tools to alert you to any differences.

- Anything that is the numbered target of a cross-reference, such as a figure, may not be inside a WeBWork exercise. The exercise may go on to have a life of its own independent of its parent PreTeXt project, and then such a number makes no sense.
- Certain aspects of specifying borders of a PreTeXt `<tabular>` are not realizable in a PGML table. Specifically,
 - Specifying column-specific top border attributes are not implemented.
 - Cell-specific bottom border attributes are not implemented.
 - `medium` and `major` table rule-thickness attributes will be handled as if they were `minor`.
- When constructing a list (`<ul>` or `<ol>`) specifying some number of columns (using the `@cols` attribute) will be ignored. PGML markup has no way to declare multicolumn lists.

Chapter 39

Hosting Your Online Version

You have HTML output, and now where do you put it? A fundamental design decision is that you only need to simply upload your HTML files to a hosting service and since all the links are relative, readers should be able to read your whole book with no more effort than that from you. By design, no extraordinary configuration or privileges are necessary on the server.

Users of the PreTeXt-CLI (Section 5.2) have access to `pretext deploy`, which walks users through the process of deploying their document to the free [GitHub Pages](#) service, even if they don't manually manage their Git revision history.

Otherwise, for the choice of a **hosting service** you may have a fundamental decision to make. Mostly this applies to authors who are employees of an institution, yet have the freedom to control the copyright on their scholarly work. But there is information here for independent scholars and for other employees.

- You love your institution, and plan to stay for a good long time. They have implicitly (or explicitly) supported your project with time and/or money. A URL with the institution's domain name on a freely-accessible project is good advertising for the institution. Bandwidth is huge, IT is super reliable and helpful, all this is no-cost to you. Read the next scenario, but you have a good situation, so you might as well use it.
- You are not really attached to your institution, and five years from now you may be somewhere else. Consider hosting your project externally, so it is not tied to your institution.

Or maybe policy on faculty web pages, or crummy content management systems, make it difficult or impossible to host your project. Or it is buried five levels deep with an impossible URL. Point out the situation to your Provost or Dean, with examples of how *other institutions* do it right. Remember that your colleagues may be writing monographs and textbooks for commercial publishers, likely with institutional support, and selling their copyright. Your institution should be *proud* to host your project prominently. If a reasoned, rational approach does not improve the situation, then consider hosting your work elsewhere.

If you are hosting at your institution, that is a great outcome. There is no cost to you, and everybody is happy. Lobby for a great URL, like `platypus.mammal-institute.org` and the rest should take care of itself. The rest of this section is about the second situation.

To arrange hosting yourself,

1. Purchase a domain name, it should not be a real big annual expense. Choose something professional, rather than just your name (though your name does have a natural appeal). And maybe something general enough that you can host your next book under that same domain name. The idea here is to *own* the domain name, so your book can move anywhere, but that domain name will always point to the book. This name should be *owned and controlled by you*, not your institution, not GitHub, not 5GBFree.com.
2. Sign up for, and perhaps pay for, a hosting service that lets you point your domain name at the site.
 - Oscar Levin explains that [GitHub Pages](#) is free, super-easy to use if you already use `git`, and makes using your domain name ("custom URL") nearly trivial. (2017-09-08)
 - Mitch Keller likes the "Swift" plan at [A2 Hosting](#) at about \$60 annually. (2017-07-05)

Now you are set, and control distribution of your scholarly publication. If you are bothered by the thought of having expenses while you make your work freely available to the world, then consider generating some modest income. For example, sell Google ads against your pages. (Why should *this* disturb anybody? I don't get it.) Or roll a small royalty into the print-on-demand version, see [Chapter 43](#).

There are a few practical details to think about. Eventually others will link to your book, and you will also release updates. First, think about creating a simple high-level directory that will be stable, short, easy to type, and easy to remember. Controlling your domain name (above) is the first step. Then consider that you may also distribute a PDF version, and you may someday write a second book. So, for example, your URL might look like:

```
https://platypus.mammal-institute.org/aota/html
https://platypus.mammal-institute.org/aota/aota.pdf
```

For the first URL, the html directory would contain all of the PreTeXt HTML output, and especially, an `index.html` file which most any web server will serve up when the URL ends with a directory. PreTeXt has tools to help you with creating the `index.html` file.

An improvement on the above is to have stable generic URLs for the current version, and dated, or versioned, URLs for older versions you may wish to keep in place (as a record, or for instructors who want to stick with an old version). It is a bit more work to maintain, but will lessen the frequency that an old version of your work is promoted as the last word.

```
https://platypus.mammal-institute.org/aota/latest
https://platypus.mammal-institute.org/aota/aota.pdf
https://platypus.mammal-institute.org/aota/html-2017-08-15
https://platypus.mammal-institute.org/aota/aota-2017-08-15.pdf
```

Chapter 40

L^AT_EX Fonts

Part of a book’s style is the choice of fonts. As of 2019-11-09 we have a better understanding of the use of fonts in L^AT_EX to the point where we can design interfaces that will make it simpler for you to experiment with different choices and preserve various features that PreTeXt enables.

The first thing to understand is that the x_el_at_ex engine is much more capable of employing modern fonts. T_EX was built in the late 1970’s when computer resources were at a premium, and the idea of mixing mathematics with non-Western languages and scripts may have been fanciful. The pdf_la_te_x engine is rooted in this history. We now have the Unicode standard, thoroughly integrated into web browsers, and companion scalable OpenType fonts. In contrast to T_EX, X_eT_EX was designed to work better with a multitude of fonts. So we organize this section by this distinction.

40.1 Processing with pdf_la_te_x

Fonts used by L^AT_EX come in **encodings**. The original encoding is known as OT1 and organizes glyphs (shapes of individual characters) 128 at a time. Since this puts regular Latin letters in one group, and accented letters into another group, this makes automatic hyphenation impossible if accented letters are used (as in many European countries and much of the Americas). The 1990 T_EX Users Group (TUG) meeting in Cork, Ireland, formulated new and improved encodings. The T1 encoding is one result, and so it is known as the **Cork** encoding (and also as EC). It groups 256 glyphs together at a time. So you need only know that a T1 encoding is better than an OT1 encoding. If you want to know more (much more), locate “L^AT_EX font encodings” by Mittlebach, Fairbairns, & Lemberg.

The [L^AT_EX Font Catalogue](#) is a great resource for locating different fonts. The fonts and their packages are likely already installed (this information is provided), or there is enough information about how to install the package in a standard way. Each gives exact directions on how to enable the font for a document. For example, [Iwona](#), a sans-serif font with support for mathematics, can be employed as the document font simply by including the following in the preamble:

```
\usepackage[math]{iwona}
\usepackage[T1]{fontenc}
```

This suggested use of fontenc package (not to be confused with the fontspec package) indicates the availability of a T1 encoding.

Note: when the L^AT_EX Font Catalogue says “OTF or TTF available” then the font can also be used with x_el_at_ex. When it says “OTF or TTF only” then the font *cannot* be used via pdf_la_te_x. Read on.

40.2 Processing with x_el_at_ex

TrueType fonts (TTF, *.t_tf files) have been improved upon by the creation of OpenType fonts (OTF, *.o_tf files). The main difference is that OTF fonts have a variety of **features** which can be selected or not. It is also easier to directly

select a particular glyph (realization of a character) by specifying its numerical code point as a Unicode character. For the remainder, we will reference OTF fonts only, but nearly everything applies equally well to TTF fonts.

The main difference with xelatex and OTF fonts is that xelatex expects the font files to be part of the system software and are managed by the operating system and its tools. So, for example, in the example above installing the iwona package will place files into your **texmf** tree, where they remain unknown to your operating system. So you will need to learn how to use your operating system to locate and install OTF files (or make them known to your operating system). With luck, popular fonts may be easy to install using your system's package manager.

So installing fonts into a system is a bit of a hurdle, and as a style writer, you are reliant on authors who use your style to understand this requirement.

Locating an OTF font. Since I have the $\text{\LaTeX}$ iwona package, I can expect the OTF version of the font to be somewhere. I know I have the $\text{\LaTeX}$ package by using a TeXLive tool to search for the style file.

```
kpsewhich iwona.sty
```

will return the full path to that file.

On my Linux system, with TeXLive installed via a Debian/Ubuntu package, I can locate the main OTF file for the Iwona font by exploring the relevant directories (fonts/opentype looks promising), or I can use the system find utility to search for filenames with iwona or Iwona in the name. There it is:

```
/usr/share/texlive/texmf-dist/fonts/opentype/nowacki/iwona/Iwona-Regular.otf
```

You can simultaneously determine if you have a font, and if it is known to your system with the `fc-list` command from the fontconfig utility. The produces a lot of output, with one line per file, but it includes file names and the human-readable name of each font. You can pipe the output through `grep` to find what you are looking for, for example

```
fc-list | grep "Latin Modern"
```

Making an OTF font known to the system. On my Linux system, I need to copy this file (and its companions) into the system directory

```
/usr/local/share/fonts
```

and reboot, or rebuild the font cache with

```
fc-cache -f -v
```

Finally, I can check that the font is known to the system with

```
fc-list
```

On a Mac, we have several reports for how to do this.

- Mitch Keller reports on 2019-01-02 that “I opened the directory on my hard drive containing the OTF file, double clicked on the font (opens the macOS application Font Book, which comes with the OS), and then clicked the Install Font button.”
- Karl-Dieter Crisman reports on 2019-07-01 that “I was able to use some fonts already existing in TeXLive with a symbolic link to the Font Book, as suggested at [Ask Different](#).”

```
ln -s
  /usr/local/texlive/2018/texmf-dist/fonts/opentype/public/lm/
  '/Library/Fonts/Latin Modern'
```

“where 2018 indicates when I updated my TeXLive distribution.”

Information about an OTF font. To learn more about this particular font TeXLive provides an executable, `otfinfo`, which I can use as (omitting the long directory path here for clarity)

```
otfinfo -i Iwona-Regular.otf
```

The first part of the output is

```
Family:           Iwona
Subfamily:        Regular
Full name:        Iwona-Regular
PostScript name:  Iwona-Regular
```

The critical piece of information here is the font's name: `Iwona-Regular`. I can also learn which OpenType features are available

```
$ otfinfo -f Iwona-Regular.otf

aalt  Access All Alternates
c2sc  Small Capitals From Capitals
csp   Capital Spacing
dlig  Discretionary Ligatures
dnom  Denominators
frac  Fractions
hist  Historical Forms
kern  Kerning
liga  Standard Ligatures
lnum  Lining Figures
numr  Numerators
onum  Oldstyle Figures
ordn  Ordinals
pnum  Proportional Figures
sinf  Scientific Inferiors
smcp  Small Capitals
ss02  Stylistic Set 2
sup   Superscript
tnum  Tabular Figures
zero  Slashed Zero
```

This will all be useful shortly.

Using an OTF font with xelatex. Depending on the package, now it can be as simple as simply including `\usepackage{}` in your preamble and the font can be used with different weights and shapes, and certain features are enabled. However, sometimes you want more control, or you want to use more than one font in a document. Now the `fontspec` package is your tool of choice.

The simplest example of using `fontspec` would be to recall the font name from above, and in your preamble use

```
\setmainfont{Iwona-Regular}
```

This font name has the hyphen in lieu of a space, but names with spaces are just fine. An optional argument will let you customize the use of the font, such as turning on some of the features. Read the `fontspec` documentation for all the finer points. Hopefully this has given you a start.

Note that `fontspec` has a `Path=` option. A tempting shortcut is to just point to the `*.otf` file in your `texmf` tree. But as a style writer, this is a really bad idea, since other authors who use your style may have their font files in

a different location. Better to employ the font *by name* and require authors to understand (or learn) their system software.

Checkpoint 40.2.1 Create a $\text{\LaTeX}$ file (from scratch) containing some Myanmar (Burmese) characters. The [Kermit Project UTF-8 Sampler](#) is a great source for these sorts of experiments. The Noto font is a project to make a huge collection of harmonious fonts for much of the world's languages. Use what you have learned here to render your sample using a Noto font via both `pdflatex` and `xelatex`. Extra credit if you can explain the name Noto.

Plans, as of 2019-11-09. We have experimental code in place to allow a style writer to place font information into the $\text{\LaTeX}$ preamble in the right place. We are adding font-changing commands into other configurations in the preamble, so that pieces of a document (e.g. the page header) will automatically use a particular font easily.

We have long been using the $\text{\LaTeX}$ `polyglossia` package to manage multiple languages in one document, and will continue to do so, perhaps. This package relies heavily on `fontspec`. We intend to hide the details of `polyglossia` from authors and from style writers, relying on just the specification of `@xml:lang` attributes to control font changes.

40.3 Font Notes

40.3.1 Dyslexic Font

There is an OTF font, with an open license, that purports to help readers with dyslexia.

- Website is opendyslexic.org/.
- In Debian/Ubuntu it is available in a package called `fonts-opendyslexic`.
- When used with `xelatex` the relevant names of the font are `OpenDyslexic` and `OpenDyslexicMono`.

40.4 LaTeX Font Configuration (Ubuntu/Debian Linux)

Actual font installation is missing here, since I (RAB) cannot recall just when or how certain fonts arrived on my system. Certainly they were almost all via Ubuntu/Debian packages, though they could have been specific to $\text{\TeX}$ Live. Specifically, the `texlive-fonts-recommended` and `texlive-fonts-extra` are two packages that will make many fonts available to $\text{\TeX}$ / $\text{\LaTeX}$ on an Ubuntu/Debian system. The following is offered in the hope that it will be useful to other publishers on other Unix-like systems.

There is a system directory

```
/etc/fonts/conf.d
```

with a wide variety of configuration files for various fonts, or collections of fonts. Here I find files (symlinks, really)

```
65-fonts-lmodern.conf
65-fonts-texgyre.conf
```

The first points to the extensive Latin Modern fonts, which are an improved version of the original Computer Modern fonts, and are $\text{\PreTeXt}$'s default font for out-of-the-box $\text{\LaTeX}$. We have never had a report of these not being available in an author's $\text{\TeX}$ distribution. The file indicates that the fonts can be found at

```
/usr/share/texmf/fonts/opentype/public/lm
/usr/share/texmf/fonts/opentype/public/lm-math
```

The second configuration file points to multiple fonts from the [TeX Gyre Collection](#) of GUST: Polska Grupa Użytkowników Systemu $\text{\TeX}$. Examining the file indicates these fonts can be found at:

```
/usr/share/texmf/fonts/opentype/public/tex-gyre
/usr/share/texmf/fonts/opentype/public/tex-gyre-math
```

As of 2019-11-09 these were the only T_EX fonts known to my system in OTF format. This despite having directories full of fonts at:

```
/usr/share/texlive/texmf-dist/fonts/opentype
/usr/share/texlive/texmf-dist/fonts/truetype
```

and more. You might have similar directories with the year of your version of T_EXLive as a directory. The solution is to create a new file (as root) named

```
/etc/fonts/conf.d/09-texlive-fontconfig.conf
```

with contents

```
<?xml version="1.0"?>
<!DOCTYPE fontconfig SYSTEM "fonts.dtd">
<fontconfig>
  <dir>/usr/share/texlive/texmf-dist/fonts/opentype</dir>
  <dir>/usr/share/texlive/texmf-dist/fonts/truetype</dir>
</fontconfig>
```

and then running `fc-cache -f -v` to update what fonts are known to the system. It is possible that you can put this file somewhere in your home directory if you do not have administrative access, but we have not tested that approach. Note that some versions of the above file that you might find online will include a `type1` directory. It is best to *not* include this directory, since these fonts are best used only with `pdflatex` and if known to the system they can mistakenly be incorporated by `xelatex`, with disastrous results. Typically you will get an unusable `pdf` and your `xelatex` run will have the error message

```
xdvipdfmx:fatal: pdf_ref_obj(): passed invalid object
```

near the very end of the command-line output.

Apparently, the `texlive-fontconfig.conf` file is not distributed with Debian Linux as this [search](#) will demonstrate. Please report any change in this situation.

When installing the Open Dyslexic font via an Ubuntu package (2020-04-28), `xelatex` became confused by the presence of Web Open Font Format (WOFF) versions which were installed along with the OTF versions. The solution is to create a new file (as root) named

```
/etc/fonts/conf.d/70-no-woff.conf
```

with contents

```
<fontconfig>
  <selectfont>
    <rejectfont>
      <glob>/usr/share/fonts/woff/*</glob>
    </rejectfont>
  </selectfont>
</fontconfig>
```

and then running `fc-cache -f -v` to update what fonts are known to the system. See the tex.stackexchange.com post.

Chapter 41

L^AT_EX Styles

Print and PDF output is created when PreTeXt outputs a L^AT_EX file, which can subsequently be easily converted to PDF with engines like pdf_latex and xel_atex. There are many ways to influence the style (look, appearance) of this output, enhancing the content, but without changing it. This chapter is directed at publishers who desire to create an attractive and consistent design for their books or articles.

Please read this chapter thoroughly once before embarking. Certain important points are made in certain contexts, yet are universally applicable. If you cherry-pick, you will miss them. Note also that some simple changes, like font size, are described in [Chapter 30](#).

41.1 Preparation

We do not encourage authors to make small adjustments in style, especially if they have few skills in book design. Instead, they should choose a design built by others that will fit their needs and desires. We do encourage publishers with design skills to create complete and harmonious designs, and to donate these back to PreTeXt with an open license, for use by all authors. This chapter assumes you are such a publisher. Further, it assumes you have certain technical skills. Specifically

- Good familiarity with basic T_EX and L^AT_EX.
- The ability to debug L^AT_EX compilations gone bad.
- Willingness to study several L^AT_EX packages that may be new to you.
- Willingness to mimic and experiment with basic eXtensible Stylsheet Language (xsl).

Fortunately, it is easy to start small, get good results, and expand your skills further.

Begin by creating a file that is a new xsl stylesheet. You can likely safely copy a mature one from the xsl/latex directory. Be certain to keep the first few declarations. The `<xsl:import>` is critical, since it will “pull in” all the basic code for the PreTeXt conversion to L^AT_EX. You will be overriding and appending to that code (which PreTeXt has made straightforward). You can start with an absolute path from your filesystem root, but once public a relative path will be necessary. Remove all of the `<xsl:template>` elements, leaving a hollow shell to begin working with.

What we are doing here is similar to the discussion of “extra xsl stylesheets” in [Section 28.2](#), only thicker. String parameters are also described in this Guide, at [Section 28.1](#).

41.2 Overview

Some changes in style are effected by setting string parameters that exist for use at the command line. However, the more flexible features come from the selection by PreTeXt of certain L^AT_EX packages. These have been chosen for their flexibility, maturity, and stability. They should be part of a full L^AT_EX installation, especially one based on TeXLive. We presume each author has a similar installation. Please let us know of any exceptions. Please try to avoid

requiring new packages as part of your style, and if necessary, be sure they are mainstream ones. Start a discussion on the development forum if you think it is warranted or necessary. It may be difficult and error-prone for you to employ and integrate an obscure package, and it will cause problems for authors who want to use your style.

This is an incomplete list of the primary packages we employ, and their general purpose. They, and their documentation, can be easily found at the [Comprehensive T_EX Archive Network](#), aka CTAN.

Table 41.2.1 Principal L^AT_EX packages used for styles

geometry	Specification of the sizes of paper, margins, headers and footers
titlesp	Headers and footers (part of titlesec)
titlesec	Titles of divisions
tcolorbox	Boxes, colors, etc. for <example>, <remark>, etc.

41.3 Page Shape

Various dimensions of a printed page, including the page itself, may be adjusted using the geometry package, so study the documentation of this package to explore possibilities. The options of this package may be also set on the command line. To make options part of your style, place the <xsl:param> declaration in your stylesheet as follows (note the two sets of quotes):

```
<xsl:param name="latex.geometry" select="'foo'"/>
```

This will have the effect of placing the following line into the preamble of the resulting L^AT_EX output file, in the right place:

```
\geometry{foo}
```

Of course, you will want to use something meaningful, such as

```
<xsl:param name="latex.geometry" select="'a4paper,total={16cm,25cm}'"/>
```

to produce in the L^AT_EX output

```
\geometry{a4paper,total={16cm,25cm}}
```

This is typical and illustrates two important universal points. First, PreTeXt puts surrounding infrastructure in place. In this case the geometry package is loaded, and in an order that does not cause conflicts, plus the \geometry{} command itself is placed and output by PreTeXt. As a style writer, you simply provide the package options you desire. Second, “garbage in, garbage out.” It is very easy to make a typo in your style, and have the L^AT_EX compilation fail. This is why we assume you are comfortable with L^AT_EX compilation and debugging, and not every author should be a style writer.

Set the document font size first, for this will influence later choices. The string parameter, latex.font.size (which has pt as part of the value) will set an optimal line width. This line width should translate to about 75 characters per line, at the upper end of recommendations for an optimal width. It will also match closely (but not exactly) to line lengths in HTML output. You can look into the L^AT_EX output before you experiment to see the value used when the stock \geometry{} command is issued. You are certainly able to override this width, but read the next paragraph carefully.

This raises two important more universal points. You should expect to repeatedly examine the L^AT_EX output as you develop a style. And most important—are you tempted to use a small font, and *increase* the line width so as to cram more material onto the page, so your book is shorter, and sells via print-on-demand for \$7 rather than \$8? Then you misunderstood that there are time-tested recommendations for the optimal number of characters per line for human readers, and we just counseled you that PreTeXt is *already at the high end of these recommendations*. You have a certain freedom as a style writer. Use it responsibly. Enhance the content provided by authors, don’t degrade it.

This is the place to think about headers and footers on the page, since you will want to make room for them, and with spacing away from the primary content. See [Section 41.4](#).

41.4 Headers and Footers

The `titleps` package cooperates with “traditional” $\text{\LaTeX}$ divisions, such as `\chapter` and `\section`, and the `titlesec` package, to pick up the titles of divisions automatically and migrate them to headers and footers on a page. The `ps` is short for “page style”, and the documentation is a PDF file *within* the distribution for the `titlesec` package. Primarily, we let $\text{\LaTeX}$ manage the selection of its page styles for various pages of an overall document: `empty`, `plain`, `headings`, and `myheadings`. PreTeXt does some management in the front matter. As a style writer it is not your concern where these styles are employed, but you do influence what information they contain and where it is placed on a page.

Add an `<xsl:template>` to your stylesheet that begins with

```
<xsl:template match="book" mode="titleps-style">
```

This would then set a collection of commands from the `titleps` package for a `<book>`, which will be placed in the correct place in the $\text{\LaTeX}$ preamble. (See [Section 41.8](#).) You can renew existing page styles or create new page styles. However you should always make your last line a declaration of the overall page style, for example,

```
\pagestyle{headings}
```

Note that a book may use the `empty` and `plain` styles for some pages, so you may need to renew those styles to be harmonious with other changes you have made.

The definition of this template will override (replace) the definition given in any imported stylesheet. You can replace the value of the `@match` attribute with `article` to make your style apply to a PreTeXt `<article>`. If your style will be used for both books and articles, and you want the style to be identical for both, you can expand the `@match` attribute to have the value `article|book`. To have different styles for a book versus an article, make two separate templates.

The `titleps` package allows at least twelve options per page style: even-numbered page versus odd-numbered page with two-sided printing; left, center, right; header or footer. There are semi-automatic customizable rules, variable widths allowing hanging styles, choices of marks (division at page-start versus division at page-end versus new-division-mid-page, including combinations at the same time), and more. Note that the $\text{\LaTeX}$ system of `\markleft` and `\markboth` has been abandoned. (Did I hear you say, “Good riddance!”?)

Some care must be taken with using `\thechapter` to get the chapter number, since strange things happen in the frontmatter and backmatter, where chapters are numberless. One solution is to use `\ifthechapter{ }{ }` which allows you to control the behavior dependent on whether there is a current chapter number. For example, if you wanted the center of the even-numbered pages to contain “Chapter 3: Derivatives” you would use

```
\sethead[ ][\ifthechapter{Chapter \thechapter: }{\chaptertitle}][ ]{ }{ }
```

which would then just put “Index” in the index, and nothing at all in the preface. A similar approach could be used to deal with section numbers and titles in the case that the introduction of a chapter is multiple pages, using `\ifthesection{ }{ }`.

Note also that the macro `\chaptertitle` will resolve to `Chapter` or `Appendix` as necessary.

41.5 Titles of Divisions

The `titlesec` package cooperates with “traditional” $\text{\LaTeX}$ divisions, such as `\chapter` and `\section`, to style the start of each division, containing its PreTeXt `<title>` and in most instances, its number. If a division is credited to (multiple) `<author>`, then that information can be styled, and there are plans (2018-09-30) for epigraphs.

PreTeXt manages numbered versus unnumbered divisions, the correct level for one-off divisions like a `<preface>` or `<appendix>`, and the specialized divisions such as `<exercises>` and `<references>`. A style writer creates two styles at each level of the hierarchy, for a numbered variant, and an unnumbered variant. `titlesec` uses a numberless key to indicate the latter. A named template, such as

```
<xsl:template name="titlesec-section-style">
```

would produce text containing complete `titlesec` commands `\titleformat` and `\titlespacing` in both numbered and unnumbered variants.

There are five $\TeX$ macros created by PreTeXt at the start of each division. For example, `\authorsptx` is a comma-separated list of the content of all the `<author>` elements for the division, in the order given. Look in the $\TeX$ output to find the others nearby. A robust style will include this information, even if the first use of the style may not have any divisions credited to others.

The table of contents and the index are created by a single $\TeX$ macro. This creates a small technical challenge, since PreTeXt never has a chance to write the contents of the heading and must take what it is given. The upshot is that the `\titleptx` macro will be wrong or empty. So instead, use the `titlesec` device of using the macro parameter #1 for the title in the correct argument of `\titleformat`. We have enabled this possibility through the package's explicit option. We believe the numberless variant of a `<chapter>` or a `<book>`, and the numberless variant of a `<section>` of an `<article>`, are the only places this is necessary. 2019-09-30: we will contemplate if this should be the rule and the `\titleptx` macro will go away. Advise if you see a good answer, either way.

2019-09-30: these templates are highly likely to break into two modes (format and spacing), with a `@match` that can react to `<chapter>`, `<section>`, etc., perhaps differently for books versus articles. The change will only imply some minor editing to achieve the same end result, so don't hold back waiting.

41.6 Environments and Blocks

Objects like `<example>` and `<remark>` are almost invariably children of a division, numbered, and work best with a `<title>`. `tcolorbox` is a massive package, that we have taken to as a solution to many under-the-bonnet technical problems, such as a hanging indent for numbers of `<exercise>` or laying out the panels of a `<sidebyside>`. But it also allows an incredible variety of styling options for these intermediate chunks of text. Think of variable placements of numbers and titles, borders and boxes, and colored backgrounds. With freedom comes responsibility! With a light touch, you can *help* your reader navigate the inherent structure of your PreTeXt source.

Whatever you call them: environments, blocks, or information objects, almost every one can be styled separately (2019-09-30: not captioned items yet, such as `<figure>`). For example,

```
<xsl:template match="example" mode="tcb-style">
  <xsl:text>colback=pink,</xsl:text>
</xsl:template>
```

would cause the background of every `<example>` to be light red in color. To make every PreTeXt element that is a variant of an `<example>` look identical, use one of the **entities** defined in `xsl/entities.ent`,

```
<xsl:template match="&EXAMPLE-LIKE;" mode="tcb-style">
  <xsl:text>colback=pink,</xsl:text>
</xsl:template>
```

. This would affect `<example>`, `<question>`, and `<problem>`, and future-proof your style when there is a demand for `<illustration>` as a new kind of “example-like.”

There is only one mode, but it can handle a variety of PreTeXt elements in the `@match`. The text produced by the template will be supplied in a named `tcolorbox` style via a `\tcbset{}` command. It is marginally more complicated than that. PreTeXt will manage certain aspects of creating a `tcolorbox`, such as forming the `tcolorbox` title to be a string like “Example 4.5 The Chain Rule.”, or just “Example 4.6” when an author does not include a PreTeXt `<title>`. As a style writer, you can change the font, color and placement of that string, but not the use of the type-name, the number, the title, or their order. The text of your template is additive, meaning it is in addition to what the stock PreTeXt conversion provides. Your options come last, so will be new and effective, or will replace what the base conversion to $\TeX$ does. So in the example above, the base conversion to $\TeX$ has `colback=white` (rather than the default gray!), and this will be overridden since pink will come later.

41.7 Preamble and Experimentation

Two hooks are provided to allow for arbitrary additions to the $\TeX$ preamble. It should go without saying that this is for experimenting with new features and is no way supported, including, but not limited to, interactions with

current $\text{\LaTeX}$ packages in use, or those added in the future.

Arbitrary text ($\text{\LaTeX}$) used as the values of the string parameters `latex.preamble.early` and `latex.preamble.late` will be added to the very beginning, or very end (respectively), of the generated preamble.

41.8 XSL 101

If you have read this far, and read carefully, you have been exposed to several key principles of writing XSL. Basically you are creating templates which the base $\text{\LaTeX}$ conversion will “call” in exactly the right place. In a procedural language these might be called **hooks**. Besides a smooth integration with the rest of PreTeXt there are several advantages:

- A template using a `@match` can apply narrowly or broadly. Witness the example above for `<example>` versus `&EXAMPLE-LIKE;`. This attribute is similar to a `this` pointer in an object-oriented language and the modal template is not dissimilar to a method.
- PreTeXt has a discovery phase when it constructs the $\text{\LaTeX}$ preamble. If your PreTeXt source has no `<example>` in it, then there will be no associated `tcolorbox` style added to the preamble, and the $\text{\LaTeX}$ `example` environment will not be defined. So the preamble is exactly what your document needs, and no more (mostly). The `@match` attribute makes this possible.
- You have seen named templates, which will feel familiar if you know procedural languages. While perhaps comfortable, they are way less powerful, and we noted that we may get rid of them.
- The `<xsl:import>` mechanism allows us to keep base definitions and override others. So as you develop your style, you do not need to start from scratch.

Be aware that *every single character* that you put inside the `<xsl:text>` element will get copied literally into the preamble of your $\text{\LaTeX}$ output, including newlines and spaces you use to indent in your XSL. Conversely, any *white-space* inside your template, but *between* the `<xsl:text>` elements is ignored. So we like to use multiple `<xsl:text>` elements (except we were a bit lazy getting this out the door initially—do as we say, not as we do) and explicitly create newlines with the `
` character. Your goal is to have a  $\text{\LaTeX}$  preamble with no blank lines and no unintended indentation. We often create comment lines (with two leading `%` characters to aid with readability).

See the PreTeXt website for recommendations for books on XSL if you are encouraged to learn more.

41.9 Testing and Debugging

The sample article tries to have one of everything, plus a few torture tests. It is a good place to test initially, especially with the `tcb-style` templates. The sample book is less haphazard, but does have most of the structure a typical book would have. So if you are designing for a book it is a good place to test page styles, headers and footers, and division titles. (2019-09-30: it is possible `<part>` has not been tested thoroughly enough yet.)

When things go bad, such as a non-obvious $\text{\LaTeX}$ compilation, it can help to stop working with PreTeXt source, and instead edit the generated $\text{\LaTeX}$ until the problem is understood.

We do not expect to provide great support for this process. First, because new code and basic support already keeps us busy, and second, because you have the freedom to really make a mess, and thus you should take responsibility for the problem. When you are certain that PreTeXt has done something wrong or inadvisable, please, please do post in the development forum with a careful explanation and a (minimal) example. It will happen. Just don't use the forums as a replacement for this documentation or a bit of sleuthing through the $\text{\LaTeX}$ that *you* are now creating. With freedom comes responsibility. Thanks.

41.10 Justified Text

We employ the $\text{\LaTeX}$ package, `microtype` to enhance $\text{\LaTeX}$'s algorithms for spacing text. See [Section 30.3](#) for some explanation. The package admits a great deal of fine-tuning, and we make it easy to pass in options via a template included as part of the styling.

If you were using `pdf\latex` you might include in your styling:

```
<xsl:template name="microtype-options">
  <xsl:text>protrusion=true,expansion</xsl:text>
</xsl:template>
```

This would have the effect of putting

```
\usepackage[protrusion=true,expansion]{microtype}
```

into your $\text{\LaTeX}$ output file. Study the microtype documentation for possibilities.

41.11 Planned Additions

Some items to include, as of 2019-09-30:

- Color schemes, so an author can use a style and just change the colors with a very simple stylesheet.
- Font control, so a style writer has simple instructions that navigate $\text{\LaTeX}$'s procedures.
- Flexibility with the building blocks of a title page.
- Cover design and placement.

Chapter 42

(*) Cover Design

Notes: Rationale (promotion). Procedures for print-on-demand (generally). Tools (Illustrator, GIMP, Inkscape). ISBN placement. Capable students can do design for you.

Covers can be modified for use in an electronic PDF produced from the $\LaTeX$ conversion, see [Section 30.14](#).

Chapter 43

Print-On-Demand

If you are both author and publisher, you may wish to make your book available in a physical form, but may be reluctant to purchase and store thousands of copies, or to take orders and arrange shipments. Then **print-on-demand** might be the solution for you.

A print-on-demand service is a manufacturer and distributor of printed books, which are typically only printed once ordered, or in extremely small quantities. They can provide many of the manufacturing and fulfillment services a traditional supplies. Some provide services you pay for that will produce a cover, provide editorial services, or assist with marketing.

We list three such services below, but first describe some commonalities, pro and con.

Updates	Generally, you provide a PDF of your text, and we have tried, with the print option, to make output that is amenable to this situation (see Section 30.1). A real advantage of print-on-demand is that you can usually update this PDF at any time, without much trouble. You will need to decide how to indicate versions (or printings?) of your work. Perhaps we will have tools and advice about this soon.
Covers	You may need to provide a cover, typically as a PDF meeting some exact specifications. Though you may be able to choose a fairly generic look through a template or wizard. Or pay to have one created for you. See also Section 30.14 and Chapter 42 .
Price	You may choose to sell at your cost, or you may wish to make a profit on each sale. (Note: as copyright-holder you can do this, no matter what license you have chosen, review Chapter 25). A 450-page hardcover book might be sold by a print-on-demand manufacturer to an online bookstore, including some profit for the manufacturer, for \$23. If you, as author, want \$5 profit, and the online bookstore wants \$7 for fulfillment, shipping, and profit, the cost to your reader is now \$35. In order for the online bookstore to give the appearance of discounting your book to \$35, you may need to declare a suggested retail price of \$49.95. So pricing takes a bit of thought. Or guesswork, since the discounting algorithm is not public. Note in the above scenario, the print-on-demand manufacturer may sell you, the publisher, small quantities at a better price, such as ten copies for \$170, shipping included.
ISBN	An International Standard Book Number is a unique identifier of books and necessary for others to distribute and sell your book. See details for each manufacturer below. Much like a domain name for your book's website (see Chapter 39), this may be something you wish to control and own, foregoing the convenience of somebody else providing and owning it for you.

In order of increasing professionalism and decreasing convenience, we describe three print-on-demand manu-

facturers we are familiar with, plus three others. Additions, corrections, updates, and alternatives are all welcome.

Lulu.com This site caters to people making photo books for relatives, in addition to more serious projects. Account setup may be trivial, an ISBN number may not even be needed, and you may have options for distribution beyond readers simply ordering direct from the site. This might be a good choice for drafts you will use in your own classes, if having your university bookstore print copies is not a good alternative. (2017-11-25)

Kindle Direct Publishing

Until early 2018, Kindle Direct Publishing (KDP) was a service known as CreateSpace. Some of the information below refers to this predecessor. (2021-04-21).

This company is owned by Amazon.com. They manufacture and distribute serious books, in addition to music and film. Distribution through Amazon is nearly automatic. There is also “Expanded Distribution”, which starts to look more like Ingram (next). If royalties are small, using direct deposit might be the most convenient (international sales all get converted to dollars for authors in the US). (2017-11-25)

CreateSpace attempts to make sure you have the rights to your content. So if they find your book freely available on the Internet, their “Content Validation Request Team” becomes suspicious and investigates. This has caused a few authors a few headaches and delays in making their book available for sale, though all have been successful eventually. (2018-03-06)

If you want to offer your project electronically for no cost, you may need to do an end run. We have a report that you can put your project on KDP for a low price (e.g. \$1), then offer it on Apple’s similar service for free, and then exercise a price match guarantee back on KDP. Reports on this technique encouraged. (2021-04-21)

Ingram Spark

IngramSpark (formerly Lightning Source) is a division of Ingram, which is a very large printer, also providing services to major publishers. Creating an account is not trivial, and you need to provide your own ISBN number. In return, your book is available at Amazon.com and many other online bookstores automatically, and is in many ways indistinguishable from offerings of large commercial publishers. There are also options for international distribution. You can also control settings for discounts and returns. (2018-03-06)

Current startup pricing for publishers is forty-nine dollars for either a print book, or print and e-book together. A recent change seems to be that, like KDP, they will now provide a free ISBN if you don’t have one; however, the free ISBN is only usable for as long as you continue to use IngramSpark. (2021-07-21)

LAD Custom Publishing

LAD primarily helps faculty create course packets while remaining copyright compliant, so it has experience with openly licensed educational resources if you need print copies of PreTeXt titles without other print options. In addition, the [LAD Bookstore](#) can be used to print and market your own text without overhead, though as with other solutions an ISBN is free only so long as you use LAD. One particular advantage for some authors is you can communicate with representatives on the phone or via email, not just through an automated web form. (2024-07-18)

Blurb Blurb specializes in photo books, and uses Ingram for printing. Sizes are limited, and costs are more than the other services. Direct experiences would be a welcome addition. (2018-03-06)

Nook Press Nook Press is a service of Barnes & Noble, and books appear only through their online store. We have no additional information, so direct experiences would be a welcome addition. (2018-03-06)

We currently have no good information about distributing EPUB or Kindle electronic versions for profit. (2017-11-25).

Chapter 44

Publication File Reference

This chapter is reference material for the **publication file**, whose employment is described in [Section 26.1](#), where you can also find an explanation of the shorthand syntax used here to describe elements of this file.

44.1 Common Options

These are options that might affect several conversions, though the influence may vary from one conversion to the next.

44.1.1 Common Chunking Option

Many outputs are produced as a collection of several files, others may only produce a single file. A **chunk** is a generic, informal term for a portion of an output, typically a single file. This is given as a level, see [Section 26.2](#) for more about levels. The specification is in the attribute

```
/publication/common/chunking/@level
```

with a whole number as the value (generally 0 is allowed). This is critical for the construction of HTML output, and ignored for $\text{\LaTeX}$ output.

44.1.2 Table of Contents Level Option

The depth of the entries in a Table of Contents is given as a level, see [Section 26.2](#) for more about levels. The specification is in the attribute

```
/publication/common/tableofcontents/@level
```

with a whole number as the value. A value of 0 is used to indicate that no Table of Contents is desired. Absent this setting, reasonable defaults are supplied, which can vary by conversion and document type.

44.1.3 Exercise Component Visibility

An `<exercise>` can appear in four different locations: mixed in with paragraphs and blocks (“inline”), inside an `<exercises>` specialized division (“divisional”), inside a `<worksheet>` specialized division (“worksheet”), or inside a `<reading-questions>` specialized division (“reading”). Also, a `<project>`, and similar (“project”), behaves in many ways like an `<exercise>`. All of these elements (or their `<task>`) may have `<statement>`, `<hint>`, `<answer>`, and `<solution>` (“components”).

Where such exercises first appear (not, for example, in back-of-the-book solutions) publication switches can control the visibility of the components. There are five elements, each with four attributes, which can take on the values of yes or no. The default is yes, which should be most useful in the early stages of a project. The element

`<exercise-project>` is a misnomer, but is used to avoid confusion with some other use of `project`. See [Section 26.4](#) for more explanation.

Table 44.1.1 Exercise Component Visibility Settings

Publication File Entry	Attributes: yes or no values
<code>common/exercise-inline</code>	<code>@statement</code> , <code>@hint</code> , <code>@answer</code> , <code>@solution</code>
<code>common/exercise-divisional</code>	<code>@statement</code> , <code>@hint</code> , <code>@answer</code> , <code>@solution</code>
<code>common/exercise-worksheet</code>	<code>@statement</code> , <code>@hint</code> , <code>@answer</code> , <code>@solution</code>
<code>common/exercise-reading</code>	<code>@statement</code> , <code>@hint</code> , <code>@answer</code> , <code>@solution</code>
<code>common/exercise-project</code>	<code>@statement</code> , <code>@hint</code> , <code>@answer</code> , <code>@solution</code>

44.1.4 Fillin Options

The style of fill-in-the-blanks (from a `<fillin>`) is set by the attributes

- *Text Fillin.*

Set

`/publication/common/fillin/@textstyle`

to values `underline`, `box`, or `shade` to influence the style of a fillin within text. The default is `underline`.

- *Math Fillin.*

Set

`/publication/common/fillin/@mathstyle`

to values `underline`, `box`, or `shade` to influence the style of a fillin within math. The default is `shade`.

44.1.5 Em Dash Width Options

The width of spaces on either side of an em dash (the `<mdash/>` element) is set with the

`/publication/common/@emdash-space`

attribute having values `none` (the default) or `thin`. See [Subsubsection 4.1.4.7](#) for more explanation.

44.1.6 Watermarks

The text of a watermark is given as the content of the

`/publication/common/watermark`

element. An optional scale factor may be given as a positive rational number with the

`/publication/common/watermark/@scale`

attribute. The default scale is `0.5`.

In the text itself avoid obscure characters or symbols, and do not use any markup. Keep it simple. When used with $\text{\LaTeX}$ output the characters `\\` will survive in the text to create multiple lines of text in the watermark. This feature is implemented for $\text{\LaTeX}$ and `HTML` output. See [Section 26.5](#) for more detail and an example.

44.1.7 Mermaid

Various themes are available for Mermaid diagrams. To use a theme, set

`/publication/common/mermaid/@theme`

with a value of the theme name. The default theme is `default`. See [Subsubsection 4.14.3.5](#) for more information on Mermaid diagrams.

44.1.8 QR code image

QR codes for videos and interactive elements can have an image placed in their center. This image file should be square, and stored in the external assets folder. To use this image in your QR codes, use

```
/publication/common/qrcode/@image
```

with a value of the file path, relative to external assets folder.

Note that the presence of such an image will increase the raw size of QR codes, which are then generally shrunk to fit. This will result in a finer grain for the QR code pixels.

44.1.9 Journal name

When an article is intended for publication in a particular journal, the name of the journal can be specified with

```
/publication/common/journal/@name
```

with a value being one of a list of supported journal short name codes. Setting this option will control the format of bibliography entries and the format of $\LaTeX$ that can be produced.

A complete list of the supported journals, together with their short name codes, will be available in [Appendix Q](#).

44.1.10 Worksheet and Handout Margins

Margins for worksheets and handouts can be specified using a combination of attributes on the element

```
/publication/common/worksheet
```

using attributes @margin, @top, @right, @bottom, and @left. The values of these attributes should include units, either in or cm. Setting @margin will set all four margins to its value, but this can be overridden by setting (some of) the other attributes. The default margin is 0.75in.

For example, setting `<worksheet margin="1in" left="2in"/>` will set the top, right, and bottom margins to 1in and the left margin to 2in. Setting `<worksheet top="1cm" bottom="2cm"/>` will set the top margin to 1cm and the bottom margin to 2cm, while leaving the left and right margins at their default of 0.75in.

These settings will be used for both the $\LaTeX$ and HTML conversions. See also [Subsection 44.3.9](#) for more information on how these margins are used in the $\LaTeX$ conversion.

44.1.11 Worksheet and Handout Headers/Footers

Headers and footers for worksheets and handouts can be specified in the publication file (they can also be edited in the browser when producing HTML output). First-page and running headers and footers are specified separately, and each can have a value for left, center, and right positions.

That is, each of the elements

```
/publication/common/worksheet/first-page-header
/publication/common/worksheet/first-page-footer
/publication/common/worksheet/running-header
/publication/common/worksheet/running-footer
```

can have attributes @left, @center, and @right, whose values are the desired content for that position in the header or footer.

For example, to add a name and date line to the top of the first page, one would include the publication file entry

```
<first-page-header left="Name: _____" right="Date: _____"/>
```

(inside the `<worksheet>` element of `<common>`).

44.2 Numbering Options

How various items get numbered can be controlled by specifying levels. For pieces of content (e.g. `<example>`, `<fn>`, ...) the level will be the number of separators in the number, usually periods. To fully understand these options, carefully read [Section 26.2](#) and [Section 26.3](#).

You can stop numbering divisions at some depth. For example, you may want your `<book>` (without parts) to have numbered `<subsection>`, but numbers on the few `<subsubsection>` is just too messy and not necessary. Then you would set the publication file entry

```
/publication/numbering/divisions/@level
```

to the value 3 to number `<chapter>`, `<section>`, and `<subsection>`, only. (Note this is the exception, you will get at most *two* separators in a division number in this case.) So you can think of this as the *maximum* level for numbers on divisions. Just be careful not to try to number other objects (described next) to a greater level, as that is impossible, and so will generate a warning and the default will be substituted.

For items within a division, various groups of objects are numbered consecutively, with a hierarchy given by a level. These numbers are controlled by

```
/publication/numbering/blocks/@level
/publication/numbering/projects/@level
/publication/numbering/equations/@level
/publication/numbering/footnotes/@level
```

whose values should be integers (zero is a possibility) that do not exceed the maximum level specified for divisions.

Chapters of a `<book>` may start numbering from something other than one. This feature is not available in a book with structural parts (rather than decorative parts). Read about this at the end of [Section 26.3](#) and [Best Practice 26.3.1](#). If you still want to proceed, then set the publication file entry

```
/publication/numbering/divisions/@chapter-start
```

to the desired value of the first chapter.

A `<book>` with `<part>` may have the parts numbered to reflect two different structures, **decorative** or **structural**. Set the publication file entry

```
/publication/numbering/divisions/@part-structure
```

to decorative (the default) or structural.

When a `<book>` is designed to have structural parts, then there is an expectation that the part divisions are important or relevant (at least moreso than in the case of decorative parts). Then, to choose to number objects (blocks, equations, etc.) at level 0, crossing part boundaries, strikes us as an odd choice. So odd that there is a [small bug at issue #1650](#), which might be solved by simply banning this combination.

For more on divisions, and their numbering see [Section 4.6](#).

44.3 PDF (L^AT_EX) Options

These options affect the conversion to L^AT_EX, which can in turn be converted into PDF. See [Chapter 30](#) for a more general overview of this conversion with more details. This includes some existing options that will eventually migrate to the publication file.

44.3.1 L^AT_EX Style Option

A limited number of custom L^AT_EX styles are available and more are in development (see [Chapter 41](#)). The

```
/publication/latex/@latex-style
```

attribute can be set to one of the provided styles to build the L^AT_EX or PDF output with that style. As of 2024-10-04, the supported styles (and values of this attribute) are AIM, chaos, CLP, dyslexic-font, and guide. (Note that some of these are meant as demos and not intended for production.)

44.3.2 L^AT_EX Print Option

The conversion to L^AT_EX can produce a PDF optimized for print, or optimized for use electronically on a screen. The

```
/publication/latex/@print
```

attribute can have the value `yes` to produce a print version, or the value `no` to produce an electronic version. The default is an electronic version. See [Section 30.1](#) for more detail. Note that the use of page numbers in cross-references has different default behavior based on this option (which can be overridden, see [Subsection 44.3.7](#)).

44.3.3 L^AT_EX Sides Option

The conversion to L^AT_EX can produce a PDF designed for printing on only one side of the page, or on both sides of the page. The

```
/publication/latex/@sides
```

attribute can have the value `one` for a one-sided version, or the value `two` to produce a two-sided version. The default depends on if the output is electronic (one-sided) or print (two-sided). See [Section 30.4](#) for more detail.

44.3.4 L^AT_EX Page Matching

A two-sided PDF will contain extra blank pages to ensure that certain divisions (usually parts and chapters) always open on an odd-numbered page. Since a one-sided PDF will omit these extra blank pages, the page numbers may get out of sync between the two versions. The

```
/publication/latex/@open-odd
```

attribute can be used to force the page numbers back into sync by either also inserting the extra blank pages in a one-side PDF version (by setting the attribute to value `add-blanks`) or by simply skipping the page number for pages that would have been blank in the pagination (by setting the attribute to value `skip-pages`). If the attribute value is `no` for a one-sided PDF output (the default), then no adjustment is made and divisions open on the next consecutive page whether odd- or even-numbered.

Note that this attribute is ignored for two-sided PDF output, where extra blank pages are always inserted. Also note that page-matching between one- and two-sided PDF versions may require a consistent specification of the `/publication/latex/@pageref` option ([Subsection 44.3.7](#)). See [Section 30.10](#) for a more thorough discussion of page number fidelity.

44.3.5 L^AT_EX Font Sizes

The document-wide point size can be specified with the

```
/publication/latex/@font-size
```

attribute can have the value 8, 9, 10, 11, 12, 14, 17, 20, which are interpreted in points (“pt”) as the unit of measure. The default value is 10. See [Section 30.5](#) for details.

44.3.6 L^AT_EX Page Shape Options

The following options affect the “shape” of each page, so are in a way less “global” than some of the previous options. These are both attributes of the

```
/publication/latex/page
```

element.

- *Text Alignment, Right Edge.*

Set

/publication/latex/page/@right-alignment

to values flush (default) or ragged to influence the right edge of the block of text on a page. See [Section 30.3](#) for more.

- *Text Alignment, Bottom Edge.*

Set

/publication/latex/page/@bottom-alignment

to values flush or ragged (default) to influence the bottom edge of the block of text on a page. See [Section 30.3](#) for more.

The content of the element

/publication/latex/page/geometry

element will feed directly into a L^AT_EX `\geometry{}` element, with no modifications. (In other words, all whitespace, such as newlines and indentation, will be preserved.) Be sure to carefully read the advice, an example, and further detail at [Section 30.6](#).

The attribute

/publication/latex/page/@crop-marks

may be set to the value none, or not specified at all, and nothing will happen. Otherwise, it is set to a paper size, then the logical page (whose size is presumably set by the geometry package) will be centered on the physical page specified, with marks at each corner delineating the logical page. Some publishers request camera-ready copy to have these indications. Paper sizes are arguments to the crop package, and as of 2023-05-19 the possible values were:

a0, a1, a2, a3, a4, a5, a6,
b0, b1, b2, b3, b4, b5, b6,
letter, legal, executive

There is no error-checking of these values. See [Section 30.6](#) for more and examples.

44.3.7 Page Numbers in Cross-References

Set

/publication/latex/@pageref

to values yes or no to enable or disable the use of page numbers in cross-references (typically achieved with the `<xref>` element). The default varies, as it is dependent on the print option ([Subsection 44.3.2](#)). See [Section 30.9](#) for more on the defaults.

44.3.8 Hyperlink Highlighting

Set

/publication/latex/@link-highlight

to underline to draw an underline beneath hyperlinks (`<xref>` and `<url>`) in the PDF output. The default value is none. This option requires no additional L^AT_EX packages. See [Section 4.41](#) for accessibility motivation.

44.3.9 L^AT_EX Worksheet Options

By default, worksheets are formatted differently than other pages, including customizable margins, workspace between exercises, and on their own pages. This separate formatting can be ignored, causing worksheets to be treated like any other division, using the

/publication/latex/worksheet/@formatted

attribute, setting the value to no. The default value is yes.

See also [Subsection 44.1.10](#) for how to specify margins for formatted worksheets.

44.3.10 Covers

The

```
/publication/latex/cover
```

element has attributes `@front` and `@back` whose values can be paths to PDF files to use as the first and last page of the output PDF. The paths should be relative to your directory of external files, much like a raster image you might bring to your project being given by a `@source` attribute. See [Section 5.6](#) for more about managed directories, and [Section 30.14](#) for more detail about covers.

44.3.11 L^AT_EX Asymptote Links

The conversion to L^AT_EX can provide a link to an HTML version of each Asymptote graphic. The

```
/publication/latex/asymptote/@links
```

attribute can have the value `yes` to produce links, or the value `no` to not create links. Note that a base URL must be set for this feature to be functional ([Subsection 29.1.6](#)). The default is `no`. See [Section 30.8](#) for more detail.

44.3.12 Draft Mode

Set

```
/publication/latex/@draft
```

to values `yes` or `no` to enable or disable the use of the L^AT_EX draft mode. The default is `no`. See [Section 5.9](#) for more detail.

44.3.13 Snapshot Record

Set

```
/publication/latex/@snapshot
```

to values `yes` or `no` to enable or disable the generation of a “snapshot” record of the LaTeX packages in use. The default is `no`. See [Section 30.19](#) for more detail.

44.3.14 Page Breaks

A publisher can declare `@xml:id` values for which a pagebreak (`\newpage`) will precede the corresponding element in output. This is only valid for certain element types. Use

```
/publication/latex/insertions/@pagebreaks
```

to provide a whitespace-separated list of `@xml:id` values. You may use multiple `<insertions>` elements, each with its own `@pagebreaks` to help organize pagebreaks by chapter, etc.

See [Section 30.7](#).

44.4 Online (HTML) Options

These options affect the base conversion to web pages (online, HTML). Many, but not all, will affect subsequent conversions based on HTML, such as a conversion to EPUB or Jupyter notebooks. See [Chapter 29](#) for a more general overview of this conversion, including options that will eventually migrate here.

44.4.1 HTML Analytics

The

```
/publication/html/analytics
```

element can have the following attributes:

- @google-gst: a Google **global site tag**, which is an ID you get from Google. *Do not copy the identification numbers from another project—be sure to obtain your own for your project.*
- @statcounter-project, @statcounter-security: ID numbers you get from StatCounter. *Do not copy the identification numbers from another project—be sure to obtain your own for your project.*

Setting these attributes to non-empty strings is the signal to add the relevant code to each of the pages of your HTML output. See [Section 29.7](#) for more.

44.4.2 HTML Base URL

The

```
/publication/html/baseurl/@href
```

attribute may be given as a complete URL for the top-level of where HTML output is hosted. Of course, this may be different for different publishers. The value is a directory, and so should end with a slash (path separator). See [Subsection 29.1.6](#) for reasons why you might want to specify this.

44.4.3 HTML Banner Options

The

```
/publication/html/banner/@subtitle
```

and

```
/publication/html/banner/@byline
```

attributes may be specified as either yes (the default) or no to control whether the subtitle and author byline are displayed in the banner. See [Subsection 29.1.2](#) for reasons why you might want to specify this.

44.4.4 HTML Embedded Calculator

The

```
/publication/html/calculator
```

element has the following attribute:

- @model: used to control which calculator is available on every page. Possible values are:
 - geogebra-classic
 - geogebra-graphing
 - geogebra-geometry
 - geogebra-3d
 - none

The default is none. See [Subsection 29.1.3](#) for more.

44.4.5 HTML Favicon

The

```
/publication/html/
```

element has the following attribute:

- @favicon: used to control how icon files for a favicon are embedded into each HTML page:
 - none
 - simple

The default is none. See [Subsection 29.1.4](#) for more, including an explanation of what a favicon is.

44.4.6 HTML WeBWork Dynamism

The

```
/publication/html/webwork
```

element has the following attributes, each of which can have a value of static or dynamic:

- @inline, default value: dynamic
- @divisional, default value: static
- @reading, default value: static
- @worksheet, default value: static
- @project, default value: dynamic

The attribute names suggest the type of exercise or project-like that will be affected. See [Subsection 38.5.2](#) for more details and the rationale for the defaults.

44.4.7 HTML ActiveCode Programming Window

The

```
/publication/html/calculator
```

element has the following attribute:

- @activecode: used to control if an ActiveCode window is available on every page, and if so, which language it understands. Possible values are:
 - python (Python)
 - javascript (JavaScript)
 - html (HTML)
 - sql (SQL)
 - c (C, Runestone server only)
 - cpp (C++, Runestone server only)
 - java (Java, Runestone server only)
 - kotlin (Kotlin, Runestone server only)
 - python3 (Python 3, Runestone server only)
 - octave (Octave, Runestone server only)
 - none

For a build hosted at a Runestone server, python is the default. For an HTML build hosted elsewhere, none is the default. See [Subsection 29.1.5](#) for more.

44.4.8 HTML Index Page

The

/publication/html/index-page

element can have the following attribute:

- @ref: the @xml:id of a division which will be a complete page at whatever level the document is chunked (broken into smaller pages).

An index.html page will be created which redirects immediately to this page. Many web servers will serve this page when a URL stops with the enclosing directory. So the Table of Contents (<book>, <article>) or the <frontmatter> are common choices. See [Subsection 29.1.1](#) for more.

44.4.9 HTML Knowlization

In a conversion to HTML a wide variety of content can be “born” in a **knowl**. Publisher switches are specified via multiple attributes of a single

/publication/html/knowl

element, with values of yes (do use a knowl) or no (do not use a knowl, present content normally). For example,

```
<knowl remark="yes"/>
```

would make every “remark-like” element occur as a knowl where the content is first introduced. Note that these are distinctly different than cross-reference knows. For more detail see [Section 29.2](#).

This table gives the various attribute names, the default value, and an indication of the elements affected. Note that some items are automatically born knowled (e.g. footnotes) and so there is no option to control that behavior. Also, items such as <figure>, when included in a <sidebyside> are not influenced by the relevant option and are not born knowled. There are four flavors of <exercise> and so four options. A <hint>, <answer>, or <solution> is automatically knowled as a plot-spoiler where the author is expecting the reader to do something. In contrast, an “example-like” is expository, so the knowlization is configurable.

Table 44.4.1 Knowlization options

Attribute	Default	Element(s)
theorem	no	<theorem>, <lemma>, ...
proof	yes	<proof>
definition	no	<definition>
example	yes	<example>, <problem>, ...
example-solution	yes	<hint>, <answer>, <solution>
project	no	<project>, <activity>, ...
task	no	<task>
remark	no	<remark>, <note>, ...
objectives	no	<objectives>
outcomes	no	<outcomes>
figure	no	<figure>
table	no	<table>
listing	no	<listing>
list	no	<list>
exercise-inline	yes	within narrative
exercise-divisional	no	inside <exercises> division
exercise-worksheet	no	inside <worksheet> division
exercise-readingquestion	no	inside <readingquestions> division

44.4.10 HTML Tabbed viewer

The

```
/publication/html/exercises
```

element can have a `@tabbed-tasks` attribute. The value is a space-separated, or comma-separated, list of types of `<exercise>` or PROJECT-LIKE. The values in this list come specify up to four of the five possible types of exercises: `divisional`, `inline`, `reading`, `project`. Note that exercises inside a `<worksheet>` are explicitly ineligible. See [Section 3.9](#) for careful descriptions of these types.

44.4.11 HTML Platforms

The

```
/publication/html/platform
```

element can have an attribute `@host` with values:

- `web`: the default, meant for self-hosting with no server configuration, features, or assumptions
- `runestone`: output meant for hosting on a Runestone server ([Chapter 32](#))

Here **platform** refers to the server where the HTML output will eventually be hosted. The effect is to create minor variations in the output to take advantage of extra features of the indicated platform.

44.4.12 Portable HTML

To limit the number of files and directories created by the HTML conversion, you can set the

```
/publication/html/platform/@portable
```

attribute to the value "yes". This will result in the HTML using hosted CSS and Javascript files, rather than including them adjacent to the rest of your output. This is especially useful in conjunction with setting the [chunking variable 44.1.1](#) to 0 to get a single html file.

44.4.13 HTML Style (Theme)

The

```
/publication/html/css
```

element can have a `@theme` to specify the theme used to style the pages. See [Section 29.6](#) for examples of what different themes look like. Depending on the theme chosen, there may be other attributes that can be set. See [List 44.4.2](#) below for details.

The "legacy" stylesheets are still available. Any publication file that references `html/css/@style` will use the corresponding legacy style sheet (`default`, `crc`, `wide`, `oscar-levin`) as well as the colors selected for it. However, the full range of configuration is no longer available. If your publisher file mixes legacy style parts in an unexpected way (`toc="crc"` but `sidebar="wide"`) you will get one or the other complete style.

New themes other than "default-modern" require a build step involving `node.js`. If `node` is not installed (try `node --version` to check), following the directions in [Appendix F](#). If you are using the `pretext/pretext` script, you will need to, one time, run `npm install` in the `script/cssbuilder` directory (this step is done automatically if you are using the CLI). After this initial setup, the build script will automatically be invoked when you build a document.

To rapidly test out different themes and options while using the CLI (see [Subsection 5.2.3](#)) you can use `pretext build --theme` to rebuild the theme without a full book rebuild. If you are using the `pretext` script (see [Chapter 47](#)) you can use the flag `-c theme`. Note that doing this assumes the HTML needed for the different themes is the same. This should generally be the case, but if you encounter odd rendering issues you should do a full rebuild.

List 44.4.2 Theme attribute options

default-modern	The following attributes can be set on CSS to modify the appearance of default-modern: • <code>@palette</code> controls the colors used. Options include: "blue-red", "blue-green", "green-blue", "greens", and "blues". • <code>@primary-color</code> override the primary color set by the palette. This, and all other color attributes, can be in CSS color format (hex, rgb, hsl etc.). • <code>@secondary-color</code> override the secondary color set by the palette. • <code>@provide-dark-mode</code> can be set to "no" to disable dark mode on the theme. This may be desirable if the work has significant elements that do not render correctly with a dark background and you are not able to modify them to do so. • <code>@primary-color-dark</code> override the primary color used in dark mode.
tacoma, greeley, boulder	The following attributes can be set on CSS to modify the appearance of tacoma, greeley, and boulder. They function in the same way as the corresponding attributes for default-modern. • <code>@primary-color</code> • <code>@primary-color-dark</code> • <code>@provide-dark-mode</code>
salem, denver	The following attributes can be set on CSS to modify the appearance of salem and denver. • <code>@palette</code> controls the colors used. Options include: "bold", "bold2", "primaries", "primaries2", "ice-fire", "earth-sea", "leaves", and "slate". • <code>@color-main</code> override the primary color set by the palette. • <code>@color-do</code> override the color used for elements that ask the reader to do something (e.g. an exercise). • <code>@color-fact</code> override the color used for elements that present a fact (e.g. a theorem). • <code>@color-meta</code> override the color used for elements that provide meta information to the reader (e.g. goals, remarks). • <code>@provide-dark-mode</code> - see default-modern. • <code>@primary-color-dark</code> - see default-modern.

Finally, if you wish to do development on a custom theme using SCSS, you can set `/publication/html/css/@theme` to "custom". If you do so, you need to also set `@entry-point` to be the path to an SCSS file that will be used to build the theme. This path should be relative to the xml file that is your document root.

44.4.14 HTML Search

The

/publication/html/search

element can have the following attributes:

- @variant: a string specifying how search queries are handled. Possible values are textbook, reference, default and none. For historical reasons, using the value default is synonymous with using textbook, so it is not necessary. If you do not specify this entry, then that is equivalent to setting the value to textbook. This is search provided natively.
- @google-cx: a Google **cx number**, gained from configuring search for a site. Setting this attribute to a non-empty string is the signal to add the relevant code for a search box in the masthead.

Note that if you elect both native search and Google search the two search boxes will overlay each other and one will not be usable. See [Section 29.8](#) and [Section 29.9](#) (respectively) for more.

44.4.15 HTML Video Embedding

The

/publication/html/video

element can have the following attribute:

- @privacy: allowed values are yes or no.

Setting this to yes (the default) prevents certain tracking cookies from being used. Currently only supported for videos from YouTube. See [Subsection 29.1.7](#) for more.

44.4.16 Short Answer Responses

The

/publication/html

element can have an attribute @short-answer-responses with values:

- graded: the default, only show an area for responses when they can be graded or scored (such as when hosted on Runestone, [Chapter 32](#)).
- always: include an editable response area, even if it cannot be submitted for feedback.

See [Subsection 4.12.9](#) for details.

44.4.17 L^AT_EX Asymptote “Click to Enlarge” Links

The conversion to HTML can provide a “Link to full-sized image” link below each Asymptote graphic. The

/publication/html/asymptote/@links

attribute can have the value yes to produce links, or the value no to not create links. Note that a base URL must be set for this feature to be functional ([Subsection 29.1.6](#)). The default is no. See [Subsection 29.1.8](#) for more detail.

44.4.18 HTML Feedback Button

The

/publication/html/feedback

element requires an @href attribute that is a complete URL. The content of the element is optional, and will provide alternate text for the button. This alternate text should have no markup—just text. See [Subsection 29.1.9](#) for explanations, details, and a caveat.

44.4.19 HTML Navigation Options

The

`/publication/html/navigation`

element can have the following attributes:

- `@logic` with values `linear` (default) and `tree`.
- `@upbutton` with values `yes` (default) and `no`.

See [Subsection 29.1.10](#) for explanations and details.

44.4.20 HTML Table of Contents Options

The

`/publication/html/tableofcontents`

element can have the following attributes:

- `@focused` with values `no` (default) and `yes`. Enabling a focused TOC makes the TOC expandable/collapsible and initially hides TOC items not on the path to the active webpage.
- `@preexpanded-levels` with values 1–6 (0 is the default). This value controls how many levels from the root of the TOC are initially expanded outside of the path to the current page. Either 0 or 1 will result in the root level of the TOC being expanded (it is on the path to the active page and thus always is expanded). 2 will result in the first two levels of the TOC being visible (e.g. parts and chapters or chapters and sections). 3 would result in three initially visible levels (e.g. parts/chapters/sections), etc...

See [Subsection 29.1.10](#) for explanations and details.

44.4.21 xref Options

The

`/publication/html/cross-references`

element can have the following attributes:

- `@knowled` with values `maximum` (default), `never` and `cross-page`. `maximum` will render xref's to structural elements as HTML links and xref's to smaller elements as knows. `never` forces all xref's to be rendered as traditional HTML links. `cross-page` bases the rendering decision on if the xref and its target are on the same page. If so, the xref is always rendered as an HTML link. If the link and target are on different pages, the behavior is the same as `maximum`.

See [Section 29.2](#) for explanations and details.

44.4.22 Embed Page Button

The

`/publication/html`

element can have an attribute `@embed-button` with values:

- `no`: the default, do not display button.
- `yes`: show a button in the navigation bar, that when clicked, displays a popup allowing the user to copy an iframe code snippet to paste into their website or LMS.

44.4.23 Interactives Options

The

```
/publication/html
```

element can have an attribute `@design-width` with an integer value. This will be used as the base value for converting the percentage based widths of an interactive into a pixel values when that is required. The default value is 600 (100% width is 600px).

The

```
/publication/html/interactives
```

element can have an attribute `@resize-behavior` that is used to control how interactives respond to window resizing. Possible values are:

- `fixed-height` (default): interactives will maintain their initial height as the window size changes. When the content becomes too wide to display, a horizontal scrollbar will appear.
- `responsive`: interactives will resize dynamically as the window size changes to maintain their aspect ratio.

In addition to this global setting, you can add child elements to

```
/publication/html/interactives
```

to set the resize behavior for specific types of interactives. Each child element should be named after the interactive type (e.g. `geogebra` or `desmos`) and have a `@resize-behavior` attribute with the same possible values as above. Settings on these child elements will override the global setting for that specific interactive type. Known interactive types that can be specified this way are:

- `calcplot3d`
- `circuitjs`
- `d3`
- `desmos`
- `doenetml`
- `geogebra`
- `iframe`
- `javascript`
- `jsxgraph`
- `sage`

44.5 Reveal.js Slideshow Options

See [Section 34.1](#) for a more general overview of this conversion.

44.5.1 Reveal.js Appearance

The

```
/publication/revealjs/appearance
```

element can have the following attributes:

- `@theme`: the base name of a file that is a reveal.js theme. (See the [reveal.js documentation](#) for available themes.) For example if the desired theme/CSS file is `css/theme/solarized.css`, then set the value of this attribute to `solarized`.
- `@custom-css`: a string separated list of custom css files to load. These may be local files or remote URLs. For local files, it is recommended to place them in the **external** files directory so that they are automatically copied from your source folder to the output directory. (Refer to [Section 5.6](#) for more information on external files.)

44.5.2 Reveal.js Controls

The

```
/publication/revealjs/controls
```

element can have the following attributes:

- `@backarrow`: for the visibility of a “back” arrow. Values are `faded`, `hidden`, or `visible`. The default is `faded`.
- `@display`: for the overall visibility of controls. Values are `yes` or `no`. The default is `yes`.
- `@layout`: for the location of the controls. Values are `edges` or `bottom-right`. The default is `bottom-right`.
- `@tutorial`: for animation of suggested navigation arrows. Values are `yes` or `no`. The default is `yes`.

44.5.3 Reveal.js Navigation

The

```
/publication/revealjs/navigation
```

element can have the following attribute:

- `@mode`: for the arrangement of the navigation through slides. Values are `default`, `linear`, or `grid`. The default is `default`. The value `default` refers to the default mode of Reveal.js and is synonymous with `grid`. In other words, the choice of `default` or `grid` results in identical behavior.

See [Section 34.1](#) for more explanation of the two different navigation modes.

44.5.4 Reveal.js Resources

You may wish to host the reveal.js CSS and javascript files locally or obtain them online. Be sure to read in [Section 34.1](#) about the necessity of providing the complete MathJax library [Section 34.1](#) if your document has mathematics and you opt for local resources.

The

```
/publication/revealjs/resources
```

element can have the following attributes:

- `@host`: The values `local` and `cdn` will retrieve various Javascript and CSS files (such as the theme) relative to the slideshow HTML file or from an online Content Delivery Network (respectively). PreTeXt will maintain a link to a current, updated, CDN version and this will change without notice. This attribute may be given a URL or a local path, though this has not been tested, so should be considered experimental.

44.6 EPUB Options

These options are specific to a conversion to EPUB. But because an EPUB file is a compressed archive of HTML files, many of the options in [Section 44.4](#) can have an effect. See [Chapter 29](#) for a more general overview of the conversion to HTML.

44.6.1 EPUB Cover Image

The

```
/publication/epub/cover/@front
```

attribute has a value that is a path, relative to the external directory of the managed directories scheme (see [Section 5.6](#)), that is an image to be used as a cover for the EPUB. See [Chapter 31](#) for more.

44.7 Source Options

44.7.1 Directory Management

Two directories of additional files need to be specified, as paths relative to the main PreTeXt source file, in order that they can be managed automatically for the construction of various output formats. Study [Section 5.6](#) carefully for the exact details. To set these directories, the

```
/publication/source/directories
```

element must have the following two attributes:

- @external: a directory of files produced independently of your project.
- @generated: a directory of files produced automatically via PreTeXt tools, from aspects of your PreTeXt source.

It is an error to only specify one of the two directories. It is all or nothing.

44.7.2 Versions

Different versions of your source can be constructed by marking elements as belonging to different components. Each component has a name of your choosing, and then you elect components to include or exclude via the publication file. See [Section 27.2](#) for the exact details and examples.

To specify components to include, the

```
/publication/source/version/@include
```

attribute should be a space-separated list of component names. For example, if <version> has

```
include="videos labs"
```

then a version would be created that includes all of the videos and all of the labs (since the author placed them into these components), but would not contain any elements whose component is genome. Note that videos, labs, and genome are names peculiar to the organization of your project, and are not PreTeXt element names (even if they are used consistently with similar element names, such as <video>).

44.7.3 Extras

Some source material is optional, and may be included based on the publisher and the audience. The values of the attributes below are filenames. They could be absolute paths (which is not very portable, and so likely a bad idea), or relative paths. In the latter case, they are relative to the source file that contains the <pretext> element (the “main” file if you have modularized your source). Temporarily using an absolute path can be useful when you are having problems with any of these files and want to eliminate one variable while debugging.

The

/publication/source

element can have the following attributes:

- @customizations: A filename for a structured file of <custom> elements, each with a @name attribute and content meant to be substituted into source. See [Section 27.1](#) for details about the use of customizations.
- @private-solutions: A filename for a structured file of <hint>, <answer>, <solution> that will see limited distribution (for example, instructors only). See [Section 37.2](#) for details about the use of a private solutions file.

44.8 WeBWorK Options

WeBWorK exercises in a PreTeXt project can be processed in two ways: using a local copy of pg, or using a webwork2 server. The variables here control which method is used where, and how it is used. These are also discussed in [Chapter 38](#).

See [Subsection 44.4.6](#) for publisher file attributes that control WeBWorK's dynamic behavior in HTML.

44.8.1 Static Processing

The attribute

/publication/webwork/@static-processing

can have value webwork2 (the default) or local. When WeBWorK exercises are processed to make static PreTeXt representations, this is the method that will be used. Either communicating with a webwork2 server or a local copy of pg. The fastest processing happens if you use local. See [Section 38.1](#).

44.8.2 PG Location

The attribute

/publication/webwork/@pg-location

gives the absolute file path to a local copy of the pg repository, for when static processing is local. The default value is /opt/webwork/pg. See [Section 38.1](#).

44.8.3 Server

The URL to the webwork2 server is in the attribute

/publication/webwork/@server

It should include the protocol (e.g. http or https) and not include a trailing slash. The server should be version 2.16 or later. The default value is https://webwork-ptx.aimath.org. See [Subsection 38.5.1](#).

44.8.4 Course

The name of the host course on the webwork2 server is in the attribute

/publication/webwork/@course

The default value is anonymous. See [Subsection 38.5.1](#).

44.8.5 User

The username that signs in to the host course to process exercises is in the attribute

/publication/webwork/@user

The default value is anonymous. See [Subsection 38.5.1](#).

44.8.6 Password

A password (for the host user in the host course) is in the attribute

`/publication/webwork/@password`

The default value is anonymous. See [Subsection 38.5.1](#). (If the webwork2 server is version 2.18 or earlier, instead of @password, there should be both @coursepassword and @userpassword.)

44.8.7 Task Reveal

The attribute

`/publication/webwork/@task-reveal`

is used to control how tasks in a multi-task exercise are made available to the reader. This applies to live interactive renderings of the exercises, either in HTML, in WeBWorK, or otherwise. Possible values are:

- preceding-correct (all preceding tasks must be correctly answered before the next is available)
- all (all tasks are available)

See [Subsection 38.5.2](#).

Chapter 45

Publisher FAQ: Frequently Asked Questions

This is a list of answers to frequent questions, in no particular order.

1. *Why does the conversion to HTML use a fixed width for the text?*

There is an optimal number of characters per line for human readers, based on research and centuries of book design. So we set a fixed width such that the default font comes close to achieving this optimal value. We also use responsive design to accomodate the constraints of a small screen as best as possible. A reader will not want to have to carefully resize a browser window to achieve the optimal width, nor should a line of text spread to many, many characters across a very expansive screen. See [Principle 4](#).

2. *I do not want my examples in knowls (nor my inline exercises, nor proofs).*

You can change that! See [Section 29.2](#).

3. *Knowls are fantastic! Can I have more?*

Start at [Section 29.2](#) and follow a cross-reference to details on the various options.

4. *Why are my knowls empty?*

When viewing the HTML version on your laptop or local computer as files, do not expect knowls to render properly. This is a known bug/feature, and there is nothing to be done about it, unless you run a web server on your own machine, which fortunately is a very easy thing to do (see [Section 5.11](#) about testing HTML output locally). Think of it this way: the knowl content comes from a server, but on your laptop there is no web server. You are just looking at files.

A possible added confusion is that some knowls, such as proofs, will appear. That is because their content is embedded in the page, not taken from a file.

5. *How can I change the colors in the HTML version?*

Start at [Subsection 44.4.13](#) to learn about how to specify alternate styles for HTML output, including an easy way to specify an existing alternate color file.

6. *Something looks wrong in the HTML output. How can I customize the layout of the HTML version?*

If there are some anomalies in the HTML version of your book, probably that was just an oversight and can be fixed easily. Send a message to pretext-support@googlegroups.com describing the problem and *including a live link to the page showing the error*. Do not make a minimal example. (CSS issues are handled in a completely different way than other software issues.)

The long-term plan is to have a variety of different layout options, which can be chosen as easily as choosing a color scheme. See [Principle 7](#). The first step is to rewrite the current CSS so that it is easy to develop alternate layouts. That should be done by the end of Spring, 2018. Then people can develop new styles! Until that happens, either suffer with the current style, or hack away at your own peril. (Note that the PreTeXt support groups will not provide any help with hacking the layout, but a lot of help will be available when it is time to develop alternate layouts.)

7. *Why does the HTML output load so many external resources?*

The subtext perhaps being, “Why shouldn’t I host these on my own server?” A main goal for PreTeXt is to spare authors the headaches of learning new technologies just so they can get their content in front of readers. That knowledge should be built into software, so an author can work at a higher level, explaining the intricacies of their discipline. So we only assume an author can place locally-built HTML output onto some public server they have permission to use. Any extra enabling technology we do not want to create ourselves gets pulled from other public servers. MathJax, both code and fonts, is a good example, as one of the enabling projects. Perhaps it is *the* enabling project.

This way,

- Authors can concentrate on their writing, not updating services on their server.
- Servers that are hostile to ad-hoc configurations (think “learning management systems”) are not an impediment to hosting projects.
- For the most part, updates to external resources happen automatically. This allows authors and PreTeXt developers to concentrate on other aspects of their work.

We get MathJax from a **content delivery network** (CDN). Once we have that dependency, then fonts and search from Google, CSS and Javascript from the American Institute of Mathematics, and other components, all have the same dependency: a decent internet connection. Our experience over several years is that these resources have good uptimes and good bandwidth, and so are not a source of problems. A good offline version, with resources packaged via a script, would be a good long-term project.

Finally, we do not load minor resources indiscriminately. Something in your source should suggest they are necessary and we perform those checks, document-wide. However, since a cross-reference is usually implemented as a knowl, and we cannot be sure what a knowl might contain, we do tend to load resources on *every* page, even if only needed once. We hope to improve this situation. And you are encouraged to help if you have technical skills in these areas.

8. *Searching my PDF output is broken.*

PreTeXt goes to great lengths to make a high-quality PDF, but if you manipulate it by adding in new pages, or adjust the intermediate $\text{\LaTeX}$ to use other fonts, you run the risk of breaking some of the features.

A ligature is a combination of two characters into one, like a lower-case “f” followed closely by a lower-case “i” without a dot. These can confuse a search. Verbatim text sometimes ends up with “smart” quotes, where left and right versions are inverted. This frustrates copying source code into an actual program. And so on. If you see problems like this with un-customized PDF output, we would like to hear about it.

9. *My LMS breaks my HTML.*

Suppose your HTML output seems to work fine, but once you place it inside your university’s Learning Management System (LMS) it no longer works very well.

A likely culprit is that your LMS is adding material to your files, ruining them in the process.

10. *I get a bad line break in HTML with a hyphen between text and mathematics.*

When you author something like `<m>x</m>-axis` the code produced by MathJax may allow a web browser to break a line just before the hyphen. Yes, this looks very bad. No, it is not a problem PreTeXt can easily solve. So we will wait for MathJax and web browsers to do a better job. You might let us know when this happens?

11. *My publication file (or other auxiliary file) does not seem to be effective.*

Does the filename, or any other directory *in the complete path to the file*, have a space in it? A common culprit might be My Documents on Windows. If so, the failure is likely our fault. Doing a test where you move your project someplace clear of filenames or directory names with spaces would be a big help to us. And then a careful report of the offending situation will let us make a fix for the next author or publisher. Thank-you.

12. *Should I edit the $\text{\LaTeX}$ file created to make a PDF?*

See [Section 30.18](#) and especially the contained [Best Practice 30.18.1](#).

Part V

Developer's Guide

Chapter 46

Processing with xsltproc

The executable program `xsltproc` implements Version 1.0 of the **eXtensible Stylesheet Language (XSL)** . This is a declarative language that walks the hierarchical tree of an XML source file, and for each element describes some output to produce before, and after, recursively processing the contained elements. (That is a simplified description.)

`xsltproc` is typically installed by default on Linux systems and as part of Mac OS. See the PreTeXt website for details for Windows systems. The most basic operation is to provide `xsltproc` with an XSL stylesheet from the PreTeXt distribution and an XML document of your creation that is valid PreTeXt. This is done at the command-line, inside of a terminal or shell. Describing command-line operations, along with file and directory management, is beyond the scope of this guide, so consult another resource if this is unfamiliar. So here is a hypothetical simple example:

```
rob@lava:~/pretext$ xsltproc xsl/pretext-html.xsl ~/books/aota/animals.xml
```

By default, `xsltproc` writes output to `stdout` (the screen), which you could redirect to a file, or you could use the `-o` switch to send the output to a named file. However, PreTeXt automatically writes to a file whose name is derived from the `@xml:id` attribute of the top-level `<book>` or `<article>` tag. If no such attribute is given the filename will be derived from `book-1` or `article-1`. All output is produced in whatever the current default directory is, so you will likely want to set this beforehand.

The `xsl` subdirectory of the PreTeXt distribution contains a variety of XSL stylesheets, which I will also refer to as **converters** or **conversions** . The ones that you will use as an author all have filenames of the form `xsl/pretext-XXX.xsl`, where `XXX` is some indication of the output produced. Conversions to $\text{\LaTeX}$ or HTML output are the two most mature converters.

Note that authors are not responsible for creating XSL stylesheets. Stock conversions are part of the PreTeXt distribution, and anybody is welcome to assume a source document is valid PreTeXt and create new conversions to process it to existing, or as yet unimagined, formats.

46.1 Setup

There are two components to processing your document, the PreTeXt stylesheets and the `xsltproc` program. We work at the **command-line** inside of a **terminal** or **console** . If you do not know what this is, it will seem very primitive at first. Sometimes the old ways are the best ways. This will be called a “Command Prompt” in Windows or a “Terminal” on a Mac. In Linux it may be known as a “console” or a “shell”. A tutorial, which is Linux-specific, can be found at [Ryan’s Tutorials](#) and certainly others exist.

The operating system on a Mac is built on Unix, which is very similar to Linux, so most of the directions here will be little changed between the two. Procedures can be very different in Windows ([Appendix M](#), [Appendix N](#)). One alternative is [CoCalc](#) which provides a full Linux computer for free in your web browser, so that may be an excellent place for initial experiments.

Step 1: PreTeXt. You need to obtain the PreTeXt stylesheets, which are the main part of PreTeXt . Since you are reading this, it may be possible that you have this already. You can use `git` to **clone** the PreTeXt from the GitHub

repository, and then be sure to checkout the dev branch to have the latest version. This is the best way to go, and you should only download the repository as a zip file once for an initial experiment, and then switch to using a clone instead.

Once you have a clone of the repository, you can issue `git pull`, and git will update your local copy with any recent changes. You should do this *regularly* — meaning on the order of *daily*. See the [FAQ entry](#) for more about why we *expect* you to do this.

See the [PreTeXt site](#) for details and commands for this step, right on the main page.

Step 2: `xsltproc`. This is the command-line program which takes your document and a PreTeXt stylesheet to together produce output. On Linux or a Mac you probably already have it installed as part of system software. On Windows it is not so simple.

In either case see the website for details about verifying you have this, or how to install it.

46.2 Processing

At a command prompt in your terminal or console adjust the path names for the two files and execute:

```
xsltproc /path/to/pretext/xsl/pretext-html.xsl /path/to/quickstart.xml
```

In the current working directory you should now find the file `article-1.html` which you can view in a web browser. (You will want an internet connection since various parts of the page come from the network. Someday we will create output for the offline situation.) It will look very plain, but you should be able to read the sentence.

Now, try the following, again with adjusted paths, and all on one line:

```
xsltproc -o quick.tex /path/to/pretext/xsl/pretext-latex.xsl
/path/to/quickstart.xml
```

In the current working directory you should now find the file `quick.tex` which you can process with `pdflatex` or `xelatex` at the command line as below. If you do not have $\text{\LaTeX}$ installed on your system, you could process this file within a variety of online services, and CoCalc would be an obvious choice.

```
pdflatex quick.tex
```

In the current working directory you should now find the file `quick.pdf` which you can view or print with standard PDF viewing software. You could even send it to a print-on-demand service to get nice hardback books, though I suspect sales will not be great.

Note that if your project includes multiple files you will need to pass the `-xinclude` flag to `xsltproc`, though this is not needed for this simple example. An example of this is the command

```
xsltproc -xinclude /path/to/pretext/xsl/pretext-html.xsl /path/to/index.xml
```

For more on this see [Section 4.39](#) and [Section 5.3](#).

That's it. You now know all the basics of authoring with PreTeXt, since you have produced two radically different output formats with identical content from the exact same structured input, via two different command lines. Everything you need to author a complete article or textbook, and produce it in many different formats, is just an extension or variation on what you just did. Let us look at a few simple extensions right away before being more methodical.

46.3 Modular Source Files

If you use the `<xi:include>` mechanism for modular source files, you must process your source slightly differently.

Add the switch `-xinclude` to your invocation of `xsltproc`, just after `xsltproc`, but before the filenames for the stylesheet and the top-level source file. Note that for some versions of `xsltproc` it might be necessary to use two dashes for the switch, `--xinclude`. So now a typical invocation (using one dash) might look like

```
xsltproc -xinclude xsl/pretext-html.xsl ~/books/aota/animals.xml
```

It is easy to forget the `-xinclude` switch. Empty output, or cryptic error messages, are your first clue to this simple, but common, mistake.

46.4 String Parameters

To pass string parameters to `xsltproc`, use a command like the following (possibly with `--xinclude`).

```
$ xsltproc -o animals.tex --stringparam latex.font.size "20pt"
$ /path/to/xsl/pretext-latex.xsl ~/books/aota/animals.xml
```

You can use as many `stringparam` as you like on the command-line (or in your scripts). The quotation marks are not strictly needed in this example, but if the value of the parameter has spaces, slashes, etc., then you need to quote-protect the string from the command-line processor, and either single or double quotes will work (and protect the other kind).

46.5 Extra XSL Stylesheets

If you want to use a custom XSL stylesheet, as described in [Section 28.2](#), it is a simple matter of using that custom as the XSL file fed to `xsltproc`. That is, enter something like the following.

```
$ xsltproc -xinclude -o animals.tex ./xsl/custom-latex.xsl ~/books/aota/animals.xml
```

Note that using this method, it is necessary to import the stock XSL using the `@href` instead of `@pretext-href`, as in

```
xsl:import href="path/to/pretext/xsl/pretext-latex.xsl"
```

. You need to specify the full path to the pretext/xsl directory, or else put your custom XSL in the pretext/user directory and use a relative path (`../xsl/pretext-latex.xsl`).

46.6 Publication File

To employ a publication file using `xsltproc` you use a string parameter ([Section 28.1](#)) named `publisher`. This should have a path that is relative to the main file for the document. For example, assuming `pod.xml` and `fauna.xml` are in the same directory

```
xsltproc -stringparam publisher pod.xml pretext-html.xsl aota.xml
```

This file should reduce the many other string parameters in use, and reduce the need for extra XSL files ([Section 28.2](#)).

46.7 Images Described in Source

As we discussed in [Section 5.8](#), there are many advantages to describing images directly in the source of your PreTeXt document. However, these must be processed with various “helper” programs such as `asy`, `pdflatex`, and `sage`.

The issue with this is that XSL is not a general purpose programming language, and so in particular, cannot call the “helper” programs. The general strategy is to use XSL to identify and isolate the parts of a document that lie in the elements designed for graphics languages. A Python script, the pretext script, employs these XSL stylesheets and then feeds each image file to the appropriate helper program.

This script has a variety of options, so we document it fully in [Chapter 47](#).

46.8 Author Tools

The `authors-report.xsl` stylesheet, found in the `xsl/utilities` directory, will report all of the provisional cross-references and all of the properly prefixed todo-comments. The report is organized by all of the divisions in use in your project. It is meant to be simple in appearance, just text.

Apply this stylesheet just like you would any other:

```
$ xsltproc -xinclude path/to/pretext/xsl/utilities/authors-report.xsl path/to/main.ptx
```

46.9 Keeping Your Source Up-to-Date

This section describes a tool you can use to automate the process of adjusting your source when there are deprecations. Generally, there is an xsl stylesheet which will convert your XML source to another XML source file, fixing many of the deprecations automatically. However, it is the nature of XML processing that your source file will undergo some cosmetic changes. For example, the order of attributes is never relevant, so an XML-to-XML conversion is free to re-order the attributes of an element, perhaps different from how you like to author them.

So you have two choices:

- Process your source with any of the provided conversions and edit by hand until the warnings all disappear.
- Run the deprecation-fixing conversion and accept the changes in XML formatting. (Read on for more specifics about these changes.)

You perform this conversion using `xsl/utilities/fix-deprecations.xsl` on an XML source file in the usual way. By default, output appears on the console, so you will want to specify an output file, for example with the `-o` flag of `xsltproc`. You will discover a safety measure that requires you to also use a parameter, which you can pass in to `xsltproc` with the `-stringparam` command-line argument.

One choice of the parameter will result in just “copying” your source file and making all the cosmetic source format changes (we refer to this here as **normalization**). This might be a useful thing to do first, all by itself, either as a first step, or an exploratory experiment. The other value of the parameter will actually make changes, and report some information about progress.

Here are some notes:

1. Be sure to experiment on copies of your source in a scratch directory. Send your output to another directory. When finished, use a diff tool to inspect the actual changes made. You can record your eventual changes using revision-control. (See [Git for Authors](#).)
2. Do not enable `xinclude` processing or else your several files will all be merged into one as output and any modularity of your source will be lost.
3. Every single bit of indentation and whitespace in your source will be preserved, except perhaps for some blank lines near the top of your source files, and limited exceptions noted below.
4. Attributes will likely be re-ordered, with normalized spacing between them.
5. Empty elements will have any spaces removed from the end of the tag.
6. Elements with no content may be written with a single empty tag.
7. CDATA sections will be converted to text acceptable to the XML parser. In other words, the CDATA wrapper will be removed and dangerous characters (`&`, `<`, `>`) will be replaced by equivalent entities (such as `&`). If you have many matrices expressed in $\text{\LaTeX}$ and wrapped in a CDATA, this might be a big change. See [Subsubsection 4.1.4.2](#) for background.
8. The output files will be labeled as having UTF-8 encoding.
9. It could be necessary to run this conversion more than once if deprecations build on one another. In other words, we do not update specific conversions, but rely on regular use to keep source up-to-date.
10. It should be safe to run this conversion repeatedly, even after new deprecations are added. In fact, it is encouraged.
11. The PreTeXt source file `examples/sample-errors-and-warnings.xml` is intentionally full of lots of bad stuff. You can experiment with it, should you want to see interesting things happen. We have already performed the normalization step, so you can concentrate on substantive changes.

To process a directory with multiple source files, I would proceed as follows. First make three temporary directories, `/tmp/original`, `/tmp/normal`, `/tmp/clean`, and copy my source files into `/tmp/original`. Then, using a BASH shell, and inputting the command all on one long line.


```
rob@lava:/tmp/original$ for f in *.xml; do xsltproc -o ../normal/$f -stringparam fix normalize
$ /home/rob/pretext/xsl/utilities/fix-deprecations.xsl $f; done
```

This will loop over every XML file in the current working directory, /tmp/original, running the normalization conversion on each file, with the output files using the same filename, but now being placed in the /tmp/normal directory. If you change to the /tmp directory, then you can compare the results. I like to use the `diff` utility provided by `git`.

```
rob@lava:/tmp$ git diff original normal
```

Or, try this for a view that might be more informative.

```
rob@lava:/tmp$ git diff --word-diff original normal
```

You may only do the above once, on your first use of this conversion stylesheet. You will see how your style of authoring XML will undergo some minor changes. We can repeat the above to actual make the changes necessary due to PreTeXt deprecations. Make /tmp/normal the working directory.

```
rob@lava:/tmp/normal$ for f in *.xml; do xsltproc -o ../clean/$f -stringparam fix all
$ /home/rob/pretext/xsl/utilities/fix-deprecations.xsl $f; done
```

And as above, you can now compare the normal and clean directories to see actual changes. If you are satisfied with the changes, you can copy the files in the clean directory back onto your source files. If you are using revision-control (you are, aren't you?) then you can make a commit that holds these changes. (See [Git for Authors](#).) Or maybe even make two commits, one from the normalization step, and a second with the substantive changes.

46.10 File Management

PreTeXt, at its core, is the formal specification of the XML vocabulary, as expressed in the DTD ([Section 5.4](#)). We have provided converters to process source files into useful output. However, we have not yet built a point-and-click application for the production of a book. So you need to take some responsibility in a large project for managing your files, both input and output. We have tried to provide flexible tools to make an author's job easier. The following is advice and practices we have successfully employed in several book projects.

Source. I am fond of describing my own books with an initialism formed from the title. So *A First Course in Linear Algebra* becomes *FCLA*, and in file and directory names becomes *fcla*. So I have a top-level directory *books* and then *books/fcla*, but this directory is not the book itself, this is all the extra stuff that goes along with writing a book, much of it in *books/fcla/local*. The actual book, the part everybody sees with an open license, lives in *books/fcla/fcla*. This subdirectory has files like *COPYING*, which is a free software standard for license information, and *README.md* which is a file in the simplistic Markdown format that is picked up automatically by GitHub and displayed nicely at the book's repository's main page. Subdirectories include *src* for the actual XML files, *xsl* for any customizing XSL ([Section 28.2](#)), and *script* for shell scripts used to process the book (see below).

I do not use any additional directory structure below *src* to manage modular files for a book, since the XML and the `--xinclude` mechanism manage that just fine. I see little benefit to extra subdirectories for organization and some resulting inconvenience. I do typically have a single subdirectory *src/images* for raster images and other graphics files.

I believe it is critically important to put your project under revision control, and if licensed openly, in a public GitHub repository. So the *books/fcla/fcla* directory and all of its contents and subdirectories is tracked as a `git` repository and hosted on GitHub. Because this directory is *source* I try very hard to *never* have any temporary files in these directories since I do not want to accidentally incorporate them into the `git` repository. As a general rule-of-thumb, only original material goes in this directory and anything that can be re-created belongs outside.

A tutorial on `git` would be way outside the scope of this guide, but Beezer and Farmer *have* written *Git For Authors*, so perhaps look for that.

Image Files. Some images are raster images (e.g. photographs) that are not easily changed, and perhaps unlikely to be changed. Other images will come from source-level languages via the pretext script. For your convenience, this script has a command-line option that allows you to direct output (graphics files) to a directory of your choice.

In the early stages of writing a book, I put image files produced from source code in a directory outside of what is tracked by `git`. It is only when a project is very mature that I begin to include completed graphics files into the `src/images` directory for tracking by `git`.

Build Scripts. When you have a mature book project, the various files, processing options, and a desire for multiple outputs can all get a bit confusing. Writing simple scripts is a good idea and the investment of time doing this early in a project will pay off through the course of further writing and editing. The particular setup you employ is less important.

I have fallen into the habit of using the `make` program. It allows me to define common variables upfront (such as paths to the PreTeXt distribution and the main directory for the project it applies to). Then I can easily make “targets” for different outputs. So, for example I typically go `make pdf` or `make html` to produce output, and have simple companion targets so that I can go `make viewpdf` or `make viewhtml`. Other targets do things like checking my source against the DTD (Section 5.4). I have split out the variable definitions in a way that a collaborator can join the project and simply edit the file of definitions just once to reflect their setup, and still participate in future upgrades to the script by pulling from GitHub and not overwrite their local information.

My use of `make` is a bit of an abuse, since it is really designed for large software projects, with the aim of reducing duplicative compilations and that is not at all the purpose. You could likely have exactly the same effect with a shell script and a case (or switch) statement.

My general strategy is to assemble all the necessary files into a temporary directory (under `/tmp` in Linux) by copying them out of their permanent home, copy customizing XSL into the right place (typically `pretext/user`), run the pretext script as necessary and direct the results to the right place, and finally copy results out of the temporary directory if they are meant to be permanent. Interesting, an exception to staging all these files is the source of the book itself which is only read for each conversion and then not needed for the output. So you can just point directly to a top-level file and the `xinclude` mechanism locates any other necessary source files.

A good example of this general strategy is the use and placement of image files for HTML output. It is your responsibility to place images into the location your resulting HTML files expect to locate them. By default, this is a subdirectory of the directory holding the HTML files, named `images`. You will want to copy images, such as photographs, out of your main source directory (`src/images?`). But you may be actively modifying source code for diagrams, and you want to re-run the pretext script for each run, and make sure the output of the script is directed to the correct subdirectory for the HTML output. Running the pretext script frequently can get tiresome, so maybe you have a makefile target `make diagrams` that updates a permanent directory, outside of your tracked files in the repository, and you copy those files into the correct subdirectory for the output. That way, you can update images only when you are actively editing them, or when you are producing a draft that you want to be as up-to-date as possible. As a project matures, you can add images into the directory tracked by `git` so they are available to others without getting involved with the pretext script.

We did not say it would be easy, but we feel much of this sort of project management is outside the scope of the PreTeXt project itself, while in its initial stages, and existing tools to manage the complexity are available and documented. (We *have* been encouraged to create sample scripts, which we may do.) Just remember the strategy: stage necessary components in a temporary directory, build output in that directory, copy out desired semi-permanent results, and limit additions to the source directory to that which is original, or mature and time-consuming to reproduce.

46.11 Doctesting Sage Code

Adding computer code to your textbook is a tricky proposition. You can propose that it is merely an illustration, and not meant to have all the necessary details, or you can make it exact, correct and executable, and then risk inevitable changes to render your code obsolete. At least you have the option of editing and reposting online versions quickly and easily.

One of our main motivations for this project was mixing in code from the powerful, open source, mathematical software project, Sage (Section 3.17). When you add example Sage code to illustrate mathematical ideas, you are then encouraged to also include expected output in the `<output>` element. Here comes one of the powerful advantages of XML source and XSL processing.

The `pretext/xsl/pretext-sage-doctest.xsl` stylesheet, used in the usual way, will create one, or several file(s), in *exactly* the format Sage expects for automated testing. So all your words are gone, and all your Sage input and output is packaged so Sage can run all the `<input>` and compare the results to the expected `<output>`. See [Subsection 44.1.1](#) for details on obtaining more than one file.

We have many years' experience testing hundreds of non-trivial Sage examples from textbooks, for linear algebra and abstract algebra. Roughly every six months, we discover ten to twenty examples that fail. Frequently the failures are trivial (usually output gets re-ordered), but some are significant changes in behavior that leads us to re-word surrounding guidance in the text, and in a few cases the failures have exposed bugs introduced into Sage. It has been relatively easy to do this maintenance on a regular basis, and if it had not been done, the accumulated errors would be enough to greatly degrade confidence in the accuracy of the examples.

Exact details for this process can be found in [Section 4.38](#). Note that Sage is really just a huge Python library, so it might be possible to test pure Python code with this facility, but we have not tested this at all. Similar support for other languages can be considered if requested for use in a serious project.

Chapter 47

The pretext Script

Note that this *entire chapter* documents a lower-level tool for producing output from PreTeXt source. Most authors and publishers should be using the PreTeXt-CLI, see [Chapter 5](#).

XSL is a very powerful language for text processing. However, it cannot do everything. The pretext/pretext script is a Swiss Army Knife of sorts to operate on parts of your document and manage processing that requires the application of external programs, such as $\text{\LaTeX}$ and Sage. It can also produce entire documents in different output formats. Its principal routines are shared with the PreTeXt-CLI.

47.1 Running pretext/pretext

pretext is a Python program (aka script), so you will need to have the Python 3 interpreter on your system (version 3.6 or better expected as of 2021-05-21) which you can run at a command-line (aka terminal, console, command prompt). So your first step is to install Python (see [Appendix C](#)) or check your installed version and the exact name of your executable (see [Section C.1](#)).

The pretext script is located in a directory of the PreTeXt distribution also named pretext. So we often refer to it as the pretext/pretext script to avoid confusion with all the other places we say PreTeXt. As a simple check on your ability to run the script, with a suitable path you can run

```
/path/to/pretext/pretext -h
```

to get a summary of the commands. Since the command lives in the pretext directory, you may prefer to change to that directory for your first attempt. On a Unix-like system (Linux, Mac) you may need to go `./pretext` since the program is not on your PATH. Thus, depending on your executable name ([Section C.1](#)), whether the script is in your path, and whether you have made the file executable, you might need to run

```
python /path/to/pretext/pretext -h
```

or

```
python3 /path/to/pretext/pretext -h
```

Some of the processing may take a long time, or you may experience trouble. There are two switches to enable more verbose output in your terminal or console.

```
pretext -v [command arguments]
```

will provide progress indicators, which may be comforting for long runs, while

```
pretext -vv [command arguments]
```

will provide progress indicators along with additional technical information that will help you or a fellow author to discern where a problem lies. If you ask for help, please *include all of this output, from start to finish*, including the command you use and the current directory, and *do not assume you know exactly which part is the relevant part*.

47.2 Overview

Generally, you use the `pretext/pretext` script by supplying a **component** (`-c`) and a **format** (`-f`), along with your PreTeXt source as the very last argument. The component may be some limited subset of your document that needs specialized treatment, such as an image described by the Asymptote language (`-c asy`). Or it could be the entire document (`-c doc`) being converted to some format as a final output form. So `-c asy -f svg` would produce Asymptote images in svg files, while `-c all -f latex` would produce the entire document as a $\text{\LaTeX}$ file.

47.3 Example Use

Here is a typical example of using `pretext/pretext`. You have several (or many!) diagrams and figures in your PreTeXt source, all authored in the TikZ language, and so packaged up within `<latex-image>` elements. Your $\text{\LaTeX}$ /PDF output looks beautiful, since PreTeXt simply inserts the TikZ code into the right place in the generated `*.tex` file, and you have done this several times until your figures look just right.

Now you need to generate the svg versions of your images that will accompany your HTML version of your book and provide nice scalable graphics. This is exactly the sort of chore the `pretext/pretext` script was designed for. You might run (all on one line)

```
pretext -vv -c latex-image -f svg
-d ~/books/aota/images ~/books/aota/animals.xml
```

Here `-c` is specifying the “component” of your book to process, and `-f` is specifying the “format” of what is being produced. The `-d` argument specifies a directory where the output ends up, in this case a collection of svg files, one per image. Note that the PreTeXt file, `animals.xml` in the above example, is the main source file for the document. The script should be run on the entire document, even if all the images are in one file.

47.4 Strategy

Much like the build advice at the end of [Section 46.10](#), the `pretext/pretext` script collects necessary bits into a system-created temporary directory, does its work, and copies out the desired results. So in the example of the previous section, each chunk of TikZ code is isolated, your $\text{\LaTeX}$ macros are copied from `<docinfo>`, and a syntactically correct $\text{\LaTeX}$ file is produced (one per image). Then `pretext` calls your system’s $\text{\LaTeX}$ executable on each of these files to produce a one-page PDF. This is then cropped and converted to an svg version, which at the end is copied to the location specified in the `-d` argument.

Some insight into failures can be found in the temporary directory where all this processing happened. (We leave the directory, and its contents, behind for the system to clean-up next time the system is rebooted). Early in the `-vv` doubly-verbose output, this directory is reported after the string `temporary directory:.`

Some notes:

- If you have modularized your source across more than one XML file, then be sure to provide your top-level file as the final argument to the script, just like you would for an invocation of `xsltproc`. It is important to understand that your source is one huge “source tree” and your file-by-file modularization is never respected or recognized in any way. In particular, use of the `xinclude` mechanism is handled by the script, and you should not apply the script to each of your source files individually. If image production (or some other task) takes a long time, see [Section 47.10](#) for a way to have the script restrict its action to only a portion of your project.
- Do not place the script, or configuration files, anywhere else (except as recommended for your personal copy of the configuration file). The locations are *critically important* for locating other files, such as the stylesheets used to isolate parts of your project for processing.
- Much of the work of this script happens in the temporary directory described above. We leave a lot of intermediate work behind in this directory. Often, exploring this directory is helpful when debugging problems, or a failure to finish successfully. For an example, see [Section 47.5](#).

47.5 Debugging Image Generation

A principal use of the pretext/pretext script is to isolate source code from `<latex-image>` sections, package them up as proper `*.tex` files, run $\text{\LaTeX}$ to make cropped PDF versions, and then convert these to other formats such as SVG or PNG.

Much of this activity happens in a temporary directory, and it is similar for Asymptote images and other options of the script. If you use the `-vv` switch described above, then these temporary directories will be noted in the debugging output requested, and a complete list will be the last line of output. In this case, and if the script encounters an error prior to successful execution, then these directories will be left in place. Looking to see what files end up there, and what those files contain, is often useful in determining the step where the script fails, and maybe even why.

With no chatter requested, or just progress indications (`-v`) these temporary directories will be cleaned out as the last part of successful execution.

Another option is to ask for the actual source files (`*.tex`, `*.asy`, etc.) as the output of a run. This is accomplished with the `-f source` option when invoking pretext/pretext. If the right packages or macros are not being employed in these files, this is an easy way to get at the source files for inspection and analysis, and is a good first check on problematic execution.

It is possible to design an image whose dimensions exceed the page size of typical $\text{\LaTeX}$ output. An example is an image you might use only for some HTML output, such as a slideshow. In this case, construction and conversion to an output format like SVG may fail. There is a string parameter, `latex.geometry`, that allows you to provide options to the $\text{\LaTeX}$ [geometry package](#) which influence page size. At a minimum, you need to override the page size, plus it might be best to override the “printable” area as well. (See [Section 47.8](#) for more on string parameters, which will be obsolete in the future.) Here is a sample *addition* to a call to the pretext script. Note that you cannot put the `-x` switch last, so it must precede some other option.

```
-x latex.geometry "paperwidth=21in,paperheight=21in,total={20in,20in}"
```

Since the image will ultimately be cropped, there is little harm in making the overall page grossly oversized. However, if the $\text{\LaTeX}$ code you are using employs lengths like `\linewidth` or `\textwidth` (which we will claim is not a good practice), then your image might be very sensitive to how you set the page geometry. Note that restricting the scope ([Section 47.10](#)) might be useful if this accommodation is only needed for one image.

47.6 Configuring External Helper Programs

Our main processor, `xsltproc`, is not a general-purpose compiler, and does not “call” external programs. That is the *raison d’être* of the pretext/pretext script. You will see a configuration file, `pretext/pretext.cfg`, as part of the distribution. Read the comments at the top of this file, but foremost, realize that you are not meant to edit this file. It is a template, and any changes you make will be overwritten with an original version when you update. Instead, make a copy and place it as `user/pretext.cfg` within the distribution. For instance, if your distribution is in the pretext directory, then the commands

```
cd /path/to/pretext
mkdir user
cp pretext/pretext.cfg user/
```

would be appropriate. The script will look for the user copy first, and if not found, then fallback to the generic version.

The entries of this file are the names of executable files that perform certain tasks as part of the script’s functions. If it seems that certain helper programs are not being found, you can provide full path names, and that may solve the problem.

47.7 Python Prerequisites for the pretext/pretext script

There are several external programs that the script relies on that are again Python packages. See [Section C.3](#) for instructions and a list.

47.8 Publication Files, String Parameters

A publication file ([Section 26.1](#)) can be used to control various options that are independent of the authoring process, or for some conversions may be necessary. And the managed directories scheme requires use of a publication file (see [Section 5.6](#)). Use the `-p` switch to specify this file, using a path that is relative to your PreTeXt source (or an absolute path if it seems to be ignored).

Sometimes you might have need to pass string parameters to the PreTeXt script, though this will eventually be something an author will rarely do, and is more likely necessary for developers. This is accomplished with the `-x` flag, followed by a space separated list of (stringparam, value) pairs. Do not use parentheses, just separate with spaces. But note that `-x` cannot be the last option passed to the script since it makes it hard for the script to “see” the filename for your source.

For example:

```
pretext -vv -x debug.datedfiles no debug.chunk 0
      -f html -c doc -p ~/books/aota/pub.xml
      -d /tmp/aota-html ~/books/aota/animals.xml
```

47.9 Extra Stylesheet

There are situations when an “extra” XSL stylesheet is necessary for processing your XML source. A good example is the supported method for styling PDF output, described at [Chapter 41](#). Historically, various customizations have been supported by “thin” XSL stylesheets ([Section 28.2](#)), but as certain themes have become apparent we have moved these customizations to simpler techniques (e.g. publication files, [Section 26.1](#)). Other than styling PDF, you should think carefully about if an additional stylesheet is necessary.

The `-X` (`--XSL`) switch is used to specify an “extra” XSL stylesheet that will be applied to your source. Note that it is now possible to do *anything* you want to your source and so can create run-time failures at any point in the process. Here are the simple mistakes to avoid.

- If you are expecting HTML output because you used `-f html`, then your stylesheet supplied in the `-X` argument will certainly import the base `xsl/pretext-html.xsl` stylesheet. When you understand this, then you will understand that perhaps we should call this an “alternate” stylesheet. This advice applies equally well to extending the base `xsl/pretext-latex.xsl` stylesheet.
- The import just described will behave better for others (meaning co-authors, or authors who fork your project) if the import uses a relative path, meaning relative to location of the extra stylesheet. We therefore suggest using a user directory, placed as a peer of the `xsl` directory, as described in [Section 28.2](#).
- So a common mistake is to use `-X` to point to an extra stylesheet someplace close-by your project’s source files, when you have sensibly setup a relative import of the base stylesheet, and instead should point to the *copy* you have placed in user so that the import is effective on everybody’s system.

As of 2021-08-04, this technique is only effective for HTML, PDF, and $\text{\LaTeX}$ outputs. It may be natural for some other output formats (e.g., EPUB), and perhaps possible for others (e.g., braille). Make a feature request for expanding applicability.

47.10 Restricting the Scope

The `-r` (`--restrict`) switch deserves special mention. It is followed by the value of an `@xml:id` attribute present in your source XML file. Then whatever action the script is asked to perform, it will only act on a subtree of the hierarchy, rooted at the element with the given `@xml:id` value.

So if your images are complex or numerous (or both!) and take a long time to process, you can restrict attention to whatever part of the document you are actively editing, and you can even restrict to a single `<image>` and so produce just a single graphics file.

47.11 Methods

The `-M` (`--method`) switch is used in some cases to specify the *method* by which some process is achieved. Supported situations and values are listed below. Defaults are listed first, in brackets.

Table 47.11.1 PreTeXt Script Methods

Situation	Values
<code>-c asy</code>	[server], local
<code>-c latex-image</code>	[xelatex], pdflatex
<code>-c doc -f pdf</code>	[xelatex], pdflatex
<code>-c preview</code>	[fast], slow

For generation of preview images for interactive elements, `fast` is the default of a 5 second timeout to allow the page to load; if the interactive element is taking longer to load, using `slow` will double the timeout.

47.12 Output

If you use the script to make a single file, such as your complete project as a PDF or an EPUB, then you can use the `-o` switch to specify this file, otherwise the file will land in the current working directory.

If your output consists of many files, such as all the HTML for your complete project, then you can specify a directory with the `-d` switch. Again, the default is the current working directory.

If you specify one of the components in [List 5.6.2](#) and you are using a publication file to specify directories that are managed ([Section 5.6](#)) then the multiple files for that component will automatically be placed into necessary directories by default (rather than in the current working directory). Of course, you can override this behavior by specifying a directory with `-d`. For example, for many operating systems, by using `-d .` you can have the results land in the current working directory. (Note there is a necessary period there.)

In the early stages of a project, you might rebuild your images regularly. But you may not always want those results landing within directories under revision control. Later in a project, your images may be relatively stable, and you want to distribute them with your source, perhaps so others can re-purpose them in handouts or other materials. To accommodate this, make two publication files, and in one make a relative path for the generated components that is outside of the main directory that is under revision control. Likely you will need some file path syntax for a parent directory, such as `../` on Unix-like systems. Then you can switch from testing to distribution, and back, easily.

47.13 pretext/pretext Capabilities

List 47.13.1 pretext/pretext Capabilities

Again, the command `pretext -h` will remind you of the various options for the script and is the most likely list to be correct and up-to-date. The following is a brief summary, in general terms, of what is possible.

Complete Conversions

With `-c doc` and various choices of the format (`-f`) the script will execute a complete conversion. In some cases, this is a convenience compared with just using the `xsltproc` processor. In other cases the conversion is very complex and multiple (arcane) output files must be packaged up in very specific ways and no author would want to manage it all. Conversion to EPUB is one example ([provisional cross-reference: `epub conversion`]).

LaTeX Graphics

LaTeX has a variety of languages for specifying images, such as `xypic`, `pgfplots`, and `TikZ`. By including the necessary packages or setup commands in `docinfo/latex-preamble`, these can all be generated at once, in the manner of the example earlier.

Asymptote Images described by the Asymptote language can be processed in a manner entirely similar to that for images described with $\text{\LaTeX}$ graphics languages. By default an online server will be used for the image generation, or you may elect to use an `asy` executable on your system and locatable via the `pretext.cfg` configuration file.

Sage Plot If you have a version of Sage installed on your system, you can specify the path to the executable and obtain images described by Sage code. See [Subsubsection 4.14.3.4](#) for more information.

All Formats If you desire images in a wide variety of formats, the option `-f all` will oblige.

YouTube Thumbnails

For each YouTube video (or itemized playlist) you specify, the script will go the YouTube site and grab a thumbnail image for that video (or first video from the itemized playlist). These get used in static formats, such as PDF.

Preview Images

Like a thumbnail for a YouTube video, other interactive content can benefit from a still image for use with static formats. Our strategy is to render the content with a “headless” web browser and capture an image automatically. For example, you might write some custom Javascript to allow a reader to interact with a graph, and you would like an image of the graph, along with its interactive sliders and checkboxes, to appear in the PDF version of your text. Recognize that this image will necessarily be the content at its initial start-up. Get a screenshot manually if you want something better.

Note that for all this to work, you need to properly serve your project’s HTML output with the interactive content. (A local server may be a possibility, but we have not tried. See [Section 5.11](#). Or you can try a temporary public server, see [Section 5.12](#)). Then you need to announce the location of this hosted HTML, which is accomplished via a “base URL” in a publication file. Start at [Subsection 29.1.6](#) for details on this publisher variable. Note that the chunk level in your publication file used in this process *must* match the chunk level used when you build your HTML output, and mismatches for other publisher variables could have some ill effects.

The automatic screenshots will then be managed by the Python `playwright` package, which you will need to have installed in your Python virtual environment (see [Section C.3](#)).

Mathematics Representations

Conversions to EPUB, braille, and other formats require creating conversions of mathematics elements locally as part of the conversion. This requires having MathJax installed locally, see [Appendix G](#). Structured files of these representations can be obtained by setting the component with `-c math`. Possible formats (`-f`) are `svg`, `mml` (MathML), `braille` and `speech`.

WeBWorK Various conversions of WeBWorK problems are facilitated using either a WeBWorK server or a local copy of the PG repository. See [Chapter 7](#) for the details of this procedure.

Chapter 48

Coding Conventions

48.1 Author and Publisher Input

When an author or publisher provides input to regulate some change in how their content is realized follow these procedures.

- String parameters (which are discouraged anyway) should default to an empty string initially when defined. For technical reasons, this makes them easier to deprecate later.
- Input should produce a global variable, whose possible values are known exactly. This variable should be set in one location, with other similar variables, and then it should be used later. There should be no need to further error-check this variable later. In other words, do not mix error-checking input and employment of that input—rigorously separate them (like separating content from presentation!).
- Use a `<xsl:choose>` inside a `<xsl:variable>` that first parrots legitimate input. Next check for attempts to set the input to something “invalid”. Use `<xsl:message>` to report:
 - the name of the input being checked
 - a description of valid values
 - “, not” followed by the invalid value supplied (via `<xsl:value-of/>` typically)
 - “Using the default value”, followed by that value
- Then don’t forget to actually set the default value, either due to no input, or erroneous input.
- Done right, this variable will usually meet the requirements above.
- If input is binary, have the variable just described take on one of two values. Only. Then construct a new boolean variable based on these two strings. Name the variable with a `b-` prefix. This will be more reliable to use later.

48.2 Run-Time Messages

XSL provides a `<xsl:message>` element, which writes to the console (the `stderr` stream really, I believe). This makes it useful for debugging (like a `print` statement in another language). A mix of text, `<xsl:apply-templates/>`, and `<xsl:value-of/>` works well. Include delimiters (colons perhaps) so you don’t get fooled by unexpected empty strings that are part of your problem anyway.

But our point here is to discuss messages broadcast to an author or publisher.

- Use `<xsl:message>` to communicate with authors and publishers as described above in [Section 48.1](#) while negotiating their input used to control their content.

- Otherwise, during actual processing, use `<xsl:message>` *only* when there is the potential for a real disaster in the output, such as a $\LaTeX$ file that will not compile at all, meaning more than simple recoverable errors.
- Do not duplicate parent-child relationships that are expressed in the schema. There are good validation tools for that job.
- Consider using our auxiliary “validation-plus” stylesheet for detecting more complicated relationships that the schema cannot, or does not, detect.
- See the advice above ([Section 48.1](#)) about checking author and publisher input and providing a “safe” result so there is no danger of creating broken output and thus less temptation to use `<xsl:message>` at all.
- Reserve adding `terminate="yes"` for extreme situations where there is absolutely no hope of recovery. This should be a last resort.
- Place `terminate="no"` (the default) in your code only when some future removal of deprecated code means this message will transition to `terminate="yes"`. Include clear code comments describing the situation.
- Prefix messages with
 - `PTX:WARNING:` for advisories
 - `PTX:ERROR:` for recoverable problems
 - `PTX:FATAL:` for case when processing terminates
 - `PTX:BUG:` for situations that are unexpected (and problematic)

and align the start of messages in column 14 (so they all line up).

- There is a template with `mode="location-report"` you can use to provide standardized assistance for an author or publisher looking for the place to correct a problem (in lieu of line numbers, which are not available).

48.3 Locating Other Nodes

Should you need to locate a node some other place in a document, via some sort of a reference by an identifier, be sure to use the interface provided by the `id-lookup-by-name` named template. It accepts a string (the value of the identifier in the reference) and returns an `xml:id` value that can be used with the `xsl:id()` function to return the desired node (as a node-set). Use a pattern such as the following, paying particular attention to the name of the variable supplied to the `id()` function.

```
<xsl:variable name="target-id">
  <xsl:call-template name="id-lookup-by-name">
    <xsl:with-param name="name" select="normalize-space(@ref)"/>
  </xsl:call-template>
</xsl:variable>
<xsl:variable name="target" select="id($target-id)" />
```

48.4 Documentation

Careful documentation must accompany new features. You *cannot* leave this task for somebody else to clean-up after you, and writing the documentation is likely to cause you to revisit your code to introduce small improvements or squash bugs.

New PreTeXt language elements are described briefly in the overview at [Chapter 3](#). This should make new authors *aware* of what elements are available, and maybe have enough instruction to get them started. Then the topics at [Chapter 4](#) should have full details. These two sections should link back-and-forth to each other. Examples are welcome, but should be short and succinct. Elaborate examples should be contributed to the *Showcase Article*.

New options for publishers should be detailed *very tersely* in the reference section, [Chapter 44](#). This section is organized lexicographically with respect to the XPath expressions describing the entries of the publication file. Then a more careful description, with examples if necessary, should appear in one of the chapters of the publisher part ([Part IV](#)) according to which aspects of a conversion are affected. These two sections should link back-and-forth to each other.

Here are some conventions to follow, which will help authors and publishers have a better experience wandering through the Guide. Please observe them in your own contributions.

- *The Guide.*

We usually just say “the Guide” or “in this Guide”, with no additional formatting, and just the capital “G” indicative of a formal title.

- *Modularization.*

When an author’s source is modularized ([Section 5.3](#)) we refer to the outermost, or main, file as the “top-level” file. This is the one file that has the overall `<pretext>` element in it.

- *Git Branches.*

Reference the primary branch of a git repository as `main`.

Chapter 49

Styling with CSS and SCSS

The PreTeXt conversion to HTML deliberately produces plain, semantic markup and leaves nearly all of the visual presentation to CSS (see [Section 29.6](#) for the publisher-facing view). This allows the same content to be styled in many different ways, and for new themes to be added without changing the conversion. It also allows the styling to be shared across different output formats, such as web and print.

To work well in this system, HTML generated by XSL or Javascript should avoid hard-coded inline styles attributes and instead use classes and CSS to control the styling.

Every HTML book loads a single `theme.css` file, which is not written by hand but built from SCSS source by a small node script. This chapter is a map of that source, so you can find your way around to adjust an existing theme or contribute a new one. You do not need to be a web developer; a working knowledge of CSS and a willingness to read existing files is enough.

49.1 The css Directory

All styling source lives in the top-level `css` directory, whose `README.md` is the authoritative reference. The folders are:

- **targets**—each **target** is a self-contained final product: one entry file that compiles to one CSS file. The modern HTML themes are in `targets/html`, one folder per theme, named for a city (default-modern, salem, denver, greeley, boulder, tacoma). Non-web targets (epub, kindle, revealjs, the print worksheet) live here too. A file used by only one target belongs in that target's folder.
- **components**—source shared across targets, grouped by what it styles: `chunks` (PreTeXt containers like exercises and knows), `elements` (small pieces such as figures and lists), `page-parts` (table of contents, navigation bar, banner), `interactives` (Sage, Runestone, WeBWorK), and `helpers` (reusable mixins). Editing a component affects every target that uses it.
- **colors**—color **palettes** a theme can draw on, plus the master list of color variables (see [Section 49.4](#)).
- **fonts**—the mechanism a theme uses to select fonts.
- **dist**—the built, ready-to-use CSS files (and source maps). *Generated output—never edit by hand* (see [Section 49.2](#)).
- **legacy**—plain CSS for older themes that predate the SCSS system, kept for backward compatibility; new work does not go here. `other` holds CSS that is not part of a book, such as the PreTeXt catalog.

49.2 Building the CSS

The distributed CSS files are built from scss (or “Sass”) source files. scss is a superset of css that adds variables, nesting, functions, and a module system (@use) for splitting styles across files. This source is built into ordinary css for use by the browser.

Plain css is valid scss, so for the most part, adding styles to elements just involves writing normal css. The extra features of scss are only needed when you want to differentiate the css output for different themes or integrate with the color palettes. The [Sass basics guide](#) is a good resource for learning about scss features.

The build script is `script/cssbuilder`, run with `node`. In normal use you never invoke it directly—the PreTeXt script and the CLI build whatever theme a document needs. The one-time setup is to install `node` and run `npm install` once in `script/cssbuilder` (the CLI does even this for you). See [Appendix F](#) for `node` and `npm`. Once that is done, you can build all of the CSS targets from the `cssbuilder` directory with:

```
script/cssbuilder$ npm run build
```

When working on the css for a feature, you will likely not want to build all the theme files and then have to rebuild your test book. Instead, you will likely want to update just the css directly in your book so you that you can examine styling updates without rebuilding the entire project or moving files around. The simplest way is to use the PreTeXt script’s `-c` theme flag to rebuild only the theme (See [Chapter 47](#)) directly to your book.

For finer control over which theme is built (to rapidly check different themes), or to have the CSS automatically rebuild when you change the source, you can run the build script directly with `node`. From the `script/cssbuilder` directory, do:

```
script/cssbuilder$ npm run build -- -t theme-default-modern -o path_to_yourbook/_static/pretext/css
-w
```

The `--` is required so the flags reach the script rather than `npm`. Useful flags: `-t` (one target; `-l` lists them), `-o` (output directory), `-w` (watch and rebuild on changes), and `-c` (a json string of options and variables, the same ones a publisher sets, for trying colors and options). Run `npm run build -- -h` for the full list; details are in `script/cssbuilder/README.md`.

49.3 How a Theme Is Assembled

Each PreTeXt work relies on a single `theme.css` file for all of its styling. That file is built from a single scss entry file (for example `targets/html/default-modern/theme-default-modern.scss`) does three things: it declares the variables a publisher may override (each marked `!default`); it pulls in components and a color palette with `@use`; and it finishes by emitting the css variables defined by the imported files.

The `css/README.md` describes ways themes are differentiated. In general, the files in a `target/` folder are only expected to be used by that target, so they can contain rules that are specific to that theme. Anything in the `components` folder is shared across targets, so it must work well across different themes and color palettes.

Simple, local styles for a small component can likely be written without much extra work to integrate with various themes. The most likely complications are setting colors ([Section 49.4](#)), fonts ([Section 49.5](#)), or items that need to be placed or sized relative to other elements on the page ([Section 49.6](#)).

49.4 Colors and Palettes

In general, styling rules should not contain hard-coded colors. A color that looks good in one theme (or set of colors for a theme) may not fit into another. And a color that works well in light mode may make something unreadable in dark mode. Instead, colors should be expressed as css custom properties (variables) that a theme can be set to different values.

For example, links in a PreTeXt document are set to `color: var(--link-text-color)` instead of something like `color: blue`. `--link-text-color` is set to a default color in `colors/_color-vars.scss`. But then each color palette overrides it as needed to fit the palette’s overall color scheme. In a theme that uses the dual palette, `--link-text-color` is defined as being the same as `var(--primary-color)`, which is set to match the color selected in the publisher file of the work.

The easiest way to guarantee that new CSS will work reasonably well with every existing theme/mode is to use the existing color variables. `colors/_color-vars.scss` is the master list of every color, given semantic names (`--link-text-color`, not `--pretty-blue`). `palette` files (such as `_palette-dual.scss`) starts from that list, overrides the entries it cares about, and produces an scss map named `$colors`. Each theme uses one of those palettes to set the color variables it will use.

Say you are adding a feature that should look something like existing buttons. To match the existing style, first find the color variables used (Section 49.7). Once you identify the relevant variables, such as `--button-background`, you can use them in your new CSS.

Best Practice 49.4.1 If you want to have something be white or black (in light mode), you should generally use `var(--content-background)` or `var(--body-text-color)` respectively. Those are good semantic choices for “the color of the background” and “the color of foreground content” and will change appropriately in different palettes.

If you want to add finer control over the colors used by a new feature, add a new color variable to `colors/_color-vars.scss`, and use it in your CSS. To do so, you would write something like:

```
| "dropdown-background": var(--button-background),
```

This ensures that the new variable has a default value (the button background color). Any palette that targets multiple colors or dark mode will already have a value for `--button-background`, so your new variable will automatically work with those. Then, to customize the new variable for a particular palette like `_palette-dual.scss`, you would add an entry to the palette file, such as: `"dropdown-background": var(--secondary-color)`. In a theme that uses the dual palette (like `default-modern`), the dropdown background will now be the secondary color, which is a different color than the button background. In a theme that uses a palette that does not customize `--dropdown-background`, the dropdown background will be the same as the button background.

49.5 Font and Icons

`css/fonts` contains code to set up different font stacks for each theme. These font stacks are rendered into CSS variables:

- `--font-body`—the font used for the main text
- `--font-headings`—the font used for headings
- `--font-monospace`—the font used for code and other monospaced text

When writing new CSS, you can use these variables to ensure that your new feature uses the same fonts as the rest of the theme.

Icons for various buttons and controls are provided for by the Google Material Symbols font. To use an icon from this font, add the classes `icon material-symbols-outlined` to an element (typically a `<span>` or `<i>`) and set its content to the name of the icon you want. For example, to get a magnifying glass icon, you would write:

```
| <span class="icon material-symbols-outlined" aria-hidden="true">search</span>
```

The `aria-hidden="true"` is important for accessibility, as it tells screen readers to ignore the icon, which is typically decorative and does not convey information on its own. Equivalent information should be provided through other means, such as text labels or ARIA attributes.

49.6 Sizing and Positioning Elements

Elements that are placed or sized relative to the page or other top level structures of the page will likely need to leverage scss variables to work well across different themes. Many files have variables defined that look like this one from `css/components/page-parts/_body.scss`: `$content-width: 600px !default`; That variable can be overridden by a file that @uses it, but otherwise will have a default value of 600px. That variable can be used in style rules:

```
| .ptx-page {
|     max-width: $content-width;
|     margin-left: auto;
```

```
    margin-right: auto;
}
```

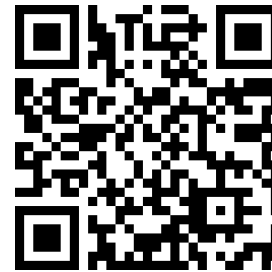
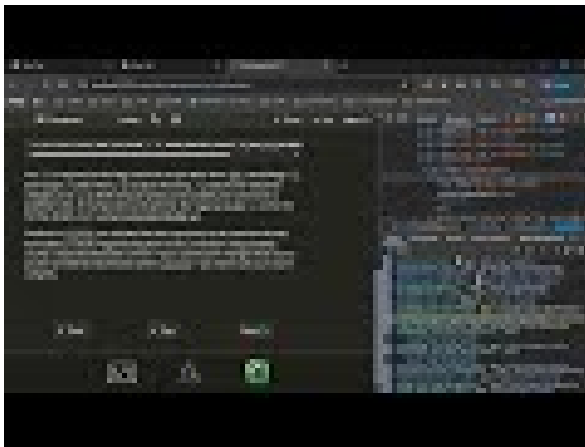
When the css is built, the variable will be replaced by the appropriate value in each build target.

A few of the most important sizing values are also rendered into css variables that can be referenced from anywhere. For example, `var(--base-content-width)` can be used to reference the standard width of the content area. Look for spots where scss variables are rendered into css variables like this:

```
:root {
  --base-content-width: #{ $content-width };
  --content-padding: #{ $content-side-padding };
}
```

49.7 Finding Styles and Making Changes

If you are modifying existing styles or trying to identify how those will interact with your new content, begin by using your browser's developer tools to inspect the element you want to change. Identify the style rules that are currently applied and where those rules are located in the source—the browser should tell you which scss file was responsible for each rule. You can also use the browser's developer tools to test changes to the style in real time before modifying the source.



[Standalone](#)
[In Context](#)

Figure 49.7.1 Using the browser's developer tools to find and test styles

Then using that information, find the relevant source file in the css directory. Make your changes, and then rebuild the theme ([Section 49.2](#)) to test. Make sure to test different screen sizes (desktop, tablet, mobile) using your browser's developer tools as well as in both light and dark modes.

If the rules you have modified or added are in “shared” files like those in components, be aware that your change will affect every theme that uses that component, so you should check how it looks in other themes.

You generally should have a commit for the CSS that is separate from the commit for the content changes that require it. For example, you might have one commit that updates the XML to produce new HTML and another commit that updates the corresponding scss styles for that content.

The files in `css/dist` are checked into the repository so the common themes work without a build. However, your pull request should not *not* include updates to those files (stale builds tend to cause merge conflicts with other work that has been in progress). The individual merging the pull request is responsible for rebuilding them and committing the changes.

Follow the usual conventions when submitting ([Chapter 51](#) and [Chapter 53](#)), and raise larger ideas, such as a brand-new theme, on the pretext-dev discussion group first.

Chapter 50

Contributing Localizations

We rely heavily on those fluent in other languages to provide translations of terms like Chapter and Example. See [Section 4.40](#) for more about this feature. This chapter is meant to help you get started if you would like to contribute to PreTeXt development in this way.

Some notes:

- We know it is hard to translate subtleties of one language into another. Do the best you can! Something is better than nothing. And somebody may come along later with an improvement. See the en-US file for explanations, this is the only documentation about each term that gets translated.
- Do not copy the documentation from the en-US file into your file. Add comments if you feel your work needs some explanations. Do try to keep the order and organization the same.
- When we add new features, sometimes new translations are needed. We appreciate maintainers who regularly come back to add in more. But anybody can hop in and add new translations to an existing file. We try to be careful to have all the “missing” translations commented-out so it is easy to see what needs work.
- Explore the files in `xsl/localizations`, including the README file, which should provide you enough to get started.

Thanks in advance for continuing the great work that allows us to support readers, authors, and publishers in many parts of the world!

Chapter 51

Code Style

51.1 Indentation

Indentation of XSL, and indentation of Python, is four spaces per level of indentation. Please do not be stubborn and suggest that you should be allowed to behave differently. With many contributors it is more important that every developer does it the same, than for one developer to suggest that their personal preference is better. Oh, and never tabs (see [Section 53.4](#)).

51.2 Whitespace

When a change has a lot of whitespace changes, or trivial re-formatting, please isolate those on a separate commit. Then suggest to the committer that the full 40-character hash of the commit (once finalized) be added to the `.git-blame-ignore-revs` file to keep `git blame` functional. To learn more, see [Rob Allen's post](#), and [Section 53.8](#).

Chapter 52

Debugging

52.1 Debugging Parameters

There are a variety of command-line string parameters ([Section 28.1](#)) for use by developers testing new code or diagnosing problems. As of 2021-02-14 we are collecting their definitions in the `pretext-common.xsl` stylesheet early on. Search on `debug.` to find them. Elsewhere the names might be a bit more variable until we get them all rounded up in one place, but usually the name contains `debug` somewhere. No documentation here, these are not documented (or stable), and only intended for authors as a stop-gap measure before being replaced in function. Read the code for more, while some more permanent versions will be mentioned in this chapter.

52.2 Testing Procedures

When developing a new feature or trying to fix a bug, it can be helpful to test a “before” version of some output format versus a current or “after” version. Sometimes you can lay the groundwork for a new feature by adding some code which has zero effect, but leaves a place where a small addition enables the feature. Then you can be somewhat confident your additions will also not ruin existing (desirable) behavior. Here are some suggestions for doing this with PreTeXt.

- Make a “before” version before you start. Or do a `git stash save` to put away uncommitted changes and then make that initial version. Use `git stash pop` to bring the changes back. You can also temporarily `git checkout 1234abcd` some commit, and `git checkout topic-branch` to come back. You can have `sample-article.tex.old` around to compare with a new and constantly updated `sample-article.tex`. I routinely remove `*.tex` between tests, so the `*.tex.old` version is named so it does not get deleted.
- The `sample-article.xml` document is meant to have one of most everything and lots of extreme examples. If your changes affect chapters, and other aspects of a book, then the `sample-book.xml` document can also be used.
- Be careful about adding new content since you can get a lot of automatically-generated id’s that change, making it hard to see real changes. By managing location (early in a list of commits) and building at specific commits you can test new code that needs new sample uses.
- $\text{\LaTeX}$ output is one big file. Use the `debug.chunk` string parameter, set to `0` to get one big HTML file.
- Use the `debug.datedfiles` string parameter set to `no` so that file headers (knows!) do not all have spurious changes.
- `git diff sample-article.tex.old sample-article.tex` works well, and oftentimes it works even better with `--word-diff`. You really want your test output off in some scratch directory. If not, be careful, and you may need the `--no-index` option so you are not fooled by no apparent changes. (Those options need a double-dash to lead, they look like one right now.)

- Having `<today/>` and `<timeofday/>` in your test document you can be sure there is always at least one or two changes, and prevent some confusion due to mistakes.
- When testing HTML output, especially for chunking or knowls, putting two versions in different parallel directories will allow git to compare the entire directories with `git diff /tmp/before /tmp/after`.
- Every number is hard-coded in HTML output, so changes to numbering are best tested there. The $\text{\LaTeX}$ conversion may react differently (a mix of hard-coded and automatically-generated numbers), so should also be tested.

It can be very surprising how many subtle bugs can be revealed by very small, or very few, changes discovered by these procedures.

Chapter 53

Git

53.1 Getting Started

Todo: preliminary list as of 2019-04-26:

- Install git.
- Learn to use git at command-line, front-ends will ruin your life.
- GitHub is not git, but we use it to advantage. Explain the difference.
- Point to Git for Authors.

53.2 Commit Messages

Commit messages are a critical tool when locating and reviewing changes to the project. In a way, they are like an index of a book. We take great care to have useful and concise commit messages, and to this end, we are likely to edit yours. But you can help by doing some simple things, so we do not have to.

- Begin with a capital letter.
- Do not end with a period.
- Do not repeat information available by examining the commit itself. Bad: “Changes in foo.xml”
- No longer than roughly 60 to 70 characters, and never onto multiple lines.
- You can find lots of advice about phrasing commit messages with action words, or in ways that describe how the code will be different. These make for good reading, but we do not adhere slavishly to any one formulation.
- For whatever reason, we isolate and prefix messages with the name of a relevant conversion such as “HTML: ”, or other areas, such as “Guide: ” or “Schema: ”. Scroll the list of commit messages to see the range.
- For pull requests we will append “(PR #nnnn)”. (This provides a permanent record somewhat independent of GitHub, and GitHub will utilize this number.)

53.3 Whitespace

Whitespace refers to those pesky characters that you really cannot see visually, but which are definitely (different) bytes in your source code. Spaces, non-breaking spaces, tabs, newlines (carriage return? Enter?). Here is the problem. Somebody else left a blank space at the end of a line mid-sentence, perhaps because they manually added a newline there to make a line shorter and more readable (especially if it is code). Now you open that file with your editor, which is set to strip trailing whitespace from lines. Maybe you make an edit, realize it was not what you needed and remove the edit. The rest of your work is off in some other file. But now the original file has a single one-character deletion, and that will be carried forever in the git history as a change you made to the work of the previous manual-line-breaker.

Solution: the original space introduced at the end of the line in mid-sentence should never have been part of a commit, and should never have been merged in the first place. The fault is really not yours, and having your editor strip trailing whitespace is probably a good thing. But if you find these sorts of mistakes, they should be on a commit of their own, clearly labeled as fixing whitespace (see [Section 51.2](#)). But do not be the person who introduces the spurious whitespace in the first place! We will likely ask you to clean it up before your pull request is merged.

Whether or not a newline is the last character of a file seems to be a common “correction” that editors make routinely. Let us say that a newline should be there, so feel free to add such newlines (on its own commit).

When I do `git diff` at the command-line, extraneous whitespace (trailing characters, blank lines that are really not at all blank) are shown as bright red rectangles. Can’t miss ’em. Find a similar tool that works for you. Or `grep` with a pattern like `$` (that’s a space and a dollar-sign.) And learn to configure your editor to do the right thing routinely (and no more).

53.4 Tabs

A **tab** is the worst sort of whitespace! (See [Section 53.3](#).) There should never ever be any tabs present in any contribution to PreTeXt. The problem: the “width” of a tab is different for each person who opens a file. Two spaces, four spaces, eight spaces, you name it. So if indentation is accomplished with a mix of tabs and spaces, there is a good chance it might look right to you, but looks wrong to many other developers. I hit the tab key on my keyboard all the time. But my editor is set to supply *four spaces* as a result. See if your editor will behave similarly.

53.5 Reviewing Pull Requests

A **pull request** (PR) is a way that developers suggest and contribute new code to a git repository. The *next* section describes how to make one, and might be interesting reading after you finish *this* section (but not now!). Here we will describe how you can test out a pull request, in order to help with evaluation, or if you are simply curious.

You will need a **clone** of the repository where the pull request resides. If you have a fork (see next section) that is fine, too. We will illustrate with a real PR, [GitHub #2096](#).

1. At a command-line, with a working directory at the top-level of the pretext repository, issue

```
git fetch origin pull/2096/head:andrew-pointer-css-2096
```

- `git` is the command-line git executable.
- `fetch` means we are going to grab a collection of commits from somewhere. This is not a `pull` and that will be incorrect here right now.
- `origin` is the somewhere, a repository that has the commits we want. This name translates to the repository hosted on GitHub, which is where you obtained your clone or fork. This is known as a **remote**, and you can have several with different names that you have assigned to them. `git remote -vv` will list all your remotes for you.
- `pull/2096/head` locates *all* the commits for PR #2096. Don’t let the use of `pull` fool you.
- `andrew-pointer-css-2096` is a pointer, internal to my clone, that helps me identify the PR, especially when I have many in play at the same time. In git it is known as a **branch**. Andrew wrote it, I have a

reminder of the topic, and I find it helpful to have the actual identifying number around. You are free to do anything you like here for a name of the branch—whatever works for you.

2. Now you have one, or more, commits on your system, and a name that helps you locate them as a branch. Presuming you are on your main branch (master here, but possibly main) you will want to switch to the branch with these new commits as a putative change to the code for you to experiment with. Easy:

```
git switch andrew-pointer-css-2096
```

3. Now you will want to use the pretext/pretext script ([Chapter 47](#)) to produce whatever output you want to test or examine. The point is, you now have the proposed modifications available to you for use.
4. Optional. The PR could be several days old, or maybe it has been weeks or months. So it may be a branch off the state of the code from the past, and the code has moved on and evolved (through accepted pull requests). While you have andrew-pointer-css-2096 as your active branch you can go:

```
git rebase master
```

This will move the branch to be a deviation from the most recent version of the code. (I say to myself as I type, “git rebase ONTO master.”) It is possible this is not what the original author of the PR intended, but for PreTeXt this is rarely an issue. It is also possible that the movement creates contradictions (**conflicts**). Don’t panic if this happens, just go:

```
git rebase --abort
```

and it will be like it never happened.

5. When you are finished, return to the master branch with

```
git switch master
```

and then delete the branch with

```
git branch -d andrew-pointer-css-2096
```

Aah, git will not let you do that. It is not clear if this might be important work and you really should not be deleting it. git cannot tell if this is something original you just created or something that is just a copy of what is safe and sound on GitHub. So, go:

```
git branch -D andrew-pointer-css-2096
```

using the semi-dangerous -D flag. Now your clone is back to its original state.

This is the procedure described on [StackOverflow: How can I check out a GitHub pull request with git?](#). On an open pull request within GitHub you can find “command line instructions.” These suggest making a (empty) branch in your repository, then doing a pull *from the fork belonging to the author of the pull request*. This means the second step will be very different every time since it uses the fork and also a branch name devised by the pull request author. Functionally equivalent, but we find it more complicated.

When you learn more about the use of git you will discover there are many additional things you can do to modify and experiment with pull requests. Here we are concentrating on the first step: getting a pull request onto your system so you can employ it.

53.6 Creating Pull Requests

Contributions to the code repository are accomplished with a **pull request**. The short version is that you have a copy of the repository and you make a collection of changes on your copy. Then you make a request to have your changes “pulled into” the official (main, canonical) repository. A pull request is a concept independent of GitHub, but one of GitHub’s advantages is that it makes a pull request very easy to create and manage.

One-Time Initial Setup.

1. Make an account on [GitHub](#) if you do not have one already. A username that bears some correspondence to your real name or favorite email name is helpful.
2. Log into your (new) account.
3. Go to the official repository for PreTeXt. Find a button labeled Fork, roughly in the upper-right corner. Click on it. This will make a copy (a **fork**) of the official repository in your GitHub account. This repository is called a fork since you are going to make improvements there and your version will diverge from the official version. The fork will “know” where it came from.
4. Install a command-line version of Git on your local computer. Heed the advice above about using front-ends.
5. You are now going to make a local copy of your fork. Think of it as a mirror—you will do your best to keep the copy and the fork in-sync. Git calls this a **clone**. In your fork (i.e. in your account, find a green button partway down the right side. Clicking on it will bring up a textbox with a URL you can copy. Now at the command-line, execute something similar to

```
git clone https://github.com/mjsmith/pretext.git
```

Your clone will also “know” where it came from.

6. That finishes setup. You can check that all is well by running

```
git remote -vv
```

and the response should be something like

```
origin      https://github.com/mjsmith/pretext.git (fetch)
origin      https://github.com/mjsmith/pretext.git (push)
upstream    https://github.com/PreTeXtBook/pretext.git (fetch)
upstream    https://github.com/PreTeXtBook/pretext.git (push)
```

origin is an alias for the location of the repository you cloned. And upstream is an alias the fork uses to know the location of the official repository.

Preparing Your Changes. The following all happens on your local computer, using your clone, at the command-line.

1. Create a **branch** for your work, and switch into it.

```
git checkout -b my-big-improvement
```

The name you choose will not ever be part of the official repository, but it will be part of the record on GitHub. So you do not have to be too careful, but it should be informative.

2. Use a text editor to make changes to existing files, or to create and populate new ones. As you save the affected files, you can type

```
git diff
```

to see the changes to existing files. See the rest of this guide for particulars about the code.

3. When finished, you will package up your changes as a **commit**, the fundamental unit of a git repository. Throughout this process (and at any other time), you can type

```
git status
```

to see how your repository is changing.

If you have created new files, you need to stage them. You can see these files’ status changing if you run `git status` before and after. To stage a new file,

```
git add xsl/pretext-esoteric-format.xsl
```

You do exactly the same thing for existing files you have changed. Run `git status` before and after.

```
git add xsl/pretext-common.xsl xsl/pretext-latex.xsl
```

Running `git status` should now show that all affected files (changed, new) are now in the staging area, and no files with changes are left behind. You can preview the commit with

```
git diff --cached
```

If you need to edit some more, go ahead, and be sure to add your new changes into the staging area. Now you are ready to make your commit.

```
git commit -m "Create a new conversion to an esoteric format"
```

Now `git status` should show something of a clean slate. You can also run

```
git show-branch
```

to get a pictorial version of your branch.

Creating the Pull Request. Now you will communicate your changes (on a branch on your local computer) to GitHub as a request for incorporation into the official repository.

1. First, **push** your branch to your fork on GitHub. Recall that this repository is known as `origin`. On your local computer, at the command-line,

```
git push origin my-big-improvement
```

2. Now move to your web browser and your fork on GitHub, which now has a copy of the `my-big-improvement` branch. You should see a prominent message about your new branch, and a green button labeled `Compare & pull request`. Click on it.
3. Now you have a screen titled “Open a pull request”, where you can describe the purpose of the new code. Then click on the green button labeled `Create pull request`.

That’s it. The developers responsible for approving pull requests will be notified automatically and receive your code in a way they can test and review it in their own forks/clones of the repositories. You can see the pull request in action at the appropriate area of the official repository. Pretty slick.

Pull Request Verification. It is often advisable to add an example of any new markup, new situation, or bug-provoking content into the sample article as part of a pull request. Please do so, as appropriate. However, because we do rolling releases, all code gets thoroughly tested. For this reason *please* put all changes to the sample article, and only changes to the sample article, onto a first commit. Code should then follow on subsequent commits. This makes a big difference in our ability to test quickly and accurately. Thanks. (Ask if you need help rearranging your commits to achieve this, or see [Section 53.7](#).)

Modifications to a Pull Request. To Do: describe how a pull request might iterate to approval/merge.

Cleaning Up.

1. At any time after pushing your branch to your clone you can/should switch to the default branch (`dev` now, but changing to `master` later).
- ```
git checkout dev
```
2. Your pull request ends when the lead developers **merge** your branch into the main branch that everybody uses. The commit will have your name on it, as part of the permanent record. But the commit may have changed slightly between initiating the pull request and its subsequent merge. You will want to remove your original branch from your clone on your local computer.



```
git checkout dev
git branch -d my-big-improvement
git branch -D my-big-improvement
```

The second command will fail, as a safeguard against deleting branches with temporary (but important) work on them. The capital “D” is a “forced deletion” so should be used with care! But it is the right thing to do here, since your work has been incorporated into the official repository.

3. But, of course, you want your new improvement like everybody else. So you are now going to pull it from the official repository into your clone on your local computer. Remember that the official repository is known as upstream.

```
git checkout dev
git pull upstream dev
```

4. Technically, you could now totally trash your fork (making your clone disconnected), and make a new fork and clone for your next contribution. Instead, you can sync your clone with the fork.

```
git checkout dev
git push origin dev
```

Now all three repositories (clone, fork, official) look the same and have your contribution. Before your next contribution you will want to pull from upstream into your clone, and then push that into origin (your fork).

## 53.7 Forming Logical Commits

There is an art to making a pull request that is easier to review, and which will be useful to others later (such as when using `git bisect` to isolate the introduction of a bug). Here are some notes:

- Always rebase your sequence of commits onto master before creating a pull request. Any (rare) conflicts should be your responsibility. If we delay in getting to a review, then maybe conflicts are our responsibility.
- Make logical commits. Changes to common templates, HTML-specific templates,  $\text{\LaTeX}$ -specific templates, etc. should all be on separate commits. Contributions to the Guide, and examples for testing in the sample article, should follow (even if testing examples may have been an early commit on your branch during development). Make schema changes *last* since it will be easier for us to manufacture derived files as an add-on to your work.
- Do not put partial work on two disjoint files into one commit, come back later, add a second commit with more work in each file, and call it good. Likely there should be two commits—relevant code in one file, related code in the other file. This is a general suggestion: a stream-of-consciousness commit history is of no benefit to anybody, even you.
- Do not make a mistake (typo, whitespace, logical error) in one commit, and then fix it several commits later as part of the same pull request. If you made a mistake, learn how to make the change/fix so it looks like it never happened. You do not want other developers to think you make mistakes, do you?
- Done right, a pull request with no changes will be merged as-is with no changes to the commit hashes. Consider that a goal, and we will congratulate you when it happens the first time (and expect it from then on!)

Alright, those are high expectations. How do you make a well-formed sequence of commits? This is not a git tutorial, but we will make some suggestions. git has what is called a **staging area** where you can gradually place a collection of changes before making them part of a single commit. The command `git commit -a` is a bad habit and breaking yourself of it will help you learn to be more flexible about how you package changes into a commit via the staging area. Finally `-m` is a useful option for making (or changing) a commit message without being thrown into an editor.

- If your most recent commit (or only commit!) is lacking you can add new changes into it by adding them to the staging area and using `git commit --amend` to introduce them into the commit.

- `git reset HEAD~n` will return your files to a state as if you have made no commits (presuming you had *n* of them in the first place). Your edits will all be available (this is *not* a “hard” reset, and forget that we even mentioned such a possibility). Then you can selectively stage portions of your work with tools like `git add <file>` or `git add -p` and build up individual logical commits in the staging area. Of course, you do not always need to reset all of your commits on a branch, perhaps only a few will need reworking. Caution: do not reset so many commits that you blow past branch pointers and lose them, such as `master`.
- You can do an interactive rebase with `git rebase -i HEAD~n`. This unwinds *n* commits on the current branch, makes a little script for replaying them in the proper order, and throws you into an editor with the script. Exit the editor and the script runs. With that description/process nothing interesting happens. What is interesting is that you can edit the script to affect the replay.

You can rearrange the order of the commits. But this only works if you know that interchanged commits do not build on one another. For example, I often start developing a new feature by designing the PreTeXt markup and making an example in the sample article. But once I am all done, I move it to be *later* than the code, as an example of how the new code will behave (and I do not leave the sample article in a broken state).

Suppose you have ten commits on a branch, and you discover that the third-newest has a small mistake. Correct the mistake and make a throw-away commit with just that correction on it. Now, do an interactive rebase with the newest *four* commits (you just added one), `git rebase -i HEAD~4`. Edit the script to place the throwaway commit just after the commit with the mistake, and do not leave it as `pick`, but edit that action to `fix` (an `f` is all you really need). Be sure to remove the line that has the original version of the throwaway commit. Exit your editor. Poof! Three commits and the mistake is corrected. (Do not use the `squash` action, that accumulates commit messages, which we do not want.)

This is analogous to `git --amend` except it is needed for the times when you see a change that needs to be made several commits ago.

- The `git cherry-pick <commit>` command allows you to recycle existing commits onto new branches. Suppose your current branch ends up having two very different projects on it. Make a new branch and cherry-pick the commits for one project onto it. Now do an interactive rebase on the original branch and remove those commits from the script. Of course, this assumes you know the two sets of commits are independent of each other.

Here is a different solution to the same problem. Do a `git reset` of the entire branch, as described above. Build up the first project into a set of commits. Now do `git stash save` to put all the remaining edits for the second project into the **stash** temporarily). Switch to `master`, make a new branch, `git stash pop` to get all the edits back, and start building up the second branch. Note that this assumes the second project does not build on the first project.

- Be bold! But maybe make backups first? You can often abandon things that are not going well. If `git rebase master` has conflicts you can look around at the files affected, and then you can give up with `git rebase --abort`. There are other bail-outs for other commands.
- The commands `git log`, `git diff`, and `git status` are your friends. It is never a mistake to use them more than necessary.

## 53.8 Blame

The command `git blame` is simultaneously funny, useful, and powerful. A line of code you do not understand can be traced back to its originating commit. We maintain a list of commits (in `.git-blame-ignore-revs`) that have no real new code and make just cosmetic changes, so you may want to use this feature. To use:

```
git blame --ignore-revs-file .git-blame-ignore-revs birds.xml
```

You can also make this happen every time with a local configuration change, to wit:

```
git config blame.ignoreRevsFile .git-blame-ignore-revs
```

To learn more, see [Rob Allen's post](#), and [Section 51.2](#).

# Appendices

## Appendix A

# Welcome to the PreTeXt Community

Thank-you for your interest in PreTeXt, and welcome! This appendix is meant to answer some questions you may have about how this open-source project is organized, how you can get help, and how you can contribute back.

PreTeXt is not  $\text{\LaTeX}$  and it is not Word. Some “features” of those languages are intentionally missing, but more importantly, the mind-set expected of authors is completely different. One of the most important distinctions is that PreTeXt treats the author, the publisher, and the reader (which might be an instructor) as separate entities—even when those are all the same person.

Newcomers to PreTeXt should focus on the author role. Make sure the structure of your book is marked-up properly. If something looks wrong in your output, assume it is a problem with the source markup. If the source is correct but the output does not look as you wish, leave that as a problem for the publisher, to be addressed *after* you have finished writing the content.

## A.1 Help and Support

There is a documentation area at the project website. Presumably, that is where you found this *PreTeXt Guide*. This is what another software project might call the *User’s Manual*. So start here. Re-read [Section 1.1](#) on the project philosophy and the principles in [List 1.1.1](#) at regular intervals. [Chapter 3](#) is meant to inform you of the features of PreTeXt, without getting into all the details. It will frequently refer you to [Chapter 4](#) for all those details.

PreTeXt is fundamentally a specification of a set of elements and attributes, a topic discussed in [Chapter 6](#). This chapter also discusses validation. Once comfortable, but before authoring lots of material, take the time to get validation working, and use it regularly. Do not save it for last.

There are many examples available in an area on the website. Compare PreTeXt source to the resulting output (in both directions). The “sample article” is not always pretty, since it is used for testing, but it does try to have one of everything.

When the above is not sufficient, the pretext-support Google Group is the right place to ask questions. If you are trying to determine which elements to use to accomplish something, provide some context. Do not ask, “How do I print a line of text upside-down?” Instead say, “I am writing a monograph on mammalian vision, and I’d like the reader to use a mirror to view a line of text written upside down. What is the best way to do that?” (I think the answer would be: make an image and include that, rather than trying to get reflected text.) Sometimes a quick search of the Goggle Group may yield insights. Overall, do your best not to be a [Help Vampire](#).

## A.2 Weekly Drop-Ins

Since May 2020 there has been a standing weekly online gathering of PreTeXt authors and developers on Friday afternoons. And now there is a companion session on Tuesday afternoons, and a Getting Started session on Mondays. A weekly schedule of these, and other activities, is posted to the pretext-announce Google Group, so look there for particulars. Once in a while there are scheduled discussions on development topics or seminars on topics of interest to authors and developers (such as building an index, see [Subsection 4.27.2](#)). But mostly there is no agenda and

no schedule. Come late, leave early, drop-in, drop-out. If you are present, then you are implicitly available to help someone with questions, even if you are working on something yourself. The rest of us may lurk and/or join in. We can use break-out rooms if a subgroup wants to run a discussion, work on a targeted project, or help somebody get started one-on-one. The model is the hotel lobby after dinner at a workshop, but you provide your own contraband.

So this is a great place to discuss a support question (likely with the person who wrote the code!) or a place to get involved with development.

## A.3 Feature Requests and Reporting Problems

We use the **issue tracker** at the GitHub site as an organized to-do list. You do not need to know anything about `git` to use this forum. Just make a new issue, or make a comment on an existing issue. Frequently, a discussion on the groups will culminate with the creation of a new issue. It is nice to have a link from the discussion to the issue, and vice-versa.

If you are asked to create an issue in response to a discussion you initiated, please consider doing so. It will save the other volunteers just a bit of time to work on other parts of PreTeXt. And you will also get email-for-life as the issue is discussed and eventually closed. It is a very helpful contribution.

As you gain more experience, you will identify bugs, obsolete instructions, typos, etc. Search the issues to see if there is something relevant you can add. For example, your particular version of a bug might provide the key insight into identifying the cause. When you are certain something is wrong, and there is no need to discuss it in the groups, feel free to go straight to making an issue. For something like a list of typos, make a single issue and just keep editing your initial post.

If you have some structural problem, see if you can reproduce it by adding into the minimal example, and post that *entire* source file. If the rendering in HTML is a bit off or you think it exhibits a bug, be sure to post a link to a *live* example, not just a verbal description and *definitely not* a screenshot. If you need an easy, quick, free, temporary location to host one file or your whole project as HTML, try [Netlify Drop](#).

If you are posting code snippets, and copying out of an editor using Dark Mode, try `ctrl-shift-V` when you paste and perhaps the Dark Mode color scheme will go away—making it easier for some to read.

## A.4 Contributing

As a project that is licensed openly, we welcome contributions. And this does not necessarily mean you need to learn our primary language, XSL. For ideas, find the issues on GitHub that have the label `contributor project`. As one example, it would be very helpful if a member of the community would create and maintain a Wikipedia page. That is a skill that is very distinct from the other skills used to create and maintain other parts of the project. See [Issue #207](#).

The documentation is authored in PreTeXt, so you know how to create additions, clean-up obsolete parts, and fix typos. If you know `git` and GitHub, then a pull request (on a new branch!) is a very economical way for us to manage contributions. Even if you do not know GitHub, we can easily accept files written in PreTeXt that contain changes to the documentation. For example, files supporting localization to new languages (see [Section 4.40](#)) are ideal for simply submitting an entire file. (Send files to us by email, or post as attachments on one of the Google groups.)

Conversions are written in XSL, a declarative language. It has a steep learning curve, but is very powerful for an application like this. Start small, and we do not mind helping you along with suggestions and critiques. Do not, do not, begin an ambitious task unless your skills are up to it.

When you have gained significant experience as an author, and have a good feel for the questions asked by other authors due to your material participation in the pretext-support group, then it may be time to apply to be part of the pretext-dev group. This where design discussions are held and nasty bugs are squashed collectively.

## A.5 Personal Email

How do we put this politely? Personal email to the core PreTeXt developers should be your last resort. We are not unfriendly—just the opposite. We would love to hear from you, but in the groups. Here is the rationale:

- You may get a better answer from somebody who is not the most active developer, but understands your particular need better than anybody else. You will never get that answer with a personal email.
- Developers teach university courses, travel to professional meetings, sleep at night, take naps, turn off their email for big coding pushes, and sometimes travel in the wilderness and are offline for days at a time. You are likely to get quick responses from the core developers through the groups, but if they are not available a personal email may get a slower response. (I am inclined to answer posts on the groups before I work through personal email.)
- Many contributors prefer to provide help in a public forum because their efforts are then more widely recognized.
- Your question and its answer are searchable by others. (The groups and issues are public.) A personal email is no help to anybody else.
- We prefer to make as many decisions as possible *openly*. So a discussion on a public group or site is there for all to see, now and later.
- We depend on granting agencies for much of our funding. Membership in groups, forks on GitHub, activity in the groups, number of issues, and number of contributors to the repository, are all crude measures of the health of the project. Personal emails add nothing to those measures.
- Everybody who has committed their big writing project to PreTeXt likes to see an active, responsive, friendly community supporting its use and growth. Just by asking a necessary question, you can add to that community.

We understand that nobody likes to pop their head up and ask a “stupid question.” But it is counterproductive to do personally what we can do better collectively. The groups are friendly forums (we will enforce that if we ever have to) and everybody there made an initial post once. And the group members largely enjoy sharing their advice, experience, and knowledge. So, please make a contribution simply by saving the personal emails for that which is really personal. Thanks, and we’ll look forward to chatting with you on the groups!

# Appendix B

## Best Practices

|                       |                                                           |
|-----------------------|-----------------------------------------------------------|
| Best Practice 4.1.2   | A CDATA Section is Never Necessary                        |
| Best Practice 4.1.8   | Understand the Importance of Careful Markup               |
| Best Practice 4.5.1   | Use <code>@xml:id</code> Frequently                       |
| Best Practice 4.5.4   | Take Care Referencing Anonymous Lists                     |
| Best Practice 4.5.7   | Be Rational About Numbering Variations                    |
| Best Practice 4.8.1   | Provide Informative Titles Liberally                      |
| Best Practice 4.9.1   | Keep Inline Mathematics Short                             |
| Best Practice 4.9.3   | Authoring Punctuation after Mathematics                   |
| Best Practice 4.11.3  | Use Only a Few Columns for Lists                          |
| Best Practice 4.12.1  | Use <code>label</code> attribute for Runestone Components |
| Best Practice 4.14.1  | Build 3-D Asymptote Figures                               |
| Best Practice 4.14.8  | Preferred Image Formats                                   |
| Best Practice 4.19.1  | Vertical Rules in Tables                                  |
| Best Practice 4.19.2  | Tables are Difficult                                      |
| Best Practice 4.25.2  | Understand the Role of a Preface                          |
| Best Practice 4.27.1  | Capitalization of Index Entries                           |
| Best Practice 4.32.1  | Craft URLs Carefully                                      |
| Best Practice 5.2.1   |                                                           |
| Best Practice 26.3.1  | Use Chapter Zero Carefully                                |
| Best Practice 30.6.1  | Avoid Too Much Text on a Printed Page                     |
| Best Practice 30.18.1 | Only Edit $\LaTeX$ Files Rarely                           |
| Best Practice 49.4.1  |                                                           |

# Appendix C

## Python

### C.1 Python Executable

While the majority of the processing of your text is accomplished with xsl transforms, we rely on Python for numerous other tasks and conveniences. So, sooner or later, you will want, or need, to get Python working on your system. Part of our attraction to Python is that it is very popular and so should run very similarly (identically?) on a variety of systems: Linux, Mac, and Windows. So the quicker we can get you to a platform-neutral setup, the better.

On Linux and Mac, you will likely have at least some version of Python already installed. On Windows, you may need to explicitly install it. (We hope to provide some more careful advice here for Windows users. For now, see [Section M.4](#) and [Appendix N](#).)

Python 2 is no longer being developed and is officially dead. That does not mean you do not have it on your system. And for older systems, it may well be the default. PreTeXt requires at least Python 3.8.5 (current as of 2022-08-31). At a command-line, or in a terminal or console, or at a command-prompt, try

```
python --version
```

and if the result is unacceptable, then try

```
python3 --version
```

Remember which variant you need—we will just routinely use the generic python.

### C.2 Python Virtual Environment

There is a very good chance your system has a version of Python installed already since various other programs rely on it. You will be much happier if various Python programs you need for PreTeXt do not get entangled with your Python programs installed as system software. And if we need to help you debug some related problems we will ask you to work in a virtual environment anyway. We will follow a [primer](#) that has been recommended by Brad Miller, founder of Runestone Interactive. See that page for more details.

These are condensed instructions without much explanation.

1. Do not install the old-school virtualenv module.
2. Make a directory anywhere you like named `virtual-python` and change into it.
3. `python3 -m venv ptx` makes a virtual environment named `ptx`. (Requires Python 3.6 or later.)

Done. But how do you use it?

1. To activate your virtual environment, be sure you are in the `virtual-python` directory. For Mac or Linux, with a bash shell go



```
source ptx/bin/activate
(ptx) $
```

There are other scripts for other shells, take a look around.

For Windows, go

```
ptx/Scripts/activate.bat
(ptx) >
```

Notice that your prompt has changed to remind you that now anything you do with Python will be “sandboxed” in this virtual environment and not get mixed up with your system Python.

2. Now you can do any Python-related task using your virtual environment, and can change directories if desired.
3. When you are finished with your virtual environment, on Mac or Linux with a bash shell you simply go

```
(ptx) $ deactivate
$
```

For Windows, similarly

```
(ptx) > deactivate
>
```

We have one report, on a Mac, of deactivation dropping you into a different virtual environment named (base). A way to leave that virtual environment is to run

```
(base) $ conda deactivate
```

This appears to be due to Anaconda somehow creating a default base environment initially. If you have this experience and find a simple solution, please report it. Some details at [Stack Overflow 54429210](https://stackoverflow.com/questions/54429210) may be helpful.

## C.3 PIP Install

PIP, in the open source tradition of recursive acronyms, stands for “PIP Installs Packages”. This package manager helps you obtain software and keep it up-to-date. There are three packages that are indispensable for use with PreTeXt so you can practice doing installations now via your virtual environment.

If you have installed the PreTeXt-CLI already using pip, then all the programs below are already installed. Otherwise (for example, if you are using xsltproc) you will need to install the following manually.

To install all required additional python libraries, in your virtual environment, navigate to the pretext directory and run

```
(ptx) $ pip install
```

You can also install individual libraries, as in

```
(ptx) $ pip install lxml
```

With lxml, you have a collection of Python routines that interface with the same base libraries for xsl processing as the xsltproc executable. A second library is requests which moderates communications with online servers and is necessary to communicate with WeBWorK servers and with a YouTube server that provides thumbnail images for static versions of videos. The pdfCropMargins package provides a tool that will crop images during their production with the pretext script. The pyMuPDF library then converts the PDF that was cropped to SVG and PNG images. Finally, playwright uses a Chromium headless browser to take static screenshots of interactive elements of your project.

Note that right after you install playwright then you want to run

```
playwright install
```

one time only. This will install three web browsers (Chromium, Firefox, WebKit) and FFmpeg where they should only be available to this package.

**Table C.3.1 PIP-installable Python Packages**

| Package        | Purpose                                       |
|----------------|-----------------------------------------------|
| lxml           | XSL processing                                |
| requests       | Communication with online servers             |
| pdfCropMargins | Crop images during production                 |
| playwright     | Automatic screenshots of interactive elements |
| pyMuPDF        | Convert images to SVG and PNG                 |

# Appendix D

## Text Editors, Spell Check

This appendix has information about using various text editors efficiently with PreTeXt source, along with suggestions for spell-checking. The choice of an editor that suits you is a big part of being a productive author. We are partial to Visual Studio Code because it is available free, is open-source, has a range of features, has community support (plug-ins), and has cross-platform support (Linux, Mac, and Windows). So we lead with Visual Studio Code, but also include Sublime Text, Emacs, XML Copy Editor, and vi/vim. A summary table of schema-aware editors can be found at [Section 6.7](#).

### D.1 Visual Studio Code

**Oscar Levin**

VS Code is a free and open source, cross platform text editor from Microsoft. The package [PreTeXt-tools](#) provides highlighting and snippets for PreTeXt by extending the XML language support of VS Code.

### D.2 Sublime Text

**Dave Rosoff**

Sublime Text is a fast cross-platform editor with thousands of user-contributed packages implemented in its Python API. It is not free or open-source, although most of the user-contributed packages are both. Development is active as of December 2024. However after build 4180 of Sublime Text 4, the PreTeXt support is broken.

Here we outline several of the most important Sublime Text features that will help you to minimize your typing overhead and work more efficiently with your PreTeXt project. We also introduce the no longer maintained PreTeXtual package designed to help PreTeXt authors work more efficiently.

Sublime Text 2, 3 and 4 are available for an unlimited evaluation period, but a license must be purchased for continued use.

#### D.2.1 Settings

Sublime Text settings are stored and managed in a collection of JSON files as key-value pairs, in files that have a `.sublime-settings` extension. You change the settings by visiting these files and editing the values away from their defaults.

To edit your Sublime Text settings, you can use the Preferences/Settings — User menu (Sublime Text/Preferences... on OS X). Make sure that when you go to edit Settings, you always choose the User option. Changes to Default settings files will be overwritten when Sublime Text updates. It is recommended to use the Default files to see what settings are available to change. There are a lot, and not all are documented.

All Sublime Text users should be aware that a particular view (buffer) may receive settings in several different ways, e.g., from global default settings, from global OS-specific settings, from package-provided settings, from user-provided settings, and so on.

Key bindings are also stored in files with a similar format. There are only a limited number of keyboard shortcuts available, although Sublime Text does support multi-step shortcuts like Emacs. If you find that you wish to reassign shortcuts, this is certainly possible through the Preferences/Key Bindings — User menu (Sublime Text/Preferences... on OS X).

## D.2.2 Package Control

Sublime Text's Python API exposes a lot of the Sublime Text internals to plugin and package authors. Packages extend Sublime Text's functionality, much like Emacs major modes. A package usually consists of some Python scripts that define Sublime Text events and actions, some text-based configuration files (XML/JSON/YAML files defining language syntax, symbol recognition, custom snippet insertion triggers and contexts, keybindings for new and old commands, etc.), and perhaps some other stuff too. These typically get bundled into a .zip archive that is disguised with the unusual extension `.sublime-package`. These archives live in the Packages directory, accessible via the Preferences menu (the Sublime Text/Preferences menu on OS X). Sublime Text monitors the Packages directory for changes and reloads all affected plugins on the fly.

The first thing you should do after installing Sublime Text is install the Package Control package. This package manager operates within Sublime Text to automatically fetch updates for packages you have installed (unless you disable this feature). You can also list currently installed packages, find new packages to investigate, remove packages, etc.

Thousands of user-contributed packages are available for easy installation via Package Control. It is possible to maintain packages by hand, since most package authors publish via GitHub, but Package Control is the universally recommended method of obtaining, managing, and removing packages for your installation.

1. Visit the [Package Control download site](#).
2. Find the Sublime Text console command (make sure the correct version of Sublime Text is selected) and copy it to the clipboard.
3. Open the Sublime Text console (Ctrl-`) and paste the command into the window that appears, then press Enter.

Having installed Package Control, you can use the command palette to deploy its commands, such as Install Package, List Packages, and Remove Package. See the documentation for more. A few packages that are especially useful are recommended throughout this section, and summarized in [Subsection D.2.7](#).

## D.2.3 Project Management

Like many modern editors, Sublime Text has good project management features. These allow files that are part of a larger project to work together. For example, Sublime's Goto Anything command allows quick access to any file in a project. The Find in Project command permits users to search and replace (with or without regular expressions) across an entire project. Matches are displayed in a text buffer and double-clicking opens the relevant file at the appropriate position.

The sidebar provides a convenient view of all of the files and directories in a project—or, if you like, a filtered view, where files of your choice are excluded. The MBXTools package ([Subsection D.2.6](#)) also makes some use of project-specific settings in order to provide some of its functionality.

### D.2.3.1 The Open Folder Command

The easiest way to make use of the project management functionality is to store related files in a single directory and its subdirectories. If you then use the File/Open Folder... command, the entire directory is opened and all its subdirectories and files are shown in the sidebar. You can toggle the sidebar with either the command palette or directly with Ctrl+K, Ctrl+B (Cmd+K, Cmd+B on OS X).

By making use of this command you are already using project management, even if you never save your project. Sublime Text always has an implicit project open if you don't open an explicit one. This is good enough for many users a lot of the time, since it provides the most useful feature (Find/Find in Project). The Goto/Go To Symbol in Project command is also useful, but not fully implemented in MBXTools ([Subsection D.2.6](#)). Some of the benefits of explicit project management are outlined below.

### D.2.3.2 Explicit Projects

To save your project explicitly, use the Project menu to choose Save As Project... and choose an appropriate name and location. For a PreTeXt XML project, this would probably be the same name and location as the document root file. Use the Project menu commands to open and close your project.

There are a few benefits to using an explicit project to group files.

- You can group together files and folders in different parts of the file system, instead of being restricted to subtrees.
- You can have project-specific settings that are different from Sublime Text's defaults and different from your user preferences ([provisional cross-reference: subsection-settings]).
- Sublime's project workspaces will remember which files you had open when you last closed the project, and at which positions.
- If you get very fancy, you can have multiple workspaces for the same project, with different filters and views for different purposes.
- It is fine to include .sublime-project files in Git repositories, but .sublime-workspace files should *never* be so included (according to the Sublime Text documentation).

### D.2.3.3 Using the Sidebar

The project sidebar allows you to view the entire directory tree (rooted at the folder you opened with the Open Folder command), or, if you've opened an explicit project as described above, all of its files and folders. You can use the sidebar to copy, move, rename, delete, and duplicate files, for example, as well as opening them.

The package SideBarEnhancements is highly recommended (install via Package Control). It makes the sidebar much more useful.

An alternative to the sidebar that Emacs users especially will find helpful is the [dired package](#). The link is to a git repository since the package is no longer available from Package Control. This package allows you to browse the directory tree in a Sublime Text buffer. You can rename and move files within it—using all your favorite Sublime commands, including multiple selections ([Subsection D.2.4](#)). You might also try the SublimeFileBrowser package, which is actively maintained, available in Package Control, and seems to provide similar functionality.

## D.2.4 Multiple selections

Multiple selections are the single most useful and irreplaceable feature of Sublime Text, the one that will keep you coming back. From the documentation:

Any praise about multiple selections is an understatement.

The base functionality of multiple selections is simple. Hold down the Ctrl key (Cmd on os x), and click somewhere in the open view to get a second cursor. Continue to add more cursors. All of them will behave together when you type: text will be inserted, most snippets or other text commands function as usual, etc. Even mouse commands work in an intuitive way with multiple selections.

It is hard to explain exactly what makes multiple selections so powerful. You just have to try it for yourself. Here is a typical example. In a structured document, many bits of text occur quite frequently—element and attribute names, for example. You may want to update several occurrences of a fragment at once—making several identical changes. Sublime's Quick Add Next command (Ctrl+D/Cmd+D) makes this a snap.

1. Place the caret somewhere in the word you'd like to modify.

2. Use **Quick Add Next** to expand your (empty) selection to the current word.
3. Use **Quick Add Next** again to add the next instance to the selection, which will then typically be disconnected.
4. Continue to **Quick Add Next** as many times as you like. Use **Quick Skip Next** (**Ctrl+K**, **Ctrl+D/Cmd+K**, **Cmd+D**) to jump over instances you would like to leave alone. If you go too far and select in error, hit **Ctrl+U/Cmd+U** to undo.
5. Make your modification, only one time.

Another example that occurs frequently when authoring XML is when you use the **Wrap with Tag** snippet (**Alt+Shift+W/Ctrl+Shift+W**). This snippet wraps the selection(s) in a `<p>` tag, with the tag name highlighted in both the start and end tags. If the `p` element is not what you wanted, just type. Both tags are replaced. This is a huge benefit to the XML author that makes essential use of multiple selections, even though you are barely aware of this as you use the feature.

Column selection allows you to select a rectangular area of a file. This is unbelievably useful when editing a structured document. There are lots of ways to do it ([see the Sublime Text documentation](#) for an almost exhaustive list), but the most frequently used is to hold down **Shift** while clicking and dragging with the right mouse button (on OS X, hold down **Option** while dragging with the right mouse button). See the documentation for keyboard-based shortcuts.

Column selection becomes even more useful when used in combination with the keyboard shortcuts for moving and selecting, such as **Ctrl+Shift+Right** (select to end of word) and **Shift+End** (select to end of line).

Yet another example of the appallingly great utility of multiple selection comes when copying and pasting from a different file format. Suppose you have copied some lines of text and wish each such line to become a list item in your **PreTeXt** source.

1. Use column selection, as described above, to select each line individually.
2. Use **Wrap with Tag** to wrap each of the selected lines with matched begin/end `<li>` tags, all at once.
3. Now you have to select the lines again, to wrap them with matched begin/end `<p>` tags. First, hit **Shift+End** to select to end of line.
4. If your lines are wrapped, you may need to hit **Shift+End** again to get to the end of the wrapped lines.
5. Now you've selected too far: the `</li>` are selected as well. Hold down **Ctrl+Shift** and hit the left arrow twice (unselect by word). (After a little practice, steps like this seem automatic.)
6. Use **Wrap with Tag** to wrap each of the selected lines with matched begin/end `<p>` tags, all at once.

This does take a little mouse-work, but the keystroke savings can be considerable. (The **Emmet** package, described in [Subsection D.2.5](#), provides an even quicker way to do this task and much more complicated ones.)

There are so many incredibly handy ways to use multiple selections that we will forgo any further examples to leave the reader the pleasure of discovering her own favorites. One particularly helpful package is **Text Pastry**, which provides some auto-numbering and text insertion commands that work nicely with multiple selections. There are also a handful of packages that extend multiple selection functionality, such as **PowerCursors** and **MultiEditUtils**. **PowerCursors** allows you to add cursors and manipulate them without using the mouse. **MultiEditUtils** provides additional text processing commands designed to work with multiple selections.

### D.2.5 Emmet

**Emmet** is the most downloaded plugin for Sublime Text (1.82 million installs via Package Control). It is mostly used by HTML and CSS authors and provides a lot of functionality for them. It is also useful for writing XML, as we see below. The main benefits of working with **Emmet** are ease of tag creation, manipulation, and removal.

**Emmet** by default overrides Sublime's binding for the **Tab** key, endowing it with new behavior (the command **Expand Abbreviation**). This new behavior is to create a matching XML tag pair for whatever word is to the left of the caret, or with whatever words are selected. For example, if you were to type `ol` and press the **Tab** key, the resulting text would be

```

```

with the caret positioned between the two newly created tags. Pressing Tab a further time moves the caret to the right of the end tag.

Emmet will produce any word it does not recognize into a matched tag pair when the Expand Abbreviation command is run. Some XML elements are empty, though. Within a matched tag pair, the command Split/Join Tag (Ctrl+Shift+`/Cmd+Shift+`) will contract it into an empty tag, removing any text between the existing begin and end tags. (If the caret is *inside* a tag for an empty element, this command replaces the empty element with a matching begin/end tag pair.)

The default behavior (creating tag pairs whenever Tab is pressed) interferes with Sublime Text's usual Tab-completion, which may be undesirable. It may be disabled by setting

```
"disabled_keymap_actions": "expand_abbreviation_by_tab"
```

in the Preferences/Package Settings/Emmet/Settings — User file. The functionality of Expand Abbreviation will still be available through Ctrl+E.

For a more involved example of abbreviations, suppose you have pasted the items of an ordered list. Now you need to structure it with ol, li, and so on.

Lists are often good.

You can provide list items with <c>@xml:id</c>.

You probably don't want to number them, though.

The desired output is:

```

 <li xml:id="item1">Lists are often good.
 <li xml:id="item2">You can provide list items with <c>@xml:id</c>.
 <li xml:id="item3">You probably don't want to number them, though.

```

Using Emmet, one produces it by executing the Wrap as you Type command (Ctrl+Shift+G/Ctrl+W) and entering the following expression in the minibuffer.

```
ol>li[xml:id=item$]*>p
```

The > symbol denotes a child element, the square brackets (with or without assignment) denote an attribute list, the \$ provides the line-based numbering, and the \* specifies wrapping each selected line with the indicated subtree (so each line is wrapped with <li><p>, instead of the entire selection).

Emmet can produce a large hierarchy of nested XML tags at various levels using this abbreviation syntax. For example, suppose you know that you will need to produce a tag structure of the following form.

```
<section xml:id="">
 <introduction>
 <p></p>
 </introduction>
 <subsection xml:id="">
 <p></p>
 <p></p>
 <figure></figure>
 <p></p>

 </subsection>
</section>
```

```

 <p></p>
 </conclusion>
</section>

```

Admittedly, this is a bit much, but it makes the point. The Emmet “abbreviation” for this structure is:

```
section[xml:id]>introduction>p^(subsection[xml:id]>p*2+figure+p+ol>li*3)^conclusion>p
```

Upon typing this string and placing the caret to the right of it, hit Ctrl+E (or Tab, if you didn’t disable the Emmet default). The entire tree structure is created immediately, with tab stops for the missing attribute values and for each matching begin/end pair.

The Expand Abbreviation As You Type command allows you to tweak such abbreviations interactively. Hit Ctrl+Alt+Enter and type the expression above into the minibuffer at the bottom of the window, watching the tree appear as you type.

Emmet is a very powerful package that can do much more than is outlined here. However, it is by default mostly adapted to writing CSS and HTML. Customizing it to work more directly with PreTeXt is an ongoing project. You can discover more about Emmet by examining the [Emmet documentation](#) or poking around in the Settings and Keymap files.

## D.2.6 PreTeXtual—a Sublime Text package for PreTeXt

PreTeXtual is a Sublime Text package designed to assist authors using PreTeXt. It is no longer supported and will not work in the latest version of Sublime Text 4.

The package owes its inspiration and much of its code to the excellent [LaTeXTools](#) package. Please let the author know of any bugs you find or any features you would like to see included in MBXTools by [creating a GitHub issue](#).

### D.2.6.1 Installation

**via Package Control.** It is recommended to install PreTeXtual via [Package Control](#). If you have not installed Package Control yet, you should do that first (and restart Sublime Text afterwards).

After Package Control is installed, use the Install Package command to search for the PreTeXtual package, and select it from the Quick Panel to install. This method of installation allows Package Control to automatically update your installation and show you appropriate release notes.

**via git.** You may also install PreTeXtual via git. Change directories into your Packages folder. To find the Packages folder, select Browse Packages from the Preferences menu (from the Sublime Text 4 menu on OS X). Make sure you are in the Packages folder and *not* Packages/User.

Then, run

```
git clone https://github.com/daverosoff/PreTeXtual.git
```

and restart Sublime Text (probably not necessary).

### D.2.6.2 Usage

You can activate the package features by enabling the PreTeXt syntax. The syntax definition looks for .mbx or .ptx file extensions. If your PreTeXt files end with .xml, you will either need to add a comment to the first line of each file (after the XML declaration):

```
<!-- PTX -->
```

or you will need to enable the syntax manually using the command palette. To enable it manually, open a PreTeXt file and press Ctrl+Shift+P (Cmd+Shift+P on OS X) and type pretext. Select “Set Syntax: PreTeXt” from the list of options.

You should see the text “PreTeXt” in the lower right corner if you have the status bar visible (command palette: Toggle Status Bar).

There are only a few features implemented so far.



1. If you have some sectioning in your PTX file, hit `Ctrl+R` (`Cmd+R` on OS X) to run the Go To Symbol command. You should see a panel showing all the divisions' `@xml:id` names.
2. If you have been using `@xml:id` to label your stuff, try typing `<xref ref=` (the beginning of a cross-reference). Sublime Text should show you a panel containing all `@xml:id` values along with the elements they go with. Choose one to insert it at the caret and close the `xref` tag. Alternatively, type `ref` and hit `Tab` to activate the `xref` snippet. Then hit `Ctrl+l` followed by `x` or `Ctrl+l` followed by `Ctrl+Space` to bring up the completions menu. There are several variants of the `ref` snippet, namely `refa`, `refp`, and `refpa`.
3. Type `chp`, `sec`, `ssec`, or `sssec` and hit `Tab` to activate the division snippets. A blank `title` element is provided and the cursor positioned within it. As you type, the `@xml:id` field for the division is filled with similar text mirroring the title you are entering.

#### D.2.6.3 Known issues

1. When manually adding an `xref` (not using the snippets or autocomplete), you will frequently see a spurious "Unrecognized format" error.
2. The `ref` snippet does not bring up the quick panel. Should it?
3. Recursive search through included files for labels is not yet implemented.  
This will only work for `xref` completion, not Go To Symbol.
4. Nothing has been tested on OS X or Linux.

#### D.2.7 Recommended Packages

1. Package Control
2. Emmet
3. SideBarEnhancements
4. PowerCursors
5. MultiEditUtils
6. Text Pastry
7. Git or SublimeGit
8. SublimeLinter
9. MBXTools

## D.3 Aspell

Aspell is a spell-checker which you can easily configure to skip every piece of text used as a name of an XML element, and to skip `PreTeXt` elements that are likely to contain text that is not really made up of words and sentences. For example, the element `<chapter>` might not be flagged by some mainstream spell checkers, but most likely the element `<mdash/>` will be flagged by every spell checker, including the default configuration of Aspell. And your `<m>` elements are full of `LaTeX`, not words. Unfortunately, Aspell will not follow your `xi:xinclude` directives, so you need to run it against each of your files if you have modularized your source. A command-line invocation would like like:

```
aspell -c ~/aota/src/fish.xml
```

Installation on Linux should be straightforward through your distribution's package manager. On a Mac, the executable, and a dictionary for your language, can be installed easily via MacPorts ([provisional cross-reference: macports]). Recent improvements on Windows (ca. 2019-06-06) perhaps imply that wsl or the Ubuntu Linux install may provide an easy avenue. Please report steps that result in a successful Windows installation, so we can include them here.

Configuration is achieved via a “hidden” file at the top of your home directory, namely `.aspell.conf`. A Mac will try to keep you away from hidden files, which are the ones whose name begins with a period. Let SublimeText give you an assist here. In [Listing D.3.1](#) we show a first run at a useful configuration file. You *definitely* want to add the SGML filter, since this is what tells Aspell that you are working on XML files<sup>1</sup>, so that all element names, attributes, etc. will not be checked. The remainder is a suggested list of PreTeXt elements to skip. Suggestions for additions are welcome here.

#### Listing D.3.1 Aspell Configuration File

```
add-filter sgml

elements with lots of code
but not "pre", since it is like a "p"
add-sgml-skip c
add-sgml-skip cd
add-sgml-skip program
add-sgml-skip console
add-sgml-skip sage

image formats
add-sgml-skip latex-image
add-sgml-skip asymptote
add-sgml-skip sageplot

elements that display XML elements
add-sgml-skip tag
add-sgml-skip tage
add-sgml-skip attr

initialisms and friends
add-sgml-skip init
add-sgml-skip acro
add-sgml-skip abbr

math (latex)
add-sgml-skip m
add-sgml-skip me
add-sgml-skip men
add-sgml-skip md
add-sgml-skip mdn
add-sgml-skip usage # in "notation"

not really content, by and large
add-sgml-skip docinfo
```

You can run Aspell in a sort of batch, non-interactive mode by adding the `-a` switch and then providing your source on standard input and directing results from standard output. But I do not find it very useful.

---

<sup>1</sup>SGML is the precursor of XML.

## D.4 emacs

Jason Underdown reports on 2016-05-12 that emacs' [nXML mode](#) works well with a schema, and Mitch Keller reports on 2021-04-18 that specifically a RELAX-NG schema may be used. The two versions of the RELAX-NG schema for PreTeXt can be found at

```
pretext/schema/pretext.rng
pretext/schema/pretext.rnc
```

The first is the “real” version, while the second is an entirely equivalent (compact) syntax that is meant to be a bit more human-readable, so perhaps either may be employed.

On 2021-04-18, Mitch Keller reports success with placing the following `schemas.xml` file in his PreTeXt source file directories. Note that you may need to adjust the path in the `@uri` attribute, and the `@pattern` attribute implies that Mitch has used a `.ptx` suffix for all his source filenames. If you are already using emacs, you should have no trouble making the necessary adjustments.

```
<?xml version="1.0"?>
<locatingRules xmlns="http://thaiopensource.com/ns/locating-rules/1.0">
 <uri pattern="*.ptx" uri="../../pretext/schema/pretext.rnc"/>
</locatingRules>
```

You simply put your cursor at any point in the document, start a new tag with `<` and then call the `completion-at-point` function (I bound it to the key-chord: `C-<`) to get a list of possible completions. Or you can start typing a few characters to narrow the list of possibilities. It will also let you know if the element you are trying to insert is invalid.

—Jason Underdown

## D.5 XML Copy Editor

Michael Doob reports on 2017-02-03 that [XML Copy Editor](#) works well, in particular on Windows. This is an open source program, for Windows and a variety of popular Linux distributions, that supports both DTD and RELAX-NG schemas. It is less of a general programmer's editor and more like dedicated tools for working strictly with XML documents.

## D.6 vi, vim

### D.6.1 Using vi and PreTeXt together

Any smart editor, and `vi` is no exception, allows the addition of new commands to make repetitive tasks easier. Since creating documents with PreTeXt markup often is repetitive, it makes sense to explore the techniques of creating additional commands in `vi` to make editing easier.

The expectation for this section is that the reader is able to use `vi` to create and edit files, but no greater depth of knowledge is assumed. The term `vi` is meant to be inclusive, that is, it includes `vim`, `gvim` and the like.

#### D.6.1.1 Modes of vi: Command, Normal, and Insert

One of the most fundamental properties of `vi` is that every editing task may be accomplished using the keyboard. For different editing contexts, it is advantageous to have the keyboard strokes have different meaning; these different interpretations are called the *modes* of `vi`.

Here are the pertinent modes of `vi`:

- *Normal mode*: This mode is for changing position within the file. For example, `j` moves down, `2w` moves forward two words, and `-2}` moves backwards two paragraphs. It is also used for block operations on text: `-3dd` is used to delete three previous lines, or `2.yy` will yank the next two sentences. Usually `vi` starts in normal mode.
- *Insert mode*: This mode is for inserting new text into the file. Typing `Galloping Gertie leapt into the air.` causes that text to be inserted at the current position in the file.
- *Command mode*: This mode is to execute commands. Typing `/abc` in normal mode initiates a forward search for the first occurrence of “abc” via the command mode. Similarly `:w` will use command mode to write out the current working material (buffer) to the disk.

Now a quick review of the keys used to move between modes: From a given position in a file in Normal mode, using `[i]` or `[a]` will change to Insert mode and insert text before or after that position. Similarly, `[I]` or `[A]` will insert text in front of or directly following the current line, and `[O]` or `[o]` will insert text above or below the current line. Returning to Normal mode is done using `[Esc]`.

If `[/]`, `[?]`, or `[:]` is typed when in normal mode, a small one-line window opens up (called the *command line*) to receive text. This entered text is terminated by `[Enter]`. The `[/]` or `[?]` initiates a forward or backward search for the entered text. The `[:]` sends the entered text to the `vi` program for further processing.

Figure D.6.1 show the keys used to move between modes.

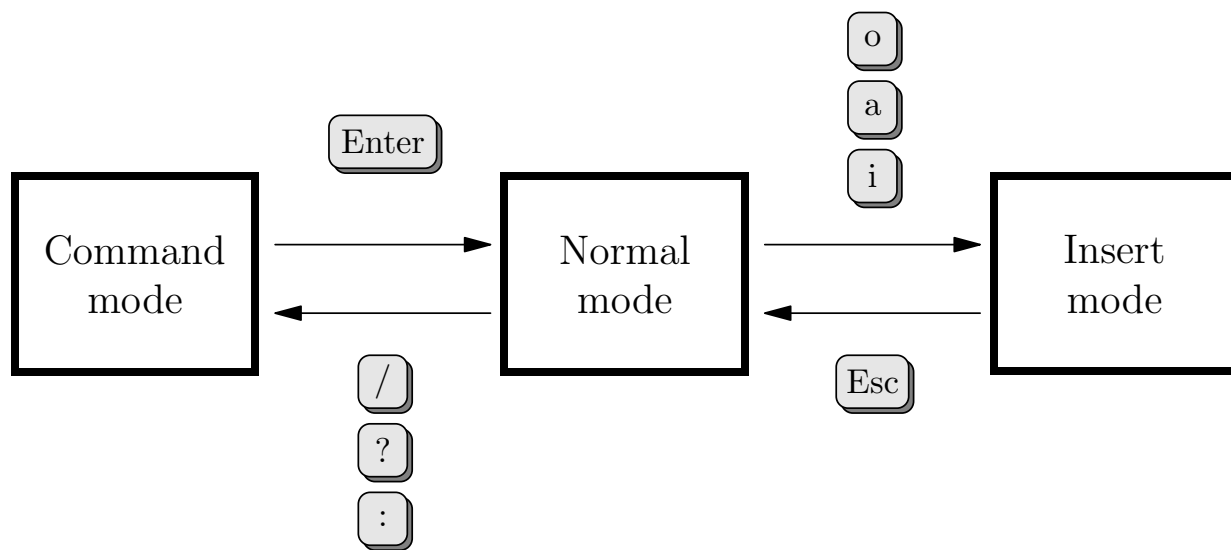


Figure D.6.1 Keys for moving between different modes in `vi`

An exclamation point prefix `!` in Command mode sends that command to the operating system. For example, entering `:!date` will cause the results of the operating system command `date` to be displayed on the command line. In addition the percent character `%` gets expanded to the name of the file being edited. If your operating system uses `ls -l` to list file information, then `:!ls -l %` will give the properties of the file being edited.

### D.6.1.2 Using `:set`

The Command mode in `vi` allows users to change their interaction with the editor. (Remember that entering `:` changes to Command mode, and any text string entered in Command mode is terminated by `[Enter]`.) For example, entering `:set number` will cause line numbers to appear on the left. They are not in the file itself, of course, but are there for the convenience of the user. Entering `:set nonumber` will remove these line numbers. Similarly `:set autoindent` will cause a new line to preserve the indentation of the previous one and `:set expandtab` will replace the tab character by an appropriate number of spaces (both are very useful for writing PreTeXt documents). Some useful `set` commands are given in Table D.6.2.

Table D.6.2 Some useful parameters for the set command

Command	Resulting change
<code>:set autoindent</code>	A new line preserves indentation
<code>:set expandtab</code>	Inserted tabs converted to spaces
<code>:set rows=n</code>	Set number of lines displayed
<code>:set columns=n</code>	Set the display size of each line
<code>:set list</code>	Show tabs and carriage returns
<code>:set tabstops=n</code>	<code>Tab</code> inserts <i>n</i> spaces
<code>:set</code>	Show all current settings

### D.6.1.3 A little editing etiquette

The XML files used with PreTeXt are ordinary text files. This makes it easy for coauthors to email them back and forth in order to expand and improve the content. There are a few potential problems, and hence some useful precautions.

- If a `Tab` is entered and not expanded to spaces, different editors may display the text with different alignments.
- If there are extra spaces at the end of a line, there may be odd line wrapping.

Fortunately, these are easy to avoid.

- A tab character may be found in the usual manner for searches: `:` `/` `Tab` will find the next tab; it can be removed and replaced by spaces.
- A space before the end of a line can be found with the search `:` `/` `Space` `$` (vi will interpret `$` as the end of a line rather than as a dollar sign). The spaces at the end of the line can then be removed.

There is another feature of vi that is helpful in this respect. Using `:set list` will make the tab and end of line characters visual as `^I` and `$`. This makes the appropriate deletions easy. It is good editing etiquette to do so.

### D.6.1.4 Abbreviations

The `:abbreviation` command allows the replacement of longer expressions by shorter ones. Try this: in Command mode enter

```
:abbreviate ups University of Puget Sound
```

and then (in Insert mode) type

```
I enjoyed my visit to the ups.
```

If all goes well, the abbreviation is expanded and the text is

```
I enjoyed my visit to the University of Puget Sound.
```

Now suppose you want to write My favourite letter of the Greek alphabet is upsilon. Looks like trouble with the last word, but in fact all is well. Abbreviations are not expanded until the next character after the abbreviation is read. If the next character after the abbreviation is either a letter or a number, no expansion takes place. Careful observation of the original example reveals that the abbreviation is not expanded until the period is entered. Here is another example: You want to write I love pushups. What about the end of the last word? No problem! The ups is expanded only if it is at the beginning of a word.

Amusingly enough, when in Command mode, the `abbreviate` command can itself be abbreviated to `ab`.

The choice of an abbreviation is essentially arbitrary. However, if desiring an abbreviation within “I enjoyed my visit to the Technische Hogeschool Eindhoven”, it would be folly to use `:ab the Technische Hogeschool Eindhoven`. The abbreviation should be mnemonic, but avoid actual words.

Now consider the following problem: what if the `Enter` key is one of the desired characters in the abbreviation? Since that key terminates Command mode, it appears impossible. Not so! The characters `<enter>` (that’s seven of them) will be replaced by a single character equivalent to `Enter`.

There are other characters that are treated in the same manner:

Table D.6.3 Text equivalents to keyboard entries

Desired key	Text equivalent (not case sensitive)
	<enter> or <cr>
	<bs>
	<ins>
	<del>
	<esc>
	<left>
	<right>
	<up>
	<down>
	<home>
	<end>
	<C-x>
	<M-y>

Here is a useful PreTeXt example: Define the abbreviation

```
ab gm <m></m><left><left><left><left>
```

and then enter (in Insert mode)

It follows from  $\text{gm}\log(\theta)=0$  that  $\text{gm}\theta=1$ .

It will (almost) be expanded to

```
It follows from <m>\log(\theta)=0</m> that <m>\theta=1</m>.
```

That's "almost" because it is necessary to move the cursor past the `</m>` when leaving the mathematics input. It's pretty easy to see how this abbreviation works. The first seven characters `<m></m>` are expanded unchanged and then the cursor moves to the left four times to put it right where it needs to be to enter the mathematics. Careful observation will reveal a little trick used in this example: `gm` is followed by a `\` and so terminated the abbreviation correctly. If the variable were  $x$ , then inputting `gm $x$`  would not work. A workaround: enter a `Space` after the `gm`. This terminates the abbreviation and it will work as desired. (Actually, there will be an extra space before the  $x$ , which causes no ill effect, but if true perfection is desired, using `gm $Space$ Backspace` will eliminate it.)

When `:ab` is used, it is in effect in all modes. When abbreviations are for Input mode only (as is the present case), then `iab` may be used and is usually preferable.

Here is another useful (nonmathematical) example. Define (for use in Input mode)

```
:iab gp <p><cr></p><up>.
```

Then entering "gp`Enter`" on a new line will create three lines appearing like:

```
<p>
```

```
</p>
```

That is, the first and third line start and end a paragraph and the cursor, represented by `|` is at the beginning of the second line. If autoindent is set, (see [Table D.6.2](#)), the line indentations are preserved. With this definition, the frequent task of starting a new paragraph appropriately formatted may be carried out using only three key strokes.

A further example:

```
:iab gcom <!--<CR><CR>--><Up>
```

is useful to entering comments. Starting a line with "gcom`Tab`" will help create nicely indented comments.

Want to know what abbreviations are in effect? Just enter `:ab` and they will be listed.

### D.6.1.5 Maps

Maps, like abbreviations, are shortcuts that save extra key strokes. There is a difference in method: the `map` command uses *key bindings* that (re)define the meanings of key strokes. For example, inputting (while in Normal mode) `:map <C-X> :w<CR>` will define (map) the `[Ctrl] + [X]` key combination so that it is equivalent to typing the rest of the definition, `:w<CR>`. This three-key sequence, of course, just changes to Command mode, writes the current buffer to disk, and returns to Normal mode.

Presumably this newly defined key binding is meant to be invoked in Normal mode. To do so in Insert mode would be a mistake; to avoid this, there is an `nmap` command that defines key binding for Normal mode only. Similarly, there is an `imap` command for Insert mode only. Thus defining a key binding using `:nmap <C-X> :w<CR>` will make it valid only in Normal mode. Unlike abbreviations, the binding takes effect the moment the key is pressed.

There is a cute technique to use the same binding in Normal and Insert modes. The key binding `:imap <C-X> <esc><C-X>a` defines a binding for Insert mode. The first character, `<esc>`, changes to Normal mode; the next character, `<C-X>`, will use the Normal mode definition of `<C-X>`. The final character `a` returns to the previous position in Insert mode.

We can make special use of the special characters `!` and `%` as described at the end of [Subsubsection D.6.1.1](#).

The author using PreTeXt usually edits an XML file, say `myfile.xml`. This file is then processed using `xsltproc` in conjunction with an `xsl` file, say `myfile.xsl`. Often the `xsl` file is `pretext-html.xsl` or `pretext-latex.xsl`. The usual command used by the author is `xsltproc myfile.xsl myfile.xml`. With this in mind, we could define a map: `:nmap <C-X> !xsltproc myfile.xsl myfile.xml<CR>` so that the XML file could be processed with a single keystroke. Even better: `:nmap <C-X> :w<cr>:!xsltproc myfile.xsl %<CR>` defines a map that first writes the file being edited to disk and then processes it with `xsltproc`. This command is independent of the particular file being edited.

A final somewhat complicated but very useful definition:

```
imap <C-A> <Esc>yiwi<<Esc>ea></<Esc>pa><Esc>F<i
```

which is a truly cryptic sequence of key strokes. Here is what they do:

Table D.6.4

Key strokes	Interpretation
<code>&lt;esc&gt;</code>	Leave input mode
<code>yiw</code>	Save (yank) word above cursor
<code>i&lt;</code>	Insert <code>&lt;</code>
<code>&lt;esc&gt;</code>	Go to Normal mode
<code>e</code>	Move to end of word
<code>a&gt;&lt;/&lt;esc&gt;</code>	append <code>&gt;&lt;/</code> and return to Normal mode
<code>p</code>	Paste the saved word
<code>a&gt;&lt;Esc&gt;</code>	append <code>&gt;</code> and leave Insert mode
<code>F&lt;</code>	Move to preceding <code>&lt;</code>
<code>i</code>	Go to Insert mode

The effect of this map: Entering `abc<C-A>` will change `abc` to `<abc>|</abc>` where `|` is the position of the cursor. The definition needs to be a bit convoluted so that it works with both single letter and multiple-letter words.

To list all of your defined maps, just use `:map`.

### D.6.1.6 Saving abbreviations and maps

The abbreviations and maps defined during an editing session disappear when the session is over. There are a number of ways to retain them over different sessions.

- Ephemeral use in one file: For most versions of vi, after entering Command mode the history may be accessed using the `\(\uparrow)` key. Backing up to the previous definition of an abbreviation or map and pressing `[Enter]`

will reinstate the definition. The line can also be edited if changes are desired (very useful while developing new abbreviations and maps). This history is preserved between editing sessions.

- Repeated use in several files in one directory: The abbreviations and maps can be saved in a text file, say `mymaps.txt`. A line within the file might look like

```
:ab ups University of Puget Sound
```

Vi is then started with the `-s mymaps.txt` option to initialize the definitions.

- Repeated use for files in different directories: Put the abbreviations and maps in the file `.vimrc` (`_vimrc` for Windows) in your home directory.

## D.7 Spell Checking Output

While browsing your HTML output, it is possible to have the web browser's native spell checker highlight spelling errors. Create a bookmark with the following for its URL (all at once on one line, there are three lines here just for clarity):

```
javascript:document.body.contentEditable=!document.body.isContentEditable;
document.designMode=(document.designMode === 'on') ? 'off' : 'on';
void 0
```

Now when viewing a page and you click this bookmarklet, JavaScript will change browser settings so that the native spell checker highlights spelling errors. This is done by making the text content of the page editable, which is normally what is needed for the browser's spell checker to kick in. While this is active, you will not be able to click most links and buttons. Click the bookmarklet again to toggle things back to how they were.

Some browsers may have extensions that you can install for spell checking the content of a page. For example, Chrome has [Webpage Spell-Check](#).



# Appendix E

## Schema Tools

This appendix has technical information about tools that work with the RELAX-NG schema and our additional validation stylesheet. See [Chapter 6](#) for a more general overview.

### E.1 Jing and Trang

These tools come from James Clark, an author of the RELAX-NG syntax. `trang` is a converter between different formats for schemas. An author should not ever need it. Though a PreTeXt developer might find it useful, and it is a by-product of the `jing-trang` install described below.

`jing` is our recommendation for RELAX-NG validation by authors, and works very well.

`jing` and `trang` are packaged (separately) for Debian/Ubuntu Linux, see [Subsection E.1.1](#). Reports of other similarly easy installations for other operating systems, to be included here, are especially welcome. We have employed the Debian/Ubuntu versions and also versions built from source, see [Subsection E.1.2](#).

#### E.1.1 `jing` on Ubuntu and CoCalc

[CoCalc](#) runs Ubuntu when you create a terminal window. The necessary Ubuntu Linux (Debian) package is also named `jing` and is installed as part of the CoCalc setup. So on Ubuntu, you need to install this package, and on CoCalc you will need to copy over the `schema/pretext.rng` version of the schema. If you have a CoCalc subscription, it would be even better if you made a clone of the PreTeXt repository, so it would be trivial to update and always have the latest version of the schema in your CoCalc account.

Now it is straightforward to execute `jing`:

```
jing /path/to/pretext.rng aota.xml
```

Presumably something similar to the above will work for any Linux distribution that has packages for `jing`.

Note that if you have modularized your source files (see [Section 5.3](#)), you only need to provide the top-level file as an argument to `jing`. In particular, the subsidiary files are certain to fail validation since they do not have a `<pretext>` root element.

## E.1.2 Install jing from Source

If you cannot install this tool easily as part of your system, you can still follow the notes below to build from source. Many authors have done this successfully. Installation notes for jing and trang follow.

**Clone** Clone the git repository at [github.com/relaxng/jing-trang](https://github.com/relaxng/jing-trang) with the command

```
git clone https://github.com/relaxng/jing-trang
```

which assumes you have a command-line version of git installed. You will end up with a new jing-trang directory, which you will certainly want *outside* of your PreTeXt files and *outside* of your project files.

**readme.md** Follow the instructions in the `readme.md` found at the top level of the jing-trang distribution, observing the following notes keyed to the four steps. These helpful notes come courtesy of the experiences of Jahrme Risner, Mitch Keller, Bruce Yoshiwara, Ken Levasseur, and Jessica Sklar on a variety of operating systems.

1. It is necessary to have a developer's version of java on your machine. Try which java to see if one is already available. Any output here might help in the next step. References here to the JDK is the Java Development Kit. There are specific directions for MacOS ([Section K.1](#)) and Windows ([Section M.6](#)).
2. You will need to have the JAVA\_HOME environment variable set correctly. You can try `echo ${JAVA_HOME}` in a Linux console to see if it produces anything sensible and/or consistent with Step 1. Ken Levasseur notes that on MacOS you can go

```
echo $(/usr/libexec/java_home)
```

and the output is what you set to the JAVA\_HOME variable.

On Windows you may need to set Environmental Variables in the Windows System Properties GUI, both here and in Step 4.

3. Setting your working directory to the root directory of the jing-trang distribution should not cause any particular difficulties.
4. You may need to install the ANT tool, almost certainly on Windows. Again, on Windows you may need to set an ANT\_HOME environment variable. On Linux, this may be all set for you already as a system tool.

The README suggests changing slashes on Windows. But you may already be using a shell that gives Unix-like behavior. So try both directions, if necessary. Also, the README suggests running `./ant test` (or `.\ant test`). Doing this on Windows may yield a BUILD FAILED message, but jing may still work.

**Results** Change into the build subdirectory. You may have more files here, but the two you really want are jing.jar and trang.jar. So if you have these, you are in good shape.

If you have modularized your source files ([Section 5.3](#)) then you need one more library. Look in the top-level lib directory (a peer of build) for xercesImpl.jar. You have two and a half choices now.

- Copy `lib/xercesImpl.jar` into build.
- Or make a “symbolic link” to the third JAR archive. Be sure you are in build and go

```
ln -s ../lib/xercesImpl.jar
```

which will just make your operating system think this third archive is in the build directory.

- A third option would be to adjust/augment some classpath information below and send us a report of success. We have not tested this approach.

Now you are ready to use `jing` to validate a PreTeXt document. Note that the commands below require the XML version of the schema, which is the non-compact version. Also, these are the commands if you build `jing-trang` from source. If you install a system-supplied version, then consult the man pages, or similar, for syntax which is likely much easier and direct.

For a document contained in a single file, run (with suitable working directory and path prefixes).

```
java -jar jing-trang/build/jing.jar pretext.rng my-book.xml
```

For a document modularized across several files using the `xinclude` mechanism, issue as one single command line (again, with suitable working directory and path prefixes). This presumes you have moved or symlinked the `lib/xercesImpl.jar` file into the build directory. Do not leave any spaces around the equals-sign, we have split that line there for readability, so the `-D` option should not have any spaces in its argument.

```
java -classpath jing-trang/build
-Dorg.apache.xerces.xni.parser.XMLParserConfiguration=
 org.apache.xerces.parsers.XIncludeParserConfiguration
-jar jing-trang/build/jing.jar pretext.rng my-book.xml
```

It may go without saying that scripting this task will make you more likely to do it as often as is necessary and you will save yourself much time, and a little frustration, in the process.

Jahrme Risner provides the following setup he uses to make it very convenient to regularly validate his sources. This is a “shell script”, which a Linux user might add to their `~/.bashrc` file. Note that we have again split the line with the `-D` option at the equals-sign, without a line-continuation character. Do not do that in your version.

```
ptx-check() {
 java\
 -classpath ~/GitHub/jing-trang/build\
 -Dorg.apache.xerces.xni.parser.XMLParserConfiguration=
 org.apache.xerces.parsers.XIncludeParserConfiguration\
 -jar ~/GitHub/jing-trang/build/jing.jar\
 ~/GitHub/pretext/Schema/pretext.rng "$1"
}
```

Then he can simply go

```
~ $ ptx-check my-book.xml
```

at anytime. Note that you might have to provide or adjust some of the paths above for your situation. And there are other ways to script tasks like this.

## E.2 PreTeXt Validation Plus

The second step of validation is our “validation-plus” stylesheet. Fortunately, there is nothing to install. Use it just like the conversions you have already been doing. In the PreTeXt distribution, in the `schema` directory you will find `pretext-validation-plus.xsl`. This is a stylesheet, unique to PreTeXt, which will carefully analyze your source to find any exceptions that the RELAX-NG schema was not designed to catch. You use the stylesheet like any other,

```
xsltproc ~/pretext/schema/pretext-validation-plus.xsl ~/books/aota/animals.xml
```

with suitably adjusted paths, and be sure to provide the `-xinclude` switch if your source is modularized across multiple files (Section 5.3). No news is good news, but each exception found should provide enough explanation for you to locate, and correct, the problem. These messages are under PreTeXt’s control, so please report any that are not helpful enough. That’s it—easy.

# Appendix F

## Node and npm

To run a Javascript program *outside* a web browser requires a program that can interpret the Javascript language. A popular choice is `node.js`, whose executable is simply `node`. Programs designed for execution by `node` often build on other programs. These are all organized in packages, which can be managed by the Node Package Manager, known as `npm` for short. A basic purpose of `npm` is to manage versions and dependencies among packages.

So the first step is to install both `node` and `npm` on your system. [Instructions for installing node and npm](#) can be found at [nodejs.org](http://nodejs.org). These programs are meant to be cross-platform, so once you do these two operating-system-specific installations, we can proceed with generic instructions.

Now `node` should be on your path, and you can try

```
which node
```

to see if your operating system can locate it automatically. If not, then you will need to edit your personal copy of the `pretext.cfg` configuration file to have the `node` key provide a path to the executable (see [Section 47.6](#)).

Some useful `npm` commands are listed below, in the form of examples. These commands depend on running in a **package** a folder containing a file named “`package.json`”. So before trying to run any of them, make sure to navigate to such a folder (e.g. `script/mjsre` or `script/cssbuilder`). In particular, note `npm install` which must be run before using a script.

When using `node` with `git` (see [Appendix I](#)), you generally should check `package.json` and `package-lock.json` into source control, but ignore the entire `node_modules` folder.

**Table F.0.1 Useful `npm` commands**

<code>npm install</code>	Install the packages listed in the “ <code>package.json</code> ” file to the folder <code>node_modules</code> . This
<code>npm list</code>	Full tree of installed packages (local)
<code>npm list mathjax-full</code>	Just one package
<code>npm view speech-rule-engine version</code>	<i>Available</i> version

## Appendix G

# Offline MathJax

PreTeXt uses the well-known  $\text{\LaTeX}$  syntax for mathematics. One fundamental reason for this choice is that the [MathJax](#) Javascript library is so capable at rendering  $\text{\LaTeX}$  inside of an HTML web page. Of course, it is not hard to render  $\text{\LaTeX}$  syntax inside of  $\text{\LaTeX}$  output! But what about output formats that are not processed by  $\text{\LaTeX}$  or Javascript, such as EPUB or braille? That is where running MathJax offline (locally) comes into play.

### G.1 MathJax and Speech Rule Engine

MathJax is packaged as a `node.js` program. A component of MathJax is Speech Rule Engine (SRE), which converts  $\text{\LaTeX}$  to text that represents a spoken version of the mathematics, and to Nemeth braille, a braille code for expressing mathematics.

Any `node.js` script requires installing Node. See [Appendix F](#) for instructions on doing so. Once Node is installed, you can use the following instructions to complete the installation of MathJax and the SRE.

Follow the directions here *exactly*. Do not free-style and think some modification will have a better or identical result. In a terminal, set your working directory with

```
cd /path/to/pretext/script/mjsre
```

Install both packages at the same time with a single command:

```
./update-sre
```

It is possible you will need to make this simple script executable. On Unix-style systems (Linux, Mac OS) this can be done with the `chmod` command. As this is a bash script, Windows users must use Window's Subsystem for Linux or examine the contents of the script to run the commands manually.

The `update-sre` script is supplied by PreTeXt. It will create a file and a directory:

```
/path/to/pretext/script/mjsre/package-lock.json
/path/to/pretext/script/mjsre/node_modules
```

Neither will be tracked by `git`. Since SRE changes faster than MathJax, this will replace MathJax's expectation for a version of SRE with a potentially newer version. So do not be alarmed if it appears MathJax is missing a dependency, that is intentional.

Now you have copies of MathJax and SRE that can be used by the `pretext/pretext` script to process mathematics offline into useful formats for conversions to EPUB, Kindle, and braille.

### G.2 Mathematics Representations

Once MathJax is installed properly, the `pretext/pretext` script will be able to produce an EPUB version of your project. The script will first analyze your document, isolating all of the mathematics. These are then processed by

MathJax and produce a file of **representations** of the mathematics, either as SVG images or as MathML versions. These representations are then inserted properly into the eventual output. This process is all automatic but explains why it is possible to produce two different types of EPUB. A good test of your installation is to use the pretext/pretext script to make an EPUB ([provisional cross-reference: epub production]) or the script can be used to simply produce structured files of these alternate representations (see [provisional cross-reference: script math representations] or paragraph just below).

With SRE also installed, text (speech) becomes a possible representation, as well as Nemeth braille. This makes possible an EPUB version that is all text, and there are online sites that will turn this EPUB into an audio book (of not very good quality). The braille representations are one component of the production of braille output. ([provisional cross-reference: braille production], [Mathematics Representations](#))

If you use the pretext/pretext script with a math component, and a format of svg or mathml, you will produce a file of these representations, structured by XML (naturally). These files are not much use by themselves, but may be of interest.

# Appendix H

## LibLouis

[liblouis](#) is an open-source library for low-level translation of phrases into braille, supporting many, many languages. We rely on Python bindings for this library to translate all of the literary text (non-math) to braille for a document, so this is an essential piece of the pipeline. Once installed, its use is transparent. This appendix outlines two alternative installation methods, current as of 2025-12-19, on Linux, Mac, and Windows Subsystem for Linux. Contributions of adjustments for other operating systems are welcome.

### H.1 Installing as a system package

For many operating systems, one may install the liblouis Python bindings as a system package. For instance, on Debian/Ubuntu systems and Windows Subsystem for Linux, run

```
sudo apt install python3-louis
```

and on Mac

```
brew install liblouis
```

To test the bindings, the following should complete successfully in a Python interpreter

```
>>> import louis
```

If you are following best practices by using a [Python virtual environment](#) for your PreTeXt project, you will need to give the environment access to system packages since liblouis is now installed as a system package. You can do this when creating the virtual environment using

```
python3 -m venv /path/to/venv --system-site-packages
```

If you have an existing virtual environment configured for your PreTeXt project, you may alternatively edit one line in the file `pyvenv.cfg` in the virtual environment's main directory:

```
include-system-site-packages = true
```

### H.2 Installing from source

While the python3-louis system package appears to be updated frequently, if you need a more recent version of liblouis or simply feel a little more adventurous, you may wish to install liblouis directly from source following these instructions:

- Download the `liblouis-X.YY.0.tar.gz` archive from the [downloads page](#). Releases are tracked at the [GitHub release page](#). The `liblouisutdml` package is not necessary.

- `tar -xvf liblouis-X.YY.0.tar.gz` into a scratch directory like `/tmp`.
- Switch to being root (`sudo`) and `cd` into the directory created by the extraction.
- In your terminal run

```
./configure --enable-ucs4
make
make install
```

The “ucs4” flag enables 32-bit Unicode support, which is necessary for running tests later.

- Read `/tmp/liblouis-X.YY.0/python/README.md` and perform two steps, still as root, from within the directory structure in `/tmp`: install the bindings into your Python distribution and run the Python tests. Instructions for setting up a Python virtual environment are also available in that file.
- I do not do anything special to clean-up afterwards, and of course, my `/tmp` goes away on the next reboot. I also do not do anything special when installing the next version, I just follow the same procedure as a fresh install.



# Appendix I

## Revision Control: git

Authoring a textbook without revision control is like driving without a seat belt. Sooner or later, you will wish you had used it. `git` is a popular program for revision control for software projects, and works quite well with PreTeXt, though not perfectly. Notes here are designed to help. For more on `git` itself, in the context of authoring a book, see [Git for Authors](http://pretextbook.org/gfa/html), by Robert Beezer and David Farmer at [pretextbook.org/gfa/html](http://pretextbook.org/gfa/html).

**Word Wrap.** `git` is designed for code, where a newline often expresses the end of a statement. In PreTeXt, it might make sense to author an entire (long) paragraph without any newlines. If so, a line-oriented file diff is not so useful. Fortunately, `git` has a flag, `--word-diff`, which does an excellent job of displaying small edits precisely.

**Messages for Commits and Merges.** When you make a commit or merge, you can supply a message at the command line with the `-m` argument. Otherwise you get thrown into an editor, with the default being `vi`, which can be hard to get out of if you have not used it before. Better, as Joe Fields suggests, is to tell `git` which editor you want to use. To set `pico` as the default editor, the one-time command-line incantation would be:

```
git config --global core.editor "pico"
```

You can also directly edit the configuration file at `~/.gitconfig`. More suggestions can be found on [this thread](https://stackoverflow.com/questions/2596805) on StackOverflow at [stackoverflow.com/questions/2596805](https://stackoverflow.com/questions/2596805).

## Appendix J

# Conversion from $\text{\LaTeX}$

As part of the [UTMOST](#) project, we offer a service to help convert existing textbooks from  $\text{\LaTeX}$  to PreTeXt. The service is free if you are planning to release your book with an open license. The conversion will only be 95 percent correct, but that means it will take you 20 times less effort than converting it yourself.

Before converting your book, familiarize yourself with PreTeXt to the point of being able to compile the sample article and sample book into HTML and PDF. Check that you can edit the source files and the resulting output files behave as expected. That way, you will be on familiar ground when you finish the last 5 percent of the conversion.

The actual conversion will be done by David Farmer from the American Institute of Mathematics. The first hurdle is to get your  $\text{\LaTeX}$  files to David. The preferred method is:

1. Put your *entire*  $\text{\LaTeX}$  source into a GitHub repository. Do not edit or restructure in any way, nor provide just a subset for “testing,” since this only complicates the process. You do not need to include the image files.
2. If the repository is private, make David a collaborator so he can access it. (GitHub username: davidfarmer.)
3. Email [farmer@aimath.org](mailto:farmer@aimath.org) with your request for conversion, including the URL of your repository and a brief description of your project.
4. When the first draft of the conversion is ready, you will receive a pull request from David via GitHub. Go ahead and accept it, and now your repository will have new files that are the PreTeXt source.
5. Spend some time to review the files carefully, looking for consistent mis-interpretations of your intent. Convert to HTML and PDF and see how they look. There may be some back-and-forth with you explaining what your  $\text{\LaTeX}$  was trying to do, and David improving the conversion to PreTeXt.
6. At some point it will be up to you to take ownership of the PreTeXt source and finish the conversion.

This service is for authors who wish to consider having PreTeXt as the “official” source of their textbook. It is not feasible to maintain  $\text{\LaTeX}$  source and expect to have all of the features of PreTeXt.

# Appendix K

## MacOS Installation Notes

This appendix explains how to install necessary software on Apple's Mac.

### K.1 Java

The jing and trang schema tools ([Appendix E](#)) require the Java Development Kit (JDK). These instructions come from Mitch Keller and Jane Butterfield on 2019-07-23. You may need to change some version numbers over time.

#### List K.1.1 Install Java on MacOS

1. From [jdk.java.net/12/](http://jdk.java.net/12/) get an open source JDK. In other words, not the proprietary version from Oracle.
2. Unzip the file you download until you have a directory called `jdk-12.0.1.jdk`. Put that directory somewhere useful. We will pretend that you put it in your home directory and that your user is called jane, which means that the directory is now at `/Users/jane/jdk-12.0.1.jdk`.
3. In a Terminal, run

```
export JAVA_HOME=/Users/jane/jdk-12.0.2.jdk/Contents/Home
```

adjusted appropriately. A good check is to get this same directory back from the command

```
echo $(/usr/libexec/java_home)
```
4. If you are doing this to install jing and trang, then, *in the same Terminal window*, `cd` into the jing-trang directory and run `./ant`.

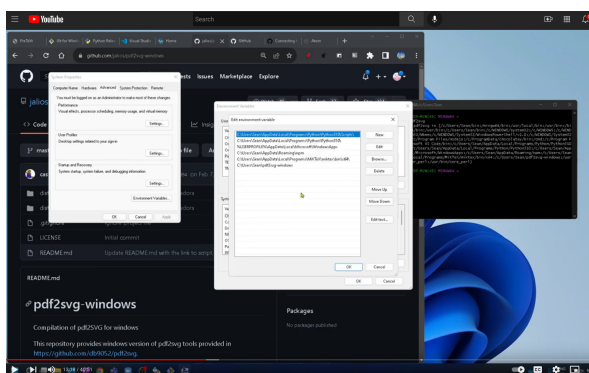
## Appendix L

# Installing the PreTeXt-CLI on Windows

Sean Fitzpatrick

The PreTeXt-CLI is the simplest way for authors to get started using PreTeXt, and it is the recommended method on all systems, including Windows. Details on using the CLI can be found starting in [Section 5.2](#). Setting up the PreTeXt-CLI on Windows is considerably simpler than the method presented in [Appendix M](#), but there is still some effort required. In particular, you will have to install several pieces of software to get your system ready for using PreTeXt.

If you prefer a video walkthrough, one is provided below. The video is approximately 40 minutes in length, but it goes through the entire installation process, from adding the necessary software, to setting up [GitHub](#).



[Standalone](#)  
[In Context](#)

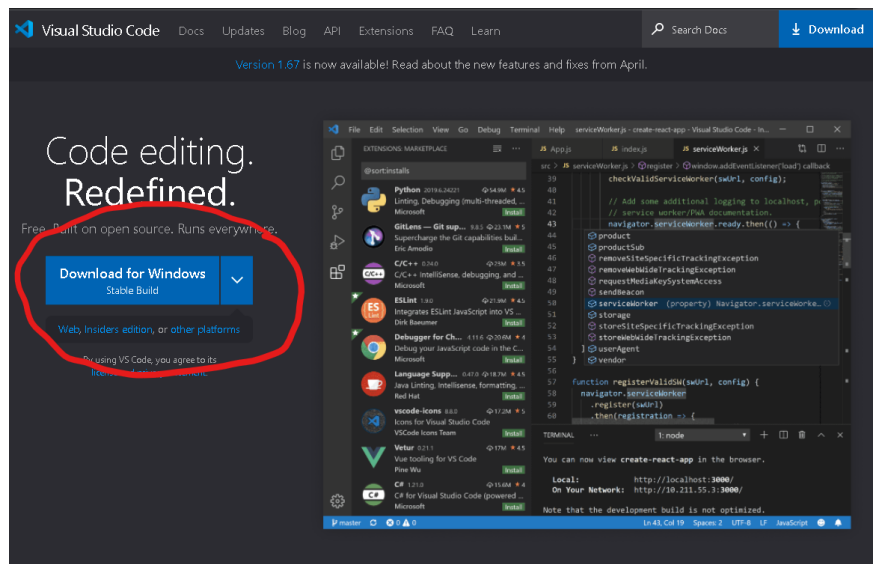
## L.1 Software prerequisites

We begin with the programs you will want to have installed prior to using the PreTeXt-CLI. All software can be downloaded free of charge, and is relatively easy to install.

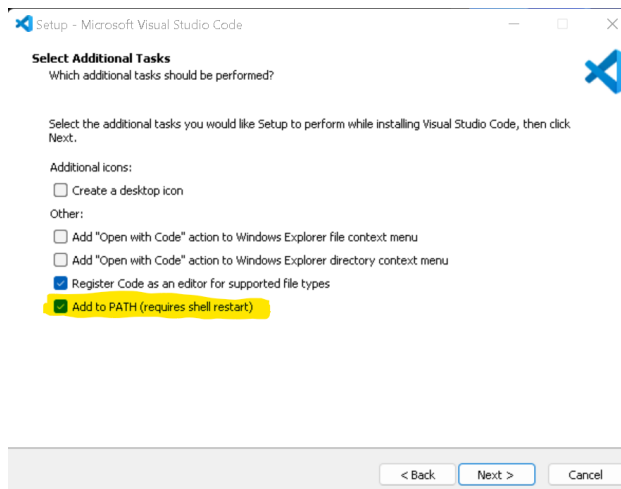
### L.1.1 VSCode

VSCode is short for Visual Studio Code. This is a text editor developed by Microsoft; we recommend it not because it plays well with Windows (although it does), but because of the availability of the pretext-tools add-on, which will let you run the PreTeXt-CLI without leaving the editor.

To install, the editor, first download the installer from [the VSCode website](#).



When running the installer, be sure that the “Add to PATH” option is selected:

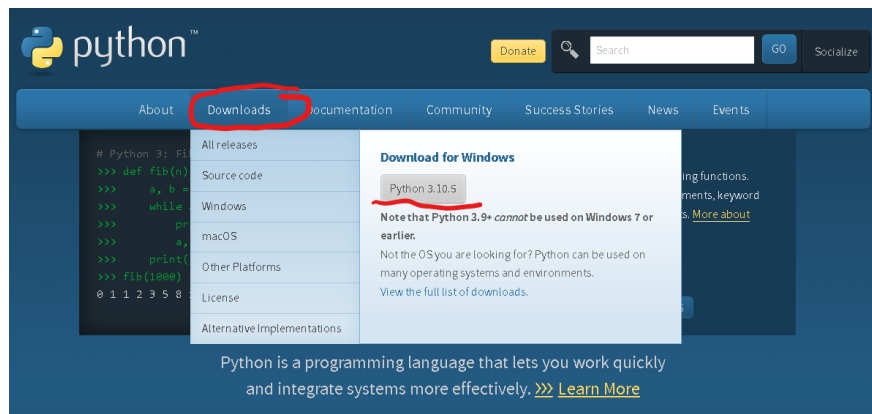


You should now have VSCode installed on your system.

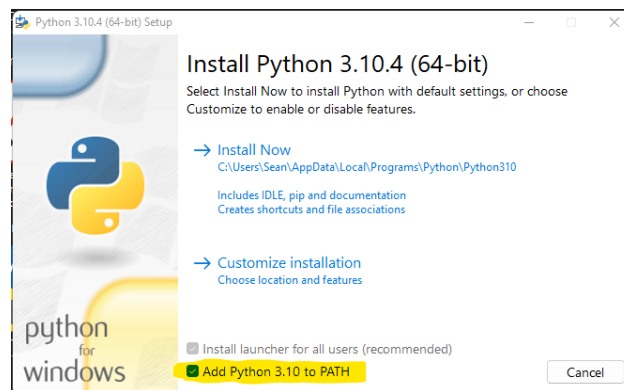
## L.1.2 Python

One difficulty with using PreTeXt on Windows is the fact that Python is not core software, and there are many different ways to install it. The recommended method for installing Python is to get it directly from [python.org](https://python.org). Other Python installations, such as Anaconda, or even Miniconda, include a lot of extras that we don't need.

To install Python, simply download the installer from the [Python website](https://python.org).



When you run the installer, be sure to check off the box to add Python to the Windows PATH during installation. This will ensure that Python commands can be run from the command line without needing to provide the path to the Python program.

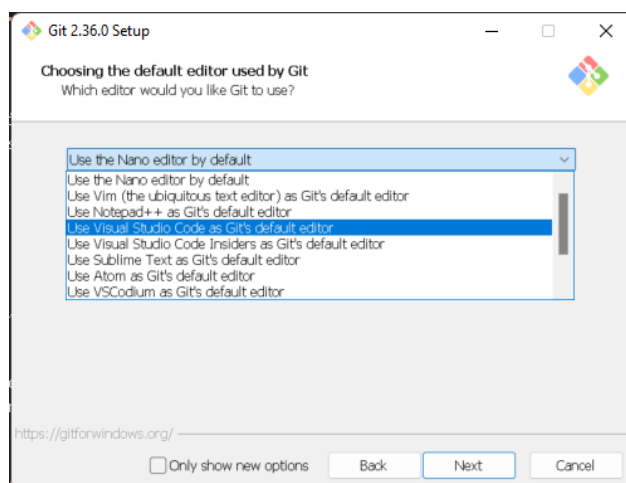


### L.1.3 Git for Windows

The next piece of software we need to install is Git for Windows. This will provide tools for allowing us to interact with textbook source on GitHub. More importantly, it provides us with the **Git Bash** terminal, which is what we will be running all of the commands for PreTeXt.

You can get Git for Windows at [gitforwindows.org](https://gitforwindows.org).

The installation process for Git for Windows can seem quite complicated, as there are a lot of options, and many of them are quite technical. You can safely choose the defaults throughout, unless there is a particular setting you're familiar with that you wish to change. The only thing you might want to do is change the default editor. Since we have already installed VSCode, we can choose to use that instead of an editor like Emacs or Nano that you may be unfamiliar with.

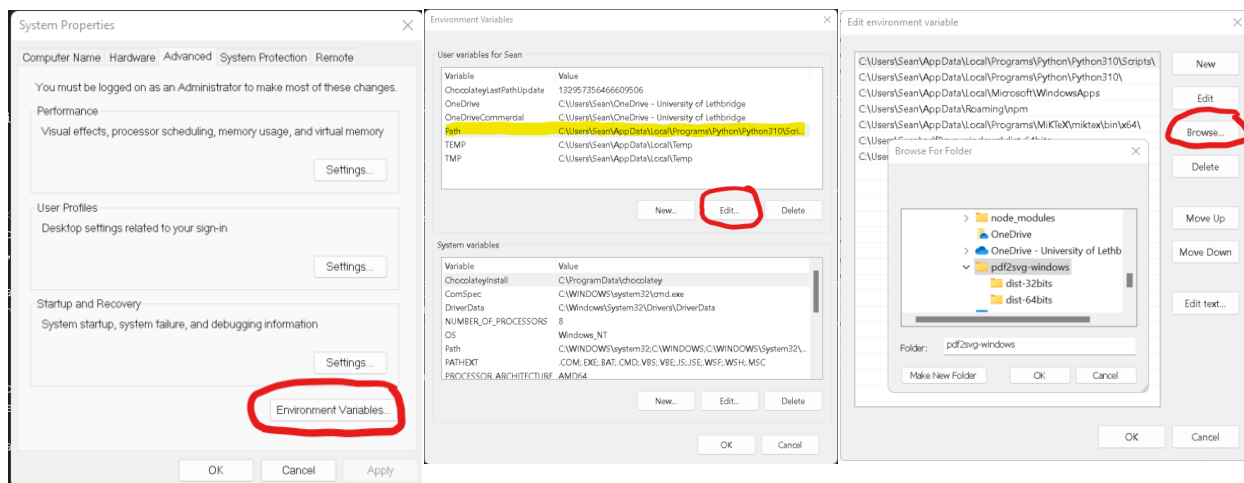


### L.1.4 Other software

A number of additional programs may be needed to assist in processing your book, depending on what elements your book contains.

**TEX** You will almost certainly need to be able to process TEX, either to produce the PDF version of your book, or to process TEX images for the HTML version of your book.

There are two ways to install TEX on Windows: [MikTeX](#), or [TeXLive](#). Although TeXLive is the default TEX distribution for Linux and MacOS platforms, most Windows users find MikTeX easier to use, since it comes with package management software to assist with automatically installing needed TEX packages. One thing to keep in mind: with MikTeX, you *must* run an initial update from the MikTeX package manager before it will work correctly.



**Sage** If your book includes Sage components, such as sageplot (but *not* Sage Cells), you will need to install Sage to process them. Sage is available at [www.sagemath.org](http://www.sagemath.org). Click on the download button, and follow the instructions in the installer.

#### GitHub Desktop

If you find it difficult to manage git from the command line, GitHub provides a graphical user interface for the Windows environment, called [GitHub Desktop](#). This provides a point-and-click interface for synchronizing your local changes with GitHub.

## L.2 Installing the CLI

You now have all the necessary software installed. Next, we need to set up Git Bash, and install the PreTeXt-CLI.

### L.2.1 Setting up Git Bash

One change that you will want to make right away is setting the default working directory for Git Bash. In your home folder (C:\Users\Sean in our example), look for a file called `.bashrc`. If it doesn't already exist, open Git Bash, navigate to this folder, and type `touch .bashrc` to create the file. Open the file (it is a plain text file), and add the line `cd C:/Users/Sean`, where you should replace the directory with the one you want to use.

Next, we need to set up ssh authentication, for more efficient communication with GitHub. We assume that you already have a GitHub account; if not, you should create one at [github.com](https://github.com). There are existing instructions available online at [docs.github.com](https://docs.github.com). To begin with, please follow the instructions provided there for generating an ssh key using Git Bash, and adding it to GitHub.

There is one aspect of the instructions that is not quite correct. When you get to the step for [adding the ssh key to the ssh agent](#), the instructions will tell you to add your ssh key to the ssh agent using a relative path, with a line such as `ssh-add ~/.ssh/id_ed25519`. **You must use a full path here.** Instead, type `ssh-add /c/Users/Sean/.ssh/id_ed25519`, where as usual you should replace Users/Sean with your own directory.

There is one more step you will want to complete. The ssh-agent program will not start automatically when you open Git Bash. To change this, we follow the instructions provided at [gist.github.com/bsara/5c4d90db3016814a3d2fe38d314f9c23](https://gist.github.com/bsara/5c4d90db3016814a3d2fe38d314f9c23).

1. In the file `/c/Users/Sean/.ssh`, open the config file (or create it using the `touch` command in Git Bash if it doesn't already exist), and add the following lines:

```
Host github.com
 Hostname github.com
 IdentityFile ~/.ssh/id_rsa
```

Note that the second and third lines are indented by one space. Note also that this assumes an rsa key, but you may have chosen a different encryption method. You may, for example, want to replace `id_rsa` with `id_ed25519`.

2. In the file `/c/Users/Sean/.bash_profile`, (again, create it if it does not already exist) add the following lines:

```
test -f ~/.profile && . ~/.profile
test -f ~/.bashrc && . ~/.bashrc
```

3. In the file `/c/Users/Sean/.ssh`, we need to add the following script, which will automatically start the ssh-agent.

```
Start SSH Agent
#-----

SSH_ENV="$HOME/.ssh/environment"

function run_ssh_env {
 . "${SSH_ENV}" > /dev/null
}

function start_ssh_agent {
 echo "Initializing new SSH agent..."
 ssh-agent | sed 's/^echo/#echo/' > "${SSH_ENV}"
 echo "succeeded"
 chmod 600 "${SSH_ENV}"

 run_ssh_env;

 ssh-add ~/.ssh/id_rsa;
```



```
}

if [-f "${SSH_ENV}"]; then
 run_ssh_env;
 ps -ef | grep ${SSH_AGENT_PID} | grep ssh-agent$ > /dev/null || {
 start_ssh_agent;
 }
else
 start_ssh_agent;
fi
```

You should now be set up to work with GitHub on your Windows machine. When you clone a GitHub repository containing a PreTeXt book, be sure to choose the `ssh` option rather than `HTML`.

### L.2.2 Installing the CLI

Finally, we are ready to install the PreTeXt-CLI. This is perhaps the easiest step of the whole process. From the Git Bash terminal, first type `which python` to confirm that Python has been successfully added to the `PATH`. You should see the path to your Python program if things are working correctly. If you don't, you may need to reinstall Python, or you can manually add it.

To install the CLI, simply type `pip install pretext`.

Congratulations! You are now ready to start using PreTeXt on your Windows machine. If you did not do so previously, there is one additional step you may want to take. In VSCode, go to File, then Preferences, then Extensions in the menu, or type `Ctrl-Shift-X`. In the search bar that comes up, type "pretext-tools", and install the package that comes up. This will equip VSCode with syntax highlighting for `.ptx` files, as well as some automatic tag completion. (For example, typing `thm` and then the `Tab` key will automatically insert the XML markup for a theorem.)

Additionally, from the command palette, (View, then Command Palette in the menu, or `Ctrl-Shift-P`) if you type "pretext", you will see some built-in options for building your book using the CLI.

# Appendix M

## Windows Installation Notes

Dave Rosoff

The recommended method for setting up PreTeXt on Windows now uses the PreTeXt-CLI. For instructions on setting up the CLI, see [Appendix L](#). This appendix explains how to install the older PreTeXt toolchain using `xsltproc`, and is intended primarily for developers. It has been tested on Windows 7, 8, and 10. We assume that none of the listed tools or equivalents have been previously installed. That may complicate matters. This is especially true if you use Cygwin, or if you have already installed Python on your machine. PreTeXt compatibility with existing Python installations is addressed elsewhere in this document ([Section C.1](#)).

If you have Windows 10, be sure to read about WSL in [Appendix N](#), which could be a whole lot easier to setup and maintain.

### M.1 Setup

In this section, we do some initial setup, establish notation, and issue warnings. Some of the steps in this process are dangerous. Typos could lead to an unstable system, or possibly even to unrecoverable system errors. Double-check everything.

#### M.1.1 Notation

Strings enclosed in `<angle brackets>` are variables whose values you should substitute in typed expressions. `<username>`, for example, should be replaced with your Windows username (e.g., mine is `drosoff`). Throughout this installation process it is very important to pay attention to the direction of slashes `/` and backslashes `\`.

#### M.1.2 Initial Windows setup

It is easier to see what is happening if your Windows file browser is not set up to hide file extensions from you. Disable the “hide file extensions” behavior before proceeding. In Windows 7/8, this can be done through the Control Panel. In Windows 10, there is a checkbox somewhere in the ribbon for it.

##### List M.1.1 Initial Windows Setup

- On Windows 7 or 8:
  1. Open the Start Menu and type “Control Panel”. Select the Control Panel entry from the popup list.
  2. Type “Folder Options” into the search box in the Control Panel window. Select Folder Options when it appears.
  3. Select the View tab.

4. Uncheck the box for “Hide extensions for known file types”.
  5. Click OK until there are no more OKs to click.
- On Windows 10:
    1. Open the Start Menu (icon shaped like a Window at the bottom left, typically). Select the File Explorer option, right above Settings, from the popup list.
    2. Click on this, and then select View from the “ribbon lists” of options along the top.
    3. After clicking this, on the right there should be a check box for “File Name Extensions”. Click this box; that should do it.

### M.1.3 A word on path names

An appallingly large fraction of the difficulties of using GNU/Linux-based utilities with Windows come from the differences in formatting path names. Windows path names begin with a drive letter (usually “C”) and a colon. Like all path names, they describe a path in a rooted tree. The root directory (folder) in Windows is called \, a backslash. Note the direction carefully. Children of the root node are either subdirectories or files in the root directory (leaves). The path to my downloads folder is:

```
C:\Users\drosoff\Downloads\
```

The trailing backslash is often unnecessary, but it is an easy way to see immediately whether a path name refers to a file or to a directory. Windows path names are not case-sensitive.

Linux/Mac OS X path names are quite similar, but lack drive letters, start with an explicit reference to the root, use forward slashes, and are case-sensitive (more or less so, in Mac’s case). A path to a typical Linux user’s download folder might be

```
/home/typical.username/Downloads
```

Again, Linux pathnames are case-sensitive and Mac OS X pathnames are typically ‘case-preserving’. The Git Bash shell for Windows is an emulation of a Linux environment, and the utilities within it expect path names that follow Linux conventions. So we conform to this expectation as follows.

1. Remove the colon, but keep the drive letter.
2. All backslashes \ become slashes /.
3. Add an initial slash preceding the drive letter.

The path name to my Windows download folder becomes

```
/c/users/drosoff/Downloads
```

Even though Git Bash is pretending to be a Linux shell, path names are still the underlying Windows path names, and therefore are not case sensitive. You can verify this using tab-completion.

Path names that begin with the drive letter (Windows) or the root / (Linux/Mac OS X) are called **absolute path names**. Their referents do not depend on the location from which the path name is invoked. **Relative path names**, on the other hand, begin in the so-called **current working directory**. A relative pathname might look something like this:

```
../../examples/sample-article/sample-article.xml
```

The symbol `..` is a shortcut for the parent of the current directory. Thus, the relative path name above means from where we are, ascend two levels, then descend into the examples and sample-article subdirectories, and find the file sample-article.xml.

Path names that contain spaces are *evil*, and should be avoided in many cases. Unfortunately, all Windows default program installation locations contain at least one space (directory name “Program Files”). This does not appear to cause problems, except when installing ImageMagick (Section M.5). To be extra careful, you could always choose an installation location that is free of space characters.

### M.1.4 Do I have 64-bit Windows?

Find out on Windows 7:

1. Open the start menu and type “Computer”. Right-click the Computer item in the popup menu.
2. Select Properties from the drop-down menu.
3. Read in the right-hand side of the pane to find the “System” heading.
4. From the “System type” entry, read whether you have a 32- or a 64-bit OS.

Now you are ready to begin installing the various pieces of the PreTeXt puzzle. The first one, xsltproc ([Section M.2](#)), is the most annoying. You might want to go get a snack or another cup of coffee.

## M.2 Installing xsltproc

### M.2.1 xsltproc binaries

This is the most annoying part of the installation. Obtain four zip archives from [Igor Zlatkovic’s FTP site](#) that hosts the most recent Libxml binaries for Windows. At the time of this writing, the 64-bit binaries were considered experimental. I have had no trouble using the 32-bit binaries on my 64-bit Windows 7 system, so I suggest that all PreTeXt users download the most recent 32-bit version of the following libraries:

#### List M.2.1 xsltproc Zip Files

1. iconv (filename something like iconv-1.9.2.win32.zip)
2. libxml2 (filename something like libxml2-2.7.8.win32.zip)
3. libxslt (filename something like libxslt-1.1.26.win32.zip)
4. zlib (filename something like zlib-1.2.5.win32.zip)

We only need a handful of files from these archives. So the simplest thing is to leave them in your Downloads folder and grab what we need. Create a new folder C:\xsltproc (it can be anywhere, as long as it’s a new location). We’ll call this location <xsltproc> in case you named your folder something different.

Extract the following files from the four zip archives you downloaded above into <xsltproc>:

#### List M.2.2 xsltproc Extracted Zip Files

1. From iconv-\*.win32.zip:
  - (a) iconv-\*.win32\bin\iconv.dll
2. From libxml2-\*.win32.zip:
  - (a) libxml2-\*.win32\bin\libxml2.dll
  - (b) libxml2-\*.win32\bin\xmlint.exe
3. From libxslt-\*.win32.zip:
  - (a) libxslt-\*.win32\bin\libxslt.dll
  - (b) libxslt-\*.win32\bin\libxslt.dll
  - (c) libxslt-\*.win32\bin\xsltproc.exe
4. From zlib-\*.win32.zip:
  - (a) zlib-\*.win32\bin\zlib1.dll

## M.2.2 Change PATH environment variable

*Note: if you prefer not to meddle with this, it can be avoided.* Now, we need to make sure your system can find these files when we need them.

### List M.2.3 Path Environment Variable for `xsltproc`

1. Open the Start menu and start typing “Edit the system environment variables”. Select this option when it becomes visible.
2. Click the Environment Variables button near the bottom of the dialog.
3. In the bottom part of the dialog labeled “System environment variables”, look for a variable named PATH. You may need to scroll.
  - (a) If you do not find one, create it using the New... button. Make sure to use all capital letters. (This really shouldn’t happen. Make sure you are editing the *system* environment variables, not the *user* environment variables.)
  - (b) If you do find the PATH variable, select it and click the Edit... button.
4. Regardless of which of steps 1 and 2 you followed, now you should see a dialog with two text fields. Your variable name should be PATH.
5. If you created this variable, populate the second field with the full path name of `<xsltproc>`, the location where you put the seven files from Igor’s zip archives. For me this looks like `C:\xsltproc`.
6. If you are editing the PATH variable, place the cursor in the existing value and press the End key, so that the cursor moves to the back of the line. The PATH string is a ;-delimited list of full path names, so append the string `<xsltproc>`; (note the semicolon) to the existing value. If you named `<xsltproc>` as we suggested above, then the last part of your PATH variable is now `C:\xsltproc;`.
7. Click OK to save changes.

Congratulations, you have successfully installed `xsltproc`.

Note that you have installed the `xml lint` utility as part of this procedure. This utility will allow some text editors to lint your PreTeXt files, that is, to automatically detect and highlight errors, and perhaps even to explain them.

## M.3 Installing git

In this section we install the git version control system and some tools to interact with it, including a fairly full-featured emulation of the bash command line shell. I strongly recommend you use the Git Bash shell or another bash emulation, so that you can use Linux commands referenced elsewhere.

### M.3.1 Steps to install git

#### List M.3.1 git Installation

1. Visit the official git [git-scm.com/download/win](https://git-scm.com/download/win) (download starts automatically) and obtain the latest binary for your system.
2. Find the installer in your Download location and run it.
3. Choose whatever location you like for the git installation folder. I recommend you use the default.
4. At the “Adjusting your PATH environment” dialog, select either of the first two items. I recommend the second, “Add git executable to Windows PATH”. This will allow you to use git from within other

Windows programs, such as Sublime Text or other text editors, which can be extremely convenient. If you are apprehensive about adding git to the Windows PATH, select the first option. I do not recommend the third option.

5. Accept the default options for all the remaining prompts.

### M.3.2 Changing the path with .bashrc

In [Subsection M.2.2](#), we promised that you could avoid messing with the Windows environment variables. If you install something else later that wants to use `xsltproc`, then this might not be the best idea. But if you are only going to use it from within Git Bash, then this will work fine.

From the Git Bash command prompt, enter this line of text and hit Enter. Do not make any typos. You should substitute your value of `<xsltproc>` where indicated, but make sure to conform to the conventions at the end of [Subsection M.1.3](#) regarding Windows path names in Git Bash. (I warned you this was going to be annoying.)

```
echo "export PATH=<xsltproc>:$PATH" >> ~/.bashrc
```

You may get a message from Git Bash the next time you run it about `.bash_profile`, which you may safely ignore. Congratulations, you have successfully installed git.

## M.4 Installing Anaconda

Anaconda is a well-regulated development environment for Python under Windows, and I recommend it for users who do not already have Python installed. The essential pretext script has [github.com/PreTeXtBook/pretext/pull/252](https://github.com/PreTeXtBook/pretext/pull/252) to support both Python 2 and Python 3. Therefore we make no recommendation about which Python version to choose.

If you already have a working Python installation, skip to [Section M.5](#).

You have some choice with your Anaconda installation. It actually supports the installation of several independent Pythons side-by-side (since both 2.7 and 3.x are in active use, this is more reasonable than it seems). If you do not need Python for anything else or are simply a minimalist, Miniconda is also an option. Miniconda installs no packages, but these can be installed via the conda utility at the command line later.

1. Download either Miniconda, Python 2.7, or Python 3.5 from the [www.continuum.io/downloads](http://www.continuum.io/downloads).
2. Run the installer and accept all the default suggestions.

Congratulations, you have successfully installed Python.

If you do not care about images, you can stop here. Much of the PreTeXt functionality is already present. However, to use the pretext/pretext script to create SVG images from sources like PDF/PNG images, Sage, Asymptote, or TikZ, you need to install ImageMagick using the directions in [Section M.5](#).

## M.5 Installing ImageMagick

Visit the [imagemagick.org/script/download.php#windows](http://imagemagick.org/script/download.php#windows) and grab a binary. If you have a 64-bit Windows installation ([Subsection M.1.4](#)), use the recommended version. If you have a 32-bit installation, find the version whose filename is obtained from that of the recommended version by substituting x86 for x64. For example, if the recommended version's filename is:

```
ImageMagick-7.0.1-6-Q16-x64-dll.exe
```

then a good choice for a 32-bit Windows would be

```
ImageMagick-7.0.1-6-Q16-x86-dll.exe
```

**List M.5.1 imagemagick Installation**

1. Run the installer from your download location.
2. Accept the license agreement.
3. Choose a default installation location that has no spaces in its folder name. The default choice “Program Files” causes problems because of path name issues. I chose  
     c:\ImageMagick-7.0.1-Q16\  
     It matters because the ImageMagick utility convert is used by the pretext script to convert your images into different formats. The pretext script will have a lot of trouble with path names that contain spaces.
4. When confronted with “Select additional tasks”, make sure that the boxes for “Add application directory to your system path” and “Install legacy utilities” are checked.
5. If you like, carry out the procedure to verify your installation.

Congratulations, you have successfully installed ImageMagick.

## M.6 Installing jing

The jing utility for schema validation can be installed in Windows. To install, first make sure Java is installed on your machine. You can either install the official Oracle JDK from [www.oracle.com/technetwork/java/javase/downloads/index.html](http://www.oracle.com/technetwork/java/javase/downloads/index.html) or, if you prefer an open license, [jdk.java.net/12/](http://jdk.java.net/12/). Download the .zip file, and extract to a folder such as C:\Java. (Avoid installing to the Program Files folder due to the space in the folder name.)

Next, follow the installation instructions in [Section E.1](#), ignoring the BUILD FAILED message. You will almost certainly want to copy the file xercesImpl.jar from the lib. folder into build.

If you are using Git Bash for processing, you may want to set up a command shortcut to avoid the rather lengthy command given in [Section E.1](#), especially if you have modularized source. Before proceeding, make sure that you have configured your Windows environment variables to include a path to Java. For example, if you installed JDK 12.0.1 in C:\Java you will want the path C:\Java\jdk-12.0.1\bin.

As suggested in [Section E.1](#), you can set up a command in .bashrc. Git Bash on Windows requires a .bash-profile to load .bashrc. Open Git Bash and change to your user directory: `cd /c/Users/your.name`. Windows does not like file names beginning with a period, so if they do not already exist, you’ll need to create them in Git Bash. Do `touch .bash-profile` and `touch .bashrc` to create the files. Use your favorite text editor to add the line

```
if [-f ~/.bashrc]; then . ~/.bashrc; fi
```

to .bash-profile

You can then create the ptx-check command as in [Section E.1](#). Absolute paths are recommended. Using the Git Bash syntax for Windows paths, your definition in .bashrc will look something like the following:

```
ptx-check() {
 java\
 -classpath /c/jing-trang/build\
 -Dorg.apache.xerces.xni.parser.XMLParserConfiguration=
 org.apache.xerces.parsers.XIncludeParserConfiguration\
 -jar /c/jing-trang/build/jing.jar\
 /c/PreTeXt/pretext/schema/pretext.rng "$1"
}
```

The above assumes that jing-trang is installed at C:\jing-trang and that PreTeXt has been cloned at C:\PreTeXt\pretext. Adjust your paths as needed. You can now run validation from Git Bash using

```
ptx-check ~/path/to/project/index.ptx > jing_report.txt
```

and the `jing_report.txt` file will be created in your working directory.

## M.7 What's Missing

Development of a Windows-compatible pretext script ([Chapter 47](#)) is mostly complete. If you need help with pretext, contact Dave Rosoff. There are still a few use cases that haven't been tested, mostly those to do with Asymptote.

A Windows installer for Sage is available from several mirrors at [www.sagemath.org/download-windows.html](http://www.sagemath.org/download-windows.html). The installer provides Sage Math, the Sage Math Shell, and the Sage Math Notebook (Jupyter).

If you find any problems or bugs, please let us know at the PreTeXt Support group in Google Groups, or email `drosoff AT collegeofidaho DOT edu`.



## Appendix N

# Windows Subsystem for Linux

Windows Subsystem for Linux is no longer the recommended method for Windows PreTeXt users. Instead, consider using either the web-based GitHub solution described in [Chapter 2](#) or the handy PreTeXt-CLI as discussed in [Section 5.2](#).

PreTeXt developers who use Windows may be interested in the information in this appendix but do note that some information may be out of date.

The **Windows Subsystem for Linux** (WSL) may be installed on computers running Windows 10. Several distributions of Linux are available. We will use Ubuntu as an example; working with other distributions is quite similar. The latest news and announcements about WSL may be found at [msdn.microsoft.com/commandline/wsl/about](https://msdn.microsoft.com/commandline/wsl/about).

**What is my OS build?** An easy way to find your OS Build: Hold down the Windows key and R key simultaneously. Enter `winver.exe` in the resulting new window. The next window that opens will have the OS Build in small type near the top.

**Installing WSL.** If you have Windows 10 with OS Build greater than 16215 (August 2016), then installing WSL is not difficult. Just follow the (reasonably straightforward) instructions given by Microsoft at the address [msdn.microsoft.com/en-us/commandline/wsl/install\\_guide](https://msdn.microsoft.com/en-us/commandline/wsl/install_guide).

Upon completion of the installation, you should

- be able to use the bash command from the PowerShell window,
- have your own WSL userid (distinct from Windows),
- have your own WSL password (distinct from Windows).

**A little background about using about the command line.**

- You type in commands (terminated by the Enter key) and the operating system responds. For example, if you type in `date`, the operating system responds with (what it considers to be) the date. Using the command line is an ongoing conversation between you and the operating system.
- The `sudo` command: when a command starts with `sudo`, the rest of the command is executed with administrative privileges. This is needed, for example, to install software or update the operating system. You must give your password when you run `sudo` (although you get a little window of time after the first usage when it is not necessary to do so).
- The `sudo apt-get update` command: this is used to resynchronize the local listing of installed packages with those in the official repository.
- The `sudo apt-get upgrade` command: this is used to bring all the local software up to date with those in the official repository.

Run `sudo apt-get update` followed by `sudo apt-get upgrade` with a newly system to bring it up to date. It is a good idea to repeat this frequently to have the latest software on your computer.

**Installing software.** The default configuration of wsl does not have the software needed for creating documents with PreTeXt.

The program `xsltproc` is used to create your readable documents. It is installed with the command `sudo apt-get install xsltproc`.

You are now ready to set up PreTeXt.

**Putting PreTeXt on your computer.** Here are the steps necessary to get the PreTeXt software onto your computer:

- Make a new directory `mkdir pretext`
- Make your own clone of the PreTeXt repository `git clone https://github.com/PreTeXtBook/pretext.git`
- Move to the new directory `cd pretext`
- Initialize the new directory with `git pull`

This last command synchronizes your files with those in the official repository. You should run it frequently to keep your files up to date.

**The simplest example.** Here is a brief description of the use of wsl to create readable files. You, as the author, create the XML file. The system will contain an appropriate xsl file that translates your XML file to something readable.

Several editors come with wsl by default including NANO, PICO, VI, and VIM. In addition, editing is possible using NOTEBOOK.EXE. Here are the steps to follow:

1. Type the command `cd` to align yourself in your home directory.
2. Use one of the editors to create a file called `hw.xml` (you could use the command `nano hw.xml`), and add the following text:

```
<?xml version="1.0" encoding="UTF-8" ?>
<pretext>
 <article xml:id="hw">
 <p>Hello, World!</p>
 </article>
</pretext>
```

3. Run the command `xsltproc pretext/xsl/pretext-html.xsl hw.xml` Upon completion, you should have a file called `hw.html`.
4. You now want to view the `hw.html` file in a browser, This is done with the command `explorer.exe hw.html`.

The edit-xsltproc-view cycle just given may seem daunting at first blush. Some things that can help:

- Pressing the up arrow when at the command line displays the previously executed commands. Hitting the enter key while such a command is displayed executes it. This saves a lot of retyping.
- It is possible to define aliases to shorten commands. Your local Linux guru can show how this is done.
- It is possible to define scripts to shorten multiple commands. Your local Linux guru can show how this is done.

# Appendix O

## Transitioning to the PreTeXt-CLI

Sean Fitzpatrick

This appendix is intended for authors of existing (“legacy”) PreTeXt projects who want to transition from using `xsltproc` to the PreTeXt-CLI.

### O.1 Setup

We will assume you have the following:

- An existing PreTeXt project, in a folder called `PROJECT`.
- A Python installation at version 3.8 or newer.
- The PreTeXt-CLI, installed using the instructions in [Section 5.2](#).

For usage of the CLI, see [Section 5.2](#).

The first thing we will do is to create a new folder. Call this folder `PROJECT-CLI`, and put it at the same level as the `PROJECT` folder. In a terminal (or in Python), enter the `PROJECT-CLI` directory, and run the `pretext new book` command. This will create the directory structure that the PreTeXt-CLI expects.

You now should see the following:

- A `project.ptx` file
- An `assets` folder
- A `generated-assets` folder
- A `publication` folder
- A `source` folder

Within the `source` folder, there will be a file called `main.ptx`, and within the `publication` folder, there will be a publisher file, called `publication.ptx`.

### O.2 Transferring files

In your original project, you probably have one folder containing your PreTeXt source, and you may have other folders containing external files, such as images, or code for interactive elements. If you use additional `XSL`, such as a LaTeX style sheet, you may have a folder for those files as well.

You want to copy your files as follows:

1. Copy all of your PreTeXt source files into PROJECT-CLI/source.

If you already have a file called `main.ptx`, it is fine to let this file overwrite the one created by the `pretext init` command. If you do not have such a file, you can either delete `main.ptx`, or rename your top-level file to `main.ptx`.

**Note:** if you do not end up with a file called `main.ptx`, or if your file with this name is not your top-level file, see [Section O.3](#) for details on how to edit the `project.ptx` file accordingly.

2. Copy each folder containing external assets such as images into the `assets` folder. Note that you want to copy the *folders* and not the contents of those folders. This should result in folders such as `PROJECT-CLI/assets/images`.
3. If applicable, copy over your `xsl` folder, and any other relevant folders. But do not copy over your output or publication folders.

## O.3 Updating docinfo, the publication file, and the project manifest

Open both your original `publication.ptx` file, and the one created by the `pretext init` command. The file created by `pretext init` will contain some directory management details, such as:

```
<source>
 <directories external="../assets" generated="../generated-assets"/>
</source>
```

If your publication file did not already have a `<source>` element, copy this from the auto-generated publisher file into your own. If it does, adjust your existing content to match the director structure needed by the PreTeXt-CLI.

When this is done, replace the publication file created by `pretext init` with your own.

If your `<docinfo>` contains a `<brandlogo>`, you may need to change the `@source` attribute from `logo.png` to `images/logo.png`, where `logo.png` is the name of the file used for your brand logo. This assumes that the file you use for your brand logo is contained in `PROJECT-CLI/assets/images/`.

Next, we need to update the `project.ptx` file that was created by `pretext init`. Opening the file, you will see there are several components: `html`, `latex`, `pdf`, and `subset`. The `subset` component is quite useful for editing: specify an `xml:id` from your book (such as a chapter or section), and you can use the command `pretext build subset` to do a partial build of your project.

In each component, ensure that the `<source>` tag points to the top level file of your project. By default, this is `source/main.ptx`, but if you kept a different file name during the step in [Section O.2](#), you will need to change this line to point to the correct file.

Similarly, ensure that the `<publication>` tag points to the correct publication file.

If you use any string parameters that cannot be transitioned to the publisher file, you can add them in your `project.ptx` manifest. For example, if you have WeBWorK problems and use the static preview feature, your `xslproc` executable would have contained

```
--stringparam webwork.divisional.static no
```

In the `<html>` section of `project.ptx`, you can add

```
<stringparam key="webwork.divisional.static" value="no"/>
```

You can also specify a WeBWorK server as a string parameter, using `key="server"`, with `@value` set to the server you use.

Finally, if there is any component for which you use additional `xsl`, you can specify this in the manifest as well. For example, if you have a LaTeX style sheet `xsl/latex-style.xsl`, then in both the `<latex>` and `<pdf>` sections, you should add the line

```
<xsl>xsl/latex-style.xsl</xsl>
```

See [Section O.4](#) below for further details on using custom xsl.

## O.4 Updating your custom XSL

Because the PreTeXt-CLI uses its own copy of PreTeXt, we have to change how the custom xsl imports the PreTeXt style sheets.

At time of writing, import of the `entities.ent` file, which defines all the different components of PreTeXt, does not seem to work automatically. Our first step will be to make a copy of this file, and place it in the `xsl` folder, next to our custom xsl style sheet.

If you still have a copy of the PreTeXt repository, you can copy `entities.ent` from `pretext/xsl`. If not, open a Python terminal, and then run:

```
from pretext import static
static.core_xsl('pretext-latex.xsl', as_path=True)
```

This will output the path to the PreTeXt xsl, and in that folder you can find `entities.ent` and copy it to `PROJECT-CLI/xsl`.

Next, open your custom xsl file. You will need to change two lines. First, you will have a line that looks something like

```
<!ENTITY % entities SYSTEM "../xsl/entities.ent"/>
```

Change the string `../xsl/entities.ent` to simply `entities.ent`. (Alternatively, instead of copying this file as above, you could enter the full path.)

Second, for each style sheet that you import, you will have a line that looks something like

```
<xsl:import href="../xsl/pretext-latex.xsl"/>
```

if you are using custom xsl for LaTeX. Replace `href="../xsl/pretext-latex.xsl"` with `pretext-href="pretext-latex.xsl"`. This tells the PreTeXt-CLI to import the xsl style sheet that it ships with.

At this point, you should be ready to try building your project with the PreTeXt-CLI. The advice above is based on a particular case study, using APEX Calculus. The requirements of your particular project may differ somewhat from the steps presented here.

As development proceeds on the PreTeXt-CLI, some of this advice may change. In particular, in a future version, it should no longer be necessary to make a copy of `entities.ent`.

# Appendix P

## CLI versions 1 vs 2

Oscar Levin

This appendix describes the differences between the PreTeXt-CLI version 1.x and 2.x. Generally, projects started in version 1 of the CLI will continue to work in version 2.

The primary difference between the versions of the CLI is the format of the project manifest (`project.ptx`): in version 1, most properties were described as XML elements, where they are now given as attributes. The functions of each attribute for the version 2 manifest is described in [Subsection 5.2.7](#). Here we illustrate how these relate to the *legacy* elements.

In [Listing P.0.1](#) we show a project manifest containing almost all options available. Then [Listing P.0.2](#) shows the same project manifest in version 2 format.

## Listing P.0.1 Example of version 1 project manifest

```
<?xml version="1.0" encoding="utf-8"?>
<project>
 <targets>
 <target name="web">
 <format>html</format>
 <source>source/main.ptx</source>
 <publication>publication/publication.ptx</publication>
 <output-dir>output/web</output-dir>
 </target>
 <target name="print" pdf-method="xelatex">
 <format>pdf</format>
 <source>source/main.ptx</source>
 <publication>publication/publication.ptx</publication>
 <output-dir>output/print</output-dir>
 <output-filename>mybook.pdf</output-filename>
 </target>
 <target name="runestone">
 <format>html</format>
 <source>source/main.ptx</source>
 <publication>publication/rs-publication.ptx</publication>
 <output-dir>published/mydoc-id</output-dir>
 <stringparam key="author.tools" value="yes"/>
 <stringparam key="html.css.extra" value="external/custom-styles.css" />
 <xsl>xsl/runestone.xsl</xsl>
 </target>
 </targets>
 <executables>
 <latex>latex</latex>
 <pdflatex>pdflatex</pdflatex>
 <xelatex>xelatex</xelatex>
 <asy>asy</asy>
 <sage>sage</sage>
 <pdfpng>convert</pdfpng>
 <pdfeps>pdftops</pdfeps>
 <node>node</node>
 <liblouis>file2brel</liblouis>
 </executables>
</project>
```

**Listing P.0.2 Example of version 2 project manifest**

```
<?xml version="1.0" encoding="utf-8"?>
<project ptx-version="2">
 <targets>
 <target
 name="web"
 format="html"
 source="source/main.ptx"
 publication="publication/publication.ptx"
 output-dir="output/web" />
 <target
 name="print"
 format="pdf"
 latex-engine="xelatex"
 source="source/main.ptx"
 publication="publication/publication.ptx"
 output-dir="output/print"
 output-filename="mybook.pdf" />
 <target
 name="runestone"
 format="html"
 platform="runestone"
 source="source/main.ptx"
 publication="publication/rs-publication.ptx"
 output-dir="published/mydoc-id"
 xsl="xsl/runestone.xsl">
 <stringparams author.tools="yes" html.css.extra="external/custom-styles.css"
 />
 </target>
 </targets>
</project>
```

One advantage of version 2 of the CLI is that many of the options are now optional, or can be applied at the project level to all targets. The example shown above can be further simplified, as shown in [Listing P.0.3](#), since most values are the defaults anyway.



## Listing P.0.3 Simplified project manifest in version 2 format

```

<?xml version="1.0" encoding="utf-8"?>
<project ptx-version="2" publication="publication">
 <targets>
 <target
 name="web"
 format="html" />
 <target
 name="print"
 format="pdf"
 output-filename="mybook.pdf" />
 <target
 name="runestone"
 format="html"
 platform="runestone"
 publication="rs-publication.ptx"
 output-dir="published/mydoc-id"
 xsl="xsl/runestone.xsl">
 <stringparams author.tools="yes" html.css.extra="external/custom-styles.css"
 />
 </target>
 </targets>
</project>

```

Attributes for the <project> element that describe paths are common roots for any paths specified in <target> elements.

Notice that there is no longer a place to specify your *executables* in the manifest. This is because your executables are generally specific to a particular system, not a project. So instead, you specify executables in the file `executables.ptx` which is ignored by git. As with the version 2 manifest, the name or path to the executables are given as values of attributes. An example is shown in [Listing 5.2.10](#).

The project manifest can be further simplified by setting attributes for the <project> element. See [Subsection 5.2.7](#) for details.

Hopefully the examples above make it clear how the two versions compare for basic options. Below we list a few more differences between how you specify options in the version 1 and version 2 manifests.

<b>@ptx-version</b>	For the CLI to know you are using a version 2 manifest, you <i>must</i> include the @ptx-version attribute in the <project> element, with value “2”.
<b>string parameters</b>	In version 1, you would specify string parameters as a sequence of <stringparam> elements, each with @key and @value attributes. In version 2, you use a single <stringparams> element with attributes for each key set to the value for the string parameter’s value. See the example in <a href="#">Listing P.0.2</a>
<b>zipped output</b>	In version 1, you could get a zipped version of your html output by setting the format of the target to html-zipped. In version 2, this is replaced by setting the @compression attribute to “zip” in the <target> element.
<b>Braille options</b>	In version 2, you specify what embossing method you use for Braille formatted targets using the @braille-mode attribute, with value either “emboss” or “electronic”.
<b>WeBWoRK Sets</b>	In version 1, you could generate webwork sets using the “webwork-sets” format. In version 2, this is replaced by setting the @format attribute to “webwork” in the <target> element. Such a target can be compressed using the @compression attribute.
<b>Local asymptote</b>	In version 2, you can require that asymptote elements are generated using a local version of asymptote (instead of the server). You can specify this by setting the @easy-method attribute to “local”, in either the <target> element, or as an attribute of <project> to use this method for all targets.

## Appendix Q

# List of Supported Journal Styles

This appendix contains the list of supported journals and their short name codes. To specify one of these journals, use the publication file. See [Subsection 44.1.9](#).

The code for a journal is taken from the list of abbreviations used by MathSciNet available at <https://mathscinet.ams.org/msnhtml/serials.pdf>.

Table Q.0.1 Journals supported by PreTeXt

Full Journal Name	Code
AMS journals	ams
Elsevier journals	elsevier
Springer Nature journals	springer-nature
Taylor Francis journals	taylor-francis
Annals of Pure and Applied Logic	ann-pure-appl-logic
Bulletin (New Series) of the American Mathematical Society	bull-amer-math-soc
Conformal Geometry and Dynamics	conform-geom-dyn
Electronic Journal of Combinatorics	electron-j-combin
Experimental Mathematics	exp-math
Journal of Algebraic Geometry	j-algebraic-geom
Journal of the American Mathematical Society	j-amer-math-soc
Mathematics of Computation	math-comp
Quarterly of Applied Mathematics	quart-appl-math
Proceedings of the American Mathematical Society	proc-amer-math-soc
Representation Theory	represent-theory
Theory of Probability and Mathematical Statistics	theory-probab-math-statist
Transactions of the American Mathematical Society	trans-ams

## Appendix R

### Example List of Notation

Symbol	Description	Page
$\nabla$	gradient operator	<a href="#">113</a>
$\binom{n}{k}$	binomial coefficient	<a href="#">168</a>

# Appendix S

## GNU Free Documentation License

Version 1.3, 3 November 2008

Copyright © 2000, 2001, 2002, 2007, 2008 Free Software Foundation, Inc. <[www.fsf.org/](http://www.fsf.org/)>

Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

**0. PREAMBLE.** The purpose of this License is to make a manual, textbook, or other functional and useful document “free” in the sense of freedom: to assure everyone the effective freedom to copy and redistribute it, with or without modifying it, either commercially or noncommercially. Secondly, this License preserves for the author and publisher a way to get credit for their work, while not being considered responsible for modifications made by others.

This License is a kind of “copyleft”, which means that derivative works of the document must themselves be free in the same sense. It complements the GNU General Public License, which is a copyleft license designed for free software.

We have designed this License in order to use it for manuals for free software, because free software needs free documentation: a free program should come with manuals providing the same freedoms that the software does. But this License is not limited to software manuals; it can be used for any textual work, regardless of subject matter or whether it is published as a printed book. We recommend this License principally for works whose purpose is instruction or reference.

**1. APPLICABILITY AND DEFINITIONS.** This License applies to any manual or other work, in any medium, that contains a notice placed by the copyright holder saying it can be distributed under the terms of this License. Such a notice grants a world-wide, royalty-free license, unlimited in duration, to use that work under the conditions stated herein. The “Document”, below, refers to any such manual or work. Any member of the public is a licensee, and is addressed as “you”. You accept the license if you copy, modify or distribute the work in a way requiring permission under copyright law.

A “Modified Version” of the Document means any work containing the Document or a portion of it, either copied verbatim, or with modifications and/or translated into another language.

A “Secondary Section” is a named appendix or a front-matter section of the Document that deals exclusively with the relationship of the publishers or authors of the Document to the Document’s overall subject (or to related matters) and contains nothing that could fall directly within that overall subject. (Thus, if the Document is in part a textbook of mathematics, a Secondary Section may not explain any mathematics.) The relationship could be a matter of historical connection with the subject or with related matters, or of legal, commercial, philosophical, ethical or political position regarding them.

The “Invariant Sections” are certain Secondary Sections whose titles are designated, as being those of Invariant Sections, in the notice that says that the Document is released under this License. If a section does not fit the above definition of Secondary then it is not allowed to be designated as Invariant. The Document may contain zero Invariant Sections. If the Document does not identify any Invariant Sections then there are none.

The “Cover Texts” are certain short passages of text that are listed, as Front-Cover Texts or Back-Cover Texts, in the notice that says that the Document is released under this License. A Front-Cover Text may be at most 5 words, and a Back-Cover Text may be at most 25 words.

A “Transparent” copy of the Document means a machine-readable copy, represented in a format whose specification is available to the general public, that is suitable for revising the document straightforwardly with generic text editors or (for images composed of pixels) generic paint programs or (for drawings) some widely available drawing editor, and that is suitable for input to text formatters or for automatic translation to a variety of formats suitable for input to text formatters. A copy made in an otherwise Transparent file format whose markup, or absence of markup, has been arranged to thwart or discourage subsequent modification by readers is not Transparent. An image format is not Transparent if used for any substantial amount of text. A copy that is not “Transparent” is called “Opaque”.

Examples of suitable formats for Transparent copies include plain ASCII without markup, Texinfo input format, LaTeX input format, SGML or XML using a publicly available DTD, and standard-conforming simple HTML, PostScript or PDF designed for human modification. Examples of transparent image formats include PNG, XCF and JPG. Opaque formats include proprietary formats that can be read and edited only by proprietary word processors, SGML or XML for which the DTD and/or processing tools are not generally available, and the machine-generated HTML, PostScript or PDF produced by some word processors for output purposes only.

The “Title Page” means, for a printed book, the title page itself, plus such following pages as are needed to hold, legibly, the material this License requires to appear in the title page. For works in formats which do not have any title page as such, “Title Page” means the text near the most prominent appearance of the work’s title, preceding the beginning of the body of the text.

The “publisher” means any person or entity that distributes copies of the Document to the public.

A section “Entitled XYZ” means a named subunit of the Document whose title either is precisely XYZ or contains XYZ in parentheses following text that translates XYZ in another language. (Here XYZ stands for a specific section name mentioned below, such as “Acknowledgements”, “Dedications”, “Endorsements”, or “History”.) To “Preserve the Title” of such a section when you modify the Document means that it remains a section “Entitled XYZ” according to this definition.

The Document may include Warranty Disclaimers next to the notice which states that this License applies to the Document. These Warranty Disclaimers are considered to be included by reference in this License, but only as regards disclaiming warranties: any other implication that these Warranty Disclaimers may have is void and has no effect on the meaning of this License.

**2. VERBATIM COPYING.** You may copy and distribute the Document in any medium, either commercially or noncommercially, provided that this License, the copyright notices, and the license notice saying this License applies to the Document are reproduced in all copies, and that you add no other conditions whatsoever to those of this License. You may not use technical measures to obstruct or control the reading or further copying of the copies you make or distribute. However, you may accept compensation in exchange for copies. If you distribute a large enough number of copies you must also follow the conditions in section 3.

You may also lend copies, under the same conditions stated above, and you may publicly display copies.

**3. COPYING IN QUANTITY.** If you publish printed copies (or copies in media that commonly have printed covers) of the Document, numbering more than 100, and the Document’s license notice requires Cover Texts, you must enclose the copies in covers that carry, clearly and legibly, all these Cover Texts: Front-Cover Texts on the front cover, and Back-Cover Texts on the back cover. Both covers must also clearly and legibly identify you as the publisher of these copies. The front cover must present the full title with all words of the title equally prominent and visible. You may add other material on the covers in addition. Copying with changes limited to the covers, as long as they preserve the title of the Document and satisfy these conditions, can be treated as verbatim copying in other respects.

If the required texts for either cover are too voluminous to fit legibly, you should put the first ones listed (as many as fit reasonably) on the actual cover, and continue the rest onto adjacent pages.

If you publish or distribute Opaque copies of the Document numbering more than 100, you must either include a machine-readable Transparent copy along with each Opaque copy, or state in or with each Opaque copy a computer-network location from which the general network-using public has access to download using public-standard network protocols a complete Transparent copy of the Document, free of added material. If you use the latter option, you must take reasonably prudent steps, when you begin distribution of Opaque copies in quantity, to ensure that

this Transparent copy will remain thus accessible at the stated location until at least one year after the last time you distribute an Opaque copy (directly or through your agents or retailers) of that edition to the public.

It is requested, but not required, that you contact the authors of the Document well before redistributing any large number of copies, to give them a chance to provide you with an updated version of the Document.

**4. MODIFICATIONS.** You may copy and distribute a Modified Version of the Document under the conditions of sections 2 and 3 above, provided that you release the Modified Version under precisely this License, with the Modified Version filling the role of the Document, thus licensing distribution and modification of the Modified Version to whoever possesses a copy of it. In addition, you must do these things in the Modified Version:

- A. Use in the Title Page (and on the covers, if any) a title distinct from that of the Document, and from those of previous versions (which should, if there were any, be listed in the History section of the Document). You may use the same title as a previous version if the original publisher of that version gives permission.
- B. List on the Title Page, as authors, one or more persons or entities responsible for authorship of the modifications in the Modified Version, together with at least five of the principal authors of the Document (all of its principal authors, if it has fewer than five), unless they release you from this requirement.
- C. State on the Title page the name of the publisher of the Modified Version, as the publisher.
- D. Preserve all the copyright notices of the Document.
- E. Add an appropriate copyright notice for your modifications adjacent to the other copyright notices.
- F. Include, immediately after the copyright notices, a license notice giving the public permission to use the Modified Version under the terms of this License, in the form shown in the Addendum below.
- G. Preserve in that license notice the full lists of Invariant Sections and required Cover Texts given in the Document's license notice.
- H. Include an unaltered copy of this License.
- I. Preserve the section Entitled "History", Preserve its Title, and add to it an item stating at least the title, year, new authors, and publisher of the Modified Version as given on the Title Page. If there is no section Entitled "History" in the Document, create one stating the title, year, authors, and publisher of the Document as given on its Title Page, then add an item describing the Modified Version as stated in the previous sentence.
- J. Preserve the network location, if any, given in the Document for public access to a Transparent copy of the Document, and likewise the network locations given in the Document for previous versions it was based on. These may be placed in the "History" section. You may omit a network location for a work that was published at least four years before the Document itself, or if the original publisher of the version it refers to gives permission.
- K. For any section Entitled "Acknowledgements" or "Dedications", Preserve the Title of the section, and preserve in the section all the substance and tone of each of the contributor acknowledgements and/or dedications given therein.
- L. Preserve all the Invariant Sections of the Document, unaltered in their text and in their titles. Section numbers or the equivalent are not considered part of the section titles.
- M. Delete any section Entitled "Endorsements". Such a section may not be included in the Modified Version.
- N. Do not retitle any existing section to be Entitled "Endorsements" or to conflict in title with any Invariant Section.
- O. Preserve any Warranty Disclaimers.

If the Modified Version includes new front-matter sections or appendices that qualify as Secondary Sections and contain no material copied from the Document, you may at your option designate some or all of these sections as invariant. To do this, add their titles to the list of Invariant Sections in the Modified Version's license notice. These titles must be distinct from any other section titles.

You may add a section Entitled "Endorsements", provided it contains nothing but endorsements of your Modified Version by various parties — for example, statements of peer review or that the text has been approved by an organization as the authoritative definition of a standard.

You may add a passage of up to five words as a Front-Cover Text, and a passage of up to 25 words as a Back-Cover Text, to the end of the list of Cover Texts in the Modified Version. Only one passage of Front-Cover Text and one of Back-Cover Text may be added by (or through arrangements made by) any one entity. If the Document already includes a cover text for the same cover, previously added by you or by arrangement made by the same entity you are acting on behalf of, you may not add another; but you may replace the old one, on explicit permission from the previous publisher that added the old one.

The author(s) and publisher(s) of the Document do not by this License give permission to use their names for publicity for or to assert or imply endorsement of any Modified Version.

**5. COMBINING DOCUMENTS.** You may combine the Document with other documents released under this License, under the terms defined in section 4 above for modified versions, provided that you include in the combination all of the Invariant Sections of all of the original documents, unmodified, and list them all as Invariant Sections of your combined work in its license notice, and that you preserve all their Warranty Disclaimers.

The combined work need only contain one copy of this License, and multiple identical Invariant Sections may be replaced with a single copy. If there are multiple Invariant Sections with the same name but different contents, make the title of each such section unique by adding at the end of it, in parentheses, the name of the original author or publisher of that section if known, or else a unique number. Make the same adjustment to the section titles in the list of Invariant Sections in the license notice of the combined work.

In the combination, you must combine any sections Entitled "History" in the various original documents, forming one section Entitled "History"; likewise combine any sections Entitled "Acknowledgements", and any sections Entitled "Dedications". You must delete all sections Entitled "Endorsements".

**6. COLLECTIONS OF DOCUMENTS.** You may make a collection consisting of the Document and other documents released under this License, and replace the individual copies of this License in the various documents with a single copy that is included in the collection, provided that you follow the rules of this License for verbatim copying of each of the documents in all other respects.

You may extract a single document from such a collection, and distribute it individually under this License, provided you insert a copy of this License into the extracted document, and follow this License in all other respects regarding verbatim copying of that document.

**7. AGGREGATION WITH INDEPENDENT WORKS.** A compilation of the Document or its derivatives with other separate and independent documents or works, in or on a volume of a storage or distribution medium, is called an "aggregate" if the copyright resulting from the compilation is not used to limit the legal rights of the compilation's users beyond what the individual works permit. When the Document is included in an aggregate, this License does not apply to the other works in the aggregate which are not themselves derivative works of the Document.

If the Cover Text requirement of section 3 is applicable to these copies of the Document, then if the Document is less than one half of the entire aggregate, the Document's Cover Texts may be placed on covers that bracket the Document within the aggregate, or the electronic equivalent of covers if the Document is in electronic form. Otherwise they must appear on printed covers that bracket the whole aggregate.

**8. TRANSLATION.** Translation is considered a kind of modification, so you may distribute translations of the Document under the terms of section 4. Replacing Invariant Sections with translations requires special permission from their copyright holders, but you may include translations of some or all Invariant Sections in addition to the original versions of these Invariant Sections. You may include a translation of this License, and all the license notices in the Document, and any Warranty Disclaimers, provided that you also include the original English version of this

License and the original versions of those notices and disclaimers. In case of a disagreement between the translation and the original version of this License or a notice or disclaimer, the original version will prevail.

If a section in the Document is Entitled “Acknowledgements”, “Dedications”, or “History”, the requirement (section 4) to Preserve its Title (section 1) will typically require changing the actual title.

**9. TERMINATION.** You may not copy, modify, sublicense, or distribute the Document except as expressly provided under this License. Any attempt otherwise to copy, modify, sublicense, or distribute it is void, and will automatically terminate your rights under this License.

However, if you cease all violation of this License, then your license from a particular copyright holder is reinstated (a) provisionally, unless and until the copyright holder explicitly and finally terminates your license, and (b) permanently, if the copyright holder fails to notify you of the violation by some reasonable means prior to 60 days after the cessation.

Moreover, your license from a particular copyright holder is reinstated permanently if the copyright holder notifies you of the violation by some reasonable means, this is the first time you have received notice of violation of this License (for any work) from that copyright holder, and you cure the violation prior to 30 days after your receipt of the notice.

Termination of your rights under this section does not terminate the licenses of parties who have received copies or rights from you under this License. If your rights have been terminated and not permanently reinstated, receipt of a copy of some or all of the same material does not give you any rights to use it.

**10. FUTURE REVISIONS OF THIS LICENSE.** The Free Software Foundation may publish new, revised versions of the GNU Free Documentation License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns. See [www.gnu.org/copyleft/](http://www.gnu.org/copyleft/).

Each version of the License is given a distinguishing version number. If the Document specifies that a particular numbered version of this License “or any later version” applies to it, you have the option of following the terms and conditions either of that specified version or of any later version that has been published (not as a draft) by the Free Software Foundation. If the Document does not specify a version number of this License, you may choose any version ever published (not as a draft) by the Free Software Foundation. If the Document specifies that a proxy can decide which future versions of this License can be used, that proxy’s public statement of acceptance of a version permanently authorizes you to choose that version for the Document.

**11. RELICENSING.** “Massive Multiauthor Collaboration Site” (or “MMC Site”) means any World Wide Web server that publishes copyrightable works and also provides prominent facilities for anybody to edit those works. A public wiki that anybody can edit is an example of such a server. A “Massive Multiauthor Collaboration” (or “MMC”) contained in the site means any set of copyrightable works thus published on the MMC site.

“CC-BY-SA” means the Creative Commons Attribution-Share Alike 3.0 license published by Creative Commons Corporation, a not-for-profit corporation with a principal place of business in San Francisco, California, as well as future copyleft versions of that license published by that same organization.

“Incorporate” means to publish or republish a Document, in whole or in part, as part of another Document.

An MMC is “eligible for relicensing” if it is licensed under this License, and if all works that were first published under this License somewhere other than this MMC, and subsequently incorporated in whole or in part into the MMC, (1) had no cover texts or invariant sections, and (2) were thus incorporated prior to November 1, 2008.

The operator of an MMC Site may republish an MMC contained in the site under CC-BY-SA on the same site at any time before August 1, 2009, provided the MMC is eligible for relicensing.

**ADDENDUM: How to use this License for your documents.** To use this License in a document you have written, include a copy of the License in the document and put the following copyright and license notices just after the title page:

Copyright (C) YEAR YOUR NAME.

Permission is granted to copy, distribute and/or modify this document

under the terms of the GNU Free Documentation License, Version 1.3

or any later version published by the Free Software Foundation;

with no Invariant Sections, no Front-Cover Texts, and no Back-Cover Texts.



A copy of the license is included in the section entitled "GNU Free Documentation License".

If you have Invariant Sections, Front-Cover Texts and Back-Cover Texts, replace the “with... Texts.” line with this:

with the Invariant Sections being LIST THEIR TITLES, with the Front-Cover Texts being LIST, and with the Back-Cover Texts being LIST.

If you have Invariant Sections without Cover Texts, or some other combination of the three, merge those two alternatives to suit the situation.

If your document contains nontrivial examples of program code, we recommend releasing these examples in parallel under your choice of free software license, such as the GNU General Public License, to permit their use in free software.

# Glossary

**alt text.** A text description of an image or other visual content. Its purpose is to describe content to readers who are unable to see the page.

**AMSMath.** A  $\text{\LaTeX}$  package containing a wide variety of common math symbols.

**Asymptote.** A package for drawing graphs, diagrams or pictures which may be used with  $\text{\LaTeX}$  or PreTeXt.

**attribute.** In XML, tags can have attributes, which provide more information about the element. For example, in `<section permid="dwf">` the tag is “section”, the attribute is “permid”, and the value of that attribute is “dwf”.

**BibTeX.** A  $\text{\LaTeX}$  package for typesetting bibliographies.

**branch.** In git, the files in a repository can exist in multiple versions which are independent of each other, but are able to be merged. Each independent version is a **branch**

**commit.** In git, the act of declaring that changes to the files in a branch are now a permanent part of that branch.

**copyleft.** An arrangement whereby software or artistic work may be used, modified, and distributed freely on condition that anything derived from it is bound by the same condition.

**Creative Commons.** An organization which has created several open licenses for creative works.

The Creative Commons licenses are: CC BY, CC BY-SA, CC BY-ND, CC BY-NC, CC BY-NC-SA, and CC BY-NC-ND. The strings BY, SA, ND, and NC represent, respectively, Attribution, Share Alike, No Derivatives, and Noncommercial.

**D3.js.** A JavaScript library for animating data. The name comes from “Data Driven Documents”.

**Desmos.** A graphing calculator that can be embedded in a Web page.

**division.** In PreTeXt, a part, chapter, section, subsection, subsubsection, appendix, or paragraphs of a document, other similar components such as readingquestions, exercises, or glossary, as well as various similar content markup outside the main content.

**DVI.** A file format intermediate between  $\text{\LaTeX}$  and PS or PDF. Stands for *Device Independent*.

**EPS.** Encapsulated Postscript: a form of Postscript designed to hold a fragment of a page, to be included in a larger document.

**escape character.** Every markup language has **escape characters** which play a special role. In XML there are two escape characters: `&` and `<`. In  $\text{\TeX}$  there are 10 escape characters: `#` `$` `%` `&` `\` `^` `_` `{` `}` and `~`.

**FontAwesome.** A  $\text{\LaTeX}$  package containing many popular icons.

**GeoGebra.** An interactive environment particularly suited for constructing demonstrations in Euclidean geometry.

**GFDL.** GNU Free Documentation License: a copyleft license for manuals, textbooks, and other written materials.

**GIF.** Graphics Interchange Format: a lossless bitmap image format.

**git.** A version control system which is used to track changes to computer files. Git is particularly useful for large projects involving multiple authors.

**GitHub.** A commercial website which hosts repositories for software projects, with the assumption that users use git to transfer material between GitHub and other systems.

**GPL.** GNU General Public License: a copyleft license which gives users the right to run, study, share and modify software.

**HTML.** Hypertext Markup Language: the markup language used to describe the content of Web pages.

**JavaScript.** A computer language in common use for interactivity in Web pages.

**JPEG.** A method for compressing digital images, where the amount of compression can be adjusted, allowing for a tradeoff between image size and image quality. Named for the Joint Photographic Experts Group, which developed the standard. Files with JPEG compression usually have the extension .jpg.

**Jupyter.** An interactive computing environment designed to support a wide variety of computer languages. Named after the languages Julia, Python and R.

A **Jupyter notebook** is a Web-based environment running Jupyter, typically accessed through a browser.

**JSXGraph.** A JavaScript library for interactive geometry, function plotting, and data visualization.

**knowl.** Similar to a hyperlink, except the referenced material appears inside the current Web page. Usually indicated by a dotted underline.

Pronounced “knoll” as in “a knoll is a small hill”.

**ⒶT<sub>E</sub>X.** A markup language used for books and papers with lots of mathematics, which is built on T<sub>E</sub>X. PreT<sub>E</sub>Xt uses ⒶT<sub>E</sub>X as an intermediate format to produce PDF and print output.

**main branch.** In git, the name of the default branch.

**Markdown.** A plain text markup language which is easy to use, is limited in its capabilities (compared to PreT<sub>E</sub>Xt or ⒶT<sub>E</sub>X, for example), but can convert to many other formats.

**markup language.** A computer language that uses tags to define the content of a document. XML, HTML, Markdown, and ⒶT<sub>E</sub>X are examples of markup languages.

**master branch.** In git, the former name of the default branch. Currently “main” is the preferred.

**MathJax.** A JavaScript package for converting math ⒶT<sub>E</sub>X to presentation MathML or other visual formats.

**MBX.** PreT<sub>E</sub>Xt was once known as “MathBook XML”, commonly abbreviated as MBX. This abbreviation appears in many historical references.

**MP3.** A format for storing audio content.

**MP4.** A format for storing video content.

**MyOpenMath.** A free and open online homework system.

**Ogg.** A format for storing video content.

**origin repository.** In git, the default name for the remote repository from which the local repository was cloned.

**PDF.** Portable Document Format: a file format designed for printing documents, where the appearance of the pages is identical on all devices.

**PDFLaT<sub>E</sub>X.** One of the available programs to convert ⒶT<sub>E</sub>X to PDF.

**pgfplots.** A ⒶT<sub>E</sub>X package for drawing graphs of functions.

**PNG.** Portable Network Graphics: a raster image format that supports lossless data compression. PNG offers a similar functionality to GIF but is not subject to patent protection.

**Postscript.** A page description language and file format designed for printing documents.

**PreT<sub>E</sub>Xt.** The markup language and document preparation system for creating the next generation of textbooks. And the generation after that.

**PSTricks.** A ⒶT<sub>E</sub>X package for drawing diagrams and pictures.

**pull.** In git, the process of transferring information from a remote repository to a local repository.

**pull request.** In git, a suggestion that the changes to the files in one branch be merged into another branch (typically in another repository).

**push.** In git, the process of transferring information from a local repository to a remote repository.

**Python.** A general-purpose programming language, noted for making it easy (compared to other programming languages) for a programmer to read the code written by someone else. Sage is written in Python.

**QR code.** Quick Response code: a type of two-dimensional bar code, usually consisting of a grid of small black-and-white squares.

**R.** A programming language and software environment for statistics; available in Sage.

**Sage.** An open source computer algebra system for a wide range of symbolic and numerical mathematical computations.

**Sage cell.** A text box in which the user can run Sage commands, designed to be included in a Web page.

**Sage interact.** A interactive demonstration, with sliders and boxes to adjust parameters, created as a single Python function in Sage.

**siunitx.** A  $\LaTeX$  package for typesetting scientific units.

**SVG.** Scalable Vector Graphics: an XML vector image format for two-dimensional graphics.

**tag.** In XML, tags are enclosed in angle brackets and are used to describe the contents of a document. For example, if “All about even” was the title of a book about numbers that are multiples of 2, then the XML source of that book might mark up the title using the `title` tag, as `<title>All about even</title>`. In XML, most tags occur in pairs which surround their content, with the closing tag beginning with a “slash” `/`. However, some tags which do not contain content are “self-closing”, meaning that they end with a slash, such as `<mdash/>` and `<pretext/>`.

**$\TeX$ .** A typesetting language with high-quality automatic line- and page-breaking, specifically designed for mathematics.

**three.js.** A JavaScript library for animating 3-dimensional graphics on a Web page.

**TikZ.** A  $\LaTeX$  package for drawing diagrams and pictures.

**unicode.** The accepted standard for encoding text in any language.

**upstream repository.** In git, the usual name for the remote repository from which the local repository was originally forked.

**URL.** Uniform Resource Locator: the “web address” of a Web page (or other online content) consisting of a domain name, optionally followed by strings separated by slashes (`/`) which indicate specific content in the domain.

**Vimeo.** A commercial website for hosting videos.

**WebM.** A format for storing video content.

**WeBWorK.** An open-source online homework system for math and sciences courses.

**Wolfram CDF.** Wolfram Computable Document Format: a system for creating interactive demonstrations hosted in the Wolfram Cloud.

**XeLaTeX.** One of the available programs to convert  $\LaTeX$  to DVI or PDF, the executable is `xelatex`.

**XML.** eXtensible Markup Language: the base syntax for the PreTeXt language. Angle brackets enclose matching begin and end tags. Tags can nest hierarchically, and can have attributes. HTML is another markup language that uses XML syntax.

**XSL.** eXtensible Stylesheet Language: the language that describes how to convert XML to other formats. PreTeXt’s XSL stylesheets are used to convert PreTeXt XML to  $\LaTeX$ , HTML, ePub, and other formats.

**xsltproc.** A command-line program which takes as input an XSL file and an XML file, using the contents of the XSL file to transform the contents of the XML file. It is now preferred to use the PreTeXt CLI for transforming.

**xypic.** A  $\LaTeX$  package for drawing simple commutative diagrams.

**YouTube.** A commercial website for hosting videos.

## References

- [1] *The Chicago Manual of Style*. Fifteenth Edition, The University of Chicago Press. 2003.
- [2] Knuth, Donald E., *Literate Programming*. The Computer Journal, 1983, [www.literateprogramming.com/knuthweb.pdf](http://www.literateprogramming.com/knuthweb.pdf).
- [3] Fred Leise, Kate Mertes, Nan Badgett, *Indexing for Editors and Authors: A Practical Guide to Understanding Indexes*, American Society of Indexers, 2008

# Index

- $\LaTeX$ , 163, 397
  - exceptions: < and &, 163
- $\TeX$ , 398
- RELAX-NG, 140
- see reference in index, see <see>
- ISBN, 278
- XSL, 312
- method, 313
- restrict, 312
- M, 313
- X, 312
- r, 312
- amsmath, 46
- author.tools, 137
- build, 124
- deploy, 125
- generate, 125
- jing, 142
- new, 124
- pretext build, 124
- pretext deploy, 125
- pretext generate, 125
- pretext new, 124
- pretext support, 130
- pretext view, 125
- pretext, 310
- pretext script, 197, 309
- project.ptx, 126
- trang, 141
- view, 125
- xinclude, 110
- xmlint, 143
- xsltproc, 302, 311
- \amp, math alignment, 46
- accessibility, 26, 187
  - hyperlink underlining, LaTeX, 285
  - mathematics, 113
- accessibilty, 112
- ActiveCode
  - window in HTML, 221, 288
- <activity>, see project-like elements
- <algorithm>, see theorem-like elements
- aligned math, 46
- alphabetized index entries, 199
- alt text, 396
- Amazon.com, 279
- AMSMath, 396
- analytics, 225, 287
- anonymous list, 95
- answer
  - to example-like elements, 168
  - to <exercise>, 174
- <answer>
  - of an <exercise>, 174
- ASCII file, 3
- <aside>, 198
  - PreTeXt code for, 199
- aspect ratio, see also video, 106
- <assumption>, see axiom-like elements
- Asymptote, 396
- Asymptote links
  - HTML, 222, 292
  - LaTeX, PDF, 286
- attribute, 396
- author tools, 137
- <axiom>, 169
- axiom-like elements (<assumption>, <axiom>, <conjecture>, <heuristic>, <hypothesis>, <principle>), 169
  - PreTeXt code for, 169
- back matter, 24
- base URL, 221
- base url
  - HTML output, 287
- bibinfo, 97
- BibTeX, 396
- <biographical>, see <aside>
- <blockquote>
  - PreTeXt code for, 200
- blocks, 16, 36

- Blurb, 279
- bold, *see* formatting
- braille, 246
  - best practices, 110
- branch (git), 396
- bulleted list, *see* unordered list
- calculator
  - embedded in HTML, 221, 287
- <caption>
  - of <figure>, 188
  - of <table>, *see* <title>
- <case>
  - of proof, 167
- CC, 396
- CDATA, 31
- CDF, 398
- cell
  - of a table, 194
  - table, 92
    - multi-line, 93
- cell alignment
  - table, 93
- <chapter>, *see* division
- characters
  - exceptional, 21, 31
  - nontrivial, 21
- checkpoint, *see* inline exercise
- chunking option
  - chunking, 280
- citation, 40, 42
- <claim>, *see* theorem-like elements
- CLI, 122
  - help, 123
  - v debug, 130
  - build, 124
  - deploy, 125
  - generate, 125
  - new, 124
  - support, 130
  - view, 125
  - executables, 129
  - format, 126
  - project manifest, 126
  - stringparam, 129
  - targets, 126
  - xmlid-root, 129
- clickable area exercises, 63
- clone, 329
- coding exercises, 69
- column
  - of a table, 195
  - of an exercise group, 178
  - table, 92
- command-line, 123, 302
- commit (git), 396
- common option
  - chunking, 280
  - handout margins, 282
  - journal name, 282
  - qrcode, 282
  - table of contents level, 280
  - watermark, 281
  - worksheet headers and footers, 282
  - worksheet margins, 282
- common options, 280
  - fillin, 281
- common pattern, 142
- <conclusion>
  - of a division, 161
  - limitations, 162
  - PreTeXt code for, 162
  - of a project-like element, 171
- <conjecture>, *see* axiom-like elements
- console, 19, 83, 123, 302
- container, 40
- <convention>, *see* remark-like elements
- conversion, *see* XSL stylesheet
- converter, *see* XSL stylesheet
- copyleft, 396
- <corollary>, *see* theorem-like elements
- cover image
  - EPUB, 296
- covers
  - LaTeX, 234, 286
  - PDF, 234, 286
- Creative Commons, 396
- <creator>
  - of axiom-like elements, 169
  - of theorem-like elements, 167
- cross reference
  - in index, 199
  - in text, 198
- cross-reference, 16, 40
  - index, 100
- CSS, 318
- custom versions, 214
- customization, 218
- customizations, 214
- D3.js, 396
- data URL, 105
- definition, 167
- <definition>
  - PreTeXt code for, 168
- deprecated, 137
- <description>
  - for accessibility, 187

- of `<image>`, 187
- Desmos, 396
- detached proof, 167
- division, *see also* structural division, 43, 161, 396
  - containing other divisions, 161
  - specialized, 44
- `<docinfo>`, 186, 191
- document structure, 161
- document type definition, 142
- DTD, *see* document type definition
- DVI, 396
- elective knows, 289
- electronic option
  - LaTeX, PDF, 229, 284
- embed page button, 293
- emphasis, 160
- entry of a table, *see* cell
- enumerate, *see* list
- environments, *see* blocks
- EPS, 396
- EPUB
  - cover image, 296
- epub, 236
- EPUB publisher options, 296
- equation, 163
  - aligned
    - PreTeXt code for, 163
  - displayed
    - PreTeXt code for, 163
  - in-line
    - PreTeXt code for, 163
  - numbered, 163
- escape (character), 396
- essay question exercises, 69
- example
  - comparison to project-like elements, 170
- `<example>`, *see also* example-like elements, 168
  - structured, 168
    - PreTeXt code for, 169
  - unstructured, 168
    - PreTeXt code for, 168
- example-like elements (`<example>`, `<problem>`, `<question>`), 168
  - answer, 168
  - hint, 168
  - solution, 168
  - structured
    - PreTeXt code for, 169
  - `<title>`, 168
  - unstructured
    - PreTeXt code for, 168
- exceptional characters, 21, 31
- exercise, 18
  - component visibility, 280
  - divisional exercise, 18
  - inline exercise, 19
  - interactive, 60
    - clickable area, 63
    - coding, 69
    - essay question, 69
    - fill-in, 63
    - free-response, 69
    - horizontal mixed-up blocks, 62
    - horizontal Parsons problems, 62
    - matching, 63
    - mixed-up blocks, 61
    - multiple-choice, 61
    - Parsons problems, 61
    - select text, 63
    - short answer, 69
    - true/false, 61
- `<exercise>`, 173
  - `<answer>`, 174
  - `<hint>`, 174
  - inline, 176
  - PreTeXt code for, 174
  - renaming, 186
  - `<solution>`, 174
  - `<statement>`, 174, 180
  - WeBWorK, 180
  - with `<task>`
    - PreTeXt code for, 175
- exercise components
  - visibility, 280
- `<exercisegroup>`, 176
  - columns, 178
  - instructions, 176
  - PreTeXt code for, 177
- `<exercises>`
  - grouping, 176
- `<exercises>` division, 176
- `<exploration>`, *see* project-like elements
- extensions, *see* file extensions
  - MathJax, 53
- external link, 198
- external reference, *see also* URL, 25, 104
- extra stylesheets, 219
- `<fact>`, *see* theorem-like elements
- favicon, 221, 288
- feedback button
  - HTML, 222, 292
- figure, 20
- `<figure>`, 188
  - containing `<sidebyside>`, 189
  - inside `<sidebyside>`, 189
  - PreTeXt code for, 188



- without caption, *see* `<image>`
- file extension
  - ps, *see* Postscript
- file extensions
  - for graphics, 187
- fill-in exercises, 63
- fillin
  - options, 281
- `<fn>`, 198
- font, *see* formatting
- font size options
  - LaTeX, PDF, 284
- FontAwesome, 396
- footnote, 198
  - long, *see* `<aside>`
  - restrictions, 198
- footnotes, 29
- fork, 329
- formatting, 160
- formula, *see* equation
- free standing quotation, *see* `<blockquote>`
- free-response exercises, 69
- front matter, 23
  
- GeoGebra, 396
- GFDL, 396
- GIF, 396
- git, 396
- GitHub, 396
- glossary, 44
- Google Custom Search Engine, 226
- GPL, 396
- graphics
  - file formats, 187
- graphics formats, 187
  - generating from  $\text{\LaTeX}$  source, 191
- group work exercises, 70
  
- `<h>`, 199
- handout margins
  - common option, 282
- `<heuristic>`, *see* axiom-like elements
- hint
  - to example-like elements, 168
  - to `<exercise>`, 174
- `<hint>`
  - of an `<exercise>`, 174
- `<historical>`, *see* `<aside>`
- host
  - HTML, 290
- hosting, 125
- HTML, 397
  - Asymptote links, 222, 292
  - banner content, 220
  - banner content options, 287
  - feedback button, 222, 292
  - index page, 220, 289
  - navigation, 293
  - tabbed viewer, 290
  - table of contents, 293
- HTML publisher options, 286
- hyperlinks, *see* cross reference, *see* external link
- `<hypothesis>`, *see* axiom-like elements
  
- `<identity>`, *see* theorem-like elements
- `<idx>`, 199
- image, 17, 71, 137, 307
  - raster image, 71
  - vector graphic, 72
- `<image>`, 187
  - file formats, 187
  - inside `<figure>`
    - PreTeXt code for, 188
  - PreTeXt code for, 187
  - without caption, 187
- including files, *see* modular source files
- index, 24
  - advice on, 100
  - cross-reference, 100
  - index entry, 24
- index entries, 199
  - PreTeXt code for, 199
- index page
  - HTML, 220, 289
- Ingram, 279
- IngramSpark, 279
- Inkscape, 136
- inline exercise, 176
  - PreTeXt code for, 174
  - renaming, 186
- `<input>`, 196
- `<insight>`, *see* remark-like elements
- interactive exercises, 60
  - clickable area, 63
  - coding, 69
  - essay question, 69
  - fill-in, 63
  - free-response, 69
  - horizontal mixed-up blocks, 62
  - horizontal Parsons problems, 62
  - matching, 63
  - mixed-up blocks, 61
  - multiple-choice, 61
  - Parsons problems, 61
  - select text, 63
  - short answer, 69
  - true/false, 61
- interactive SageMath code, *see* `<sage>`

- internal cross reference, [198](#)
- internationalization, [25](#), [111](#)
- `<introduction>`
  - of a division, [161](#)
  - limitations, [162](#)
  - PreTeXt code for, [162](#)
  - of a project-like element, [171](#)
- `<investigation>`, *see* project-like elements
- italic, *see* formatting
- itemize, *see* unordered list
- JavaScript, [397](#)
- journal name
  - common option, [282](#)
- JPEG, [397](#)
- jpg, *see* JPEG
- JSXGraph, [397](#)
- Jupyter, [397](#)
- keyboard keys, [35](#)
- keys
  - keyboard, [35](#)
- kindle, [236](#)
- Kindle Direct Publishing, [279](#)
- knowl, [397](#)
- knowls
  - elective, [289](#)
- `\label`, *see* `xml:id`
- LAD Custom Publishing, [279](#)
- LaTeX
  - Asymptote links, [286](#)
  - covers, [234](#), [286](#)
  - electronic option, [229](#), [284](#)
  - font size option, [284](#)
  - hyperlink highlighting, [285](#)
  - open odd option, [284](#)
  - page breaks, [286](#)
  - page matching, [284](#)
  - pagebreaks, [286](#)
  - print option, [229](#), [284](#)
  - sides option, [284](#)
  - snapshot, [235](#), [286](#)
  - style option, [283](#)
- LaTeX
  - using to generate images, [191](#)
- LaTeX packages, [53](#)
- LaTeX publisher options, [283](#)
- `<latex-image>`
  - PreTeXt code for, [192](#)
- `<latex-image-preamble>`, [191](#)
- `<lemma>`, *see* theorem-like elements
- levels, [211](#)
- liblouis, [361](#)
- Lightning Source, [279](#)
- line, [45](#)
- link
  - external, [198](#)
  - internal, *see* cross reference
- list, [18](#), [56](#), [165](#)
  - anonymous, [95](#)
  - columns, [58](#)
  - description list, [18](#)
  - displayed in columns, [166](#)
  - label, [56](#)
  - marking, [165](#)
  - named, [20](#), [95](#)
  - numbering, [165](#)
  - of notation, [24](#), [102](#), [168](#)
  - ordered, [165](#)
    - PreTeXt code for, [165](#)
  - ordered list, [18](#)
  - unordered, [166](#)
    - PreTeXt code for, [166](#)
  - unordered list, [18](#)
- listing, [20](#)
- literal text, [22](#), [39](#)
- literate programming, [26](#), [120](#), [141](#)
- localization, [25](#), [111](#)
- Lulu.com, [279](#)
- macro, [49](#)
- main branch, [397](#)
- manifest, [126](#)
- margin note, *see* `<aside>`
- Markdown, [397](#)
- markup language, [2](#), [160](#), [397](#)
- master branch, [397](#)
- matching exercises, [63](#)
- mathematics, [17](#), [46](#), *see also* equation, [163](#)
  - LaTeX, [163](#)
    - best practices, [55](#)
    - display mathematics, [17](#)
  - environments
    - aligned equation, [163](#)
    - displayed equation, [163](#)
    - in-line equation, [163](#)
  - formula, [163](#)
  - inline mathematics, [17](#)
  - mathematical results, *see also* theorem-like elements, [23](#)
- MathJax, [163](#), [359](#), [397](#)
- MathJax extensions, [53](#)
- Maxwell's equations, [113](#)
- MBX, [397](#)
- `<md>`
  - PreTeXt code for, [163](#)
- merge, [330](#)
- Mermaid themes, [281](#)

- mixed-up blocks exercises, 61
  - horizontal, 62
- modular source files, 130
- MP3, 397
- MP4, 397
- `<mrow>`
  - PreTeXt code for, 163
- multiple-choice exercises, 61
- MyOpenMath, 397
- myopenmath exercise, 24, 103
- named list, 95
- named pattern, 142
- navigation
  - HTML, 293
- Node, 358
- nontrivial characters, 21
- Nook Press, 279
- normalization, 305
- notation, 24, 102
- notation (to be included in a notation list)
  - PreTeXt code for, 168
- `<notation>` (to be included in a notation list), 168
- notation list, 24, 102
- `<notation-list/>`, 168
- `<note>`, *see* remark-like elements
- npm, 358
- numbering, 211
  - list items, 165
- numbering options, 283
- `<observation>`, *see* remark-like elements
- Ogg, 397
- `<ol>`
  - PreTeXt code for, 165
- online platforms, 290
- open odd option
  - LaTeX, PDF, 284
- ordered list, 165
  - PreTeXt code for, 165
- origin repository, 397
- `<output>`, 196
- `<p>`, 162
  - inside `<sidebyside>`, 189
  - PreTeXt code for, 160
  - use inside `<li>`, 165
- packages
  - LaTeX, 53
- Page breaks
  - LaTeX, PDF, 286
- page layout, 188
  - in `<worksheet>`, 181
- page matching
  - LaTeX, PDF, 284
- Pagebreaks
  - LaTeX, PDF, 286
- panel, *see* side-by-side panel
- paragraph, 16, 162
- paragraphs, 162
- paragraphs, 162
- Parsons problems, 61
  - horizontal, 62
- pattern, 142
  - common pattern, 142
  - named pattern, 142
- PDF, 397
  - Asymptote links, 286
  - covers, 234, 286
  - electronic option, 229, 284
  - font size option, 284
  - open odd option, 284
  - page breaks, 286
  - page matching, 284
  - pagebreaks, 286
  - print option, 229, 284
  - sides option, 284
  - style option, 283
- PDF format
  - as image, 187
- PDF publisher options, 283
- PDFLaTeX, 397
- percent encoding, *see also* URL encoding, 104
- pgfplots, 397
- platform
  - HTML, 290
- PNG, 397
- PNG image format, 187
- porism
  - renaming another tag to obtain, 186
- portable
  - HTML, 290
- portable html, 290
- Postscript, 397
- PreTeXt, 397
- PreTeXt code for
  - `<aside>`, 199
  - `<blockquote>`, 200
  - `<idx>`, 199
  - aligned equations, 163
  - axiom-like elements (`<assumption>`, `<axiom>`, `<conjecture>`, `<heuristic>`, `<hypothesis>`, `<principle>`), 169
  - `<conclusion>` of a division, 162
  - `<definition>`, 168
  - displayed equation, 163
  - example
    - structured, 169
    - unstructured, 168

- example-like elements (`<example>`, `<problem>`, `<question>`), 168, 169
- `<exercise>`, 174
  - inline, 174
  - with `<task>`, 175
- `<exercisegroup>`, 177
- `<figure>`, 188
- `<image>`, 187
- `<image>` (inside `<figure>`), 188
- in-line equation, 163
- index entries, 199
- `<introduction>` of a division, 162
- `<latex-image>`, 192
- `<md>`, 163
- `<mrow>`, 163
- `<ol>`, 165
- ordered list, 165
- `<p>`, 160
- project-like elements (`<activity>`, `<exploration>`, `<investigation>`, `<project>`), 171
- proof (of theorem-like element), 167
- `<reading-questions>`, 179
- remark-like elements (`<convention>`, `<insight>`, `<note>`, `<observation>`, `<remark>`, `<warning>`), 170
- `<rename>`, 186
- `<sage>`, 196
- `<sageplot>`, 197
- `<sbsgroup>`, 190
- `<section>`, 161, 162
- `<sidebyside>`, 189
- `<subsection>`, 162
- `<table>`, 194
- `<tabular>`, 194
- theorem-like elements (`<algorithm>`, `<claim>`, `<corollary>`, `<fact>`, `<identity>`, `<lemma>`, `<proposition>`, `<theorem>`), 167
- `<ul>`, 166
- unordered list, 166
- `<worksheet>`, 182
- pretext script, 191
  - prerequisites, 191
- `<principle>`, *see* axiom-like elements
- print option
  - LaTeX, PDF, 229, 284
- print-on-demand, 278
- privacy
  - video, 221, 292
- private solutions, 254
- problem
  - as homework, *see* `<exercise>`
- `<problem>`, *see* example-like elements
- program, 19, 84
- programmer's editor, 3
- `<project>`, 170
  - PreTeXt code for, 171
- project manifest, 126
- project-like elements (`<activity>`, `<exploration>`, `<investigation>`, `<project>`), 170
  - PreTeXt code for, 171
- `<proof>`, 167
  - detached, 167
  - outside of theorem-like elements, *see* proof, detached
  - PreTeXt code for, 167
- `<proposition>`, *see* theorem-like elements
- ps (file extension), *see* Postscript
- PSTricks, 397
- publication file, 210
- publisher file, *see* publication file
- publisher options
  - common, 280
  - EPUB, 296
  - HTML, 286
  - html interactives, 294
  - LaTeX, 283
  - numbering, 283
  - PDF, 283
  - reveal.js appearance, 295
  - reveal.js controls, 295
  - reveal.js navigation, 295
  - reveal.js resources, 295
  - reveal.js slideshow, 294
  - source, 296
  - WeBWorK, 297
- pull (git), 397
- pull request, 328, 397
- push (git), 397
- Python, 397
- QR code, 397
- qrcode
  - common option, 282
- `<question>`, *see* example-like elements
- quotation mark
  - double, 199
  - single, 199
- quotations, 199
  - inline, 199
  - long, *see* `<blockquote>`
- R (programming language), 397
- raster image, 71
- `<reading-questions>`
  - PreTeXt code for, 179
- reference, 20
- `<axiom>`

- PreTeXt code for, 169
- <remark>, 170
  - PreTeXt code for, 170
- remark-like elements (<convention>, <insight>, <note>, <observation>, <remark>, <warning>), 169
  - PreTeXt code for, 170
- <rename>, 186
  - for <exercise>, 186
  - PreTeXt code for, 186
- renaming blocks, 38
- reveal.js slideshow publisher options, 294
- revision control, 5
- row
  - of a table, 194
  - table, 92
- rule
  - table, 92
- Sage, 22, 398
  - cell, 398
  - interact, 398
- <sage>
  - PreTeXt code for, 196
- Sage cell, 22
- SageMathCell
  - for static images, *see* <sageplot>
  - interactive, 196
- <sageplot>
  - PreTeXt code for, 197
- <sbsgroup>
  - PreTeXt code for, 190
- schema, *see also* XML vocabulary, 131, 140
- scientific units, 25
- script, 307
- SCSS, 318
- search, 291
  - Google, 226, 291
  - native, 225, 291
- <section>, *see also* division, 161
  - PreTeXt code for, 161, 162
- <see>, 199
- <seealso>, 199
- select text exercises, 63
- short answer exercises, 69
  - response area, 292
- SI units, *see* scientific units
- side-by-side group, 23, *see also* <sbsgroup>, 190
- side-by-side panel, 23, 96
  - panel, 96
    - captioned item, 97
    - sub-captioned item, 97
- <sidebyside>, 188
  - containing <figure>, 189
  - containing <p>, 189
  - inside <figure>, 189
  - PreTeXt code for, 189
- sides option
  - LaTeX, PDF, 284
- siunitx, 398
- slides, 26, 120, 247
- solution
  - to example-like elements, 168
  - to <exercise>, 174
- <solution>
  - of an <exercise>, 174
- solutions
  - private, 254
- sorted index entries, 199
- source, 3
- source publisher options, 296
- special symbols, *see*  $\LaTeX$
- specialized division, 44
- Speech Rule Engine, 359
- STACK exercise, 25, 104
- standalone files, 129
- <statement>
  - of an <exercise>, 174, 180
  - of axiom-like elements, 169
  - of theorem-like elements, 167
- string parameters, 218
- stringparam, 129, 218
- structural division, 15, 43
- style
  - HTML, 224, 290
  - print, 234, 271
- style option
  - LaTeX, PDF, 283
- stylesheet
  - extra, 219
- <subexercises>, 176
- subheadings (in index), 199
- <subsection>, *see* division
  - PreTeXt code for, 162
- SVG, 398
- svg (Scalable Vector Graphics) format, 187
- tabbed viewer
  - HTML, 290
- table, 20, 91
  - breakable, 93
  - cell, 92
  - cell alignment, 93
  - column, 92
  - entry, 92
  - multi-line cell, 93
  - row, 92
  - rule, 92

- `<table>`, 193
  - deciding when to use instead of `<figure>`, 194
  - not for visual layout, *see also* side-by-side group, 193
  - PreTeXt code for, 194
  - `<title>`, 194
  - without title or caption, *see* `<tabular>`
- table of contents
  - HTML, 293
  - level option, 280
- tabular, 92
  - breakable, 93
- `<tabular>`
  - PreTeXt code for, 194
  - without title or caption, 194
- tag, 398
- term, 160
- `<term>`
  - in a definition, 167
- terminal, 123, 302
- theme
  - development, 318
  - HTML, 290
- theorem, 167
- `<theorem>`
  - PreTeXt code for, 167
  - `<title>`, 167
- theorem-like elements (`<algorithm>`, `<claim>`, `<corollary>`, `<fact>`, `<identity>`, `<lemma>`, `<proposition>`, `<theorem>`), 167
  - PreTeXt code for, 167
  - proofs of, 167
- three.js, 398
- TikZ, 191, 398
  - inside `<latex-image>`, 192
  - loading in `<latex-image-preamble>`, 191
- timed assessments, 70
- title, 17, 45
  - very long, 45
- `<title>`
  - of axiom-like elements, 169
  - of example-like elements, 168
  - of remark-like elements, 170
  - of `<table>`, 194
  - of theorem-like elements, 167
- top-level content, 44
- top-level file, 130
- true/false, 61
- `<ul>`
  - PreTeXt code for, 166
- unicode, 398
- Unicode characters, 109
- units, *see* scientific units
- unordered list, 166
  - PreTeXt code for, 166
- upstream repository (git), 398
- URL, 25, 104, 398
  - base, 221
  - data, 105
  - query string, 105
  - URL encoding, 104
  - visual, 105
- `<url>`, 198
- valid schema, 131, 142
- validation
  - additional, 141
- validator, 131, 140
- vector graphic, 72
- vector graphics
  - Inkscape, 136
- verbatim text, 22, 39
- versions, 215
  - custom, 214
- video, 25, 106
  - aspect ratio, 106
  - embedding, 292
  - HTML, 221
- Vimeo, 398
- `<warning>`, *see* remark-like elements
- watermark, 212
  - common option, 281
- web accessibility, 26
  - mathematics, 113
- web accessibility, 112
- WebM, 398
- WeBWorK, 398
  - HTMLdynamism, 288
  - WeBWorK exercise, 24
  - WeBWorK exercises, 144, 257
- WeBWorK exercise, 180
- WeBWorK publisher options, 297
- whitespace, 4
  - in code, 327
  - tabs, 327
- Wolfram CDF, 398
- worksheet, 19
- `<worksheet>`, 181
  - PreTeXt code for, 182
- worksheet headers and footers
  - common option, 282
- worksheet margins
  - common option, 282
- XeLaTeX, 398
- XML, 398
- XML application, *see also* XML vocabulary, 2

- XML editor, [3](#)
- XML vocabulary, *see also* XML application, [2](#), [131](#)
- xmlid-root, [129](#)
- xref
  - publisher options, [293](#)
- XSD, [141](#)
- XSL, [302](#), [398](#)
- XSL stylesheet, [302](#)
- xsltproc, [398](#)
- xypic, [398](#)
- YouTube, [398](#)